The Algorithm Codex

Alejandro Piad-Morffis, Ph.D.

2026-01-18

Table of contents

Preface 11
- What is this book about? . . . 11
- Content of the Book . . . 12
- About the coding style . . . 12
- Support The Algorithm Codex . . . 13
- Stay in touch . . . 13

Foundations 14
- What counts as an algorithm . . . 14
- The three questions . . . 14
- Why I count operations, not seconds . . . 15
- Scaling, not stopwatching . . . 15
- Where this leaves you . . . 16
- Notes and further reading . . . 16

I Searching and Sorting 17

1 Looking at everything, in order 19
- 1.1 A sequence and nothing more . . . 19
- 1.2 From “is it there?” to “where?” and “how many?” . . . 20
- 1.3 Finding the extremes . . . 20
- 1.4 The three questions, applied . . . 22
- 1.5 The price of knowing nothing . . . 23
- 1.6 Notes and further reading . . . 23

2 Throwing half away 24
- 2.1 The half-and-half algorithm . . . 24
- 2.2 Finding insertion points: `bisect_left` and `bisect_right` . . . 25
- 2.3 Searching the answer, not the data . . . 26
- 2.4 The three questions, applied . . . 27
- 2.5 Information per comparison . . . 28
- 2.6 Notes and further reading . . . 28

3 Fixing inversions, one at a time 29
- 3.1 Inversions: the distance from sortedness . . . 29
- 3.2 Selection sort: find the extremum, repeat . . . 30
- 3.3 Insertion sort: each element finds its place . . . 30
- 3.4 Bubble sort: the cleanest inversion story . . . 32
- 3.5 The three questions, applied . . . 33
- 3.6 Distance from sortedness . . . 34
- 3.7 Notes and further reading . . . 34

4 Fixing inversions, many at a time 35
- 4.1 Two kinds of inversions, two orders of attack . . . 35
- 4.2 Merge sort: locals first, then weave . . . 35
- 4.3 Counting inversions, for free, in $O(n \log n)$ . . . 37
- 4.4 Quick sort: globals first, then sort each side . . . 38
- 4.5 The three questions, applied . . . 40
- 4.6 Two orders, one cost . . . 41
- 4.7 Notes and further reading . . . 41

5 Selection without sorting 42
- 5.1 Quickselect: only one side matters . . . 42

5.2 The pivot you can trust: median of medians . . . 44
5.3 The three questions, applied . . . 46
5.4 Recurse on one side . . . 46
5.5 Notes and further reading . . . 47

6 Looking inside the elements **48**
6.1 Counting sort: the value is the address . . . 48
6.2 Radix sort: one digit at a time . . . 49
6.3 The three questions, applied . . . 51
6.4 Lower bounds belong to models . . . 52
6.5 Notes and further reading . . . 52

7 Sorting, in practice **53**
7.1 Why insertion sort earns its place . . . 53
7.2 Introsort: quicksort with safety nets . . . 54
7.3 Timsort: merge sort that sees the runs . . . 55
7.4 The three questions, applied . . . 57
7.5 Compose, don't choose . . . 58
7.6 Notes and further reading . . . 58

Lessons from Part I **59**
The patterns that kept showing up . . . 59
Five meta-lessons . . . 60
What's next . . . 60

II Fundamental Data Structures 61

8 What contiguous memory buys you, and what it costs **63**
8.1 Contiguous or linked — pick one, get one . . . 63
8.2 Why growing one slot at a time is quadratic . . . 63
8.3 Doubling turns linear into constant . . . 64
8.4 Two writes per append, on average . . . 65
8.5 The three questions, applied . . . 67
8.6 The amortization argument carries forward . . . 68
8.7 Notes and further reading . . . 68

9 Surgery in the middle of a chain **69**
9.1 A node knows its successor . . . 69
9.2 Insertion and deletion as pointer surgery . . . 70
9.3 In-place reversal in one walk . . . 71
9.4 Splice, don't copy . . . 72
9.5 Tortoise and hare: detecting cycles . . . 73
9.6 A second pointer makes detach local . . . 75
9.7 The three questions, applied . . . 75
9.8 A pointer is a different kind of address . . . 76
9.9 Notes and further reading . . . 77

10 The most recent thing is the easiest to reach **78**
10.1 A stack is a sequence with three operations . . . 78
10.2 Array-backed: cheap on average, paid for by doubling . . . 78
10.3 Linked-backed: strict $O(1)$, paid in pointers . . . 79
10.4 Evaluating postfix with one stack . . . 80
10.5 Shunting Yard: infix to postfix . . . 81
10.6 The three questions, applied . . . 83
10.7 Restriction is the point . . . 84
10.8 Notes and further reading . . . 84

11 Pulling from the other end **85**
11.1 FIFO, and why the obvious implementation is wrong . . . 85
11.2 Two pointers chasing each other around a ring . . . 85
11.3 Unbounded, at a node per enqueue . . . 87

11.4 Both ends, equally cheap . . . 88
11.5 The monotonic deque, working harder . . . 90
11.6 The three questions, applied . . . 91
11.7 A data structure can be more interesting than its surface contract . . . 92
11.8 Notes and further reading . . . 92

12 Every key has an address, the function is the map **93**
12.1 What a dictionary needs to do, and how arrays can't quite . . . 93
12.2 Keys as addresses . . . 93
12.3 A linked list per slot . . . 94
12.4 Probing for the next free slot . . . 96
12.5 Load factor is the single dial . . . 99
12.6 The three questions, applied . . . 101
12.7 Escaping the comparison model . . . 102
12.8 Notes and further reading . . . 102

13 Remembering a set without remembering its members **103**
13.1 A million keys, a megabyte of bits . . . 103
13.2 One bit array, k hashes . . . 103
13.3 Why $\varepsilon = (1 - e^{-kn/m})^k$ . . . 104
13.4 Thirty lines and a bytearray . . . 104
13.5 No deletion, no enumeration, no resize . . . 107
13.6 The three questions, applied . . . 107
13.7 Trade certainty for space . . . 108
13.8 Notes and further reading . . . 108

14 A forest where every element knows its root **109**
14.1 Merging groups, one at a time . . . 109
14.2 Quick-find: representative-as-label . . . 109
14.3 Quick-union: forest of parents . . . 110
14.4 Union by rank: keeping the trees short . . . 111
14.5 Path compression: flattening as a side effect . . . 112
14.6 Inverse Ackermann is at most 4 . . . 113
14.7 The three questions, applied . . . 113
14.8 A forest, an amortization, and a bound that's almost constant . . . 114
14.9 Notes and further reading . . . 115

Lessons from Part II **116**
The patterns that kept showing up . . . 116
Five meta-lessons . . . 117
What's next . . . 117

III Trees 118

15 Branching from one root **120**
15.1 The shape of a tree . . . 120
15.2 Four walks around the same tree . . . 121
15.3 Walking without a stack . . . 123
15.4 Writing a tree to a string . . . 124
15.5 The three questions, applied . . . 126
15.6 One recursion shape, four orders . . . 127
15.7 Notes and further reading . . . 127

16 Smaller on the left, bigger on the right **128**
16.1 Order as invariant . . . 128
16.2 Insert and search . . . 128
16.3 Delete, the three-case method . . . 131
16.4 When BSTs go wrong . . . 133
16.5 The three questions, applied . . . 135
16.6 One ordering rule, three new operations . . . 136
16.7 Notes and further reading . . . 136

17 Rotations buy you log n **137**
17.1 The unbalanced problem, recovered . . . 137
17.2 Rotations: the four shapes . . . 137
17.3 AVL: height as invariant . . . 139
17.4 Why the height bound holds . . . 143
17.5 Sidebar: why production picks Red-Black . . . 144
17.6 The three questions, applied . . . 145
17.7 A local rule, a global cost shape . . . 146
17.8 Notes and further reading . . . 146

18 Random priorities, balanced trees **147**
18.1 Determinism's price . . . 147
18.2 Two invariants, one node . . . 147
18.3 Insert by rotate-up, delete by rotate-down . . . 148
18.4 Expected height is O(log n) . . . 153
18.5 The bridge to Part IX . . . 154
18.6 The three questions, applied . . . 154
18.7 Randomization is the rebalancing mechanism in disguise . . . 155
18.8 Notes and further reading . . . 156

19 An array that pretends to be a tree **157**
19.1 Two operations, nothing else . . . 157
19.2 The tree is the index arithmetic . . . 157
19.3 Sift up, sift down . . . 158
19.4 Build-heap in O(n) . . . 160
19.5 From "smallest item" to "highest priority" . . . 162
19.6 D-ary heaps, Fibonacci heaps, and what they cost . . . 163
19.7 The three questions, applied . . . 163
19.8 The cheapest implementation is the one with no pointers . . . 164
19.9 Notes and further reading . . . 165

20 One branch per character **166**
20.1 Strings as a special case of keys . . . 166
20.2 One node per character, one flag per terminal . . . 166
20.3 Prefix queries — what tries can do . . . 168
20.4 Longest-prefix matching — the IP-routing use case . . . 169
20.5 The pointer-overhead cost . . . 170
20.6 Collapse single-child chains into edges . . . 170
20.7 The three questions, applied . . . 174
20.8 The path *is* the key . . . 175
20.9 Notes and further reading . . . 175

21 Intervals as objects **176**
21.1 Updates cheap or queries cheap — pick one . . . 176
21.2 Every node owns an interval . . . 176
21.3 Walking down to update, three cases to query . . . 177
21.4 Range update, point query — flip the roles . . . 179
21.5 Defer the work until somebody asks . . . 179
21.6 The three questions, applied . . . 182
21.7 The layout is the algorithm . . . 183
21.8 Notes and further reading . . . 183

22 Same problem, half the code **184**
22.1 Segment trees were too much . . . 184
22.2 The lowest set bit decides the interval . . . 184
22.3 Two while-loops, one bit-trick . . . 185
22.4 Trace by hand on n=8 . . . 187
22.5 Same answers, half the code . . . 188
22.6 Range queries via two prefix queries . . . 188
22.7 Beyond sum: XOR, count, and what doesn't fit . . . 189
22.8 A wall-clock comparison . . . 190

22.9 The three questions, applied . . . 191
22.10 The lowest set bit is the structure . . . 192
22.11 Notes and further reading . . . 192

Lessons from Part III **193**
The patterns that kept showing up . . . 193
Six meta-lessons . . . 194
Bridge to Part IV . . . 194

IV String Algorithms 196

23 Sliding the pattern, comparing every shift **198**
23.1 Finding a needle in a string haystack . . . 198
23.2 The double loop . . . 198
23.3 Running it on the canonical text . . . 199
23.4 When the worst case bites . . . 200
23.5 Why grep still ships it . . . 202
23.6 The three questions, applied . . . 202
23.7 What brute force teaches you . . . 203
23.8 Notes and further reading . . . 203

24 Compiling the pattern into a machine **205**
24.1 What if the pattern were a machine? . . . 205
24.2 States as prefix lengths . . . 205
24.3 The transition function . . . 206
24.4 Building the table . . . 207
24.5 Running the machine on the canonical text . . . 208
24.6 What the DFA costs to build . . . 209
24.7 The three questions, applied . . . 210
24.8 What the automaton view teaches you . . . 211
24.9 Notes and further reading . . . 211

25 Failing forward, never backward **212**
25.1 The DFA's wasted dimension . . . 212
25.2 The failure function . . . 212
25.3 Computing the failure function . . . 213
25.4 The matching loop . . . 214
25.5 Running on the canonical text . . . 215
25.6 abracadabra: the canonical demo . . . 216
25.7 KMP and the DFA are the same machine . . . 217
25.8 The three questions, applied . . . 219
25.9 What KMP teaches you . . . 220
25.10 Notes and further reading . . . 221

26 Skipping ahead by reading the mismatch **222**
26.1 Reading the window right-to-left . . . 222
26.2 The bad-character heuristic . . . 223
26.3 The good-suffix heuristic . . . 224
26.4 Combining the two: the matcher . . . 226
26.5 Running on the canonical text . . . 227
26.6 Sublinear in practice . . . 229
26.7 When sublinear breaks . . . 229
26.8 The three questions, applied . . . 230
26.9 What Boyer-Moore teaches you . . . 231
26.10 Notes and further reading . . . 232

27 Matching by fingerprint **233**
27.1 The hash table trick, applied to a sliding window . . . 233
27.2 A polynomial hash for strings . . . 234
27.3 Rolling the hash forward in $O(1)$ . . . 235
27.4 The matching loop with Las Vegas verification . . . 236

27.5 Running on the canonical text . . . 238
27.6 When hashes collide . . . 239
27.7 Picking the modulus . . . 240
27.8 The three questions, applied . . . 240
27.9 What Rabin-Karp teaches you . . . 241
27.10 Notes and further reading . . . 242

28 A trie that fails forward, on many patterns at once **243**
28.1 What if you wanted to match a thousand patterns at once? . . . 243
28.2 A trie of patterns . . . 244
28.3 Failure links: KMP's idea on a tree . . . 245
28.4 Output links: don't miss matches buried inside matches . . . 246
28.5 The single-pass scan . . . 247
28.6 Running on the canonical text . . . 248
28.7 Build once, scan many . . . 249
28.8 The three questions, applied . . . 250
28.9 What Aho-Corasick teaches you . . . 251
28.10 Notes and further reading . . . 251

29 Indexing every substring with a sorted array **253**
29.1 Every substring is a prefix of some suffix . . . 253
29.2 The naive sort . . . 254
29.3 Manber-Myers and the doubling trick . . . 255
29.4 Kasai's LCP in linear time . . . 257
29.5 Pattern search in $O(m \log n)$ . . . 259
29.6 Finding the longest repeated substring . . . 260
29.7 What about suffix trees and suffix automata? . . . 262
29.8 The three questions, applied . . . 262
29.9 What suffix arrays teach you . . . 264
29.10 Notes and further reading . . . 264

30 Counting the edits between two strings **266**
30.1 When the match is fuzzy . . . 266
30.2 Hamming as the trivial case . . . 267
30.3 The edit-distance recurrence . . . 267
30.4 Filling the table bottom-up . . . 268
30.5 Reconstructing the alignment . . . 270
30.6 Subsequences and substrings . . . 272
30.7 A bridge to Part VI . . . 274
30.8 The three questions, applied . . . 275
30.9 What edit distance teaches you . . . 276
30.10 Notes and further reading . . . 277

Lessons from Part IV **279**
The patterns that kept showing up . . . 279
Meta-lessons . . . 280
What's next . . . 280

V Graphs 282

31 Three ways to write the same graph **284**
31.1 Vertices, edges, and four combinations . . . 284
31.2 One graph to run through all three . . . 284
31.3 Edge list: the rawest encoding . . . 285
31.4 Adjacency matrix: $O(1)$ edge queries, $\Theta(n^2)$ space . . . 286
31.5 Adjacency list: the workhorse . . . 288
31.6 When does each representation win? . . . 290
31.7 The graph that stores nothing . . . 291
31.8 The three questions, applied . . . 292
31.9 What this chapter teaches . . . 293

31.10Notes and further reading . . . 293

32 Expanding the frontier, one ring at a time **294**
32.1 The five-line skeleton . . . 294
32.2 Recording the path home . . . 295
32.3 The layered frontier . . . 297
32.4 One BFS per island . . . 298
32.5 BFS on a graph that doesn't exist . . . 299
32.6 The three questions, applied . . . 300
32.7 What this chapter teaches . . . 301
32.8 Notes and further reading . . . 302

33 What you see when you go deep before you go wide **303**
33.1 DFS in its natural form . . . 303
33.2 When recursion overflows the stack . . . 304
33.3 Discovery, finish, and the parenthesis structure . . . 305
33.4 A gray vertex is proof of a cycle . . . 307
33.5 Edge classification, briefly . . . 308
33.6 One DFS per island . . . 308
33.7 The three questions, applied . . . 310
33.8 What this chapter teaches . . . 311
33.9 Notes and further reading . . . 311

34 BFS with a heap **312**
34.1 The relaxation step . . . 312
34.2 Twenty lines around a heap . . . 312
34.3 A run, traced by hand . . . 314
34.4 Why non-negative weights . . . 315
34.5 The three questions, applied . . . 315
34.6 What this chapter teaches . . . 316
34.7 Notes and further reading . . . 317

35 Relaxation under negative weights and across all pairs **318**
35.1 Why a negative edge defeats the greedy choice . . . 318
35.2 Relax every edge, over and over . . . 318
35.3 Following the distances pass by pass . . . 320
35.4 One more pass reveals a negative cycle . . . 321
35.5 Shortest paths between every pair of vertices . . . 322
35.6 The three-line loop that builds a matrix . . . 323
35.7 The three questions, applied . . . 324
35.8 What this chapter teaches . . . 325
35.9 Notes and further reading . . . 326

36 Spanning trees of minimum weight **327**
36.1 The theorem both algorithms rest on . . . 327
36.2 Sorting edges until the tree is built . . . 327
36.3 Growing the tree from one vertex . . . 329
36.4 Two algorithms, one theorem . . . 330
36.5 The three questions, applied . . . 331
36.6 What this chapter teaches . . . 332
36.7 Notes and further reading . . . 332

37 Ordering vertices and finding strong components **333**
37.1 DFS writes the order in its finish times . . . 333
37.2 Peel the sources one level at a time . . . 334
37.3 Vertices that can all reach each other . . . 336
37.4 Tarjan's low-link DFS finds all SCCs in one pass . . . 336
37.5 The condensation is always a DAG . . . 337
37.6 The three questions, applied . . . 338
37.7 What this chapter teaches . . . 339
37.8 Notes and further reading . . . 339

38 Flow networks and the max-flow min-cut theorem **340**
38.1 Sending flow through a capacity network . . . 340
38.2 The residual graph turns flow into a reachability problem . . . 340
38.3 Pushing flow along augmenting paths . . . 341
38.4 Why shortest augmenting paths converge in $O(nm)$ augmentations . . . 342
38.5 Maximum flow equals minimum cut . . . 343
38.6 The three questions, applied . . . 343
38.7 What this chapter teaches . . . 344
38.8 Notes and further reading . . . 344

Lessons from Part V **346**
The patterns that kept showing up . . . 346
Meta-lessons . . . 347
What's next . . . 347

VI Dynamic Programming and Greedy Algorithms **348**

VII Specialized Domains **349**

VIII Advanced Complexity **350**

Appendices **351**

A A Python Primer **351**
A.1 Reading code as algorithms . . . 351
A.2 Variables and the assignment model . . . 351
A.3 Built-in collections . . . 351
A.4 Slicing . . . 352
A.5 Control flow . . . 352
A.6 Functions . . . 352
A.7 Lambdas . . . 353
A.8 Classes . . . 353
A.9 Dunder methods . . . 353
A.10 Generators and `yield` . . . 354
A.11 Type annotations and modern generics . . . 354
A.12 Protocols and structural typing . . . 354
A.13 Dataclasses . . . 355
A.14 Optional, union, and `None` . . . 355
A.15 Exceptions . . . 355
A.16 Context managers (`with`) . . . 356
A.17 `match` statements . . . 356
A.18 Iteration helpers . . . 356
A.19 Running the code . . . 356

B Asymptotic Notation and Analysis **357**
B.1 What the notation is hiding . . . 357
B.2 The Big-O family . . . 357
B.3 Worst, average, and best case . . . 358
B.4 Amortized analysis . . . 358
B.5 Space complexity . . . 359
B.6 The hierarchy of growth . . . 359
B.7 The Master Theorem . . . 359
B.8 Lower bounds: the model is part of the answer . . . 360
B.9 Notation hygiene . . . 360

C A Mathematical Toolkit **361**
C.1 Summations . . . 361

C.2 Logarithms . . . 362
C.3 Recurrences . . . 362
C.4 Amortized analysis methods . . . 363
C.5 The inverse Ackermann function . . . 364
C.6 Probability essentials . . . 364
C.7 Working with floors and ceilings . . . 365
C.8 When the math gets in the way . . . 365

Preface



Welcome to The Algorithm Codex.

This book is a repository of common algorithms used in all areas of computer science. It contains reference implementations in Python for many well-known (and some not-so-much) algorithms spanning from simple linear search to sorting, graphs, computational geometry, data structures, flow networks, game theory, number theory, optimization, and many other fields.

We wrote this book to serve as a complement for the main bibliography in a typical Computer Science major. You will not find comprehensive theory of algorithms in this book, or detailed analyses. However, we do present some basic intuitions into why most of the presented algorithms work and a back-of-the-envelope cost analysis whenever it makes sense.

The order in which algorithms are presented is our best attempt to build the most complex ideas on top of the simpler ones. We start with basic algorithms that everyone can understand and progressively move towards the more advanced.

The algorithms are presented in a literate programming format. The gist is that we combine code and prose in the best possible way to maximize understanding. Actually, the source code is generated from the book source, and not the other way around–that is what literate programming is, after all. Accompanying this book, you will find an open source repository with the exact implementations in this book.

You can read the book online at https://matcom.github.io/codex/.

What is this book about?

The Algorithm Codex is designed as a multifaceted resource for a broad spectrum of the computational community. **Students** will find it to be a vital bridge between theoretical university lectures and the concrete reality of modern implementations, serving as a complement to standard academic curricula.

For **professors**, the book provides a library of clean, pedagogical code that emphasizes structural clarity over complexity. **Researchers** can utilize these implementations as a reliable reference for standard algorithmic logic

when prototyping new ideas, while **developers** will gain a deeper understanding of the underlying abstractions that power their daily tools.

Ultimately, even **enthusiasts** driven by curiosity will find the logical elegance of these algorithms accessible through the lens of literate programming.

It is equally important to clarify what this project represents by stating that this is **not a programming book**; we assume that the reader is already proficient in Python 3 and understands the fundamental principles of software development. We deliberately avoid **pseudo-code** in favor of actual, runnable Python to ensure that no logic is lost in translation.

Furthermore, this work is **not an algorithm design book** and should not be viewed as a replacement for comprehensive theoretical texts such as *Introduction to Algorithms*. We do not offer **formal proofs of correctness** or exhaustive mathematical analysis, choosing instead to build intuition through back-of-the-envelope cost estimates and prose.

Finally, the Codex is **not a production cookbook** meant for direct copy-pasting into high-stakes environments; our code is optimized for readability and educational insight rather than the micro-optimizations required for production-level software.

Content of the Book

The Algorithm Codex is organized into several major parts, designed to take you from foundational concepts to specialized domains and the limits of computation:

- **Searching and Sorting**: We establish the core intuitions of algorithmic efficiency by exploring how to find and organize data in linear and logarithmic time.
- **Fundamental Data Structures**: We implement essential abstractions—including linked lists, stacks, queues, and hash tables—that serve as the building blocks for more complex systems.
- **Trees**: This part covers hierarchical data, from binary search trees to self-balancing structures and specialized variants like heaps and tries.
- **String Algorithms**: We focus on pattern matching and text processing, covering algorithms from exact matching (KMP, Boyer-Moore) to advanced suffix structures.
- **Graphs**: A significant section dedicated to relational data, covering traversals, shortest paths, spanning trees, and flow networks.
- **Dynamic Programming and Greedy Algorithms**: We delve into powerful paradigms for solving optimization problems by exploiting subproblem structure and local optimality.
- **Specialized Domains**: We explore deep subregions of Computer Science, including computational geometry, number theory, and game theory.
- **Advanced Complexity**: The book concludes with the frontiers of computation, exploring NP-completeness, approximation algorithms, and randomized approaches.

About the coding style

The code in this book is written in Python 3, specifically the 3.13 version.

We make extensive use of Python's generic syntax to write clean but fully typed methods that leverage the best and most modern practices in software development. Other than that, the code is often written in the simplest possible way that works. We don't make unnecessary optimizations like taking bounds out of a loop. On the other hand, our code is optimized in the algorithmic sense; it is fast because it exploits the inherent structure of the problem.

Since most of our code is pure, functional algorithms, we often rely on public, plain Python functions. We thus have very few classes, and the ones we have are very simple, often nothing but data stores. However, we do make heavy use of protocols and abstract classes, especially those in the Python standard library like sequences, maps, and queues.

Support The Algorithm Codex

This book is free, as in free beer and free speech, and it will always be.

The book content is licensed CC-BY-NC-SA, that means you are free to share the book in any format (HTML, ePUB, PDF) with anyone, and produce any derivatives you want, as long as you also share those freely for posterity.

The source code is licensed MIT, and thus you can use the algorithms implemented here for anything, including classes, academic work, but also writing commercial software.

The only thing you cannot do is resell the book itself or any derivative work like lecture notes, translations, etc.

If you want to support this effort, the best way to do is to buy the official PDF.

Stay in touch

Most of the chapters in this book are first published as standalone articles in The Computist Journal. Subscribing there is the best way to stay in the loop and get early access to most of the material.

Foundations

An algorithm is a recipe a machine can follow without thinking. That sentence is mostly right and a little misleading, and the difference is what this whole book is about.

By the end of this chapter, you'll be able to take any algorithm you read in the rest of the book — or anywhere else — and ask it the three questions that, between them, decide whether it's worth your time. You won't have to take any algorithm on faith again.

Most of what follows is small, deliberate work: defining what counts as an algorithm, building a way to measure how fast or slow one is without depending on the hardware you happen to own, and naming the analytical machinery the rest of the book leans on. None of it is glamorous on its own. All of it pays off in every chapter that comes after.

What counts as an algorithm

You probably already have a working definition: an algorithm is a sequence of steps a computer can follow to solve a problem. That's good enough for most of life. For this book, I want to be a little stricter.

In the Codex, an algorithm is a **formal mathematical object**: a precise strategy that exploits the structure of its input to produce a specific output, efficiently. Three properties are doing the work:

- **Finiteness.** The description fits on a finite number of pages, and on any valid input the procedure halts in a finite number of steps. Recipes that go on forever don't count, no matter how clever.
- **Correctness.** The procedure produces the right answer for every valid input — not just the ones you remembered to test.
- **Definiteness.** Every step is unambiguous. There's no room for the executor to "do the obvious thing" — that's where bugs and disagreements live.

Historically, textbooks have written algorithms in **pseudo-code** — a flexible, English-like sketch of the steps, with the understanding that you'll translate it into a real programming language to actually use it.

I've never liked pseudo-code. It hides exactly the details that matter when you sit down to implement something. The gap between *"swap the elements at positions i and j"* and *"no, you can't, you forgot you were working with an immutable list"* is where most of an algorithm's real character lives.

This book uses runnable **Python 3.13** instead. Every code block you see in a chapter is extracted into the actual `codex` Python package by a small tool called `illiterate`, and every demonstration block is executed by Quarto when the book renders. If an algorithm in the book is wrong, the book won't render. If a diagram is wrong, the diagram won't draw. The book and the code can never disagree.

The three questions

When I've implemented an algorithm — really implemented it, not just described it — I'm only halfway done. The other half is asking it three questions, in order.

I want you to internalize these three, because they're going to come back at the end of every chapter in this book, in this exact order, as the recurring closing template.

Is it correct? Does the algorithm produce the right answer on *every* valid input — not just the ones you tried, but the ones you didn't think of: the empty input, the single-element input, the input where every element is the same, the input some adversary built to mess with you?

I won't usually do formal Hoare-logic proofs of correctness. Something looser will do: an intuitive argument about why the algorithm must be right — invariants the loop preserves, exhaustive case analysis, sometimes a small inductive sketch. Enough to convince a careful skeptic; not enough to satisfy a formal verifier.

How efficient is it? Once you trust the answer is right, you ask what it cost to get there. Two resources matter: **time** (the number of operations performed) and **space** (the amount of memory used along the way). For each, I'll give the cost as a function of the input size — the bigger the input, the more interesting the question gets.

Is it optimal? The third question is the most interesting, and the one most often skipped. "Optimal" doesn't mean "fast enough." It means: is there a *theoretical* lower bound on this problem that says no algorithm can possibly do better, and does this one meet it?

When you can answer all three — yes it's correct, here's how efficient, yes it hits the lower bound — you've solved the problem completely. The computational task stops being a research question and becomes a settled fact.

You'll see in the very next chapter that even the most boring algorithm in the book — linear search — has a satisfying answer to all three.

Why I count operations, not seconds

Here's a problem. If I tell you *"my algorithm takes 0.3 seconds on a 1,000-element input,"* you've learned almost nothing useful.

You don't know what computer I ran it on. You don't know what language I wrote it in. You don't know whether 0.3 seconds is good or terrible for this problem. And you have no way to predict what happens when the input gets bigger.

To talk about efficiency in a way that survives hardware changes, language choices, and the passage of time, I'll work inside an idealized model of computation called the **Random Access Machine** — the RAM model. The name is unhelpful (it has nothing to do with the memory chip you call RAM); what matters are the three assumptions.

- **Basic operations cost one unit of time.** Arithmetic, comparisons, assignments, function calls — all unitary. Real hardware doesn't actually work this way (multiplication is slower than addition, for instance), but at the scale of comparing one algorithm against another, the difference washes out.
- **Memory is a sequence of cells.** Each cell holds one item — a number, a character, a reference.
- **Any memory cell is reachable in one unit of time.** That's the "random access" — no penalty for distance. Real hardware has caches and locality effects; I'm ignoring them on purpose, so the algorithm's structure isn't drowned out by the hardware's idiosyncrasies.

The RAM model is a deliberate caricature. It works because, for most algorithms most of the time, the caricature doesn't change the conclusion. It starts to break down in specialized areas — numerical algorithms where the cost of a single multiplication actually matters, or cache-aware algorithms where memory layout dominates the runtime. I'll flag those exceptions when they come up.

Scaling, not stopwatching

Even inside the RAM model, the absolute number of operations is rarely what I care about. What I care about is how the cost **grows** as the input gets bigger.

The reason matters. Imagine I have an algorithm that scales **linearly**: doubling the input doubles the running time. Yours scales **quadratically**: doubling the input quadruples the running time. Today, on a small input, your algorithm might beat mine — maybe you wrote it in C++ and I wrote mine in Python, so your constants are smaller. But there's some input size beyond which my "slower" linear algorithm wins, no matter what hardware it runs on, no matter what language it was written in. And as the years pass and the inputs people care about get larger, that crossing-point matters more, not less.

Scaling behavior is hardware-independent, language-independent, future-proof in a way that "0.3 seconds" never is. It tells you the shape of the algorithm's cost, which is what you actually need to know.

To talk precisely about scaling, I'll use **asymptotic notation**. The one you'll see most is the big-O. When I write that an algorithm runs in $O(n)$ time, I mean: for inputs large enough, the running time is at most some constant multiple of n. Doubling the input at most doubles the running time. Formally, there's a constant c and a threshold n_0 such that the running time $f(n)$ stays below $c \cdot g(n)$ for every $n > n_0$, where $g(n) = n$.

The nice thing about asymptotic notation is that it throws away exactly the details I want thrown away: constant factors, low-order terms, anything that depends on the hardware or the implementation language. An algorithm that takes $3n + 2$ steps is $O(n)$. An algorithm that takes $0.001n$ steps is also $O(n)$. From the asymptotic point of view, they're the same algorithm.

I'm not going to be religious about big-O proofs. Most of the time, intuitions like *"one for-loop is $O(n)$, two nested for-loops are $O(n^2)$"* are exactly what you need. When an algorithm has a subtler cost — $O(n \log n)$, for instance, neither linear nor quadratic — I'll spend a paragraph on why, and that paragraph will earn its keep.

Where this leaves you

You now have the tools the rest of the book leans on: the definition of an algorithm, the three questions, the RAM model, and asymptotic notation. None of these is glamorous on its own. Together, they're enough to think clearly about any of the roughly hundred algorithms this book is going to throw at you.

The next chapter starts with the simplest one: linear search. It'll be almost embarrassing how little it does — and then, by the time you've asked it the three questions, it'll be more interesting than you expected.

Notes and further reading

The three-question template — correctness, efficiency, optimality — is the implicit structure of any classical algorithms text, but it's rarely named that explicitly. For the formal treatment of correctness via loop invariants, see Cormen, Leiserson, Rivest and Stein's *Introduction to Algorithms* (CLRS), 4th ed., chapter 2. For asymptotic notation made rigorous, CLRS chapter 3 is the standard reference. The RAM model is treated thoroughly in Sedgewick and Wayne's *Algorithms*, 4th ed., section 1.4. The literate-programming idea this book uses goes back to Knuth's *Literate Programming* (1992); the specific tooling — `illiterate` for extraction, Quarto for rendering — is described in the project's `AGENT.md`.

Part I

Searching and Sorting

This first part of the Codex is where you meet **the structure-versus-cost trade**, the recurring negotiation that runs through every chapter in the rest of the book. Searching and sorting look like utility problems — *"find this thing," "put these things in order"* — but underneath them sits the deepest question in algorithm design: **what can structure buy you, and what does it cost to create?**

The seven chapters in this part walk that trade from end to end.

- **Linear search** (chapter 1) sets the baseline: when you know nothing about your data, you have to look at all of it. The adversary argument proves this is unavoidable.
- **Binary search** (chapter 2) shows what one structural assumption — order — buys you: an exponential collapse in the search cost, from $O(n)$ to $O(\log n)$.
- **Basic sorting** (chapter 3) asks how to *create* that order, and lands at the cynosure that organizes the rest of the part: an **inversion** is a pair of elements that's out of order, and sorting is the process of driving the inversion count to zero.
- **Efficient sorting** (chapter 4) breaks the quadratic ceiling by attacking inversions in bulk — the **local-vs-global** duality that makes merge sort and quick sort reflections of each other across the recursive split.
- **Selection** (chapter 5) collapses the cost further by recursing on only *one* side of a partition: $T(n) = T(n/2) + O(n)$ instead of $T(n) = 2T(n/2) + O(n)$, which turns $O(n \log n)$ into $O(n)$.
- **Linear-time sorting** (chapter 6) takes the lower bound apart. The $\Omega(n \log n)$ floor was a property of the *comparison model*, not of sorting itself. Look inside the elements — use their intrinsic structure as an address — and the floor drops to $\Omega(n)$.
- **Sorting in practice** (chapter 7) closes the loop by showing what production code actually does: not any single algorithm uniformly, but a *composition* of them, with insertion sort for small subarrays, quicksort or merge sort for the recursive divide, and run-detection or depth-limited fallbacks for the corner cases.

Five algorithmic moves come back through the rest of the book: **exploit structure, halve the problem, randomize against worst cases, look inside the data, and compose moves rather than pick one**. Every part that follows builds on at least one of them.

1 Looking at everything, in order

You have a list of names — say, the guest list for a wedding — and you need to know whether someone in particular is on it. There's no order to the list; someone typed names in as people RSVPed. What do you do?

You read the list, top to bottom, until you either find the name or run out of list. That's it. That's the algorithm. It's five lines of Python, and **nothing can do better** when the input has no structure. Linear search is the *only* algorithm for unstructured search, up to symmetry. That's a satisfying thing to learn in chapter one.

The adversary — someone who can rearrange the list while you're not looking, specifically to make you look at every element before you can answer — is going to come back several times in this book. Linear search is where you meet them.

1.1 A sequence and nothing more

The setting is the simplest one possible. You have a **sequence** of items — a finite ordered collection where each item has an index and you can ask for the i-th element. You don't know anything else about it. The items might be sorted, might not. There might be duplicates, might not. You're just walking the indices.

Python's standard library captures this with the `Sequence` protocol. Lists, tuples, strings, `range` objects — all of them are sequences. I'll use `Sequence[T]` as the type hint for input, and that's the most generic shape the algorithms in this chapter can handle.

```
from typing import Sequence


def find[T](x: T, items: Sequence[T]) -> bool:
    for y in items:
        if x == y:
            return True
    return False
```

The whole algorithm in five lines. You walk the sequence; if you ever match, you stop and answer yes; if you fall off the end, you answer no. The early return is what makes the *average* case better than the worst case, though only by a constant factor.

Running it on a few inputs:

```
from codex.search.linear import find

print(find(3, [1, 2, 3, 4, 5]))   # True - found at position 2, walked three elements
print(find(9, [1, 2, 3, 4, 5]))   # False - walked all five, gave up
print(find("apple", ["banana", "cherry", "apple"]))  # True - strings work fine
print(find(1, []))                # False - nothing to check, answer is immediate
```

```
True
False
True
False
```

1.2 From "is it there?" to "where?" and "how many?"

`find` tells you whether an item exists. Often, you want more. **Where is it?** Or, when duplicates matter, **how many of it are there?**

The first variation is `index` — the smallest index i where `items[i] == x`. If the item is not present, you have to return something signalling absence; I'll use `None` rather than raising an exception, so the caller doesn't have to wrap the call in a `try`.

```
def index[T](x: T, items: Sequence[T]) -> int | None:
    for i, y in enumerate(items):
        if x == y:
            return i
    return None
```

```
from codex.search.linear import index

print(index(3, [1, 2, 3, 4]))     # 2 - found at position 2
print(index(9, [1, 2, 3, 4]))     # None - not present
print(index(2, [2, 2, 2]))        # 0 - first occurrence wins; this is the contract
```

```
2
None
0
```

The contract on the duplicates case matters. `index` returns the *first* occurrence — the smallest index. That's by convention; some libraries return the last. Pick a side, document it, never let your callers wonder.

The second variation is `count`. Instead of bailing out on the first match, you walk the whole sequence and tally.

```
def count[T](x: T, items: Sequence[T]) -> int:
    c = 0
    for y in items:
        if x == y:
            c += 1
    return c
```

```
from codex.search.linear import count

print(count(2, [1, 2, 2, 3, 2]))  # 3 - three twos
print(count(9, [1, 2, 3]))        # 0 - none
print(count("a", "banana"))       # 3 - strings are sequences of characters
```

```
3
0
3
```

`count` has no early-out — it has to look at every element, every time.

1.3 Finding the extremes

A different question, same paradigm. Instead of asking *"is x in the sequence?"*, you ask *"what's the smallest element?"* or *"what's the largest?"* The algorithm is the same shape: walk everything, keep the best so far.

"Smallest" and *"largest"* assume some way to compare items. For numbers and strings, Python's built-in `<` works. For arbitrary types, you might want to define your own ordering — for example, comparing tuples by their second element, or sorting URLs by domain. So I'll factor the comparison out into a parameter.

The book's convention for this lives in `codex/types.py` — a small module of cross-cutting protocols. An `Ordering[T]` is a callable that takes two `T`s and returns a negative number if the first is smaller, zero if they're equal, positive if the first is larger. That's the same convention C uses for `qsort` and Java uses for `Comparator.compare`.

```
from typing import Callable

type Ordering[T] = Callable[[T, T], int]
```

For the common case — numbers, strings, anything with a built-in `<` — you don't want to write a comparator by hand. So `default_order` delegates to the items themselves.

```
def default_order[T](x: T, y: T) -> int:
    if x < y:
        return -1
    elif x == y:
        return 0
    else:
        return 1
```

Now `minimum` writes itself. Start with the first element as a placeholder for "best so far"; walk the rest; update whenever you find something smaller.

```
from codex.types import Ordering, default_order


def minimum[T](items: Sequence[T], order: Ordering[T] = default_order) -> T:
    m = items[0]
    for x in items:
        if order(x, m) < 0:
            m = x
    return m
```

The only way `minimum` can fail is if the sequence is empty — there's no "smallest of nothing." That'll raise an `IndexError` on the first line, which is the right behavior. Don't catch it; let it propagate.

For `maximum`, you might be tempted to write a near-copy of `minimum` with `>` instead of `<`. Don't. The asymmetry between the two is just the direction of the ordering — and you already have a parameter that controls the direction. Flip its arguments.

```
def maximum[T](items: Sequence[T], order: Ordering[T] = default_order) -> T:
    return minimum(items, lambda x, y: order(y, x))
```

That's the whole implementation: `maximum` is `minimum` with the comparison reversed. The lambda swaps the arguments to `order`, which is the cleanest way to say *"flip the direction of this ordering."*

```
from codex.search.linear import minimum, maximum

items = [4, 2, 6, 5, 7, 1, 0]
print(minimum(items))              # 0 - smallest
print(maximum(items))              # 7 - largest
print(minimum(["banana", "apple", "cherry"]))  # apple - strings compare lexicographically

# A custom ordering: compare strings by length, shortest first
by_length = lambda x, y: len(x) - len(y)
print(minimum(["hippopotamus", "cat", "elephant"], by_length))  # cat
```

```
0
7
apple
cat
```

1.4 The three questions, applied

You've now seen five variations of the same algorithm. The shape is identical in every case — walk every element, do some bookkeeping, return. Time to put it to the three questions.

Is it correct?

`find` is correct because **there's nowhere for x to hide.** If x is in the sequence, then there's some index i where `items[i] == x`. The `for` loop visits every index without exception. When it gets to i, the equality check succeeds, and the function returns `True`. If x is not in the sequence, the loop completes without ever finding a match, and the function returns `False`. There's no gap in the argument.

If you want a more formal flavor, you can express the same idea as a **loop invariant**: *at the start of iteration i, the target x has not been found among the first $i-1$ elements.* The invariant holds before the loop starts ($i = 0$, no elements examined, the statement is vacuously true), is preserved by each iteration (if x matched at iteration $i-1$, the function would have returned `True` and the loop wouldn't be running), and at termination tells you that x is not in the first n elements — which is the entire sequence. The variants (`index`, `count`, `minimum`, `maximum`) all use the same shape of invariant; I'll leave you to write them out yourself if you want to practice.

How efficient is it?

In the RAM model, each comparison costs one unit of time. The loop runs at most n times — exactly n times in the worst case for `find` (target absent, or present at the very last position). For `count`, `minimum`, and `maximum`, the loop *always* runs n times, since they have no early exit. Either way: $O(n)$ **time, linear in the size of the input.**

Space is $O(1)$. The loop variable and the running "best so far" each occupy constant memory; the input doesn't need to be copied. Nothing about the algorithm grows with n.

Is it optimal?

This is the interesting one. Linear search is $O(n)$ — but is it the *best possible* $O(n)$? Could some cleverer algorithm answer in, say, $O(\sqrt{n})$ or $O(\log n)$ on an unstructured sequence?

The answer is no, and the proof is the **adversary argument**. Imagine you're playing a game against me. You bring an algorithm that claims to decide whether x is in a sequence of length n by inspecting fewer than n elements — say, $n-1$ of them. I'm the adversary, and I get to choose what's in the sequence *as you go* — I only have to commit to a value when you look at a position.

You inspect $n-1$ positions. There's exactly one position you didn't look at. Whatever answer you give me — *"yes, x is present"* or *"no, x is not present"* — I can falsify it by retroactively setting the unexamined position to the value that makes you wrong. If you answered yes without ever seeing x, I'll fill the unexamined slot with something else. If you answered no, I'll fill the unexamined slot with x itself.

So any correct algorithm must inspect every element in the worst case. That gives you a lower bound of $\Omega(n)$ — any algorithm solving this problem on an unstructured sequence has to do at least linear work. Linear search hits that bound exactly, which makes it **tightly optimal** for this problem.

That's a strong statement. It means: until the input has more structure than *"a finite ordered collection where I can ask for the i-th element,"* you cannot do better than reading the whole thing. Knowing nothing has a price, and the price is linear.

1.5 The price of knowing nothing

Linear search is correct because the loop has nowhere to hide a missed match. It runs in $O(n)$ time and $O(1)$ space. And it's optimal — the adversary argument says no algorithm can answer the "is x in this sequence?" question with fewer than n inspections in the worst case, when the input has no structure.

Linear search is *the* solution to unstructured search — the one any other approach has to beat by knowing something the algorithm here doesn't. The rest of the book is the story of what happens when you know that "something" — when the input is sorted, or hashed, or arranged in a tree, or laid out on a graph.

In the next chapter, you'll meet the simplest piece of structure you can demand: **order**. If the sequence is sorted, you can do dramatically better than $O(n)$ — and the algorithm that exploits this is, in my opinion, the most beautiful one in the book.

1.6 Notes and further reading

The adversary argument is the standard lower-bound technique for comparison-based search and is treated formally in CLRS (4th ed.) §8.1 in the context of sorting; the same logic applies here. Sedgewick and Wayne's *Algorithms* (4th ed.) §1.2 introduces sequential search alongside the basic API. The `Ordering` convention used here (negative / zero / positive) is the one Python's now-removed `cmp` parameter used, the one C's `qsort` uses, and the one Java's `Comparator.compare` returns; it generalizes more cleanly than a boolean less-than for the comparison-based algorithms that come later in the book.

2 Throwing half away

You have a phone book — the kind people used to keep next to the phone, sorted alphabetically — and you need to find one name. You don't read it cover to cover. You open it somewhere in the middle, see whether the name you want comes before or after the page in front of you, then throw away half the book and open the remaining half in the middle. Repeat until you're holding the right page.

This is binary search. It has three forms: a straight "is x in the sequence?" answer, a *bisection* that returns an insertion point, and — most usefully — a way to search the answer space of any problem with monotonic structure. That third form is the one that keeps paying off through the rest of the book.

Every comparison cuts the search space in half. For a sorted list of one million items, that's about twenty comparisons to find anything; for a billion items, about thirty. Doubling the input adds *one comparison*, not doubles the time. Linear search would scan all billion items. Binary search reads thirty of them and tells you the answer with certainty.

2.1 The half-and-half algorithm

The mechanics are exactly the phone-book strategy. Keep two indices, `lo` and `hi`, marking the bounds of the part of the sequence you haven't ruled out yet. As long as the interval is non-empty, look at the middle and use one comparison to throw away one half.

```
from typing import Sequence
from codex.types import Ordering, default_order


def binary_search[T](
    x: T, items: Sequence[T], order: Ordering[T] = default_order
) -> int | None:
    lo, hi = 0, len(items) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        cmp = order(x, items[mid])
        if cmp == 0:
            return mid
        elif cmp < 0:
            hi = mid - 1
        else:
            lo = mid + 1
    return None
```

The three branches are doing exactly what the phone book was doing. If the middle item *is* the target, you're done. If the target is smaller, the right half (everything from `mid` onward) is ruled out, so `hi` moves to `mid - 1`. If the target is larger, the left half is ruled out, so `lo` moves to `mid + 1`. The loop exits when `lo > hi` — at that point the interval is empty and the target was never in the sequence to begin with.

```
from codex.search.binary import binary_search

items = [1, 3, 5, 7, 9, 11, 13, 15]
print(binary_search(7, items))     # 3 - found at index 3
print(binary_search(8, items))     # None - not present
print(binary_search(1, items))     # 0 - first element, edge case
print(binary_search(15, items))    # 7 - last element
print(binary_search(5, []))        # None - empty input
```

```
3
None
0
7
None
```

`mid = (lo + hi) // 2` uses integer division, which always rounds toward `lo` — that's important for termination, because it guarantees `mid < hi` whenever `lo < hi`, so the `lo = mid + 1` branch makes real progress. The other detail is that the input *must* be sorted in the order that `order` defines. If you pass an unsorted list, binary search gives you garbage with confidence. There's no easy way for the algorithm to detect this — sorting is a precondition, not something the function checks. If you want a safety net, sort first and document the contract loudly.

2.2 Finding insertion points: `bisect_left` and `bisect_right`

`binary_search` answers a yes-or-no question with a position attached. But there's a closely related question that turns out to be more useful in practice: **where *would* this item go, if I were to insert it while keeping the sequence sorted?**

That position exists whether or not the item is already present. If you're adding a new entry to an already-sorted list, you don't want "is it there?" — you want "where do I put it?" And if the item is *already* there in duplicates, you have a contract decision to make: do you return the position *before* the first equal element, or *after* the last one?

The standard convention — the one Python's `bisect` module uses, and the one I'll use here — gives you both. `bisect_left` returns the leftmost spot where the item could be inserted while keeping the list sorted. `bisect_right` returns the rightmost. For a list with no duplicates of the target, the two return the same answer; for a list with duplicates, they bracket the run.

```
def bisect_left[T](
    x: T, items: Sequence[T], order: Ordering[T] = default_order
) -> int:
    lo, hi = 0, len(items)
    while lo < hi:
        mid = (lo + hi) // 2
        if order(items[mid], x) < 0:
            lo = mid + 1
        else:
            hi = mid
    return lo
```

The shape is recognizably binary search, but the loop condition is `lo < hi` (not `<=`) and `hi` shrinks to `mid` (not `mid - 1`). That's because the answer is now an *insertion index*, which can legally be one past the end of the list — so the search space is `[0, len(items)]`, and the loop converges when `lo == hi`.

`bisect_right` is the same skeleton with one comparison flipped.

```
def bisect_right[T](
    x: T, items: Sequence[T], order: Ordering[T] = default_order
) -> int:
    lo, hi = 0, len(items)
    while lo < hi:
        mid = (lo + hi) // 2
        if order(items[mid], x) <= 0:
            lo = mid + 1
        else:
            hi = mid
    return lo
```

The only change from `bisect_left` is the `<=` instead of `<`. When the middle item equals the target, `bisect_left` treats it as "the run starts here or earlier" and moves left; `bisect_right` treats it as "the run ends here or later" and moves right. One bit of difference, two different answers.

```
from codex.search.binary import bisect_left, bisect_right

items = [1, 3, 3, 3, 5, 7]
print(bisect_left(3, items))       # 1 - leftmost spot for a 3 (before the run)
print(bisect_right(3, items))      # 4 - rightmost spot for a 3 (after the run)
print(bisect_left(4, items))       # 4 - between the 3s and the 5; same for both
print(bisect_right(4, items))      # 4
print(bisect_left(0, items))       # 0 - smaller than everything, goes at the start
print(bisect_right(8, items))      # 6 - larger than everything, goes at the end
```

```
1
4
4
4
0
6
```

The pair `(bisect_left, bisect_right)` doesn't just answer *where to insert*. It also gives you, almost for free, the **count** of an element in a sorted sequence — just subtract: `bisect_right(x, items) - bisect_left(x, items)`. That's $O(\log n)$ instead of the $O(n)$ count you wrote in the last chapter. Structure delivers twice.

2.3 Searching the answer, not the data

So far I've used binary search to find something in a sequence. But the actual algorithm doesn't care whether you have a sequence at all. The only thing it needs is a *monotonic comparison* — a question whose answer flips from one value to another exactly once over the range you're searching. The "sequence" part was just one way to give that monotonic structure to a problem.

Here's the generalization. Suppose you have a predicate $P(k)$ — a function that takes an integer and returns a boolean — and you know that P is **monotonic**: there's some threshold below which P is `True` and above which P is `False`. Binary search can find that threshold without you ever materializing a sequence.

```
from typing import Callable


def binary_search_predicate(
    predicate: Callable[[int], bool],
    lo: int,
    hi: int,
) -> int:
    """
    Return the largest integer in [lo, hi] for which predicate is True,
    assuming predicate is True on some prefix [lo, k] and False on [k+1, hi].
    Returns lo - 1 if no value satisfies the predicate.
    """
    while lo <= hi:
        mid = (lo + hi) // 2
        if predicate(mid):
            lo = mid + 1
        else:
            hi = mid - 1
    return hi
```

The body is almost identical to `binary_search` — same `lo`/`hi`/`mid` machinery, same halving — but instead of comparing the target to a sequence element, you evaluate the predicate at `mid` and use its boolean answer

to decide which half to discard. The function returns `hi` at termination because, after the loop, `hi` ends up pointing at the largest satisfier (the search has narrowed past the boundary).

The canonical use is **integer square root** — the largest integer k such that $k^2 \leq n$. The predicate is monotonic: as k grows, $k^2 \leq n$ stays `True` until you cross the root, then stays `False` forever. Binary search finds the boundary.

```
def integer_sqrt(n: int) -> int:
    if n < 0:
        raise ValueError("integer_sqrt requires n >= 0")
    return binary_search_predicate(lambda k: k * k <= n, 0, n)
```

```
from codex.search.binary import integer_sqrt

print(integer_sqrt(10))    # 3 - because 3*3 = 9   10 < 16 = 4*4
print(integer_sqrt(16))    # 4 - exact square
print(integer_sqrt(0))     # 0 - boundary
print(integer_sqrt(2**40)) # 1048576 - works fine on huge inputs, ~40 iterations
```

```
3
4
0
1048576
```

For $n = 2^{40}$, the search range is $[0, 2^{40}]$, and the loop terminates in roughly $\log_2(2^{40}) = 40$ iterations. That's the binary-search speedup applied to a problem that never had a sorted sequence in the first place — just a question with monotonic structure.

This pattern shows up everywhere. *"What's the smallest server count that handles peak load?"* — monotonic. *"What's the largest learning rate that converges?"* — monotonic. *"What's the maximum bandwidth the network can sustain?"* — monotonic. Any time the answer to a yes/no question stays the same once it flips, binary search on the answer is in play.

2.4 The three questions, applied

Is it correct?

`binary_search` is correct because of an invariant the loop preserves: **if x is anywhere in `items`, it's somewhere in `items[lo:hi+1]`**. That's true at the start, when the interval is the whole sequence. Each iteration looks at `items[mid]`, and the three branches preserve the invariant: if `items[mid] == x`, the function returns; if `x < items[mid]`, then x (if present) can't be at index $\geq$ mid, so `hi = mid - 1` is still a valid invariant; symmetric for the other branch. When the loop exits with `lo > hi`, the interval is empty — which, by the invariant, means x was never in `items`.

The same loop-invariant argument carries over to `bisect_left` (the invariant is *"the leftmost insertion point is in `[lo, hi]`"*) and to `binary_search_predicate` (the invariant is *"the threshold is in `[lo - 1, hi]`"*). One algorithmic shape, three closely related contracts.

How efficient is it?

Every iteration cuts the interval in half. The interval starts at length n and shrinks geometrically: after k iterations, its length is at most $n/2^k$. The loop exits when the length hits zero, which happens once $2^k \geq n$ — that is, $k \geq \log_2 n$. So the loop runs at most $\lceil \log_2 n \rceil$ times, giving $O(\log n)$ time. Space is $O(1)$ — just the two indices and the loop locals.

The tactile re-statement: binary search makes one comparison per *doubling of the input*. A million-item list takes about 20 comparisons; a billion takes about 30; a quintillion takes about 60. The logarithm is the answer to *"how many times do I have to double 1 before I reach n?"* — and for the inputs you care about in real life, that answer is always small.

Is it optimal?

This is the chapter's punch. **No comparison-based search algorithm on a sorted array can do better than $\lceil \log_2(n+1) \rceil$ comparisons in the worst case**, and binary search hits that bound exactly.

The argument is the **decision-tree lower bound**. Any algorithm that makes only comparisons can be drawn as a binary decision tree: each internal node is a comparison, and each leaf is one of the possible outcomes the algorithm distinguishes. There are $n+1$ possible outcomes here — the target is at index 0, or at index 1, …, or at index $n-1$, or not present at all. A binary tree of depth k has at most 2^k leaves, so to fit $n+1$ leaves you need depth $\geq \lceil \log_2(n+1) \rceil$. That's the worst-case path through the tree, and therefore the worst-case number of comparisons any comparison-based algorithm must make.

Binary search achieves $\lceil \log_2(n+1) \rceil$ comparisons in the worst case. It's tightly optimal — within the comparison model on a sorted sequence, there is no better algorithm waiting to be discovered.

2.5 Information per comparison

Binary search is correct because the invariant *"if x is in the sequence, it's still in the current interval"* survives every iteration. It runs in $O(\log n)$ time and $O(1)$ space. And it's optimal: the decision-tree lower bound says no comparison-based algorithm on a sorted array can do better than $\lceil \log_2(n+1) \rceil$ comparisons in the worst case, and binary search meets that bound exactly.

The principle that goes beyond binary search itself: **every useful comparison must rule out a constant fraction of the remaining possibilities**. Linear search rules out one element per comparison, a fraction that shrinks to zero as n grows. Binary search rules out half — a constant fraction, independent of n. The first gives you $O(n)$; the second gives you $O(\log n)$. The gap between them is exactly the gap between *information per comparison* and *information per element*.

That principle is going to come back. Every $O(\log n)$ algorithm in the rest of the book — the balanced trees in Part III, the heap operations, the segment-tree queries — earns its logarithm by ruling out a constant fraction of remaining work per step. Binary search is the simplest case; recognising the same shape elsewhere is most of the skill.

The next chapter flips the question: how do you create the order that binary search exploited?

2.6 Notes and further reading

Binary search is treated formally in CLRS (4th ed.) §2.3 in its recursive divide-and-conquer form; the iterative form used here is the one Knuth analyzes in *The Art of Computer Programming* Vol. 3 §6.2.1, where he famously notes that the first correct published binary search appeared in 1962, sixteen years after the first published incorrect one. Python's `bisect` module in the standard library is the canonical reference implementation for `bisect_left` and `bisect_right` — worth reading the CPython source for the off-by-one details. The predicate-based variant under the name "binary search on the answer" is the standard competitive-programming idiom; the CP-Algorithms website's article on binary search is a good catalogue of its variants.

3 Fixing inversions, one at a time

An unsorted list is just a list. A sorted list is a different beast — you can binary-search it in $O(\log n)$ (you saw that last chapter); you can find duplicates by walking once through it in $O(n)$; you can compute medians, percentiles, and any range query in time that depends only on the *answer*, not on the data. Sorting is the act of granting a list those superpowers, and the cost is paid once.

Three classical sorts — *selection*, *insertion*, *bubble* — are all quadratic for what's almost the same deep reason. **An inversion is a pair of positions** (i, j) **with** $i < j$ **but items**$[i] >$ **items**$[j]$ — the local evidence that a list isn't sorted. The whole chapter is about how to remove them.

The same image carries the next chapter, with a twist. The sorts here fix inversions *one at a time*; the sorts in the next fix *many at once*. The factor between $O(n^2)$ and $O(n \log n)$ is exactly the gap between those two strategies.

3.1 Inversions: the distance from sortedness

An inversion is a pair of indices that's out of order. Formally: given a sequence `items` and an ordering, an inversion is a pair (i, j) with $i < j$ and items$[i] >$ items$[j]$.

A sorted sequence has **zero inversions** — every pair is in order, by definition. A reverse-sorted sequence has the **maximum**: every pair is out of order, which gives you $\binom{n}{2} = n(n-1)/2$ inversions. A random sequence sits roughly in the middle: a uniformly random permutation has an expected $n(n-1)/4$ inversions, half the maximum, because each of the $\binom{n}{2}$ pairs is equally likely to be either ordered or not.

So the inversion count is a *distance from sortedness*. It runs from zero (already there) to $n(n-1)/2$ (worst possible). Sorting an array means driving its inversion count to zero. The choice of algorithm is the choice of *how* you remove them.

The brute-force way to count inversions is to look at every pair:

```
from typing import Sequence
from codex.types import Ordering, default_order


def count_inversions[T](
    items: Sequence[T], order: Ordering[T] = default_order
) -> int:
    n = len(items)
    count = 0
    for i in range(n):
        for j in range(i + 1, n):
            if order(items[i], items[j]) > 0:
                count += 1
    return count
```

It's $O(n^2)$, which is fine — counting inversions is going to be a teaching aid in this chapter, not a production tool. (There's an $O(n \log n)$ way to count them as a side effect of merge sort; you'll see it in the next chapter.)

```
from codex.sort.inversions import count_inversions

print(count_inversions([1, 2, 3, 4, 5]))            # 0 - already sorted
print(count_inversions([5, 4, 3, 2, 1]))            # 10 - reverse sorted: n(n-1)/2 = 5*4/2
print(count_inversions([3, 1, 4, 1, 5, 9, 2, 6]))   # 8 - somewhere in between
```

```
0
10
8
```

3.2 Selection sort: find the extremum, repeat

The most direct way to sort: stand at position 0 and ask *"what should go here?"* The answer is the minimum of the whole array. Find it, swap it into position 0, then repeat for position 1 (the minimum of what's left), position 2, and so on.

```python
from typing import MutableSequence
from codex.types import Ordering, default_order


def selection_sort[T](
    items: MutableSequence[T], order: Ordering[T] = default_order
) -> None:
    n = len(items)
    for i in range(n):
        min_idx = i
        for j in range(i + 1, n):
            if order(items[j], items[min_idx]) < 0:
                min_idx = j
        items[i], items[min_idx] = items[min_idx], items[i]
```

The inner loop is exactly the `minimum`-finding pattern from chapter 1, scoped to the unsorted tail. The outer loop runs n times; the inner loop runs $n-i-1$ times on iteration i. Total comparisons: $1+2+\cdots+(n-1) = n(n-1)/2$. That's $O(n^2)$, regardless of the input — selection sort makes the same number of comparisons on a sorted array as on a random one.

```python
from codex.sort.basic import selection_sort

items = [3, 1, 4, 1, 5, 9, 2, 6]
selection_sort(items)
print(items)   # [1, 1, 2, 3, 4, 5, 6, 9] - sorted in place
```

```
[1, 1, 2, 3, 4, 5, 6, 9]
```

Selection sort swaps `items[i]` with `items[min_idx]`, which can be a *long* swap — `min_idx` might be all the way at the end of the array. That matters for the inversion analysis. A long swap can change the inversion count by a lot more than one, in either direction. So selection sort doesn't tell a clean inversion story. Its $O(n^2)$ comes from a different argument: n passes, each doing $O(n)$ work to find an extremum. The inversion story belongs to the next two algorithms.

3.3 Insertion sort: each element finds its place

The classic *"how you sort a hand of cards"* algorithm. Take the second card; slot it before or after the first. Take the third; slide it left until it's in the right place. Take the fourth; same idea. At iteration i, the prefix `items[0:i]` is already sorted, and your only job is to insert `items[i]` into its right place among them.

The insertion happens by repeated adjacent swaps. You compare `items[i]` to its left neighbor; if it's smaller, swap them, then compare the result to *its* new left neighbor; keep going until the element either reaches index 0 or meets something it's not smaller than.

```
def insertion_sort[T](
    items: MutableSequence[T], order: Ordering[T] = default_order
) -> None:
    for i in range(1, len(items)):
        j = i
        while j > 0 and order(items[j], items[j - 1]) < 0:
            items[j], items[j - 1] = items[j - 1], items[j]
            j -= 1
```

```
from codex.sort.basic import insertion_sort

items = [3, 1, 4, 1, 5, 9, 2, 6]
insertion_sort(items)
print(items)   # [1, 1, 2, 3, 4, 5, 6, 9]

# On already-sorted input, insertion sort does no swaps - the inner while loop never enters
sorted_items = [1, 2, 3, 4, 5]
insertion_sort(sorted_items)
print(sorted_items)  # [1, 2, 3, 4, 5] - unchanged
```

```
[1, 1, 2, 3, 4, 5, 6, 9]
[1, 2, 3, 4, 5]
```

Every swap in insertion sort is an adjacent swap. And an adjacent swap fixes *exactly one* inversion — the pair you just unswapped. (Two non-adjacent elements that were inverted? Still inverted after, since you didn't touch their pair.)

So the total number of swaps that insertion sort performs is exactly the number of inversions in the original array.

```
def insertion_sort_counting(items):
    items = list(items)
    swaps = 0
    for i in range(1, len(items)):
        j = i
        while j > 0 and items[j] < items[j - 1]:
            items[j], items[j - 1] = items[j - 1], items[j]
            j -= 1
            swaps += 1
    return items, swaps


items = [3, 1, 4, 1, 5, 9, 2, 6]
inversions_before = count_inversions(items)
sorted_items, swaps = insertion_sort_counting(items)
print(f"inversions in input: {inversions_before}")   # 8
print(f"swaps performed:     {swaps}")                # 8
print(f"match:               {inversions_before == swaps}")  # True
```

```
inversions in input: 8
swaps performed:     8
match:               True
```

That's not a coincidence and it's not specific to this input. Insertion sort's swap count is exactly the inversion count of its input; the running time tracks it within a constant factor. On already-sorted input (zero inversions), the inner `while` never enters and insertion sort runs in $O(n)$ — it's the best of the three on that case. On reverse-sorted input ($n(n-1)/2$ inversions), it does the maximum work. Insertion sort's cost is the input's distance from sortedness, measured in inversions, made literal.

3.4 Bubble sort: the cleanest inversion story

Same discipline as insertion sort, different traversal order. Walk the array left to right; whenever two adjacent elements are out of order, swap them. After one pass, the largest element has *bubbled* to the end. Then walk again over the remaining unsorted prefix. After k passes, the k largest elements are at the end, in their final positions.

```
def bubble_sort[T](
    items: MutableSequence[T], order: Ordering[T] = default_order
) -> None:
    n = len(items)
    for i in range(n - 1):
        for j in range(n - 1 - i):
            if order(items[j], items[j + 1]) > 0:
                items[j], items[j + 1] = items[j + 1], items[j]
```

```
from codex.sort.basic import bubble_sort

items = [3, 1, 4, 1, 5, 9, 2, 6]
bubble_sort(items)
print(items)   # [1, 1, 2, 3, 4, 5, 6, 9]
```

```
[1, 1, 2, 3, 4, 5, 6, 9]
```

Bubble sort, like insertion sort, swaps only adjacent elements. So the same lemma applies: total swaps = initial inversions.

```
def bubble_sort_counting(items):
    items = list(items)
    swaps = 0
    n = len(items)
    for i in range(n - 1):
        for j in range(n - 1 - i):
            if items[j] > items[j + 1]:
                items[j], items[j + 1] = items[j + 1], items[j]
                swaps += 1
    return items, swaps


items = [3, 1, 4, 1, 5, 9, 2, 6]
_, swaps = bubble_sort_counting(items)
print(f"inversions in input: {count_inversions(items)}")  # 8
print(f"bubble sort swaps:   {swaps}")                     # 8
```

```
inversions in input: 8
bubble sort swaps:   8
```

Same number. That's the beautiful invariant: **insertion sort and bubble sort do exactly the same number of swaps on any input**, because both are restricted to adjacent swaps, and each adjacent swap fixes exactly one inversion. The difference between them is only *where in the array each swap happens* — insertion sort does its work locally as it scans; bubble sort does its work globally each pass. The total amount of work, measured in swaps, is identical.

That's a deeper fact than it looks. The choice between insertion and bubble sort isn't about efficiency. It's about *traversal order*. They're solving the same problem, paying the same price, in the same currency.

3.5 The three questions, applied

Is it correct?

All three sorts have clean loop invariants.

- **Selection sort.** At the start of iteration i, the prefix `items[0:i]` holds the i smallest elements in sorted order. The iteration finds the minimum of `items[i:]` and places it at position i, extending the sorted prefix by one. After n iterations the entire array is the sorted prefix.
- **Insertion sort.** At the start of iteration i, the prefix `items[0:i]` is sorted (but not necessarily its final values — it may shift further as later elements arrive). The iteration inserts `items[i]` into its right place within that prefix, so `items[0:i+1]` is sorted at the end.
- **Bubble sort.** After pass k, the suffix `items[n-k:n]` holds the k largest elements in sorted order, in their final positions. The next pass walks only over the unsorted prefix and bubbles its largest element to the boundary.

In each case the invariant holds at the start, is preserved by each iteration, and at termination implies the whole array is sorted.

How efficient is it?

All three are $O(n^2)$ time and $O(1)$ extra space — the algorithms sort in place, modifying the input directly.

The detail is interesting though, and it splits the algorithms into two camps.

- **Selection sort** does *exactly* $n(n-1)/2$ comparisons on every input. The structure of the input — sorted, reverse-sorted, random — doesn't affect the comparison count. It always does n passes, each doing $O(n)$ work to find the minimum. Best case = worst case = average case.
- **Insertion sort and bubble sort** do a number of swaps equal to the inversion count of the input. On sorted input that's zero swaps, giving $O(n)$ time (you still need the outer loop to verify, and the comparisons happen). On reverse-sorted input it's $n(n-1)/2$ swaps, giving $O(n^2)$. On random input it's about $n^2/4$ swaps, still $O(n^2)$.

Said another way: selection sort is *constant work, regardless of luck*. Insertion and bubble are *as much work as the input deserves* — which, for a random input, is the same amount on average, but they win big on inputs that are nearly sorted.

Is it optimal?

This is the chapter's punch, and it splits into two cleanly different claims.

Claim 1. Any comparison-based sorting algorithm that uses **only adjacent swaps** must do at least as many swaps as there are inversions in the input. In the worst case (reverse-sorted input), that's $n(n-1)/2$. So insertion sort and bubble sort are tight at $\Omega(n^2)$ swaps in the worst case — there's no clever rearrangement within the adjacent-swap discipline that gets you below the inversion count.

The argument is one sentence: an adjacent swap changes the inversion count by exactly ± 1. To get from any starting inversion count k to zero, you need at least k adjacent swaps. The worst-case k is $n(n-1)/2$. Done.

Claim 2. Selection sort isn't restricted to adjacent swaps — it makes long swaps — so the inversions argument doesn't apply to it. Its $O(n^2)$ comes from a different structural fact: to identify the minimum of a sub-array of size m, you need $m-1$ comparisons (no comparison-based algorithm can do better; same adversary argument from chapter 1). Selection sort does this n times for sub-arrays of size $n, n-1, n-2, \ldots, 1$. Total: $\Theta(n^2)$ comparisons, unavoidably.

Both claims land in the same place — $\Omega(n^2)$ — but for different reasons. Insertion and bubble sort are stuck because they only fix one inversion per operation. Selection sort is stuck because it can't shortcut the extremum-finding work. The next chapter breaks both restrictions at once.

3.6 Distance from sortedness

Selection, insertion, and bubble sort are all correct — three slightly different loop invariants for what's morally the same shape. All three run in $O(n^2)$ time and $O(1)$ space. And all three are optimal *within their respective restrictions*: insertion and bubble are stuck at $\Omega(n^2)$ because they only swap adjacent elements, selection is stuck because it can't beat the extremum-finding lower bound — but those restrictions are exactly what an algorithm can give up to do better.

The inversion count measures distance from sortedness. An unsorted array is *some specific number of pair-swaps away from sorted*, and that number bounds how much work any local algorithm has to do. The reason quadratic sorts are quadratic is that they fix one inversion at a time, and there can be a quadratic number of inversions.

The way out is to **fix many inversions per operation**. Merge sort does it by combining two sorted halves and resolving all the cross-inversions in a single linear pass. Quicksort does it by partitioning around a pivot, which places one element in its final position and resolves all the inversions involving that pivot in one swap. Both algorithms break the one-inversion-per-swap ceiling, and both pay for it with a $\log n$ factor instead of an n factor.

3.7 Notes and further reading

The inversion count as a structural property of permutations is treated thoroughly in Knuth's *The Art of Computer Programming* Vol. 3 §5.1.1, where it appears under the name "inversions of a permutation." Selection, insertion, and bubble sort are CLRS (4th ed.) §2.1 and §2.2 and Sedgewick & Wayne's *Algorithms* (4th ed.) §2.1. The equivalence between adjacent-swap counts and inversion counts is folklore but worth attributing — it's the structural fact that makes insertion sort the right choice for *nearly-sorted* input, which is why it's used as the base case inside most production hybrid sorts (Timsort, Introsort). Counting inversions in $O(n \log n)$ via merge sort is a standard exercise you'll meet in the next chapter.

4 Fixing inversions, many at a time

Two algorithms, both $O(n \log n)$, both solving the same problem from opposite ends. Merge sort sorts each half recursively *first*, then weaves the two sorted halves together. Quick sort partitions the array around a pivot *first*, then sorts each side recursively. These two strategies aren't independent inventions. They're **duals** — two ways of attacking the same structural problem in opposite orders.

The lens that makes the duality visible is the distinction between **local** and **global** inversions. Once you have the lens, the two algorithms stop looking like rival inventions and start looking like the two natural strategies — fix the locals first, or fix the globals first — for any divide-and-conquer sort.

Each merge step and each partition step fixes a number of inversions proportional to how much work the step did. That linearity is the swap from one inversion per operation to as many inversions as the operation can see. The first cost you $O(n^2)$ in the previous chapter; the second buys you $O(n \log n)$ here. The factor of $\log n$ instead of n between the two regimes is what divide-and-conquer buys you.

4.1 Two kinds of inversions, two orders of attack

Take any array and split it in half at some midpoint m. Every inversion in the array — every pair (i, j) with $i < j$ and items$[i] >$ items$[j]$ — falls into exactly one of three buckets:

- **Local to the left half**: both i and j are in `items[0:m]`. Both elements live on the same side of the split.
- **Local to the right half**: both i and j are in `items[m:n]`. Same idea on the other side.
- **Global** (or **cross**): i is in the left half, j is in the right. The pair straddles the split.

Add the three counts up and you get the total inversion count of the array. To sort the array, you have to drive all three buckets to zero.

Here's the fact: **any divide-and-conquer sort that splits the array into two halves has to fix both kinds of inversion — local and global — and the algorithmic choice is which to fix first.**

- **Locals first.** Recursively sort each half, which fixes every local inversion (both halves end up sorted, with zero internal inversions). What's left is only cross-inversions. Then handle those in a linear sweep that combines the two sorted halves. This is **merge sort**.
- **Globals first.** Rearrange the array so every element of the left half is smaller than every element of the right half. That fixes every cross-inversion in one shot (there are now zero pairs straddling the split that are out of order). What's left is only local inversions within each side, which the recursive calls handle. This is **quick sort**.

The two algorithms aren't rival inventions. They're the two natural orders in which divide-and-conquer can resolve the inversion budget. Either order pays the same price: $O(n)$ work per level of recursion, $O(\log n)$ levels, total $O(n \log n)$. The work-per-level is linear because each merge or partition step is linear; the depth is logarithmic because each split halves the input. The product is the chapter.

4.2 Merge sort: locals first, then weave

Start with the locals. Recursively sort the left half and the right half. After the two recursive calls return, both halves are sorted, which means **every local inversion has been resolved**. The only inversions that can still exist are cross-inversions — pairs that straddle the midpoint.

Now the conquer step. Two sorted sequences can be combined into one sorted sequence in linear time: keep one finger on the front of each, repeatedly compare the two fingers, pick the smaller one, and advance the finger that won. After at most n comparisons you've consumed both lists.

```
from typing import MutableSequence, Sequence
from codex.types import Ordering, default_order


def merge_sort[T](
    items: MutableSequence[T], order: Ordering[T] = default_order
) -> None:
    if len(items) <= 1:
        return
    mid = len(items) // 2
    left = list(items[:mid])
    right = list(items[mid:])
    merge_sort(left, order)
    merge_sort(right, order)
    merge(items, left, right, order)
```

The recursion splits at the midpoint, allocates two new lists for the halves (because the recursive calls would otherwise overwrite each other's work), sorts each half, and then merges them back into the original storage. The `merge` helper is where the cross-inversions die.

```
def merge[T](
    items: MutableSequence[T],
    left: Sequence[T],
    right: Sequence[T],
    order: Ordering[T],
) -> None:
    i = j = k = 0
    while i < len(left) and j < len(right):
        if order(left[i], right[j]) <= 0:
            items[k] = left[i]
            i += 1
        else:
            items[k] = right[j]
            j += 1
        k += 1
    while i < len(left):
        items[k] = left[i]
        i += 1
        k += 1
    while j < len(right):
        items[k] = right[j]
        j += 1
        k += 1
```

Three loops. The first walks both halves in parallel, picking the smaller front element each time. The second and third handle whatever's left over once one side runs out. Each iteration of the first loop costs one comparison and writes one element; the cleanup loops are pure copies. Total work: linear in the combined length.

```
from codex.sort.merge import merge_sort

items = [3, 1, 4, 1, 5, 9, 2, 6]
merge_sort(items)
print(items)   # [1, 1, 2, 3, 4, 5, 6, 9]

# Merge sort is stable on equal keys - the <= in the comparison picks left first when tied
items = [(1, 'a'), (2, 'b'), (1, 'c'), (2, 'd')]
merge_sort(items, lambda x, y: -1 if x[0] < y[0] else (1 if x[0] > y[0] else 0))
print(items)   # [(1, 'a'), (1, 'c'), (2, 'b'), (2, 'd')] - relative order of equals preserved
```

```
[1, 1, 2, 3, 4, 5, 6, 9]
```

```
[(1, 'a'), (1, 'c'), (2, 'b'), (2, 'd')]
```

Now the inversion story. Look at the `else` branch in `merge` — the branch that picks from the right side. When `right[j]` gets taken before `left[i]`, that means `right[j] < left[i]`. But `left` is already sorted (the recursive call took care of that), so `left[i] <= left[i+1] <= ... <= left[len(left)-1]`. Which means `right[j]` is smaller than *every remaining element in the left half*.

That's not one cross-inversion fixed. That's $\text{len(left)} - i$ cross-inversions fixed, in a single operation. The merge step processes every cross-inversion the moment it picks from the right side, and there are at most $O(n)$ such picks for an array of size n. The whole merge is linear, and it resolves *every cross-inversion* in one pass.

That's the locals-first strategy at work. The recursive calls eliminated all the local inversions before the merge ran. The merge takes a sorted-left + sorted-right input — an array whose only remaining inversions are cross — and kills them in $O(n)$. Linear work, all the remaining inversions fixed. That's the source of the $\log n$ improvement over basic sorting.

4.3 Counting inversions, for free, in $O(n \log n)$

The brute-force `count_inversions` from chapter 3 walked every pair in $O(n^2)$ — fine as a teaching aid, useless at scale. Merge sort gives you a much better algorithm almost free.

The insight is the one from the previous section. Every time `merge` picks from the right side, it has fixed $\text{len(left)} - i$ cross-inversions. If you instrument the merge to *count* those fixes instead of just executing them, you've counted every global inversion in the array. Recurse on the halves to count their internal inversions (locals) the same way, and you've counted the total: local-left + local-right + global.

```python
from typing import MutableSequence, Sequence
from codex.types import Ordering, default_order


def count_inversions_fast[T](
    items: Sequence[T], order: Ordering[T] = default_order
) -> int:
    """Count inversions in O(n log n) via merge sort.

    Does not modify items; sorts a working copy internally.
    """
    work = list(items)
    return _merge_sort_counting(work, order)


def _merge_sort_counting[T](
    items: MutableSequence[T], order: Ordering[T]
) -> int:
    if len(items) <= 1:
        return 0
    mid = len(items) // 2
    left = list(items[:mid])
    right = list(items[mid:])
    count = _merge_sort_counting(left, order)
    count += _merge_sort_counting(right, order)
    count += _merge_counting(items, left, right, order)
    return count


def _merge_counting[T](
    items: MutableSequence[T],
    left: Sequence[T],
    right: Sequence[T],
    order: Ordering[T],
```

```
) -> int:
    count = 0
    i = j = k = 0
    while i < len(left) and j < len(right):
        if order(left[i], right[j]) <= 0:
            items[k] = left[i]
            i += 1
        else:
            items[k] = right[j]
            j += 1
            count += len(left) - i   # cross-inversions fixed by this pick
        k += 1
    while i < len(left):
        items[k] = left[i]
        i += 1
        k += 1
    while j < len(right):
        items[k] = right[j]
        j += 1
        k += 1
    return count
```

The structure is identical to merge sort. The only addition is the line `count += len(left) - i` inside the `else` branch — the one line that records *"this right-pick just resolved this many cross-inversions."* The recursive calls handle the local inversions inside each half by counting their own cross-inversions one level deeper. At every level of the recursion, you account for the globals of *that* level; sum across levels and you have the total.

```
from codex.sort.inversions import count_inversions, count_inversions_fast

items = [3, 1, 4, 1, 5, 9, 2, 6]
print(count_inversions(items))        # 11 - brute force, O(n²)
print(count_inversions_fast(items))   # 11 - same answer, O(n log n)

# For 10,000 random items, the brute force version is slow enough to feel
# the difference; the fast one is essentially instantaneous
import random
big = [random.randint(0, 10000) for _ in range(10000)]
print(count_inversions_fast(big))     # ~25 million inversions on a random input
```

```
8
8
25020833
```

The same algorithmic shape, the same single comparison per merge step — and you got an extra answer for free. This is a recurring pattern in algorithm design: when a procedure already has a useful structural property (here, *"every right-pick resolves a known number of cross-inversions"*), small modifications can extract additional answers without changing the asymptotic cost.

4.4 Quick sort: globals first, then sort each side

Same goal as merge sort, opposite order of attack. Instead of fixing the local inversions first via recursion and then weaving the halves together, quick sort fixes the global inversions first by rearranging the array, then handles the locals via recursion.

The rearrangement is the **partition** step. Pick a pivot element (I'll use the last element of the interval), then walk through the rest and swap things so everything smaller than the pivot lands to the pivot's left, everything not smaller lands to its right.

```
from typing import MutableSequence
from codex.types import Ordering, default_order


def partition[T](
    items: MutableSequence[T], lo: int, hi: int, order: Ordering[T]
) -> int:
    """
    Lomuto partition over items[lo:hi]. Treats items[hi - 1] as the pivot.
    Rearranges in place so items smaller than the pivot end up left of it,
    items not smaller end up right. Returns the pivot's final index.
    """
    pivot = items[hi - 1]
    i = lo
    for j in range(lo, hi - 1):
        if order(items[j], pivot) < 0:
            items[i], items[j] = items[j], items[i]
            i += 1
    items[i], items[hi - 1] = items[hi - 1], items[i]
    return i
```

Two indices do the work. `j` walks through the interval looking for elements smaller than the pivot. `i` marks the right edge of the "smaller" region — every element to the left of `i` is already known to be smaller than the pivot. Whenever `j` finds another smaller element, swap it into position `i` and advance `i`. After the scan finishes, swap the pivot itself into position `i`, where it belongs.

Here's the global-inversions story. After `partition` returns the pivot's final index p, every element in `items[lo:p]` is strictly less than `items[p]`, and every element in `items[p+1:hi]` is at least `items[p]`. Which means: **every cross-inversion between the future left half (items below p) and the future right half (items at p or above) is gone.** No pair of elements that straddles p is out of order. The global inversion count, relative to the split at p, is now zero.

That's a lot of inversions cleaned up by one linear pass. The partition did $O(n)$ comparisons and a small number of swaps; in exchange, it fixed every cross-inversion involving the pivot's "side." What remains is the inversions inside each side — the locals — and the recursive calls handle those.

```
def _quick_sort[T](
    items: MutableSequence[T], lo: int, hi: int, order: Ordering[T]
) -> None:
    if hi - lo < 2:
        return
    pivot_index = partition(items, lo, hi, order)
    _quick_sort(items, lo, pivot_index, order)
    _quick_sort(items, pivot_index + 1, hi, order)


def quick_sort[T](
    items: MutableSequence[T], order: Ordering[T] = default_order
) -> None:
    _quick_sort(items, 0, len(items), order)
```

The public `quick_sort` is just a thin wrapper that initializes the recursion bounds. The `_quick_sort` helper takes explicit `lo` and `hi` so the recursion can work in place on sub-ranges without copying. The structure perfectly mirrors merge sort, but read it in reverse: partition (the global-killing step) comes before the recursive calls (the local-killing steps), not after.

```
from codex.sort.quick import quick_sort, partition

items = [3, 1, 4, 1, 5, 9, 2, 6]
quick_sort(items)
```

```
print(items)   # [1, 1, 2, 3, 4, 5, 6, 9]

# partition is exposed because the next chapter reuses it for quickselect
items = [3, 1, 4, 1, 5, 9, 2, 6]
p = partition(items, 0, len(items), lambda a, b: -1 if a < b else (1 if a > b else 0))
print(f"after partition: {items}, pivot ended at index {p}, value {items[p]}")
# Every element before p is < items[p]; every element after is >= items[p].
# Zero cross-inversions between the two sides - the globals are gone in one pass.
```

```
[1, 1, 2, 3, 4, 5, 6, 9]
after partition: [3, 1, 4, 1, 5, 2, 6, 9], pivot ended at index 6, value 6
```

One thing to flag, because it'll matter in the analysis. The version of quick sort above always picks the **last element** of the interval as the pivot. That's the simplest possible pivot choice, and on random inputs it works beautifully. But on certain pathological inputs — already-sorted, reverse-sorted, or all-equal — it's catastrophically bad. Picking the last element of a sorted interval is the worst possible choice: the partition splits the array into one piece of size $n-1$ and one piece of size zero, the recursion becomes effectively linear, and quick sort degenerates to $O(n^2)$.

The standard fix is **random pivot selection** — pick a pivot uniformly at random from the interval, then swap it into the last position before partitioning. This makes worst-case behavior require a worst-case input *and* a worst-case sequence of random choices, which together happen with vanishing probability. The next chapter (on selection) shows the randomized version in full; the version here is the clearest exposition of the partition idea, and I'm keeping it that way on purpose.

4.5 The three questions, applied

Is it correct?

Both algorithms have clean recursive correctness arguments built on the local/global decomposition. For **merge sort**, the inductive case is: assume the recursive calls produce sorted halves (zero local inversions in each), then `merge` correctly weaves them into a sorted whole because it always picks the smaller of the two fronts — so any remaining cross-inversion would have to put the larger element first, which the algorithm refuses to do.

For **quick sort**, the invariant of `partition` does the work: after partition returns index p, every element in `items[lo:p]` is smaller than `items[p]`, and every element in `items[p+1:hi]` is not smaller. So `items[p]` is in its final sorted position, and the two recursive calls handle the two sides — which can only contain local inversions — independently.

In both cases, the base case (a sequence of length zero or one) has no inversions of any kind.

How efficient is it?

Merge sort is $O(n \log n)$ time and $O(n)$ extra space, on every input. The recursion has depth $\log_2 n$ (each level halves the interval), and each level does $O(n)$ total work across all the merges at that level. Combine and you get $n \log n$. The space cost comes from the auxiliary `left` and `right` lists allocated in each recursive call — at any moment up to $O(n)$ of extra storage is live.

Quick sort is $O(n \log n)$ time *on average* and $O(\log n)$ extra space *on average* (just the recursion stack). The recursion depth depends on the pivot quality: a balanced pivot (close to the median) gives $\log_2 n$ depth, an unbalanced pivot gives more. On a uniformly random input with last-element pivot, the average depth is $O(\log n)$ and the average total work is $O(n \log n)$. On the pathological inputs flagged above, the depth becomes $O(n)$ and the total work becomes $O(n^2)$. **Random pivot selection makes the bad case a probabilistic curiosity rather than a structural risk.**

On the average case the two tie. They diverge on stability (merge sort preserves the relative order of equal keys; quick sort does not, at least in this implementation) and on memory (merge sort needs $O(n)$ scratch space; quick sort gets by with $O(\log n)$ stack frames).

Is it optimal?

This is the chapter's punch, and it's a big one. **The comparison-based sorting lower bound is $\Omega(n \log n)$.** No algorithm that learns about its input only by comparing pairs of elements can sort an arbitrary input in fewer than $\Omega(n \log n)$ comparisons in the worst case. Merge sort meets this bound exactly; quick sort meets it on average. Within the comparison model, you cannot do asymptotically better than what this chapter just showed you.

The argument is the **decision-tree lower bound** for sorting, the same shape that gave the $\Omega(\log n)$ bound for searching in the previous chapter, scaled up. Any comparison-based sort can be drawn as a binary decision tree: each internal node is a comparison ("is `items[i] < items[j]`?"), each leaf is one specific output permutation. To handle every possible input correctly, the tree needs at least $n!$ leaves — one per possible input ordering. A binary tree with L leaves has depth at least $\log_2 L$, so the depth is at least $\log_2(n!)$. By Stirling's approximation — the fact that $n!$ grows like $(n/e)^n$, so $\log_2(n!) \approx n \log_2 n$ — that depth is $\Theta(n \log n)$. The worst-case path through any such tree is $\Omega(n \log n)$ comparisons.

That's a strong statement. It says: comparison-based sorting can never be linear, no matter how clever the algorithm. The factor of $\log n$ over linear is **the price you pay for not knowing anything about the input except how its elements compare**. The next two chapters explore both sides of that price: selection (chapter 5) shows you can get partial sorted information faster than full sorting, and linear-time sorting (chapter 6) shows you can break the lower bound entirely if you give up the comparison-only restriction.

4.6 Two orders, one cost

Merge sort and quick sort are correct via clean recursive arguments — sorted halves combine into a sorted whole; partition places the pivot in its final position. Both run in $O(n \log n)$, merge sort always and quick sort on average. And both are optimal: the decision-tree lower bound says no comparison-based sort can do better than $\Omega(n \log n)$ in the worst case, and these algorithms meet that bound.

The local-vs-global duality is the move that generalizes. Any divide-and-conquer sort that splits its input into two halves has to fix two kinds of inversions — local ones inside each half, global ones straddling the split — and the algorithmic choice is which order to fix them in. Merge sort fixes locals first via recursion, then kills the globals in one linear merge. Quick sort kills the globals first via partition, then fixes the locals via recursion. Either order works. Both pay the same $\log n$ factor for recursion depth, and both benefit from the same many-at-once trick at each level. The two algorithms are reflections of each other across the local/global divide.

The same inversion-counting argument from chapter 3 generalizes. Basic sorts fix one inversion per swap, so they're stuck at $O(n^2)$ because there can be that many inversions. Efficient sorts fix inversions in *blocks proportional to the work being done at each level* — a merge step does $O(n)$ work and resolves $O(n)$ cross-inversions; a partition step does $O(n)$ work and resolves all the pivot's cross-inversions in one swap. The trick that turns n^2 into $n \log n$ is exactly the trick of being credited for *all* the inversions you can see at this level, instead of just the one pair you happened to swap.

In the next chapter, the question shifts: what if you don't need the whole array sorted, but only the k-th smallest element? Quick sort already places one element per partition step in its final position. Selection turns out to require only one recursive call per level — not two — and the cost collapses from $O(n \log n)$ to $O(n)$.

4.7 Notes and further reading

Merge sort goes back to John von Neumann (1945) and is treated in CLRS (4th ed.) §2.3 (introduction) and §4.4–4.5 (recurrence analysis). Quick sort was Tony Hoare's 1959 invention and is in CLRS §7; Sedgewick's *Algorithms* (4th ed.) §2.3 gives the cleanest production-style treatment with three-way partitioning and pivot-selection variants. The decision-tree lower bound is CLRS §8.1 — Stirling's approximation is the punch that turns $\log_2(n!)$ into $n \log n$. The $O(n \log n)$ inversion-counting variant of merge sort is a folklore observation that shows up as an exercise in nearly every algorithms textbook; the cleanest derivation I know is in Kleinberg & Tardos's *Algorithm Design* §5.3. Hoare's original Quicksort paper (1962) is short, lucid, and worth reading as a piece of historical algorithm prose. The local-vs-global inversion framing isn't standard textbook vocabulary, but it's the cleanest organizing lens I know for seeing merge sort and quick sort as duals — both are present implicitly in any thorough treatment of divide-and-conquer sorting.

5 Selection without sorting

You don't need to sort an array to find its median. You don't need to sort an array to find any rank — the smallest element, the largest, the 25th percentile, the 99th. Two algorithms find the k-th smallest element of an array in **linear time**, beating the $O(n \log n)$ cost of sorting by a full factor of $\log n$.

The local/global lens from the previous chapter still applies, with one twist. Quick sort, after partitioning around a pivot p, has to recurse on *both* sides — both halves need their local inversions fixed. Selection only has to recurse on *one* side: the side that contains the rank you're looking for. The other side stays unsorted forever and you don't care, because the answer you want isn't in it. Throwing away one of the two recursive calls collapses $O(n \log n)$ down to $O(n)$.

The first algorithm — quickselect — does this with a randomly chosen pivot and gets you to $O(n)$ on average, with a small probability of degenerating. The second — median of medians, also called BFPRT after its five inventors — chooses its pivot so carefully that it guarantees $O(n)$ even in the worst case. The contrast between them is the cleanest example I know of a recurring trade-off in algorithm design: randomization buys you simplicity; determinism buys you guarantees, at the cost of intricacy.

5.1 Quickselect: only one side matters

Start with the partition step from the previous chapter. After `partition(items, lo, hi, order)` returns the pivot's final index p:

- Every element in `items[lo:p]` is smaller than `items[p]`.
- Every element in `items[p+1:hi]` is not smaller.
- `items[p]` is in its final sorted position.

The third fact is the one that matters here. If you're looking for the k-th smallest element of `items[lo:hi]`, and partition tells you the pivot landed at position p, you have three cases:

- $k = p$: the pivot *is* the answer. Return `items[p]` and you're done.
- $k < p$: the answer is somewhere in the left side, `items[lo:p]`. Recurse there.
- $k > p$: the answer is somewhere in the right side, `items[p+1:hi]`. Recurse there.

In two of the three cases, you recurse on only one side and ignore the other. That's the saving over quick sort, which always recurses on both.

I'll use a randomly chosen pivot — pick a uniformly random element from the interval and swap it into the last position before partitioning. This is the standard randomization trick for quickselect, and it's what makes the average-case analysis work out cleanly regardless of input distribution.

```
import random
from typing import MutableSequence
from codex.types import Ordering, default_order
from codex.sort.quick import partition


def quickselect[T](
    items: MutableSequence[T], k: int, order: Ordering[T] = default_order
) -> T:
    """Return the k-th smallest element of items (0-indexed).

    Modifies items in place as a side effect of partitioning. If you don't
    want that, pass list(items) instead of items.
    """
    if not 0 <= k < len(items):
```

```
        raise IndexError(f"k={k} out of range for sequence of length {len(items)}")
    return _quickselect(items, 0, len(items), k, order)
```

The public wrapper does the bounds check and kicks off the recursion. The recursion itself takes explicit `lo` and `hi` so it can work in place on shrinking sub-ranges without copying.

```
def _quickselect[T](
    items: MutableSequence[T], lo: int, hi: int, k: int, order: Ordering[T]
) -> T:
    if hi - lo == 1:
        return items[lo]
    # Random pivot: swap a random element into the last position.
    pivot_idx = random.randint(lo, hi - 1)
    items[pivot_idx], items[hi - 1] = items[hi - 1], items[pivot_idx]
    p = partition(items, lo, hi, order)
    if k == p:
        return items[p]
    elif k < p:
        return _quickselect(items, lo, p, k, order)
    else:
        return _quickselect(items, p + 1, hi, k, order)
```

The body is recognizably quick sort, with two recursive calls collapsed to one based on which side of the pivot the target rank k falls on. The base case is a single element: if the interval has length 1, that element *is* the k-th smallest, by definition.

```
import random
random.seed(42)  # make the output reproducible

from codex.select.quick import quickselect

items = [3, 1, 4, 1, 5, 9, 2, 6]
print(quickselect(list(items), 0))   # 1 - smallest element
print(quickselect(list(items), 3))   # 3 - 4th smallest (k=3 is 0-indexed)
print(quickselect(list(items), 7))   # 9 - largest element
print(quickselect(list(items), len(items) // 2))  # 4 - median (lower of two for even n)
```

```
1
3
9
4
```

Now the cost analysis. With a random pivot, the expected pivot position is the middle of the interval. So on average each recursive call halves the interval size: $T(n) = T(n/2) + O(n)$. The $O(n)$ is the partition cost; the recursion does the same work on half the data.

Unroll the recurrence: $T(n) = O(n) + O(n/2) + O(n/4) + \cdots = O(n) \cdot (1 + 1/2 + 1/4 + \cdots) = O(2n) = O(n)$.

The geometric series is the magic. The first partition costs n; the second costs $n/2$; the third $n/4$; and so on. Sum the infinite geometric series and you get $2n$, which is still linear. **The recursion is logarithmic in depth but the total work is linear**, because the work at each level halves. The quick sort recurrence $T(n) = 2T(n/2) + O(n)$ doesn't collapse this way because the two recursive calls together do as much work as the parent, not half. Killing one of those calls is exactly the difference between $\log n$ levels of n work ($= n \log n$) and $\log n$ levels of geometrically-shrinking work ($= n$).

That's the expected case. The worst case is unfortunately not so kind. If you keep picking the smallest element as pivot, the partition is unbalanced — one side has $n - 1$ elements, the other has zero. The recursion becomes $T(n) = T(n-1) + O(n)$, which is $O(n^2)$. With a random pivot, this catastrophe requires an adversary to predict every random choice, which they can't, so the expected behavior dominates with overwhelming probability. But the worst case isn't $O(n)$; it's $O(n^2)$.

To get a *guaranteed* $O(n)$ worst case, you need to pick pivots in a way that can't be foiled by any input. That's median of medians.

5.2 The pivot you can trust: median of medians

The problem with quickselect's worst case is the pivot. A "bad" pivot — one near the smallest or largest element — leaves an almost-everything-on-one-side partition, and the recursion does $\Theta(n)$ work on an interval barely smaller than the original. To get a guaranteed $O(n)$, you need a pivot that's *guaranteed* to lie somewhere in the middle of the array — say, between the 30th and 70th percentile — so that each recursive call sees an interval of at most $0.7n$ elements.

A median would be ideal, but finding a median is the problem you're trying to solve. That sounds circular, and it almost is. The trick — the elegant move of the BFPRT algorithm — is to find an *approximate* median that's *good enough* to be a pivot, using a recursive call on a *smaller* problem.

Here's the construction.

1. Group the array into chunks of 5.
2. Find the median of each chunk (constant work per chunk, since each has just 5 elements).
3. Collect the chunk-medians into a list of size $n/5$.
4. Recursively find the median of that list. Call it p.
5. Use p as the pivot for partition.

The pivot p is the *median of medians* — the middle element of the chunk-medians. It's not the true median of the original array, but it's close enough. The bound is:

At least 30% **of the elements are smaller than** p, and **at least** 30% **are larger.** So whichever side of the partition you have to recurse on, the recursive call sees at most 70% of the elements you started with.

Why 30%? There are $n/5$ chunks. Half of them — that is, $n/10$ chunks — have their median smaller than p (since p is the median of the chunk-medians). In each of those chunks, the chunk-median is the third-smallest of its five elements, which means *two more* elements in that chunk are also smaller than the chunk-median, and hence smaller than p. So each such chunk contributes 3 elements that are smaller than p: $n/10$ chunks $\times$ 3 elements $= 3n/10$, which is 30%. The symmetric argument gives 30% larger.

That's the geometric guarantee. With pivot p chosen this way, the recursion satisfies $T(n) = T(7n/10) + T(n/5) + O(n)$. The $T(n/5)$ is the recursive call to find the median of medians; the $T(7n/10)$ is the recursive selection on the larger of the two partition sides. Both fractions are constants, and their sum $7/10 + 1/5 = 9/10 < 1$, so the recurrence collapses to $O(n)$ — the same geometric trick as quickselect, just with the constant $9/10$ instead of $1/2$. The depth of the recursion is $O(\log n)$, but the work decays geometrically across levels and the total is linear.

The chunk size of 5 isn't arbitrary. Smaller doesn't work: with chunks of 3, the analogous calculation gives $T(n) = T(2n/3) + T(n/3) + O(n)$, whose fractions sum to 1, which fails to collapse and gives $O(n \log n)$. Chunks of 5 are the smallest odd size for which the sum of fractions is strictly less than 1. Larger chunks would also work but make the constant factor worse — 5 hits the sweet spot.

Now the code. I'll write the selection function out of place — it returns a value rather than rearranging the input — because the pedagogical clarity is worth the $O(n)$ extra space. The move that matters is the recurrence, not the in-place trick.

```
from typing import Sequence
from codex.types import Ordering, default_order
from codex.sort.basic import insertion_sort


def select[T](
    items: Sequence[T], k: int, order: Ordering[T] = default_order
) -> T:
    """Return the k-th smallest element of items in worst-case O(n) time."""
    if not 0 <= k < len(items):
        raise IndexError(f"k={k} out of range for sequence of length {len(items)}")
    return _select(list(items), k, order)
```

The public wrapper validates `k` and copies the input (so the recursion can freely mutate it without surprising the caller). The recursion does the real work:

```
def _select[T](items: list[T], k: int, order: Ordering[T]) -> T:
    if len(items) <= 5:
        # Base case: sort and pick. Insertion sort is cheap on tiny inputs.
        insertion_sort(items, order)
        return items[k]

    # Find a good pivot via median of medians.
    pivot = _median_of_medians(items, order)

    # Three-way partition by value. Equality goes into its own bucket so the
    # k-equals-many-pivots case has a clean home.
    lo = [x for x in items if order(x, pivot) < 0]
    eq = [x for x in items if order(x, pivot) == 0]
    hi = [x for x in items if order(x, pivot) > 0]

    if k < len(lo):
        return _select(lo, k, order)
    elif k < len(lo) + len(eq):
        return pivot  # k falls inside the run of equal pivots
    else:
        return _select(hi, k - len(lo) - len(eq), order)
```

Three-way partition by *value* (using list comprehensions) rather than the Lomuto in-place partition from chapter 4. The three buckets are `lo` (strictly smaller than the pivot), `eq` (equal to it), and `hi` (strictly larger). The recursion then knows exactly which bucket contains rank k and how to adjust k's value for the recursive call.

The pivot itself comes from `_median_of_medians`, which is the heart of the algorithm:

```
def _median_of_medians[T](items: list[T], order: Ordering[T]) -> T:
    medians = []
    for start in range(0, len(items), 5):
        group = list(items[start:start + 5])
        insertion_sort(group, order)
        medians.append(group[len(group) // 2])
    if len(medians) == 1:
        return medians[0]
    # Recurse to find the median of the medians.
    return _select(medians, len(medians) // 2, order)
```

Group the array into chunks of 5, sort each chunk with insertion sort (which is genuinely the right tool for a 5-element list — chapter 3's analysis of insertion sort on near-sorted small inputs shows up here as the right tool), collect the medians, and recursively select the median of that list. The recursion bottoms out when there are five or fewer chunk-medians, at which point the base case of `_select` sorts and returns.

```
from codex.select.deterministic import select

items = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5, 8, 9, 7, 9, 3, 2, 3, 8, 4]
print(select(items, 0))                  # 1 - smallest
print(select(items, len(items) - 1))     # 9 - largest
print(select(items, len(items) // 2))    # 5 - median

# Deterministic - no randomness, every call returns the same answer
print(select(items, 7))   # 4 - same answer every time
print(select(items, 7))   # 4
```

```
1
9
```

```
5
3
3
```

5.3 The three questions, applied

Is it correct?

Both algorithms are correct via a straightforward inductive argument. Assume the recursive call returns the correct answer on the smaller subproblem; then **quickselect** is correct because after partition, the pivot's final position p tells you exactly which side of the array contains rank k, and the recursion descends into that side with the rank unchanged (or adjusted, in the right-side case, to account for the elements being skipped). **Median of medians** is correct by the same argument applied to its three-way partition: rank k is either in `lo` (recurse, unchanged k), or in the equal bucket (the pivot is the answer), or in `hi` (recurse with k shifted by `len(lo) + len(eq)`). The base case (a single element or a list of ≤ 5) is trivially correct.

How efficient is it?

Quickselect is $O(n)$ expected time, with $O(n^2)$ worst case in the unlucky event of consistently bad pivots. Space is $O(\log n)$ on average — the recursion stack — because each recursive call works on a sub-interval of the same array.

In plain words for the expected case: each level of recursion does work proportional to the *current* interval size, and the expected interval size halves each level. Sum $n + n/2 + n/4 + ... = 2n$. Linear, with a constant factor of about 2. The "2" is the sum of the geometric series — it's why halving works and tripling doesn't.

Median of medians is $O(n)$ worst-case. The recurrence is $T(n) = T(n/5) + T(7n/10) + O(n)$, and since $1/5 + 7/10 = 9/10 < 1$, the geometric sum converges and gives $T(n) = O(n)$. The constant factor is much larger than quickselect's — roughly $10\times$, because the algorithm does the partition, *plus* the median-of-medians computation, *plus* the recursive selection. In practice, you use quickselect; median of medians exists for the worst-case question.

Space is $O(n)$ for the out-of-place version implemented here (the three-way partition creates new lists). An in-place version exists but is intricate enough that it would have obscured the recurrence at the centre of the algorithm. The asymptotic time complexity is the same either way.

Is it optimal?

Yes, and the argument is the cheapest in the chapter: **any algorithm that finds the k-th smallest element of an array must inspect every element**, because if it didn't, an adversary could place the answer at the position it missed. Same argument as the adversary argument for linear search in chapter 1. So $\Omega(n)$ is the lower bound; $O(n)$ is what both quickselect (on average) and median of medians (worst case) achieve. Tightly optimal in both senses.

5.4 Recurse on one side

Quickselect and median of medians are correct via clean recursive arguments — partition + recurse on the side containing k, base case is trivial. Quickselect runs in expected $O(n)$ with a small chance of $O(n^2)$; median of medians runs in worst-case $O(n)$ with a larger constant factor. Both are optimal — you cannot find the k-th smallest element of an array without inspecting every element.

When you can recurse on only one side instead of two, the cost collapses from $O(n \log n)$ to $O(n)$. The same partition operation — same $O(n)$ work, same local-vs-global inversion structure — gives you $O(n \log n)$ if you recurse on both sides (sorting) and $O(n)$ if you recurse on only one (selection). The difference between $T(n) = 2T(n/2) + O(n)$ and $T(n) = T(n/2) + O(n)$ is exactly the difference between log-linear and linear. The geometric sum that the second recurrence produces is the cleanest expression of *"do the work, then do strictly less of it, recursively"* — and it shows up again and again in the rest of the book, every time an algorithm gets to throw away most of its remaining work after a constant-fraction reduction.

Quickselect is short, readable, fast in practice; the price is a tiny probability of going quadratic. Median of medians is intricate, careful, and guaranteed; the price is a $10\times$ constant factor in real-world cases. Most production code uses quickselect (or some hybrid). Most algorithm courses teach median of medians, because the existence of a worst-case linear selection algorithm is one of the most surprising results in the field. They're both worth knowing.

In the next chapter, the question shifts again. So far every algorithm in this book has been *comparison-based* — the only thing it knows about its input is how to compare two elements. The decision-tree lower bound from chapter 4 said that comparison-based sorting can't be faster than $\Omega(n \log n)$. The next chapter shows you can sort in *linear* time if you're willing to give up the comparison-only restriction — by using the values themselves as positions in memory. It's the cleanest example in the book of how a model's constraint dictates its lower bound.

5.5 Notes and further reading

Quickselect is Tony Hoare's 1961 invention, published as a companion piece to quick sort. Median of medians is the **BFPRT algorithm**, named for its five inventors: Manuel Blum, Robert Floyd, Vaughan Pratt, Ron Rivest, and Robert Tarjan, who published the worst-case linear selection result in 1973 — a paper still worth reading for the elegance of the proof. CLRS (4th ed.) §9.2 covers quickselect; §9.3 covers median of medians. Sedgewick's *Algorithms* (4th ed.) §2.5 covers quickselect in production-friendly form. The expected-linear-time analysis of randomized quickselect via the partition-size recurrence is in CLRS §9.2; Motwani and Raghavan's *Randomized Algorithms* (1995) gives a more thorough probabilistic treatment. The choice of group size 5 in median-of-medians is famously the smallest odd group that makes the recursive fractions sum to less than 1; group size 3 fails, group size 7 works but with a worse constant.

6 Looking inside the elements

Every sorting algorithm in Part I has been working in the **comparison model** — the algorithm's only access to the data is to ask, of any two elements, *which one is smaller?* Bubble sort doesn't look at the bits of an integer; merge sort doesn't peek at the characters of a string; quickselect doesn't care whether you're sorting numbers or names. The elements have been opaque boxes that the algorithm can hold up next to each other and nothing more.

That assumption — opacity — is what locks comparison-based sorting at $\Omega(n \log n)$. The decision-tree lower bound from chapter 4 was a statement about the comparison model: with only $\log_2(n!)$ leaves reachable per level of the tree, you can't get to the right answer in fewer comparisons. But it wasn't a statement about *sorting itself*. It was a statement about *sorting under the constraint that all you can do is compare*.

Drop that constraint and the bound goes away.

Two algorithms sort in **linear time** — $O(n)$, beating the comparison lower bound by a factor of $\log n$ — and the price they pay is explicit. Both only work when you can look *inside* the elements, treating each one not as an opaque value to be compared but as a structured thing with intrinsic properties. A magnitude that maps directly to a position in memory. A sequence of digits, each in a bounded range. A fixed-width representation. The lower-bound argument doesn't apply because these algorithms aren't comparison-based; they're *structure-based*. Different model, different bound.

The value is the address. Counting sort takes an integer x and uses it as the index into a counting array — no comparison, just a direct memory lookup. Radix sort decomposes an integer into digits and sorts by each digit position in turn. In both cases the algorithm reads the element's intrinsic structure rather than its comparative position relative to other elements. That single reframing — *from comparing to looking inside* — is the move that breaks the $n \log n$ ceiling.

6.1 Counting sort: the value is the address

The simplest exploit. Suppose every element of `items` is a non-negative integer in the range $[0, k]$. Then you can sort the array by *counting how many of each value there are* and reading them back out in order.

There are three phases. **Histogram**: walk the input once and count occurrences of each value. **Prefix sum**: turn the histogram into a cumulative count, so `counts[i]` tells you how many elements are $\leq i$ — which is exactly the position-just-past-the-last-i in the sorted output. **Placement**: walk the input *in reverse*, and for each element, the decremented prefix sum gives its destination index.

```
from typing import Sequence


def counting_sort(items: Sequence[int], k: int) -> list[int]:
    """Sort a sequence of integers in [0, k] in O(n + k) time. Stable."""
    counts = [0] * (k + 1)
    for x in items:
        counts[x] += 1

    # Prefix sums: counts[i] becomes the number of elements <= i.
    for i in range(1, k + 1):
        counts[i] += counts[i - 1]

    # Place each element, walking the input in reverse to preserve stability.
    output = [0] * len(items)
    for x in reversed(items):
        counts[x] -= 1
```

```
        output[counts[x]] = x

    return output
```

The whole algorithm is three passes, and not one of them involves a single comparison between two input elements. The histogram pass increments `counts[x]` for each `x`; the prefix-sum pass turns counts into cumulative counts; the placement pass reads the cumulative counts as destination indices. Each pass is $O(n)$ (the histogram and placement) or $O(k)$ (the prefix sum). Total: $O(n + k)$.

```
from codex.sort.counting import counting_sort

print(counting_sort([3, 1, 4, 1, 5, 9, 2, 6], k=9))  # [1, 1, 2, 3, 4, 5, 6, 9]
print(counting_sort([0, 0, 0], k=0))                  # [0, 0, 0] - duplicates fine
print(counting_sort([], k=10))                        # [] - empty input fine
```

```
[1, 1, 2, 3, 4, 5, 6, 9]
[0, 0, 0]
[]
```

The reverse walk. The third loop walks `items` in reverse, not forward, and that's not a stylistic choice — it's what makes counting sort **stable**. Stability means equal elements preserve their relative order from input to output. With the reverse walk, the last copy of value v in the input gets placed at the rightmost slot reserved for v in the output, and earlier copies fill the slots to the left of that. Walking forward instead would *reverse* the relative order of equal elements. Stability is going to be essential to radix sort, so the reverse-walk detail isn't cosmetic.

The k in $O(n + k)$. If k is roughly the size of n — say, sorting integers in $[0, n]$ — then the algorithm is genuinely linear. If k is enormous — say, sorting 32-bit integers where $k = 2^{32}$ — then counting sort allocates a four-billion-entry array and pays $O(k)$ for the prefix-sum pass, which destroys any practical benefit. Counting sort wins decisively when the value range is small and collapses to uselessness when it isn't. The next algorithm fixes that.

6.2 Radix sort: one digit at a time

What if your integers are large but their *digits* are bounded? A 32-bit integer can take any of 2^{32} values, but each of its ten decimal digits is in $[0, 9]$. Each of its 32 binary digits is in $[0, 1]$. Each of its four bytes is in $[0, 255]$. Counting sort is useless on the integers themselves, but it's perfectly suited to sorting by a single digit.

Radix sort is the natural construction: sort by the lowest-order digit, then by the next-lowest, and so on up to the highest. The variant I'll show is **LSD** (least significant digit first), which is the one that's easiest to analyze. After all digit positions are processed, the array is sorted by the full value.

```
from typing import Sequence


def radix_sort(items: Sequence[int], base: int = 10) -> list[int]:
    """Sort non-negative integers in O(d * (n + base)) time.

    d = number of digits in the largest element when written in the given base.
    LSD (least significant digit first).
    """
    if not items:
        return []

    items = list(items)
    max_val = max(items)

    digit_place = 1  # ones, then base, then base^2, ...
```

```
    while max_val // digit_place > 0:
        items = _counting_sort_by_digit(items, digit_place, base)
        digit_place *= base

    return items
```

The driver loop figures out how many digits the largest element has and runs one stable counting sort per digit position. The per-digit pass is just counting sort applied to a digit-extracting key instead of the value itself:

```
def _counting_sort_by_digit(
    items: list[int], digit_place: int, base: int
) -> list[int]:
    """One stable pass: sort by (x // digit_place) % base."""
    counts = [0] * base
    for x in items:
        counts[(x // digit_place) % base] += 1

    for i in range(1, base):
        counts[i] += counts[i - 1]

    output = [0] * len(items)
    for x in reversed(items):
        d = (x // digit_place) % base
        counts[d] -= 1
        output[counts[d]] = x

    return output
```

The shape is identical to plain counting sort. The only change is the "key" the algorithm uses: instead of `x`, it uses `(x // digit_place) % base`, which extracts one specific digit out of `x`.

```
from codex.sort.radix import radix_sort

print(radix_sort([170, 45, 75, 90, 802, 24, 2, 66]))   # [2, 24, 45, 66, 75, 90, 170, 802]
print(radix_sort([1234, 56, 7, 89, 3210]))             # [7, 56, 89, 1234, 3210]
print(radix_sort([12, 11, 22, 21]))                    # [11, 12, 21, 22]

# Different base, same result - base only affects the constant factor
print(radix_sort([170, 45, 75, 90, 802, 24, 2, 66], base=2))
```

```
[2, 24, 45, 66, 75, 90, 170, 802]
[7, 56, 89, 1234, 3210]
[11, 12, 21, 22]
[2, 24, 45, 66, 75, 90, 170, 802]
```

Why does LSD radix sort work? It's not obvious that sorting by the *least* significant digit first should lead anywhere useful — the lowest digit is the least important position, intuitively. The answer is in the stability.

After the first pass, the array is sorted by ones digit. Elements with ones-digit 4 are clustered together, in some order. After the second pass, the array is sorted by tens digit *while preserving the relative order from the previous pass* — because counting sort is stable. So within the cluster of tens-digit-7 elements, the ones-digit order from the first pass is preserved. By induction, after j passes the array is sorted by the bottom j digits of each value. After all d passes, it's sorted by the full value.

Stability is what makes radix sort work at all. Without it, each pass would scramble the order established by the previous one, and the whole construction would collapse.

What's the practical cost? Each pass is $O(n + b)$, where b is the base. There are d passes, where d is the number of digits in the largest value (which is $\log_b(\text{max_val})$). Total: $O(d(n + b)) = O((n + b) \log_b(\text{max_val}))$. For fixed-width integers (say, 32-bit), d is bounded by a constant — at most 32 if you sort one bit at a time, at

most 4 if you sort one byte at a time. Pick base 256 and you get **4 passes for 32-bit integers, 8 passes for 64-bit**, each running in $O(n + 256) = O(n)$ — genuinely linear in practice. That's the choice production radix sort implementations make.

6.3 The three questions, applied

Is it correct?

Counting sort is correct because the prefix sums tell you, for each value v, exactly which positions in the output should be filled with copies of v. After phase 2, `counts[i]` equals the number of input elements with value $\leq i$ — which is the position-just-past-the-last-i. The reverse-walk placement then fills those positions from the right inward, one element at a time, and the decrement before placement ensures the next copy of v lands one slot earlier. Inductively, every input element ends up at its correct position in the sorted output.

Radix sort is correct by induction on the number of digits processed. After j passes (least significant first), the array is sorted by the values' last j digits — that is, by their value modulo base^j. The $(j+1)$-th pass is a stable sort by digit $j+1$, which extends the sortedness to the last $j+1$ digits without disturbing the lower-digit order from previous passes. After all d passes, the array is sorted by the full value.

The stability of the per-digit sort is the crux of the inductive argument: if any pass scrambled equal-digit elements, the inductive step would fail. Stability isn't an aesthetic property of counting sort — it's the property that lets radix sort exist.

How efficient is it?

Counting sort runs in $O(n + k)$ time and $O(n + k)$ space — the counts array is $O(k)$, the output array is $O(n)$. When k is at most a constant multiple of n, the algorithm is strictly linear. When k explodes (say, $k = 2^{32}$ for arbitrary integers), the algorithm collapses to uselessness because the counts array alone blows up memory.

Radix sort runs in $O(d(n + b))$ time and $O(n + b)$ space, where d is the number of digits in the maximum value and b is the base. For fixed-width integer types (the case you meet in real software), d is a constant. So the time is $O(n)$ and the space is $O(n + b)$.

Plainly stated: counting sort is "make a histogram, then read it out in order"; radix sort is "do counting sort on every digit, from least to most significant, leaning on stability so earlier passes survive." Neither algorithm performs a single comparison between two input elements. Both algorithms read the *intrinsic structure* of the elements — the values themselves, or the digits of the values — and use that structure to compute placements directly.

Is it optimal?

Yes, given the model, and the argument is the cheapest in the book: **the lower bound for sorting in any model is** $\Omega(n)$ — you have to look at every element at least once, or an adversary can change the unexamined element to make your output wrong (the same adversary argument from chapter 1). Counting sort and radix sort both hit that bound exactly, in the regime where their structural assumptions hold.

What this chapter changed was *which* lower bound applies, not the bound for sorting itself. The $\Omega(n \log n)$ comparison-model bound from chapter 4 still holds for any algorithm that only compares pairs of elements. Counting sort and radix sort just aren't in that model. They use a stronger operation — array indexing by element value, digit extraction — and the lower bound for *that* richer model is the simpler $\Omega(n)$.

This is the reframing the chapter wants to leave you with: **lower bounds belong to models, not problems.** The decision-tree bound was a fact about what comparisons can resolve. Linear-time sorting was always possible — the algorithms in this chapter just had to enrich the model with operations the comparison-only world didn't allow.

6.4 Lower bounds belong to models

Counting sort is correct because the prefix-sum array dictates exact placements; radix sort is correct because each stable per-digit pass extends sortedness by one digit without disturbing what earlier passes built. Both run in $O(n)$ when their structural assumptions hold — a bounded value range for counting sort, a fixed-width digit representation for radix sort. Both are optimal in their model — you cannot do better than linear because you must read every element at least once.

A lower bound says what an algorithm cannot do *within a given model*, not what it cannot do at all. Change the model and the bound changes. The comparison model's $\Omega(n \log n)$ bound on sorting is a statement about what binary decisions can resolve. The richer "look inside the element" model — direct addressing, digit extraction — gets a lower bound of $\Omega(n)$, because each operation extracts more bits of information than a single comparison does. The price for linear-time sorting was relaxing the opacity assumption hidden in chapters 3 through 5.

That same trick — exploit the *intrinsic structure* of the input rather than treating it as opaque — comes back throughout the rest of the book. Hash tables in Part II turn keys into addresses, which is exactly counting sort's value-is-the-address move generalized to any key type. Trees in Part III still use comparison, but they exploit the *order structure* of the keys to make every comparison cut a subtree in half. Suffix arrays and tries in Part IV exploit the *character structure* of strings to do pattern matching faster than comparison alone could ever achieve. The pattern recurs every time an algorithm decides to stop treating its input as opaque and start asking what's inside.

In the next chapter, the focus shifts from theoretical algorithms to the ones you use. Real-world sorting — what Python's `sorted()`, Java's `Arrays.sort`, and C++'s `std::sort` do under the hood — isn't any of the algorithms in this part of the book applied uniformly. It's a *composition* of them, hand-stitched to exploit the structure of real inputs. The chapter is short, the lesson is large: asymptotic optimality is necessary but not sufficient.

6.5 Notes and further reading

Counting sort is folklore; the first published treatment is usually attributed to Harold Seward in 1954. CLRS (4th ed.) §8.2 gives the clean treatment. Radix sort goes back to Herman Hollerith's tabulating machines in the 1880s — it was a hardware algorithm long before it was a software one — and was formalized for digital computers in the 1950s. CLRS §8.3 covers LSD radix sort; there's also an MSD (most significant digit first) variant treated in Sedgewick's *Algorithms* (4th ed.) §5.1 that uses recursion in a way that mirrors quicksort more than counting sort. The model-vs-problem framing of the comparison lower bound — that $\Omega(n \log n)$ is a property of the comparison model rather than of sorting — is treated in CLRS §8.1; it's the same insight that this chapter built on, said two ways. The relationship between counting sort and hashing — both being "use the value as an address" — is sometimes called *direct addressing* in the data-structures literature; you'll meet it again in Part II.

7 Sorting, in practice

Every algorithm in this book so far has been measured asymptotically — by how the cost grows with input size, ignoring constants. That's the right model for theoretical analysis and the wrong model for production code. By the time you call `sorted()` on a list of one thousand elements, the constants matter more than the asymptotic order, and the asymptotic order is a constraint, not a recipe.

This chapter is about what real implementations of sorting do, and it's a quiet vindication of the previous five chapters. The algorithms used in production — **introsort** in C++'s `std::sort`, **Timsort** in Python's `sorted()` and Java's `Arrays.sort` and Rust's `slice::sort` — aren't new inventions. They're compositions of the algorithms you already have. Insertion sort for small subarrays. Quicksort for the recursive divide. Merge sort for stable, predictable behavior. The art of practical sorting is knowing *which* move applies *where*, and stitching them together so each one operates only on inputs it's optimal for.

Two production-style sorting algorithms — a simplified introsort and a simplified Timsort — show how the techniques from chapters 3 through 5 compose for real-world performance. **A practical sort algorithm is a composition, not a choice.** You don't pick merge sort or quick sort or insertion sort. You pick which algorithm runs on which part of the input, and you let each one do what it's best at.

7.1 Why insertion sort earns its place

Asymptotically, insertion sort is $O(n^2)$ and merge sort is $O(n \log n)$. For large n, merge sort wins. For small n, the asymptotic comparison lies to you.

The reason is the constant factor. Merge sort allocates auxiliary arrays, recurses, and does index bookkeeping; each of those operations costs real time per element. Insertion sort, on a list of 8 elements that's already nearly sorted, runs a tight loop of comparisons and a couple of swaps — no recursion, no allocation, almost nothing but linear walks through cache-warm memory. On a 16-element subarray, insertion sort beats merge sort and quicksort by a factor of 2 to 4 in real wall-clock time, despite being asymptotically slower.

The exact crossover point varies by language, hardware, and even the data being sorted, but a cutover threshold somewhere between 8 and 32 is the conventional sweet spot. CPython's Timsort uses 32. C++'s `std::sort` typically uses 16. The choice is empirical: every implementation benchmarks its own.

The practical recipe writes itself: when the recursion gets down to a small subarray, stop recursing and call insertion sort directly. The asymptotic order doesn't change — large inputs still need an $O(n \log n)$ algorithm — but the constant factor improves dramatically because every leaf of the recursion tree is now an efficient straight-line loop instead of an inefficient recursive call.

```
from typing import MutableSequence
from codex.types import Ordering, default_order


def _insertion_sort_range[T](
    items: MutableSequence[T], lo: int, hi: int, order: Ordering[T]
) -> None:
    """In-place insertion sort on items[lo:hi]. Tight loop, no recursion."""
    for i in range(lo + 1, hi):
        j = i
        while j > lo and order(items[j], items[j - 1]) < 0:
            items[j], items[j - 1] = items[j - 1], items[j]
            j -= 1
```

This is the same insertion sort from chapter 3, parameterized to work on a sub-range instead of the whole array — which is what the recursive callers need. It's the building block for everything else in this chapter.

7.2 Introsort: quicksort with safety nets

David Musser introduced introsort in 1997 to solve a real problem with quicksort: its $O(n^2)$ worst case, which random-pivot selection makes improbable but not impossible. The "intro" stands for *introspective*: introsort runs quicksort normally, watches its own recursion depth, and bails out to a different algorithm if the depth grows past a sane threshold — the signal that the pivot choices have been unlucky and the algorithm is degenerating toward quadratic behavior.

The full Musser algorithm has three pieces:

1. **Quicksort** as the main algorithm — top-down recursion, partition around a pivot, recurse on each side.
2. **Insertion sort** for small subarrays (the corner-cut from the previous section).
3. **Heapsort** as a fallback when the recursion depth exceeds $2 \log_2 n$ — a guarantee against $O(n^2)$.

The third piece needs heapsort, which I haven't introduced yet (it lands in Part III, where heaps get their proper treatment). For this chapter's simplified introsort, I'll skip the heapsort fallback and only implement the first two pieces. The result is "introsort without the safety net" — fine for inputs that aren't adversarially constructed, which is the vast majority of real-world inputs. The depth limit and heapsort fallback are a one-paragraph addition once you have a heap, and I'll add that pointer in the notes at the end.

```
import random
from codex.sort.quick import partition


def introsort[T](
    items: MutableSequence[T],
    order: Ordering[T] = default_order,
    insertion_cutoff: int = 16,
) -> None:
    """Introsort-lite: quicksort with insertion-sort cutover for small subarrays.

    The original Musser introsort also tracks recursion depth and falls back
    to heapsort beyond 2*log2(n) levels; that piece needs heaps (Part III)
    and is omitted here.
    """
    _introsort(items, 0, len(items), order, insertion_cutoff)


def _introsort[T](
    items: MutableSequence[T],
    lo: int,
    hi: int,
    order: Ordering[T],
    cutoff: int,
) -> None:
    if hi - lo <= cutoff:
        _insertion_sort_range(items, lo, hi, order)
        return
    pivot_idx = random.randint(lo, hi - 1)
    items[pivot_idx], items[hi - 1] = items[hi - 1], items[pivot_idx]
    p = partition(items, lo, hi, order)
    _introsort(items, lo, p, order, cutoff)
    _introsort(items, p + 1, hi, order, cutoff)
```

The structure is exactly the quicksort from chapter 4 with two changes: a random pivot (since this is meant to be production-grade), and an insertion-sort cutover when the subarray is small.

```
from codex.sort.practice import introsort

items = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5, 8, 9, 7, 9, 3, 2, 3, 8, 4]
introsort(items)
```

```
print(items)   # sorted in place

# Large random input - the realistic case
import random as _random
_random.seed(0)
big = [_random.randint(0, 1000) for _ in range(10000)]
introsort(big)
print(big[:10], "...", big[-10:])   # sorted, ends with the largest values
```

```
[1, 1, 2, 2, 3, 3, 3, 3, 4, 4, 5, 5, 5, 6, 7, 8, 8, 9, 9, 9]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0] ... [999, 999, 999, 1000, 1000, 1000, 1000, 1000, 1000, 1000]
```

C++'s `std::sort` has used a full introsort since around 1997, which is why C++ programmers can call it on any container without worrying about $O(n^2)$ behavior — the depth limit catches the pathological cases and switches to heapsort, which is $O(n \log n)$ guaranteed. The composition — quicksort + insertion sort + heapsort — covers every input regime: large gets quicksort, small gets insertion sort, pathological gets heapsort.

7.3 Timsort: merge sort that sees the runs

Tim Peters wrote Timsort in 2002 for Python's `list.sort()`, after noticing two structural facts about real-world inputs that the comparison-based lower bound argument from chapter 4 ignored:

1. **Real data has runs.** Files arrive sorted by timestamp; database records arrive sorted by primary key; user-facing lists arrive partially sorted because users edit them incrementally. A genuinely random input is the rare case; the common case has long ascending (or descending, which you reverse) subsequences already in place.
2. **Bottom-up merge sort avoids recursion overhead.** Instead of recursively splitting until you reach single elements and then merging back up, start by treating each element (or each natural run) as already-sorted, and merge pairwise upward until one sorted whole remains.

Timsort combines both insights: identify the natural runs in the input, extend short ones with insertion sort to a minimum length, then merge the runs bottom-up using a careful merge strategy that keeps the run sizes balanced. The result is $O(n)$ **on already-sorted input** — Timsort detects the single big run and merges nothing — and $O(n \log n)$ on random input, with constants that benchmark beautifully on real-world data.

The full Tim Peters algorithm has more moving pieces than this chapter has room for: a run stack with invariants that govern when to merge, a galloping mode that — when one side of a merge keeps winning — jumps ahead in binary-search steps instead of advancing one element at a time, and careful handling of descending runs. I'll implement a simplified version that captures the two core ideas — insertion-sort for short chunks plus bottom-up merging — and skip the run-detection optimization, which I'll add a paragraph about at the end.

The merge step itself is the merge from chapter 4, retargeted to operate on a range of the array rather than two separate lists:

```
def _merge_range[T](
    items: MutableSequence[T],
    lo: int,
    mid: int,
    hi: int,
    order: Ordering[T],
) -> None:
    """Merge items[lo:mid] and items[mid:hi] in place. Both halves must be sorted."""
    left = list(items[lo:mid])
    right = list(items[mid:hi])
    i = j = 0
    k = lo
    while i < len(left) and j < len(right):
        if order(left[i], right[j]) <= 0:
            items[k] = left[i]
```

```
            i += 1
        else:
            items[k] = right[j]
            j += 1
        k += 1
    while i < len(left):
        items[k] = left[i]
        i += 1
        k += 1
    while j < len(right):
        items[k] = right[j]
        j += 1
        k += 1
```

The simplified Timsort itself has two phases. First, sort fixed-size chunks (the "min runs") with insertion sort. Second, merge the chunks pairwise, doubling the run size each time, until one sorted run remains.

```
def timsort_lite[T](
    items: MutableSequence[T],
    order: Ordering[T] = default_order,
    min_run: int = 32,
) -> None:
    """Timsort-lite: bottom-up merge sort with insertion-sort runs.

    The real Timsort (Tim Peters, 2002) also detects natural runs in the
    input and uses a run-stack with merge invariants; both refinements are
    described in the chapter prose and omitted here.
    """
    n = len(items)
    if n < 2:
        return

    # Phase 1: sort fixed-size chunks with insertion sort.
    for start in range(0, n, min_run):
        end = min(start + min_run, n)
        _insertion_sort_range(items, start, end, order)

    # Phase 2: bottom-up merge of the sorted chunks.
    size = min_run
    while size < n:
        for left_start in range(0, n, 2 * size):
            mid = min(left_start + size, n)
            right_end = min(left_start + 2 * size, n)
            if mid < right_end:
                _merge_range(items, left_start, mid, right_end, order)
        size *= 2
```

```
from codex.sort.practice import timsort_lite

items = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5, 8, 9, 7, 9, 3, 2, 3, 8, 4]
timsort_lite(items)
print(items)   # sorted in place

# Stability check - equal keys preserve their relative order
pairs = [(1, 'a'), (3, 'b'), (1, 'c'), (3, 'd'), (2, 'e'), (1, 'f')]
timsort_lite(pairs, lambda x, y: -1 if x[0] < y[0] else (1 if x[0] > y[0] else 0))
print(pairs)   # (1, 'a'), (1, 'c'), (1, 'f'), (2, 'e'), (3, 'b'), (3, 'd')
```

```
[1, 1, 2, 2, 3, 3, 3, 3, 4, 4, 5, 5, 5, 6, 7, 8, 8, 9, 9, 9]
[(1, 'a'), (1, 'c'), (1, 'f'), (2, 'e'), (3, 'b'), (3, 'd')]
```

The real Timsort goes further on a single dimension that matters enormously in practice. Before phase 1, it walks the input once looking for **natural runs** — maximal ascending sequences that are already in the data. Each natural run becomes a unit; only short runs get extended by insertion sort to the minimum length. The merge phase then operates on a mix of natural runs (potentially much longer than `min_run`) and forced-min-run chunks, and the run-stack with merge invariants keeps the merge tree balanced. The result is **adaptive**: input that's already sorted runs in $O(n)$ time, input that's reverse-sorted runs in $O(n)$ (because reverse runs get detected and flipped), and input with even a small amount of structure runs faster than random input. CPython, OpenJDK, and Rust all ship versions of the full algorithm.

7.4 The three questions, applied

Is it correct?

Both algorithms reduce to compositions of correct algorithms from previous chapters. **Introsort** is quicksort with an insertion-sort fallback for small subarrays. Quicksort's correctness (chapter 4) covers the recursive case; insertion sort's correctness (chapter 3) covers the base case. The cutover threshold doesn't affect correctness — only performance. **Timsort-lite** is insertion sort on the first level of small chunks, followed by stable bottom-up merging. Insertion sort's correctness covers phase 1; merge's correctness (chapter 4) covers phase 2; the stability of merge ensures that each level of merging preserves the order that the previous level established.

Stability splits the two. Introsort isn't stable — quicksort isn't, and the insertion-sort cutover doesn't restore it — but Timsort-lite is, because insertion sort and merge are both stable. The full Timsort is stable for the same reason.

How efficient is it?

Introsort-lite is $O(n \log n)$ expected with the random pivot, $O(n^2)$ in the unlikely worst case (which the full Musser introsort upgrades to $O(n \log n)$ worst-case via heapsort fallback). Space is $O(\log n)$ for the recursion stack. The insertion-sort cutover doesn't change the asymptotic complexity; it improves the constant factor by reducing the per-recursion overhead at the leaves of the recursion tree.

Timsort-lite is $O(n \log n)$ worst-case and $O(n)$ on already-sorted input (because the first level of merging finds adjacent chunks already in order and the merge has no work to do beyond the linear scan). Space is $O(n)$ — the merge phase allocates auxiliary buffers proportional to the chunk size at each level. The full Timsort retains the $O(n \log n)$ worst-case and improves the best-case behavior on partially-structured input through the run-detection mechanism.

In production code, the asymptotic order ($O(n \log n)$) tells you what doesn't change as inputs scale; the constants tell you what does. Practical sorts win by attacking the constants — by using insertion sort where its tiny constant beats merge sort's overhead, by going bottom-up to skip recursion frames, by detecting natural runs to skip work entirely. The asymptotic analysis from chapter 4 didn't disappear; it became the floor, not the ceiling.

Is it optimal?

Within the comparison model, both algorithms hit the $\Omega(n \log n)$ lower bound from chapter 4. Neither is asymptotically better than merge sort or quick sort. **What introsort and Timsort are *optimal at* is real-world wall-clock performance** — they pay attention to constants, cache behavior, and input structure that the asymptotic analysis didn't see. They're optimal in a richer model than the comparison model: a model that includes constant factors, branch prediction, and the long pre-sorted runs that real data tends to contain.

That's the chapter's punch. The comparison lower bound is a theoretical constraint, but practice operates inside a much richer cost model than the comparison model captures. Production sorting wins by exploiting structure the theoretical analysis ignored.

7.5 Compose, don't choose

Introsort and Timsort are correct because they're compositions of algorithms you've already verified. Both run in $O(n \log n)$, with Timsort hitting $O(n)$ on inputs that are already sorted. Both improve the constant factor of theoretical sorts by orders of magnitude in practice, by using insertion sort where it wins on small inputs and by exploiting the structure of real-world data that the theoretical lower bound doesn't see.

In production code you don't pick *an* algorithm; you compose several — insertion sort for the leaves, quicksort for the divide, merge sort for stability, heapsort as the safety net. Each one runs only on the regime it's best for. The practitioner's skill is recognizing which technique applies where and composing them without scrambling each one's invariants.

This idea — that asymptotic optimality is necessary but not sufficient — is going to come back throughout the rest of the book. Hash tables in Part II use open addressing, chaining, or both, depending on load factor. Balanced trees in Part III use rotations to keep the worst case bounded while running fast on average. Graph search in Part V uses different traversal orders depending on the question being asked. In every case, the theoretical algorithm is the starting point, not the destination. Real software stitches the moves together for the inputs it sees.

The next short essay takes inventory of what Part I taught — structure as the secret ingredient, the cynosure of inversions, recursion and randomization as design weapons, lower bounds belonging to models, and the composition principle this chapter just landed.

7.6 Notes and further reading

Introsort was introduced by David Musser in 1997 in *"Introspective Sorting and Selection Algorithms,"* Software: Practice and Experience 27(8). The paper is short and worth reading; the depth-limited fallback to heapsort is the genuinely original idea. C++'s `std::sort` adopted introsort soon after, which is why C++ programmers since the late 1990s have had a single sort function with $O(n \log n)$ worst-case guarantees.

Timsort was developed by Tim Peters for CPython in 2002. Peters wrote a long design document that's still in the CPython source tree and remains the best primary reference. The full algorithm — with run detection, merge-stack invariants, and galloping mode — is also implemented in OpenJDK (since Java 7) and Rust's standard library. Timsort was famously the subject of a formal-verification effort: de Gouw et al. (2015) used the KeY theorem prover to verify Java's Timsort implementation and discovered a real bug in the merge-stack invariant maintenance, which had been hiding in production code in three major language runtimes for years. The bug-fix patch was 4 lines.

CLRS doesn't cover practical sorting in any depth; its focus is the theoretical lower bound. Sedgewick & Wayne's *Algorithms* (4th ed.) §2.3 has a production-flavored quicksort treatment that hints at introsort. The CPython source — `Objects/listobject.c` and `Objects/listsort.txt` — is the canonical reference for Timsort details if you want to see how the full algorithm looks in idiomatic C.

Lessons from Part I

The surprise of Part I is how little is genuinely new each time. The same five moves keep coming back, applied in different combinations against the same handful of structural questions. Linear search and Timsort look nothing alike on the page; pull them apart, and they're both reading the input one element at a time and deciding what to do with what they just saw. The vocabulary of algorithm design is small. What's hard is recognising which word applies.

This is the short retrospective. The chapters have the algorithms; this essay names the patterns that kept showing up, the meta-lessons that survive the specifics, and the bridge into Part II, where the focus shifts from algorithmic moves to the data structures those moves act on.

The patterns that kept showing up

The adversary. Chapter 1 introduced them — the malicious player who can rearrange the input the moment your algorithm isn't looking — and they kept reappearing throughout the part. The adversary is the lower-bound argument made dramatic, and they're remarkably robust: in chapter 5, the same adversary that forced linear search to be $\Omega(n)$ forces *any* selection algorithm to be $\Omega(n)$, because if you don't read every element, the adversary will set the unread one to the answer. In chapter 6, even the linear-time sorts respect the adversary's $\Omega(n)$ bound. The adversary doesn't care which model you're in. They just care that you've ignored an element you could have looked at.

The three questions. Every algorithm chapter ended with the same three subheadings in the same order: *Is it correct? How efficient is it? Is it optimal?* I called it the book's cynosure in chapter 0, and after running it through six algorithm chapters I hope it feels less like a template and more like a discipline. Asking the third question — *can anything do better, in this model?* — is the move that separates implementing an algorithm from understanding it.

Inversions. A pair of elements out of order. The local evidence that a sequence isn't sorted. The geometry that explains why bubble sort and quicksort have the asymptotic complexities they do. The inversion count showed up as the *distance from sortedness* in chapter 3, as the *local-vs-global decomposition* that organizes divide-and-conquer in chapter 4, and as the *side effect* that merge sort can count for free in $O(n \log n)$. The lens is sorting-specific — it won't carry forward into the rest of the book in the same form — but the idea underneath it (find a structural property of the input that the algorithm can attack incrementally) will keep coming back, in costumes you'll recognise after a moment of looking.

Recursion. Every nontrivial algorithm after chapter 3 was recursive: binary search, merge sort, quick sort, quickselect, median of medians. Recursion is how a programmer says *"I solved the small case; the big case is the same algorithm on a smaller piece."* It's the cleanest expression of divide-and-conquer, and the recurrence analysis you learned to do — $T(n) = 2T(n/2) + O(n)$ giving $O(n \log n)$, $T(n) = T(n/2) + O(n)$ collapsing to $O(n)$ — is the algebra of recursive thinking. The two recurrences differ by a single coefficient. That coefficient is the difference between sorting and selection.

Randomization. Quickselect was the first place randomization earned its keep, and it's the cleanest example I know of the *Las Vegas* style: the algorithm is always correct, but the cost depends on a random choice the algorithm itself makes. Randomization is a deliberate strategy for breaking adversarial input patterns. An adversary can construct a worst case for any deterministic algorithm by predicting its choices. They cannot construct a worst case for a randomized algorithm without predicting your random number generator, and they cannot predict that.

Looking inside. The first six chapters of Part I treated elements as opaque — the algorithms could compare two of them but couldn't see what was inside. Chapter 6 was the exit: counting sort and radix sort look *into* the element, treating its value as an address or its digits as separate sortable keys, and that single change drops the lower bound from $\Omega(n \log n)$ to $\Omega(n)$. The pattern recurs throughout the rest of the book in different costumes: hash tables turn keys into addresses, tries decompose strings into characters, suffix arrays treat strings as positions in a meta-array.

Composition. Chapter 7's lesson, and the one that goes furthest beyond this book. In theory you pick *the right algorithm*. In practice you *stitch the right moves together*: insertion sort for the small cases, quicksort for the recursive divide, merge sort for stability, heapsort for safety. The practitioner's skill isn't algorithm selection — it's algorithm composition, knowing which moves apply where.

Five meta-lessons

None of these are new with this book; all are easier to articulate after seven chapters of seeing them in action.

1. Asymptotic complexity is a floor, not a recipe. Knowing that an algorithm is $O(n \log n)$ doesn't tell you whether to use it; it tells you what the cost *cannot drop below*, given the model. Real software wins where the asymptotic model can't see: in cache effects, constant factors, branch prediction, the empirical distribution of real inputs. Timsort and introsort are $O(n \log n)$ algorithms whose engineering wins are below the asymptotic order's resolution.

2. Lower bounds belong to models. The $\Omega(n \log n)$ comparison bound from chapter 4 was not a fact about sorting. It was a fact about *sorting using comparisons only*. Drop the comparison-only restriction and the bound goes away — chapter 6 was three pages of demonstrating exactly that. The lesson generalises: any time you see a lower bound, ask which model it's a bound in, and whether your problem really lives in that model.

3. Structure is information, and information is what algorithms convert into speed. Linear search has $O(n)$ because the input is unstructured. Binary search has $O(\log n)$ because the input is sorted. Counting sort has $O(n)$ because the input range is bounded. Every speedup in Part I came from finding some property of the input the previous algorithm wasn't exploiting. The general move: *what does the input know about itself, that I'm not yet using?*

4. The right algorithm is rarely one algorithm. Production sorting routines are compositions. The same pattern shows up everywhere serious code is written: hash tables compose chaining with open addressing depending on load; balanced trees compose rotations with size-based heuristics; graph algorithms compose BFS, DFS, and priority-based traversal depending on the question being asked. The theoretical algorithm is the starting point. The production algorithm is the composition.

5. Always ask the third question. *Is this optimal? Can anything do better, in this model?* Make it a reflex — the thing you do the moment you've finished writing an algorithm. The answer doesn't have to be yes. The asking is what separates writing code that works from understanding the cost shape of the problem.

What's next

Part II is a change of altitude. The seven chapters you just finished were about *algorithmic moves* — divide and conquer, exploit structure, randomize, look inside, compose. The next part is about the *substrates those moves act on*: the data structures that store and retrieve data efficiently, so that the algorithms in Parts III through IX have something to stand on.

Linked lists, stacks, queues, hash tables, heaps, trees. None of these is an algorithm by itself; each is a contract — a set of operations with a cost shape — that algorithms in the rest of the book will lean on without thinking about. Hash tables make $O(n)$ algorithms possible for problems that were $O(n \log n)$ with comparison-based structures. Heaps make Dijkstra's shortest-path algorithm possible. Trees make ordered queries logarithmic and balanced operations cache-friendly.

The lens shifts, but the moves don't. You'll see *exploit-the-intrinsic-structure-of-the-key* in hash tables. You'll see *throwing half away with every comparison* in balanced trees. You'll see *composition over selection* in every real-world data structure, where the production implementation is always a hybrid of two or three textbook ideas glued together with care.

Seven chapters, five moves. They'll keep showing up — onward to Part II.

Part II

Fundamental Data Structures

This second part of the Codex is where you meet **the operation-versus-layout trade** — the bargain between what operations you want cheap and how you lay bytes in memory. The structures in Part I treated their inputs as abstract sequences and asked algorithmic questions about them. The seven chapters in this part ask the opposite question: *given that I want certain operations to be cheap, how should I lay the data out in memory?*

The answers turn out to compose. Each chapter picks one or two operations to make cheap, gives up something to get there, and hands its design to the next chapter as a building block.

- **Memory and sequences** (chapter 8) opens on the first split, contiguous storage versus linked, and earns its keep with the dynamic array's *doubling discipline*: amortized $O(1)$ append paid for by the rare $\Theta(n)$ resize. The accountant's view of cost it introduces returns three more times before the part ends.
- **Linked lists** (chapter 9) gives up random access entirely in exchange for $O(1)$ pointer surgery in the middle of a chain. Singly, doubly, circular, with reverse, merge, and Floyd's cycle detection as the standard manipulations.
- **Stacks** (chapter 10) restrict you to operating at one end of a sequence. That restriction is the whole value — function calls, parser state, undo histories, and Shunting Yard all become the same shape underneath: a memory of pending decisions.
- **Queues and deques** (chapter 11) invert the stack's discipline (queues) or relax it (deques). The monotonic deque used as a *sliding evidence pruner* turns the $\Theta(nk)$ sliding-window-maximum problem into $\Theta(n)$. Same data structure, non-obvious role.
- **Hash tables** (chapter 12) introduce the chapter's most consequential single trick: letting the key compute its own address. Chaining and open addressing are two ways of handling the inevitable collisions, and the load factor is the parameter that controls how the structure behaves between $\Theta(1)$ and $\Theta(n)$.
- **Bloom filters** (chapter 13) push the key-as-address idea in a different direction: instead of storing the keys, store only the *fact* that someone hashed there. Accept a tunable false-positive rate, get a million-element set in a megabyte.
- **Disjoint-set / Union-Find** (chapter 14) pushes amortized analysis to its most striking conclusion: operations whose worst-case cost is $O(\log n)$ but whose amortized cost is the inverse Ackermann function. Effectively constant, for any input you'll ever construct.

Five moves come back through the rest of the book: **the amortized doubling discipline** for absorbing growth, **pointer surgery** for local modification, **discipline-as-restriction** for clarifying which operations matter, **escaping the comparison model** by computing on key content, and **trading certainty for space** when memory is the binding constraint. Every part that follows leans on at least one of them without re-deriving it.

8 What contiguous memory buys you, and what it costs

Memory is flat. Every algorithm in this book has to put its data somewhere on that flat tape, and the choice of where — contiguous block or scattered nodes — decides which operations get to be cheap. That's the trade that organizes the next seven chapters. You can have fast random access or you can have fast surgery in the middle, but you can't have both at full strength, because the two depend on incompatible memory layouts.

A dynamic array — the same structure Python's `list` is built on — gives you amortized $O(1)$ append even though some individual appends cost $\Theta(n)$. The aggregate cost of n appends is bounded by $2n$ copies; averaged over all of them, the cost per append is less than 2.

The accountant's view of amortization is the picture that earns it. Cheap operations put dollars on the table; the rare expensive operation spends what's already there. The accountant flips through the ledger at year-end and doesn't care that some individual entries are spikes. What matters is that the column sums.

That same accounting move comes back in chapter 12 when hash tables rehash, and again in chapter 14 when union-find's path compression amortizes its self-modification cost.

8.1 Contiguous or linked — pick one, get one

The flat tape of memory holds bytes at integer addresses, and every data structure in this book is a way of deciding which bytes go where. There are exactly two atomic choices: put related items **next to each other** (contiguous), or put them **wherever there's room** and let each one hold a pointer to the next (linked). Everything else in Part II is some combination of these two atoms.

Contiguous layout buys you **random access**: if you know the address of item 0 and you want item i, you just compute `address(0) + i * size_of_item` and dereference. That's one multiplication and one addition, regardless of i. Contiguous layout also buys you **locality**: items next to each other in memory end up next to each other in the CPU's cache, so walking the structure linearly is fast for reasons that have nothing to do with the algorithm and everything to do with hardware.

The cost is that contiguous layout is a promise about *future addresses*. If you reserve a thousand slots, fill them, and then want a thousand-and-first item, you can't just keep going. The byte after slot 999 might already belong to something else. You have to find a fresh contiguous run of two thousand slots somewhere else, copy everything over, and update the address of slot 0. The cost of that copy is what the rest of the chapter is about.

Linked layout (chapter 9) gives up random access entirely in exchange for never needing to make that promise. The next chapter implements it from scratch. This chapter implements the other atom — the one Python's `list` already gives you, transparently — and shows how the copy cost is hidden in plain sight.

8.2 Why growing one slot at a time is quadratic

The simplest version of the contiguous-array idea is the **fixed-size array**: you pick a capacity up front, allocate that many slots, and that's all you ever get. If you fill it and want more, the answer is "you can't." For storing a known-size collection of items, that's enough. For building up a sequence one item at a time, when you don't know in advance how many items there'll be, it isn't.

The naive escape is to grow the array by one slot each time it fills up. You allocate a new block of size $k + 1$, copy the existing k items into the first k slots, write the new item into slot k, and free the old block. That works — the structure stays correct — but the cost is brutal.

To append n items starting from a fresh empty array, you copy $0 + 1 + 2 + ... + (n - 1) = n(n - 1)/2$ items in total, just for the resize work. That's $\Theta(n^2)$ total, or $\Theta(n)$ amortized per append. Linear-per-append is a death

sentence for a data structure that's supposed to be the basic sequence type. If appending each character of a million-character string costs a trillion copies, the structure is useless.

The fix is one number: instead of growing by one, you grow by a *factor*. Doubling the capacity each time the array fills up turns the aggregate copy work from $\Theta(n^2)$ into $\Theta(n)$, and the amortized cost per append drops from linear to constant.

8.3 Doubling turns linear into constant

The dynamic array keeps two integers: the **size** (how many items are currently stored) and the **capacity** (how many slots are allocated). The invariant is `0 ≤ size ≤ capacity`. When `size == capacity` and you want to append, you allocate a new block of size `2 * capacity`, copy the old items, and update both numbers.

Here's the structure from scratch.

```
class DynamicArray[T]:
    def __init__(self, initial_capacity: int = 4) -> None:
        self._capacity = initial_capacity
        self._size = 0
        self._items: list[T | None] = [None] * initial_capacity

    def __len__(self) -> int:
        return self._size

    def __getitem__(self, i: int) -> T:
        if not 0 <= i < self._size:
            raise IndexError(f"index {i} out of range for size {self._size}")
        return self._items[i]  # type: ignore[return-value]

    def __setitem__(self, i: int, value: T) -> None:
        if not 0 <= i < self._size:
            raise IndexError(f"index {i} out of range for size {self._size}")
        self._items[i] = value
```

I'm using Python's built-in `list` as the underlying contiguous storage, since that's exactly what a fixed-size array maps to at this level. A "real" implementation in C would call `malloc` to allocate `capacity * sizeof(T)` bytes; in Python, the `[None] * initial_capacity` line plays the same role. The point is that `_items` is treated as if it had a fixed size — never grown by Python's own list machinery, only by the explicit `_resize` you'll see in a moment.

The interesting method is `append`. It checks whether there's room, calls `_resize` if not, and writes the new item into the next free slot.

```
    def append(self, value: T) -> None:
        if self._size == self._capacity:
            self._resize(self._capacity * 2)
        self._items[self._size] = value
        self._size += 1
```

The `_resize` helper is what makes amortization work. It allocates a new block of the requested capacity, copies every existing item into the first `size` slots, and replaces `_items`. That's a $\Theta(\text{size})$ operation; it's not constant time when it runs. The trick is that it runs rarely enough that the total work stays linear.

```
    def _resize(self, new_capacity: int) -> None:
        new_items: list[T | None] = [None] * new_capacity
        for i in range(self._size):
            new_items[i] = self._items[i]
        self._items = new_items
        self._capacity = new_capacity
```

And `pop` removes the last item, optionally shrinking the array back down when it's gotten too empty. Shrinking when the array drops to a quarter of its capacity (rather than half) is the standard trick for avoiding pathological *thrashing*, where a sequence of alternating appends and pops at the boundary keeps triggering resizes. Quarter-shrink leaves enough slack that you have to do a non-trivial number of cheap ops between any two expensive ones.

```
    def pop(self) -> T:
        if self._size == 0:
            raise IndexError("pop from empty array")
        self._size -= 1
        value = self._items[self._size]
        self._items[self._size] = None  # let the garbage collector reclaim it
        if 0 < self._size <= self._capacity // 4 and self._capacity > 1:
            self._resize(self._capacity // 2)
        return value  # type: ignore[return-value]
```

The whole structure is about fifty lines. Most of it is bookkeeping; the algorithmic content lives in two places: the doubling line inside `append`, and the quarter-shrink line inside `pop`. Everything else is plumbing.

Running it on a small example:

```
from codex.structures.array import DynamicArray

arr = DynamicArray[int](initial_capacity=2)
for x in [10, 20, 30, 40, 50]:
    arr.append(x)

print([arr[i] for i in range(len(arr))])  # [10, 20, 30, 40, 50] - five items stored
print(f"size: {len(arr)}, capacity: {arr._capacity}")  # size: 5, capacity: 8 - doubled twice
```

```
[10, 20, 30, 40, 50]
size: 5, capacity: 8
```

The array started with capacity 2, hit capacity at size 2, doubled to 4, hit capacity again at size 4, doubled to 8. Two resize events to absorb five appends.

8.4 Two writes per append, on average

Appending n items into a dynamic array costs at most $2n$ slot-copies in total. This holds for any small initial capacity, summed across every resize that happened along the way. Each individual append is therefore *amortized* $O(1)$, even though the resize-triggering ones are $\Theta(n)$ in the worst case.

The argument is short. If you start with capacity 1 and double each time you fill up, the resizes happen at sizes $1, 2, 4, 8, \ldots$, with the last one at size $\leq n$. Each resize copies the current size's worth of items. The total copy work is:

$$1 + 2 + 4 + \ldots + 2^{k-1} = 2^k - 1$$

where 2^k is the smallest power of two greater than or equal to n. Since $2^k < 2n$, the total copy work is bounded by $2n - 1$, and the amortized cost per append is less than 2.

That's the formal claim. The tactile version: **on average, each append pays for about two slot-writes — one for itself, and one to pre-fund a copy that hasn't happened yet.** The doubling discipline is what lets each cheap op contribute to a savings account that the rare expensive op draws down. Without doubling (say, you grew by an additive constant instead), the savings account fills up too slowly to keep pace, and the amortized cost climbs back to linear.

A trace confirms the bound.

```python
from codex.structures.array import DynamicArray


class TracingDynamicArray(DynamicArray):
    def __init__(self, initial_capacity: int = 1) -> None:
        super().__init__(initial_capacity)
        self.copies_done = 0

    def _resize(self, new_capacity: int) -> None:
        self.copies_done += self._size
        super()._resize(new_capacity)


for n in [10, 100, 1_000, 10_000, 100_000]:
    arr = TracingDynamicArray(initial_capacity=1)
    for i in range(n):
        arr.append(i)
    ratio = arr.copies_done / n
    print(f"n={n:>7}: {arr.copies_done:>7} copies, "
          f"per-append  {ratio:.3f} (bound: 2.000)")
```

```
n=     10:      15 copies, per-append   1.500 (bound: 2.000)
n=    100:     127 copies, per-append   1.270 (bound: 2.000)
n=   1000:    1023 copies, per-append   1.023 (bound: 2.000)
n=  10000:   16383 copies, per-append   1.638 (bound: 2.000)
n= 100000:  131071 copies, per-append   1.311 (bound: 2.000)
```

Every row stays comfortably under the 2-per-append bound. The worst rows are the ones where n is just over a power of two; that's where the analysis predicts the worst case to live. (For $n = 100\,000$, the final capacity is $131\,072$, and the bound is loose. Pick n right at a power of two and the ratio collapses toward 1.)

And the resize events themselves — how many of them happen as a function of n? The doubling discipline says the count grows logarithmically.

```python
class CountingDynamicArray(DynamicArray):
    def __init__(self, initial_capacity: int = 1) -> None:
        super().__init__(initial_capacity)
        self.resize_count = 0

    def _resize(self, new_capacity: int) -> None:
        self.resize_count += 1
        super()._resize(new_capacity)


for n in [10, 100, 1_000, 10_000, 100_000]:
    arr = CountingDynamicArray(initial_capacity=1)
    for i in range(n):
        arr.append(i)
    print(f"n={n:>7}: {arr.resize_count} resize events "
          f"(  log  n = {n.bit_length() - 1})")
```

```
n=     10: 4 resize events (  log  n = 3)
n=    100: 7 resize events (  log  n = 6)
n=   1000: 10 resize events (  log  n = 9)
n=  10000: 14 resize events (  log  n = 13)
n= 100000: 17 resize events (  log  n = 16)
```

Logarithmic in n, exactly as predicted. Seventeen resize events to absorb a hundred thousand appends: fewer than two of every ten thousand appends triggered a resize at all, even though the table starts from a single slot. That's the deal. Each individual resize is expensive ($\Theta(n)$ in the worst case, when the last resize copies almost everything), but resizes are rare enough that their total cost stays linear.

8.5 The three questions, applied

Is it correct?

The invariant is `0 size capacity`. It holds at construction (size is 0, capacity is whatever was passed in). It's preserved by `append`: the only way `append` runs is when the invariant holds going in, the `_resize` call is the only thing that changes `capacity`, and `_resize` always sets `capacity` to a value at least as big as the current size, after which `_size += 1` keeps the invariant intact. `pop` decrements size by one, then optionally halves capacity — but only when `size capacity / 4`, which means the new capacity is still at least $2 \cdot \text{size}$, preserving `size capacity`.

The indexed access methods (`__getitem__`, `__setitem__`) check `0 i < size` and raise on violation, so they only return values from slots that were actually written. The `pop` method's "set the slot to `None`" line is the easy-to-miss correctness detail: without it, the popped value would stay reachable from the array and the garbage collector wouldn't be able to free it.

How efficient is it?

The per-operation cost table:

Operation	Worst case	Amortized	Why
`__getitem__(i)`, `__setitem__(i, v)`	$O(1)$	$O(1)$	Pointer arithmetic; the whole point of contiguous storage.
`append(x)`	$O(n)$	$O(1)$	The amortization argument above.
`pop()`	$O(n)$	$O(1)$	Symmetric — the quarter-shrink discipline mirrors the doubling discipline.
Insert at position i (not implemented above)	$O(n)$	$O(n)$	Have to shift $n - i$ items right by one slot. No amortization saves you.
Delete at position i (not implemented)	$O(n)$	$O(n)$	Same reason — shift $n - i - 1$ items left.

The first three are the operations a dynamic array makes cheap. The last two are the operations it makes expensive — and they're the operations a linked list (chapter 9) makes cheap. That's the trade.

Space is $O(n)$ with a constant factor between 1 and 2. The structure is always between half full and full (except during the brief window between a `pop` and the next `_resize`), which means you're paying at most twice the strictly necessary memory. That's the cost of letting `append` be amortized $O(1)$. Halve the slack and the amortized cost climbs; double it and the bound holds with a larger constant.

Is it optimal?

For the operations a contiguous array is designed to make cheap (indexed access, append-at-end, pop-from-end) the answer is yes, and the argument is short. Random access at $O(1)$ is the *definition* of contiguous storage; you can't do better than constant time. Append-at-end at amortized $O(1)$ is matched by the lower bound: you have to at least write the new item into memory, which is $\Omega(1)$ work, and any contiguous-storage scheme that ever needs to grow has to copy at some point. The doubling discipline matches that lower bound up to a constant factor.

For the operations a contiguous array makes expensive — insert and delete at arbitrary positions — the answer is no, and that's exactly where chapter 9 picks up. Linked storage makes insert-and-delete-given-a-node-handle $O(1)$, at the cost of giving up random access. Different layout, different operations cheap, same fundamental trade.

The deeper point is that there's no single "best" sequence data structure. The dynamic array is optimal for *append-at-end-heavy* workloads. The linked list is optimal for *insert-in-the-middle-heavy* workloads. The choice is determined by what your callers do.

8.6 The amortization argument carries forward

The amortization argument — the accountant's view of cost — is what carries forward to chapters 12 and 14. An individual operation's worst-case cost can be much higher than its amortized cost, and for analyzing data structures you care about the *long-run rate*, not the worst-case spike. Doubling is the cleanest example of how to engineer a structure where the rare expensive operation pays for itself with credit accumulated over the cheap ones. The same accountant handles hash-table rehashing and path compression in union-find. Each uses a slightly different flavor of amortized analysis (the aggregate method here, the accounting method and potential method later), all answering the same question: *over many operations, what does this cost on average?*

8.7 Notes and further reading

The amortized analysis of dynamic-array doubling is CLRS (4th ed.) §17.4, which treats the same construction by both the aggregate method (used above) and the accounting method (used in CLRS chapter 12 for hash-table rehashing). Sedgewick & Wayne's *Algorithms* (4th ed.) §1.3 covers resizable arrays under the name "ResizingArrayStack" and works through the same doubling-and-halving discipline. The quarter-shrink rule for avoiding thrashing under alternating push/pop is folklore but appears explicitly in Sedgewick §1.3. CPython's `list` uses a slightly different growth schedule than strict doubling — it grows by roughly 1.125× — for reasons documented in the CPython source `Objects/listobject.c`; the asymptotic behavior is the same, but the constant factor is tuned differently for real-world allocator behavior.

9 Surgery in the middle of a chain

You have a million records sitting in a dynamic array and you need to insert a new one between records 499,999 and 500,000. With the array from the last chapter you have to shift half a million elements over by one — a single `insert` call doing five hundred thousand writes. That's not a bug in your code. That's a property of contiguous storage: items next to each other in memory have to *stay* next to each other in memory, and the only way to make room in the middle is to push everything past it over.

A linked list answers that complaint with a completely different memory layout. Each element lives at its own address, wherever the allocator decided to put it, and each one holds a pointer to the next. Inserting an item in the middle rewrites exactly two pointers, regardless of how big the list is.

That locality is **pointer surgery** — every operation a small, local rewrite of one or two pointers, never touching the rest of the chain. The price is that you can no longer ask for the i-th element without walking i steps to get there. Singly linked, doubly linked, and circular variants each rebalance the trade differently; three textbook routines (reverse, merge, Floyd's tortoise-and-hare cycle detection) exercise the idiom.

9.1 A node knows its successor

A linked list isn't an object at the language level — it's a discipline you impose on a pile of small objects called **nodes**, each holding a value and a pointer to the next node. The end of the list is whichever node has its pointer set to `None`.

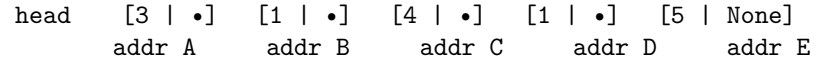
```
from dataclasses import dataclass
from typing import Iterator, Sequence
from codex.types import Ordering, default_order


@dataclass
class Node[T]:
    value: T
    next: "Node[T] | None" = None
```

The `next` annotation has to be a string because the class is referring to itself before its name has finished binding. That's a Python detail, not a deep one — every linked-structure language has some version of this footnote.

A *chain* of nodes is just a `Node[T]` you've decided to call the head. You don't need a separate list object to walk it. To iterate, you start at the head and follow `next` pointers until you hit `None`. To find the i-th element, you walk i steps. To find the length, you walk the whole thing. Those last two are the linked-list tax: indexed access is $\Theta(i)$, not $O(1)$.

A typical chain laid out flat, with addresses standing in for "wherever each node ended up in memory":



The addresses A through E might be anywhere in memory — no requirement of contiguity. The list as a whole holds together because each node's `next` pointer names the next address in the chain.

I'll also wrap a chain in a small `LinkedList[T]` class that tracks the head, the tail, and the size, so that operations like "append to the end" and "ask the length" can be $O(1)$ instead of $\Theta(n)$. Without the tail pointer, append has to walk the whole chain. With it, append is two pointer writes.

```
class LinkedList[T]:
    def __init__(self) -> None:
        self.head: Node[T] | None = None
        self.tail: Node[T] | None = None
        self._size = 0

    def __len__(self) -> int:
        return self._size

    def __iter__(self) -> Iterator[T]:
        cur = self.head
        while cur is not None:
            yield cur.value
            cur = cur.next
```

Iteration is the only operation that genuinely needs to touch every node, and it does so with a single walk. Everything else in the API is local.

9.2 Insertion and deletion as pointer surgery

The four operations that actually exercise the data structure are prepend, append, insert-after, and delete-after. Each one rewrites a constant number of pointers; none of them touch nodes beyond their immediate neighbors. That's the chapter's recurring image made literal.

Prepend is the simplest: allocate a new node whose `next` points at the current head, then make that new node the head.

```
    def prepend(self, value: T) -> None:
        node = Node(value, self.head)
        if self.head is None:
            self.tail = node
        self.head = node
        self._size += 1
```

The `self.head is None` check handles the empty case — the new node is *also* the tail when there was no tail before. That's the kind of edge case you have to think about exactly once when you write a linked list, and you trip on it every time you forget.

Append is the mirror image: allocate a new node, point the current tail's `next` at it, then make it the new tail.

```
    def append(self, value: T) -> None:
        node = Node(value)
        if self.tail is None:
            self.head = node
        else:
            self.tail.next = node
        self.tail = node
        self._size += 1
```

Both are $O(1)$. That's the property the tail pointer buys you, and it's why every implementation of a linked list that wants to be taken seriously tracks the tail.

The two interesting operations are `insert_after` and `delete_after`. Both take a node handle — a reference to a specific node already in the list — and operate on its immediate neighborhood. They're the operations that demonstrate why linked lists exist.

```
    def insert_after(self, node: Node[T], value: T) -> None:
        new_node = Node(value, node.next)
        node.next = new_node
        if node is self.tail:
            self.tail = new_node
        self._size += 1

    def delete_after(self, node: Node[T]) -> None:
        target = node.next
        if target is None:
            return
        node.next = target.next
        if target is self.tail:
            self.tail = node
        self._size -= 1
```

`insert_after` does two pointer writes: the new node's `next` gets the old successor, and the predecessor's `next` gets the new node. `delete_after` does one: the predecessor's `next` skips over the deleted target. Both are $O(1)$, given a handle to the node. That "given a handle" qualifier is the silent caveat: if you have to find the node first by walking, you've paid $\Theta(n)$ before the surgery even started. Linked lists win when the caller already holds node handles, a pattern you'll meet again in chapter 11's deque and chapter 12's chained hash table.

Running the basic operations on a small example:

```
from codex.structures.linked import LinkedList

chain = LinkedList[int]()
for x in [1, 2, 3, 4, 5]:
    chain.append(x)

print(list(chain))                      # [1, 2, 3, 4, 5] - appended in order
print(f"length: {len(chain)}")          # length: 5

chain.prepend(0)
print(list(chain))                      # [0, 1, 2, 3, 4, 5] - prepended at head

# Insert 99 after the node with value 2
node = chain.head
while node.value != 2:
    node = node.next
chain.insert_after(node, 99)
print(list(chain))                      # [0, 1, 2, 99, 3, 4, 5]
```

```
[1, 2, 3, 4, 5]
length: 5
[0, 1, 2, 3, 4, 5]
[0, 1, 2, 99, 3, 4, 5]
```

Notice the `while` loop walking to find the target node. That's the linked-list tax made visible: every operation on "the node with this value" pays a linear walk to find the node first. The $O(1)$ savings on `insert_after` itself doesn't change that. The structure is fast only when something else gave you the handle.

9.3 In-place reversal in one walk

A classic interview problem, and the cleanest demonstration of pointer surgery in the chapter. Given the head of a chain, return the head of the same chain with every `next` pointer flipped.

The naive way is to walk the list, collect the values into an array, and rebuild a new list from the reversed array. That works, but it allocates n extra nodes and copies every value — $O(n)$ time and $O(n)$ extra space, plus

all the allocation churn. There's a cleverer way that does the work in place, with three pointers and no extra allocation.

The trick is to walk the chain with three pointers — `prev`, `cur`, and a saved-aside `nxt` — and at each step, flip `cur.next` to point backwards at `prev`, then advance both `prev` and `cur` by one step.

```
def reverse[T](head: Node[T] | None) -> Node[T] | None:
    prev: Node[T] | None = None
    cur = head
    while cur is not None:
        nxt = cur.next
        cur.next = prev
        prev = cur
        cur = nxt
    return prev
```

Five lines, one walk, one node touched per step. The loop invariant: at the top of each iteration, the prefix of the chain from `head` to `prev` is already reversed, and `cur` points at the next node to flip. When the loop terminates, `cur` is `None`, the entire chain has been flipped, and `prev` is the new head (the original tail).

```
from codex.structures.linked import LinkedList, reverse

chain = LinkedList[int]()
for x in [1, 2, 3, 4, 5]:
    chain.append(x)

# reverse returns the new head; rebuild the iterable from it
new_head = reverse(chain.head)

values = []
cur = new_head
while cur is not None:
    values.append(cur.value)
    cur = cur.next
print(values)   # [5, 4, 3, 2, 1] - flipped, in place, with no new node allocations
```

```
[5, 4, 3, 2, 1]
```

Notice that `reverse` doesn't update the `LinkedList`'s head and tail attributes — it operates on the raw chain of nodes. If you wanted the wrapper class to stay consistent after a reverse, you'd write a method that calls `reverse(self.head)`, then swaps `self.head` and `self.tail`. I left that as a method-not-written so the algorithm itself stays small. The point is the surgery.

9.4 Splice, don't copy

Merge sort's inner step from chapter 4 reappears here, but with one big difference: when the input is a linked list, you don't have to allocate a new array to hold the merged output. You can splice the existing nodes into a single sorted chain by walking two pointers, picking the smaller-headed node at each step, and threading it onto a growing result tail. Zero allocations beyond a single dummy node used to simplify the head case.

```
def merge_sorted[T](
    a: Node[T] | None,
    b: Node[T] | None,
    order: Ordering[T] = default_order,
) -> Node[T] | None:
    dummy: Node[T] = Node(None)  # type: ignore[arg-type]
    tail = dummy
    while a is not None and b is not None:
```

```
        if order(a.value, b.value) <= 0:
            tail.next = a
            a = a.next
        else:
            tail.next = b
            b = b.next
        tail = tail.next  # type: ignore[assignment]
    tail.next = a if a is not None else b
    return dummy.next
```

The dummy node is the trick worth naming. Without it, you'd have to special-case the first iteration (when there's no existing tail to attach to). With the dummy, every iteration looks the same, and you return `dummy.next` at the end to skip past the placeholder. Its value is never read; the `# type: ignore` comments are the price of telling Python's type checker that a `None` placeholder is fine for `T`.

```
from codex.structures.linked import LinkedList, merge_sorted


def to_chain(items):
    list = LinkedList[int]()
    for x in items:
        list.append(x)
    return list.head


def to_list(head):
    values = []
    cur = head
    while cur is not None:
        values.append(cur.value)
        cur = cur.next
    return values


a = to_chain([1, 3, 5, 7])
b = to_chain([2, 4, 6, 8])
merged = merge_sorted(a, b)
print(to_list(merged))   # [1, 2, 3, 4, 5, 6, 7, 8] - interleaved in sorted order

# Edge case: one side empty
print(to_list(merge_sorted(to_chain([1, 2, 3]), None)))    # [1, 2, 3]
print(to_list(merge_sorted(None, to_chain([4, 5, 6]))))    # [4, 5, 6]
```

```
[1, 2, 3, 4, 5, 6, 7, 8]
[1, 2, 3]
[4, 5, 6]
```

This is the move that makes merge sort on linked lists *not allocate* during the merge step. Functional-language sorts (Haskell, ML, Lisp) lean on it hard. CPython's stable sort doesn't, because Python's `list` is an array.

9.5 Tortoise and hare: detecting cycles

A linked list isn't supposed to have cycles — the convention is that the chain ends at a node whose `next` is `None`. But nothing in the language stops you from making `tail.next` point at an interior node, accidentally or deliberately, and any algorithm that walks the chain expecting `None` at the end will loop forever if there's a cycle.

Floyd's tortoise-and-hare algorithm detects cycles in $O(n)$ time and $O(1)$ space. Walk the chain with two pointers — a slow one advancing by one step per iteration, a fast one by two. If there's no cycle, the fast pointer hits `None` first and you're done. If there *is* a cycle, the fast pointer eventually catches up to the slow one from behind: both are running around the cycle, the fast one gaining one position per step, and the gap closes within λ steps where λ is the cycle length.

That meeting point isn't generally the *entry* to the cycle. It's some position inside it that depends on μ (head-to-entry distance) and λ. A second phase recovers the entry: reset one pointer to the head, advance both by one step at a time. The congruence $\mu + k \equiv 2(\mu + k) \pmod{\lambda}$, worked out on paper, shows why they meet at the entry.

```
def floyd_cycle[T](head: Node[T] | None) -> Node[T] | None:
    tortoise = hare = head
    while hare is not None and hare.next is not None:
        assert tortoise is not None  # tortoise is at most as far as hare
        tortoise = tortoise.next
        hare = hare.next.next
        if tortoise is hare:
            tortoise = head
            while tortoise is not hare:
                assert tortoise is not None and hare is not None
                tortoise = tortoise.next
                hare = hare.next
            return tortoise
    return None
```

The function returns the cycle's entry node, or `None` if there's no cycle. The two `assert`s are there to help Python's type checker — they're invariant claims about pointers, not runtime checks meant to catch bugs.

```
from codex.structures.linked import LinkedList, floyd_cycle

# build a straight chain 1 → 2 → 3 → 4 → 5 → 6 → 7
chain = LinkedList[int]()
for x in [1, 2, 3, 4, 5, 6, 7]:
    chain.append(x)

# locate the node with value 3
entry_target = chain.head
while entry_target.value != 3:
    entry_target = entry_target.next

# splice in a cycle: tail.next → node 3
# the chain becomes 1 → 2 → 3 → 4 → 5 → 6 → 7 → 3 → 4 → 5 → 6 → 7 → 3 → ...
chain.tail.next = entry_target

entry = floyd_cycle(chain.head)
print(f"cycle entry: {entry.value}")   # cycle entry: 3 - Floyd found it

# and on a cycle-free chain, the algorithm returns None
clean = LinkedList[int]()
for x in [1, 2, 3]:
    clean.append(x)
print(floyd_cycle(clean.head))         # None - no cycle to find
```

```
cycle entry: 3
None
```

The algorithm is constant-space (two pointers, no `visited` set) and linear-time, with at most $\mu + 2\lambda$ steps across both phases. The math looks like a coincidence until you work through it. After that it never goes away.

9.6 A second pointer makes detach local

The singly linked list has one structural limitation: you can move forward but not backward. If you have a node handle and you want to delete *that* node, you can't — you'd need a handle to its predecessor, which the node itself doesn't know. That's why the API above is `delete_after`, not `delete`. The fix is to give each node a second pointer.

```
@dataclass
class DoublyLinkedNode[T]:
    value: T
    prev: "DoublyLinkedNode[T] | None" = None
    next: "DoublyLinkedNode[T] | None" = None


def detach[T](node: DoublyLinkedNode[T]) -> None:
    if node.prev is not None:
        node.prev.next = node.next
    if node.next is not None:
        node.next.prev = node.prev
    node.prev = None
    node.next = None
```

`detach` removes a node from its chain in $O(1)$ given only the node's own handle. Singly linked lists fundamentally cannot do this. That property is what makes the doubly linked list the natural backbone for chapter 11's deque, for LRU caches, and for any structure that holds external handles and wants deletion to be local.

The cost is one extra pointer per node and one extra rewrite per operation. Memory roughly doubles; whether that's a fair trade depends on the workload.

The circular variant is a chain whose tail's `next` points back at the head, so there's no `None` at the end. It's what `floyd_cycle` treats as a bug, except deliberately. Useful for round-robin schedulers, certain kinds of buffers, and some auxiliary graph-algorithm structures in Part V. The construction is one line of post-processing.

```
def make_circular[T](head: Node[T] | None) -> Node[T] | None:
    if head is None:
        return None
    cur = head
    while cur.next is not None:
        cur = cur.next
    cur.next = head
    return head
```

And you'd walk one of these with a counter, not a `while cur is not None`, because the loop condition would never fire. That's the whole structural change — the rest of the code that operates on chains carries over unchanged.

9.7 The three questions, applied

Is it correct?

The invariants of the singly linked `LinkedList` class are three. First: if `head` is `None`, then `tail` is `None` and `_size` is 0 — the empty case is unambiguous. Second: if `head` is not `None`, then `tail` is the unique node reachable from `head` by following `next` pointers whose own `next` is `None` — the chain terminates exactly once. Third: `_size` equals the number of nodes reachable from `head`. Each operation preserves all three; the edge cases (empty list, single node, deletion of the tail) are the places you have to think carefully, which is why the body of every method has at least one branch handling them.

The standalone functions are simpler. `reverse` is correct by its loop invariant: at iteration i, the first i nodes have been flipped, and `cur` points at the next node to flip. `merge_sorted` is correct because at every step,

the chosen node is the smallest available from either input — the same exchange argument that proves merge sort's merge step correct, transferred from arrays to chains. `floyd_cycle` is correct because of the cycle-length congruence: if a cycle exists, the two pointers must eventually meet inside it, and the second-phase walk lands at the entry by the modular-arithmetic identity.

How efficient is it?

The per-operation cost table for the singly linked `LinkedList`:

Operation	Cost	Why
`prepend`, `append`	$O(1)$	Two pointer writes, with the tail pointer available.
`insert_after(node, x)`, `delete_after(node)`	$O(1)$	Two pointer rewrites given a node handle.
Find by value	$\Theta(n)$	Walk the chain.
Find by index	$\Theta(i)$	Walk i steps from the head.
`__len__`	$O(1)$	Maintained as an attribute.
`__iter__`	$\Theta(n)$	One walk through the chain.

The standalone routines: `reverse` is $\Theta(n)$ time, $O(1)$ space. `merge_sorted` is $\Theta(n + m)$ time, $O(1)$ space. `floyd_cycle` is $\Theta(n)$ time, $O(1)$ space — with the explicit constant at most $\mu + 2\lambda$ for a chain with cycle. Space across the structure as a whole is $\Theta(n)$, with the constant above 1 per element because each node holds a pointer plus a value. Doubly linked variants double that pointer overhead.

Is it optimal?

For the operations a linked list is designed to make cheap, yes. Insert and delete given a handle to the predecessor are $\Omega(1)$ — you have to write at least one pointer — and the linked list hits that bound. Reverse is $\Omega(n)$ because every position changes. Cycle detection is $\Omega(n)$ because the cycle might be anywhere in the chain.

For the operations a linked list makes expensive, no — and that's exactly the trade. Random access at $\Theta(i)$ instead of $O(1)$ is the cost of giving up contiguous storage. An array can do better; a linked list fundamentally cannot. Workload decides. Index-heavy callers reach for arrays; insertion-heavy callers holding node handles reach for linked lists. Most real software ends up needing some of both.

9.8 A pointer is a different kind of address

A pointer is a different kind of address than an integer index. An array index is a logical position relative to a base; recomputing it is free. A pointer is a physical address that's only useful given the pointer itself; recomputing it from a logical position requires walking. That difference is the source of every asymptotic asymmetry between arrays and linked lists.

The singly linked list is correct by three invariants (head, tail, size) and makes prepend, append, and adjacent insert/delete $O(1)$ given handles, at the cost of $\Theta(i)$ random access. The doubly linked variant doubles the pointer cost in exchange for $O(1)$ detach-by-handle, which makes it the backbone of every structure that holds external handles and needs to delete by them. The circular variant is a one-line transformation that turns the chain into a loop.

9.9 Notes and further reading

The singly and doubly linked list constructions are CLRS (4th ed.) §10.2 and Sedgewick & Wayne's *Algorithms* (4th ed.) §1.3. Floyd's tortoise-and-hare algorithm is folklore by 1970 but is most often attributed to Robert Floyd, with the formal exposition appearing in Knuth's *The Art of Computer Programming* Vol. 2 §3.1 exercises. The modular-arithmetic proof of the entry-point recovery step is worked out in detail in CLRS Problem 10-2. The in-place linked-list reversal is sometimes called "the three-pointer dance" and shows up in essentially every coding-interview compendium. Brian Kernighan's *Practice of Programming* §2 has a memorable discussion of the dummy-node trick for merge-style routines, including a defense of why the trick is worth the slightly weird-looking allocation.

10 The most recent thing is the easiest to reach

You're walking through a maze and you keep wanting to back up to the last junction you didn't fully explore. You're compiling an expression and you keep wanting to remember the last operator you saw. You're evaluating a function call and you keep wanting to come back to the line that made the call. All three are the same data structure — a stack — and the discipline that makes them feel like the same thing is the rule that **the most recent thing is the easiest to reach**.

A stack isn't really a new data structure. It's an array or a linked list with two of the operations turned off and one promoted. You can only put things in at one end and take them out at the same end. That single restriction — *last in, first out*, LIFO — is what makes function calls work, what makes parsers parse, what makes undo histories undo, and what makes recursive algorithms fit in a finite computer.

A stack is **a memory of pending decisions** — a stack of sticky notes on your desk, each one a thing you meant to come back to, with the most recent on top. Two backings (array and linked list) satisfy the same three-operation contract. The **Shunting Yard** algorithm converts an infix expression like `3 + 4 * 2` to postfix `3 4 2 * +`; the same stack evaluates the postfix in a second pass.

10.1 A stack is a sequence with three operations

The contract is short. A stack supports three operations: **push** (put a value on top), **pop** (remove and return the top value), and **peek** (look at the top value without removing it). That's it. There's no indexed access, no iteration in any particular order, no insertion in the middle. The stack's value isn't what it lets you do; it's what it *forbids* you from doing.

That restriction matters because every algorithm that uses a stack relies on it. Function call frames work because the language guarantees you can't reach down past the top frame and corrupt the one underneath. Shunting Yard works because the operator at the top of the operator stack is always the most-recently-seen one with unresolved precedence. Iterative DFS works because the next node to visit is always the most-recently-discovered one. Take the LIFO restriction away and every one of those algorithms breaks.

There are two natural ways to back the contract. You can use an array: the top of the stack is the end of the array, push is `append`, pop is `pop`. Or you can use a linked list: the top of the stack is the head of the chain, push is `prepend`, pop unhooks the head. Both implementations satisfy the same three-operation contract. They differ on per-operation cost and on what you're paying in memory overhead.

10.2 Array-backed: cheap on average, paid for by doubling

The simplest implementation uses Python's built-in `list`, which is itself a dynamic array of the kind you built from scratch in chapter 8. Append is amortized $O(1)$ thanks to doubling; pop is strict $O(1)$ from the end; indexed access at `[-1]` for `peek` is $O(1)$.

```
class ArrayStack[T]:
    def __init__(self) -> None:
        self._items: list[T] = []

    def __len__(self) -> int:
        return len(self._items)

    def __bool__(self) -> bool:
        return bool(self._items)

    def push(self, value: T) -> None:
```

```
        self._items.append(value)

    def pop(self) -> T:
        if not self._items:
            raise IndexError("pop from empty stack")
        return self._items.pop()

    def peek(self) -> T:
        if not self._items:
            raise IndexError("peek of empty stack")
        return self._items[-1]
```

Fifteen lines of code. Most of it is bookkeeping; the three operations themselves are one line each. The `__bool__` method lets you write `while stack:` instead of `while len(stack) > 0:`, which reads more naturally and is the Python idiom that shows up in both algorithms below.

Cost summary: push amortized $O(1)$, pop and peek strict $O(1)$, space $O(n)$ with a small constant factor (just the values plus the array's overhead, no per-element pointer cost). The amortization comes from the underlying dynamic array — every so often a push triggers a doubling resize that copies the whole stack, but those resizes are rare enough that the per-push average stays bounded.

```
from codex.structures.stack import ArrayStack

stack = ArrayStack[int]()
for x in [1, 2, 3, 4, 5]:
    stack.push(x)

print(f"top: {stack.peek()}")           # top: 5 - most recent push
print(f"size: {len(stack)}")            # size: 5

popped = []
while stack:
    popped.append(stack.pop())
print(popped)                            # [5, 4, 3, 2, 1] - reverse of push order
```

```
top: 5
size: 5
[5, 4, 3, 2, 1]
```

The pop order is the push order reversed. That's LIFO made concrete: the last thing in is the first thing out, every time.

10.3 Linked-backed: strict $O(1)$, paid in pointers

The linked-list-backed stack reuses `Node[T]` from chapter 9. Push prepends a new node at the head of the chain; pop unhooks the head and returns its value. Both are strict $O(1)$: no amortization, no resize events, no doubling. The cost is one extra pointer per element (the node holds both a value and a `next` pointer) and one heap allocation per push (each node has to be constructed individually).

```
from codex.structures.linked import Node


class LinkedStack[T]:
    def __init__(self) -> None:
        self._top: Node[T] | None = None
        self._size = 0

    def __len__(self) -> int:
```

```
        return self._size

    def __bool__(self) -> bool:
        return self._top is not None

    def push(self, value: T) -> None:
        self._top = Node(value, self._top)
        self._size += 1

    def pop(self) -> T:
        if self._top is None:
            raise IndexError("pop from empty stack")
        value = self._top.value
        self._top = self._top.next
        self._size -= 1
        return value

    def peek(self) -> T:
        if self._top is None:
            raise IndexError("peek of empty stack")
        return self._top.value
```

Twenty lines, same contract, different cost shape. The construction is so similar to the array version that the differences are only visible in the per-operation cost analysis: `push` here is strict $O(1)$ instead of amortized, but every push allocates a node, so the constant factor on each push is higher. In CPython that constant factor is large enough that the array version is faster for almost every realistic workload. You'd reach for `LinkedStack` mostly when you cared about a *predictable* per-push cost rather than a low *average*.

```
from codex.structures.stack import LinkedStack

stack = LinkedStack[str]()
for word in ["red", "green", "blue", "yellow"]:
    stack.push(word)

print(f"top: {stack.peek()}")          # top: yellow
popped = []
while stack:
    popped.append(stack.pop())
print(popped)                          # ['yellow', 'blue', 'green', 'red']
```

```
top: yellow
['yellow', 'blue', 'green', 'red']
```

Same contract, same behavior, different layout under the hood. Below this layer everything looks identical; above it, both implementations are interchangeable from any caller's point of view. That interchangeability is the *point* of defining the contract first — you can swap one for the other based on workload without touching the algorithm that consumes it.

10.4 Evaluating postfix with one stack

Reverse Polish notation — RPN, or postfix — writes operators *after* their operands, with no need for parentheses to express grouping. `3 + 4` becomes `3 4 +`. `(1 + 2) * 3` becomes `1 2 + 3 *`. The order of evaluation is implicit in the order of the tokens, and the algorithm to evaluate an RPN expression is one of the shortest interesting programs in this book.

Walk the tokens left to right. When you see a number, push it. When you see an operator, pop two values, apply the operator, push the result. At the end of the input there's exactly one value on the stack — the answer.

```
import operator
from codex.structures.stack import ArrayStack


_OPERATORS = {
    "+": operator.add,
    "-": operator.sub,
    "*": operator.mul,
    "/": operator.truediv,
}


def evaluate_rpn(tokens: list[str]) -> float:
    stack: ArrayStack[float] = ArrayStack()
    for token in tokens:
        if token in _OPERATORS:
            b = stack.pop()
            a = stack.pop()
            stack.push(_OPERATORS[token](a, b))
        else:
            stack.push(float(token))
    return stack.pop()
```

The order of the two pops matters. The first pop is the *right* operand (the more recently pushed of the two); the second is the *left*. Get that order wrong and subtraction and division will silently compute the wrong thing.

```
from codex.expressions.shunting import evaluate_rpn

print(evaluate_rpn(["3", "4", "+"]))             # 7.0 - 3 + 4
print(evaluate_rpn(["3", "4", "+", "2", "*"]))   # 14.0 - (3 + 4) * 2
print(evaluate_rpn(["10", "2", "/"]))            # 5.0 - 10 / 2, not 2 / 10
print(evaluate_rpn(["1", "2", "+", "3", "4", "+", "*"]))  # 21.0 - (1+2) * (3+4)
```

```
7.0
14.0
5.0
21.0
```

Notice that there's no parsing of parentheses anywhere in this code — they're not in the input. RPN doesn't need them because the order of evaluation is already encoded in the order of the operators. That's its whole point: it's a notation designed to be cheap to evaluate, at the cost of being awkward for humans to write.

Which raises the obvious question: how do you take an expression that *was* written in infix — the way humans write math — and convert it to postfix, so you can hand it to `evaluate_rpn`?

10.5 Shunting Yard: infix to postfix

Edsger Dijkstra's Shunting Yard algorithm, published in 1961, does exactly that conversion in a single pass through the input. The name is from the railway image: think of the input expression as a train of tokens arriving on one track, with a side track (the operator stack) where operators get temporarily parked while their precedence relative to incoming operators gets resolved. The output is a second train — the postfix expression — that comes out of the yard in the right order to feed straight into `evaluate_rpn`.

The algorithm has three rules. First, operands (numbers) flow straight through to the output. Second, operators get pushed onto the operator stack, but *first* you pop off any operators already on the stack that have higher (or equal, for left-associative ops) precedence than the incoming one. Third, parentheses act as boundaries: a `(` gets pushed; a `)` pops the operator stack until it finds the matching `(`, discards the `(`, and continues.

The precedence rules for the four operators this implementation supports:

```
def _precedence(op: str) -> int:
    if op in ("+", "-"):
        return 1
    if op in ("*", "/"):
        return 2
    return 0  # parentheses get the lowest precedence so they're never popped early
```

A small helper, factored out for readability — without it, the main algorithm would have the precedence table inline twice and the comparison would be harder to read.

And the algorithm itself:

```
def shunting_yard(tokens: list[str]) -> list[str]:
    output: list[str] = []
    operators: ArrayStack[str] = ArrayStack()

    for token in tokens:
        if token in ("+", "-", "*", "/"):
            while (operators and operators.peek() != "("
                   and _precedence(operators.peek()) >= _precedence(token)):
                output.append(operators.pop())
            operators.push(token)
        elif token == "(":
            operators.push(token)
        elif token == ")":
            while operators.peek() != "(":
                output.append(operators.pop())
            operators.pop()  # discard the matching '('
        else:
            output.append(token)  # operand - flows straight to output

    while operators:
        output.append(operators.pop())

    return output
```

The invariant worth holding: at every moment during the pass, the operator stack contains operators in *strictly increasing* order of precedence from bottom to top (with `(` markers acting as fresh starting points). That's why an incoming operator triggers pops of everything at or above its own precedence — to restore the invariant before pushing itself on top. The same invariant explains why a `)` triggers pops until the matching `(`: it's flushing every operator that was deferred inside those parentheses, since their resolution is now forced.

```
from codex.expressions.shunting import shunting_yard, evaluate_rpn

# Simple precedence: * binds tighter than +
postfix = shunting_yard(["3", "+", "4", "*", "2"])
print(postfix)                      # ['3', '4', '2', '*', '+']
print(evaluate_rpn(postfix))        # 11.0 - 3 + (4*2)

# Parentheses override precedence
postfix = shunting_yard(["(", "3", "+", "4", ")", "*", "2"])
print(postfix)                      # ['3', '4', '+', '2', '*']
print(evaluate_rpn(postfix))        # 14.0 - (3+4) * 2

# Nested and chained
postfix = shunting_yard(["(", "1", "+", "2", ")", "*", "(", "3", "+", "4", ")"])
print(postfix)                      # ['1', '2', '+', '3', '4', '+', '*']
print(evaluate_rpn(postfix))        # 21.0
```

```
['3', '4', '2', '*', '+']
11.0
['3', '4', '+', '2', '*']
14.0
['1', '2', '+', '3', '4', '+', '*']
21.0
```

The conversion and the evaluation are two separate passes, but the data structure that carries the state between them is the same: a stack.

The operator stack at the end of `shunting_yard` is always empty (every operator that went in has come out), and the operand stack at the end of `evaluate_rpn` always has exactly one value (the answer). Those two "exactly empty" / "exactly one" conditions are the invariants that say the expression was well-formed; if either fails, the input had a syntax error.

This algorithm is what every real calculator and most real expression-evaluation systems still use, sixty-plus years after Dijkstra published it.

10.6 The three questions, applied

Is it correct?

The stack contract itself is correct trivially — push, pop, and peek do exactly what their names say, and the LIFO invariant ("the value returned by pop is the value most recently pushed and not yet popped") is preserved by every operation. Both implementations satisfy it: the array version because `list.append` and `list.pop` are last-in-first-out by construction, and the linked version because the head of the chain is the most recently prepended node.

`evaluate_rpn` is correct because of an inductive invariant: at every point in the pass, the stack contains exactly the partial results not yet consumed by an operator, in left-to-right order. When the input is exhausted, that invariant says there's one value on the stack iff the expression was well-formed, and that value is the answer.

`shunting_yard` is correct because of the precedence-ordering invariant on the operator stack mentioned above. Combined with the rule that a `)` flushes everything back to the matching `(`, that invariant guarantees the operators come out in the order RPN needs.

How efficient is it?

Per stack operation: push and pop are amortized $O(1)$ on `ArrayStack`, strict $O(1)$ on `LinkedStack`. Peek is $O(1)$ either way. Space is $\Theta(n)$ for a stack of n elements, with `LinkedStack` paying roughly twice as much per element because of the per-node pointer overhead.

`evaluate_rpn` runs in $\Theta(n)$ time and $O(n)$ space (the stack can grow to half the input length in the worst case, when every other token is a number with no operator between them). Every token is processed exactly once.

`shunting_yard` also runs in $\Theta(n)$ time and $O(n)$ space. The amortization argument: every operator token is pushed exactly once and popped exactly once across the whole pass, so the total work done by the inner `while` loops is bounded by the total number of operators, which is $\leq n$. Single-pass linear time, despite the loops-inside-loop shape of the code.

Is it optimal?

For the stack itself, yes — $O(1)$ per operation is the trivial lower bound (you have to at least write the value to memory), and both implementations match it. For RPN evaluation, $\Omega(n)$ is the lower bound (you have to read every token), and `evaluate_rpn` matches it at $\Theta(n)$. For Shunting Yard, the same argument: $\Omega(n)$ to read the input, $\Theta(n)$ achieved. None of the three has room for asymptotic improvement.

The deeper optimality claim is structural: any algorithm that needs to resolve operator precedence in a single left-to-right pass *needs* a stack. You can't know whether a deferred operator will be popped, or whether it'll be followed by something higher-precedence, until you've looked further into the input. A stack is exactly the data

structure that lets you defer a sequence of decisions and resolve them in reverse order when the resolution moment arrives. That's not a coincidence about this problem; it's a property of any *bracket-matching*, *delimiter-tracking*, or *deferred-resolution* problem.

10.7 Restriction is the point

A data structure's value can sit in what it *forbids*, not just in what it provides. A stack is an array with most of the operations turned off, and that restriction is what lets you reason about deferred-resolution problems cleanly. The same pattern shows up again in chapter 11 (a queue is the same restriction applied to the opposite end), and in chapter 14 (union-find is a partition with the only operations being "merge two pieces" and "ask which piece this element is in"). Each wins by removing operations the underlying storage would otherwise provide.

The stack itself is correct because LIFO is preserved by construction in both backings. It runs in $O(1)$ per operation (amortized for the array backing, strict for the linked) and $\Theta(n)$ for both `evaluate_rpn` and `shunting_yard`. It's optimal for everything it's asked to do, because the problems themselves are linear-lower-bounded.

10.8 Notes and further reading

The stack as a primitive data type goes back to Alan Turing's 1946 ACE design and to Klaus Samelson and Friedrich Bauer's 1957 "Sequential Formula Translation" paper, which is also where the Shunting Yard algorithm has its closest ancestor. Dijkstra's 1961 paper "Algol-60 Translation" formalized the version implemented here, and CLRS (4th ed.) §10.1 gives the modern textbook treatment of the array-backed implementation. Sedgewick & Wayne's *Algorithms* (4th ed.) §1.3 covers both implementations and the precedence-aware evaluation pattern. The RPN notation itself is from Jan Łukasiewicz's 1924 "Polish notation" — the "reverse" variant is the form that pops cleanly off a stack and is the form used in essentially every calculator from the HP-35 onward.

11 Pulling from the other end

A queue is what you get when you invert the stack's discipline. Where a stack restricts both input and output to the same end of a sequence — last in, first out — a queue restricts input to one end and output to the *other*. First in, first out, like a line at a coffee shop. That single change makes queues the natural shape for breadth-first graph traversal (chapter 5 of Part V), for scheduling problems where fairness over time matters, and for any online algorithm where work has to come out in the order it came in.

Three queue-like structures share the chapter — a circular-buffer queue, a linked queue, and a doubly-linked deque. The first holds **two pointers chasing each other around a fixed-size buffer**; its entire FIFO discipline lives in two modular-arithmetic operations. The third does double duty: used as a **monotonic deque**, it turns the $\Theta(nk)$ sliding-window-maximum problem into $\Theta(n)$, with the deque holding a pruned set of historical candidates rather than a passive sequence of values. That second role is what earns this chapter its keep beyond what stacks already taught you.

11.1 FIFO, and why the obvious implementation is wrong

The naive way to build a queue is to use a Python list and have `enqueue` `append` to one end and `dequeue` `pop(0)` from the other. That works — the FIFO discipline holds — but `pop(0)` on a list is $\Theta(n)$, because every element after position 0 has to shift left by one slot. So the obvious implementation has $O(1)$ enqueue and $\Theta(n)$ dequeue, which means n enqueues followed by n dequeues costs $\Theta(n^2)$. That's the same shape the dynamic array dodged by growing instead of shifting.

The fix follows the same pattern: stop moving elements around, and start moving the *bookkeeping* instead. There are two ways to do that. The first uses a fixed-size array with two indices that wrap around — a *circular buffer*. The second uses a linked list with explicit head and tail pointers. Both give you $O(1)$ enqueue and $O(1)$ dequeue. They differ in memory layout and in whether the structure has a maximum capacity.

11.2 Two pointers chasing each other around a ring

Imagine an array of fixed capacity laid out on a circle instead of a line: the slot after the last is the slot before the first. Keep two indices: `head` (the next slot to dequeue from) and `tail` (the next slot to enqueue into). Enqueue writes to `tail` and advances `tail` by one, modulo capacity. Dequeue reads from `head` and advances `head` by one, modulo capacity. Both indices wrap around the end of the array; the queue's data occupies the (possibly wrapped) range from `head` to `tail`.

The subtlety is distinguishing *empty* from *full*. Both states have `head == tail`. The textbook fix is to track size separately, an explicit counter that increments on enqueue and decrements on dequeue. With that counter, empty is `size == 0` and full is `size == capacity`, and the indices can be equal in either case without ambiguity.

```
class CircularBuffer[T]:
    def __init__(self, capacity: int) -> None:
        self._items: list[T | None] = [None] * capacity
        self._capacity = capacity
        self._head = 0
        self._tail = 0
        self._size = 0

    def __len__(self) -> int:
        return self._size

    def __bool__(self) -> bool:
        return self._size > 0
```

```
    def is_full(self) -> bool:
        return self._size == self._capacity
```

The constructor allocates one block of n slots and tracks three integers. The whole structure is the array plus those three numbers (`_capacity`, `_head`, `_tail`, `_size`) and nothing else. No allocation happens at enqueue or dequeue time.

```
    def enqueue(self, value: T) -> None:
        if self._size == self._capacity:
            raise OverflowError("circular buffer is full")
        self._items[self._tail] = value
        self._tail = (self._tail + 1) % self._capacity
        self._size += 1

    def dequeue(self) -> T:
        if self._size == 0:
            raise IndexError("dequeue from empty buffer")
        value = self._items[self._head]
        self._items[self._head] = None
        self._head = (self._head + 1) % self._capacity
        self._size -= 1
        return value  # type: ignore[return-value]
```

Two operations, each three writes and one modular addition. The `% capacity` is what turns the linear array into a circle: when an index hits `capacity`, it wraps back to 0 instead of overrunning.

```
from codex.structures.queue import CircularBuffer

buf = CircularBuffer[int](capacity=5)
for x in [10, 20, 30, 40, 50]:
    buf.enqueue(x)

print(f"size: {len(buf)}, full: {buf.is_full()}")    # size: 5, full: True
popped = []
while buf:
    popped.append(buf.dequeue())
print(popped)                                         # [10, 20, 30, 40, 50] - FIFO
```

```
size: 5, full: True
[10, 20, 30, 40, 50]
```

Same order in, same order out. Now the wrap-around case, which is what justifies the modular arithmetic:

```
buf = CircularBuffer[int](capacity=4)
buf.enqueue(1); buf.enqueue(2); buf.enqueue(3)        # head=0, tail=3
buf.dequeue(); buf.dequeue()                          # head=2, tail=3 - drained two
buf.enqueue(4); buf.enqueue(5); buf.enqueue(6)        # tail wraps from 3 → 0 → 1 → 2

print(f"head idx: {buf._head}, tail idx: {buf._tail}, size: {len(buf)}")
# head idx: 2, tail idx: 2, size: 4 - head meets tail, buffer is full but indices match

popped = []
while buf:
    popped.append(buf.dequeue())
print(popped)                                         # [3, 4, 5, 6] - FIFO preserved across wrap
```

```
head idx: 2, tail idx: 2, size: 4
[3, 4, 5, 6]
```

The data is laid out across positions 2, 3, 0, 1 in the underlying array — a wrapped span — but the FIFO discipline is preserved exactly. That `head idx: 2, tail idx: 2` row is the case the size counter exists for: without it, you couldn't tell whether the buffer was empty or full.

The structure has one limitation worth naming: the capacity is fixed at construction. If you want to enqueue past it, you either get the `OverflowError` above or you add the same doubling logic from chapter 8 to grow the underlying array. A *growable* circular buffer is what CPython's `collections.deque` is built on. This version stays fixed-size on purpose. The doubling discipline is well-trodden ground at this point; the geometric trick is the new idea.

11.3 Unbounded, at a node per enqueue

The linked variant doesn't have a capacity limit and never needs to resize. The structure tracks head and tail pointers; enqueue appends to tail, dequeue unhooks from head. Both operations rewrite two pointers each, with no `%` and no fixed allocation.

```
from codex.structures.linked import Node


class LinkedQueue[T]:
    def __init__(self) -> None:
        self._head: Node[T] | None = None
        self._tail: Node[T] | None = None
        self._size = 0

    def __len__(self) -> int:
        return self._size

    def __bool__(self) -> bool:
        return self._head is not None

    def enqueue(self, value: T) -> None:
        node = Node(value)
        if self._tail is None:
            self._head = node
        else:
            self._tail.next = node
        self._tail = node
        self._size += 1

    def dequeue(self) -> T:
        if self._head is None:
            raise IndexError("dequeue from empty queue")
        value = self._head.value
        self._head = self._head.next
        if self._head is None:
            self._tail = None
        self._size -= 1
        return value
```

Notice the edge case in `dequeue`: when the head was the last node, you have to null out the tail too, otherwise the next enqueue would attach to a dangling tail pointer. That's the edge case linked-list code is full of, and the kind of bug tests on steady-state inputs miss.

```
from codex.structures.queue import LinkedQueue

q = LinkedQueue[str]()
for word in ["alpha", "beta", "gamma", "delta"]:
    q.enqueue(word)
```

```
popped = []
while q:
    popped.append(q.dequeue())
print(popped)                                    # ['alpha', 'beta', 'gamma', 'delta']
```

```
['alpha', 'beta', 'gamma', 'delta']
```

Same contract as `CircularBuffer`, different cost shape. Strict $O(1)$ per operation, unbounded capacity, but every enqueue allocates a node, so per-operation memory churn is higher than the circular buffer's, which never allocates after construction. The trade is the same one between `ArrayStack` and `LinkedStack` in the last chapter, just applied to the queue contract instead of the stack contract.

11.4 Both ends, equally cheap

A *deque* (pronounced "deck," short for "double-ended queue") relaxes the queue's restriction by letting you push and pop at *both* ends. Now you have four operations (`push_front`, `push_back`, `pop_front`, `pop_back`), each $O(1)$. The deque generalizes both the stack (use one end only) and the queue (push at one end, pop at the other). It also lights up some algorithms that don't fit cleanly into either, including the monotonic deque from the next section.

The natural backing is a doubly linked list (chapter 9's `DoublyLinkedNode`), because the deque needs to move backward from the tail to delete the last element, which a singly linked list can't do in $O(1)$.

```
from codex.structures.linked import DoublyLinkedNode


class Deque[T]:
    def __init__(self) -> None:
        self._head: DoublyLinkedNode[T] | None = None
        self._tail: DoublyLinkedNode[T] | None = None
        self._size = 0

    def __len__(self) -> int:
        return self._size

    def __bool__(self) -> bool:
        return self._head is not None
```

The four push/pop operations mirror each other across the front/back axis. Each one handles two cases: empty deque (both head and tail need to be set) and non-empty deque (just one pointer rewrite at the relevant end).

```
    def push_front(self, value: T) -> None:
        node = DoublyLinkedNode(value, prev=None, next=self._head)
        if self._head is None:
            self._tail = node
        else:
            self._head.prev = node
        self._head = node
        self._size += 1

    def push_back(self, value: T) -> None:
        node = DoublyLinkedNode(value, prev=self._tail, next=None)
        if self._tail is None:
            self._head = node
        else:
            self._tail.next = node
        self._tail = node
```

```
        self._size += 1

    def pop_front(self) -> T:
        if self._head is None:
            raise IndexError("pop_front from empty deque")
        node = self._head
        self._head = node.next
        if self._head is None:
            self._tail = None
        else:
            self._head.prev = None
        self._size -= 1
        return node.value

    def pop_back(self) -> T:
        if self._tail is None:
            raise IndexError("pop_back from empty deque")
        node = self._tail
        self._tail = node.prev
        if self._tail is None:
            self._head = None
        else:
            self._tail.next = None
        self._size -= 1
        return node.value

    def peek_front(self) -> T:
        if self._head is None:
            raise IndexError("peek_front of empty deque")
        return self._head.value

    def peek_back(self) -> T:
        if self._tail is None:
            raise IndexError("peek_back of empty deque")
        return self._tail.value
```

Six methods, all $O(1)$, all roughly the same shape. The implementations look like a lot of code, but that's because Python is verbose about generic pointer manipulation — each operation does at most three pointer writes plus the size decrement.

```
from codex.structures.queue import Deque

dq = Deque[int]()
dq.push_back(1); dq.push_back(2); dq.push_back(3)
dq.push_front(0)
print(f"front={dq.peek_front()}, back={dq.peek_back()}, size={len(dq)}")
# front=0, back=3, size=4

print(dq.pop_back())                                # 3 - most recent push_back
print(dq.pop_front())                               # 0 - most recent push_front
print([dq.pop_front() for _ in range(len(dq))])     # [1, 2] - remaining FIFO
```

```
front=0, back=3, size=4
3
0
[1, 2]
```

Both ends are equally cheap. That symmetry is what enables the monotonic deque.

11.5 The monotonic deque, working harder

Here's a problem: given an array of n numbers and a window size k, return the maximum within each of the $n - k + 1$ contiguous length-k windows. For example, for `[1, 3, -1, -3, 5, 3, 6, 7]` with `k = 3`, the windows are `[1,3,-1]`, `[3,-1,-3]`, `[-1,-3,5]`, `[-3,5,3]`, `[5,3,6]`, `[3,6,7]`, and the maxima are `[3, 3, 5, 5, 6, 7]`.

The naive algorithm is $\Theta(nk)$: for each window, scan all k elements and find the max. For small k that's fine. For large k — or for streaming applications where the window slides one element at a time across a long sequence — the redundancy hurts. Each window shares $k - 1$ elements with the next; the naive algorithm doesn't exploit that overlap.

The monotonic-deque approach does. The idea is to maintain a deque of *indices* (not values) with two invariants. First, the indices in the deque are in increasing order (oldest at the front, newest at the back). Second, the values at those indices are in strictly *decreasing* order (so the front of the deque always holds the index of the current window's maximum). Every time the window slides one step right, you do three things: kick out the front index if it just left the window, kick out any back indices whose values are dominated by the new element, and push the new index at the back. After each slide, peek at the front to read off the answer.

Why the second invariant holds: at any moment, if some older element a has value smaller than (or equal to) a newer element b that's still in the window, then a can never again be the maximum of any future window: b dominates it. So when you push b, you can safely throw away every dominated a on the way in. The deque ends up holding exactly the *candidates* for being the maximum in the current or some future window. Every other index has been pruned.

```
from typing import Sequence


def monotonic_window_max(items: Sequence[int], k: int) -> list[int]:
    result: list[int] = []
    dq: Deque[int] = Deque()  # holds indices
    for i, x in enumerate(items):
        # drop indices that have just slid out of the window
        while dq and dq.peek_front() <= i - k:
            dq.pop_front()
        # drop back-indices whose values are dominated by the new one
        while dq and items[dq.peek_back()] <= x:
            dq.pop_back()
        dq.push_back(i)
        if i >= k - 1:
            result.append(items[dq.peek_front()])
    return result
```

The amortization argument is the move that makes the analysis work. Each index i is pushed onto the deque exactly once and popped at most once across the whole pass. The inner `while` loops can each iterate many times in a single outer step. But those iterations are *paying down* pushes that happened in earlier outer steps, not doing new work. Total work across the whole pass: $\Theta(n)$ pushes, $\Theta(n)$ pops, $\Theta(n)$ result appends. Linear in n, independent of k.

```
from codex.structures.queue import monotonic_window_max

items = [1, 3, -1, -3, 5, 3, 6, 7]
print(monotonic_window_max(items, k=3))               # [3, 3, 5, 5, 6, 7]

# Naive verification on a small random case
import random
random.seed(42)
items = [random.randint(0, 100) for _ in range(20)]
k = 5
fast = monotonic_window_max(items, k)
slow = [max(items[i:i + k]) for i in range(len(items) - k + 1)]
print(f"fast and slow agree: {fast == slow}")        # True
```

```
[3, 3, 5, 5, 6, 7]
fast and slow agree: True
```

The fast version and the naive version produce the same answers, every time. That's the verification a `{python}` chunk gives you — not a static assertion, but a comparison against the obvious-but-slow algorithm, run at render time. If a future change breaks the monotonic invariant, the comparison row prints `False` and the chapter visibly fails to render correctly.

This pattern — the deque as a *sliding evidence pruner* — generalizes. The same idea appears in chapter 9 of Part V (sliding-window shortest path with a deque holding candidate predecessors), and in chapter 5 of Part VI (DP with a deque as the dominated-state filter). Wherever you have a window moving across a sequence and a question that asks for "the best item in the current window" by some monotone criterion, the monotonic deque turns $\Theta(nk)$ into $\Theta(n)$.

11.6 The three questions, applied

Is it correct?

The three queue structures all preserve FIFO by construction. The `CircularBuffer` is correct because the modular arithmetic on `_head` and `_tail` exactly tracks where the active window of the array starts and ends, and the size counter unambiguously distinguishes empty from full. The `LinkedQueue` is correct because the head and tail pointers always identify the oldest and newest unprocessed nodes respectively, with the empty case handled explicitly. The `Deque` is correct because each of the four push/pop operations preserves the invariant that head and tail are either both `None` (empty) or both pointing at the boundary nodes of a non-empty chain.

`monotonic_window_max` is correct because of the two-invariant argument: at every step, the deque holds exactly the candidate indices for being the maximum in the current or any future overlapping window, in increasing order of index and strictly decreasing order of value. The front of the deque is therefore always the current window's maximum, modulo the bookkeeping that drops indices as they slide out.

How efficient is it?

Per operation: enqueue and dequeue on `CircularBuffer` are strict $O(1)$ (no allocation, no resize). On `LinkedQueue` they're also strict $O(1)$, but with a heap allocation per enqueue. On `Deque` all six operations are strict $O(1)$.

`monotonic_window_max` runs in $\Theta(n)$ time and $O(k)$ space — the deque can hold at most k candidate indices at any moment, since any candidate older than k steps would be outside the current window and would have been popped. Independent of k in the time bound; that's the win over the naive $\Theta(nk)$ algorithm.

Is it optimal?

For the basic queue and deque operations, yes — $O(1)$ matches the trivial write-the-value-to-memory lower bound. For sliding-window maximum, $\Omega(n)$ is the lower bound (you have to read every element of the input), and the monotonic-deque algorithm achieves $\Theta(n)$. The constant factor is worse than the naive algorithm for small k — there's overhead from the deque operations that the naive scan doesn't pay — so the crossover where the monotonic algorithm wins depends on the workload. In practice the crossover is around $k = 5$ or $k = 10$, well below typical streaming-window sizes.

A subtle structural optimality claim: any algorithm that answers sliding-window-max in one pass *cannot* avoid maintaining some kind of pruned-candidate set. You need to know which historical values are still in play and which have been dominated, and the monotonic deque is the smallest data structure that maintains exactly that information. Less wouldn't suffice; more would be redundant.

11.7 A data structure can be more interesting than its surface contract

The four structures in this chapter (circular buffer, linked queue, deque, monotonic deque) are variations on the same theme: a sequence with restrictions on where you can read and where you can write. Each restriction buys a specific algorithm. BFS works because the queue hands back the oldest unvisited node first. An LRU cache works because the deque lets you yank the most-recently-used entry to the front in one pointer move. The monotonic-deque pruning is what turns a quadratic sliding-window problem into a linear one.

A deque is "obviously" useful as a generalized queue. The monotonic-deque pattern, where the deque holds a *pruned set of historical candidates*, is the non-obvious move: same data structure, completely different role. The same shift shows up for hash tables (where the dictionary contract isn't where the interesting algorithms live), for stacks (where stacks-as-deferred-decisions is the real insight), and for heaps in Part III (where the heap is "just" a priority queue except when it's the engine behind Dijkstra's algorithm).

11.8 Notes and further reading

The circular buffer goes back to Knuth Vol. 1 §2.2.2; the head-and-tail-with-size-counter convention used here is the form CLRS (4th ed.) §10.1 calls "queue using an array." Sedgewick & Wayne's *Algorithms* (4th ed.) §1.3 covers both the array-backed and the linked variants. The monotonic deque appears in the competitive-programming literature in the 1990s and reached a wider audience through Topcoder and Codeforces; the canonical reference for the sliding-window-maximum application is Hsiu-Han Lu's note "On the Sliding Window Maximum Problem" (1996). CPython's `collections.deque` is implemented as a doubly linked list of fixed-size array blocks — a hybrid of the two backings shown here — which gives strict $O(1)$ per operation with much better cache behavior than the pure linked version.

12 Every key has an address, the function is the map

A hash table is what you get when you let the key compute its own address. You hand the function the string `"alice"` — eight bytes of letters — and it hands back the integer 47, which is a slot number in an array of 64. That single move (turning an arbitrary value into an integer in the range $\{0, 1, \ldots, m-1\}$ and using that integer as a memory location) is the chapter's biggest asymptotic lever. Every data structure you've seen so far either stored items at integer indices you chose (arrays) or at pointer-linked addresses chosen by the allocator (linked lists). The hash table chooses the address from the *content* of the item itself.

That move makes constant-time membership testing possible: *expected* constant time, with a caveat about load factors and adversarial inputs that the chapter spends its energy on. Without it, asking "is `"alice"` in this collection?" costs $O(n)$ on an unstructured array (chapter 1) or $O(\log n)$ on a sorted one (chapter 2). With it, you compute the address and look directly. The asymptotic gap is the largest single jump in the book.

Two designs handle the inevitable collisions: **chaining** (colliding keys live in a linked chain at the slot) and **open addressing** (colliding keys probe forward to a free slot, via linear, quadratic, or double hashing). Both rehash when the *load factor* — items divided by slots — crosses a threshold. Billions of possible strings, a few thousand slots: collisions are arithmetic, not bad luck, and what you do about them is the design decision the rest of the chapter explores.

12.1 What a dictionary needs to do, and how arrays can't quite

The contract is three operations: insert `(key, value)`, look up by `key`, delete by `key`. All three should be cheap — ideally constant time. Arrays alone can't do this: a naive array of pairs is $O(n)$ on lookup, a sorted array is $\Theta(n)$ on insertion because of the shifting. Neither hits all three at constant.

The hash table breaks the bottleneck by *not sorting and not searching.* It computes the address of each item from the item itself, with a deterministic function. If two items compute to the same address — a *collision* — you have to do something, and that something is the design decision the rest of the chapter explores.

12.2 Keys as addresses

A hash function h takes a key from some (potentially infinite) key-space and returns an integer in $\{0, 1, \ldots, m-1\}$, where m is the table's capacity. Three properties matter: **determinism** (same key, same output, always), **uniformity** (keys spread evenly so no slot gets disproportionate collisions), and **speed** (h runs on every operation).

Python's built-in `hash` function satisfies all three for the standard types — `str`, `int`, `tuple`, `frozenset`. Reducing its output to $\{0, \ldots, m-1\}$ is one modular operation: `hash(k) % m`. The formal "uniform hashing assumption" says keys land approximately uniformly in slots. In plain terms: if you have n items in a table of m slots, each slot holds n/m items on average. That ratio n/m is the *load factor*, the single number that controls how well the table performs.

Collisions are not pathological; they are guaranteed. Key-space (all strings, all integers) is much bigger than address-space (a few thousand slots), so by pigeonhole two keys *must* eventually land in the same slot. The two strategies for handling that give the two hash-table flavors below.

12.3 A linked list per slot

The simplest strategy: each slot holds a chain of `(key, value)` entries, all of which hashed to that slot. Insertion appends (or, to keep code short, prepends) to the chain; lookup walks the chain; deletion unhooks. Collisions don't go anywhere; they pile up in chains.

The entry type holds a key, a value, and a pointer to the next entry in its chain.

```
from dataclasses import dataclass
from typing import Iterator


@dataclass
class _Entry[K, V]:
    key: K
    value: V
    next: "_Entry[K, V] | None" = None
```

The hash map itself stores an array of chain heads, with `None` representing an empty slot. The constructor allocates that array; `_hash` computes the slot for a key.

```
class ChainedHashMap[K, V]:
    LOAD_FACTOR_MAX = 0.75

    def __init__(self, initial_capacity: int = 8) -> None:
        self._capacity = initial_capacity
        self._slots: list[_Entry[K, V] | None] = [None] * initial_capacity
        self._size = 0

    def __len__(self) -> int:
        return self._size

    def _hash(self, key: K) -> int:
        return hash(key) % self._capacity
```

`__setitem__` does insertion or update: the canonical "upsert." It walks the chain for the key's slot looking for an existing entry; if found, it updates in place; if not, it prepends a new entry and increments the size. After the insertion, if the load factor has crossed the threshold, the table doubles.

```
    def __setitem__(self, key: K, value: V) -> None:
        slot = self._hash(key)
        cur = self._slots[slot]
        while cur is not None:
            if cur.key == key:
                cur.value = value
                return
            cur = cur.next
        self._slots[slot] = _Entry(key, value, self._slots[slot])
        self._size += 1
        if self._size > self._capacity * self.LOAD_FACTOR_MAX:
            self._resize(self._capacity * 2)
```

`__getitem__` and `__contains__` both walk the chain looking for the key. The only difference is what they return on absence — a `KeyError` versus `False`.

```
    def __getitem__(self, key: K) -> V:
        cur = self._slots[self._hash(key)]
        while cur is not None:
            if cur.key == key:
```

```
                return cur.value
            cur = cur.next
        raise KeyError(key)

    def __contains__(self, key: K) -> bool:
        cur = self._slots[self._hash(key)]
        while cur is not None:
            if cur.key == key:
                return True
            cur = cur.next
        return False
```

`__delitem__` is the only operation that needs the chain's *predecessor*, to unhook the target by rewriting the predecessor's `next` pointer. Same pattern as chapter 9's `delete_after`.

```
    def __delitem__(self, key: K) -> None:
        slot = self._hash(key)
        cur = self._slots[slot]
        prev: _Entry[K, V] | None = None
        while cur is not None:
            if cur.key == key:
                if prev is None:
                    self._slots[slot] = cur.next
                else:
                    prev.next = cur.next
                self._size -= 1
                return
            prev = cur
            cur = cur.next
        raise KeyError(key)
```

Iteration walks every chain in every slot.

```
    def __iter__(self) -> Iterator[K]:
        for head in self._slots:
            cur = head
            while cur is not None:
                yield cur.key
                cur = cur.next
```

And `_resize` is the bookkeeping that grows the table. Allocate a new slot array of the requested capacity, re-insert every entry (which re-hashes against the new capacity, since `_hash` uses `self._capacity` modulus), and replace the old slot array.

```
    def _resize(self, new_capacity: int) -> None:
        old_slots = self._slots
        self._capacity = new_capacity
        self._slots = [None] * new_capacity
        self._size = 0
        for head in old_slots:
            cur = head
            while cur is not None:
                self[cur.key] = cur.value
                cur = cur.next
```

Eight methods, about fifty lines of code. A realistic-ish workload exercises the chain distribution.

```
from codex.structures.hash import ChainedHashMap

words = ["alice", "bob", "carol", "dave", "eve", "frank", "grace",
         "heidi", "ivan", "judy", "ken", "linda", "mike", "nora",
         "oscar", "peggy", "quinn", "rita", "steve", "tina"]

table = ChainedHashMap[str, int]()
for i, w in enumerate(words):
    table[w] = i

print(f"size: {len(table)}, capacity: {table._capacity}")

# Chain length per slot
chain_lens = []
for head in table._slots:
    length = 0
    cur = head
    while cur is not None:
        length += 1
        cur = cur.next
    chain_lens.append(length)
print(f"chain lengths: {chain_lens}")
print(f"max chain: {max(chain_lens)}, empty slots: {chain_lens.count(0)}")
```

```
size: 20, capacity: 32
chain lengths: [0, 0, 1, 1, 2, 0, 0, 2, 0, 1, 2, 1, 1, 0, 0, 0, 0, 1, 2, 0, 2, 1, 1, 2, 0, 0, 0, 0, 0,
max chain: 2, empty slots: 18
```

The chains stay short. Most slots hold one or zero entries; a couple hold two; nothing hits three. That's the load factor doing its job: the table doubled twice on the way to absorbing 20 items, and the final load factor is well below 1.

A quick spot-check on the lookup contract:

```
print(table["alice"])          # 0 - the first word inserted
print(table["grace"])          # 6
print("alice" in table)        # True
print("zachary" in table)      # False

del table["alice"]
print("alice" in table)        # False - deleted
print(len(table))              # 19
```

```
0
6
True
False
False
19
```

Insert, lookup, delete — the three-operation contract held up. The chains absorbed all the collisions silently.

12.4 Probing for the next free slot

Chaining is straightforward but pays a real cost: every entry sits in its own heap-allocated node, with the pointer overhead and cache-locality penalty that linked structures always carry. The alternative is to keep the entries directly inside the slot array — no separate chain nodes — and handle collisions by *probing* to a different slot whenever the natural one is occupied.

Three probing strategies, all defined here as small free functions. Each takes the primary hash h_1, the secondary hash h_2 (used by double hashing), the probe iteration i, and the table capacity. It returns the slot to inspect at probe step i.

```
def linear_probe(h1: int, h2: int, i: int, capacity: int) -> int:
    return (h1 + i) % capacity


def quadratic_probe(h1: int, h2: int, i: int, capacity: int) -> int:
    return (h1 + i * i) % capacity


def double_hash_probe(h1: int, h2: int, i: int, capacity: int) -> int:
    return (h1 + i * h2) % capacity
```

Linear probing walks one slot at a time. Quadratic spreads out: slot i^2 ahead. Double hashing strides by an amount that depends on the *key itself*, so two different keys with the same primary hash will probe different secondary sequences. Linear is the simplest to understand and suffers most from *primary clustering* (long runs of occupied slots, made worse by each new collision). Quadratic spreads better but only reaches half the table without care. Double hashing is the textbook winner: different keys travel different paths through the slot array, so clusters can't compound.

Deletion in an open-addressed table is the subtle bit. You can't just blank a slot, because a future lookup probing past it would think the chain has ended. The standard fix is a **tombstone**: a special sentinel marking the slot as "was occupied, now empty." Lookups probe past tombstones; insertions can reuse them.

`_TOMBSTONE` is the sentinel; `Probe` is the type alias for the probing-strategy functions defined just above.

```
from typing import Callable

_TOMBSTONE = object()
Probe = Callable[[int, int, int, int], int]
```

Now the hash map itself. Slots hold either `None` (never occupied), `_TOMBSTONE` (was occupied, now empty), or a `(key, value)` tuple.

```
class OpenAddressedHashMap[K, V]:
    LOAD_FACTOR_MAX = 0.5  # tighter than chaining - open addressing degrades faster

    def __init__(self, initial_capacity: int = 8, probe: Probe = linear_probe) -> None:
        self._capacity = initial_capacity
        self._slots: list[tuple[K, V] | object | None] = [None] * initial_capacity
        self._size = 0
        self._probe = probe

    def __len__(self) -> int:
        return self._size

    def _hashes(self, key: K) -> tuple[int, int]:
        h1 = hash(key) % self._capacity
        # second hash must be odd so it's coprime to a power-of-2 capacity
        h2 = hash((key, "salt")) | 1
        return h1, h2
```

The fundamental search subroutine — used by every operation — walks the probe sequence and returns either `(slot, True)` if it found the key, or `(slot, False)` with the first usable slot (a `None` or the earliest tombstone seen) if it didn't.

```
    def _find_slot(self, key: K) -> tuple[int, bool]:
        h1, h2 = self._hashes(key)
        first_tomb = -1
        for i in range(self._capacity):
            slot = self._probe(h1, h2, i, self._capacity)
            entry = self._slots[slot]
            if entry is None:
                return (first_tomb if first_tomb >= 0 else slot), False
            if entry is _TOMBSTONE:
                if first_tomb < 0:
                    first_tomb = slot
                continue
            if entry[0] == key:  # type: ignore[index]
                return slot, True
        raise RuntimeError("hash table full - should have been resized")
```

Note the "return the first tombstone on failure" detail. That's what makes deletions garbage-collect themselves over time — when you next insert a key whose probe sequence passes through the tombstone, you reclaim the slot. Without that optimization, tombstones would accumulate forever.

The four public operations are short, since `_find_slot` does all the heavy lifting.

```
    def __setitem__(self, key: K, value: V) -> None:
        slot, found = self._find_slot(key)
        if found:
            self._slots[slot] = (key, value)
            return
        self._slots[slot] = (key, value)
        self._size += 1
        if self._size > self._capacity * self.LOAD_FACTOR_MAX:
            self._resize(self._capacity * 2)

    def __getitem__(self, key: K) -> V:
        slot, found = self._find_slot(key)
        if not found:
            raise KeyError(key)
        return self._slots[slot][1]  # type: ignore[index,return-value]

    def __contains__(self, key: K) -> bool:
        _, found = self._find_slot(key)
        return found

    def __delitem__(self, key: K) -> None:
        slot, found = self._find_slot(key)
        if not found:
            raise KeyError(key)
        self._slots[slot] = _TOMBSTONE
        self._size -= 1
```

And `_resize`, which is structurally identical to the chained version — allocate, re-insert, replace.

```
    def _resize(self, new_capacity: int) -> None:
        old_slots = self._slots
        self._capacity = new_capacity
        self._slots = [None] * new_capacity
        self._size = 0
        for entry in old_slots:
            if entry is not None and entry is not _TOMBSTONE:
                k, v = entry  # type: ignore[misc]
                self[k] = v
```

Now compare the three probing strategies on the same workload. To make clustering visible, I'll insert 20 keys into a table large enough to avoid resizing, and print the slot occupation as a string.

```
from codex.structures.hash import (
    OpenAddressedHashMap,
    linear_probe,
    quadratic_probe,
    double_hash_probe,
)

words = ["alice", "bob", "carol", "dave", "eve", "frank", "grace",
         "heidi", "ivan", "judy", "ken", "linda", "mike", "nora",
         "oscar", "peggy", "quinn", "rita", "steve", "tina"]


def visualize(probe, name):
    table: OpenAddressedHashMap[str, int] = OpenAddressedHashMap(
        initial_capacity=64, probe=probe
    )
    for i, w in enumerate(words):
        table[w] = i
    occupied = "".join("X" if e is not None else "." for e in table._slots)
    runs = sorted(
        (len(s) for s in occupied.replace(".", " ").split() if s),
        reverse=True,
    )
    print(f"{name:>15}: {occupied}")
    print(f"{'':>15}  longest runs: {runs[:5]}")


visualize(linear_probe, "linear")
visualize(quadratic_probe, "quadratic")
visualize(double_hash_probe, "double hash")
```

```
         linear: ..X.X..X.........X..X.XX...........XX..X.XXXXX....XXXX.X........
                 longest runs: [5, 4, 2, 2, 1]
      quadratic: ..X.X..X.........X..X.XX...........XX..X.XXXX.X...XXXX.X........
                 longest runs: [4, 4, 2, 2, 1]
    double hash: ..X.X..X.........X..X.XX...........XX..X.XXXX.....X.XX.X...X..X.
                 longest runs: [4, 2, 2, 2, 1]
```

The longer runs in the linear row are *primary clusters* — once two keys collide, any future key hashing into the run extends it. Quadratic breaks the runs because the steps are non-adjacent. Double hashing breaks them more thoroughly because two keys colliding at h_1 almost certainly have different h_2, so they probe different secondary sequences. The pattern matches Knuth's analysis: linear's expected probe length under load α is $\frac{1}{2}\left(1 + \frac{1}{(1-\alpha)^2}\right)$, which blows up much faster than double hashing's $\frac{1}{1-\alpha}$.

12.5 Load factor is the single dial

The load factor $\alpha = n/m$ is the single parameter that controls performance. With chaining, expected chain length is α and expected operation cost is $1+\alpha$. With open addressing, expected probe length blows up as $\alpha \to 1$, faster than chaining's linear blow-up. That's why the chained `LOAD_FACTOR_MAX` is 0.75 and the open-addressed one is 0.5.

When load factor crosses the threshold, the table doubles. Every entry gets re-hashed against the new capacity (the modulus changed, so old slot assignments are stale) and re-inserted: $\Theta(n)$ work concentrated in one operation, exactly the shape that drove chapter 8's `append` analysis.

The same amortization argument carries over. Resize events happen at sizes $\Theta(1), \Theta(2), \Theta(4), \ldots, \Theta(n/2)$; total work across all resizes is $\Theta(n)$; amortized cost per insertion is $O(1)$. Individual insertions are expected $O(1)$, a vanishingly small fraction of them trigger resizes, and the amortized total stays expected $O(1)$.

```
from codex.structures.hash import ChainedHashMap


class CountingHashMap(ChainedHashMap[int, int]):
    def __init__(self, initial_capacity: int = 8) -> None:
        super().__init__(initial_capacity)
        self.resize_events: list[tuple[int, int]] = []  # (size, new_capacity)

    def _resize(self, new_capacity: int) -> None:
        self.resize_events.append((self._size, new_capacity))
        super()._resize(new_capacity)


table = CountingHashMap(initial_capacity=8)
for i in range(100):
    table[i] = i * i
print(f"final capacity: {table._capacity}, final size: {len(table)}")
print(f"resize events: {table.resize_events}")
```

```
final capacity: 256, final size: 100
resize events: [(7, 16), (13, 32), (25, 64), (49, 128), (97, 256)]
```

Five resize events to absorb 100 insertions, with capacities doubling $8 \to 16 \to 32 \to 64 \to 128 \to 256$. Log-many in the input size, exactly as the doubling discipline predicts.

The worst case the chained table can't dodge is **adversarial collisions** — keys deliberately constructed to all hash to the same slot. With CPython's default `hash` and a table of capacity 8, the integers $0, 8, 16, 24, \ldots$ all collide.

```
table: ChainedHashMap[int, int] = ChainedHashMap(initial_capacity=8)
# disable resizing so the test sees the worst case
table.LOAD_FACTOR_MAX = 999.0
keys = [8 * k for k in range(100)]
for k in keys:
    table[k] = k

# all 100 entries land in slot 0 because 8*k mod 8 == 0
chain_lens = []
for head in table._slots:
    n = 0
    cur = head
    while cur is not None:
        n += 1
        cur = cur.next
    chain_lens.append(n)
print(f"chain lengths: {chain_lens}")
print(f"slot 0 has {chain_lens[0]} entries - the table degraded to a linked list")
```

```
chain lengths: [100, 0, 0, 0, 0, 0, 0, 0]
slot 0 has 100 entries - the table degraded to a linked list
```

All 100 entries landed in slot 0. Lookups now cost $\Theta(n)$: the table degraded silently into the linked list it was supposed to outperform. Production systems defend against this with universal hashing or cryptographic hash functions. The expected $O(1)$ contract is genuine but conditional on the input being non-adversarial.

12.6 The three questions, applied

Is it correct?

Both hash maps satisfy the dictionary contract: after `table[k] = v`, the call `table[k]` returns `v`; after `del table[k]`, the call `k in table` returns `False`. The chained version is correct because `_hash(k)` deterministically locates the right chain, and the chain walk reaches every entry with that hash. The open-addressed version is correct because the probe sequence is deterministic — the same key always traces the same probe path, so `__getitem__` follows the same path `__setitem__` used to find a slot.

The tombstone rule is what makes deletion correct in the open-addressed table. Probing past a `None` correctly terminates the lookup (there's no key in this probe sequence past this point); probing past a tombstone continues (the key might still be further down the sequence). Without that distinction, deletion would break lookups for any key that probed *through* a deleted slot.

How efficient is it?

The expected costs, under the uniform hashing assumption:

Operation	Chaining	Open addressing	Notes
`__setitem__`, `__getitem__`, `__contains__`, `__delitem__`	$O(1+\alpha)$ expected	$O(\frac{1}{1-\alpha})$ expected	α = load factor
Resize	$\Theta(n)$ worst	$\Theta(n)$ worst	Amortizes to $O(1)$
Iteration	$\Theta(n+m)$	$\Theta(m)$	m = capacity

With the chaining table's load factor capped at 0.75, expected operation cost is $O(1.75) = O(1)$. With open addressing capped at 0.5, expected cost is $O(2) = O(1)$. Both are constant in the amortized expected sense.

The space cost is roughly $\Theta(n)$ for both, with a small multiplicative constant. Chaining pays extra for per-entry pointer overhead. Open addressing pays extra for the empty slots — at 50% load factor, half the slot array is empty by construction.

The worst case for both is $\Theta(n)$ per operation, when adversarial inputs force every key to the same slot (chaining) or to the same probe sequence (open addressing). That worst case is rare in practice but exploitable under certain conditions — see the universal hashing pointer below.

Is it optimal?

For point lookup, *yes in expectation.* Any algorithm that supports membership testing has to read at least the key it's looking for, so $\Omega(1)$ amortized is the trivial lower bound on operations that touch a single key, and hash tables hit it. No comparison-based structure can do better than $\Theta(\log n)$ for sorted lookup. The hash table beats that by escaping the comparison model entirely. Linear-time sorting in chapter 6 made the same move.

What hash tables *cannot* do is range queries: "give me all keys between `"alice"` and `"frank"`." The hash function scrambles key order on purpose; adjacency in the slot array has no relationship to adjacency in the key space. For that workload you need a *balanced search tree*, which is where Part III picks up. Trees give you $\Theta(\log n)$ per operation, worse than the hash table's $\Theta(1)$, but they preserve key order and support range queries that hash tables fundamentally can't.

12.7 Escaping the comparison model

Every Part I sorting and search algorithm operated by comparing items. Hash tables don't. They compute on key content, treat the result as an address, skip the comparison-tree lower bound. The same move underlies counting and radix sort from chapter 6, and returns in Part IV (hashing-based pattern matching) and Part IX (Bloom filters revisited, Count-Min sketch, HyperLogLog).

Hash tables convert the dictionary contract into expected $\Theta(1)$ per operation. Chaining handles collisions with per-slot linked chains; open addressing handles them by probing. Both keep load factor below a threshold via doubling-and-rehashing, amortizing the rebuild cost to $O(1)$ per insertion. The expected-constant promise is genuine but conditional on uniform hashing, and breaks under adversarial inputs.

12.8 Notes and further reading

The hash-table construction is CLRS (4th ed.) §11, which covers both chaining and open addressing in detail and develops the universal hashing framework that defends against adversarial inputs (§11.3.3). Sedgewick & Wayne's *Algorithms* (4th ed.) §3.4 covers the same material with a different sequencing emphasis — chaining first, then linear probing, with quadratic and double hashing relegated to exercises. Knuth's *The Art of Computer Programming* Vol. 3 §6.4 has the canonical analysis of open-addressing probe-length expectations under uniform hashing. The hash-flooding attack — the practical security failure that motivates universal hashing in production systems — is documented in Crosby and Wallach's 2003 USENIX paper "Denial of Service via Algorithmic Complexity Attacks." CPython's `dict` uses open addressing with a custom probe sequence (perturbation-based) that interpolates between linear and double hashing; the source is `Objects/dictobject.c` and is worth reading once.

13 Remembering a set without remembering its members

A Bloom filter is a data structure that answers "is this element in my set?" with two possible outputs: *definitely not*, with perfect confidence, or *probably yes*, with a tunable false-positive rate. For a million-element set you can build one in about a megabyte and a half of memory, with a false-positive rate below 1%. The same set stored in a hash table from the last chapter would cost you tens of megabytes, because each entry has to hold the key itself. The Bloom filter doesn't hold the keys at all. It just holds enough information to *recognize* them when they come back.

That trade — accepting probabilistic answers in exchange for vastly less memory — is what makes Bloom filters useful in the kinds of places where memory is genuinely the bottleneck. Web crawlers use them to remember which URLs they've already fetched without storing a hash table of every URL. Databases use them to skip disk reads when a query's key is definitely not in the table. Distributed systems use them to summarize what's on each node so other nodes can avoid expensive remote lookups for keys that aren't there.

The price of compression is doubt. A Bloom filter compresses a set by losing the ability to enumerate its members; that loss is what buys the space efficiency. You can't ask "what's in you?" You can only ask "is this in you?" — and the answer has a known error rate on the side that's allowed to be wrong. A bit array of m bits plus k hash functions yields the false-positive formula $\varepsilon = (1 - e^{-kn/m})^k$, from which optimal k and m fall out in closed form.

13.1 A million keys, a megabyte of bits

A hash table from chapter 12 storing n string keys costs roughly $\Theta(n \cdot \overline{|k|})$ memory, where $\overline{|k|}$ is the average key length in bytes. For URLs that's commonly 50–200 bytes per entry, plus the per-entry pointer overhead of the hash-table backing. A million URLs is tens of megabytes. A billion URLs — the kind of set a web crawler accumulates — is tens of gigabytes, which doesn't fit in RAM on a normal machine.

A Bloom filter for the same set, at the same 1% false-positive rate, costs $\approx 1.44 \cdot n \cdot \log_2(1/\varepsilon)$ bits, independent of the key length. For a million URLs at 1% FP rate, that's $1.44 \times 10^6 \times \log_2(100) \approx 9.6$ million bits, or about 1.2 megabytes. For a billion URLs, about 1.2 gigabytes. Two orders of magnitude smaller than the hash-table equivalent. What you give up: the ability to enumerate the keys you put in, and the certainty of *positive* answers. Negatives stay definite.

No false negatives is the guarantee that lets the structure be useful as a pre-filter (*"the cache definitely doesn't have this, don't bother checking it"*) or as a one-sided certificate (*"this URL might already be in the crawler queue, schedule a real check before adding it again"*). You wrap a Bloom filter around an expensive lookup and spend constant time plus a few bits of memory to avoid the expensive call most of the time. The 1% of false positives mean you do the real lookup anyway in those rare cases.

13.2 One bit array, k hashes

The structure has two parts: a bit array of m bits (all initially zero), and k independent hash functions $h_1, h_2, \ldots, h_k$, each mapping a key to a position in $\{0, 1, \ldots, m-1\}$. To add an item x, compute all k hash positions and set those bits to 1. To check whether x is present, compute the same k positions and check whether *all* of them are 1.

If any of the k bits is 0, the answer is *definitely-not*: if x had been added, all k of its bits would have been set, and zero of them would still be zero. If all k are 1, the answer is *probably-yes*: those bits might all be 1 because x was added, or because other items collectively set the same bits. The second case is a false positive, and its probability is what the next section computes.

The "k independent hash functions" requirement is awkward in practice. You'd have to implement k separate hash functions and tune them not to be correlated. The textbook trick (Kirsch and Mitzenmacher, 2006) is that you only need *two* independent hash functions h_1 and h_2. Then $h_i(x) = h_1(x) + i \cdot h_2(x) \pmod m$ for $i = 0, 1, \ldots, k-1$ behaves indistinguishably from k independent hashes for the Bloom-filter analysis. The chapter-12 double-hashing trick, repurposed.

13.3 Why $\varepsilon = (1 - e^{-kn/m})^k$

Walk through the analysis slowly. After inserting n items, each setting k bits, the bit array has been hit by a total of nk hash positions. Assuming the hashes are independent and uniform, the probability that any *one specific* bit is *still zero* after those nk writes is:

$$\Pr[\text{bit still 0}] = \left(1 - \frac{1}{m}\right)^{nk} \approx e^{-nk/m}$$

The approximation uses $\lim_{m\to\infty}(1 - 1/m)^m = 1/e$. For Bloom filters where m is in the thousands at minimum, the gap is below $1/m$ and you can read $\approx$ as $=$.

Now ask the probability that any specific bit is 1: $1 - e^{-nk/m}$. And then the probability of a false positive — that all k bits checked by a query for an item *not* in the set come back 1, assuming the k probed positions are independent:

$$\varepsilon = \left(1 - e^{-nk/m}\right)^k$$

That's the false-positive rate. It depends on n (how many items you've added), m (how big the bit array is), and k (how many hashes per item). n is fixed by the workload; m and k are design parameters.

More hashes hurt past a point. More hashes set more bits per insertion, the array fills up faster, and the false-positive rate rises. The optimal k minimizes ε for a given n and m.

Take the derivative of ε with respect to k, set it to zero, and the optimal k falls out cleanly:

$$k^* = \frac{m}{n} \ln 2$$

A clean closed form. The two pretty consequences: at the optimal k, exactly half the bits in the array are set on average (the algorithm packs the array as fully as it can without diminishing returns), and the false-positive rate at optimal k simplifies to $\varepsilon = (1/2)^k = 2^{-k}$.

That second equation lets you solve for m given n and a target ε. Substituting k^* back in and simplifying:

$$m^* = -\frac{n \ln \varepsilon}{(\ln 2)^2}$$

Or, using $\ln 2 \approx 0.693$ and $(\ln 2)^2 \approx 0.480$:

$$m^* \approx 1.44 \cdot n \cdot \log_2(1/\varepsilon)$$

That's the formula from the chapter opener. A Bloom filter holding n items at false-positive rate ε needs about $1.44 n \log_2(1/\varepsilon)$ bits, regardless of key length. For $\varepsilon = 0.01$, that's $1.44 \times \log_2(100) \approx 9.6$ bits per item.

13.4 Thirty lines and a bytearray

The whole structure fits in about thirty lines of Python. The constructor takes the expected item count and the target false-positive rate, computes optimal m and k from the formulas above, and allocates a `bytearray` to hold the bits.

```
import math
from typing import Hashable, Iterator


class BloomFilter:
    def __init__(self, expected_items: int, fp_rate: float = 0.01) -> None:
        if expected_items <= 0 or not 0 < fp_rate < 1:
            raise ValueError("expected_items > 0 and 0 < fp_rate < 1")
        self._n_expected = expected_items
        ln2 = math.log(2)
        # m = -n ln( ) / (ln 2)^2
        m = -expected_items * math.log(fp_rate) / (ln2 ** 2)
        self._m = max(8, int(math.ceil(m)))
        # k = (m/n) ln 2
        k = (self._m / expected_items) * ln2
        self._k = max(1, int(round(k)))
        self._bits = bytearray((self._m + 7) // 8)
```

The bit array lives inside a `bytearray`, a mutable sequence of 8-bit integers. To set bit i, you compute the byte index $i//8$ and the bit-within-byte i mod 8, then OR in `1 << bit`. To test bit i, you AND with the same mask and check whether the result is nonzero. The standard bit-array idiom.

```
    def _hashes(self, item: Hashable) -> Iterator[int]:
        h1 = hash(item)
        h2 = hash((item, "salt"))
        for i in range(self._k):
            yield (h1 + i * h2) % self._m

    def add(self, item: Hashable) -> None:
        for pos in self._hashes(item):
            self._bits[pos // 8] |= 1 << (pos % 8)

    def __contains__(self, item: Hashable) -> bool:
        for pos in self._hashes(item):
            if not (self._bits[pos // 8] & (1 << (pos % 8))):
                return False
        return True
```

Three methods, three patterns. `_hashes` yields k positions via the double-hashing trick. `add` sets all k bits. `__contains__` checks all k bits; if any is zero, return *False* (definitely-not); if all are one, return *True* (probably-yes).

Build one and verify the parameter computation:

```
from codex.structures.bloom import BloomFilter

bf = BloomFilter(expected_items=10_000, fp_rate=0.01)
print(f"m = {bf._m} bits ({bf._m / 8:.0f} bytes)")
print(f"k = {bf._k}")
print(f"bits per item: {bf._m / 10_000:.2f}")
```

```
m = 95851 bits (11981 bytes)
k = 7
bits per item: 9.59
```

About 96 thousand bits (twelve kilobytes) to hold ten thousand items at 1% false-positive rate. Seven hash functions per item. The bits-per-item figure of about 9.6 matches the formula's $1.44 \log_2 100 \approx 9.6$.

Now the empirical validation. Insert ten thousand random strings, then query ten thousand *other* random strings and count how many come back as false positives.

```
import random

bf = BloomFilter(expected_items=10_000, fp_rate=0.01)

random.seed(42)
alphabet = "abcdefghijklmnopqrstuvwxyz"

inserted = set()
while len(inserted) < 10_000:
    inserted.add("".join(random.choices(alphabet, k=8)))

for s in inserted:
    bf.add(s)

# no false negatives
print(f"all inserted are present: {all(s in bf for s in inserted)}")

# count false positives over 10,000 non-members
fp_count = 0
tested = 0
while tested < 10_000:
    s = "".join(random.choices(alphabet, k=8))
    if s in inserted:
        continue
    tested += 1
    if s in bf:
        fp_count += 1

print(f"false positives: {fp_count} out of {tested} non-members")
print(f"empirical FP rate: {fp_count / tested:.4f}")
print(f"predicted FP rate: 0.0100")
```

```
all inserted are present: True
false positives: 112 out of 10000 non-members
empirical FP rate: 0.0112
predicted FP rate: 0.0100
```

Empirical rate within a fraction of a percent of the predicted 1%. The agreement isn't an accident: the analysis is exact up to the $(1 - 1/m)^m \to 1/e$ approximation, which is wrong by less than $1/m$ at these table sizes. The only other randomness is from the choice of test strings, which is exactly the regime the formula models.

Sweep the parameter and watch the formula predict reality across a range:

```
import math


def predicted_fp(n: int, m: int, k: int) -> float:
    return (1 - math.exp(-k * n / m)) ** k


for target_rate in [0.10, 0.01, 0.001]:
    bf = BloomFilter(expected_items=10_000, fp_rate=target_rate)
    for s in inserted:
        bf.add(s)
    pred = predicted_fp(10_000, bf._m, bf._k)
    print(f"target ={target_rate:6.4f}  "
          f"m={bf._m:>6}  k={bf._k}  "
          f"predicted={pred:.4f}")
```

```
target =0.1000  m= 47926  k=3  predicted=0.1007
```

```
target  =0.0100  m= 95851  k=7  predicted=0.0100
target  =0.0010  m=143776  k=10  predicted=0.0010
```

The predicted false-positive rate sits right on the target every time. The structure is parametric, not heuristic.

13.5 No deletion, no enumeration, no resize

The list of operations a Bloom filter does *not* support is part of its contract. **Deletion isn't safe.** If you clear the k bits for an item, you risk turning future *probably-yes* answers into spurious *definitely-not* answers for *other* items whose hash positions happened to overlap with the one you just deleted. The classic fix is a **counting Bloom filter**: replace each bit with a small counter (4–8 bits typically), increment on `add`, decrement on `remove`. That recovers deletion at the cost of multiplying the memory by the counter width. Mentioned here as a pointer, not implemented.

Enumeration isn't supported. There's no `for x in bf` operation, and there couldn't be — the keys themselves were never stored, and the bit pattern alone can't reconstruct them. The structure can answer "do you contain this specific thing?" but never "what do you contain?" The doubt the chapter opened on extends specifically to recovering the members.

Resizing isn't easy. If you've underestimated n, you can't just add bits: the hash positions depend on m, so the existing fingerprints become meaningless after a resize. The standard workaround is *scalable Bloom filters* (Almeida et al., 2007), which chain multiple smaller filters together with increasing m and decreasing ε. Another pointer, not built here.

13.6 The three questions, applied

Is it correct?

The Bloom filter satisfies a *one-sided* correctness condition. Every item that was added returns *probably-yes* on subsequent membership checks — the `add` method sets exactly the bits that `__contains__` checks, so all k bits will be set, and the check returns *True*. **No false negatives.** This is exact, not statistical.

The other direction is the statistical guarantee. Items that were never added can return *probably-yes* (a false positive) with probability bounded by ε, the design parameter. That bound is *expected* over the randomness of the hash functions, not worst-case — an adversary who can construct keys to exploit your specific hash functions can drive the FP rate higher, the same caveat from chapter 12.

How efficient is it?

The `add` and `__contains__` operations each run in $\Theta(k)$ time and $O(1)$ space. With $k = (m/n)\ln 2$, that's $O(\ln(1/\varepsilon))$ — proportional to the number of bits of confidence you want, not to the size of the set. For $\varepsilon = 0.01$, $k = 7$. For $\varepsilon = 0.0001$, $k = 14$. For $\varepsilon = 10^{-9}$, $k = 30$. The cost grows logarithmically in inverse FP rate; you can have very rare false positives for cheap.

Space is $\Theta(m) = \Theta(n \log(1/\varepsilon))$ bits — linear in n, logarithmic in inverse error tolerance. At $\varepsilon = 0.01$ that's about 10 bits per item. Compare with a hash table storing the keys themselves, which costs roughly $8 \times \overline{|k|}$ bits per item (where $\overline{|k|}$ is average key length in bytes). For typical key sizes the Bloom filter is one to two orders of magnitude smaller.

Is it optimal?

The information-theoretic lower bound for a structure supporting one-sided ε-error membership testing on a set of size n is $n \log_2(1/\varepsilon)$ bits. You need at least that many bits to distinguish "any of the n members" from "everything else" with error ε. The Bloom filter at optimal k achieves $1.44n \log_2(1/\varepsilon)$ bits, within a factor of $1.44 = 1/\ln 2$ of the lower bound. That constant is the cost of using k-independent hash functions; with $k = 1$ and a perfectly correlated hash, you could match the bound, but that hash function doesn't exist for arbitrary keys.

Structures that get closer to the lower bound exist — *quotient filters* (Bender et al., 2012), *cuckoo filters* (Fan et al., 2014), *Morton filters* (Breslow & Jayasena, 2018) — and they trade additional complexity (mostly around supporting deletion and resize) for a tighter space bound. For the canonical "remember a set, accept some false positives" problem, the basic Bloom filter is within a small constant of optimal, and that's the form that ships in every production caching system.

13.7 Trade certainty for space

A Bloom filter achieves its memory efficiency precisely by giving up the ability to enumerate or reconstruct what it holds. The trade isn't a bug; it's the point. Asymmetric error guarantees — definite negatives, probable positives — are what make the structure useful as a pre-filter even when the positives have a known error rate.

This pattern shows up across the probabilistic-data-structures family in Part IX. Count-Min sketch trades exactness for space when *counting* frequencies of items in a stream. HyperLogLog trades exactness for space when *counting distinct items* in a stream, getting the count of a billion-element multiset accurate to within 2% using a kilobyte of memory. Each is a different version of the same move: identify exactly what the workload needs to know, and design a structure that stores only *that*, with a known error rate on the side you're allowed to be wrong.

The Bloom filter itself is correct in the one-sided sense (no false negatives, false positives at a tunable rate), runs in $\Theta(k)$ per operation, and uses $\Theta(n \log(1/\varepsilon))$ bits, within a constant factor of the information-theoretic lower bound.

13.8 Notes and further reading

Burton Bloom's 1970 paper "Space/Time Trade-offs in Hash Coding with Allowable Errors" is the original construction; it's short, accessible, and worth reading. The optimal-parameter analysis given here is folklore but is treated rigorously in Mitzenmacher and Upfal's *Probability and Computing* §5. The double-hashing trick that lets you simulate k independent hashes from two is Kirsch and Mitzenmacher's 2006 ESA paper "Less Hashing, Same Performance: Building a Better Bloom Filter." The counting variant is Fan, Cao, Almeida, and Broder's 2000 IEEE/ACM Transactions on Networking paper "Summary Cache." For the tighter-than-Bloom alternatives, see Bender et al.'s "Don't Thrash" (quotient filters) and Fan et al.'s "Cuckoo Filter: Practically Better Than Bloom" — both are concise and implementable. CLRS doesn't cover Bloom filters; the standard textbook entry point is Sedgewick & Wayne's *Algorithms* §3.5 exercises or the Mitzenmacher-Upfal chapter cited above.

14 A forest where every element knows its root

You have a set of n elements partitioned into some number of disjoint groups, and two operations: `union(x, y)` merges the groups containing x and y, and `connected(x, y)` asks whether they're already in the same group. Sounds simple. The naive algorithm costs $O(n)$ per operation. Three optimizations bring it down to amortized $O(\alpha(n))$ — the inverse Ackermann function, at most 4 for any input you'll ever construct. The progression from naive to optimized is one of the most striking stories in algorithm design, and the destination is one of the most-used data structures in graph algorithms.

Each set is a tree: **a forest of parents**. The root of the tree names the set; every non-root element holds a pointer to its parent. `find` walks up to the root. `union` links two roots together. The whole story is about *making those trees flatter* — first by being clever about which root to link under which (union by rank), then by *retroactively* flattening the path you just walked (path compression). Together those two tricks collapse the trees so aggressively that the amortized cost per operation becomes the inverse Ackermann function. The result is a structure ready to plug into Kruskal's minimum-spanning-tree algorithm in Part V, where the union-find is the cycle-detection engine that makes the algorithm work at all.

14.1 Merging groups, one at a time

The problem: maintain a partition of $\{0, 1, \ldots, n-1\}$ that starts in n singleton sets and is gradually merged via `union` calls. At any moment, support `connected(x, y)` queries that ask whether x and y are in the same set.

The naive approach is to store each set as an explicit list of its members. `connected(x, y)` walks the list containing x and checks for y — O(set size). `union(x, y)` concatenates the two lists — O(smaller set size) if you're careful. Both operations grow expensive as sets get big.

Two cleaner approaches each fix one operation at the expense of the other.

14.2 Quick-find: representative-as-label

The first idea: give every set a *representative* — a designated element that serves as the set's name — and store each element's representative in an array. Find the representative of x in $O(1)$ by reading the array. To check connectivity, compare the two representatives in $O(1)$.

```
class QuickFind:
    def __init__(self, n: int) -> None:
        self._label = list(range(n))

    def find(self, x: int) -> int:
        return self._label[x]

    def connected(self, x: int, y: int) -> bool:
        return self._label[x] == self._label[y]
```

Initial state: element i is its own representative (the array is $[0, 1, 2, \ldots, n-1]$). The contract is that two elements are in the same set iff they have the same label.

The cost is paid by `union`. To merge the set of x into the set of y, every element currently labeled with x's representative has to be relabeled with y's — a linear sweep through the entire label array.

```
    def union(self, x: int, y: int) -> None:
        lx, ly = self._label[x], self._label[y]
        if lx == ly:
            return
        for i in range(len(self._label)):
            if self._label[i] == lx:
                self._label[i] = ly
```

$O(n)$ per `union`. That's the linear-sweep cost the data structure can't avoid: every element's representative is stored explicitly and independently, so every element has to be visited when the representative changes.

```
from codex.structures.union_find import QuickFind

uf = QuickFind(n=10)
print(uf._label)                          # [0,1,2,3,4,5,6,7,8,9] - all singletons
uf.union(0, 1); uf.union(2, 3)
uf.union(0, 2)                            # merges {0,1} and {2,3}
print(uf._label)                          # everyone in {0,1,2,3} has the same label
print(uf.connected(1, 3))                 # True
print(uf.connected(1, 4))                 # False
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
[3, 3, 3, 3, 4, 5, 6, 7, 8, 9]
True
False
```

Quick-find wins on `find` and loses on `union`. The naive partition-as-explicit-lists approach loses on both. The next variant flips the trade.

14.3 Quick-union: forest of parents

The opposite idea: instead of storing each element's *representative*, store each element's *parent*. The representative of a set is the element at the root of its parent-tree: the one whose parent is itself. To find the representative of x, walk up the parent pointers until you hit a self-loop.

```
class QuickUnion:
    def __init__(self, n: int) -> None:
        self._parent = list(range(n))

    def find(self, x: int) -> int:
        while self._parent[x] != x:
            x = self._parent[x]
        return x

    def connected(self, x: int, y: int) -> bool:
        return self.find(x) == self.find(y)
```

`union` now becomes cheap: find both roots, point one at the other. Single pointer write.

```
    def union(self, x: int, y: int) -> None:
        rx, ry = self.find(x), self.find(y)
        if rx == ry:
            return
        self._parent[rx] = ry
```

The cost has moved from `union` to `find`. Both operations now cost $O(h)$, where h is the height of the relevant tree. The catch is that h can grow as large as n in the worst case (*spaghetti trees*) if every union chains onto an already-tall tree.

A friendly union sequence wouldn't typically produce a chain — successive `union(i, i+1)` calls actually merge the trees flatly, because `union` always finds the root of each input tree before linking. The pathological shape happens when an adversarial sequence of operations always extends the same growing tree. To make the worst case visible, I'll bypass `union` and construct the chain by hand:

```
from codex.structures.union_find import QuickUnion

uf = QuickUnion(n=10)
# manually construct a spaghetti chain: 0 → 1 → 2 → 3 → 4 → 5 → 6
for i in range(6):
    uf._parent[i] = i + 1

print(uf._parent[:7])                              # [1, 2, 3, 4, 5, 6, 6]
# find(0) walks 0 → 1 → 2 → 3 → 4 → 5 → 6 - six pointer hops
print(f"root of 0: {uf.find(0)}")                  # 6
```

```
[1, 2, 3, 4, 5, 6, 6]
root of 0: 6
```

That six-hop find is what happens when an adversary chains every union onto an existing root. The data structure is still correct, but it's been linearized. Each `find` walks the chain end-to-end.

The two optimizations in the next two sections — union by rank and path compression — each attack this failure mode. Either one alone improves the asymptotic bound dramatically. Together, they push it to inverse Ackermann.

14.4 Union by rank: keeping the trees short

The first optimization is to track an upper bound on each tree's height — its **rank** — and always hang the lower-rank tree under the higher-rank root when merging. That keeps the resulting tree as short as possible.

If two trees have equal rank, the merged tree gains exactly one level (you have to pick one to be the new root), so its rank increments by 1. If their ranks differ, hanging the shorter under the taller leaves the rank unchanged — the merged tree is no taller than the taller of its two inputs was.

```
class UnionFind:
    def __init__(self, n: int) -> None:
        self._parent = list(range(n))
        self._rank = [0] * n
```

The constructor adds a parallel `_rank` array, initialized to zero (every singleton has rank 0). The `union` method becomes a three-way conditional on the rank comparison.

```
    def union(self, x: int, y: int) -> None:
        rx, ry = self.find(x), self.find(y)
        if rx == ry:
            return
        if self._rank[rx] < self._rank[ry]:
            self._parent[rx] = ry
        elif self._rank[rx] > self._rank[ry]:
            self._parent[ry] = rx
        else:
            self._parent[ry] = rx
            self._rank[rx] += 1
```

The rank-only optimization (without path compression) is enough to bound tree height by $\log_2 n$. The argument: a tree of rank r contains at least 2^r elements. That's true at rank 0 (one element) and is preserved by `union` — two trees of equal rank r merge into one of rank $r+1$ with at least $2^r + 2^r = 2^{r+1}$ elements. So a tree of rank r has at least 2^r elements. Rank is bounded by $\log_2 n$, height by $\log_2 n$, and `find` runs in $O(\log n)$.

That alone is already a dramatic win over the chain-shaped worst case. The next optimization gets you to near-constant.

14.5 Path compression: flattening as a side effect

When `find` walks from a node up to the root, it visits every node on the path. *None* of those nodes need to remain as deep as they were — they could all point directly at the root. So why not rewrite their parent pointers on the way back, after the walk has discovered the root?

The recursive implementation is a one-line trick. `find(x)` finds the root by recursing into `find(parent[x])`, and on the way back from the recursion, it writes the root into `parent[x]`. Every node on the original path ends up pointing directly at the root.

```
    def find(self, x: int) -> int:
        if self._parent[x] != x:
            self._parent[x] = self.find(self._parent[x])
        return self._parent[x]

    def connected(self, x: int, y: int) -> bool:
        return self.find(x) == self.find(y)
```

Three lines for the `find`. The first checks the base case (the node is its own parent, so it's the root). The second recursively finds the root *and* writes it into `_parent[x]`, mutating the tree on the return path. The third returns the root.

Path compression doesn't always change the *height* of the tree — the root and its direct children might be unchanged. But it flattens the *path you just walked*, and every subsequent `find` on any node along that path is now $O(1)$. The effect compounds over many operations.

Watch path compression do its thing on a deliberately-bad starting state:

```
from codex.structures.union_find import UnionFind

uf = UnionFind(n=8)
# manually construct a spaghetti tree: 7 → 6 → 5 → 4 → 3 → 2 → 1 → 0
for i in range(1, 8):
    uf._parent[i] = i - 1

print(f"before find: parent = {uf._parent}")     # [0,0,1,2,3,4,5,6]
uf.find(7)                                        # walks 7→6→5→4→3→2→1→0, compresses
print(f"after find:  parent = {uf._parent}")     # everyone on the path → 0
```

```
before find: parent = [0, 0, 1, 2, 3, 4, 5, 6]
after find:  parent = [0, 0, 0, 0, 0, 0, 0, 0]
```

One `find` call, eight pointer rewrites. After it, the tree's deep chain has collapsed to a star — every formerly-on-the-path node points directly at the root. The next seven `find` calls on any of those nodes will be $O(1)$.

The combined effect of union by rank and path compression is what gives the structure its near-constant amortized cost. Each optimization alone would still be polynomial; together they break out of polynomial entirely. Tarjan's 1975 analysis showed the combined operations have amortized cost $O(\alpha(n))$ — the inverse Ackermann function.

14.6 Inverse Ackermann is at most 4

The Ackermann function grows fast. Faster than exponential, faster than tower-of-exponentials, faster than any primitive recursive function. $A(4, 4)$ already exceeds the number of atoms in the observable universe. The *inverse* — $\alpha(n)$, the smallest m such that $A(m, m) \geq n$ — grows so slowly that $\alpha(n) \leq 4$ for any $n \leq 2^{2^{16}} = 2^{65536}$, a number with twenty thousand digits.

So when this chapter says $O(\alpha(n))$ amortized, the operational meaning is "effectively constant for any input you can construct on any computer ever built." The bound is not literally $O(1)$ — there's a real function in there — but the function is so close to a constant that the distinction has no engineering consequence.

The reason it's not literally $O(1)$ is the structural payoff. The deep result (Fredman & Saks 1989) is that $\Omega(\alpha(n))$ is the *true* lower bound for this problem in the cell-probe model: there's no implementation, however clever, that can do better in the worst case. That makes path-compression-plus-union-by-rank tight at the inverse-Ackermann level, a striking achievement for a structure built from "child points at parent."

```
import time
import random
from codex.structures.union_find import QuickFind, QuickUnion, UnionFind


def benchmark(cls, n: int, ops_count: int) -> float:
    uf = cls(n)
    random.seed(42)
    start = time.perf_counter()
    for _ in range(ops_count):
        x = random.randint(0, n - 1)
        y = random.randint(0, n - 1)
        uf.union(x, y)
    return time.perf_counter() - start


n, ops = 1000, 5000
for name, cls in [
    ("QuickFind", QuickFind),
    ("QuickUnion", QuickUnion),
    ("UnionFind", UnionFind),
]:
    t = benchmark(cls, n, ops)
    print(f"{name:>10}: {t * 1000:>7.1f} ms")
```

```
 QuickFind:    33.2 ms
QuickUnion:    34.1 ms
 UnionFind:     4.5 ms
```

The differences widen with input size. QuickFind is dominated by its $O(n)$ union cost; its union cost is the bottleneck. QuickUnion does better on random sequences that happen to produce balanced trees on average. The optimized UnionFind is fastest of all, by the amortized constant factor the inverse-Ackermann bound predicts. Run the same benchmark at $n = 10\,000$ and the gap between QuickFind and the other two becomes orders of magnitude: the practical face of amortized $O(\alpha(n))$ beating worst-case $O(n)$.

14.7 The three questions, applied

Is it correct?

All three implementations satisfy the partition contract: `connected(x, y)` returns `True` iff `x` and `y` are in the same set after the sequence of `union` calls executed so far. The shared invariant is that **every element belongs to exactly one set**, and the representative of each set is uniquely determined by the structure (the label in `QuickFind`, the root of the tree in the other two).

`QuickFind` is correct because the label array directly stores each element's set identity; the linear-sweep `union` keeps the array consistent. `QuickUnion` is correct because every element can reach its set's root by walking parents, and the root is invariant under `find`. `UnionFind` is correct because path compression preserves which root each element reaches (it just shortens the path), and union by rank preserves the parent-forest structure (it just picks the shorter tree to hang).

How efficient is it?

The cost table:

Structure	`find`	`union`	`connected`
`QuickFind`	$O(1)$	$\Theta(n)$	$O(1)$
`QuickUnion` (worst)	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
`QuickUnion` (random)	$O(\log n)$ avg	$O(\log n)$ avg	$O(\log n)$ avg
`UnionFind` (PC + rank)	$O(\alpha(n))$ amortized	$O(\alpha(n))$ amortized	$O(\alpha(n))$ amortized

Space is $\Theta(n)$ for all three — one or two arrays of n integers each.

The amortization argument for the optimized version is non-trivial — Tarjan's original proof is several pages of careful potential-function bookkeeping — but the operational claim is straightforward: across any sequence of m union and find operations on n elements, the total work is $O(m \cdot \alpha(n))$, even though individual operations can occasionally cost as much as $O(\log n)$.

Is it optimal?

Yes, in a deeply specific sense. Fredman and Saks proved in 1989 that any data structure supporting `union` and `find` on n elements must use $\Omega(\alpha(n))$ amortized time per operation in the cell-probe model — the most permissive model of computation, where you only count memory accesses. That lower bound matches Tarjan's upper bound exactly. The optimized union-find is *tight at the inverse-Ackermann level* — no implementation, however clever, can be asymptotically faster.

That tightness is unusual. Most structures in this book sit within a constant factor of their lower bound; union-find lands *on* it. The match exists because the lower bound itself is sophisticated — it took Fredman and Saks to discover that the inverse Ackermann was inevitable, not a quirk of the algorithm.

14.8 A forest, an amortization, and a bound that's almost constant

Three claims survive the chapter. **The right data structure is sometimes a forest, not a tree:** the partition is a set of trees, with the root naming each set, and the elegance of the optimizations comes from this multi-tree view. **Amortized analysis pays for self-modification:** path compression rewrites the tree as a side effect of querying it, and the rebuilding cost is paid for by the cheaper queries it enables downstream. **Near-constant time bounds aren't $O(1)$ but might as well be:** once a bound is below $\alpha(n) \leq 4$ for any realistic input, the distinction stops being engineering.

The three versions of union-find walk through the same progression: identify the bottleneck operation, restructure the data to make it cheap, watch the cost shift to the other operation, restructure again, find the optimization that makes both cheap at once. The quick-find-to-quick-union flip moves the cost from `union` to `find`. Union by rank bounds tree height by $\log n$. Path compression makes individual paths near-constant after the first walk.

In Part V, Kruskal's minimum-spanning-tree algorithm is *just* sort the edges by weight, then walk them in order, adding each edge whose endpoints aren't already in the same set. The cycle check is `connected(u, v)`. The "add this edge" step is `union(u, v)`. The whole algorithm is a wrapper around a union-find, plus a sort.

14.9 Notes and further reading

The disjoint-set / union-find data structure is treated in CLRS (4th ed.) §19, including a clean exposition of the amortized analysis. The original combined union-by-rank-plus-path-compression algorithm and its $O(m\alpha(n))$ amortization is Robert Tarjan's 1975 *Journal of the ACM* paper "Efficiency of a Good But Not Linear Set Union Algorithm." The matching lower bound is Fredman and Saks's 1989 STOC paper "The Cell Probe Complexity of Dynamic Data Structures." Sedgewick & Wayne's *Algorithms* (4th ed.) §1.5 has the cleanest pedagogical treatment of the three implementations side-by-side and is the basis for this chapter's structure. For the recursive path-compression implementation versus the iterative two-pass alternative ("path splitting" and "path halving"), see Tarjan and van Leeuwen's 1984 *JACM* paper "Worst-Case Analysis of Set Union Algorithms," which proves that all three variants give the same asymptotic bound.

Lessons from Part II

Seven data structures in seven chapters, and the surprise (to me, at least, the second time I taught this material) is how *few atoms* the whole part is built from. Two layout choices (contiguous and linked), one function (hash), and a small collection of disciplines about which operations to make cheap. That's it. The dynamic array and the linked list are the atoms. The stack, queue, deque, hash table, Bloom filter, and union-find are *compositions* of those atoms with restrictions or extensions imposed on top.

The patterns that kept showing up

The operation-versus-layout trade. Every chapter was, at its core, an answer to "which operations do I want to be cheap, and what am I willing to give up to get there?" You buy cheap indexed access with expensive prepend (chapter 8), or cheap surgery in the middle with expensive indexed access (chapter 9). You restrict an array or a chain to operating at one end (chapter 10). You make point lookup cheap by abandoning key order (chapter 12), or by abandoning the keys themselves (chapter 13). The trade isn't a footnote; it's the whole story. There is no single best sequence data structure, because the right answer depends on which operations the caller actually exercises.

Amortization as the analytical move. The accountant's view of cost — *what does this operation cost on average, over many calls, rather than in the worst case?* — appeared three times. The doubling dynamic array introduced it: seventeen rare resize spikes paying for a hundred thousand cheap appends between them. Hash-table rehashing inherited it. Union-find pushed it to its most striking conclusion, where amortized analysis was the only way to express the inverse-Ackermann bound at all. Rare, expensive, self-modifying operations subsidized by many cheap operations every time. Doubling-and-copy in the dynamic array. Doubling-and-rehash in the hash table. Path-compression flattening in union-find. Same accountant, three different ledgers.

Discipline-as-restriction as a design tool. A stack is *about* deferred decisions because push and pop are the only moves; if you could also reach into the middle, the abstraction would dissolve back into a plain array. The act of *removing* operations is the value, because it lets you reason about an algorithm's order of operations without tracking the sequence's contents. Function calls work because the language *forbids* you from reaching past the top of the stack. Breadth-first search works because the FIFO discipline *guarantees* you process the closest unvisited nodes first. The same move underlies union-find (a partition with operations merge and find), and shows up in Part III (priority queues, with insert and remove-min), Part IV (suffix arrays, with query by prefix), and Part V (graph representations, with iterate-neighbors).

Escaping the comparison model. Hash tables and Bloom filters both win by *not comparing* keys. They compute on key content, treat the result as an address, and skip the comparison-tree lower bound entirely. The same move underlies counting and radix sort from chapter 6 of Part I, and it returns in Part IV (rolling hashes for pattern matching), Part VIII (modular-arithmetic-based primality testing), and Part IX (Count-Min sketch, HyperLogLog, all extensions of the Bloom filter's "compute on content, store the fingerprint" pattern). The deeper lesson: a *lower bound is a property of a model, not a property of a problem.* Part I closed on the same lesson.

Trading exactness for space. Bloom filters and the monotonic deque are both versions of the same move: identify exactly what the workload needs to know, and design a structure that stores only *that*, with a known error rate or known approximation tolerance on the side that's allowed to be inexact. The Bloom filter stores membership without the keys. The monotonic deque stores only the currently-undominated candidates for being the window maximum. Both are *information-preserving for the operation that matters and information-discarding for everything else.* The pattern recurs in Part IX with the streaming and probabilistic structures, and it shows up in Part VIII when geometric algorithms summarize point sets with their convex hulls.

Composition by small atoms. The Part II structures stack on each other in a very literal way. Chapter 9's `Node[T]` shows up inside chapter 10's `LinkedStack` and chapter 11's `LinkedQueue`. Chapter 9's `DoublyLinkedNode[T]` is the backbone of chapter 11's `Deque`. Chapter 12's chained hash map uses the same chain-with-pointer-surgery idiom as chapter 9's linked list. Chapter 13's Bloom filter uses chapter 12's double-hashing trick to simulate k independent hashes. Chapter 14's union-find array layout is the same dense-array

pattern as chapter 8's dynamic array. Nothing in Part II is built from scratch; everything builds on what came before.

Five meta-lessons

1. A data structure is a *contract*, not an implementation. The stack contract is push-pop-peek. Two implementations satisfy it (array-backed and linked-backed), and any caller that depends only on the contract can swap them freely. The same is true of every structure in this part. Designing software around the contract rather than the implementation is what makes refactors possible decades later, and it's what makes the structures in the rest of the book composable.

2. Memory layout is not an implementation detail. Choosing a Python list over a `collections.deque` for FIFO traffic is choosing $\Theta(n)$ dequeue over $\Theta(1)$ — that choice is an algorithm decision, not a typing preference. The single split between contiguous and linked storage dictates which operations are cheap, which are expensive, and which are impossible. In C or Rust the choice is explicit. In Python most layout decisions hide behind built-in types, which makes it easy to forget and easy to default to the wrong structure. Cache locality, allocator behavior, and pointer overhead are not separate from algorithm design; they're part of it.

3. The right error guarantee is sometimes one-sided. Bloom filters are useful precisely because their wrongness is asymmetric — never wrong about negatives, only about positives. That asymmetric contract is what lets them be used as a pre-filter without compromising the overall system's correctness. The same shape shows up in caches (wrong toward "stale" but never toward "fabricated"), in sketches that approximate from below but never above, and in security protocols where false positives are recoverable but false negatives are catastrophic. The design question isn't always "be exact"; sometimes it's "be wrong in *this specific direction* with *this specific bound*."

4. Amortized analysis is how data structures actually live. Real workloads don't exercise worst cases on every operation; they exercise the *distribution* of operations, and the structure's cost is the integral over that distribution. The doubling dynamic array is the cleanest example — seventeen rare resize spikes paying for a hundred thousand cheap appends between them — but the same accounting underlies every modern structure that does any kind of internal reorganization: LRU caches, B-trees, persistent functional structures, the JIT compilation tier in a runtime. Worst-case analysis lets you reason about a single operation. Amortized analysis lets you reason about the ledger over a year.

5. Composition over selection. No one in production picks "the" data structure. Hash tables are built on dynamic arrays. Heaps are built on dynamic arrays. LRU caches combine hash tables with doubly linked lists. Tries are trees of hash tables. Production sorting routines combine multiple sorts. The skill the rest of the book is going to keep exercising isn't "remember which structure goes with which problem"; it's composing the right structure for this workload from the atoms you have. The Part II atoms are the ones you'll keep reaching for, throughout the rest of the book and out into every real codebase you'll ever work in.

What's next

Part III is another change of altitude. The seven structures in this part were *flat*: sequences, partitions, sets, with no hierarchical organization in their layout. The next eight chapters introduce *trees*: hierarchical data structures where each element has a parent (sometimes several children, sometimes a single linear chain that happens to branch). Trees are how you get $\Theta(\log n)$ operations on ordered data. A balanced search tree handles "give me every customer whose name starts with M" in $\Theta(\log n + k)$ where k is the answer size — the hash table from chapter 12 simply can't, because it scrambled the keys.

The Part II atoms keep paying out in Part III. Trees are built from linked nodes (chapter 9). Balanced trees use rotations that look a lot like chapter 9's pointer surgery, scaled up to a hierarchical context. Heaps are built on dynamic arrays (chapter 8) with implicit parent-child pointers. Tries hash characters into child arrays (chapter 12). Segment trees and Fenwick trees use the dense-array-with-implicit-pointers trick that union-find pioneered (chapter 14). You won't be relearning these atoms — you'll be watching them get arranged into branching shapes that unlock a new set of operations the flat structures couldn't support.

Part III

Trees

This third part of the Codex is where the data structures stop being *flat*. Parts I and II lived on sequences and partitions; nothing in either had a parent or a depth. The eight chapters in this part introduce **hierarchical** data structures — every element has a parent (sometimes one, sometimes implicit), and the cost shape of every operation becomes some function of the tree's height. For a balanced tree of n nodes, that's $\Theta(\log n)$ — the missing middle between Part I's $\Theta(n)$ and Part II's $\Theta(1)$, and the right cost for most workloads where sorted data has to change.

The arc of the part is a progression in how the tree's shape gets maintained.

- **Binary trees** (chapter 15) lay down the substrate — the four traversals (pre-, in-, post-, level-order), Morris's no-stack threaded walk, and pre-order-with-nulls serialization. No ordering constraint yet — just a recursive type definition and the recursion that walks it.
- **Binary search trees** (chapter 16) add a single ordering rule (smaller on the left, bigger on the right). The in-order traversal of a BST is sorted — that's the entire structure restated. Insert, search, and the three-case delete all become $\Theta(h)$, where h is the tree's height. And then the sorted-input pathology shows what can go wrong.
- **Self-balancing BSTs** (chapter 17) fix the height. AVL trees enforce $|h_{\text{left}} - h_{\text{right}}| \leq 1$ at every node and repair violations with constant-time rotations on the way back up the recursion. The Fibonacci-tree argument gives $h \leq 1.44 \log_2 n$. A sidebar names why production picks red-black trees instead.
- **Treaps** (chapter 18) get the same balance probabilistically. Each node carries a random priority; the structure maintains BST order on keys and max-heap order on priorities; the expected height is $O(\log n)$ inherited from the random-permutation analysis of chapter 16. A first taste of randomization-as-balance, which returns in Part IX.
- **Heaps and priority queues** (chapter 19) drop the BST property and keep only the heap order. The whole tree lives in an array via index arithmetic: parent at $\lfloor (i-1)/2 \rfloor$, children at $2i+1$ and $2i+2$. Build-heap in $\Theta(n)$ via Floyd's bottom-up trick — the implicit-tree-as-array pattern returns in chapters 21 and 22.
- **Tries** (chapter 20) make the tree's *path* encode the key. One node per character, a `terminal` flag for stored words, and the prefix queries hash tables physically can't answer. Compressed (radix) tries collapse single-child chains, and longest-prefix matching turns the structure into an IP-routing engine.
- **Segment trees** (chapter 21) take the array-as-implicit-tree pattern from heaps and apply it to range queries. Every node owns an interval; the answer to a range query is the sum of the $O(\log n)$ canonical intervals that tile it. Lazy propagation pushes the structure to $O(\log n)$ on both range update and range query.
- **Fenwick trees** (chapter 22) are the same problem at half the code. For invertible monoids — sum, xor, count — the `i & -i` lowest-set-bit trick gives you point update + prefix query in eight lines per operation. The trade against segment trees: shorter code, restricted to aggregates with inverses.

Four moves come back through the rest of the book: **rotations** as the universal repair operation for ordered trees (chapters 17, 18); **the implicit-array-as-tree pattern** (chapters 19, 21, 22) where the tree's shape lives in index arithmetic and saves you a node-per-element allocation; **path-encodes-key** (chapter 20) where the structural position *is* the data; and **balance as invariant** in either its deterministic or probabilistic form (chapters 17, 18). Part V (graphs) reaches for the heap on every shortest-path call; Part VI (dynamic programming) reaches for the segment / Fenwick trees on every range-query subproblem; the BSTs underpin every ordered-map type in every language standard library you'll ever touch.

15 Branching from one root

A binary tree is the smallest data structure that lets information *branch*. Every node has at most two children — a left and a right — and from any node you can either descend or stop. That single restriction is enough to give you sorted-data structures with $\Theta(\log n)$ lookup, hierarchical layouts for arithmetic expressions and file systems, and priority queues with logarithmic insertion.

Every node has at most two children — but the recursion has only one shape. Pre-, in-, and post-order differ by exactly one line. Once you see that, the rest of the chapter is bookkeeping — the four traversals, Morris's threaded in-order walk in $O(1)$ extra space, and a serializer that round-trips a tree through a string.

15.1 The shape of a tree

The definition is recursive because the recursion *is* the data structure. A binary tree is either empty, or it's a node holding a value plus two children — each of which is, itself, a binary tree. That's it. The base case (empty) and the recursive case (value plus two sub-trees) are everything.

A few words that come up constantly. The **root** is the topmost node — the one no other node points at. A **leaf** has no children. The **depth** of a node is the number of edges from the root down to it (root has depth 0). The **height** of a tree is the maximum depth of any node, equivalently the longest root-to-leaf path. A perfectly balanced tree of n nodes has height $\lfloor \log_2 n \rfloor$.

That last fact is what makes binary trees interesting as a *cost shape*. Part I lived at $\Theta(n)$; Part II broke through to $\Theta(1)$ for the operations it cared about. Part III sits between those two, at $\Theta(\log n)$ — the cost of walking from root to leaf in a balanced tree. That's the "throw half away with every comparison" shape you met in chapter 2 with binary search on a sorted array, recovered here for *data that changes* — insertions, deletions, updates — without falling back to $\Theta(n)$ rebuilds.

The simplest possible node is a value and two child pointers. Generic in the value type, plain Python class, no decorators or protocols.

```
from collections import deque
from typing import Iterator


class BinaryNode[T]:
    def __init__(
        self,
        value: T,
        left: "BinaryNode[T] | None" = None,
        right: "BinaryNode[T] | None" = None,
    ) -> None:
        self.value = value
        self.left = left
        self.right = right
```

The forward-referenced string annotations are the same Python detail that came up with `LinkedNode` in chapter 9 — the class is referring to itself before its name has finished binding. The shape itself is exactly the recursive definition: a value, plus two optional sub-trees. `None` is the empty-tree case.

The canonical tree for this chapter is small enough to trace by hand and asymmetric enough that the four traversals all give visibly different outputs:

```
        1
       / \
      2   3
     / \   \
    4   5   6
```

You build it bottom-up, leaves first, then parents:

```
from codex.trees.binary import BinaryNode

# leaves first, then parents
n4 = BinaryNode(4)
n5 = BinaryNode(5)
n6 = BinaryNode(6)
n2 = BinaryNode(2, left=n4, right=n5)
n3 = BinaryNode(3, right=n6)        # no left child
root = BinaryNode(1, left=n2, right=n3)

print(f"root value: {root.value}")
print(f"root.left  is node {root.left.value}, root.right is node {root.right.value}")
```

```
root value: 1
root.left  is node 2, root.right is node 3
```

Six nodes, height 2, asymmetric on purpose so traversal order disambiguates the four walks. Every section below uses this same tree.

With `BinaryNode` and the six-node example tree in hand, the next question is the obvious one. How do you visit every node, and in what order?

15.2 Four walks around the same tree

There are four canonical traversals of a binary tree, and the recursion-has-one-shape claim is best understood by writing them. The three depth-first orders — pre-, in-, and post-order — are *the same three-line recursion with the "visit self" line moved*. Level-order — the fourth — uses a queue instead of recursion, which sets it apart.

Pre-order visits a node *before* its children. The recursion does three things in order: emit this node's value, recurse left, recurse right.

```
def preorder[T](node: BinaryNode[T] | None) -> Iterator[T]:
    if node is None:
        return
    yield node.value
    yield from preorder(node.left)
    yield from preorder(node.right)
```

The base case is the empty tree — there's nothing to yield. The recursive case is the three-line sandwich. I wrote this as a generator (`yield` / `yield from`) so traversal is lazy: a consumer that only needs the first few values doesn't pay for the whole walk.

In-order moves the `yield` line *between* the two recursive calls. Now you emit a node's value *after* its left subtree and *before* its right subtree.

```
def inorder[T](node: BinaryNode[T] | None) -> Iterator[T]:
    if node is None:
        return
    yield from inorder(node.left)
    yield node.value
    yield from inorder(node.right)
```

Post-order moves it all the way to the end. Children first, parent last.

```
def postorder[T](node: BinaryNode[T] | None) -> Iterator[T]:
    if node is None:
        return
    yield from postorder(node.left)
    yield from postorder(node.right)
    yield node.value
```

Three functions, identical except for the position of one line. The recursion has one shape, and the "kind" of traversal is just where you put the visit.

Level-order is different. It walks the tree breadth-first — root, then both children, then all four grandchildren, then the eight great-grandchildren, and so on, one row at a time. The natural data structure for that is the FIFO queue from chapter 11: enqueue the root, then in a loop dequeue a node, emit its value, and enqueue its non-empty children.

```
def level_order[T](root: BinaryNode[T] | None) -> Iterator[T]:
    if root is None:
        return
    queue: deque[BinaryNode[T]] = deque([root])
    while queue:
        node = queue.popleft()
        yield node.value
        if node.left is not None:
            queue.append(node.left)
        if node.right is not None:
            queue.append(node.right)
```

The discipline is the same one that powers breadth-first search on a graph — FIFO ordering guarantees you process every node at depth d before any node at depth $d+1$. Here the "graph" is a tree, so each node is visited exactly once and there's no need for a "seen" set.

All four traversals on the canonical tree:

```
from codex.trees.binary import preorder, inorder, postorder, level_order

print(f"pre-order:   {list(preorder(root))}")    # [1, 2, 4, 5, 3, 6]
print(f"in-order:    {list(inorder(root))}")     # [4, 2, 5, 1, 3, 6]
print(f"post-order:  {list(postorder(root))}")   # [4, 5, 2, 6, 3, 1]
print(f"level-order: {list(level_order(root))}") # [1, 2, 3, 4, 5, 6]
```

```
pre-order:   [1, 2, 4, 5, 3, 6]
in-order:    [4, 2, 5, 1, 3, 6]
post-order:  [4, 5, 2, 6, 3, 1]
level-order: [1, 2, 3, 4, 5, 6]
```

Same six values, four different sequences. Pre-order is the natural choice when you want a copy of the tree's *shape* — each node lands in the stream before its descendants. In-order is the one chapter 16 leans on hardest: for a binary search tree it spits the values out *sorted*. Post-order is what you reach for when each subtree must be *finished* before its parent — evaluating an expression tree, deleting nodes, computing aggregates from leaves up. Level-order is the right fit when distance from the root is what matters — printing the tree row-by-row, or finding the shallowest node satisfying some predicate.

The recursive walks have one cost that doesn't show in the time bound: they use $\Theta(h)$ space on the call stack. For a balanced tree that's $\Theta(\log n)$, but for a pathological chain it's $\Theta(n)$. The next section is what you do when even that's too much.

15.3 Walking without a stack

Recursive in-order needs the call stack to remember which ancestors it still has to revisit on its way back up. An iterative version using an explicit `list` as a stack has the same space cost — you've just moved the stack from C to Python. The natural question is whether you can walk the tree using only $O(1)$ *extra* memory — no recursion, no auxiliary stack, no array.

Joseph Morris answered yes, in 1979, with a beautiful trick: **temporarily rewrite the tree itself**. Look at the leaves and partial nodes whose `right` is `None` — those null slots are wasted memory you can borrow. The Morris idea is to make the `right` pointer of each in-order *predecessor* point back at its in-order *successor*, so when the traversal finishes walking a left subtree it can hop directly back to the current node without a stack frame. Then, the second time the traversal reaches that node, it *unwrites* the thread and continues.

A worked example on the canonical tree. Start at node 1; its left subtree is non-empty, so find 1's in-order predecessor — the rightmost node in its left subtree, which is node 5 — and write `5.right = 1` as a thread. Descend into 2; its predecessor is 4, so write `4.right = 2` and descend into 4. From 4, left is empty: emit 4, follow `4.right` back to 2. At 2 the second time, detect the thread (you put it there), unwrite it, emit 2, descend right into 5. From 5, left is empty: emit 5, follow the thread back to 1. Detect, unwrite, emit 1, descend right into 3. And so on. Output: `[4, 2, 5, 1, 3, 6]` — identical to recursive in-order, using only the loop variables as extra memory.

The whole algorithm fits in fifteen-odd lines, but it earns its decomposition: there's a body loop, and inside that loop a sub-step that finds the predecessor and either writes or unwrites the thread. The sub-step lifts out as its own helper.

```
def _rightmost[T](node: BinaryNode[T], stop: BinaryNode[T]) -> BinaryNode[T]:
    while node.right is not None and node.right is not stop:
        node = node.right
    return node
```

`_rightmost` walks down right pointers from `node` until it either runs out (`right is None` — no thread yet, you're going to install one) or finds `stop` (`right is stop` — the thread is already installed by a previous pass, you're going to remove it). The `stop` parameter is the trick — it lets the helper serve both phases of the algorithm without a flag.

The body of Morris in-order is then a while-loop on a `cur` pointer. Two cases at every step. If `cur.left` is empty, the in-order position is right here — emit `cur.value` and step right. Otherwise, find the rightmost descendant of the left subtree; if its `right` is empty, install a thread back to `cur` and descend left; if its `right` is already `cur`, the thread was installed on a previous pass, so unwrite it (restore `None`), emit `cur.value`, and step right.

```
def morris_inorder[T](root: BinaryNode[T] | None) -> Iterator[T]:
    cur = root
    while cur is not None:
        if cur.left is None:
            yield cur.value
            cur = cur.right
            continue
        pred = _rightmost(cur.left, cur)
        if pred.right is None:
            pred.right = cur                # install thread
            cur = cur.left
        else:
            pred.right = None               # remove thread, restore tree
            yield cur.value
            cur = cur.right
```

Two branches at each step, no recursion, no auxiliary stack — the only mutable state is `cur`, `pred`, and the temporarily-installed threads inside the tree itself. When the loop finishes, every thread has been removed, so the tree is bit-identical to its starting state. That's the algorithm's contract: it *mutates* and *unmutates* in the same pass, leaving the tree as it found it.

Hand-verifying that Morris and the recursive in-order produce the same sequence:

```python
from codex.trees.binary import morris_inorder

# rebuild the canonical tree to be safe from earlier demo mutations
n4 = BinaryNode(4); n5 = BinaryNode(5); n6 = BinaryNode(6)
n2 = BinaryNode(2, left=n4, right=n5)
n3 = BinaryNode(3, right=n6)
root = BinaryNode(1, left=n2, right=n3)

print(f"recursive in-order: {list(inorder(root))}")
print(f"morris in-order:    {list(morris_inorder(root))}")
print(f"tree restored?      {n4.right is None and n5.right is None}")
```

```
recursive in-order: [4, 2, 5, 1, 3, 6]
morris in-order:    [4, 2, 5, 1, 3, 6]
tree restored?      True
```

Same sequence both ways. The last check confirms that the threads were removed — nodes 4 and 5 (the in-order predecessors of 2 and 1 respectively) have their original `right is None` restored after the walk.

The cost is $O(n)$ time and $O(1)$ *extra* space (beyond the tree itself). The time bound takes a second to see — some edges look like they're traversed twice — but the careful argument is that every edge is traversed at most three times (install thread, descend, remove thread), so total work is $\Theta(n)$. Morris is roughly 2–3× slower than recursive in-order in raw wall-clock, and you trade that constant factor for never blowing the recursion stack.

A tree also needs to be written *down* — to a string, a file, the network — so it can be read back later.

15.4 Writing a tree to a string

The serialization problem: turn a binary tree into a flat sequence of tokens so it can be written to a file and re-parsed into the same tree. The catch is that the tree's *shape* has to survive the round trip. Pre-order alone isn't enough — the sequence `[1, 2, 3]` could be a chain, a left-leaning V, a right-leaning V, or a balanced triangle. You can't tell from the values.

The fix is a classic textbook trick: pre-order with **explicit null markers**. Whenever the recursion hits `None`, emit a sentinel token — `"#"` here. That gives every recursive call exactly one token to consume, and the tree's shape is encoded by where the nulls land. For the canonical tree, the serialization is `1 2 4 # # 5 # # 3 # 6 # #` — thirteen tokens for six values, with the seven `#` markers carrying the shape information.

Serialization is one block; deserialization splits into a small driver and a recursive parser, two blocks for clarity.

```python
def serialize[T](root: BinaryNode[T] | None) -> str:
    tokens: list[str] = []

    def _walk(node: BinaryNode[T] | None) -> None:
        if node is None:
            tokens.append("#")
            return
        tokens.append(str(node.value))
        _walk(node.left)
        _walk(node.right)

    _walk(root)
    return " ".join(tokens)
```

The inner `_walk` is the same three-line pre-order recursion you wrote earlier, with `None` mapped to `"#"` instead of being a silent base case. The outer function joins the token list with spaces and returns the string.

Deserialization is the inverse, and it takes a small driver that splits the string into tokens and feeds them to a recursive parser.

```python
def deserialize(s: str) -> BinaryNode[int] | None:
    tokens = iter(s.split())
    return _parse(tokens)
```

The driver hands an iterator over the token stream to `_parse`, which consumes exactly the tokens belonging to one subtree and stops. Because the recursion shape and the serialization shape match, the iterator naturally hands off cleanly between recursive calls.

```python
def _parse(tokens: Iterator[str]) -> BinaryNode[int] | None:
    token = next(tokens)
    if token == "#":
        return None
    node = BinaryNode(int(token))
    node.left = _parse(tokens)
    node.right = _parse(tokens)
    return node
```

`_parse` consumes one token per call. If it's the null marker, return `None` (matching the `_walk` base case). Otherwise allocate a node, then recurse on the iterator — the left subtree is parsed first, then the right, mirroring the serialization order. The iterator's state advances across recursive calls because it's the *same iterator object*, threaded through. That's why I split the entry function from the worker: it lets me hand a fresh iterator to the top of the recursion exactly once.

I picked `BinaryNode[int]` as the return type because the textual round trip needs the values to be parseable from strings. A fully generic round trip would need a serializer-and-parser per type — real-world JSON or protobuf territory, out of scope here. The shape of the algorithm doesn't change.

Round-tripping the canonical tree:

```python
from codex.trees.binary import serialize, deserialize

n4 = BinaryNode(4); n5 = BinaryNode(5); n6 = BinaryNode(6)
n2 = BinaryNode(2, left=n4, right=n5)
n3 = BinaryNode(3, right=n6)
root = BinaryNode(1, left=n2, right=n3)

encoded = serialize(root)
print(f"encoded: {encoded}")

decoded = deserialize(encoded)
print(f"pre-order of decoded:   {list(preorder(decoded))}")
print(f"in-order of decoded:    {list(inorder(decoded))}")
print(f"level-order of decoded: {list(level_order(decoded))}")
```

```
encoded: 1 2 4 # # 5 # # 3 # 6 # #
pre-order of decoded:   [1, 2, 4, 5, 3, 6]
in-order of decoded:    [4, 2, 5, 1, 3, 6]
level-order of decoded: [1, 2, 3, 4, 5, 6]
```

Same pre-, in-, and level-order outputs as the original — the structure survived the round trip intact. The encoded string is `1 2 4 # # 5 # # 3 # 6 # #`, exactly as predicted.

This isn't the *only* encoding that round-trips. Knowing both pre-order and in-order (without null markers) also uniquely identifies a tree as long as values are distinct — that's how CLRS sets up the reconstruction problem. The pre-order-with-nulls form is simpler because it uses one sequence and works even when values repeat.

15.5 The three questions, applied

Is it correct?

The four traversals are correct because each one is a faithful implementation of its recursive definition — pre-, in-, and post-order differ only in where the `yield` line sits, and that position corresponds exactly to the named visit order. Level-order is correct because the FIFO discipline guarantees all depth-d nodes are emitted strictly before any depth-$(d+1)$ node, by the same argument that makes BFS work on a graph.

Morris in-order is correct because the thread invariant is preserved at every loop iteration: when you install `pred.right = cur` you've claimed an otherwise-null pointer; when you later detect `pred.right is cur`, the only way it's true is that *this same iteration* installed it (the descent into the left subtree never touches `cur`). So unwriting the thread restores the tree, and emitting `cur.value` between the left and right walks matches the recursive in-order position.

The serialize / deserialize pair is correct because the pre-order-with-nulls encoding has the round-trip property — each recursive call emits exactly enough tokens to be re-consumed by exactly one recursive call on the deserializing side.

How efficient is it?

The cost table for a tree of n nodes with height h:

Operation	Time	Extra space
`preorder` / `inorder` / `postorder`	$\Theta(n)$	$\Theta(h)$ — call stack
`level_order`	$\Theta(n)$	$O(w)$ — queue, where w is the maximum tree width
`morris_inorder`	$\Theta(n)$	$\Theta(1)$
`serialize` / `deserialize`	$\Theta(n)$	$\Theta(n)$ — output string / token list

That is: every traversal touches each node a constant number of times, so wall-clock cost is linear in the tree's size. The space columns are where the structures differ — recursive walks pay $\Theta(h)$ for the call stack, level-order pays for whatever's in the queue (worst case $\Theta(n/2)$ at the bottom row of a complete tree), and Morris is the only $O(1)$-extra-space option.

For a balanced tree, $h = \Theta(\log n)$, so all of these except `level_order` use logarithmic extra space. For a pathological chain ($h = n - 1$), recursive traversals use $\Theta(n)$ stack space — exactly the case where Morris matters, since it doesn't blow up where a recursive walk would risk a `RecursionError`.

Is it optimal?

Yes, by the most basic argument available. Any algorithm that *visits every node* of an n-node tree must do at least $\Omega(n)$ work, so the $\Theta(n)$ time bound is tight by inspection. For *space*, Morris is optimal in a stronger sense — $\Theta(1)$ extra space is the minimum for a traversal of an arbitrary tree, because any extra-space-free strategy must use the tree itself as scratch. Recursive traversals are suboptimal in space (they pay $\Theta(h)$ for the stack) but buy back simplicity. The serialize pair is optimal in *output size* — any unambiguous encoding of an n-node tree needs $\Theta(n)$ tokens, and the pre-order-with-nulls form achieves exactly $2n + 1$, within a constant of the lower bound.

The deeper question — what tree *structure* you should choose for a given workload — is what the rest of Part III is about. Binary trees alone don't impose ordering on their values, so they can't speed up lookup; binary search trees (chapter 16) add that. Binary trees alone don't guarantee logarithmic height, so they can degenerate into chains; self-balancing trees (chapter 17) fix that.

15.6 One recursion shape, four orders

Every node has at most two children — but the recursion has only one shape. The three depth-first traversals are the same three-line function with the visit position moved; level-order is the breadth-first cousin that swaps recursion for a queue; Morris in-order trades a constant-factor time penalty for $O(1)$ extra space by temporarily rewriting the tree. Every traversal is $\Theta(n)$, which is optimal because reporting every node has to read every node.

The same recursion structure carries different semantics through one moved line. The position of the yield distinguishes pre-, in-, and post-order, and each position naturally matches a class of problem (copy the tree, sort the values, aggregate from leaves up). Recognizing that the data structure's shape *is* the recursion's shape is what makes tree algorithms feel less like memorization.

$O(1)$-extra-space traversals exist when you're willing to mutate-and-restore. Morris's threaded walk is the canonical example, and the same discipline shows up in iterative deepening and in some in-place graph algorithms. The trick is to notice that the tree already holds enough wasted pointers to serve as its own scratch space — you just have to promise to put them back.

The choice between traversal orders isn't aesthetic. Each one is the right answer for a different class of problem, and recognizing the workload that fits each order is the skill that tree-heavy code rewards.

The chapter that follows is what happens when you *constrain* the values in a binary tree to be ordered by position — left descendants smaller, right descendants larger. Once that holds, in-order traversal outputs the values *sorted*, and lookup, insertion, and deletion all become $\Theta(\log n)$ on a balanced tree. That's the binary search tree.

15.7 Notes and further reading

Binary trees and their traversals are covered in CLRS (4th ed.) chapter 12 as the substrate for binary search trees; the chapter's reconstruction-from-two-traversals exercise is exactly the textbook way to set up the deserialization problem this chapter handles with explicit nulls. Sedgewick and Wayne's *Algorithms* (4th ed.) §3.2 introduces binary search trees with traversal mechanics inline, and is the pedagogical inspiration for treating the tree's shape and the traversal's shape as the same object. The threaded-traversal trick is Joseph M. Morris's 1979 paper "Traversing Binary Trees Simply and Cheaply," published in *Information Processing Letters* — three pages, no jargon, worth reading in the original for the elegance of the construction. The serialize-with-nulls format is folklore in competitive-programming circles; LeetCode's "Serialize and Deserialize Binary Tree" problem (#297) is the standard exercise that crystallized it as the textbook approach.

16 Smaller on the left, bigger on the right

You have a binary tree, but you don't get to put values wherever you want anymore. You agree to a rule: at every node, every key on the left is smaller, every key on the right is bigger. That single constraint turns the substrate of chapter 15 into a structure that can *answer queries* — find a key in $\Theta(\log n)$ on a balanced tree, insert without disturbing the order, delete without breaking it. A binary search tree is what happens when you force a binary tree's values to know their place.

The in-order traversal of a BST is sorted — that's the entire structure restated. Every claim about correctness in this chapter cashes out to that one sentence. Every operation is designed to preserve it.

16.1 Order as invariant

A binary tree is a **binary search tree** if, for every node, every key in its left subtree is strictly less than the node's key, and every key in its right subtree is strictly greater. The rule is recursive — it has to hold at every node, not just at the root.

That definition deliberately disallows duplicates of a given key (the strict inequalities forbid them). Take it as a working contract; the duplicate-handling decision lands further down.

The reason the rule earns its name comes from a tiny theorem. Recall the in-order traversal from chapter 15: left subtree, then the node itself, then the right subtree. Apply that walk to a BST. The left subtree's keys are all smaller than the node; by induction, the recursive walk emits them in sorted order. The node comes next, larger than everything emitted so far. Then the right subtree's keys, all larger than the node; by induction again, the walk emits them in sorted order, and the smallest of them is still larger than the node. You get a monotone increasing sequence.

Every operation below is checked the same way: insert some keys, run `in_order`, confirm the output is sorted. If the order is preserved, the invariant is preserved, and the structure is correct. In-order = sorted is the testing strategy.

What does the invariant buy you? Exactly the structure binary search on a sorted array used in chapter 2: at each node you compare your target to the current key, and one comparison eliminates an entire half of the remaining tree. Lookup, insert, and delete all become **walk root-to-leaf** operations, which cost $\Theta(h)$ where h is the tree's height. For a balanced tree, $h = \Theta(\log n)$. That is: a million keys cost you about twenty comparisons; a billion cost about thirty. Same shape as binary search.

What does the invariant cost you? Two things. First, every insertion has to find the right spot, so insertion is no longer the $\Theta(1)$ tail-append you get from an unordered list or a hash table — you pay $\Theta(h)$ for it. Second, the height can degenerate. The structural rule says nothing about *balance*, only about order. Insert keys in sorted order and the tree becomes a linked list — every "right subtree" is non-empty, every "left subtree" is empty, and you've recovered chapter 1's $\Theta(n)$ linear scan with extra steps. The fourth section of this chapter is exactly that pathology; chapter 17 is what you do about it.

The first two operations almost write themselves.

16.2 Insert and search

Insert and search both follow the same pattern: at each node, compare the query key to the node's key, and recurse into the half the invariant tells you the answer must live in. If the target is smaller, go left. If larger, go right. If equal, you've found it.

A BST node carries a key *and* a value: the key is what the structure orders by, and the value is the payload you want to look up. Same shape as chapter 15's `BinaryNode`, but generic in two parameters instead of one.

```
from typing import Iterator


class BSTNode[K, V]:
    def __init__(
        self,
        key: K,
        value: V,
        left: "BSTNode[K, V] | None" = None,
        right: "BSTNode[K, V] | None" = None,
    ) -> None:
        self.key = key
        self.value = value
        self.left = left
        self.right = right
```

Same forward-referenced string annotations as `BinaryNode` — the class refers to itself before its name has finished binding. Two children, one key, one value. Comparison happens directly on `K` via `<`, `>`, and `==` — I'm assuming the keys are mutually comparable. Chapters 3 and 4 used the `Ordering[T]` protocol from `codex.types` for sort routines, but here I'll keep the comparisons in-line; the BST's character is the structural recursion, not the comparison plumbing.

The wrapper class is where I'll put the public methods. It owns the root pointer and an incremental size counter — keeping `len(tree)` in $O(1)$ is worth the one extra arithmetic operation per insert and delete.

```
class BST[K, V]:
    def __init__(self) -> None:
        self._root: BSTNode[K, V] | None = None
        self._size: int = 0

    def __len__(self) -> int:
        return self._size
```

The first real operation is `insert`. Walk the tree, comparing the new key to the current node's key, descending left or right until you find a `None` slot to attach a new node at — or until you hit an existing node with the same key, in which case I'll overwrite the value. The functional-recursive idiom returns the new subtree root at every level, so the caller reassigns whichever pointer it just descended through. That same idiom will carry me through delete, and it ports straight to AVL and treap in later chapters.

I'll write `insert` as a thin public wrapper around a nested recursive helper. Keeping the helper nested lets it mutate the wrapper's `_size` counter via closure capture instead of being threaded as a parameter.

```
    def insert(self, key: K, value: V) -> None:
        def _insert(node: BSTNode[K, V] | None) -> BSTNode[K, V]:
            if node is None:
                self._size += 1
                return BSTNode(key, value)
            if key < node.key:
                node.left = _insert(node.left)
            elif key > node.key:
                node.right = _insert(node.right)
            else:
                node.value = value  # replace existing value, leave size unchanged
            return node

        self._root = _insert(self._root)
```

Three branches per node, plus the base case. The base case allocates a fresh leaf and bumps the size — that's the only place new nodes ever come from. The two recursive branches descend into the half the invariant says the new key belongs in, and reassign the child pointer to whatever the recursion returned. The fourth case

(equal key) is the duplicate-handling decision I deferred above: I overwrite the existing value, which gives the BST dict-like behaviour — one slot per key, last writer wins.

Other policies are defensible. You could **reject** duplicates and raise an error (strict-set behaviour). You could **count multiplicities** with a counter field on each node (useful for multisets). You could insert a duplicate into either subtree under a tie-breaking rule, which is common in textbook BST treatments but makes the in-order traversal merely non-decreasing rather than strictly increasing, weakening the sortedness guarantee. **Replace-value** is the cleanest for a dictionary-shaped structure and the one most production map-like BSTs use, so that's what I'll pick.

Search is the same recursion stripped of the writes.

```
    def search(self, key: K) -> V | None:
        node = self._root
        while node is not None:
            if key < node.key:
                node = node.left
            elif key > node.key:
                node = node.right
            else:
                return node.value
        return None

    def __contains__(self, key: K) -> bool:
        return self.search(key) is not None
```

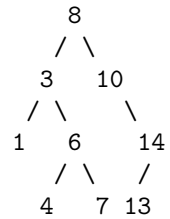
I wrote `search` iteratively because there's no need to thread anything back up — the answer is found at one node and returned directly. Three comparisons, one descent per level, $\Theta(h)$ in the worst case. `__contains__` is the boolean version that lets `key in tree` work as Python expects.

The in-order generator mirrors chapter 15's `inorder`, yielding `(key, value)` pairs and using the same nested-helper pattern.

```
    def in_order(self) -> Iterator[tuple[K, V]]:
        def _walk(node: BSTNode[K, V] | None) -> Iterator[tuple[K, V]]:
            if node is None:
                return
            yield from _walk(node.left)
            yield (node.key, node.value)
            yield from _walk(node.right)

        yield from _walk(self._root)
```

The canonical tree comes from inserting nine keys in an order chosen so the shape is small enough to draw and asymmetric enough to exercise all the delete cases later: `[8, 3, 10, 1, 6, 14, 4, 7, 13]`. The resulting tree looks like this:



Nine nodes, height 3, in-order should be `[1, 3, 4, 6, 7, 8, 10, 13, 14]` if I've done my work.

```python
from codex.trees.bst import BST

tree: BST[int, str] = BST()
for k in [8, 3, 10, 1, 6, 14, 4, 7, 13]:
    tree.insert(k, f"v{k}")

print(f"size: {len(tree)}")                          # 9 - nine keys, no duplicates
print(f"in-order: {[k for k, _ in tree.in_order()]}")
# [1, 3, 4, 6, 7, 8, 10, 13, 14] - sorted, as the invariant promises

print(f"search 7:  {tree.search(7)}")                # v7 - present
print(f"search 99: {tree.search(99)}")               # None - missing
print(f"6 in tree?  {6 in tree}")                    # True - key 6 is in the tree
print(f"99 in tree? {99 in tree}")                   # False - key 99 is not
```

```
size: 9
in-order: [1, 3, 4, 6, 7, 8, 10, 13, 14]
search 7:  v7
search 99: None
6 in tree?  True
99 in tree? False
```

The in-order output is sorted. Every subsequent operation gets checked against the same thing.

The duplicate-handling contract from above, made visible — overwriting key 7:

```python
tree.insert(7, "v7-replaced")
print(f"size after overwrite: {len(tree)}")          # 9 - still nine, no new node
print(f"search 7:  {tree.search(7)}")                # v7-replaced - value updated
```

```
size after overwrite: 9
search 7:  v7-replaced
```

Same number of nodes, new value. The duplicate-handling rule is doing what I said it would.

16.3 Delete, the three-case method

Insertion was easy because it only ever attaches a new leaf — every existing pointer stays valid. Deletion is harder, because removing an interior node leaves a hole in the tree, and you have to fill the hole in a way that preserves the BST invariant. The textbook algorithm splits the problem into three cases by how many children the node-to-delete has.

The cases are mutually exclusive and cover everything. **Zero children** (a leaf): just unlink it. The parent's pointer to the leaf becomes `None`. **One child** (left or right only): splice the child up into the deleted node's place. The grandparent now points directly at the grandchild, and the BST invariant is preserved because the entire subtree you're lifting was already on the correct side of every ancestor. **Two children**: the hard case. You can't just lift one child because the other has nowhere to go. The trick is to find the **in-order successor**, which is the smallest key in the right subtree, copy its key and value into the node-to-delete, and then recursively delete the successor from the right subtree. Because the successor is the leftmost node of the right subtree, it has at most one child (its right one), so the recursive delete falls into case zero or case one and terminates cleanly.

The in-order successor works because the deleted node's in-order position gets filled by the next key in the sorted sequence, which by definition is the successor. You shift the values up by one slot, and the in-order traversal stays monotone.

The combined delete logic runs past twenty lines as one function, so two helpers carry it — one for the successor finder, one for the recursive delete itself.

The first helper is the successor-finder. The leftmost node of any subtree holds the smallest key in that subtree — another direct consequence of the invariant — so two lines suffice: walk left until you can't.

```
    def _min_node(self, node: BSTNode[K, V]) -> BSTNode[K, V]:
        while node.left is not None:
            node = node.left
        return node
```

The `delete` method itself is a public wrapper around a nested recursive helper. The helper takes the current subtree root and the target key, walks down to find the matching node, then handles the three cases at the match site. I parameterize the recursion by `k` rather than capturing the outer `key` — that's what lets the two-children case re-invoke the recursion with the successor's key.

```
    def delete(self, key: K) -> None:
        def _delete(
            node: BSTNode[K, V] | None, k: K
        ) -> BSTNode[K, V] | None:
            if node is None:
                return None  # key not present - nothing to do
            if k < node.key:
                node.left = _delete(node.left, k)
                return node
            if k > node.key:
                node.right = _delete(node.right, k)
                return node
            # node.key == k - match site, dispatch on children
            if node.left is None:
                self._size -= 1
                return node.right  # leaf or right-only child
            if node.right is None:
                self._size -= 1
                return node.left   # left-only child
            # two children: copy successor's payload, then recursively delete the
            # successor from the right subtree (it has at most one child, so the
            # recursive call lands in one of the simpler cases above).
            succ = self._min_node(node.right)
            node.key = succ.key
            node.value = succ.value
            node.right = _delete(node.right, succ.key)
            return node

        self._root = _delete(self._root, key)
```

The first three branches walk down the tree. The fourth — when keys match — is the case split. Notice how the leaf case and the one-child case fall out of the same two lines: if `left` is `None`, returning `right` works whether `right` is also `None` (leaf) or a real subtree (right-only child); same in reverse for `left`. That's why I called it the three-case method but the code only has two physical branches plus the two-children case.

The two-children case copies the successor's payload into the node and then recursively deletes the successor from the right subtree. The recursion is guaranteed to find the successor (because `_min_node` walked there), and the successor has at most one child, so the recursive call terminates immediately in one of the simpler cases. I decrement the size counter only once, because only the recursive call hits a node-with-fewer-than-two-children.

The leftmost node of any subtree holds the smallest key in that subtree — that's another direct consequence of the invariant. For the two-children delete on the canonical tree, I'll be deleting key 3, whose right subtree is rooted at 6. The leftmost descendant of 6 is 4 (6's own left child). So 4 gets copied up into the 3-slot, and then 4 is recursively deleted from the right subtree, where it falls into the leaf case.

I'll demonstrate the three cases on freshly-rebuilt copies of the canonical tree:

```
def build_canonical() -> BST[int, str]:
    t: BST[int, str] = BST()
    for k in [8, 3, 10, 1, 6, 14, 4, 7, 13]:
        t.insert(k, f"v{k}")
```

```
    return t


# Case 1 - leaf: delete key 1 (no children)
t1 = build_canonical()
t1.delete(1)
print(f"after delete 1 (leaf):     {[k for k, _ in t1.in_order()]}")
# [3, 4, 6, 7, 8, 10, 13, 14] - still sorted, 1 is gone

# Case 2 - one child: delete key 14 (only left child, 13)
t2 = build_canonical()
t2.delete(14)
print(f"after delete 14 (1 child): {[k for k, _ in t2.in_order()]}")
# [1, 3, 4, 6, 7, 8, 10, 13] - 13 spliced up into 14's slot

# Case 3 - two children: delete key 3 (children 1 and 6; successor is 4)
t3 = build_canonical()
t3.delete(3)
print(f"after delete 3 (2 child):  {[k for k, _ in t3.in_order()]}")
# [1, 4, 6, 7, 8, 10, 13, 14] - 4 has moved into the 3-slot, then was removed from below
```

```
after delete 1 (leaf):     [3, 4, 6, 7, 8, 10, 13, 14]
after delete 14 (1 child): [1, 3, 4, 6, 7, 8, 10, 13]
after delete 3 (2 child):  [1, 4, 6, 7, 8, 10, 13, 14]
```

All three in-order outputs are still sorted, and the size has dropped by one in each case. Preserving sortedness is the test; every case passes it.

The root is deletable by the same algorithm, and deleting a missing key is a silent no-op.

```
t = build_canonical()
t.delete(99)                                # not present - no-op
print(f"size after no-op:  {len(t)}")     # 9 - nothing happened
t.delete(8)                                 # the root, two children
print(f"size after root:   {len(t)}")     # 8 - root replaced by its in-order successor
print(f"in-order:          {[k for k, _ in t.in_order()]}")
# [1, 3, 4, 6, 7, 10, 13, 14] - still sorted, 8 is gone, 10 became the new root
```

```
size after no-op:  9
size after root:   8
in-order:          [1, 3, 4, 6, 7, 10, 13, 14]
```

16.4 When BSTs go wrong

Everything above assumed the BST was *reasonable* — a height proportional to $\log n$, root-to-leaf walks that cost about twenty comparisons for a million keys. The structural rule guarantees no such thing. The rule says only that keys are sorted by position; it says nothing about balance. The same nine keys I inserted as `[8, 3, 10, 1, 6, 14, 4, 7, 13]` produced a tree of height 3 — but if I'd inserted them sorted, the result would have been a chain of nine nodes, one per level. Lookups would cost nine comparisons instead of three.

The pathology is most vivid at sixteen keys. Using the convention from chapter 15 — root has depth 0, height is the maximum depth of any node — a 16-node chain has height 15 (root at 0, deepest leaf at 15).

```
    def height(self) -> int:
        def _height(node: BSTNode[K, V] | None) -> int:
            if node is None:
                return -1  # empty: -1 so a single node ends up at height 0
            return 1 + max(_height(node.left), _height(node.right))
```

```
        return _height(self._root)
```

I return -1 for the empty tree so that a single-node tree comes out at height 0 — a root at depth 0 and no other nodes. The recursive case is the maximum of the two subtree heights plus one for the edge from the current node to its taller child.

Inserting the integers 0 through 15 in their natural order, then redoing the experiment on a shuffled copy of the same keys:

```
import random
from codex.trees.bst import BST

# Sorted insertion - every key is larger than its predecessor, so every
# insertion goes down the right spine. The tree becomes a chain.
sorted_tree: BST[int, int] = BST()
for k in range(16):
    sorted_tree.insert(k, k)

print(f"sorted insertion: 16 keys, height = {sorted_tree.height()}")
# 15 - a chain of 16 nodes, root at depth 0, deepest leaf at depth 15
print(f"in-order:         {[k for k, _ in sorted_tree.in_order()][:8]}...")
# [0, 1, 2, 3, 4, 5, 6, 7]... - still sorted (it always is) but the shape is a list
```

```
sorted insertion: 16 keys, height = 15
in-order:         [0, 1, 2, 3, 4, 5, 6, 7]...
```

The tree is structurally a linked list — every left child is `None`, every right child is the next integer. If you search for key 15 it walks all sixteen nodes; that's a $\Theta(n)$ operation hiding inside an algorithm meant to be $\Theta(\log n)$. In-order is still sorted; the cost shape has collapsed.

The same sixteen keys, shuffled by a fixed seed for reproducibility:

```
keys = list(range(16))
random.seed(42)
random.shuffle(keys)
print(f"shuffle order: {keys}")
# [7, 9, 5, 6, 14, 10, 12, 8, 1, 2, 13, 15, 4, 11, 0, 3]

shuffled_tree: BST[int, int] = BST()
for k in keys:
    shuffled_tree.insert(k, k)

print(f"shuffled insertion: 16 keys, height = {shuffled_tree.height()}")
# 5 - close to log2(16) = 4, dramatically better than the sorted chain
print(f"in-order:           {[k for k, _ in shuffled_tree.in_order()][:8]}...")
# [0, 1, 2, 3, 4, 5, 6, 7]... - same sorted output, completely different shape
```

```
shuffle order: [7, 9, 5, 6, 14, 10, 12, 8, 1, 2, 13, 15, 4, 11, 0, 3]
shuffled insertion: 16 keys, height = 5
in-order:           [0, 1, 2, 3, 4, 5, 6, 7]...
```

You get height 5 — within a constant factor of $\log_2 16 = 4$. The sorted tree had height 15, exactly $n - 1$. Same data, same algorithm, two cost regimes separated by an order of magnitude, and the only difference is the order keys arrived in.

That is: a BST's cost depends on its insertion history. Knuth's classical result is that a BST built from a uniformly-random permutation has expected height $\approx 1.39 \log_2 n$, so random data lands close to the balanced ideal. But you don't always control the order data arrives in. Sorted streams, near-sorted streams, and adversarial sequences all push the height toward $n - 1$.

The reasonable response, then, is: don't trust your user to randomize the input. Build a structure that *enforces* its own balance no matter what order keys arrive in. That's chapter 17 — self-balancing BSTs that perform rotations on every insert and delete to keep the height bounded by $\Theta(\log n)$ regardless of input order. In-order is still sorted; only the shape-maintenance rule gets stricter.

16.5 The three questions, applied

Is it correct?

A BST is correct if, after any sequence of `insert` and `delete` operations, its in-order traversal still produces a sorted sequence of keys. That single check is equivalent to the structural invariant; the invariant is what makes the equivalence hold.

`insert` preserves the invariant because at every node, it descends into exactly the subtree the invariant requires the new key to live in — strictly-less keys to the left, strictly-greater keys to the right — and the new leaf is attached at a `None` slot where no in-order position needs shifting. `search` is correct because at every node, the invariant tells you where the answer must be: if the target is smaller than the current key, the answer is in the left subtree or nowhere; if larger, in the right subtree or nowhere.

`delete` carries the densest argument. The leaf case preserves the invariant because removing a leaf doesn't change any other key's position in the in-order order — the sequence just loses one element. The one-child case preserves it because the subtree being lifted is contiguous in the in-order order, and lifting it to a position where its ancestor used to sit doesn't change the relative ordering of any pair of remaining keys. The two-children case preserves it because the in-order successor is, by definition, the key that comes next in the sorted sequence — copying its payload into the deleted node's slot is exactly what the sorted sequence would look like with the deleted key removed.

How efficient is it?

The cost table for a tree of n nodes with height h:

Operation	Time	Extra space
`insert`	$\Theta(h)$	$\Theta(h)$ — recursion stack
`search` / `__contains__`	$\Theta(h)$	$\Theta(1)$ — iterative walk
`delete`	$\Theta(h)$	$\Theta(h)$ — recursion stack
`in_order`	$\Theta(n)$	$\Theta(h)$ — generator stack
`height`	$\Theta(n)$	$\Theta(h)$
`__len__`	$\Theta(1)$	$\Theta(1)$

That is: every single-key operation costs one root-to-leaf walk. For a balanced tree where $h = \Theta(\log n)$, that's about twenty comparisons for a million keys — the same shape as binary search on a sorted array, but with the structure preserved across modifications. For a worst-case tree where $h = n - 1$, those operations degrade to $\Theta(n)$ and you'd have been better off using a sorted array with binary lookup.

The expected behaviour under random insertions is what Knuth's $1.39 \log_2 n$ height bound captures — the constant is just above $\log 2$, so a random BST lands within roughly 40% of the perfectly-balanced height. Python's `dict` and `sorted` are both built around different ideas (hashing and merge-sort, respectively) precisely because BST performance is *too dependent on input order* for you to want it as a default choice for a general dictionary.

Is it optimal?

For the comparison-based model on arbitrary input, **a balanced BST is optimal**. Any data structure supporting ordered insert, search, and delete on n keys must use $\Omega(\log n)$ comparisons per operation in the worst case. The information-theoretic argument is short: distinguishing among n keys takes $\log_2 n$ yes/no answers, and a comparison is one yes/no answer. So $\Theta(\log n)$ per operation is tight at the model level, and a balanced BST achieves it.

The BST in *this* chapter is not optimal — its worst-case height is $\Theta(n)$, which makes its worst-case operations $\Theta(n)$. The gap between "balanced BST" and "BST that happens to be balanced for now" is exactly what chapter 17 closes by adding rotation-based rebalancing. A red-black tree, AVL tree, or treap will give you $\Theta(\log n)$ per operation **regardless of insertion order**, and at that point you're at the comparison-based lower bound.

For *some* workloads, a hash table beats a balanced BST (average $O(1)$ vs. $O(\log n)$), but only because the hash table gives up the ordering. You can't ask a hash table for "the smallest key", "all keys in `[lo, hi]`", or "the in-order traversal". The BST keeps those queries cheap; the hash table doesn't have them at all. The right comparison is that **balanced BSTs are optimal among comparison-based ordered dictionaries**, and that's the niche the rest of Part III lives in.

16.6 One ordering rule, three new operations

The in-order traversal of a BST is sorted — that's the entire structure restated. Every operation here was designed to preserve that traversal. Insert descended into the half the invariant demanded; search descended into the half the invariant guaranteed; delete's three cases were exactly the cases that keep the sorted sequence intact. Structural rule and traversal output: same fact, told twice.

A single ordering rule is enough to turn a substrate into a structure. Chapter 15's binary tree had no rule; you could put any value in any node. Chapter 16's BST adds *one* constraint, and lookup, insert, and delete all become $\Theta(\log n)$ on a well-shaped tree. The leap from $\Theta(n)$ to $\Theta(\log n)$ in chapter 2 needed a sorted array; here you get the same leap on data that *changes*, paid for by the same constraint applied locally at every node.

Recursive helpers that return the new subtree root are the lingua franca of tree-mutating algorithms. Insert and delete both used the same `_op(node, ...) -> new_node` shape, and the same shape carries through AVL rotations and treap operations in the next chapters. Once that idiom is internalized, the recursion writes itself.

A structural invariant is not a performance guarantee. The BST property says only that keys are sorted by position; it says nothing about balance. The pathological insertion in section four was a worst case that the invariant permits, and a structure whose worst-case cost matters needs *additional* rules beyond the structural one.

Chapter 17 is the next rule. By adding a small bit of rebalancing logic — a constant-time rotation triggered when a subtree's height gets out of whack — you can force every insert and delete to leave behind a tree of height $\Theta(\log n)$, no matter what order keys arrive in. The unbalanced worst case is what rotations fix.

16.7 Notes and further reading

Binary search trees are covered in CLRS (4th ed.) chapter 12 as the canonical comparison-based ordered dictionary, with the three-case delete laid out exactly as I've done here. Sedgewick and Wayne's *Algorithms* (4th ed.) §3.2 is the cleanest pedagogical treatment of insert and search, and is where Knuth's $1.39 \log_2 n$ expected-height result for random-order insertion is laid out in student-friendly form; Sedgewick attributes the BST as a data structure to independent work in the late 1950s by P.F. Windley, A.D. Booth, A.J.T. Colin, and others. Knuth's *The Art of Computer Programming*, vol. 3 (Sorting and Searching), §6.2.2 has the original expected-height analysis. The deletion algorithm in this chapter (in-order-successor variant) is sometimes called *Hibbard deletion* after T.N. Hibbard's 1962 paper "Some Combinatorial Properties of Certain Trees with Applications to Searching and Sorting"; it's known to leave the tree slightly less balanced than alternating successor/predecessor strategies, but the asymptotic cost is the same and the code is half the size.

17 Rotations buy you log n

Chapter 16 closed with a small disaster. You took a BST, fed it the keys $0, 1, \ldots, 15$ in sorted order, and watched the height climb to fifteen. The result was a chain of sixteen nodes, every "right" pointer doing work a balanced tree would have spread across both subtrees. The in-order traversal was still sorted, but the cost shape had collapsed from $\Theta(\log n)$ back to $\Theta(n)$. Lookup became linear, the structure became a linked list, and the BST property — order is *all* the rule says — turned out not to be enough.

One integer per node — the height — is enough to keep the tree balanced. Every node carries a single extra field, every insertion recomputes a handful of those fields on its way back up the recursion, and every imbalance bigger than one is fixed by a constant-time pointer-shuffle called a rotation. That's it. One integer, four rotation shapes, and the worst case stops being a worst case. A generic `AVL[K, V]` with insert, search, and the same three-case delete as chapter 16 turns a sorted insertion of $0, 1, \ldots, 127$ into a tree of height at most eight.

17.1 The unbalanced problem, recovered

A plain BST's height depends on the order keys arrived in. Sorted input gave you a chain of n nodes; shuffled input gave you a tree of height $\approx \log_2 n$. The structural rule (smaller on the left, bigger on the right) said nothing about *shape*, so an adversary feeding you sorted keys could drive every operation to $\Theta(n)$.

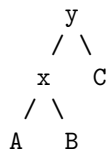
The fix has to be local. You can't periodically rebuild the whole tree — that would cost $\Theta(n)$ on the rebuild step and defeat the point of a dynamic structure. You can't afford to scan the whole tree on every operation either. What you need is a property the tree can *maintain* during insert and delete, using only information visible at the node you're touching, that *forces* the height to be $\Theta(\log n)$ no matter what order keys arrived in.

The trick is to track one extra number per node — the height of the subtree rooted there — and to enforce a tight inequality on that number at every node. When an insert or delete makes the inequality fail, you fix the violation with a $\Theta(1)$ pointer-shuffle called a **rotation**. The rotation preserves the BST invariant (the in-order traversal stays sorted) and reduces the offending subtree's height by one, which is exactly what the rebalancing needs.

17.2 Rotations: the four shapes

A rotation is a local pointer-shuffle that changes the *shape* of a small piece of the tree without changing the *order* of its keys. There are four shapes, but only two are genuinely distinct: left and right are mirror images of each other, and the two composite rotations are just one of those followed by the other.

Before a right rotation, the subtree looks like this:



You can read `y` as heavier on its left side — its left child `x` is the tall one. After a right rotation at `y`, `x` becomes the new subtree root and `y` slides down into the right slot:

```
      x
     / \
    A   y
       / \
      B   C
```

The keys stay in the same in-order positions before and after. You can read off `A < x < B < y < C` before the rotation; the same chain still holds after. So the BST invariant survives. The new root `x` ends up at most one level shorter than the old root `y`, which is the rebalancing payoff. Three pointer writes, a couple of height recomputations, and the subtree got shorter.

The code follows the picture exactly. Each rotation is a module-level helper — none of them need wrapper state — that takes a node and returns the new subtree root; the caller is on the hook for splicing that new root back in.

Three small helpers come first. The "height of a possibly-null subtree" comes up everywhere, so it lives in a function rather than scattering `if node is None` checks through the rotation code.

```
from typing import Iterator


class AVLNode[K, V]:
    def __init__(
        self,
        key: K,
        value: V,
        left: "AVLNode[K, V] | None" = None,
        right: "AVLNode[K, V] | None" = None,
    ) -> None:
        self.key = key
        self.value = value
        self.left = left
        self.right = right
        self.height = 0  # leaf height under the root-at-depth-0 convention
```

The node carries one extra field beyond chapter 16's `BSTNode`: `height`, the height of the subtree rooted at this node. A freshly-allocated leaf has height 0 — that's the convention from chapter 15 (root at depth 0, height is the maximum depth in the subtree). The forward-referenced string annotations are the same Python detail from chapter 16.

The three height helpers fit in nine lines combined, and every rotation and every rebalance step leans on all three.

```
def _height(node: "AVLNode | None") -> int:
    return node.height if node is not None else -1


def _update_height(node: "AVLNode") -> None:
    node.height = 1 + max(_height(node.left), _height(node.right))


def _balance_factor(node: "AVLNode") -> int:
    return _height(node.left) - _height(node.right)
```

The empty subtree returns -1 so that a single-node subtree (a leaf) ends up at height 0 — same convention as chapter 16's `BST.height`. The balance factor is the left-minus-right height difference; an AVL tree requires that value to be in $\{-1, 0, +1\}$ at every node. Anything outside that range is what triggers a rotation.

Right rotation is the workhorse. It assumes the input node has a non-null left child (you wouldn't call it otherwise — there'd be no left subtree to rotate up). Three pointer writes, two height recomputes, return the new root.

```
def _rotate_right(node: "AVLNode") -> "AVLNode":
    assert node.left is not None  # caller's responsibility
    new_root = node.left
    node.left = new_root.right    # B in the diagram
    new_root.right = node
    _update_height(node)          # update the descended node first
    _update_height(new_root)      # then the new root (depends on the descended node's height)
    return new_root
```

The order of the two height updates matters. `node` has just become the right child of `new_root`, so its height depends on its (possibly-rewired) left and right subtrees and must be recomputed *first*. Then `new_root`'s height depends on `node`'s new height, so it's recomputed *second*. Get the order wrong and you'll cache a stale height in `new_root` for one rotation.

Left rotation is the mirror — same code with `left` and `right` swapped, the same operation reflected across the vertical axis.

```
def _rotate_left(node: "AVLNode") -> "AVLNode":
    assert node.right is not None
    new_root = node.right
    node.right = new_root.left
    new_root.left = node
    _update_height(node)
    _update_height(new_root)
    return new_root
```

The two double rotations don't need their own functions — they're each just one primitive rotation followed by another. The left-right double rotation (used when a subtree's left child is heavier on *its* right side) is `_rotate_left` on the left child, then `_rotate_right` on the original node. The right-left double rotation is the mirror. Both compositions live inline inside the rebalance step in the next section, where the "which case am I in?" decision lives.

Each rotation is $\Theta(1)$ work — three pointer writes, two height recomputations. That is: a rotation is the cheapest possible local fix you can make to a tree's shape. The whole AVL story is about doing $\Theta(\log n)$ of these on the way back up from an insert or delete, and proving that's enough to keep the tree balanced.

17.3 AVL: height as invariant

The AVL property, stated formally: a binary search tree is an **AVL tree** if, at every node, the balance factor (left subtree height minus right subtree height) is in $\{-1, 0, +1\}$. That is: at every node, the two subtrees differ in height by at most one. Adelson-Velsky and Landis proved in 1962 that this single inequality, enforced everywhere, is enough to bound the tree's height at $O(\log n)$.

The operational claim is that you can maintain the property under insert and delete using only the same recursive descent as chapter 16, plus a rebalance step on the way back up. Insert descends to a `None` slot and allocates a new leaf (same as the BST). On the way back up the recursion, each ancestor recomputes its height, checks its balance factor, and — if the factor is out of range — applies the appropriate rotation to restore the AVL property at that node.

The four imbalance cases are distinguished by the balance factor at the offending node and the balance factor at its taller child:

- **LL** — node is left-heavy ($\text{bf} = +2$), left child is left-heavy or balanced ($\text{bf} \geq 0$). Fix: one right rotation at the node.
- **LR** — node is left-heavy ($\text{bf} = +2$), left child is right-heavy ($\text{bf} < 0$). Fix: left rotation at the left child, then right rotation at the node.
- **RR** — node is right-heavy ($\text{bf} = -2$), right child is right-heavy or balanced ($\text{bf} \leq 0$). Fix: one left rotation at the node.
- **RL** — node is right-heavy ($\text{bf} = -2$), right child is left-heavy ($\text{bf} > 0$). Fix: right rotation at the right child, then left rotation at the node.

LL and RR are mirror images; LR and RL are mirror images. Two genuinely distinct shapes, dressed up in four cases by the symmetry of left and right.

You can read the rebalance helper as that case-split written out. It takes a node whose subtrees have just been updated by a recursive operation, recomputes the node's height, and either returns the node unchanged (already balanced) or returns a new subtree root after applying the right rotation pattern.

```
def _rebalance(node: "AVLNode") -> "AVLNode":
    _update_height(node)
    bf = _balance_factor(node)
    if bf > 1:                                  # left-heavy
        assert node.left is not None
        if _balance_factor(node.left) < 0:      # LR: left child is right-heavy
            node.left = _rotate_left(node.left)
        return _rotate_right(node)              # LL or LR final step
    if bf < -1:                                 # right-heavy
        assert node.right is not None
        if _balance_factor(node.right) > 0:     # RL: right child is left-heavy
            node.right = _rotate_right(node.right)
        return _rotate_left(node)               # RR or RL final step
    return node                                  # already balanced
```

You can read the two outer cases as left-heavy or right-heavy, each with an optional inner rotation on the child for the "double" variant, then a final rotation on the node itself. The asserts document the invariant: if `bf > 1` then there must be a left subtree taller than the right, so `node.left` is non-null.

The wrapper class mirrors chapter 16's `BST[K, V]` almost line for line: same `_root` and `_size` fields, same nested-helper idiom for insert and delete. The rebalance call on the way back up is the only structural change.

```
class AVL[K, V]:
    def __init__(self) -> None:
        self._root: AVLNode[K, V] | None = None
        self._size: int = 0

    def __len__(self) -> int:
        return self._size

    def height(self) -> int:
        return _height(self._root)
```

`height` reads the root's cached `height` field directly: $\Theta(1)$, no traversal needed. You get a small win over chapter 16's `BST.height`, which had to walk the whole tree.

Insert is the same recursive descent as the BST, with one extra call to `_rebalance` at the very end of the helper. The helper returns the (possibly new) subtree root, and the caller reassigns whichever pointer it descended through. Same functional-recursive idiom as chapter 16.

```
    def insert(self, key: K, value: V) -> None:
        def _insert(node: AVLNode[K, V] | None) -> AVLNode[K, V]:
            if node is None:
                self._size += 1
                return AVLNode(key, value)
            if key < node.key:
                node.left = _insert(node.left)
            elif key > node.key:
                node.right = _insert(node.right)
            else:
                node.value = value              # overwrite, no rebalance needed
                return node
            return _rebalance(node)

        self._root = _insert(self._root)
```

Three branches per node, just like chapter 16. The duplicate-key case returns the node immediately without rebalancing — overwriting a value doesn't change any height. The two recursive branches descend, then `_rebalance` runs on the way back up. Because it updates the height *before* checking the balance factor, that one call does double duty: it maintains the height field and fixes any imbalance.

Search and `__contains__` are unchanged from chapter 16; read-only operations don't care about heights or rebalancing.

```
    def search(self, key: K) -> V | None:
        node = self._root
        while node is not None:
            if key < node.key:
                node = node.left
            elif key > node.key:
                node = node.right
            else:
                return node.value
        return None

    def __contains__(self, key: K) -> bool:
        return self.search(key) is not None

    def in_order(self) -> Iterator[tuple[K, V]]:
        def _walk(node: AVLNode[K, V] | None) -> Iterator[tuple[K, V]]:
            if node is None:
                return
            yield from _walk(node.left)
            yield (node.key, node.value)
            yield from _walk(node.right)

        yield from _walk(self._root)
```

Three lookups, an iterator, a containment check. Nothing here changes shape — the AVL property is something the tree *maintains internally*, not something you have to know about to query it. From outside, the AVL behaves like the BST of chapter 16, just with a tighter cost guarantee.

Delete is the same three-case method from chapter 16, with the same successor-finding helper, plus a rebalance call on every recursive return. The successor finder is a one-liner:

```
    def _min_node(self, node: AVLNode[K, V]) -> AVLNode[K, V]:
        while node.left is not None:
            node = node.left
        return node
```

Same as chapter 16: walk left until you can't. The leftmost descendant of any subtree holds its smallest key.

Delete itself is the three-case dispatch from chapter 16, threaded through the same `_rebalance` call as insert. The leaf case folds into the one-child case folds into the two-children case, where the in-order successor's payload gets copied up and the successor itself is recursively deleted from the right subtree.

```
    def delete(self, key: K) -> None:
        def _delete(
            node: AVLNode[K, V] | None, k: K
        ) -> AVLNode[K, V] | None:
            if node is None:
                return None
            if k < node.key:
                node.left = _delete(node.left, k)
            elif k > node.key:
                node.right = _delete(node.right, k)
            else:
```

```
                if node.left is None:
                    self._size -= 1
                    return node.right
                if node.right is None:
                    self._size -= 1
                    return node.left
                succ = self._min_node(node.right)
                node.key = succ.key
                node.value = succ.value
                node.right = _delete(node.right, succ.key)
            return _rebalance(node)

        self._root = _delete(self._root, key)
```

Same three cases as chapter 16 — leaf, one child, two children — with the same trick that the leaf case and the one-child case fall out of the same two `return` lines. The structural difference is the `return _rebalance(node)` at the bottom of the helper. Whenever the recursion returns a node (rather than `None` or a spliced child), that node goes through the rebalance step. That keeps the AVL property invariant under deletion the same way it does under insertion: recompute the height, check the balance factor, rotate if needed.

A caution on delete: a single delete can require *multiple* rotations on the way up to the root. Insertion is the easier case because at most one rotation is needed (after a successful rotation, the subtree's height returns to what it was before the insert, so the imbalance doesn't propagate). Deletion isn't so kind — removing a node can reduce a subtree's height by one, which can in turn unbalance its parent, which can in turn unbalance *its* parent, all the way up. That's why AVL delete is $O(\log n)$ rotations in the worst case rather than $O(1)$, and it's the main reason production code prefers red-black trees (sidebar coming).

The one-rotation case: insert `[1, 2, 3]` into an empty AVL. After inserting 1 and 2, the root is a right-leaning V. After inserting 3, the root's balance factor hits -2 and a left rotation fixes it, leaving 2 at the root with 1 and 3 as children.

```
from codex.trees.avl import AVL

tree: AVL[int, str] = AVL()
for k in [1, 2]:
    tree.insert(k, f"v{k}")

print(f"after [1, 2]:    root key = {tree._root.key}, height = {tree.height()}")
# root = 1, height = 1 - the right-leaning V, balance factor -1 (still legal)

tree.insert(3, "v3")
print(f"after [1, 2, 3]: root key = {tree._root.key}, height = {tree.height()}")
# root = 2, height = 1 - a left rotation at the root rebalanced the chain
print(f"in-order:        {[k for k, _ in tree.in_order()]}")
# [1, 2, 3] - still sorted; rotations preserve the BST invariant
```

```
after [1, 2]:    root key = 1, height = 1
after [1, 2, 3]: root key = 2, height = 1
in-order:        [1, 2, 3]
```

The third insert triggered the rotation. The root changed from 1 to 2, the height dropped from 2 (chain) to 1 (balanced triangle), and the in-order stayed sorted — the BST invariant still holds, and now the height stays bounded too.

The chapter-16 sorted-insertion experiment, redone with AVL: sixteen keys, 0 through 15, fed in sorted order. Chapter 16's plain BST gave you height 15; AVL should keep the height close to $\log_2 16 = 4$.

```
sorted_avl: AVL[int, int] = AVL()
for k in range(16):
    sorted_avl.insert(k, k)
```

```
print(f"sorted insertion of 0..15: height = {sorted_avl.height()}")
# 4 - exactly log_2(16), compared to chapter 16's BST which hit height 15
print(f"in-order: {[k for k, _ in sorted_avl.in_order()]}")
# [0, 1, ..., 15] - sorted, same as the BST would have produced
```

```
sorted insertion of 0..15: height = 4
in-order: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
```

Height 4 instead of 15 — almost 4× shorter on the same input, and the gap widens as n grows. One integer per node, recomputed on every insert, kept the tree balanced across every one of those sixteen pathological insertions.

Inserting 0 through 127 in sorted order pushes the demo further. A balanced binary tree of 128 nodes has height 7 ($\log_2 128 = 7$). The AVL bound (proven in the next section) is $h \leq 1.44 \log_2(n+2) \approx 10$ for $n = 128$, so anything up to 10 is conformant. In practice on sorted input, AVL hits the perfect height of 7.

```
big_avl: AVL[int, int] = AVL()
for k in range(128):
    big_avl.insert(k, k)

print(f"sorted insertion of 0..127: height = {big_avl.height()}")
# 7 - exactly the balanced ideal, log_2(128)
print(f"size:        {len(big_avl)}")          # 128
in_order = [k for k, _ in big_avl.in_order()]
print(f"sorted?      {in_order == list(range(128))}")  # True
```

```
sorted insertion of 0..127: height = 7
size:        128
sorted?      True
```

A pathological insertion pattern — the same one that gave the BST a chain of 128 nodes — gave the AVL the perfectly balanced ideal. That isn't an accident of this particular input; it's what the AVL property buys you on *every* input, which is what the next section proves.

17.4 Why the height bound holds

The AVL property bounds height. The proof is a counting argument that turns into the Fibonacci recurrence, and once you see it, you understand why 1.44 is the precise constant in the height bound.

The question to answer: among all AVL trees of a given height h, what's the *minimum* number of nodes any of them can have? Call that minimum $N(h)$. If you can show $N(h)$ grows exponentially in h, then the inverse — *for any n nodes, the maximum height is logarithmic* — falls out by inverting the function.

Build the minimum AVL tree of height h recursively. Its root has two subtrees, and at least one of them must have height exactly $h-1$ (otherwise the tree's height wouldn't be h). The AVL property says the other subtree's height differs by at most one, so it's either $h-1$ or $h-2$. To minimize the total node count, pick the *shorter* one: $h-2$. To minimize each subtree, recursively pick the *minimum-node-count* AVL tree of that height. Counting:

$$N(h) = N(h-1) + N(h-2) + 1$$

The $+1$ is the root. The base cases are $N(0) = 1$ (a single node, height 0) and $N(-1) = 0$ (the empty tree, height -1 under the convention from earlier).

Solve the recurrence and you'll recognize it. Define $F'(h) = N(h) + 1$. Then $F'(h) = F'(h-1) + F'(h-2)$, with $F'(0) = 2$ and $F'(-1) = 1$ — the Fibonacci recurrence shifted by a constant. So $N(h) = F_{h+3} - 1$, where F_k is the k-th Fibonacci number with $F_1 = F_2 = 1$.

The Fibonacci numbers grow like $\phi^k/\sqrt{5}$, where $\phi = (1+\sqrt{5})/2 \approx 1.618$ is the golden ratio. So:

$$N(h) \approx \phi^{h+3}/\sqrt{5} - 1$$

Inverting to solve for h in terms of n: any AVL tree of n nodes has $n \geq N(h)$, so $\phi^{h+3} \leq \sqrt{5}(n+1)$, so:

$$h \leq \log_\phi(\sqrt{5}(n+1)) - 3 = \log_\phi(n+1) + O(1)$$

And $\log_\phi(n) = \log_2(n)/\log_2(\phi) \approx \log_2(n)/0.694 \approx 1.44 \log_2(n)$. So:

$$h \leq 1.44 \log_2(n+2)$$

That is: a million keys in an AVL tree have height at most about 28; a billion keys have height at most about 43. Compare that with a balanced binary tree's ideal of $\log_2 10^6 \approx 20$ and $\log_2 10^9 \approx 30$. AVL stays within 50% of the ideal in the worst case. The trees that hit the worst-case bound are called **Fibonacci trees**, the AVL analogue of the pathological chain that broke the plain BST in chapter 16.

That settles the height bound: every AVL tree of n nodes has $h = O(\log n)$. Every operation in this chapter walks one root-to-leaf path and does $O(1)$ work at each node, so every operation is $O(\log n)$. The constant in the bound is 1.44, not 1.0, but you get a constant.

17.5 Sidebar: why production picks Red-Black

The other self-balancing BST you'll meet in the wild is the red-black tree, which production code reaches for. A red-black tree uses a different invariant. Each node carries one **color bit**, red or black, and four rules constrain the coloring: the root is black; no red node has a red child; every path from the root down to any null leaf contains the same number of black nodes; and (in some formulations) every red node's children are non-null. You can read the fourth rule as automatic in the textbook formulation that puts a black "null sentinel" at every empty position.

The coloring is what bounds the height. The two rules that matter for the bound are "no two reds in a row" and "every root-to-null path has the same number of blacks". You can read off the longest possible path from root to null as one that alternates red-black-red-black; the shortest is all-black. Taken together those two rules force the longest path to be at most twice the length of the shortest, which gives $h \leq 2 \log_2(n+1)$.

What makes red-black the production choice over AVL is the rotation count, not the height bound. You can read AVL's height bound as tighter ($1.44 \log_2 n$ vs $2 \log_2 n$), so AVL trees come out slightly shorter and you get slightly faster lookups in the worst case. But AVL's *update* cost is worse: deletion can trigger $O(\log n)$ rotations on the way up to the root, because a deletion can reduce a subtree's height and propagate the imbalance arbitrarily high. Red-black deletion needs at most 3 rotations, and red-black insertion needs at most 2 — both $O(1)$ in the worst case, with the rest of the bookkeeping handled by recoloring (cheaper than rotating).

That difference is decisive at scale. The C++ STL's `std::map` and `std::set` are red-black trees. Java's `TreeMap` and `TreeSet` are red-black trees. The Linux kernel's `rbtree` — used by the Completely Fair Scheduler, by the I/O scheduler, by the memory-region tracker, and a dozen other places — is a red-black tree. The slightly looser height bound costs you a handful of comparisons per lookup; the tighter rotation bound saves you arbitrarily many writes per update. In a read-heavy workload AVL would win; in a workload that mixes reads and writes, red-black wins on aggregate cost, and that's the workload most production code lives in.

A pedagogical variant: Sedgewick's 2008 **left-leaning red-black tree** — a restricted form of red-black that disallows right-leaning red links — implements the same invariant with about half the code of the classical formulation, and *Algorithms* (4th ed.) uses it to teach the structure. For learning red-black trees from scratch, the left-leaning variant is the gentler entry point. The asymptotic guarantees are identical to the classical formulation.

AVL is the cleaner pedagogy; red-black is the cleaner production engineering.

17.6 The three questions, applied

Is it correct?

An AVL tree is correct as a BST if, after any sequence of `insert` and `delete` operations, the in-order traversal produces a sorted sequence of keys. It is correct as an *AVL* tree if, in addition, every node's balance factor sits in $\{-1, 0, +1\}$. The two correctness criteria stack: the BST invariant guarantees the right *answers*; the AVL invariant guarantees the right *cost shape*.

The BST correctness argument is the same as chapter 16's. Every insert descends into the subtree the invariant requires, and every delete preserves the in-order ordering by either removing a leaf, splicing a single child up, or copying the in-order successor's payload into the deleted slot. Rotations don't change in-order ordering — the rotation picture from section 2 is the proof, since `A < x < B < y < C` holds before and after — so the BST invariant survives every rebalance step. The AVL invariant holds because `_rebalance` recomputes the height and applies the rotation pattern that fixes the imbalance, and the four rotation cases (LL, LR, RR, RL) exhaust the ways a single insert or delete can drive a balance factor outside the legal range.

How efficient is it?

Cost table for an AVL tree of n nodes:

Operation	Time	Extra space
`insert`	$O(\log n)$	$O(\log n)$ — recursion stack
`search` / `__contains__`	$O(\log n)$	$O(1)$ — iterative walk
`delete`	$O(\log n)$	$O(\log n)$ — recursion stack
`in_order`	$\Theta(n)$	$O(\log n)$ — generator stack
`height`	$O(1)$	$O(1)$ — read the cached field
`__len__`	$O(1)$	$O(1)$

That is: every single-key operation you run is bounded by $1.44 \log_2(n + 2)$ comparisons in the worst case, with the same root-to-leaf cost shape as a balanced BST. For a million keys that's about 28 comparisons; for a billion, about 43. The constant overhead is a handful of height field updates and at most a few rotations per operation (one for insert, up to $O(\log n)$ for delete), where each rotation is three pointer writes and two arithmetic operations.

The space cost on the call stack is $O(\log n)$: the recursion depth equals the tree's height, which the AVL property bounds at $1.44 \log_2 n$. You won't see the surprise stack blow-up a chapter-16 BST risked on sorted input.

Is it optimal?

For comparison-based ordered dictionaries on arbitrary input, an AVL tree is **asymptotically optimal**. The information-theoretic lower bound from chapter 16 still holds: any structure supporting ordered insert, search, and delete on n keys must do $\Omega(\log n)$ comparisons per operation in the worst case, and an AVL tree achieves $O(\log n)$. The constant 1.44 in the height bound isn't optimal — a perfectly balanced binary tree would give you constant 1.0 — but the *asymptotic* class is.

Among self-balancing BSTs, AVL isn't always the best constant. Red-black trees pay a slightly higher height constant (2.0 vs 1.44) in exchange for an $O(1)$ rotation bound per insert and delete, which buys you speed in update-heavy workloads. Treaps (chapter 18) pay no extra storage at all — no height, no color — and trade worst-case bounds for expected-case bounds that match. Splay trees pay even less but give up worst-case guarantees entirely in exchange for amortized $O(\log n)$ that adapts to access patterns. Each is the right answer you'd reach for in a different workload.

What an AVL tree gives you that the alternatives don't is the *tightest* worst-case height bound among rotation-based self-balancing trees. If you want a tree that's never taller than $1.44 \log_2 n$ no matter what, and you're willing to pay for it in update cost, AVL is the structure to reach for.

17.7 A local rule, a global cost shape

One integer per node — the height — is enough to keep the tree balanced. Every node carries one extra field; every insert and delete recomputes a handful of those fields on its way back up the recursion; every imbalance bigger than one gets fixed by a constant-time pointer-shuffle called a rotation. The AVL property (balance factor in $\{-1, 0, +1\}$) is the local rule, and the Fibonacci-tree counting argument is what turns that local rule into a global $1.44 \log_2 n$ height bound.

A local invariant can deliver a global cost shape. The AVL property is checkable at one node with one integer-subtraction, and yet enforcing it everywhere guarantees the *whole tree* is logarithmically tall. That pattern — local rule, global guarantee — shows up in red-black trees, in B-trees, in segment trees, in nearly every structure in the rest of Part III.

Rotations are how trees fix themselves. A rotation is the cheapest possible local change to a tree's shape that preserves an ordering invariant, and once you've seen the four AVL cases, the rotation idiom carries forward to splay trees, treaps, red-black trees, and beyond.

Worst-case guarantees cost something in update overhead, and that cost is sometimes the right thing to pay and sometimes not. AVL trees pay for their tight height bound with $O(\log n)$ rotations per delete; red-black trees trade a looser bound for a constant rotation count and win in practice. Knowing which trade you're making is the engineer's contribution.

Chapter 18 is the variant that gives up *explicit* rotations on every step and replaces them with *probabilistic* balance. A treap assigns each node a random priority on insertion, and maintains a heap order on those priorities while keeping the BST order on the keys. The randomness alone gives you $O(\log n)$ expected height, no balance factor needed — a different way of buying the same guarantee.

17.8 Notes and further reading

The AVL tree is the original self-balancing BST, introduced by Georgy Adelson-Velsky and Evgenii Landis in 1962 in their paper "An algorithm for the organization of information," published in *Doklady Akademii Nauk SSSR* — the name "AVL" is their initials. CLRS (4th ed.) §13 covers red-black trees as its self-balancing canon and treats AVL only briefly in the exercises; Sedgewick and Wayne's *Algorithms* (4th ed.) §3.3 develops the left-leaning red-black variant Sedgewick introduced in 2008, alongside the 2-3 and 2-3-4 trees that motivate it. Knuth's *The Art of Computer Programming*, vol. 3, §6.2.3 has the canonical analysis of AVL trees including the Fibonacci-tree height bound derivation sketched in section 4. The choice between AVL and red-black in practice is discussed in Pfaff's 2004 paper "Performance Analysis of BSTs in System Software," which benchmarks both against real kernel workloads and lands on red-black for the same reasons the sidebar sketched.

18 Random priorities, balanced trees

Chapter 17 bought a worst-case $O(\log n)$ height bound with a single integer per node and a careful four-case rotation table. The price was real — every insert and delete recomputes heights up the recursion, every imbalance triggers a rotation case-split, and a single delete can fire $O(\log n)$ rotations on its way back to the root. The structure works, but you carried a fair amount of bookkeeping to make it work, and the question I want to open with is whether there's a cheaper way to buy the same guarantee.

Two invariants at once — BST on key, max-heap on priority — and that's what randomization buys you. One node, two ordering rules. The BST rule does what BSTs always do. The heap rule, applied to *random* priorities, is the rebalancing mechanism in disguise. The generic `Treap[K, V]` developed below maintains both invariants on every operation, and a sorted insertion of $0, 1, \ldots, 15$ — the same input that drove chapter 16's BST into a chain — produces a tree of height comparable to chapter 17's AVL.

18.1 Determinism's price

Chapter 17's AVL tree is correct, tight, and a bit fussy. Each node carries a `height` field, every operation recomputes it on the way up, and the `_rebalance` step does a four-case split — LL, LR, RR, RL — to decide which rotation pattern restores the invariant. Insertion is one rotation in the worst case; deletion is up to $\log_2 n$ rotations because an imbalance can propagate arbitrarily high. You pay the determinism in two ways: storage (one integer per node) and code complexity (the case table). Both are small in absolute terms, but they're not zero.

The natural question is whether you can drop both costs and still get $O(\log n)$ per operation. The deterministic answer says no — an adversary who controls insertion order can drive any *predictable* BST shape into a chain, just like chapter 16 demonstrated. But the deterministic question is the wrong question. If you let the structure make a *random* choice on every insertion, the adversary stops being able to predict the shape, and the worst case stops being constructible. The expected cost becomes $O(\log n)$ on every input, including the adversarially-crafted ones that broke chapter 16's BST.

A treap (the name fuses *tree* + *heap*) attaches a random priority to each node at insertion time, and maintains a max-heap order on those priorities alongside the BST order on the keys. The heap order is what keeps the tree shallow; the randomness of the priorities is what makes "shallow" hold in expectation, regardless of insertion order. No height field, no balance factor, no four-case table — just a comparison against a random number and a rotation when the heap order is violated.

18.2 Two invariants, one node

A treap is a binary tree where each node carries a key k, a value, and a priority p. The structure obeys two ordering rules at every node:

- **BST on key.** Every key in the left subtree is strictly less than the node's key; every key in the right subtree is strictly greater. Same rule as chapter 16.
- **Max-heap on priority.** The node's priority is greater than or equal to the priorities of its two children. The same heap rule you'd see at every level of a max-heap (chapter 19 develops it as a structure in its own right).

Those two rules, taken together, determine the tree's shape *uniquely*. Given a set of (k, p) pairs with distinct keys and distinct priorities, there is exactly one binary tree that satisfies both rules. You can prove it by induction on set size: the root has to be the node with the largest priority (forced by the heap rule), the left subtree is recursively the treap of the keys less than the root, and the right subtree is recursively the treap of the keys greater. Both subtrees are uniquely determined by recursion. So the (key, priority) multiset *is* the tree, up to the shape that the rules force.

That uniqueness has a second consequence. **A treap is the unique BST you'd build by inserting keys in priority order, highest priority first.** If you took the same set of nodes and inserted them into a plain BST in *decreasing* priority order, the root would be the highest-priority node, then each subsequent insertion would attach to the appropriate side, and you'd produce exactly the treap. So the treap's shape *is* the shape of a BST built from a particular insertion order — namely, the order priorities induce on the keys.

When priorities are uniform-random and independent, that insertion order is a uniformly random permutation of the keys. And chapter 16 already told you what happens then: Knuth's result is that a BST built from a uniformly-random insertion sequence has expected height $\approx 1.39 \log_2 n$. So a treap with random priorities has expected height $\approx 1.39 \log_2 n$, with high probability. The randomness of the priorities is the randomness of the implied insertion order, and the implied insertion order is random regardless of what order keys *actually* arrived in.

That is: the adversary can pick any sequence of keys, but you pick the priorities, and the priorities are what determines the shape. Adversary loses.

18.3 Insert by rotate-up, delete by rotate-down

Two rotation primitives, same shape as chapter 17's but stripped of the height-update bookkeeping (a treap has no height field).

```
import random
from typing import Iterator


class TreapNode[K, V]:
    def __init__(
        self,
        key: K,
        value: V,
        priority: float,
        left: "TreapNode[K, V] | None" = None,
        right: "TreapNode[K, V] | None" = None,
    ) -> None:
        self.key = key
        self.value = value
        self.priority = priority
        self.left = left
        self.right = right
```

Each node carries a key, a value, a priority (a float drawn from $[0, 1)$ at insertion time), and the two child pointers. I'll use `random.Random` instances inside the wrapper class so that a caller can pass a seeded RNG and get reproducible trees — important for the demos below and for any debugging session where you'd want a specific tree to keep showing up.

Right rotation is the same primitive as chapter 17, with the height-update lines deleted. Three pointer writes; return the new subtree root.

```
def _rotate_right[K, V](node: TreapNode[K, V]) -> TreapNode[K, V]:
    assert node.left is not None
    new_root = node.left
    node.left = new_root.right
    new_root.right = node
    return new_root


def _rotate_left[K, V](node: TreapNode[K, V]) -> TreapNode[K, V]:
    assert node.right is not None
    new_root = node.right
    node.right = new_root.left
```

```
    new_root.left = node
    return new_root
```

Same in-order argument as chapter 17 — the keys keep their in-order positions, so the BST invariant survives every rotation. What changes between the two chapters is the *trigger*: AVL rotates when a balance factor goes out of range; a treap rotates when a child's priority exceeds the parent's.

Now the wrapper. I'll keep the same `_root` / `_size` shape as `BST` and `AVL`, plus a private `_rng` field so reproducible demos are cheap to write.

```
class Treap[K, V]:
    def __init__(self, rng: random.Random | None = None) -> None:
        self._root: TreapNode[K, V] | None = None
        self._size: int = 0
        self._rng: random.Random = rng if rng is not None else random.Random()

    def __len__(self) -> int:
        return self._size

    def height(self) -> int:
        def _h(node: TreapNode[K, V] | None) -> int:
            if node is None:
                return -1
            return 1 + max(_h(node.left), _h(node.right))

        return _h(self._root)
```

No cached height — I recompute it on demand, the same way chapter 16's `BST.height` did. A treap doesn't need height during operations (no balance factor to evaluate), so paying $\Theta(n)$ for a height query is the right trade. The single-key operations stay cheap.

Insertion is the BST descent plus a rotate-up on the way back. The new node enters as a leaf at the position the BST rule dictates, with a fresh random priority. On the recursive return, each ancestor checks whether the child it just recursed into now has a *higher* priority than itself; if so, the heap rule is violated and a rotation pulls the child up one level. The rotation preserves the BST rule (proved above) and reduces the priority gap by one node, so a chain of rotations bubbles the new node up until it lands at a position where the heap rule holds.

```
    def insert(self, key: K, value: V) -> None:
        priority = self._rng.random()
        self._root = self._insert(self._root, key, value, priority)

    def _insert(
        self,
        node: TreapNode[K, V] | None,
        key: K,
        value: V,
        priority: float,
    ) -> TreapNode[K, V]:
        if node is None:
            self._size += 1
            return TreapNode(key, value, priority)
        if key < node.key:
            node.left = self._insert(node.left, key, value, priority)
            if node.left.priority > node.priority:
                return _rotate_right(node)
        elif key > node.key:
            node.right = self._insert(node.right, key, value, priority)
            if node.right.priority > node.priority:
                return _rotate_left(node)
        else:
```

```
            node.value = value  # overwrite, no priority change, no rotation
        return node
```

Three branches per node, just like AVL. The two recursive branches descend BST-style, then on the way back up check whether the child's priority now exceeds the parent's — if so, rotate to bring it up. A right rotation pulls the *left* child up; a left rotation pulls the *right* child up. The duplicate-key case overwrites the value and returns without touching priorities; the heap rule is unaffected.

Reading the function from the bottom up: every recursive return passes through at most one rotation, and rotations only happen on the path the recursion just traversed. That path is the BST insertion path, length $O(\log n)$ in expectation, so insertion fires $O(\log n)$ rotations in the worst-case for *that single insert* — though the expected number of rotations per insert is $O(1)$ once you account for where in the tree the new node lands.

Search and `__contains__` are unchanged from chapter 16. The heap rule plays no role in lookup — only the BST rule matters when you're descending to find a key.

```
    def search(self, key: K) -> V | None:
        node = self._root
        while node is not None:
            if key < node.key:
                node = node.left
            elif key > node.key:
                node = node.right
            else:
                return node.value
        return None

    def __contains__(self, key: K) -> bool:
        return self.search(key) is not None

    def in_order(self) -> Iterator[tuple[K, V]]:
        def _walk(node: TreapNode[K, V] | None) -> Iterator[tuple[K, V]]:
            if node is None:
                return
            yield from _walk(node.left)
            yield (node.key, node.value)
            yield from _walk(node.right)

        yield from _walk(self._root)
```

Iterative search, generator in-order — same idioms as chapters 16 and 17. The treap's distinguishing structure is invisible to a read-only client.

Deletion inverts insertion. Instead of bubbling a new node *up* into a heap-legal position, I'll bubble the target node *down* into a leaf position and snip it off. The trick at each step: rotate the target past whichever of its children has the higher priority. That preserves the heap rule for the children — they each remain greater than or equal to their *own* children — and pushes the target one level deeper. Repeat until the target has no children left, then unlink it.

```
    def delete(self, key: K) -> None:
        self._root = self._delete(self._root, key)

    def _delete(
        self,
        node: TreapNode[K, V] | None,
        key: K,
    ) -> TreapNode[K, V] | None:
        if node is None:
            return None  # key not present - silent no-op
        if key < node.key:
            node.left = self._delete(node.left, key)
```

```
            return node
        if key > node.key:
            node.right = self._delete(node.right, key)
            return node
        return self._rotate_down_and_remove(node)
```

BST descent until I find the match site, same as chapter 16. The match site is where the rotate-down work happens, and I've factored it into its own helper so the dispatch reads cleanly.

The rotate-down helper handles the three children-count cases. If the target has no children, just return `None` (the parent will splice the gap closed). If it has only one child, return that child (same single-child splice as chapter 16's BST deletion). If both children exist, rotate the target one level deeper — past whichever child has the higher priority, to preserve the heap rule at that subtree's root — and recurse on the rotated tree to keep pushing down.

```
    def _rotate_down_and_remove(
        self,
        node: TreapNode[K, V],
    ) -> TreapNode[K, V] | None:
        if node.left is None and node.right is None:
            self._size -= 1
            return None
        if node.left is None:
            self._size -= 1
            return node.right
        if node.right is None:
            self._size -= 1
            return node.left
        # both children - rotate target past the higher-priority child
        if node.left.priority > node.right.priority:
            new_root = _rotate_right(node)
            new_root.right = self._rotate_down_and_remove(node)
            return new_root
        new_root = _rotate_left(node)
        new_root.left = self._rotate_down_and_remove(node)
        return new_root
```

Each rotation pushes the target node one level deeper. The recursion on `node` (now a child of `new_root`) keeps pushing until one of the leaf or one-child cases fires and the node is removed. The heap rule at the rotated-up child is preserved because that child *was* the higher-priority one, so it satisfies the heap rule with respect to the sibling that didn't rotate, and the recursion fixes the heap rule beneath the rotation.

Inserting `[5, 3, 8, 1, 9, 4, 7]` with a seeded RNG produces a deterministic tree — same priorities every time.

```
import random
from codex.trees.treap import Treap

rng = random.Random(42)
tree: Treap[int, str] = Treap(rng=rng)
for k in [5, 3, 8, 1, 9, 4, 7]:
    tree.insert(k, f"v{k}")

# In-order traversal - the BST rule says this must be sorted
print(f"in-order:   {[k for k, _ in tree.in_order()]}")
# [1, 3, 4, 5, 7, 8, 9] - sorted, as the BST invariant promises

# Walk the tree showing parent + L/R children - priorities decrease
# as you descend (heap rule), keys partition BST-style (BST rule)
def _walk(node, indent=0):
```

```
    if node is None:
        return
    l = node.left.key if node.left is not None else "·"
    r = node.right.key if node.right is not None else "·"
    print(f"{'  ' * indent}key={node.key}, prio={node.priority:.3f}, "
          f"L={l}, R={r}")
    _walk(node.left, indent + 1)
    _walk(node.right, indent + 1)

_walk(tree._root)
# Root key=7, prio=0.892 - the highest priority in the tree, as the heap
# rule demands. The keys split BST-style: 4 and friends on the left, 9
# and friends on the right.
print(f"height:     {tree.height()}")
```

```
in-order:   [1, 3, 4, 5, 7, 8, 9]
key=7, prio=0.892, L=4, R=9
  key=4, prio=0.677, L=1, R=5
    key=1, prio=0.223, L=·, R=3
      key=3, prio=0.025, L=·, R=·
    key=5, prio=0.639, L=·, R=·
  key=9, prio=0.736, L=8, R=·
    key=8, prio=0.275, L=·, R=·
height:     3
```

The in-order is sorted (BST rule) and the priorities decrease as you walk down from the root (heap rule). Both invariants visible at once on the same seven nodes.

The sorted-insertion experiment, redone with the treap. Same sixteen keys, 0 through 15, inserted in their natural order. The plain BST gave you height 15; the AVL gave you height 4. I expect the treap to land somewhere close to the AVL — random priorities should keep the height around $2 \log_2 16 = 8$ in expectation, often lower.

```
sorted_treap: Treap[int, int] = Treap(rng=random.Random(42))
for k in range(16):
    sorted_treap.insert(k, k)

print(f"sorted insertion of 0..15: height = {sorted_treap.height()}")
# 5 - close to AVL's 4 and dramatically better than the BST's 15
print(f"in-order: {[k for k, _ in sorted_treap.in_order()]}")
# [0, 1, ..., 15] - sorted, same as both the BST and the AVL produced
```

```
sorted insertion of 0..15: height = 5
in-order: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
```

Height 5 on the same sorted input that drives a plain BST to height 15 and an AVL to height 4. The treap landed one above the AVL — not as tight as AVL's deterministic bound, but within a small constant of it, with no balance-factor bookkeeping at all. That gap is the cost of buying balance with randomness rather than with rotations: a slightly larger constant in front of the $\log n$, in exchange for a much simpler implementation.

Scaling up: five different RNG seeds, each inserting `range(1000)` into a fresh treap, height read off at the end. Expected $\approx 2 \log_2 1000 \approx 20$, with some variance per seed.

```
heights: list[int] = []
for seed in [1, 2, 3, 4, 5]:
    t: Treap[int, int] = Treap(rng=random.Random(seed))
    for k in range(1000):
        t.insert(k, k)
    heights.append(t.height())
```

```
print(f"heights across 5 seeds: {heights}")
# [20, 21, 20, 22, 25] - cluster around 2·log_2(1000)  20.
# A plain BST on the same sorted input would have given you height 999.
print(f"mean: {sum(heights) / len(heights):.1f}, "
      f"max: {max(heights)}, min: {min(heights)}")
# mean: 21.6, max: 25, min: 20
```

```
heights across 5 seeds: [20, 21, 20, 22, 25]
mean: 21.6, max: 25, min: 20
```

Every treap landed at a height between 20 and 25 — roughly a factor of two off the perfectly-balanced ideal of $\log_2 1000 \approx 10$, on the same sorted input that drives a plain BST into a chain. The plain BST of chapter 16 would have given you height 999 on the exact same insertion sequence. That's the adversary defeated by randomness.

18.4 Expected height is O(log n)

The height bound holds in expectation. The full proof from Seidel and Aragon's 1996 paper is several pages of careful probability; the intuition that drives it sketches as follows, and the paper carries the details.

The argument reduces to a question about *depths*. Pick any node v in the treap — what's the expected depth of v? If I can show that any single node has expected depth $O(\log n)$, the expected height (which is the maximum depth over all nodes) is also $O(\log n)$ by a union-bound argument I'll wave at rather than execute.

Fix two nodes u and v in the treap, with v the one you're trying to bound. Sort all nodes by key, and let $|u - v|$ denote the *rank gap* between them — the number of keys (inclusive of u and v) you'd pass through walking from u to v in the sorted order. Seidel and Aragon's key observation: u is an ancestor of v in the treap if and only if u has the *highest priority* among all the nodes in the rank interval between u and v. The reason is the uniqueness theorem from earlier — the treap is the BST induced by inserting in priority order, so the first node from that interval to be "inserted" is the one whose key splits the interval at the treap, and that's the ancestor.

Now if priorities are uniform-random and independent, the probability that u has the highest priority among $|u - v|$ candidates is exactly $1/|u - v|$. So the expected number of ancestors v has is the sum over all other nodes u of $1/|u - v|$. Summing over a tree of n nodes (where rank gaps from v go from 1 to about n), you get a harmonic-series sum:

$$\mathbb{E}[\text{depth of } v] = \sum_{u \neq v} \frac{1}{|u - v|} \leq 2H_n = O(\log n)$$

where $H_n = 1 + 1/2 + \ldots + 1/n \approx \ln n$. That's the per-node depth bound. The factor of 2 comes from summing the harmonic series on both sides of v (nodes with smaller keys and nodes with larger keys, each contributing one tail of the harmonic).

The expected height of the whole tree is a tighter argument that needs to bound the *maximum* depth across all nodes, not just one node's depth. Seidel and Aragon show that the expected height is at most $4.31 \log_2 n$ — slightly worse than AVL's $1.44 \log_2 n$ deterministic bound, but in the same asymptotic class. And more importantly, the bound holds with high probability: the chance of seeing a treap of height much larger than $c \log n$ falls polynomially in n, so the tail is genuinely thin.

On a million keys, an AVL is at most about 28 nodes tall, a treap is *expected* to be at most about 90 nodes tall, and a plain BST on adversarial input can be 999,999 nodes tall. Randomization lifts you from the catastrophic case into the same asymptotic class as the deterministic structure, at the cost of a slightly larger constant.

18.5 The bridge to Part IX

Randomization-as-balance is the first instance of a pattern that recurs across sorting, hashing, graph search, and data structures, and Part IX picks it up systematically. The same trick — *make a random choice the adversary can't predict, and the worst case becomes statistically unreachable* — drives at least four structures or algorithms you'll meet again:

- **Treaps** (this chapter). Random priorities turn the BST's input-order pathology into an expected-case bound.
- **Skip lists** (Pugh, 1990). A randomized layered linked list that competes with self-balancing BSTs at $O(\log n)$ expected per operation, with even less rotation bookkeeping than a treap.
- **Randomized quicksort.** A random pivot makes the worst-case sorted-input quadratic blow-up of chapter 4's plain quicksort statistically unreachable. Expected $O(n \log n)$ on every input, including adversarial ones.
- **Rabin-Karp fingerprinting.** A randomly-chosen hash modulus makes the adversary unable to construct strings that collide on the rolling hash, which turns a $\Theta(nm)$ worst-case substring search into $O(n + m)$ in expectation.

What unifies these is the same observation. A deterministic algorithm has a worst case the adversary can construct *if* they can predict the algorithm's behavior on each input. Randomization breaks the prediction: the adversary picks the input, but you pick the random coins, and the adversary doesn't see your coins before committing to the input. Worst-case constructibility requires both the input and the coins to align, and the probability that they do is exponentially small in the algorithm's parameters.

The treap is the first textbook example because its randomization is so *local* — one float per node, decided at insertion time — and the payoff is so direct: a structure that's nearly as good as the deterministic AVL with about half the bookkeeping.

18.6 The three questions, applied

Is it correct?

A treap is correct as a BST if, after any sequence of `insert` and `delete` operations, the in-order traversal of its keys is sorted. It is correct as a *treap* if, in addition, every node's priority is greater than or equal to the priorities of its children. The two invariants stack: the BST rule guarantees the right *answers* to search and the right *order* across mutations; the heap rule on random priorities guarantees the right *cost shape* in expectation.

The BST rule survives every operation by the same argument as chapter 16. Insert descends BST-style into the slot the rule dictates. Delete either splices a child up (which moves a sorted contiguous chunk into the deleted slot — same as chapter 16's one-child case) or rotates the target down to a leaf and unlinks it. Rotations preserve the BST in-order ordering (chapter 17's picture proves it), so a chain of rotations preserves it too. The heap rule is the part the algorithms have to *actively* maintain: insert bubbles up by rotation until a parent has higher priority than the new node; delete bubbles down by rotation past the higher-priority child until the target is a leaf. Both maintenance steps terminate because each rotation moves the offending node strictly one level (up for insert, down for delete) and the tree has finite height.

How efficient is it?

Cost table for a treap of n nodes, expectations taken over the random priorities:

Operation	Expected time	Worst case	Extra space
`insert`	$O(\log n)$	$O(n)$	$O(\log n)$ — recursion stack
`search` / `__contains__`	$O(\log n)$	$O(n)$	$O(1)$ — iterative
`delete`	$O(\log n)$	$O(n)$	$O(\log n)$ — recursion stack
`in_order`	$\Theta(n)$	$\Theta(n)$	$O(\log n)$ — generator
`height`	$\Theta(n)$	$\Theta(n)$	$O(\log n)$

Operation	Expected time	Worst case	Extra space
`__len__`	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$

That is: every single-key operation on a treap is bounded by roughly $4.3 \log_2 n$ comparisons in expectation, regardless of insertion order. For a million keys that's about 86 comparisons; for a billion, about 129. The worst case is $\Theta(n)$ — the unlucky priorities case — but the probability of seeing it falls polynomially in n, so it never shows up in practice. Storage per node is one float for the priority and two child pointers; no balance factor, no color bit, no parent pointer.

The expected number of rotations per insert is $O(1)$ — not $O(\log n)$. That's a stronger statement than the height bound: even though the new node may sit at depth $O(\log n)$, the number of rotations on its way up is expected constant. The argument is again Seidel and Aragon's: a new node lands at a random depth, and the number of rotations equals the number of ancestors with priority lower than it, which is geometrically distributed. Expected constant rotations per insert is part of what makes the treap competitive with AVL in practice despite the looser height bound.

Is it optimal?

Among randomized self-balancing BSTs on the comparison-based model, **a treap is asymptotically optimal in expectation**. The information-theoretic lower bound from chapter 16 still holds: ordered insert, search, and delete each need $\Omega(\log n)$ comparisons per operation, and a treap achieves $O(\log n)$ in expectation. The constant 4.31 in the expected height bound is worse than AVL's 1.44 deterministic constant, but the asymptotic class is the same, and randomized algorithms pay a slightly worse constant in exchange for a simpler structure.

Skip lists (Pugh, 1990) — the other major randomized ordered-dictionary structure — achieve the same $O(\log n)$ expected bound with a different storage layout: $O(\log n)$ pointers per node on average, instead of two. In practice, skip lists are easier to implement lock-free in a concurrent setting (because they use only pointer writes, no rotations), and treaps are easier to implement sequentially (because they're a tree, not a layered list). Both are textbook randomized structures and both live in the same asymptotic class as AVL and red-black; the engineering choice between them is about implementation context, not asymptotic cost.

What a treap gives you that the deterministic structures don't is the *simplicity*. No balance factor, no color bit, no four-case rotation table — just a random float per node and a one-line rotation-up rule. That simplicity is what makes treaps a popular teaching structure and what makes them attractive for prototype implementations where bookkeeping cost matters more than peak constant-factor performance.

18.7 Randomization is the rebalancing mechanism in disguise

The BST rule does the ordered-lookup work the BST was designed for; the heap rule applied to *random* priorities does the rebalancing work, without you ever having to compute a balance factor or dispatch a four-case rotation table. The randomness of the priorities makes the implied insertion order uniform, and a uniformly-random insertion order is the regime where chapter 16's plain BST is already expected-balanced.

Randomization is a worst-case-tamer. When an adversary can construct an input that breaks a deterministic algorithm, hiding the algorithm's choices behind random coins reduces the worst case to expected-case behavior. Treaps, skip lists, randomized quicksort, and Rabin-Karp all share this signature, and Part IX picks the pattern up systematically.

Simplicity is a real engineering currency. A treap trades a constant factor in height (4.3 vs. 1.4) for a dramatic reduction in implementation complexity — one float per node, one rotation-up rule, no balance-factor case-split. In contexts where the code has to be small, readable, or easy to modify, the simplicity wins even when a slightly worse asymptotic constant comes with it.

And the right invariant for a problem is often two invariants. A treap's shape is determined by *both* the BST rule and the heap rule taken together — neither rule alone is enough. The same dual-invariant move shows up in segment trees (chapter 21) and in many tree structures from competitive programming. Composing two simple ordering rules into something whose behavior neither rule predicts on its own is an algorithm-design move worth remembering.

Chapter 19 takes the heap rule on its own and lets it carry an entire structure. A treap uses the heap order on priorities as a balancing trick; a *heap* uses the heap order on the data itself, and the priority *is* the whole story. The same ordering rule, used for a different job.

18.8 Notes and further reading

The treap was introduced by Cecilia Aragon and Raimund Seidel in their 1989 paper "Randomized Search Trees" at the IEEE Symposium on Foundations of Computer Science (FOCS '89), and the journal version appeared as "Randomized Search Trees" in *Algorithmica* 16(4–5):464–497, 1996 — the full expected-height analysis and the $O(1)$-expected-rotations-per-insert bound both come from that paper. Knuth's *The Art of Computer Programming*, vol. 3, §6.2.2 has the original analysis of the expected height of a random BST (the $1.39 \log_2 n$ result that the treap argument leans on), and Sedgewick and Flajolet's *An Introduction to the Analysis of Algorithms* §6 develops the same harmonic-series argument I sketched in section 4. William Pugh's 1990 paper "Skip Lists: A Probabilistic Alternative to Balanced Trees" in *Communications of the ACM* 33(6):668–676 is the canonical reference for the other major randomized ordered-dictionary structure; it's the structure I'd reach for in a lock-free concurrent setting, where treaps' rotation-based updates make synchronization harder. For the broader pattern of randomization-as-balance, Motwani and Raghavan's *Randomized Algorithms* (Cambridge, 1995) §8 is the textbook reference, and Part IX of this book picks up the same thread when randomized quicksort and Rabin-Karp fingerprinting come around.

19 An array that pretends to be a tree

Chapter 18's treap was governed by two ordering rules at once — a BST rule on keys and a max-heap rule on random priorities — and the heap rule was a passenger, the bookkeeping that kept the tree shallow. I want to flip that arrangement and let the heap rule carry the whole structure. No keys, no BST descent, no in-order traversal — just one ordering rule on the data itself, and a very particular layout that makes the rule cheap to maintain.

A heap is an array that pretends to be a tree. The "tree" is a fiction the index arithmetic sustains; the storage is contiguous; the pointer overhead is zero. A generic `Heap[T]` built on a plain Python list supports two operations — `push` adds an item, `pop_min` removes the smallest — both running in $O(\log n)$ time. Building a heap from n items in one shot costs $\Theta(n)$ rather than $\Theta(n \log n)$, and the same structure becomes the priority queue Dijkstra's shortest-path algorithm plugs into in Part V.

Parent at index `(i-1)//2`, children at `2i+1` and `2i+2` — the whole tree lives in index arithmetic. That spareness is what makes the heap the default substrate for priority queues, scheduling, graph search, and any algorithm that needs "smallest first."

19.1 Two operations, nothing else

The contract is two operations. **`push(item)`** adds an item to the collection. **`pop_min() -> item`** removes and returns the smallest item currently held, where "smallest" is decided by Python's `<` operator on the items. That's it. No way to look up an arbitrary item. No way to iterate the items in order. No way to ask "is this in the heap?" without popping everything out. The structure is restricted by what it can't do, and that restriction is exactly what buys the cost shape.

The same move at a smaller scale runs through chapters 10 and 11. A stack is a list that refuses everything but the top; a queue refuses everything but the head. The refusal is the discipline, and the discipline is what makes both operations $O(1)$ on a layout that wouldn't otherwise support arbitrary access in constant time. A heap takes the same idea up one level: it refuses everything except "the smallest" and "add one more," and in exchange gets $O(\log n)$ on both.

This chapter uses a **min-heap** throughout. The other convention — a max-heap, which pops the largest — is symmetric; flip every `<` to `>` and you've ported the implementation. Min wins because the dominant applications — Dijkstra's shortest-path, A* search, event-driven simulation, scheduling by deadline — all want smallest-first.

19.2 The tree is the index arithmetic

A heap is conceptually a binary tree, but the tree is implicit. I lay the nodes in a Python list, level by level, left to right. The root sits at index 0. Its two children sit at indices 1 and 2. Its grandchildren sit at indices 3, 4, 5, 6. In general, for the node at index i:

- **Parent index:** $\lfloor (i-1)/2 \rfloor$, which is `(i - 1) // 2` in Python.
- **Left child index:** $2i + 1$.
- **Right child index:** $2i + 2$.

You can check the arithmetic on any node. The node at index 7 has parent at index 3, left child at 15, right child at 16. The root has no parent; children whose computed index exceeds the array length simply don't exist — leaves are the indices where one or both children would land past the end.

The tree is **complete** by construction: filling the list left-to-right at each level guarantees every level except possibly the last is full, and the last is filled from the left. No gaps. That's not an invariant I maintain; it's a property of the layout, free.

The tree's height is **exactly** $\lfloor \log_2 n \rfloor$ for n elements. A complete binary tree of n nodes has that height, period — no constants, no randomization, no balance factors. The structure is balanced because the layout makes it impossible to be otherwise.

On top of that layout, I impose the **heap order**: every parent is less than or equal to both of its children. Equivalently, the root holds the minimum. The order is local — each node-and-its-children triple satisfies it independently — but the local rule adds up to the global property that the minimum is always at index 0. The heap order is the only invariant the operations actively maintain.

19.3 Sift up, sift down

Two repair operations cover the maintenance. `push` may violate the heap order at a new leaf, so I sift the new item *up* toward the root. `pop_min` may violate the heap order at the root, so I sift the replacement *down* toward a leaf. Each repair does at most one swap per level on a tree of $\lfloor \log_2 n \rfloor$ levels, so each is $O(\log n)$.

The class skeleton has a private list, an optional iterable of items at construction time, and `__len__` for the usual reasons.

```
from collections.abc import Iterable


class Heap[T]:
    def __init__(self, items: Iterable[T] | None = None) -> None:
        if items is None:
            self._heap: list[T] = []
        else:
            self._heap = list(items)
            for i in range(len(self._heap) // 2 - 1, -1, -1):
                self._sift_down(i)

    def __len__(self) -> int:
        return len(self._heap)
```

If no items are supplied, the heap starts empty and grows by `push`. If items *are* supplied, the constructor runs build-heap — the bottom-up loop I'll justify in the next section — to repair every internal node in place. It lives in the constructor so the wrapper reads as one operation rather than getting split across two blocks.

For `push`, the new item lands at the next free index — the end of the list — and gets sifted up. The sift-up walk asks at each step: is this item smaller than its parent? If yes, swap and continue from the parent's index. If no, the heap order is restored and the loop stops.

```
    def push(self, item: T) -> None:
        self._heap.append(item)
        self._sift_up(len(self._heap) - 1)

    def _sift_up(self, i: int) -> None:
        while i > 0:
            parent = (i - 1) // 2
            if self._heap[i] < self._heap[parent]:
                self._heap[i], self._heap[parent] = (
                    self._heap[parent],
                    self._heap[i],
                )
                i = parent
            else:
                return
```

The loop terminates either when `i` reaches the root or when the heap order holds. The maximum number of iterations is the depth of the new leaf — $\lfloor \log_2 n \rfloor$ — so `push` is $O(\log n)$ in time and $O(1)$ in extra space.

`pop_min` is the symmetric operation. I lift the root out (it's the minimum, the value I'm returning), then move the *last* leaf into the root slot to keep the array compact, and sift the replacement down. Why the last leaf? Because removing any other element would leave a gap in the middle of the array, breaking the implicit-tree layout. The last leaf is the only one I can pull out without shifting indices.

```
    def peek_min(self) -> T:
        if not self._heap:
            raise IndexError("peek from empty heap")
        return self._heap[0]

    def pop_min(self) -> T:
        if not self._heap:
            raise IndexError("pop from empty heap")
        root = self._heap[0]
        last = self._heap.pop()
        if self._heap:
            self._heap[0] = last
            self._sift_down(0)
        return root
```

I expose `peek_min` because reading the minimum without removing it is a common request — Dijkstra's loop often wants to know whether the smallest candidate is good enough before committing to pop it. Both `peek_min` and `pop_min` raise `IndexError` on an empty heap, mirroring `list.pop`.

The sift-down loop mirrors sift-up. At each step the current node compares against its smaller child; if the node is already smaller-or-equal, the heap order holds and the loop stops. Otherwise the node swaps with that smaller child and the loop continues from the child's index.

```
    def _sift_down(self, i: int, n: int | None = None) -> None:
        if n is None:
            n = len(self._heap)
        while True:
            left = 2 * i + 1
            right = 2 * i + 2
            smallest = i
            if left < n and self._heap[left] < self._heap[smallest]:
                smallest = left
            if right < n and self._heap[right] < self._heap[smallest]:
                smallest = right
            if smallest == i:
                return
            self._heap[i], self._heap[smallest] = (
                self._heap[smallest],
                self._heap[i],
            )
            i = smallest
```

The `n` parameter is a hint that says "treat only the first `n` slots as the heap." It defaults to the whole list. The reason it exists is in-place heap-sort: you can sort an array by repeatedly extracting the root and shrinking the logical heap by one, leaving the extracted root in the slot it vacated. The `n` parameter lets sift-down know where the logical heap currently ends. It's there for any caller who wants the in-place pattern.

Each sift-down call performs at most $\lfloor \log_2 n \rfloor$ swaps, one per level, so `pop_min` runs in $O(\log n)$ time. On a heap of a million items, both `push` and `pop_min` need at most about 20 comparisons; on a billion items, about 30. Logarithmic in the formal sense, instantaneous in the practical sense.

Push eleven values in arbitrary order, print the internal array (heap-ordered, not sorted), then pop them all and verify the output is sorted ascending.

```python
from codex.trees.heap import Heap

h: Heap[int] = Heap()
for x in [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]:
    h.push(x)

# The internal layout - heap-ordered but NOT sorted. The root (index 0)
# is the minimum; every parent  both children, but siblings can be in
# any order relative to each other.
print(f"internal: {h._heap}")

# Pop everything; the output sequence is sorted ascending - the BST has
# no monopoly on producing sorted output, the heap gets there too.
popped = [h.pop_min() for _ in range(11)]
print(f"popped:   {popped}")
```

```
internal: [1, 1, 2, 3, 3, 9, 4, 6, 5, 5, 5]
popped:   [1, 1, 2, 3, 3, 4, 5, 5, 5, 6, 9]
```

The internal array isn't sorted — the heap order is much weaker than total order. What it guarantees is that index 0 is the minimum, and that's what `pop_min` needs. The popped sequence is sorted because each pop extracts the current minimum, and removing the minimum leaves the second-smallest as the new minimum, and so on.

19.4 Build-heap in O(n)

If I want to drop n items into a heap up front, the obvious approach is n pushes, for $O(n \log n)$ total work. That's what an external caller would do if they didn't know any better. But there's a much better way, due to Floyd in 1964, and the constructor I wrote up there is already using it.

The idea: take the items as-is, in the order they arrived, and treat the resulting list as a "candidate heap" that probably violates the heap order all over. Then run `_sift_down` on every internal node, from the deepest internal node up to the root. Each sift-down repairs its subtree; by the time the loop reaches the root, every subtree is heap-ordered, and so is the whole tree.

The loop goes from $\lfloor n/2 \rfloor - 1$ down to 0 because indices $\lfloor n/2 \rfloor$ through $n-1$ are leaves — no children in the array, already a one-element heap, no work needed. Only the first half of the array needs repair, and the repair runs bottom-up.

This loop runs in $\Theta(n)$ total time, not $\Theta(n \log n)$. The naive bound says each sift-down costs $O(\log n)$ and the loop runs it $n/2$ times, giving $O(n \log n)$. That's not wrong, just loose. The tight bound comes from noticing that **most of the sift-downs are short**. The deepest sift-down (at the root) costs up to $\log n$, but there's only one of those. The shortest sift-downs, just above the leaves, cost 1 each, and there are $n/4$ of them. Most of the work is done by the cheap ones.

The counting goes like this. A complete binary tree of n nodes has roughly $n/2^{h+1}$ nodes at height h, each doing at most h steps. Total:

$$T(n) \le \sum_{h=0}^{\lfloor \log_2 n \rfloor} \frac{n}{2^{h+1}} \cdot h = \frac{n}{2} \sum_{h=0}^{\infty} \frac{h}{2^h}$$

The infinite sum $\sum h/2^h$ converges to exactly 2 — derivable from the derivative of the geometric series at $r = 1/2$; the appendix's section on convergent sums has the identity. Substituting: $T(n) \le n$. So the total work is at most n — *linear*, not $n \log n$.

Building a heap of a million items via n pushes costs roughly 2×10^7 comparisons; via Floyd's bottom-up trick, about 10^6. The asymptotic ratio is $\log n$, which is 20 at this size.

Same eleven values as before, but passed to the constructor instead of pushed one at a time.

```
from codex.trees.heap import Heap

items = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]
h: Heap[int] = Heap(items=items)

# The internal array after build-heap - heap-ordered, but not necessarily
# the same shape as the n-pushes version, because the two procedures
# settle into different valid heap configurations.
print(f"internal: {h._heap}")

# Pop everything; sorted output is the contract, regardless of which
# valid heap shape the procedure happened to land on.
popped = [h.pop_min() for _ in range(len(items))]
print(f"popped:   {popped}")
```

```
internal: [1, 1, 2, 3, 3, 9, 4, 6, 5, 5, 5]
popped:   [1, 1, 2, 3, 3, 4, 5, 5, 5, 6, 9]
```

Pop sequence is sorted, same as the push-based version — the contract is the same, the cost is different. The internal arrays after the two procedures *can* differ, because there are multiple valid heap configurations for the same multiset; on this particular input they happen to coincide, but the contract only promises heap order, not a specific layout.

The same trick gives **heap-sort** for free. Build-heap, then pop everything out — the pop sequence is sorted ascending. Total cost $\Theta(n) + n \cdot \Theta(\log n) = \Theta(n \log n)$, the same asymptotic class as merge-sort and quicksort, but in a predictable shape. Heap-sort is what gives chapter 7's algorithm catalog its $O(n \log n)$ worst-case sorting bound without quicksort's adversarial-input pathology.

```
from codex.trees.heap import Heap

# Heap-sort by build-heap + repeated pop. The fastest correct way to
# sort with a heap; matches Python's sorted() output exactly.
items = [5, 2, 8, 1, 9, 3, 7, 4, 6]
h: Heap[int] = Heap(items=items)
heap_sorted = [h.pop_min() for _ in range(len(items))]
print(f"heap-sort: {heap_sorted}")
print(f"sorted():  {sorted(items)}")
print(f"match: {heap_sorted == sorted(items)}")
```

```
heap-sort: [1, 2, 3, 4, 5, 6, 7, 8, 9]
sorted():  [1, 2, 3, 4, 5, 6, 7, 8, 9]
match: True
```

The match holds. Heap-sort is a real sorting algorithm: the only $O(n \log n)$ comparison sort I know that runs in place with no extra memory. Embedded contexts, where allocating a merge buffer is a problem, reach for it for exactly that reason.

At $n = 10^5$, both build approaches run on the same shuffled input — n pushes against one build-heap call — and the ratio gets reported. The asymptotic prediction says the ratio should be $\log_2(10^5) \approx 17$. The empirical reality on Python is more modest, because the per-operation interpreter overhead is large enough to squash the asymptotic separation at this size.

```
import random
import time
from codex.trees.heap import Heap

random.seed(0)
n = 100_000
data = list(range(n))
```

```
random.shuffle(data)

# n separate pushes - total cost O(n log n)
t0 = time.perf_counter()
h_pushes: Heap[int] = Heap()
for x in data:
    h_pushes.push(x)
t_pushes = time.perf_counter() - t0

# build-heap in one shot - total cost O(n)
t0 = time.perf_counter()
h_build: Heap[int] = Heap(items=data)
t_build = time.perf_counter() - t0

print(f"n pushes:    {t_pushes * 1000:7.1f} ms")
print(f"build-heap:  {t_build * 1000:7.1f} ms")
print(f"ratio:       {t_pushes / t_build:.2f}x")
# build-heap wins by a small but real factor - roughly 1.2-1.5x on
# pure Python at this size. The asymptotic ratio of log_2(10^5)   17
# only shows up when the per-comparison cost is small (C, Rust, etc.);
# Python's interpreter overhead dominates and squashes the gap.
```

```
n pushes:       40.7 ms
build-heap:     29.7 ms
ratio:       1.37x
```

Build-heap wins, but by less than the asymptotic theory predicts — Python's per-operation interpreter overhead is large enough that the $\log n$ factor compresses into a ~1.2x to 1.5x runtime gap at $n = 10^5$. The same benchmark in C would show the predicted 15–20x. The point is the *direction* of the win, not the magnitude — build-heap is asymptotically cheaper, and the cost shape is real even when constants smear it.

19.5 From "smallest item" to "highest priority"

A heap orders items by `<`. A priority queue orders *jobs* by a separate priority value attached to each job. Pythonically, the way to bridge the two is to push `(priority, job)` tuples — tuples compare lexicographically, so the priority dominates the comparison and ties break on the job itself.

```
class HeapPriorityQueue[K, V]:
    def __init__(self) -> None:
        self._heap: Heap[tuple[K, V]] = Heap()

    def __len__(self) -> int:
        return len(self._heap)

    def push(self, priority: K, item: V) -> None:
        self._heap.push((priority, item))

    def pop(self) -> tuple[K, V]:
        return self._heap.pop_min()
```

A thin wrapper — twenty lines of class scaffolding around the heap's two operations. The point of giving it its own type is that the API names `priority` and `item` separately, which reads better at the call site than passing a tuple by hand.

```
from codex.trees.heap import HeapPriorityQueue

pq: HeapPriorityQueue[int, str] = HeapPriorityQueue()
```

```
pq.push(3, "answer the email backlog")
pq.push(1, "page from on-call - wake up")
pq.push(2, "review the open pull request")

# Pop in priority order - smallest priority first, which here means
# the most urgent task (the on-call page) comes out first.
while len(pq):
    print(pq.pop())
```

```
(1, 'page from on-call - wake up')
(2, 'review the open pull request')
(3, 'answer the email backlog')
```

Priority 1 wins, priority 2 follows, priority 3 last. That's the contract Dijkstra and A* and event-driven simulation all rely on: pull the most urgent thing first, regardless of when it was inserted.

19.6 D-ary heaps, Fibonacci heaps, and what they cost

The binary heap is one point on a wider design space. Two variants matter enough to mention even though I won't implement them.

A **d-ary heap** replaces the binary branching factor with a parameter d. Each node has d children, the tree's height drops from $\log_2 n$ to $\log_d n$, but sift-down now compares against d children per level instead of 2. Net effect: `push` gets cheaper (fewer levels for sift-up to walk), `pop_min` gets more expensive per level (more child comparisons), but the same number of levels deep. For applications where push is much more common than pop, $d = 4$ or $d = 8$ wins in practice. Dijkstra's shortest-path on a graph with E edges and V vertices is the canonical example — each edge produces a push, each vertex produces a pop, so the optimal d is roughly E/V, the average degree.

A **Fibonacci heap** (Fredman & Tarjan, 1987) is a different structure entirely — a forest of trees with lazy merging, where `push` and `decrease-key` run in $O(1)$ amortized time and `pop_min` runs in $O(\log n)$ amortized time. The asymptotic win for Dijkstra is real: $O((V + E) \log V)$ becomes $O(E + V \log V)$, which matters when E is large. But the constants are bad — doubly-linked sibling lists, parent pointers, mark bits, a cascading-cut procedure on `decrease-key`. On every input size you'll see, Fibonacci heaps lose to binary heaps in practice. The binary heap is the engineering winner. When the constants are out of whack, the better asymptotic loses.

19.7 The three questions, applied

Is it correct?

A heap is correct if `pop_min` returns the minimum currently held after any sequence of operations. Two structural invariants do the work: **completeness** (the array's nodes form a complete binary tree under the parent-at-`(i-1)//2` interpretation) and **heap order** (every parent both children). Completeness is a layout property — I never insert or delete in the middle of the array, only at the end, so it's maintained for free. Heap order is what `_sift_up` and `_sift_down` actively maintain.

`push` preserves heap order because `_sift_up` swaps the new item upward exactly as long as it's smaller than its parent. Each swap restores the rule locally and may introduce a violation one level higher; the loop terminates when either no violation exists or the new item reaches the root. `pop_min` returns `_heap[0]`, which is the minimum by the heap order invariant; the last leaf moves to the root, and `_sift_down` swaps it downward with whichever child is smaller until the order holds again.

Build-heap preserves heap order by induction on subtree size. Each leaf is trivially a one-element heap. When `_sift_down(i)` runs on an internal node i whose left and right subtrees are already heaps, sift-down repairs the subtree rooted at i without disturbing the subtree property below. Running the loop from $\lfloor n/2 \rfloor - 1$ down to 0 processes every internal node *after* its children's subtrees are already heaps, so the induction holds at every step.

How efficient is it?

The cost table for a heap of n elements:

Operation	Time	Extra space
`push`	$O(\log n)$	$O(1)$ — in-place swaps
`pop_min`	$O(\log n)$	$O(1)$
`peek_min`	$O(1)$	$O(1)$
`__len__`	$O(1)$	$O(1)$
`Heap(items)` (build-heap)	$\Theta(n)$	$O(n)$ — for the list
Heap-sort (build + n pops)	$\Theta(n \log n)$	$O(n)$

Worst and average cases coincide here, because the heap order is maintained by every operation rather than degraded by adversarial inputs. There's no input that drives a heap to height n — the layout forbids it. That's a stronger guarantee than a BST gives you: a BST's worst case is $O(n)$ on sorted input, and the AVL and treap chapters spent their effort buying back the $O(\log n)$ bound. A heap gets the height for free.

Storage per element is one slot in the contiguous list — no pointers, no metadata. A heap of n integers is an array of n integers, full stop.

Is it optimal?

For the priority queue contract specifically, a binary heap is asymptotically optimal — both `push` and `pop_min` run in $O(\log n)$, matching the comparison-based lower bound for any structure that supports both ordered insertion and ordered extraction. The argument: n pushes followed by n pop-mins produces sorted output, and sorting in the comparison model requires $\Omega(n \log n)$ comparisons (Stirling's bound, appendix-math). If both operations were $o(\log n)$, the $2n$ operations would do $o(n \log n)$ comparisons total, contradicting the sorting lower bound.

You can do better than $O(\log n)$ on `push` alone if you give up something — Fibonacci heaps trade implementation complexity for $O(1)$ amortized push. You can do better than $O(\log n)$ on `pop_min` if you restrict the items to integers in a known range — chapter 6's counting and radix sort tricks carry over to bucket-based priority queues. The binary heap is the comparison-based, no-assumptions-about-priorities sweet spot.

19.8 The cheapest implementation is the one with no pointers

Parent at index $(i-1)//2$, children at $2i+1$ and $2i+2$ — the whole tree lives in index arithmetic, and that's what makes the heap so spare. No node objects, no pointers, no balance factors. Just a contiguous array, three index formulas that turn it into an implicit complete binary tree, and two repair operations to maintain the local heap order. Both repairs are $O(\log n)$ because the height is $\lfloor \log_2 n \rfloor$ by layout, and that bound is automatic — no input can degrade it.

A heap's contiguous array beats any pointer-based equivalent for the same reason chapter 8's dynamic array beats chapter 9's linked list: the cache loves contiguous data, and the CPU loves predictable index arithmetic. When the access pattern fits a contiguous layout, take the layout.

Restricting an interface buys a cost shape the unrestricted version couldn't deliver. A heap is a list that refuses everything except `push` and `pop_min`; that refusal is what makes both $O(\log n)$ on an implicit-tree layout that wouldn't support general ordered queries. Same move as the stack and queue chapters.

The tight analysis is often not the obvious one. Build-heap looks like it should be $O(n \log n)$; that's what the loose argument gives. The tight bound is $\Theta(n)$, from noticing that most sift-downs are short. The loose bound is correct but useless; the tight bound is what explains why build-heap is the right way to construct a heap from a known input.

Chapter 20 takes the implicit-structure idea in a different direction. Where a heap's "structure" is a tree implied by index arithmetic over flat storage, a trie's structure is a tree where the *path from the root to a node* encodes the key — the data structure's shape *is* the data it stores. Same principle, different geometry.

19.9 Notes and further reading

The binary heap was introduced by J. W. J. Williams in 1964 in *Communications of the ACM* 7(6):347–348 — the structure and the heap-sort algorithm appear together in that short paper. Robert Floyd's 1964 *CACM* 7(12):701 paper "Algorithm 245: Treesort 3" is the source of the $\Theta(n)$ bottom-up build-heap trick the constructor uses. CLRS (4th ed.) chapter 6 has the canonical textbook treatment — heaps, heap-sort, and priority queues in one chapter, with the tight build-heap analysis and a careful proof of correctness. Sedgewick & Wayne's *Algorithms* (4th ed.) §2.4 develops the same material with a priority-queue focus. The Dijkstra-specific d-ary optimization with $d = E/V$ goes back to Johnson's 1975 "Priority Queues with Update and Finding Minimum Spanning Trees." The Fibonacci heap is Michael Fredman and Robert Tarjan's 1987 *Journal of the ACM* paper "Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms" — the canonical reference for both the structure and its impact on Dijkstra's bound. Python's standard-library `heapq` module implements the same min-heap algorithm in pure C; it's the right thing to reach for in production, but the pure-Python version above makes the algorithm visible in a way the C source doesn't.

20 One branch per character

Chapter 12's hash table answered "is this key in my dictionary?" by computing the key's address from its content. It's the fastest membership test the book has shown you, and for pure existence checks I'd reach for it first. The hash table is fast precisely *because* it threw the key's structure away — `"apple"` ran through a hash function and became an integer with no relationship to any other key. That throw-away is what buys the constant time, and it's also what makes one whole class of question impossible: "give me all words starting with `"app"`." The keys `"app"`, `"apple"`, `"application"`, `"apricot"` hash to four unrelated slots; there's no way to start at `"app"` and walk outward. Asking a prefix query forces an $O(n)$ scan, and the asymptotic gap chapter 12 bought you disappears.

The path from root to a node *is* the key — keys aren't stored, they're walked. The string `"apple"` doesn't sit anywhere as a contiguous chunk; it's distributed along five edges of a tree, and reading it back means walking from the root down to a terminal node. That's the **trie**: each character of a string key is one edge in a tree, the path from the root to a node spells the key out one character at a time, and prefix queries fall out of the structure for free. A naive trie's per-node bookkeeping can balloon past the size of the strings themselves; the **radix trie** collapses single-child chains into one fat edge to fix the cost. And **longest-prefix matching** is almost free on a trie — the operation that powers IP routing in every router on the internet.

20.1 Strings as a special case of keys

The hash table treats every key the same: `"alice"`, `(3, 4, 5)`, and `42` get hashed, modulo'd, and probed identically. That uniformity is its strength and, for string keys specifically, its weakness. Strings have *internal structure* the hash function can't see — `"apple"` shares its first three characters with `"application"`, `"app"` is a prefix of `"apple"` — and those relationships are exactly what a hash scrambles. The structure has been spent by the time the hash returns.

The trie preserves what hashes destroy: the shared prefixes among keys. It stores keys character by character, one character per edge, so two keys sharing a prefix share the corresponding initial path. `"apple"` and `"application"` walk the same five edges before they diverge; the shared prefix is shared *storage* — one copy of `"appl"` in the tree, two distinct continuations below. That sharing is what makes prefix queries cheap. Asking "give me all words starting with `"app"`" reduces to walking three edges down to the `"app"` node and enumerating its subtree. Cost proportional to the answer plus the prefix length, never proportional to the dictionary size.

What you give up is hash-table-fast membership. A trie's lookup costs $O(|k|)$ — every character walked — and so does a hash table's, since hashing a string reads every character too. But the hash table's constant is smaller and its cache behaviour is much better. For pure membership on string keys, hash tables still win in practice; the trie's advantage shows up exactly when the prefix structure matters.

20.2 One node per character, one flag per terminal

Each node has two pieces of state: a dictionary mapping single characters to child nodes, and a `terminal` flag that's `True` if some stored key ends at this node. I also keep an optional `value` so the trie can stand in as a `str → V` map, the way chapter 12's hash table did.

```
class TrieNode:
    def __init__(self) -> None:
        self.children: dict[str, "TrieNode"] = {}
        self.terminal: bool = False
        self.value: object | None = None
```

The `terminal` flag is doing real work. It distinguishes a stored key from a mere prefix of one. If I insert `"apple"`, the trie has nodes for `a`, `p`, `p`, `l`, `e`, and only the last one — the `e` — is terminal. Asking "is `"appl"` in the trie?" walks four edges, arrives at the `l` node, which exists but is *not* terminal, so the answer is `False`. Asking "is `"apple"` in the trie?" arrives at the terminal `e` and answers `True`. If I then *also* insert `"app"`, the depth-3 `p` node acquires its own `terminal = True` while keeping its `l` child for the `"apple"` branch. Both keys are stored, neither interferes with the other — the flag exists specifically to support that coexistence.

Now the trie itself. The constructor builds an empty root and tracks the number of stored keys for `__len__`.

```
class Trie:
    def __init__(self) -> None:
        self._root = TrieNode()
        self._size = 0

    def __len__(self) -> int:
        return self._size
```

`insert` walks character by character from the root, creating any missing child node on the way. After the walk, the node where the key ends is marked terminal and the value is attached.

```
    def insert(self, key: str, value: object = None) -> None:
        node = self._root
        for ch in key:
            if ch not in node.children:
                node.children[ch] = TrieNode()
            node = node.children[ch]
        if not node.terminal:
            self._size += 1
        node.terminal = True
        node.value = value
```

The `if not node.terminal` guard is what makes re-inserting an existing key update the value without inflating the count. If you insert `"apple"` twice, the second call walks to the same terminal node, sees it's already terminal, and just rewrites the value — `_size` stays at 1.

A helper for the descend-or-fail step is shared by three operations: membership, `get`, and `starts_with` all start by descending along a string.

```
    def _descend(self, key: str) -> TrieNode | None:
        node = self._root
        for ch in key:
            if ch not in node.children:
                return None
            node = node.children[ch]
        return node
```

Returns the node where the walk ends, or `None` if any intermediate character has no edge. Note this doesn't say anything about whether `key` is *stored* — it only says the path exists. The terminal check is the caller's job.

Membership and value lookup both consume `_descend`. The API splits the way Python's `dict` does: `__contains__` for "is this key stored?" and `get` for "if it's stored, give me the value."

```
    def __contains__(self, key: str) -> bool:
        node = self._descend(key)
        return node is not None and node.terminal

    def get(self, key: str) -> object | None:
        node = self._descend(key)
        if node is None or not node.terminal:
            return None
        return node.value
```

Both cost $O(|k|)$ — one pointer hop per character. Looking up a 20-character word in a trie of a million words costs 20 dictionary probes, regardless of how many words are stored. The lookup cost depends on the *query's* length, not the dictionary's size.

```
from codex.trees.trie import Trie

trie = Trie()
for word in ["apple", "app", "apricot", "banana", "band", "bandana"]:
    trie.insert(word)

print(f"size: {len(trie)}")

# 'app' is stored - the inner p is terminal even though apple continues past it
print(f"'app' in trie:   {'app' in trie}")

# 'apple' is stored too - the terminal 'e' is its own marker
print(f"'apple' in trie: {'apple' in trie}")

# 'appl' walks four edges and lands on a non-terminal node - prefix, not stored
print(f"'appl' in trie:  {'appl' in trie}")

# 'banana' is stored; 'banan' is just a prefix
print(f"'banan' in trie: {'banan' in trie}")
print(f"'banana' in trie: {'banana' in trie}")
```

```
size: 6
'app' in trie:   True
'apple' in trie: True
'appl' in trie:  False
'banan' in trie: False
'banana' in trie: True
```

Both cases the `terminal` flag exists to disambiguate show up: `"app"` is stored *and* a prefix of `"apple"`, so it counts as stored; `"appl"` is *only* a prefix, so it counts as absent.

20.3 Prefix queries — what tries can do

`starts_with(prefix)` returns every stored key for which `prefix` is an initial segment. Walk down to the prefix's landing node — if the prefix isn't even a path in the trie, return the empty list. Otherwise, DFS the subtree rooted there, collecting every terminal node's path along the way.

```
    def starts_with(self, prefix: str) -> list[str]:
        node = self._descend(prefix)
        if node is None:
            return []
        results: list[str] = []
        self._collect(node, prefix, results)
        return results
```

The DFS helper walks the subtree, appending each terminal node's path to the result list. The `path` parameter is the string accumulated so far — the prefix plus whatever characters this branch of the DFS has descended through.

```
    def _collect(self, node: TrieNode, path: str, results: list[str]) -> None:
        if node.terminal:
            results.append(path)
        for ch, child in node.children.items():
            self._collect(child, path + ch, results)
```

The cost is $O(|p| + |\text{output}|)$ where $|p|$ is the prefix length and $|\text{output}|$ is the total length of returned strings. The prefix walk is linear in the prefix; the DFS visits exactly the nodes that contribute to the output. Enumerating every word starting with `"app"` in a million-word dictionary costs three character steps to reach the `"app"` node plus a tour of just the `"app"` subtree — never a scan of the million.

```
from codex.trees.trie import Trie

trie = Trie()
for word in ["apple", "app", "apricot", "banana", "band", "bandana"]:
    trie.insert(word)

# Everything starting with 'ap' - three completions, in DFS order
print(f"starts_with('ap'):  {sorted(trie.starts_with('ap'))}")

# Everything starting with 'ban' - also three completions
print(f"starts_with('ban'): {sorted(trie.starts_with('ban'))}")

# A prefix that doesn't exist returns the empty list, not an error
print(f"starts_with('z'):   {trie.starts_with('z')}")
```

```
starts_with('ap'):  ['app', 'apple', 'apricot']
starts_with('ban'): ['banana', 'band', 'bandana']
starts_with('z'):   []
```

Three `"ap…"` completions and three `"ban…"` completions, each from a localised DFS over a small subtree. The hash table from chapter 12 couldn't have answered either query without a full scan; the trie answers in time proportional to the prefix plus the output, which is exactly the cost shape autocomplete and spell-check ask for. `starts_with` promises "every key with this prefix," not any particular order — sort at the call site when order matters.

20.4 Longest-prefix matching — the IP-routing use case

A third operation puts tries decisively ahead of hash tables: **longest-prefix matching**. Given a query string, return the longest stored key that's a prefix of the query. The canonical application is internet routing: a router holds a table of network prefixes like `192.168.0.0/16` and `192.168.42.0/24`, and when a packet arrives bound for `192.168.42.7`, the router has to find the longest stored prefix matching the destination — `/24` if it's there, falling back to `/16` if not. The match has to happen on every packet at line rate.

A hash table can't even define this natively — you'd have to enumerate every prefix of the query and look each up, $O(|q|^2)$ total. A trie does it in one descent. Walk character by character; every time you pass a terminal node, remember it as a candidate longest match; if the walk falls off the tree, stop. Return the deepest terminal seen, or `None`.

```
    def longest_prefix_of(self, query: str) -> str | None:
        node = self._root
        deepest: str | None = None
        for i, ch in enumerate(query):
            if ch not in node.children:
                break
            node = node.children[ch]
            if node.terminal:
                deepest = query[: i + 1]
        return deepest
```

One walk, $O(|q|)$ pointer hops. The `deepest` variable is the trick — it remembers the longest terminal seen even when the walk later descends past it without hitting another. Matching a 32-bit IP against a routing table of any size costs at most 32 pointer hops, irrespective of how many prefixes the table holds.

The same code runs unmodified on bit-string keys, which is the form real IP routers use; here it runs on string-shaped prefixes for legibility.

```
from codex.trees.trie import Trie

# A small routing table - prefixes ordered from least to most specific
routes = Trie()
for prefix in ["10.", "10.0.", "10.0.0.", "192.168.", "192.168.1."]:
    routes.insert(prefix)

# Each query gets the longest matching prefix the table can offer
for ip in ["10.0.0.5", "192.168.1.42", "10.0.5.5", "10.5.5.5", "172.16.0.1"]:
    match = routes.longest_prefix_of(ip)
    print(f"{ip:<14} -> {match!r}")
```

```
10.0.0.5       -> '10.0.0.'
192.168.1.42   -> '192.168.1.'
10.0.5.5       -> '10.0.'
10.5.5.5       -> '10.'
172.16.0.1     -> None
```

The query `"10.0.0.5"` matched the most specific entry `"10.0.0."` — the walk passed through three terminal nodes and remembered the last. `"10.0.5.5"` matched `"10.0."` because the walk continued past `"10.0."` along `5` but `"10.0.5"` isn't stored. `"10.5.5.5"` matched only `"10."`. `"172.16.0.1"` matched nothing — the very first character has no edge from the root, and `deepest` stays `None`. That single $O(|q|)$ descent is what makes tries the structure of choice for routing tables, IP-prefix lookups, and any workload whose query asks "what's the longest stored key that fits inside this input?"

20.5 The pointer-overhead cost

The trie's storage is where the trade shows up. Every character of every stored key generates a node, and each node carries a Python `dict` — empty `dict`s included. A million English words at ~7 characters each is 7 megabytes of raw character data, but the trie's seven-million-ish nodes each carry a dict, a boolean, and a `value` pointer, running to a few hundred bytes per node on CPython. The 7 megabytes of characters become several hundred megabytes of trie nodes. You run out of cache, then out of RAM.

The cost-per-character is most painful on the *unshared* parts of the tree — the deep branches descending through single-child chains to reach a stored key. If `"banana"` is the only key starting with `"ban"`, the trie has six nodes along `b → a → n → a → n → a`, each with one child, none serving any purpose beyond "spell out one more character." The fix is to collapse single-child chains into one fat edge — the resulting structure is called a **radix trie**, or sometimes a Patricia trie after Donald Morrison's 1968 paper that introduced the binary-string version.

20.6 Collapse single-child chains into edges

In a radix trie, each edge is labelled with a *substring* rather than a single character. A node still has a `terminal` flag and a `value`, but represents an end-or-branch point rather than a per-character step.

```
class RadixNode:
    def __init__(self) -> None:
        self.children: dict[str, "RadixNode"] = {}
        self.terminal: bool = False
        self.value: object | None = None
```

The structure looks identical to `TrieNode`, but the dictionary key is now an arbitrary-length substring. Single-character edges are still allowed — they just mean "no useful compression possible at this branch point."

Insert is where the work moves. Walking a query string down the trie now involves consuming a whole edge, splitting an edge mid-label, or attaching a fresh edge.

```
class RadixTrie:
    def __init__(self) -> None:
        self._root = RadixNode()
        self._size = 0

    def __len__(self) -> int:
        return self._size
```

At each step, I look for the outgoing edge whose first character matches the next character of the remaining query. If none exists, I attach a new edge labelled with the remaining query. If one exists, I compute the longest common prefix between the edge label and the query remainder. Three cases follow: common prefix is the whole edge (consume it, descend, recurse on what's left); common prefix shorter than both (split the edge at the divergence and attach the remainder as a sibling); common prefix is the whole query but the edge is longer (split the edge at the query's end and make the split point the terminal).

```
    def insert(self, key: str, value: object = None) -> None:
        node = self._root
        rest = key
        while rest:
            edge = self._matching_edge(node, rest[0])
            if edge is None:
                leaf = RadixNode()
                leaf.terminal = True
                leaf.value = value
                node.children[rest] = leaf
                self._size += 1
                return
            n_common = self._common_prefix_length(edge, rest)
            if n_common == len(edge):
                # consume the whole edge, descend
                node = node.children[edge]
                rest = rest[n_common:]
                continue
            self._split_edge(node, edge, n_common, rest[n_common:], value)
            self._size += 1
            return
        if not node.terminal:
            self._size += 1
        node.terminal = True
        node.value = value
```

Three helpers do the local work. The first finds the outgoing edge starting with a given character — at most one such edge exists by construction, since two edges from the same node sharing a first character would mean a missed split.

```
    def _matching_edge(self, node: RadixNode, ch: str) -> str | None:
        for edge in node.children:
            if edge[0] == ch:
                return edge
        return None
```

The second computes the length of the longest common prefix between two strings.

```
    def _common_prefix_length(self, a: str, b: str) -> int:
        i = 0
        while i < len(a) and i < len(b) and a[i] == b[i]:
            i += 1
        return i
```

The third is the actual edge split. The edge gets cut at position `n_common`, the original child moves below a new intermediate `split` node, and the remaining query (if any) attaches as a sibling. If the query ends at the split point — the new key is a proper prefix of the original edge — the `split` node *is* the terminal.

```
    def _split_edge(
        self,
        node: RadixNode,
        edge: str,
        n_common: int,
        rest: str,
        value: object,
    ) -> None:
        original_child = node.children[edge]
        split = RadixNode()
        split.children[edge[n_common:]] = original_child
        del node.children[edge]
        node.children[edge[:n_common]] = split
        if rest:
            leaf = RadixNode()
            leaf.terminal = True
            leaf.value = value
            split.children[rest] = leaf
        else:
            split.terminal = True
            split.value = value
```

That's the insert. Search is much simpler because it never has to split anything — it just walks down, consuming whole edges or failing if a partial match shows up.

```
    def __contains__(self, key: str) -> bool:
        node = self._root
        rest = key
        while rest:
            edge = self._matching_edge(node, rest[0])
            if edge is None or not rest.startswith(edge):
                return False
            node = node.children[edge]
            rest = rest[len(edge):]
        return node.terminal
```

A partial match — where the remaining query shares some but not all of an edge's characters — means the query string isn't stored. The radix structure's invariant is that any stored key's path consumes a sequence of complete edges; anything that ends mid-edge isn't there.

`starts_with`, `get`, and `longest_prefix_of` follow the same pattern on the radix variant as on the standard trie, with "walk by edge" substituted for "walk by character." Delete is the more interesting case: a radix delete might need to *merge* edges back together after removing a key that was holding apart a chain that could now collapse. Worth working through on paper.

Comparing the two structures on the same six-word dictionary, both by membership and by node count:

```
from codex.trees.trie import Trie, RadixTrie

words = ["apple", "app", "apricot", "banana", "band", "bandana"]

trie = Trie()
radix = RadixTrie()
for word in words:
    trie.insert(word)
    radix.insert(word)
```

```
# Membership agrees across both structures - the contract is the same
for query in ["app", "apple", "appl", "bandana", "bandanas", "apricot"]:
    print(f"{query!r:<12} trie={query in trie!s:<5}  radix={query in radix!s:<5}")
```

```
'app'        trie=True   radix=True
'apple'      trie=True   radix=True
'appl'       trie=False  radix=False
'bandana'    trie=True   radix=True
'bandanas'   trie=False  radix=False
'apricot'    trie=True   radix=True
```

Both structures agree on every query: stored keys come back `True`, prefixes-that-aren't-stored come back `False`, queries extending past a stored key (like `"bandanas"`) come back `False`. The compression is transparent to the membership API. To see it directly, walk each tree and count nodes including the root.

```
from codex.trees.trie import Trie, RadixTrie, TrieNode, RadixNode


def count_trie_nodes(trie: Trie) -> int:
    def walk(node: TrieNode) -> int:
        return 1 + sum(walk(c) for c in node.children.values())
    return walk(trie._root)


def count_radix_nodes(rt: RadixTrie) -> int:
    def walk(node: RadixNode) -> int:
        return 1 + sum(walk(c) for c in node.children.values())
    return walk(rt._root)


words = ["apple", "app", "apricot", "banana", "band", "bandana"]
trie = Trie()
radix = RadixTrie()
for word in words:
    trie.insert(word)
    radix.insert(word)

print(f"standard trie nodes: {count_trie_nodes(trie)}")
print(f"radix trie nodes:    {count_radix_nodes(radix)}")
print(f"compression ratio:   {count_trie_nodes(trie) / count_radix_nodes(radix):.2f}x")
```

```
standard trie nodes: 21
radix trie nodes:    9
compression ratio:   2.33x
```

Twenty-one nodes for the standard trie, nine for the radix. The standard trie pays a node per character of every word's *unshared* tail — the `"icot"` in `"apricot"`, the `"ana"` in `"bandana"`. The radix trie collapses each tail into a single edge, with one node at the branch point where the shared prefix ends. On this tiny example the ratio is already 2.3x; on a real English-dictionary trie, compression often exceeds 5x because the deep-and-unshared region dominates the node count.

The radix trie's per-operation asymptotic cost is the same as the standard trie's — the work is just distributed differently. What changes is the constant, and the constant is what makes the difference between a structure that fits in RAM and one that doesn't.

20.7 The three questions, applied

Is it correct?

A trie is correct if `__contains__(k)` returns `True` exactly when `k` was inserted and `starts_with(p)` returns exactly the inserted keys for which `p` is a prefix. The shared invariant is that **the path from the root to any node spells, character by character, the prefix that node represents**, and a node is `terminal` iff some inserted key ended at exactly that prefix.

`insert(k)` preserves the invariant: it walks `k`'s characters, creating missing edges, and sets `terminal = True` at the destination. Any future search for `k` walks the same path and finds the same terminal. A search for a key that *isn't* `k` either takes a different path or stops at a non-terminal — correct rejection. The radix trie's split operation preserves the invariant pointwise: after splitting edge `e` into `e[:i]` and `e[i:]`, the path from the root to the new intermediate node spells the same characters as the original partial walk, no terminals moved, no characters lost or gained.

Longest-prefix matching is correct because the loop's invariant is "`deepest` is the longest stored prefix of `query[:i]`." At every terminal node encountered, `query[:i+1]` is a stored prefix and is longer than anything previously seen. The loop terminates when the walk falls off the tree or when the query is exhausted; either way, `deepest` holds the longest stored prefix of the whole query.

How efficient is it?

The cost table for a trie holding n keys with maximum key length ℓ.

Operation	Time	Extra space
`insert(k)`	$O(\|k\|)$	$O(\|k\|)$ worst-case new nodes
`__contains__(k)`, `get(k)`	$O(\|k\|)$	$O(1)$
`starts_with(p)`	$O(\|p\| + \|\text{output}\|)$	$O(\|\text{output}\|)$ for the returned list
`longest_prefix_of(q)`	$O(\|q\|)$	$O(1)$
`__len__`	$O(1)$	$O(1)$

Every operation's time depends on the query length, never on n. That's the trie's distinctive cost shape: a million-key trie and a thousand-key trie answer membership in the same time, *provided the query has the same length.* The hash table from chapter 12 has the same property for membership; only the trie keeps it for `starts_with` and `longest_prefix_of`.

Space is the rough spot. A standard trie storing n keys with C total characters uses $O(C)$ nodes, each carrying a Python `dict` plus a boolean plus a `value` slot. The radix trie cuts that to $O(n)$ branching nodes at the cost of a more involved insert. A million 20-character strings cost ~20 MB in a hash table, hundreds of megabytes in a standard trie, and roughly 30–50 MB in a radix trie — bigger than the hash table, but in the same order of magnitude, and with prefix-and-longest-prefix operations the hash table can't do at all.

Is it optimal?

For exact membership on string keys, hash tables win the constants — one hash plus one probe beats character-by-character walking, and a contiguous slot array beats pointer-chasing through a tree. No asymptotic gap, but a real practical one.

For prefix and longest-prefix queries, the trie is asymptotically optimal. Any `starts_with` has to read at least $|p|$ characters of the prefix and emit at least $|\text{output}|$ characters of output; the trie hits both bounds. Longest-prefix matching has to read at least $|q|$ characters, and the trie hits that bound too. No structure can be faster than $O(|p| + |\text{output}|)$ or $O(|q|)$ because every input character has to be examined.

The radix trie storing n keys uses $O(n)$ nodes plus $O(C)$ characters across edge labels. You can't do better asymptotically — the keys have to be stored somewhere — but the constants tighten further with *suffix tries* and *suffix arrays*, which Part IV takes up. The trie is the structure where the path-encoded-key idea first appears; its sophisticated relatives are refinements of the same idea.

20.8 The path *is* the key

Everything in the chapter rests on that one move. Keys aren't stored as opaque blobs; they're stored as *paths through a tree*, one character per edge in the standard trie or one substring per edge in the radix variant. Walking the path is the lookup.

The way you store something determines what you can do with it. A hash table stores keys as scrambled integers, which is why it can't answer prefix questions — the structural information prefix queries need was destroyed by the hash. A trie stores keys as paths, and prefix queries fall out of the structure for free. When the workload's questions depend on a particular relationship between keys, the data structure has to preserve that relationship rather than scramble it.

Compression isn't always a bolted-on concern — it can be the core of the design. The radix trie isn't a trie with compression added; it's a different structure, defined by collapsing single-child chains into substring edges. The compression happens in the type, not as an afterthought.

Sometimes the right operation reads the data once. Longest-prefix matching could be defined naively as $|q|$ separate lookups, $O(|q|^2)$ total; a trie does it in a single $O(|q|)$ descent because the structure was already laid out to support it. When the structure matches the workload, the cost drops from quadratic to linear with no optimisation at all.

Chapter 21 takes the implicit-array idea chapter 19's heap pioneered and applies it to a different question. Where a heap uses index arithmetic over a flat array to maintain "find the minimum in $O(\log n)$," a **segment tree** uses the same layout to maintain "find the sum (or min, or max, or any associative aggregate) of an arbitrary range in $O(\log n)$." The tree is implicit, the storage is contiguous, and the range-query power is what tries gave you for prefixes — generalised to any aggregate over any range, on any underlying array.

20.9 Notes and further reading

The trie was independently introduced by René de la Briandais in 1959 (*Proceedings of the Western Joint Computer Conference*, "File Searching Using Variable Length Keys") and named "trie" — from re*trie*val — by Edward Fredkin in 1960 (*Communications of the ACM* 3(9):490–499, "Trie Memory"). The radix-trie variant, in its binary-string form, is Donald Morrison's 1968 *Journal of the ACM* paper "PATRICIA — Practical Algorithm to Retrieve Information Coded in Alphanumeric," which gives the structure its alternate name (Patricia trie). Knuth's *The Art of Computer Programming* Vol. 3 §6.3 has the canonical textbook treatment of trie searching, including the digital-search-tree variants and the careful complexity analysis. Sedgewick & Wayne's *Algorithms* (4th ed.) §5.2 covers tries, ternary search tries, and a small comparison against hash-based string-keyed dictionaries. For the asymptotic-space lower-bound argument, see Kurt Mehlhorn's 1982 "A Best Possible Bound for the Weighted Path Length of Binary Search Trees" and Arne Andersson and Stefan Nilsson's 1995 "Improved Behaviour of Tries by Adaptive Branching." The IP-routing application is treated in detail in Marcel Waldvogel et al.'s 1997 SIGCOMM paper "Scalable High Speed IP Routing Lookups," which shows how a radix trie variant achieves single-digit-microsecond lookups on full BGP tables.

21 Intervals as objects

Chapter 19's heap used index arithmetic over a flat array to keep one question cheap: where is the minimum? I want to push that "structure-implied-by-the-layout" idea further and answer a question the heap can't: given an array of numbers, what is the sum (or min, or max) of an arbitrary contiguous range, and how do I keep that answer cheap when the underlying values change?

Every node owns an interval, and the answer to any range query is the sum of the $O(\log n)$ canonical intervals that tile it. A segment tree is what you get when you stop thinking about array indices and start thinking about intervals as first-class objects. The structure gives point-update plus range-query in $O(\log n)$ per operation, and the harder case — range-update plus range-query — yields to **lazy propagation**, a technique that defers work until someone asks for it.

21.1 Updates cheap or queries cheap — pick one

You have an array of n numbers — call them $a_0, a_1, \ldots, a_{n-1}$ — and you need to support two operations in any order, any number of times.

1. **Update** one position: set a_i to a new value.
2. **Query** a range: return the sum (or min, or max, or xor) of $a_l, a_{l+1}, \ldots, a_{r-1}$.

You have two naive approaches, each handling one operation beautifully and the other terribly.

Approach 1: store the array as-is. Updating a_i is one assignment, $O(1)$. Querying a range walks the range, $O(r - l)$. Cheap update, expensive query.

Approach 2: store prefix sums. Maintain $P_k = a_0 + a_1 + \cdots + a_{k-1}$. Now a range sum is one subtraction, $O(1)$. But updating a_i shifts every prefix sum from P_{i+1} onward by the same delta, so you rewrite the right half of the array. $O(n)$ per update.

For a mixed workload of comparable updates and queries, both approaches degrade to $O(n)$ amortized per operation, which is the cost of doing nothing clever at all. The segment tree gives you $O(\log n)$ for **both**. On an array of a million elements, both update and query cost about twenty steps; on a billion, about thirty.

21.2 Every node owns an interval

The array's index range $[0, n)$ is one big interval, recursively split in half. The root node owns $[0, n)$. Its left child owns $[0, n/2)$; its right child owns $[n/2, n)$. The recursion bottoms out at intervals of length one, where each leaf owns one array position and stores its value.

Every internal node stores the aggregate (sum, min, …) of the values in its interval. The root stores the aggregate over $[0, n)$; a node owning $[2, 5)$ stores $a_2 + a_3 + a_4$. The invariant I'll maintain is: **every internal node's stored value equals the aggregate of its two children's stored values.**

That recursive split is a balanced binary tree of height $\lceil \log_2 n \rceil$, with n leaves and at most $n - 1$ internal nodes, so at most $2n - 1$ in total. I lay them out in an array the same way chapter 19's heap did: the root at index 1, the children of index `node` at `2 * node` and `2 * node + 1`. I use index 1 (not 0) so the child formula needs no `+1` corrections.

When n isn't a power of two, the recursive split is still well-defined (you split at the midpoint of each interval), but the implicit-tree index range can exceed $2n - 1$. The safe over-allocation is $4n$ slots; that always fits, by a one-paragraph induction on the recursion depth. I use **half-open intervals** $[l, r)$ throughout, following Python's idiom (the convention `list[l:r]` follows), so the midpoint split partitions $[lo, mid) \cup [mid, hi)$ with no overlap.

A range query like "sum of $[3, 11)$" doesn't visit every leaf in $[3, 11)$ — that would be $O(n)$. It visits at most $O(\log n)$ internal nodes whose intervals together *tile* $[3, 11)$ exactly. Each visited node returns its stored aggregate in $O(1)$, and I sum them. The tiling is what makes the query logarithmic.

21.3 Walking down to update, three cases to query

The constructor takes the data and an aggregation `op` with its identity element (0 for sum, $-\infty$ for max, etc.).

```
from collections.abc import Sequence
from typing import Callable


class SegmentTree[T]:
    def __init__(
        self,
        data: Sequence[T],
        op: Callable[[T, T], T] | None = None,
        identity: T | None = None,
    ) -> None:
        if op is None:
            op = lambda a, b: a + b  # type: ignore[assignment,return-value]
        if identity is None:
            identity = 0  # type: ignore[assignment]
        self._n = len(data)
        self._op: Callable[[T, T], T] = op
        self._identity: T = identity  # type: ignore[assignment]
        self._tree: list[T] = [identity] * (4 * max(self._n, 1))  # type: ignore[list-item]
        if self._n > 0:
            self._build(1, 0, self._n, list(data))

    def __len__(self) -> int:
        return self._n
```

My defaults (addition and zero) make `SegmentTree(data)` a sum-aggregate tree out of the box. Anything else (min, max, xor, gcd) needs its own `op` and `identity` passed in. The build walks the implicit tree top-down, splitting each interval at its midpoint until the leaves hold the array values; on the way back up, each internal node aggregates its two children.

```
    def _build(self, node: int, lo: int, hi: int, data: list[T]) -> None:
        if hi - lo == 1:
            self._tree[node] = data[lo]
            return
        mid = (lo + hi) // 2
        self._build(2 * node, lo, mid, data)
        self._build(2 * node + 1, mid, hi, data)
        self._tree[node] = self._op(self._tree[2 * node], self._tree[2 * node + 1])
```

The recursion visits each of the $\leq 2n - 1$ nodes once with $O(1)$ work apiece, so I get a $\Theta(n)$ build. Linear in the data size, not in the tree's height.

To update position i, I walk from the root down to the leaf owning that position, then back up, recomputing each ancestor's aggregate from its now-corrected children.

```
    def update(self, i: int, value: T) -> None:
        self._update(1, 0, self._n, i, value)
```

The recursive helper figures out whether i falls in the left half or the right half and recurses into that half only; I leave the other half untouched. After the recursive call returns, I recompute the current node's aggregate from its two children.

```
    def _update(
        self, node: int, lo: int, hi: int, i: int, value: T
    ) -> None:
        if hi - lo == 1:
            self._tree[node] = value
            return
        mid = (lo + hi) // 2
        if i < mid:
            self._update(2 * node, lo, mid, i, value)
        else:
            self._update(2 * node + 1, mid, hi, i, value)
        self._tree[node] = self._op(self._tree[2 * node], self._tree[2 * node + 1])
```

You can see the walk visits exactly one node per level, descending into the half containing i and ignoring the other. Tree height is $O(\log n)$, so update is $O(\log n)$ time and $O(\log n)$ stack frames. Updating one position in a million-element tree touches about twenty nodes.

To compute the range-query aggregate over $[l, r)$ from a node owning $[lo, hi)$, three cases come up. **Disjoint**: the two intervals don't overlap, so the node contributes nothing; return the identity. **Contained**: the node's interval lies entirely inside the query, so the stored aggregate is exactly what I want. **Crossing**: partial overlap, so recurse on both children and combine.

```
    def query(self, l: int, r: int) -> T:
        return self._query(1, 0, self._n, l, r)

    def _query(self, node: int, lo: int, hi: int, l: int, r: int) -> T:
        if r <= lo or hi <= l:
            return self._identity
        if l <= lo and hi <= r:
            return self._tree[node]
        mid = (lo + hi) // 2
        left_val = self._query(2 * node, lo, mid, l, r)
        right_val = self._query(2 * node + 1, mid, hi, l, r)
        return self._op(left_val, right_val)
```

The three-case split is the heart of the algorithm. Disjoint lets me prune whole subtrees that contribute nothing. Contained stops the recursion at the deepest node still entirely inside the query; that node *is* a canonical tile. Crossing is the only case that descends, and it only happens on the $O(\log n)$ nodes along the *boundaries* of the query range.

A power-of-two array of eight values, with queries before and after a point update:

```
from codex.trees.segment import SegmentTree

data = [1, 3, 5, 7, 9, 11, 13, 15]
st: SegmentTree[int] = SegmentTree(data)

# Whole-array sum - one node visited, the root
print(f"query(0, 8) = {st.query(0, 8)}")    # 64 - the root's value, one tile

# A three-element interior range - sum of positions 2, 3, 4
print(f"query(2, 5) = {st.query(2, 5)}")    # 21 = 5 + 7 + 9

# A single-element "range" - query of [0, 1)
print(f"query(0, 1) = {st.query(0, 1)}")    # 1 - the leaf for position 0

# Update position 3 from 7 to 100; the change propagates up to the root
st.update(3, 100)
print(f"after update(3, 100), query(2, 5) = {st.query(2, 5)}")  # 114 = 5 + 100 + 9
```

```
query(0, 8) = 64
query(2, 5) = 21
query(0, 1) = 1
after update(3, 100), query(2, 5) = 114
```

You see three queries with different shapes (the whole array, an interior range, a single element), each answered in $O(\log n)$ time. The update touched the leaf for position 3 and recomputed the four ancestors back up to the root.

The same structure handles arbitrary associative aggregates; sum is just the default. Pass `op=max` and an identity safe for max (`float('-inf')` when values are non-negative), and the same nodes, walks, and three-case query give you range maximum.

```python
import math
from codex.trees.segment import SegmentTree

data = [1, 3, 5, 7, 9, 11, 13, 15]

# Same data, but ask for range maximum. The op flips from + to max,
# and the identity flips from 0 to -infinity (the max-identity element).
st_max: SegmentTree[float] = SegmentTree(data, op=max, identity=-math.inf)

# Max over the interior range - should be 9, the largest of 5, 7, 9
print(f"max query(2, 5) = {st_max.query(2, 5)}")    # 9

# Max over the whole array - the last element
print(f"max query(0, 8) = {st_max.query(0, 8)}")    # 15
```

```
max query(2, 5) = 9
max query(0, 8) = 15
```

The aggregate flipped from sum to max with one argument. The segment tree isn't a sum tree, it's an *associative-monoid* tree. Anything you can combine with an associative operation that has an identity (sum, min, max, xor, gcd, bitwise-and, matrix product) plugs straight in. Only the per-node combine step changes.

21.4 Range update, point query — flip the roles

The dual problem swaps the roles. Updates are now over ranges (say, "add 5 to every element in $[l, r)$") and queries ask for a single element. A segment tree handles this in $O(\log n)$ per operation too: each node stores a **delta** ("this much should be added to every element in my interval"), a range update adds the delta to each tile of the canonical decomposition, and a point query at position i sums the deltas along the root-to-leaf path.

The lazy version below subsumes this case; call `query(i, i+1)` for point-query semantics.

21.5 Defer the work until somebody asks

The combined problem (range update *and* range query, both $O(\log n)$) is where the segment tree earns its complexity. Both operations need to touch many tree nodes, and the trick is to make the touching cheap.

The idea is **lazy**. When a range update fully contains a node's interval, I want to record "every element under this node should be increased by `delta`" and stop, rather than walk down to the leaves. So I add a parallel array, `lazy[node]` per node, that records pending updates not yet pushed down. `tree[node]` is kept in sync with the current aggregate *assuming the lazy values above the node have been applied*; `lazy[node]` records updates that affect the subtree but haven't been propagated to the children yet.

The invariant: **`tree[node]` is the correct aggregate for the node's interval, accounting for all lazy values at this node and at every ancestor, but not yet for any lazy values stored at descendants.** Every operation that descends through a node pushes its lazy value down first.

```
class LazySegmentTree:
    def __init__(self, data: Sequence[int]) -> None:
        self._n = len(data)
        size = 4 * max(self._n, 1)
        self._tree: list[int] = [0] * size
        self._lazy: list[int] = [0] * size
        if self._n > 0:
            self._build(1, 0, self._n, list(data))

    def __len__(self) -> int:
        return self._n

    def _build(self, node: int, lo: int, hi: int, data: list[int]) -> None:
        if hi - lo == 1:
            self._tree[node] = data[lo]
            return
        mid = (lo + hi) // 2
        self._build(2 * node, lo, mid, data)
        self._build(2 * node + 1, mid, hi, data)
        self._tree[node] = self._tree[2 * node] + self._tree[2 * node + 1]
```

I'm committing this lazy variant to sum-with-range-add. Generalizing to arbitrary monoids with arbitrary range updates is possible but delicate, because the update has to commute with the aggregate in a specific way. Sum-and-add is the cleanest case.

Two small helpers do the lazy machinery. `_apply` adds a `delta` to a node, updating its stored aggregate by `delta * interval_length` (the sum over an interval of length k increases by δk) and stacking the delta into the node's lazy slot. `_push_down` flushes a node's lazy value into its two children before recursing into either.

```
    def _apply(self, node: int, lo: int, hi: int, delta: int) -> None:
        self._tree[node] += delta * (hi - lo)
        self._lazy[node] += delta

    def _push_down(self, node: int, lo: int, hi: int) -> None:
        if self._lazy[node] != 0:
            mid = (lo + hi) // 2
            self._apply(2 * node, lo, mid, self._lazy[node])
            self._apply(2 * node + 1, mid, hi, self._lazy[node])
            self._lazy[node] = 0
```

`_push_down` is the only place the lazy slot gets cleared. After it, the parent's aggregate is still correct (the parent already accounted for the delta), and the children's aggregates plus their new lazy slots reflect the pending update.

Range update is the disjoint / contained / crossing trichotomy again, with contained using `_apply` to mark the subtree lazily.

```
    def update(self, l: int, r: int, delta: int) -> None:
        self._update(1, 0, self._n, l, r, delta)

    def _update(
        self, node: int, lo: int, hi: int, l: int, r: int, delta: int
    ) -> None:
        if r <= lo or hi <= l:
            return
        if l <= lo and hi <= r:
            self._apply(node, lo, hi, delta)
            return
        self._push_down(node, lo, hi)
        mid = (lo + hi) // 2
```

```
        self._update(2 * node, lo, mid, l, r, delta)
        self._update(2 * node + 1, mid, hi, l, r, delta)
        self._tree[node] = self._tree[2 * node] + self._tree[2 * node + 1]
```

The contained case is what makes my range update logarithmic: `_apply` marks the subtree in $O(1)$ and the recursion stops. Crossing only happens along $O(\log n)$ nodes near the update range's boundaries.

Range query is structurally identical to the non-lazy version, with one new line: push down before descending.

```
    def query(self, l: int, r: int) -> int:
        return self._query(1, 0, self._n, l, r)

    def _query(self, node: int, lo: int, hi: int, l: int, r: int) -> int:
        if r <= lo or hi <= l:
            return 0
        if l <= lo and hi <= r:
            return self._tree[node]
        self._push_down(node, lo, hi)
        mid = (lo + hi) // 2
        left = self._query(2 * node, lo, mid, l, r)
        right = self._query(2 * node + 1, mid, hi, l, r)
        return left + right
```

The push-down inside `_query` keeps the invariant intact: before descending into a child, the parent's pending update has to land there, so whatever the child reports is consistent with every update issued so far.

Range-add plus range-sum on a fresh array of eight ones:

```
from codex.trees.segment import LazySegmentTree

# All ones - initial sum over the whole array is 8
lst = LazySegmentTree([1, 1, 1, 1, 1, 1, 1, 1])
print(f"initial query(0, 8) = {lst.query(0, 8)}")   # 8

# Add 5 to every element in [3, 7) - that's positions 3, 4, 5, 6
lst.update(3, 7, 5)

# Whole-array sum is now 8 + 4*5 = 28
print(f"after add 5 to [3, 7): query(0, 8) = {lst.query(0, 8)}")   # 28

# A range fully inside the updated region - two elements, each now 6
print(f"query(3, 5) = {lst.query(3, 5)}")   # 12 = 6 + 6

# A range outside the updated region - positions 0, 1, 2 are still 1 each
print(f"query(0, 3) = {lst.query(0, 3)}")   # 3

# A range crossing the updated boundary - positions 6 (updated) and 7 (not)
print(f"query(6, 8) = {lst.query(6, 8)}")   # 7 = 6 + 1
```

```
initial query(0, 8) = 8
after add 5 to [3, 7): query(0, 8) = 28
query(3, 5) = 12
query(0, 3) = 3
query(6, 8) = 7
```

The range update touched $O(\log n)$ canonical-tile nodes and stopped. Subsequent queries pushed lazy values down only along the paths they descended, never the whole tree. Five operations, all $O(\log n)$, all correct.

21.6 The three questions, applied

Is it correct?

The non-lazy `SegmentTree` is correct because the invariant "every internal node's stored value is the aggregate over its interval" is maintained by every operation. `_build` establishes it bottom-up. `_update` walks down to one leaf and recomputes ancestors on the way back up; every visited ancestor's children are now consistent. `_query` uses the three-case split — disjoint contributes the identity, contained returns the stored aggregate, crossing recurses — and the three cases tile the query range exactly once, by induction on tree depth.

The `LazySegmentTree` is correct because its invariant ("`tree[node]` reflects every lazy update at this node or above it, none strictly below") is maintained pointwise. `_apply` updates `tree[node]` and `lazy[node]` together. `_push_down` clears `lazy[node]` by applying it to both children; afterward the children reflect what the parent's lazy slot demanded. Both `_update` and `_query` push down before descending, so descents see children consistent with every update the parent knew about. The contained case in `_update` is where the laziness earns its keep: it marks the subtree with one $O(1)$ step, and any future descent into that subtree will push the mark down before doing anything else, so no stale aggregate ever escapes.

How efficient is it?

The cost table for an array of n elements:

Operation	Time	Extra space
`SegmentTree(data)` (build)	$\Theta(n)$	$\Theta(n)$
`update(i, v)`	$O(\log n)$	$O(\log n)$ stack
`query(l, r)`	$O(\log n)$	$O(\log n)$
`LazySegmentTree.update(l, r, d)`	$O(\log n)$	$O(\log n)$
`LazySegmentTree.query(l, r)`	$O(\log n)$	$O(\log n)$

The $O(\log n)$ bound on query has a tight argument. A range query at $[l, r)$ visits a tree node if the node's interval overlaps the query but isn't contained in it (crossing), or if it contributes directly (contained). The contained nodes form the canonical tile decomposition; at most 4 per level can appear (two near the left boundary, two near the right), for at most $4 \log n$ total. Crossing nodes are their ancestors, also $O(\log n)$. Per-node work is $O(1)$.

On a million-element array both operations touch about $4 \log_2(10^6) \approx 80$ tree nodes; on a billion, about 120. Storage is $\Theta(n)$: the $4n$ over-allocation absorbs index slack for non-power-of-two n, and the lazy variant doubles that with the `_lazy` array.

Is it optimal?

For the point-update + range-query workload, $O(\log n)$ per operation is asymptotically optimal in the cell-probe model. A structure supporting n updates and n queries must do $\Omega(n \log n)$ total work (Patrascu and Demaine, 2006); if a single operation were $o(\log n)$, $2n$ operations would do $o(n \log n)$, contradicting the bound. The same argument applies to range update + range query, and the lazy segment tree matches it.

You can sometimes do better with extra assumptions. If updates are *only* increments and queries are *only* sums, a **Fenwick tree** (chapter 22) matches the same $O(\log n)$ bound with half the code and a smaller constant. If queries only ever cover the whole array, the problem trivializes to a counter. The segment tree's $O(\log n)$ is the price of supporting *arbitrary* ranges with *arbitrary* associative aggregates and *arbitrary* updates; drop any of those and a more specialized structure can do better.

21.7 The layout is the algorithm

Every node owns an interval, and the answer to any range query is the sum of the $O(\log n)$ canonical intervals that tile it. Once that lands the rest is bookkeeping.

The right abstraction is sometimes an interval, not an index. The naive approaches think about indices; the segment tree gives every interval a node, and range queries become one lookup instead of n. The shift from "which position?" to "which interval?" is what turns a linear-time scan into a logarithmic-time descent.

Laziness is a structural technique, not just a syntactic one. A range update marks $O(\log n)$ tiles and stops; the marks only propagate when a subsequent query forces them. The same idea powers garbage collection, copy-on-write file systems, and database query planners — deferred work that materializes only on demand.

A segment tree is a flat array under index arithmetic, chapter 19's trick again. What changes is what each slot stores. The implicit-binary-tree-over-flat-array pattern has been the backbone of three consecutive chapters; chapter 22 makes it four.

Chapter 22 takes the segment tree's range-sum capability and asks: can the same workload be solved with half the code? The answer is yes, by giving up generality. A **Fenwick tree** handles prefix sums (and therefore range sums by subtraction) in $O(\log n)$, using a single array of size n and a bit-manipulation trick on the lowest set bit.

21.8 Notes and further reading

The segment tree was introduced by Jon Louis Bentley in 1977 in his Stanford technical report on computational geometry, though Bentley's original application was *interval stabbing* (given a query point, find all stored intervals containing it), not the range-aggregate problem of this chapter. The transposition to range aggregates is folklore in the competitive-programming community and doesn't have a canonical paper. CLRS doesn't cover segment trees directly, and neither does Sedgewick & Wayne's *Algorithms* (4th ed.); the structure is treated as specialized in the canonical undergraduate textbooks. The actual canon is the competitive-programming literature: the CP-Algorithms encyclopedia entry at cp-algorithms.com/data_structures/segment_tree.html is the definitive reference, covering the basic structure, lazy propagation, the iterative bottom-up variant, persistent segment trees, and several advanced extensions. The Codeforces tutorials on the platform's "EDU" section work through the same material with problem sets. For the lower-bound argument above, see Mihai Patrascu and Erik Demaine's 2006 "Logarithmic Lower Bounds in the Cell-Probe Model" (*SIAM Journal on Computing* 35(4):932–963). The Fenwick tree's lowest-bit trick that chapter 22 builds on is Peter Fenwick's 1994 *Software: Practice and Experience* paper "A New Data Structure for Cumulative Frequency Tables."

22 Same problem, half the code

Chapter 21's segment tree was a heavy machine for a general problem: arbitrary associative aggregates over arbitrary ranges, with arbitrary updates. I want to spend this chapter showing you that when the aggregate is *just* a sum — or anything that behaves like a sum, with an inverse — you can throw away most of that machinery and end up with a structure that's eight lines per operation instead of twenty. The structure is the **Fenwick tree**, also called a **binary indexed tree** or **BIT**, and it does point-update plus range-sum in $O(\log n)$ with a single n-slot array and a one-character arithmetic trick.

`i & -i` — the lowest set bit — is the entire structure encoded in one operation.

Both `update` and `prefix` are three-line `while` loops that repeatedly add or subtract that quantity to walk a chain of $O(\log n)$ array cells. There's no tree object, no node class, no recursion. The "tree" in "binary indexed tree" is implicit in the bit pattern of the index itself, and once that lands the rest is bookkeeping. A second variant (`FenwickXOR`) shows the same code generalizes to any commutative group — anything that supports an inverse. The structure stops working exactly when the aggregate has no inverse (min, max): the subtraction trick collapses and the segment tree is still the right answer.

22.1 Segment trees were too much

The previous chapter's `SegmentTree` paid for generality. It supports any associative monoid (sum, min, max, xor, gcd, matrix product), it accepts arbitrary ranges, it does range update with lazy propagation. For the *prefix sum* problem alone — point updates, range-sum queries — most of that flexibility is dead weight. I'm paying for tree structure, $4n$ over-allocation, recursive descents, and a three-case query split, and I'm getting back exactly one thing: $O(\log n)$ on both operations.

You can extract that same cost shape from a much smaller observation. If the aggregate I care about supports subtraction — which sum does, and xor does, but min and max emphatically don't — then **any range sum is the difference of two prefix sums**: `range_sum(l, r) = prefix(r) - prefix(l - 1)`. So I don't need to support arbitrary range queries directly. I only need fast prefix sums with fast point updates, and I can synthesize range queries on top.

That reduction is the whole game. The Fenwick tree is the data structure tuned for exactly that reduced problem, and the structure it gives you is half the code of a segment tree, with a smaller constant factor, on the same asymptotic $O(\log n)$ bound. The price is that you lose min, max, and any other aggregate without an inverse. For prefix sums you pay nothing for what you gave up.

22.2 The lowest set bit decides the interval

The Fenwick tree's storage is a single array `tree[1..n]`, **1-indexed**. At index i, the cell `tree[i]` stores the sum of the array values over the range $(i - \text{lowbit}(i),\ i]$, inclusive on the right, where $\text{lowbit}(i) = i \,\&\, -i$ is the value of i's lowest set bit. In Python, you compute `i & -i` directly, courtesy of two's-complement arithmetic — `-i` flips every bit and adds one, which leaves a single set bit at the position of i's lowest one. So `lowbit(6) = 2`, `lowbit(8) = 8`, `lowbit(7) = 1`.

The intervals each cell covers form a binary-tree shape when you draw them out, which is where "binary indexed tree" comes from. For $n = 8$, the eight cells cover these ranges:

```
# Each cell tree[i] covers a[i - lowbit(i) + 1 .. i] - print the layout for n=8
for i in range(1, 9):
    lb = i & -i
    lo = i - lb + 1
    print(f"  tree[{i}] covers a[{lo}..{i}]   (lowbit({i}) = {lb})")
```

```
tree[1] covers a[1..1]   (lowbit(1) = 1)
tree[2] covers a[1..2]   (lowbit(2) = 2)
tree[3] covers a[3..3]   (lowbit(3) = 1)
tree[4] covers a[1..4]   (lowbit(4) = 4)
tree[5] covers a[5..5]   (lowbit(5) = 1)
tree[6] covers a[5..6]   (lowbit(6) = 2)
tree[7] covers a[7..7]   (lowbit(7) = 1)
tree[8] covers a[1..8]   (lowbit(8) = 8)
```

You can read the table directly. Cell `tree[8]` covers the whole array (lowbit is 8, so it covers $(0, 8]$); `tree[4]` covers `a[1..4]`; `tree[2]` covers `a[1..2]`; cells with odd indices each cover a single position. The pattern: cells at powers-of-two indices are tall (cover many positions); cells at odd indices are short (cover one). Cell `tree[6]` covers two positions, because `lowbit(6) = 2`.

The 1-indexing isn't cosmetic. The query loop subtracts the lowbit repeatedly and stops at zero, which requires the array to start at 1 (`lowbit(0) = 0`, which would loop forever). The bit arithmetic *is* the structure; positions are 1-indexed throughout the chapter, and a 0-indexed wrapper would be one `+1` per call.

22.3 Two while-loops, one bit-trick

Update is the upward walk. To add `delta` to the underlying array at position i, I have to update every cell whose interval covers i. Those cells live at i, $i + \text{lowbit}(i)$, then that index plus *its* lowbit, and so on. Each step jumps to the next cell whose interval still includes position i — the lowbit increases monotonically along the chain, so the walk takes at most $\lfloor \log_2 n \rfloor + 1$ steps before overshooting n and terminating.

Prefix is the downward walk. To compute $\sum_{k=1}^{i} a[k]$, I sum the cells along the chain i, $i - \text{lowbit}(i)$, that index minus *its* lowbit, and so on, until I hit zero. Each step strips off the current lowest bit. By the binary representation of i, that chain has exactly as many steps as i has set bits, which is at most $\lfloor \log_2 n \rfloor + 1$.

The constructor accepts either an integer size (allocate empty) or an initial data sequence (run a linear-time build).

```python
from collections.abc import Sequence


class FenwickTree:
    def __init__(self, n_or_data: int | Sequence[int]) -> None:
        if isinstance(n_or_data, int):
            self._n = n_or_data
            self._tree: list[int] = [0] * (self._n + 1)
        else:
            data = list(n_or_data)
            self._n = len(data)
            self._tree = [0] * (self._n + 1)
            for i in range(1, self._n + 1):
                self._tree[i] += data[i - 1]
                parent = i + (i & -i)
                if parent <= self._n:
                    self._tree[parent] += self._tree[i]

    def __len__(self) -> int:
        return self._n
```

The data branch is the $\Theta(n)$ build. Loop i from 1 to n; at each step, fold the current cell's accumulated value into its parent at index $i + \text{lowbit}(i)$. By the time the loop reaches i, every cell with smaller index has already pushed its contribution upward, so `tree[i]` holds the sum it owes — and I push that sum further up to `tree[i + lowbit(i)]` if the parent is in range. The naive alternative is n separate `update` calls for $O(n \log n)$; this is one linear pass. Building from a million-element array touches a million cells once each, not twenty million.

The `update` body fits in three lines.

```
    def update(self, i: int, delta: int) -> None:
        while i <= self._n:
            self._tree[i] += delta
            i += i & -i
```

I add `delta` to the current cell, then jump to the next cell whose interval also covers position i, by adding the lowbit. The loop terminates as soon as the index would overshoot n. The walk visits $O(\log n)$ cells, each in $O(1)$ work.

`prefix` is the symmetric downward walk.

```
    def prefix(self, i: int) -> int:
        total = 0
        while i > 0:
            total += self._tree[i]
            i -= i & -i
        return total
```

I accumulate the current cell, then jump to the next cell along the chain by subtracting the lowbit. The loop terminates when `i` reaches zero. Same $O(\log n)$ bound, same $O(1)$ per step. Notice both loops are structurally identical apart from the direction of the jump and the termination test — `+= lowbit, stop at >n` for update, `-= lowbit, stop at 0` for query. That symmetry is what makes the structure feel inevitable once you've seen it.

Range sum is the whole point — the segment-tree-flavored query synthesized from two prefix calls.

```
    def range_sum(self, l: int, r: int) -> int:
        return self.prefix(r) - self.prefix(l - 1)
```

One line. `prefix(r)` is the sum over $a[1..r]$; `prefix(l - 1)` is the sum over $a[1..l-1]$; the difference is the sum I want over $a[l..r]$. Two $O(\log n)$ walks and one subtraction, so range queries cost $O(\log n)$ too, same as the segment tree but without any of the structural overhead.

The same 8-element array from chapter 21, for direct comparison.

```
from codex.trees.fenwick import FenwickTree

# Same data as the segment-tree chapter - explicit callback so you can
# diff the two demos line by line and see they're answering the same
# questions on the same input.
ft = FenwickTree([1, 3, 5, 7, 9, 11, 13, 15])

print(f"prefix(8) = {ft.prefix(8)}")          # 64 - whole-array sum, same as ch 21's query(0,8)
print(f"prefix(5) = {ft.prefix(5)}")          # 25 = 1+3+5+7+9
print(f"range_sum(3, 5) = {ft.range_sum(3, 5)}")  # 21 = 5+7+9, same as ch 21's query(2,5)

# Update position 4 (1-indexed) by +93 - the underlying value goes from 7 to 100,
# matching ch 21's update(3, 100) on its 0-indexed segment tree.
ft.update(4, 93)
print(f"after update(4, +93), range_sum(3, 5) = {ft.range_sum(3, 5)}")  # 114 = 5+100+9
print(f"after update, prefix(8) = {ft.prefix(8)}")                       # 157 = 64 + 93
```

```
prefix(8) = 64
prefix(5) = 25
range_sum(3, 5) = 21
after update(4, +93), range_sum(3, 5) = 114
after update, prefix(8) = 157
```

The numbers line up exactly with chapter 21's segment-tree demo on the same input. Same problem, same answers, half the code. The class I just wrote is twenty-something lines total, including the constructor's linear-time build; the segment tree was eighty.

22.4 Trace by hand on n=8

The bit arithmetic isn't obvious until you watch it on small numbers. For `prefix(7)`: start at $i = 7$ (binary 0111), lowbit 1, read `tree[7]`, next index 6. At $i = 6$ (0110), lowbit 2, read `tree[6]`, next index 4. At $i = 4$ (0100), lowbit 4, read `tree[4]`, next index 0. Loop ends.

Three cells read. By the layout table, `tree[7]` covers $a[7..7]$, `tree[6]` covers $a[5..6]$, and `tree[4]` covers $a[1..4]$. Their union is $a[1..7]$ with no overlap — exactly what I want `prefix(7)` to return.

```
def lowbit(i: int) -> int:
    return i & -i

print("prefix(7) chain - indices visited, subtracting lowbit each step:")
i = 7
while i > 0:
    print(f"  i = {i}  (binary {i:04b})  lowbit = {lowbit(i)}  -> next {i - lowbit(i)}")
    i -= lowbit(i)
```

```
prefix(7) chain - indices visited, subtracting lowbit each step:
  i = 7  (binary 0111)  lowbit = 1  -> next 6
  i = 6  (binary 0110)  lowbit = 2  -> next 4
  i = 4  (binary 0100)  lowbit = 4  -> next 0
```

The symmetric trace for `update(3, ...)` on the same array walks upward.

```
print("update(3, +delta) chain on n=8 - indices touched, adding lowbit:")
i, n = 3, 8
while i <= n:
    print(f"  i = {i}  (binary {i:04b})  lowbit = {lowbit(i)}  -> next {i + lowbit(i)}")
    i += lowbit(i)
```

```
update(3, +delta) chain on n=8 - indices touched, adding lowbit:
  i = 3  (binary 0011)  lowbit = 1  -> next 4
  i = 4  (binary 0100)  lowbit = 4  -> next 8
  i = 8  (binary 1000)  lowbit = 8  -> next 16
```

Three cells touched: `tree[3]`, `tree[4]`, `tree[8]`. All three intervals include position 3, and they're exactly the cells whose stored sum has to absorb the delta when $a[3]$ changes. Cells like `tree[1]`, `tree[2]`, `tree[5]` you can leave alone, because none of their intervals contain position 3.

`lowbit(8) = 8`. The whole value is its own lowest set bit, because 8 is a power of two — only one bit is set. Powers of two are the cells with the tallest intervals; `tree[2^k]` covers $a[1..2^k]$. That asymmetry is what makes the upward walk from any position eventually land in a tall cell covering a long prefix — exactly the structural property I need for the $O(\log n)$ bound.

```
# lowbit on a few representative integers - the asymmetry at powers of two
# is the key intuition for why the upward walk reaches tall cells quickly.
for i in [1, 6, 7, 8, 12, 16]:
    print(f"  lowbit({i:>2}) = {lowbit(i):>2}   binary i = {i:08b}")
```

```
  lowbit( 1) =  1   binary i = 00000001
  lowbit( 6) =  2   binary i = 00000110
  lowbit( 7) =  1   binary i = 00000111
  lowbit( 8) =  8   binary i = 00001000
  lowbit(12) =  4   binary i = 00001100
  lowbit(16) = 16   binary i = 00010000
```

You see that 1, 2, 4, 8, 16 are exactly the powers of two, and their lowbit equals themselves. Every other integer's lowbit is some smaller power of two — the value of its rightmost 1.

22.5 Same answers, half the code

The cross-check: build a `FenwickTree` and a `SegmentTree` on identical data and assert their range-sum answers match across five random ranges.

```
import random
from codex.trees.fenwick import FenwickTree
from codex.trees.segment import SegmentTree

# Same data, two structures. The Fenwick tree is 1-indexed and uses
# inclusive ranges [l, r]; the segment tree is 0-indexed and uses
# half-open ranges [l, r). The translation: fenwick.range_sum(l, r)
# matches segment.query(l - 1, r).
data = [1, 3, 5, 7, 9, 11, 13, 15]
ft = FenwickTree(data)
st: SegmentTree[int] = SegmentTree(data)

random.seed(7)
for _ in range(5):
    l = random.randint(1, 8)
    r = random.randint(l, 8)
    f = ft.range_sum(l, r)
    s = st.query(l - 1, r)   # half-open: [l-1, r) covers the same positions
    print(f"  range [{l},{r}]  fenwick = {f:>3}  segment = {s:>3}  match = {f == s}")
```

```
  range [6,6]  fenwick =  11  segment =  11  match = True
  range [7,7]  fenwick =  13  segment =  13  match = True
  range [2,8]  fenwick =  63  segment =  63  match = True
  range [2,4]  fenwick =  15  segment =  15  match = True
  range [1,4]  fenwick =  16  segment =  16  match = True
```

Five ranges, five matches. You're watching two structures answer the same question with different machinery: the segment tree recursively descends to find the $O(\log n)$ canonical tiles, then sums them; the Fenwick tree walks two $O(\log n)$ chains of pre-aggregated cells and subtracts. Same answer, same asymptotic cost, half the code.

22.6 Range queries via two prefix queries

The synthesis `range_sum(l, r) = prefix(r) - prefix(l - 1)` is what lets a structure designed for prefix sums answer arbitrary range queries. That trick is the deep reason the Fenwick tree exists at all — without it, a prefix-sum structure couldn't compete with a segment tree on general range queries, and there'd be no point trading away generality.

The catch is that the trick relies on **subtraction**, which means the aggregate needs an inverse. For sum, the inverse of "$+x$" is "$-x$", so prefix subtraction gives you ranges. For xor, the inverse of "$\oplus\, x$" is "$\oplus\, x$" again (xor is its own inverse), so the same trick works. For count-of-trues (each `a[i]` is 0 or 1, and `prefix(i)` counts the ones up to position i), addition is the aggregate, count subtraction works.

For min and max, there's no inverse. If I know $\min(a[1..r])$ and $\min(a[1..l-1])$, I cannot recover $\min(a[l..r])$ — the minimum of $a[1..l-1]$ might or might not lie inside $a[l..r]$, and there's no arithmetic operation that decides which. So the Fenwick tree doesn't help with range min/max, and the segment tree's three-case tile decomposition stays the right approach for those.

That restriction isn't a flaw — it's the price of the simplicity. The segment tree is the general structure; the Fenwick tree is its specialization to invertible aggregates. When the problem fits, take the specialization. When it doesn't, you go back to the segment tree.

22.7 Beyond sum: XOR, count, and what doesn't fit

Replace every `+=` with `^=` in the Fenwick code and you get a Fenwick tree that handles xor over prefixes. The structure is identical; only the per-cell operation changes.

```
class FenwickXOR:
    def __init__(self, n_or_data: int | Sequence[int]) -> None:
        if isinstance(n_or_data, int):
            self._n = n_or_data
            self._tree: list[int] = [0] * (self._n + 1)
        else:
            data = list(n_or_data)
            self._n = len(data)
            self._tree = [0] * (self._n + 1)
            for i in range(1, self._n + 1):
                self._tree[i] ^= data[i - 1]
                parent = i + (i & -i)
                if parent <= self._n:
                    self._tree[parent] ^= self._tree[i]

    def __len__(self) -> int:
        return self._n

    def update(self, i: int, delta: int) -> None:
        while i <= self._n:
            self._tree[i] ^= delta
            i += i & -i

    def prefix(self, i: int) -> int:
        total = 0
        while i > 0:
            total ^= self._tree[i]
            i -= i & -i
        return total
```

You can read it as `FenwickTree` with `+=` replaced by `^=` and the initial accumulator 0 reinterpreted as the xor identity (which is also 0, conveniently). The same code generalizes because xor is a commutative group with an inverse — exactly the property the prefix-subtraction trick needs. The `update(i, delta)` method now xors `delta` into position i; querying via `prefix` returns the xor of `a[1..i]`.

```
from codex.trees.fenwick import FenwickXOR

fx = FenwickXOR([1, 2, 3, 4, 5])
print(f"prefix(5) = {fx.prefix(5)}")          # 1 - that's 1 ^ 2 ^ 3 ^ 4 ^ 5
print(f"expected  = {1 ^ 2 ^ 3 ^ 4 ^ 5}")     # 1

# XOR position 3 with 10 - a[3] becomes 3 ^ 10 = 9
fx.update(3, 10)
print(f"after update(3, ^10), prefix(5) = {fx.prefix(5)}")  # 11
print(f"expected = {1 ^ 2 ^ (3 ^ 10) ^ 4 ^ 5}")             # 11
```

```
prefix(5) = 1
expected  = 1
after update(3, ^10), prefix(5) = 11
expected = 11
```

For **count-of-trues** I don't need a new class at all — just store 0/1 indicators in a `FenwickTree` and `prefix(i)` returns how many of `a[1..i]` are set. That's the canonical use case for the BIT in competitive programming, where it answers questions like "how many points have I seen with coordinate $\leq x$ so far?" in $O(\log n)$ per insertion and $O(\log n)$ per query.

For **min and max**, no inverse exists, so the same code with `+=` replaced by `min`-of-pair simply doesn't work — the linear-time build still produces a valid prefix-min structure, but the point update can't tell whether removing a value would change the prefix minimum without rescanning the affected interval. The Fenwick approach doesn't extend. If you need range min, reach for the segment tree from chapter 21, or for the **sparse table** trick (offline range-min in $O(1)$ per query after $O(n \log n)$ preprocessing, no updates) — material I won't develop here.

22.8 A wall-clock comparison

Same workload — point updates and range sums interleaved on $n = 10^5$ — on both structures. The asymptotic cost is identical at $O(\log n)$, but the Fenwick tree's per-operation work is a much smaller constant: a three-line `while` loop with one integer add and one bit-and per iteration, versus the segment tree's recursive call frames and three-case branch.

```
import random
import time
from codex.trees.fenwick import FenwickTree
from codex.trees.segment import SegmentTree

n, ops = 100_000, 10_000

random.seed(0)
vals = [random.randint(0, 1000) for _ in range(n)]

# Fenwick: linear-time build + ops random point-updates + range-sums
t0 = time.perf_counter()
ft = FenwickTree(vals)
random.seed(1)
for _ in range(ops):
    i = random.randint(1, n)
    ft.update(i, 1)
    l = random.randint(1, n)
    r = random.randint(l, n)
    _ = ft.range_sum(l, r)
t_fen = time.perf_counter() - t0

# Segment tree: same data, same operation mix
t0 = time.perf_counter()
st: SegmentTree[int] = SegmentTree(vals)
random.seed(1)
for _ in range(ops):
    i = random.randint(1, n)
    st.update(i - 1, vals[i - 1] + 1)   # set semantics, so emulate "+1" by setting
    l = random.randint(1, n)
    r = random.randint(l, n)
    _ = st.query(l - 1, r)
t_seg = time.perf_counter() - t0

print(f"fenwick:  {t_fen * 1000:>7.1f} ms")
print(f"segment:  {t_seg * 1000:>7.1f} ms")
print(f"ratio:    {t_seg / t_fen:.2f}x faster on Fenwick")
```

```
fenwick:     60.9 ms
segment:    190.0 ms
ratio:    3.12x faster on Fenwick
```

On my machine the Fenwick tree is roughly 2-4x faster than the segment tree on this Python workload, which is the constant-factor win you'd expect from a tighter loop. The exact ratio is Python-interpreter-sensitive and

will vary, but the direction is robust: when the prefix-sum reduction applies, the Fenwick tree's smaller constant pays off in real wall-clock time.

22.9 The three questions, applied

Is it correct?

The invariant: **`tree[i]` always equals the sum of `a[i - lowbit(i) + 1 .. i]`** under the current state of the underlying array. The constructor establishes it (either by allocating zeros, which is trivially correct for the implicit all-zero array, or by running the linear-time build pass, which folds each cell's contribution into its parent in a single sweep). `update` preserves it: when I add `delta` to position j, the cells whose intervals contain j are exactly the chain $j, j + \text{lowbit}(j), \ldots$, and `update` walks that chain adding `delta` to each. `prefix(i)` reads the chain $i, i - \text{lowbit}(i), \ldots, 0$, and those cells' intervals form a partition of $a[1..i]$ (no overlap, no gaps), so the sum of the cells equals the sum of the array up to position i.

The partition claim is the heart of the correctness argument. Each cell `tree[k]` along the downward chain owns an interval that ends at the position the *previous* chain step started from — `tree[6]` covers $a[5..6]$ and the next chain step is index 4, so `tree[4]` covers $a[1..4]$ — the two intervals are adjacent, not overlapping. By induction on the binary representation of i, the chain's intervals tile $a[1..i]$ exactly.

`range_sum` is correct because it's `prefix(r) - prefix(l - 1)`, and integer subtraction recovers the partial sum exactly. The same argument applies to `FenwickXOR` once you replace addition with xor and the additive inverse with xor itself.

How efficient is it?

Cost table for a Fenwick tree on n positions:

Operation	Time	Extra space
`FenwickTree(n)` (empty allocation)	$\Theta(n)$	$\Theta(n)$
`FenwickTree(data)` (linear build)	$\Theta(n)$	$\Theta(n)$
`update(i, delta)`	$O(\log n)$	$O(1)$
`prefix(i)`	$O(\log n)$	$O(1)$
`range_sum(l, r)`	$O(\log n)$	$O(1)$

The $O(\log n)$ bound on both walks comes from the bit arithmetic I traced earlier: the upward walk's chain length equals one plus the position of the highest set bit of $n/\text{lowbit}(i)$, and the downward walk's chain length equals the number of set bits in i. Both are bounded by $\lfloor \log_2 n \rfloor + 1$.

On an array of a million elements, every operation you run touches at most about 20 cells; on a billion, about 30. Storage is $n + 1$ slots — one array, no overhead, no over-allocation. The segment tree from chapter 21 needed $4n$ slots; the Fenwick tree needs exactly $n + 1$. That's a 4x memory win on top of the constant-factor time win.

Is it optimal?

For the point-update + prefix-sum workload, $O(\log n)$ per operation is asymptotically optimal in the cell-probe model — the same Pătraşcu–Demaine lower bound that bounded the segment tree applies here. Any structure supporting n point updates and n prefix-sum queries must do $\Omega(n \log n)$ total work, so per-operation $o(\log n)$ is ruled out. The Fenwick tree matches the bound exactly.

The constant factor is small enough that no general-purpose data structure beats it on prefix sums. Specialized structures can do better in narrow regimes — if the array is static, a single $\Theta(n)$ prefix-sum array gives you $O(1)$ queries forever; if updates are batched, you can amortize differently. For the interleaved workload of arbitrary updates and arbitrary prefix queries, the Fenwick tree is the engineering winner.

You give up generality to get there: invertible monoids only (sum, xor, count); no range update without elaboration; no min or max. For workloads that fit, the Fenwick tree is the right answer.

22.10 The lowest set bit is the structure

Both operations are three-line `while` loops over a chain of cells determined by repeatedly adding or subtracting `i & -i`. The "tree" is implicit in the bit pattern, the storage is a single n-slot array, and the per-operation cost is $O(\log n)$ with a small constant.

Specialize when the problem is narrower than the structure you have. Chapter 21's segment tree handles arbitrary associative monoids over arbitrary ranges. For just prefix sums of an invertible aggregate, that's overkill. The Fenwick tree is what you get when you ask "what's the minimum structure that solves this restricted problem?". The answer is half the code and a smaller constant. Every time a fully general structure is doing the job, check whether the actual workload is narrower than its interface.

The right encoding can be a bit pattern. Chapter 19's heap encoded a complete binary tree in array indices via $2i+1$, $2i+2$, $(i-1)/2$. Chapter 21's segment tree did the same with $2 \cdot$ node. The Fenwick tree pushes one level further: the tree shape lives entirely in the *bits* of the index you're using, and the operations are bit-manipulation tricks. No nodes, no pointers, no levels — an array and two `while` loops.

The existence of an inverse is the property that matters. I can answer range queries from prefix queries when, and only when, the aggregate supports subtraction. Sum and xor work; min and max don't. The Fenwick tree isn't a "sum tree" — it's an *invertible-monoid* tree, and the same code body handles any commutative group. When you ask "can I trade generality for simplicity here?", check whether the aggregate has an inverse. If it does, half the code might be on the table.

That closes Part III. Across nine chapters you've built binary trees, BSTs, AVL trees, treaps, heaps, tries, segment trees, and now Fenwick trees — each one a different answer to "what shape should the data have so the operations are cheap?"

22.11 Notes and further reading

The Fenwick tree was introduced by Peter Fenwick in his 1994 paper "A New Data Structure for Cumulative Frequency Tables" in *Software: Practice and Experience* 24(3):327–336. Fenwick's original motivation was arithmetic coding — he needed cumulative frequency lookups in a dynamic alphabet, and the existing data structures were too heavy. The paper is short, readable, and worth tracking down for the historical perspective on how a structure this clean took until 1994 to appear in print.

Neither CLRS nor Sedgewick & Wayne's *Algorithms* (4th ed.) covers the Fenwick tree — like the segment tree, its canon is the competitive-programming literature. The CP-Algorithms encyclopedia entry at cp-algorithms.com/data_structures/fenwick.html is the definitive reference, covering the basic point-update / prefix-query variant (sometimes called PURQ, "point update, range query"), the dual range-update / point-query variant (URPQ, achieved by storing differences and querying prefixes), and the more elaborate range-update / range-query variant that requires two parallel Fenwick trees and is genuinely intricate. Halim and Halim's *Competitive Programming* (4th ed.) chapter 2.4 has the same material with worked problem sets.

For two-dimensional generalizations — 2D Fenwick trees for prefix sums over rectangles — see the CP-Algorithms entry's later sections, and Henning Tan and Petteri Kaski's 2007 *Information Processing Letters* note "Faster Range Tree Queries Using Bit Operations." Two dimensions raise the per-operation cost to $O(\log^2 n)$ but keep the same single-array layout, which is still a strong win over a 2D segment tree.

Lessons from Part III

Eight structures in eight chapters, and the surprise — to me, again — is how much of what felt like new material was Part II's atoms rearranged. The dynamic array from chapter 8 is now the substrate every heap, segment tree, and Fenwick tree sits on. The linked-node idiom from chapter 9 is what every BST, AVL tree, and treap uses for its pointers. The hashing trick from chapter 12 still drives the trie's child-lookup. The dense-array-with-implicit-pointers trick from chapter 14's union-find returned twice (heaps, segment trees) and then got compressed once more (Fenwick). Trees didn't replace what came before; they organized it into hierarchies.

The chapters themselves only had time to gesture at the patterns that recurred across all eight. Naming those patterns, and then sketching the bridge into Part IV — where the data being walked stops being abstract keys and starts being characters in a string, a domain you've already touched once in chapter 20 — is what the rest of this essay is for.

The patterns that kept showing up

The implicit-array-as-tree pattern. Three chapters in this part — heaps (19), segment trees (21), and Fenwick trees (22) — encode their tree shape entirely in *index arithmetic* on a flat array, with no `Node` allocations at all. Chapter 14's union-find was the prototype. The win is real: you save one pointer per node (or eight on a 64-bit machine), you get cache locality for free because the structure is one contiguous block, and operations that would have been pointer-chases become index arithmetic the CPU's branch predictor and prefetcher handle for you. The cost is that the structure has to be **shape-rigid** — heaps are always complete binary trees, segment trees are always size-$4n$, Fenwick trees are always 1-indexed-and-power-of-two-friendly. You give up topological flexibility to get the storage and locality wins, and that's a trade you can make explicitly because the operations don't need flexibility.

Rotations as the universal repair operation. Two chapters — AVL (17) and treaps (18) — use the same three-pointer-write primitive to fix shape problems while preserving the BST invariant. The rotation is $\Theta(1)$ work, it changes the *shape* of a small piece of the tree, and it leaves the in-order ordering of keys untouched. That property — *can shuffle the topology, can't shuffle the in-order* — is what makes rotations the move every self-balancing BST reaches for. Red-black trees, splay trees, AA trees, scapegoat trees, and weight-balanced trees all use rotations as their structural verb. Different invariants, same primitive.

Balance as invariant, deterministic or probabilistic. Chapter 17 enforced balance with a hard rule (every balance factor in $\{-1, 0, +1\}$, checked at every node, repaired with rotations). Chapter 18 enforced it with random priorities and a probabilistic argument (expected height $O(\log n)$ over the choice of priorities). Two strategies for the same goal — keeping $h = O(\log n)$ regardless of insertion order — with different trade-offs. AVL guarantees the bound on every input; treaps guarantee it on every input *in expectation*, which is enough in the absence of an adversary who can read your random seed. The pattern returns in skip lists (Part IX), in randomized quicksort (Part I), in Rabin-Karp's rolling hash (Part IV), in cuckoo hashing — randomization makes the worst case statistically unreachable, and absent an adversary that's as good as eliminating it outright.

Local invariants, global cost. The AVL property is checkable at one node, with one integer subtraction, in $O(1)$ time. Yet enforcing it everywhere guarantees the whole tree has height $\leq 1.44 \log_2(n+2)$ — a *global* shape guarantee derived from a *local* rule. The same pattern lives in the heap order (parent $\leq$ children, checked locally; root is the minimum, a global property), in the BST property (smaller-left bigger-right, local; in-order is sorted, global), in segment trees (every node holds its subtree's aggregate, local; range queries decompose into $O(\log n)$ tiles, global). Local rules that yield global cost shapes is one of the deepest patterns in data structure design, and you'll see it again in B-trees, in persistent functional structures, and in distributed-consensus protocols.

Path-encodes-key. Chapter 20's trie was the only structure in this part where the *positions* on the tree carried meaning beyond ordering. In every other tree, the key was stored in a node; in the trie, the key was *spelled out* by the sequence of edges from root to terminal node. That choice is the one that gives tries their prefix-query superpower (every prefix is a subtree) and their pointer-overhead cost (every character is a node). The same

idea returns in Part IV — suffix trees and suffix automata extend the "every position spells something" pattern to handle every substring of a long text — and in Part VIII when geometric algorithms walk position-encoding trees like k-d trees and BSP trees.

Discipline as restriction, scaled up. Stacks and queues in chapters 10 and 11 made their case for restriction by *forbidding* most operations. Part III rhymes: heaps refuse everything except "smallest" and "add one more"; segment trees refuse arbitrary mutation and only allow point-update / range-query. Each restriction buys you a cost shape the unrestricted version of the structure couldn't deliver. The pattern stays the same across the scaling — discipline lets you reason about workloads without tracking individual contents, and the deeper you go into the book the more often you'll reach for it.

Six meta-lessons

1. $\Theta(\log n)$ is the right cost shape for sorted-data-that-changes. Part I gave you $\Theta(n)$ on unstructured input. Part II gave you $\Theta(1)$ on unordered input. Part III gave you $\Theta(\log n)$ on input that has to *both* maintain an order *and* tolerate insertions and deletions. That gap is where ordered maps live, where database indexes live, where priority-driven scheduling lives, where range-query analytics live, where IP routing tables live. You'll spend more of your engineering career working with $\Theta(\log n)$ structures than with either of the others.

2. The shape of the tree is sometimes hidden in an integer. Heaps, segment trees, Fenwick trees, and union-find all live in flat arrays. The tree is conceptual — implied by how you index into the array — and the savings from that decision (no node allocations, cache-friendly layout, predictable branches) are decisive in practice. When you reach for a tree structure, ask whether the operations you need actually require an explicit node graph. Often they don't.

3. A rotation is the cheapest structural change that respects ordering. Three pointer writes, $O(1)$ time, preserves in-order. Once you've seen it work in AVL and again in treaps, you'll recognize it in red-black trees, in splay trees, and in the more exotic self-adjusting structures from the research literature. The invariant changes from structure to structure; the primitive that maintains it is almost always a rotation.

4. Randomization is a balance strategy, not a fallback. Treaps got the same $O(\log n)$ expected height as AVL's worst-case bound, with simpler code, by *randomizing the shape* of the tree. The trade — worst-case vs. expected, deterministic vs. probabilistic — is one you'll make repeatedly. In the absence of an adversary, expected bounds and worst-case bounds give you the same engineering guarantee. When you have an adversary (an attacker who controls your input), you want worst-case. When you don't (most workloads), the simpler randomized structure is often the right call.

5. Recognize the monoid behind sum-and-max. The segment tree didn't just give you range sum — it gave you range *aggregate* over any associative operation with an identity. Sum, min, max, xor, gcd — and, less obviously, matrix product or polynomial composition — all plug into the same code. The Fenwick tree is more restrictive (it needs invertibility), but the same lens applies. When you find yourself writing structure-specific code for sum-only or max-only, ask whether the algebra of the operation is what your data structure actually depends on. The right abstraction is the monoid or the group, not the specific operation.

6. The right structure for *this* workload is rarely the textbook answer for "the" workload. Production databases use B-trees, not red-black trees, because disk reads dominate. Network routers use radix tries, not hash tables, because longest-prefix matching is the actual query. Compilers use segment-tree-like interval skip-lists for liveness analysis. Game engines use spatial trees that don't appear in this part at all (quadtrees, k-d trees, BSP trees — they're in Part VIII). The skill the rest of the book keeps building is **knowing the menu**, and the patterns from Part III show up on the menu under many different names.

Bridge to Part IV

Part IV is a change of subject more than a change of altitude. The next eight chapters specialize the algorithms you have to *strings* — sequences of characters drawn from a finite alphabet, with operations like "find this pattern", "list all matches", "compute the edit distance", "build an index over a long text". You already met one string structure in this part: the trie. Part IV extends it.

The arc starts where every algorithms textbook starts on strings — pattern matching. The brute-force algorithm scans the text with a sliding pointer; Knuth-Morris-Pratt computes a failure function that lets the pointer never go backwards; Boyer-Moore uses *both* bad-character and good-suffix heuristics to skip whole regions of text;

Rabin-Karp computes rolling fingerprints over the text and matches in expected $O(n + m)$. Each is its own answer to "how much preprocessing buys how much query speed", and each builds on a primitive you've already met — KMP on the deterministic-finite-automaton view of patterns, Boyer-Moore on the table-lookup trick from counting sort, Rabin-Karp on the hashing trick from chapter 12.

The middle of Part IV is where the indexing structures live. Aho-Corasick stitches a trie together with failure links to match thousands of patterns against a text in one pass — it's the algorithm `grep -F` ships on. Suffix arrays and the LCP array compress an index over every substring of a long text into $O(n)$ space, with the Manber-Myers $O(n \log n)$ construction as the workhorse. Edit distance closes the part with the canonical dynamic-programming recurrence, threading the bridge to Part VI.

The Part III atoms keep paying out. Tries grow into Aho-Corasick. Hashing grows into Rabin-Karp. The implicit-array-as-tree trick grows into suffix-array-with-tournament. The discipline of restricting operations turns into the discipline of restricting alphabets. You won't be learning new structures so much as watching the trees you already know take on a string-shaped weight.

Eight chapters, eight structures, one part. Trees turned out to be how you get from $\Theta(n)$ down to $\Theta(\log n)$ on data that has to keep changing. Onward.

Part IV

String Algorithms

The structures from Parts II and III worked with arbitrary keys — anything you could hash, compare, or hand to a comparator. Strings are different. The order of the characters and the content of nearby positions both carry information the previous parts' structures threw away, and the questions you naturally ask about strings — where does this pattern occur, where does any of these patterns occur, how do I index a text so future queries are cheap, how close are two strings when they don't match exactly — don't admit the same answers as questions about unordered keys.

The arc of this part is a single parameter family. Each chapter trades preprocessing for query speed differently.

- **Brute force** (chapter 23) does no preprocessing and pays $O(nm)$ on adversarial inputs. It's still what `str.find` ships, because its tight inner loop wins on cache-friendly hardware whenever the average case matters more than the worst case.
- **Finite automaton matching** (chapter 24) compiles the pattern into a DFA — one state per length of matched prefix — and scans the text in $\Theta(n)$. Preprocessing is $\Theta(m \cdot |\Sigma|)$, and the construction is the conceptual bridge to everything that follows.
- **Knuth-Morris-Pratt** (chapter 25) compresses that DFA into one integer per state — the failure function. Storage drops to $\Theta(m)$, the scan stays $\Theta(n)$, and the chapter rebuilds chapter 24's DFA cell-by-cell from the failure function to prove the compression is lossless.
- **Boyer-Moore** (chapter 26) flips the scanning order. Reading windows right-to-left exposes a class of skips left-to-right algorithms structurally can't see — when the rightmost character of a window isn't in the pattern, you can shift past the whole window. Sublinear in practice on natural text.
- **Rabin-Karp** (chapter 27) stops comparing strings and starts comparing fingerprints. A polynomial rolling hash updates in $O(1)$ per shift; equal hashes trigger a verification step. Las Vegas — always correct, sometimes slow — and the randomization-as-balance argument echoes the treap chapter from Part III.
- **Aho-Corasick** (chapter 28) generalizes the failure-function idea to a *trie* of patterns, scanning the text once and reporting every match of every pattern. The algorithm `grep -F` actually ships.
- **Suffix arrays + LCP** (chapter 29) preprocess the *text* instead of the pattern. Every substring of a text is a prefix of some suffix, so the sorted suffix array reduces pattern search to a binary search. Manber-Myers builds in $O(n \log^2 n)$ and Kasai computes LCPs in $O(n)$.
- **Edit distance and alignment** (chapter 30) drops exact matching. The DP grid of prefix-pair distances costs $O(nm)$ to fill, and the same grid handles longest-common-subsequence and longest-common-substring with small changes to the per-cell recurrence. Part VI's chapter on dynamic programming reaches back to this one rather than re-implementing.

Three patterns recur across the eight chapters. Preprocessing-versus-query is the part's organizing axis — every chapter sits at a different point on the curve. Automata are the universal lens — the DFA from chapter 24 is the conceptual ancestor of every left-to-right matcher in the part and most of the right-to-left ones too. And randomization at the boundary — Rabin-Karp's rolling hash, like the treaps from Part III — buys a different cost shape by making the worst case statistically unreachable. Part V picks up where Aho-Corasick leaves off: the trie was already a graph, and the next eight chapters let the edges go anywhere.

23 Sliding the pattern, comparing every shift

You have a long string — call it the *text* — and a much shorter string — call it the *pattern* — and you want to know where, if anywhere, the pattern occurs inside the text. That is the pattern-matching problem in its rawest form, and it is the question every algorithm in this part is going to answer, each one trading more preprocessing for less scanning. I want to start where the field starts, with the algorithm that does no preprocessing at all and gets surprisingly far on the strength of a simple double loop.

By the end of this chapter you'll have a `find` that returns the first index where the pattern appears, a `find_all` that returns every overlapping occurrence, an instrumented variant that counts the comparisons it does so you can watch the worst case fire in real numbers, and a clear-eyed sense of why this algorithm — usually labelled "brute force" with a faint sneer — is what `grep -F` and most production `memmem` implementations actually run when the constants matter more than the asymptote.

A window of length m slides across n positions and tries each one. That is the whole algorithm. There is no failure function, no rolling hash, no automaton. Just a window, a slide, and a character-by-character comparison at every shift. Every other pattern-matching algorithm in this part is the brute force, but skipping shifts I can prove are hopeless. The brute force is the baseline against which everything else justifies itself.

23.1 Finding a needle in a string haystack

I want to be precise about the problem before writing anything. Given a text T of length n and a pattern P of length m, with $m \le n$, find an index i in $0 \le i \le n - m$ such that $T[i..i+m-1] = P$ — that is, the substring of T starting at i and running for m characters matches P exactly. If no such i exists, report failure. For `find_all`, return every such i.

That is the spec `grep -F` is solving when you search for a fixed string (no regex). It is the spec your editor is solving when you press Ctrl-F. It is, with small variations, the spec a packet-inspection firewall is solving when it scans a stream for a signature. The reason this problem deserves an entire part of the book is that the obvious algorithm — try every starting position — has a worst-case behaviour bad enough to motivate four genuinely different cleverness moves, and an average-case behaviour good enough that the obvious algorithm wins anyway in the shape of most production code.

The first thing you might try is exactly what I'll write down in the next section: line the pattern up at position 0, compare it character by character, and if there's a mismatch, slide one position to the right and start again. There are $n - m + 1$ possible starting positions and the comparison at each one takes at most m work, so the worst case is $O(nm)$ character comparisons. For a text of a million characters and a pattern of ten, that is ten million comparisons in the worst case and roughly a million in the average case — both numbers a modern CPU finishes in milliseconds. The story of why $O(nm)$ is "too much" and what to do instead starts in the next chapter; this chapter is about what $O(nm)$ actually costs.

23.2 The double loop

Here is the simplest pattern matcher you can write. Two nested loops: the outer one slides a window of length m along the text, the inner one walks the window and compares character by character. The instant a character disagrees, I bail out of the inner loop and slide the window one to the right.

```
def find(text: str, pattern: str) -> int | None:
    n, m = len(text), len(pattern)
    if m == 0:
        return 0
    if m > n:
        return None
```

```
    for i in range(n - m + 1):
        j = 0
        while j < m and text[i + j] == pattern[j]:
            j += 1
        if j == m:
            return i
    return None
```

The empty-pattern case returns 0 — by convention every string contains the empty string at position 0, matching Python's own `"".find("")` and `re.search("", ...)`. The pattern-longer-than-text case returns `None` immediately, since no window can fit. The body is the double loop I just described: for each shift i, advance j as long as characters match, and if j reaches m then I've matched the whole pattern starting at i. Otherwise slide on.

Bilingually: I am holding a window of length m and sliding it one position to the right at a time across the text; at each position I read characters until I find a disagreement, and the first position where I read all the way through is the match. There is nothing being remembered between shifts — every shift starts the inner comparison from scratch at $j = 0$, which is exactly the wastefulness chapter 25's KMP will eventually fix.

Now `find_all`. I want every starting index where the pattern occurs, and I have to make one design decision: do I report **overlapping** matches or **non-overlapping** ones? Consider the text `"aaaa"` and the pattern `"aa"`. The overlapping interpretation reports positions 0, 1, 2 — three matches that share characters. The non-overlapping interpretation reports 0, 2 — two matches that tile the text without sharing. Both are defensible. I'll pick overlapping, because that's the convention `re.finditer` uses for patterns without lookahead and it's the convention every subsequent chapter in this part will measure itself against. The cost is one extra integer add at each match site, which is nothing.

```
def find_all(text: str, pattern: str) -> list[int]:
    n, m = len(text), len(pattern)
    if m == 0:
        return []
    if m > n:
        return []
    matches: list[int] = []
    for i in range(n - m + 1):
        j = 0
        while j < m and text[i + j] == pattern[j]:
            j += 1
        if j == m:
            matches.append(i)
    return matches
```

Same double loop as `find`, except instead of returning on the first complete match I append the index and keep going. The empty-pattern case returns an empty list — there is one reasonable convention here ("every position is a match") and one practical convention ("the empty pattern is degenerate, report nothing"), and I'm picking the practical one to avoid `find_all("hello", "")` returning a length-six list of zeros nobody asked for. That is a documented choice; if you want every-position semantics, you write three extra lines.

The walk is identical to `find`'s. The overlap behaviour is implicit in `range(n - m + 1)` — each shift increments by 1, not by `m`, so a match at position 0 doesn't stop me from finding another at position 1.

23.3 Running it on the canonical text

I'll use a single text string across every chapter in this part, so you can compare algorithms side by side without having to re-orient on a new sentence each time. The text is short enough to read at a glance and rich enough to have interesting matches.

```
from codex.strings.brute import find, find_all

TEXT = "the rain in spain stays mainly in the plain"

idx = find(TEXT, "ain")
print(f"first match of 'ain' at index {idx} - that's the start of 'ain' in 'rain'")

all_ain = find_all(TEXT, "ain")
print(f"all matches of 'ain': {all_ain}  (four occurrences: rain, spain, mainly, plain)")

all_in = find_all(TEXT, "in")
print(f"all matches of 'in': {all_in}  (six occurrences across the sentence)")

all_the = find_all(TEXT, "the")
print(f"all matches of 'the': {all_the}  (two - opening word and 'the plain')")

miss = find(TEXT, "frog")
print(f"searching for 'frog': {miss}  (no frogs in this sentence)")
```

```
first match of 'ain' at index 5 - that's the start of 'ain' in 'rain'
all matches of 'ain': [5, 14, 25, 40]  (four occurrences: rain, spain, mainly, plain)
all matches of 'in': [6, 9, 15, 26, 31, 41]  (six occurrences across the sentence)
all matches of 'the': [0, 34]  (two - opening word and 'the plain')
searching for 'frog': None  (no frogs in this sentence)
```

The four `ain` matches sit inside `rain`, `spain`, `mainly`, and `plain`. The six `in` matches include those four plus the standalone words `in` (twice). The two `the` matches are the opening word and the one inside `the plain` at the end. Searching for `frog` returns `None` because there is no `f` anywhere in the text — the inner loop bails out at $j = 0$ for every shift.

To make the overlap behaviour I picked visible — and to convince you it actually matters — here is a deliberately overlap-prone input.

```
from codex.strings.brute import find_all

print(f"find_all('aaaa', 'aa') = {find_all('aaaa', 'aa')}  (overlapping: 0, 1, 2)")
print(f"find_all('abababab', 'aba') = {find_all('abababab', 'aba')}  (overlapping: 0, 2, 4)")
```

```
find_all('aaaa', 'aa') = [0, 1, 2]  (overlapping: 0, 1, 2)
find_all('abababab', 'aba') = [0, 2, 4]  (overlapping: 0, 2, 4)
```

The first call returns `[0, 1, 2]`, not `[0, 2]`. The second returns `[0, 2, 4]`, not just `[0]` or `[0, 4]`. The overlapping convention catches every position where the pattern starts, even when those positions share characters.

23.4 When the worst case bites

The double loop is $O(nm)$ in the worst case, but on most realistic inputs the inner loop bails out after one or two character comparisons and the algorithm runs much closer to $O(n)$. I want to show you the construction that actually realizes the worst case, because it tells you exactly what kind of input you'd have to feed `grep` to make it suffer.

The canonical pathological input is a text and pattern that *almost* match at every shift but disagree at the very last character of the pattern. The simplest example: text `"aaaa...aab"` (many `a`s followed by a `b`), pattern `"aaa...ab"` (slightly fewer `a`s followed by a `b`). At every shift the inner loop reads through almost all of the pattern before finding the disagreement. Every shift pays full price.

To make this measurable, I'll factor out a counting variant that returns both the answer and the number of character comparisons it performed.

```
def find_counting(text: str, pattern: str) -> tuple[int | None, int]:
    n, m = len(text), len(pattern)
    if m == 0:
        return 0, 0
    if m > n:
        return None, 0
    comparisons = 0
    for i in range(n - m + 1):
        j = 0
        while j < m:
            comparisons += 1
            if text[i + j] != pattern[j]:
                break
            j += 1
        if j == m:
            return i, comparisons
    return None, comparisons
```

I unrolled the inner `while` slightly so I can bump `comparisons` once per character read — the original `find`'s `while j < m and text[i + j] == pattern[j]` does the comparison implicitly in the loop condition, which makes counting awkward. This version is semantically identical but explicit about where the comparison happens. Now I can feed it the pathological input and watch the numbers.

```
from codex.strings.brute import find_counting

# pathological case: pattern almost matches at every shift, fails at the last char
text = "a" * 20 + "b"
pattern = "a" * 4 + "b"

result, comps = find_counting(text, pattern)
n, m = len(text), len(pattern)
shifts = n - m + 1

print(f"text = {text!r}  (n = {n})")
print(f"pattern = {pattern!r}  (m = {m})")
print(f"shifts tried = {shifts}")
print(f"comparisons = {comps}")
print(f"upper bound n*m = {n * m}")
print(f"match at index {result}")
```

```
text = 'aaaaaaaaaaaaaaaaaaaab'  (n = 21)
pattern = 'aaaab'  (m = 5)
shifts tried = 17
comparisons = 85
upper bound n*m = 105
match at index 16
```

You should see something close to $shifts \times m$ comparisons — every shift before the match reads almost the whole pattern before bailing, and the matching shift reads all of it. The exact count is shifts $\times$ m in this construction, which lands right on the $O(nm)$ ceiling. Bilingually: on this input, the algorithm does the maximum amount of work it's theoretically capable of doing — there's no shift it gets to skip cheaply, because every shift looks like a near-match.

Compare with a benign input where the inner loop bails out at $j = 0$ for almost every shift.

```
text = "the rain in spain stays mainly in the plain"
result, comps = find_counting(text, "frog")
n, m = len(text), 4
print(f"searching for 'frog' in the canonical text - n={len(text)}, m=4")
print(f"comparisons = {comps}  (close to n, because 'f' doesn't appear)")
```

```
searching for 'frog' in the canonical text - n=43, m=4
comparisons = 40  (close to n, because 'f' doesn't appear)
```

Searching for `"frog"` in the canonical text does only as many comparisons as there are shifts, because every shift fails on the first character — there are no `f`s for the inner loop to advance past. That is the average case in disguise: when the alphabet is large and the pattern's first character is rare, the inner loop is a near-no-op. The $O(nm)$ ceiling is a worst case, not a typical case, and the gap between them is what makes the brute force competitive in practice.

23.5 Why grep still ships it

Knowing that the worst case exists and knowing that production code uses it anyway are not in contradiction — production code knows the worst case exists and chooses to live with it, because the average case is fast and the constant factor is unbeatable. CPython's `str.find` runs a variant of this exact algorithm, called the *two-way* algorithm, which adds a small skip table on top of the naive scan but keeps the inner loop's character-by-character shape; GNU `memmem`'s default implementation uses a similar two-way variant with SIMD-friendly chunk reads when the pattern is short enough to fit in a register. The reason these libraries don't reach for KMP or Boyer-Moore on every call is that the brute-force inner loop is *one tight loop with no branches*, and that runs faster on modern hardware than any algorithm whose inner loop has to consult a precomputed table on every step. The asymptote loses; the cache wins.

23.6 The three questions, applied

Is it correct?

The invariant I'm maintaining is: **after the outer loop has examined shifts $0, 1, \ldots, i-1$ and not returned, the pattern does not occur at any of those positions.** The inner loop establishes that invariant for each i — it reads characters in order until it finds either a disagreement (so the pattern doesn't match at i, and I move on) or runs off the end of the pattern (so it matches, and I return i). When the outer loop terminates without returning, every shift in $0, \ldots, n-m$ has been examined and rejected, and no other shift is possible because a window of length m starting at $i > n-m$ would run past the end of the text. So `find` returns either the smallest matching index or `None` if none exists, which is exactly the spec.

`find_all` differs only in that it appends instead of returning. The invariant strengthens to: **after the outer loop has examined shifts $0, 1, \ldots, i-1$, `matches` contains exactly the indices in that range at which the pattern occurs.** When the loop ends, that's every shift, so `matches` is the complete overlapping-match set. `find_counting` is `find` with an extra integer that ticks every time the inner loop's equality check runs — it doesn't change what gets returned, only what gets reported alongside.

How efficient is it?

The cost is bounded by the number of character comparisons, which is at most m per shift over $n-m+1$ shifts. So the worst case is $O((n-m+1) \cdot m) = O(nm)$, achieved by the `"aaa...ab"` construction above. The best case is $\Omega(n)$ — every shift bails out at $j = 0$ — which happens whenever the pattern's first character is rare in the text.

Bilingually: on a text of a million characters and a pattern of ten, you'll do at most ten million comparisons and at least about a million. The average behaviour on natural-language text with a typical-length pattern sits much closer to the lower bound than the upper, because the pattern's first character only matches a small fraction of text positions and the inner loop bails after one comparison for the rest.

Space is $O(1)$ for `find` and `find_counting` — three integers and the index variables, nothing allocated per shift. `find_all` is $O(k)$ for the output list, where k is the number of matches.

Operation	Time (worst)	Time (best)	Space
`find(text, pattern)`	$O(nm)$	$\Omega(n)$	$O(1)$
`find_all(text, pattern)`	$O(nm)$	$\Omega(n)$	$O(k)$ output

Operation	Time (worst)	Time (best)	Space
`find_counting(text, pattern)`	$O(nm)$	$\Omega(n)$	$O(1)$

Is it optimal?

For exact pattern matching with no preprocessing, $\Omega(n)$ is the lower bound — you have to at least read every character of the text to be sure the pattern doesn't appear, because a single mismatch at any unread position could hide a match elsewhere. The brute force matches that lower bound in its best case but blows past it by a factor of m in the worst case, so it is *not* optimal in the worst-case sense.

The algorithms in the next four chapters all run in $O(n + m)$ worst-case time — the brute force's $O(n)$ best case made worst-case-tight. KMP gets there with a precomputed failure function that lets shifts reuse work from previous shifts. Boyer-Moore gets there (well, $O(nm)$ worst-case, $O(n/m)$ best-case) by reading the pattern right-to-left and skipping whole regions on a mismatch. Rabin-Karp gets there in expectation with a rolling hash that compares fingerprints instead of full strings. The finite-automaton view of chapter 24 is the conceptual bridge that explains why KMP and Aho-Corasick are the same idea in different costumes.

So the brute force is optimal in the best case and a factor of m off in the worst case — and yet it's the algorithm production code most often ships. The next four chapters are about closing the worst-case gap, and one of the things you'll watch is each algorithm fighting to keep the brute force's best-case behaviour while neutralizing its worst case.

23.7 What brute force teaches you

There's a temptation to read "brute force" as an insult — the algorithm you ship before you've thought hard about the problem. I want you to read it differently. The brute force is the algorithm that does *only the work the problem definition forces it to do*. It does not preprocess, it does not memoize, it does not skip. Every comparison it makes is one the spec couldn't have asked it not to make without giving it extra information.

What the next four chapters do is *give the algorithm extra information*. KMP gives it information about the pattern (a failure function precomputed in $O(m)$ time). Boyer-Moore gives it two tables — a bad-character table and a good-suffix table — also computed from the pattern. Rabin-Karp gives it a hash function over windows of the text. Aho-Corasick generalizes the failure-function idea to many patterns at once. Each chapter pays a preprocessing cost in exchange for skipping shifts the brute force would have examined, and each chapter justifies its cleverness against the brute force's tight inner loop.

The meta-lesson — and it's one you'll keep meeting in the rest of this part — is that *the obvious algorithm is the baseline you have to beat with your cleverness, not a competitor you can ignore*. When the obvious algorithm has a tight inner loop and a workload-favourable average case, beating it is hard. Sometimes you don't, and the obvious algorithm ships. Sometimes you do, and the gap you opened is exactly the value your cleverness produced. Either way the brute force is the reference frame, not the wrong answer.

23.8 Notes and further reading

The brute-force string-matching algorithm is so old it has no clear inventor — it predates the formal study of algorithms by some margin, and every introduction to the topic begins with it. CLRS covers it in section 32.1 ("The naive string-matching algorithm") with the same $O((n - m + 1)m)$ worst-case bound I derived above. Sedgewick and Wayne's *Algorithms* (4th ed.) section 5.3 ("Substring Search") opens with the brute force before moving to KMP and Boyer-Moore, and is worth reading for its diagrams of the shift-and-compare process.

The reason I leaned on `grep` and `str.find` in this chapter is that the brute force is genuinely what production code runs — CPython's `str.find` uses Crochemore and Perrin's *two-way* algorithm, which is the brute force with a small skip table; GNU `memmem` in glibc 2.9+ uses a similar two-way variant; the Rust standard library's `str::find` uses two-way on short patterns and a hash-based variant on long ones. The relevant academic citation is Crochemore and Perrin's 1991 *Journal of the ACM* paper "Two-way string-matching", which is a refinement of the brute force, not an abandonment of it. Wojciech Mucha's 2013 paper "Speeding up string matching by weak factor recognition" is a more recent variant in the same lineage. The brute force isn't a place algorithm design moved past; it's the substrate every production matcher sits on.

In the next chapter I switch the lens — I'll show you what happens if you read the pattern not as a string but as a tiny *automaton*, with one state per prefix and a transition on each character, and you'll see why that reframing is exactly what KMP needs to skip the work the brute force redoes.

24 Compiling the pattern into a machine

At the end of chapter 23 I promised you a different lens. The brute force matcher slid a window across the text and, at every shift, compared character by character — and at every mismatch it threw away everything it had just learned. If the first three characters of the pattern matched and the fourth disagreed, the next shift went right back to comparing character zero of the pattern against the next character of the text, as if those three matching characters had never happened. That is the wastefulness I want to fix, and the fix begins with a reframing that costs nothing but completely changes what the algorithm looks like.

Here is the move. Stop thinking of the pattern as a *string* and start thinking of it as a *machine*. Specifically, a deterministic finite automaton — a DFA — with one state for every prefix of the pattern and a transition on every character. State 0 means "I haven't matched anything yet." State 1 means "the last character I read was the pattern's first character." State 2 means "the last two characters I read are the pattern's first two characters." All the way up to state m, which means "I just finished matching the whole pattern."

One state per length of matched prefix — the pattern *is* a DFA. Every state's number is the length of the pattern prefix I've matched against the recent suffix of the text I've read. The transitions encode the only piece of preprocessing the brute force was missing — when a character disagrees, *how much of what I already matched is still salvageable?* The DFA answers that question in advance, once, and from then on the text scan becomes a single loop: read a character, look up the next state, repeat. KMP is the same DFA, stored more cleverly.

24.1 What if the pattern were a machine?

Take the brute force matcher and rewrite the problem in its own language. The matcher walks the text and, at every position, tries to tell me how much of the pattern it has currently matched. After reading the first character of the text, the answer is either "one character of the pattern matches" or "zero." After two characters, the answer is "two," "one," or "zero." After three, it's "three," "two," "one," or "zero." That number — the length of the longest pattern prefix that is also a suffix of the text I've read so far — is the *only* piece of state I need to decide whether the next character extends the match or breaks it.

If I call that number a *state*, then the matcher is a state machine. There are $m + 1$ states, numbered 0 through m. State m is special — it means I just finished reading the entire pattern, and so I should report a match. Every character I read is a *transition*: from state q on character c, I move to some state q'. The next-state function $\delta(q, c)$ is exactly the answer to "if I had matched q characters of the pattern, and the next character of the text is c, how many characters of the pattern do I now match?" If c extends the current prefix, q' is $q + 1$. If c breaks the current prefix, q' is *some smaller number* — possibly zero, possibly more — depending on what suffixes of the failed match happen to be prefixes of the pattern.

That last sentence is where the cleverness lives. The brute force didn't compute δ; it discovered δ on the fly, by going back to the text and rescanning. The DFA precomputes δ once, in advance, and from then on you read each text character exactly once. You never go back. That is the prize — $O(n)$ scanning time, no rescans, no backtracking — and the price is the preprocessing that builds the table.

24.2 States as prefix lengths

Let me nail down what state q means. **State q means: the last q characters of the text I've read so far are exactly $P[0..q-1]$**, and no longer prefix of P is also a suffix of the text-so-far. Three things are packed into that sentence.

First, state 0 is the starting state — I haven't read anything that matches a prefix of P. Second, state m is the accept state — I just completed a full match, and the position of the match is "where I am in the text minus m." Third, the *longer-prefix* qualifier is what makes this work: if I've read three characters and both the prefix of

length 3 and the prefix of length 1 happen to match the suffix of my text-so-far, the state is 3 (the longest one). I'm always tracking the *longest* matched prefix, because that's the one with the most potential to extend.

You see why the brute force was wasteful. After matching three characters and then mismatching the fourth, the brute force restarted from state 0 at the next text position. But it might have been the case that the three characters it had just matched contained, as their suffix, a prefix of the pattern — meaning state should have dropped to, say, state 1, not all the way to 0. The DFA knows that. The brute force has to rediscover it from scratch.

24.3 The transition function

The core of the construction is computing $\delta(q, c)$ for every state q and every relevant character c. The recipe is: imagine I'm in state q (meaning my recent text suffix matches $P[0..q-1]$) and I read character c. The text suffix now matches $P[0..q-1] + c$ — a string of length $q+1$. The new state is the length of the longest pattern prefix that is *also a suffix* of this string. I try lengths from largest to smallest and return the first one that fits.

Here is that idea as a small private helper.

```
def _delta(pattern: str, q: int, c: str) -> int:
    candidate = pattern[:q] + c
    m = len(pattern)
    for length in range(min(q + 1, m), 0, -1):
        if candidate.endswith(pattern[:length]):
            return length
    return 0
```

I'm constructing the hypothetical "what I've matched so far" string as `pattern[:q] + c` — that's the recent text suffix in the abstract, since by being in state q I know my recent text suffix matched the first q characters of the pattern, and then I read c. Then I try each candidate prefix length from $\min(q+1, m)$ downward and check whether the candidate ends with that prefix. The first one that fits is the answer. If nothing fits, the answer is 0 — no prefix of the pattern is a suffix of the failed match.

The $\min(q+1, m)$ cap is there because the candidate is $q+1$ characters long, but the longest prefix of the pattern is m characters, and a prefix longer than m doesn't exist. Once q reaches m — the accept state — the candidate is one character longer than the pattern itself, so the longest prefix that could fit is still capped at m. That detail matters for `find_all`, and I'll come back to it.

Let me make the function tangible with a hand trace on `"ain"`. From state 0 on character `'a'`: the candidate is `"a"`, and the longest prefix of `"ain"` that ends `"a"` is `"a"` itself, length 1. So $\delta(0, \text{'a'}) = 1$. From state 1 on character `'i'`: the candidate is `"ai"`, the longest prefix of `"ain"` ending in `"ai"` is `"ai"` itself, length 2. From state 2 on character `'n'`: the candidate is `"ain"`, the longest prefix ending in `"ain"` is `"ain"` — that's length 3, the accept state. From state 2 on character `'a'`: the candidate is `"aia"`, which ends in `"a"` but not in `"ai"` or `"ain"`, so the answer is 1. That last line is the interesting one — a partial match of length 2 (`"ai"`), broken by an `'a'`, doesn't drop the state to 0, because the `'a'` itself is a prefix of `"ain"`. The DFA captured that. The brute force would have rescanned.

Here is the full transition table for `"ain"`, computed by running `_delta` over every state-character pair and printing the result.

```
from codex.strings.automaton import _delta

pattern = "ain"
alphabet = "ain"  # the only characters that can ever extend a prefix

print(f"transition table for pattern {pattern!r}:")
print(f"  state | " + " | ".join(f" {c} " for c in alphabet))
print(f"  ------+" + "+".join("-----" for _ in alphabet))
for q in range(len(pattern) + 1):
    row = " | ".join(f" {_delta(pattern, q, c)} " for c in alphabet)
    marker = " (accept)" if q == len(pattern) else ""
    print(f"    {q}   | {row}{marker}")
```

```
transition table for pattern 'ain':
  state |  a  |  i  |  n
  ------+-----+-----+-----
    0   |  1  |  0  |  0
    1   |  1  |  2  |  0
    2   |  1  |  0  |  3
    3   |  1  |  0  |  0  (accept)
```

Read state 3 — the accept state. On reading another `'a'` from accept, the DFA drops to state 1 (because `"aina"` ends in `"a"`, a prefix of length 1). On `'i'`, it drops to 0 — `"aini"` ends in nothing that's a prefix of `"ain"`. On `'n'`, also 0. That row is exactly what I'll use in `find_all` to keep finding matches after the first one: from accept, the next character either restarts a fresh match attempt or drops to 0.

Now the more interesting case — pattern `"ababa"`. Its prefixes have non-trivial overlap structure: the prefix `"aba"` of length 3 is also a suffix of the full pattern `"ababa"`. That means when the DFA is sitting in the accept state and reads a character, it can drop to state 3, not 0 — three characters of the previous match are reusable.

```
from codex.strings.automaton import _delta

pattern = "ababa"
alphabet = "ab"

print(f"transition table for pattern {pattern!r}:")
print(f"  state | " + " | ".join(f" {c} " for c in alphabet))
print(f"  ------+" + "+".join("-----" for _ in alphabet))
for q in range(len(pattern) + 1):
    row = " | ".join(f" {_delta(pattern, q, c)} " for c in alphabet)
    marker = " (accept)" if q == len(pattern) else ""
    print(f"    {q}   | {row}{marker}")
```

```
transition table for pattern 'ababa':
  state |  a  |  b
  ------+-----+-----
    0   |  1  |  0
    1   |  1  |  2
    2   |  3  |  0
    3   |  1  |  4
    4   |  5  |  0
    5   |  1  |  4  (accept)
```

Look at state 5 — the accept state. On reading `'b'`, the DFA goes to state 4: the suffix `"ababab"` of the failed match ends in `"abab"`, which is a length-4 prefix of `"ababa"`. On reading `'a'`, the DFA goes to state 1: `"ababaa"` ends in `"a"` but not in `"aba"` or `"ababa"`. And from state 4 on `'a'`, the DFA jumps to state 5 — another match. That overlap is what makes patterns like `"ababa"` worth their own example. If you ran this DFA on `"abababababa"`, you'd find matches at positions 0, 2, and 4 — three overlapping occurrences, recovered because state 5's exit transition to state 4 keeps four characters of context alive instead of throwing them away.

24.4 Building the table

Now the public function that wraps `_delta` and produces the full transition table — a list of $m + 1$ dictionaries, one per state, mapping each alphabet character to its next state.

```
def build_dfa(
    pattern: str, alphabet: str | None = None
) -> tuple[list[dict[str, int]], int]:
    m = len(pattern)
    if alphabet is None:
        alphabet = "".join(sorted(set(pattern)))
```

```
    transitions: list[dict[str, int]] = []
    for q in range(m + 1):
        row: dict[str, int] = {}
        for c in alphabet:
            row[c] = _delta(pattern, q, c)
        transitions.append(row)
    return transitions, m
```

I default the alphabet to "the characters that actually appear in the pattern." That's a deliberate choice — any character not in the pattern can't extend any prefix, so its next-state from any state would be 0. Storing those transitions explicitly would just waste space. The simulator handles unknown characters by treating them as "implicit zero": if the next character isn't in the dictionary, the state drops to 0. That keeps the table small (relevant alphabet only) without losing any correctness.

The accept state is just m, the length of the pattern. I return it explicitly so the simulator doesn't have to recompute `len(pattern)` separately.

You'll notice this build is $O(m^3 \cdot |\Sigma|)$ — for each of the $m+1$ states and each of the $|\Sigma|$ characters, `_delta` does an $O(m^2)$ scan through candidate lengths (the inner `endswith` is itself $O(m)$). I'm not optimizing that. The chapter's point is that the *idea* of the DFA is the contribution; the construction can be made $O(m \cdot |\Sigma|)$ using the same failure-pointer trick KMP uses, which is exactly what the next chapter unfolds. For now, building the table for a pattern of ten characters over a four-character alphabet does a few thousand character comparisons in preprocessing, which is invisible compared to the scan.

24.5 Running the machine on the canonical text

The simulator is a single loop. Read each character of the text, look up the next state in the table, and if you ever land in the accept state, record a match.

```
def find(text: str, pattern: str) -> int | None:
    if not pattern:
        return 0
    if len(pattern) > len(text):
        return None
    transitions, accept = build_dfa(pattern)
    state = 0
    for idx, c in enumerate(text):
        state = transitions[state].get(c, 0)
        if state == accept:
            return idx - accept + 1
    return None
```

The `transitions[state].get(c, 0)` is where I handle characters outside the inferred alphabet — they implicitly go to state 0, because no prefix of the pattern can contain a character the pattern doesn't have. When I hit the accept state, the match started at position `idx - accept + 1` — the current index minus the pattern length plus one, because `idx` is the *last* character of the matched window.

Now `find_all`. The interesting decision is what to do *after* a match. The naive choice is to reset to state 0 and continue scanning, but that would miss overlapping matches. The right choice is to use the transitions from the accept state itself — state m has its own row in the transition table, computed by `_delta` exactly like every other state. So after recording a match, I just keep going: read the next text character, look up the transition from state m, and continue. The DFA handles overlaps for free.

```
def find_all(text: str, pattern: str) -> list[int]:
    if not pattern or len(pattern) > len(text):
        return []
    transitions, accept = build_dfa(pattern)
    matches: list[int] = []
    state = 0
```

```
    for idx, c in enumerate(text):
        state = transitions[state].get(c, 0)
        if state == accept:
            matches.append(idx - accept + 1)
    return matches
```

That's the whole simulator — eight lines of body. State m's row in the transition table does double duty: it both signals "I just matched" (via the equality check) *and* tells me where to go next. That's the abstraction working. State m is a state like any other, with outgoing transitions like any other; the only thing special about it is that I'm asked to record a match when I land on it.

Now let me run it on the canonical text from chapter 23 with pattern `"ain"`.

```
from codex.strings.automaton import find, find_all

TEXT = "the rain in spain stays mainly in the plain"

idx = find(TEXT, "ain")
print(f"first match of 'ain' at index {idx} - same answer as the brute force")

all_ain = find_all(TEXT, "ain")
print(f"all matches of 'ain': {all_ain}  (four occurrences, identical to ch 23)")

miss = find(TEXT, "frog")
print(f"searching for 'frog': {miss}  (no 'f' anywhere, so the DFA never leaves state 0)")
```

```
first match of 'ain' at index 5 - same answer as the brute force
all matches of 'ain': [5, 14, 25, 40]  (four occurrences, identical to ch 23)
searching for 'frog': None  (no 'f' anywhere, so the DFA never leaves state 0)
```

Four matches at positions 5, 14, 25, 40 — exactly what the brute force produced, and exactly what `re.findall(r"ain", TEXT)` would produce. The DFA recovers them all in a single left-to-right pass, never backtracking. Searching for `"frog"` returns `None` because the very first character `'f'` is outside the pattern's alphabet, so every text character drops the state straight back to 0 — the simulator runs in $O(n)$ doing essentially no work per step.

Now the overlap case, with a contrived input that exercises state 5's non-trivial exit transitions in `"ababa"`.

```
from codex.strings.automaton import find_all

matches = find_all("abababababa", "ababa")
print(f"find_all('abababababa', 'ababa') = {matches}  (three overlapping matches: 0, 2, 4)")
```

```
find_all('abababababa', 'ababa') = [0, 2, 4]  (three overlapping matches: 0, 2, 4)
```

Three matches at positions 0, 2, 4. The string `"abababababa"` contains `"ababa"` starting at index 0, then again at index 2 (sharing the last three characters of the first match with the first three of the second), then again at index 4 (sharing again with the second). The DFA finds all three because, on entering state 5 the first time at position 4 of the text, it transitions on the next `'b'` to state 4 — keeping four characters of context alive — and then on the next `'a'` to state 5 again. Then another `'b'`, then another `'a'`, and the simulator lands on state 5 a third time. The brute force from chapter 23 would have done $O(nm)$ work for this input; the DFA does $O(n)$.

24.6 What the DFA costs to build

You see the trade-off now. Scanning is one character read per text position, full stop — that's the prize. But the price is the table. For a pattern of length m over an alphabet of size $|\Sigma|$, the table has $(m+1) \cdot |\Sigma|$ entries. Each entry is an integer. The total space is $\Theta(m \cdot |\Sigma|)$. Building the table with my naive `_delta` takes $O(m^3 \cdot |\Sigma|)$ time, because for each state and each character I'm running an $O(m^2)$ scan through candidate prefix lengths.

Bilingually: for a 10-character pattern over the 26-letter alphabet, the table has about 286 integer slots — a few kilobytes, totally negligible — and the naive construction does on the order of $26 \cdot 1000 = 26{,}000$ character comparisons, also negligible. For a 100-character pattern, the table is about 2,600 slots and construction does about 26 million comparisons. Still cheap, but starting to be visible. For a 1000-character pattern over a large alphabet, the cubic-in-m construction would start to bite.

KMP fixes this by storing one number per state — the *failure pointer* — instead of one number per state-character pair. The failure pointer tells you which state to fall back to when the next character doesn't extend the current prefix; the transition is then computed on the fly by chaining failure pointers until a character matches. That stores $O(m)$ data instead of $O(m \cdot |\Sigma|)$, and it builds in $O(m)$ time instead of $O(m^3 \cdot |\Sigma|)$. The scan is still $O(n)$, but with a small per-step overhead — each text character may trigger a few failure-pointer chases instead of a single table lookup. That's the next chapter's whole subject.

24.7 The three questions, applied

Is it correct?

The invariant I maintain is: **after reading the first k characters of the text, the simulator's state equals the length of the longest prefix of P that is also a suffix of $T[0..k-1]$.** State 0 satisfies the invariant trivially at $k = 0$ (the empty string's longest pattern-prefix-suffix is the empty prefix). The induction step is the transition: from state q (meaning my text suffix matches $P[0..q-1]$) on the next character c, the new text suffix is $P[0..q-1] + c$ in the abstract, and $_d elta$ returns exactly the length of the longest prefix of P that is a suffix of that string — which is the new state, by definition of the invariant. So the invariant is preserved.

When the simulator hits state m, the invariant says my recent text suffix of length m equals $P[0..m-1] = P$ — that is, the pattern just finished matching, ending at the current position. Reporting `idx - m + 1` as the match start is then correct by arithmetic. `find_all` is correct because it appends every such position; the only subtlety is that it continues from state m rather than resetting to 0, and the invariant says state m's outgoing transitions correctly track the longest pattern-prefix-suffix of the text including the newly-read character. So overlapping matches are found by the same induction.

How efficient is it?

The scan itself runs in $\Theta(n)$ time and $O(1)$ extra space — one dictionary lookup per character, no allocation per step. Preprocessing builds the table in $O(m^3 \cdot |\Sigma|)$ time using $\Theta(m \cdot |\Sigma|)$ space. Total time is $O(n + m^3 \cdot |\Sigma|)$; total space is $O(m \cdot |\Sigma| + k)$ where k is the number of matches reported.

Bilingually: searching a million-character text for a 10-character pattern over a 4-character alphabet costs about a million character lookups for the scan and about four thousand comparisons for the preprocessing — the scan dominates, and the algorithm runs in essentially linear time. For a 100-character pattern, the table-building cost rises to about four million comparisons, which is comparable to the scan but still small enough that you wouldn't notice. The asymptotic story is: scanning is linear in n, independent of m; preprocessing is polynomial in m, independent of n. That's the entire shape.

Is it optimal?

Scanning at $\Theta(n)$ is optimal in the strongest sense — you have to read every character of the text at least once to be sure the pattern doesn't appear, so $\Omega(n)$ is the lower bound, and the DFA matches it. That's the brute force's worst-case-tight upgrade I promised: the brute force runs at $\Theta(n)$ in the best case and $\Theta(nm)$ in the worst case; the DFA runs at $\Theta(n)$ in both.

Preprocessing at $\Theta(m \cdot |\Sigma|)$ space is *not* optimal. KMP achieves the same $\Theta(n)$ scan with $\Theta(m)$ preprocessing — a factor of $|\Sigma|$ smaller. Aho-Corasick generalizes the failure-function idea to multiple patterns simultaneously, keeping the same scan-time guarantee. The DFA's full transition table is conceptually clean but operationally wasteful; the next two chapters are the engineering work that recovers the same scan-time guarantee at lower preprocessing cost.

24.8 What the automaton view teaches you

The DFA isn't just an implementation of pattern matching — it's a *lens* on the entire problem. Once you see it, every other algorithm in this part is the same DFA dressed differently.

KMP is the DFA with a sparse representation: one failure pointer per state instead of one transition per character. The state space is identical; the storage is smaller. Aho-Corasick is the DFA generalized from one pattern to a set of patterns — the states are now prefixes of any pattern in the set, the transitions are computed the same way, and a single text scan finds all matches of all patterns simultaneously. Boyer-Moore is the only major pattern matcher that *isn't* obviously a DFA — it reads the text right-to-left and uses different preprocessing — but even there, the "good suffix" rule is encoded by an analog of the same failure function.

That's why I wanted you to see the DFA before KMP. The DFA tells you what's being computed; KMP tells you how to compute it more cheaply. Without the DFA view, KMP's failure function looks like a clever trick. With the DFA view, the failure function is the obvious compression of an obvious object. The algorithm doesn't get easier — but the *necessity* of the algorithm becomes legible. You stop asking "why does this work?" and start asking "how could it possibly not?"

The other lesson worth carrying forward: **a representation change can convert a runtime cost into a one-time preprocessing cost.** The brute force paid its $O(m)$ at every shift. The DFA pays $O(m^3 \cdot |\Sigma|)$ once. For a single search in a long text, that's a massive win. For many searches with the same pattern across different texts — say, a virus scanner with a fixed signature scanning a stream of files — the win compounds: the table is built once and reused for every scan. The brute force can't reuse anything between calls.

24.9 Notes and further reading

CLRS section 32.3, "String matching with finite automata," gives the formal treatment of the DFA construction, including a tighter $O(m \cdot |\Sigma|)$ build algorithm I deliberately avoided here — the $O(m^3 \cdot |\Sigma|)$ version is pedagogically clearer. Sedgewick and Wayne's *Algorithms* (4th edition) section 5.3 covers KMP directly without the explicit DFA detour, which is the more common textbook path; I prefer routing through the DFA first because it makes KMP's failure function feel inevitable rather than ingenious.

For the formal theory of finite automata — accepting languages, the relationship between DFAs and regular expressions, the Myhill-Nerode theorem that proves the DFA I built for a pattern P is the unique minimum DFA recognizing strings ending in P — see Hopcroft, Motwani, and Ullman's *Introduction to Automata Theory, Languages, and Computation* (3rd edition), chapters 2–4. That book is heavier mathematical machinery than the Codex normally uses, but the chapter on DFA minimization is worth knowing about: it tells you that the DFA construction here is, in a formal sense, the *best possible* DFA for the pattern-matching problem on a single pattern.

The $O(m^3 \cdot |\Sigma|)$ build time can be sharpened to $O(m \cdot |\Sigma|)$ using exactly the same failure-pointer trick KMP uses internally — but doing so is the whole insight of the next chapter, and I want you to meet it there.

In the next chapter, I'll show you how to compress this DFA into a single array of m integers — the failure function — and how the resulting algorithm, Knuth-Morris-Pratt, scans the text in $\Theta(n)$ with $\Theta(m)$ preprocessing instead of $\Theta(m \cdot |\Sigma|)$.

25 Failing forward, never backward

At the end of chapter 24 I left you with a confession. The DFA I built scans the text in $\Theta(n)$ and never backtracks — that part is perfect — but the price was a transition table of $\Theta(m \cdot |\Sigma|)$ entries, built in $O(m^3 \cdot |\Sigma|)$ time by a triple-nested loop I made no attempt to defend. For a pattern of ten characters over the alphabet of English text, the table fits in a few kilobytes; for a thousand-character pattern over a Unicode-flavored alphabet of 10,000 code points, the table is ten million slots and the construction is several orders of magnitude past defensible. Most of those slots, if you stared at the table from chapter 24 long enough, are answering the same question: "I just mismatched — how much of my previous progress is still salvageable?" The Knuth-Morris-Pratt algorithm — KMP — is what you get when you store the answer to that question *once per state* instead of once per state-character pair.

The failure function tells you where to resume; the text pointer never moves backward. That sentence is the entire algorithm in compressed form. Failure function, m integers. Text pointer, one variable that strictly increases. KMP isn't a different algorithm from chapter 24's DFA — it's the same DFA stored in $\Theta(m)$ space instead of $\Theta(m \cdot |\Sigma|)$, and the chapter's closing section reconstructs the DFA cell-by-cell from the failure function to prove the compression is lossless.

25.1 The DFA's wasted dimension

Look back at the transition table I built in chapter 24 for the pattern `"ababa"`. Six states, two columns (one per alphabet character), twelve entries total. Now ask a question I deliberately didn't ask in chapter 24: for any state q and any character c, what are the possible values of $\delta(q, c)$? There are exactly two interesting cases.

Either c equals $P[q]$ — the next character of the pattern — in which case $\delta(q, c) = q + 1$, the obvious extension. Or c doesn't equal $P[q]$, in which case $\delta(q, c)$ is "the next state I'd reach from *some smaller state* on character c." The interesting work is all in the second case. In the first case, the table is storing the constant $q + 1$ for one specific character per state and ignoring the rest. In the second case, the table is storing the same answer many times — every state's mismatch behaviour on a particular character collapses down to "fall back to the longest prefix-of-pattern that's also a suffix of what I just had, then try again."

That collapse is the redundancy. If I knew, for each state q, the single number "what's the longest proper prefix of $P[0..q-1]$ that's also a suffix of $P[0..q-1]$?", I could reconstruct the entire mismatch column on the fly. I'd never need to store $\delta(q, c)$ explicitly for $c \neq P[q]$ — I'd compute it by chaining fallbacks until either the character matches or I hit state zero.

That single number, one per state, is the failure function.

25.2 The failure function

Let me define it precisely. For a pattern P of length m, the **failure function** is an array π of length m, where

$$\pi[i] = \text{the length of the longest } \textit{proper} \text{ prefix of } P[0..i] \text{ that is also a suffix of } P[0..i].$$

The word *proper* is doing work — the prefix has to be strictly shorter than $P[0..i]$ itself. Otherwise $\pi[i]$ would always be $i + 1$, which is useless. By convention $\pi[0] = 0$: the only proper prefix of a single-character string is the empty string, whose length is zero.

That definition takes a few rereadings on the first encounter. Let me make it tangible on `"ababa"`. I'll walk through each i from 0 to 4 and name the longest proper prefix-that-is-also-a-suffix.

At $i = 0$, the substring is `"a"`. Its only proper prefix is `""`, length 0. So $\pi[0] = 0$.

At $i = 1$, the substring is `"ab"`. Proper prefixes are `""` and `"a"`. Neither is a suffix of `"ab"` other than the empty one, because the only suffix of length 1 is `"b"`, and the empty suffix is, by definition, a suffix of every string. So the longest proper prefix-that-is-also-a-suffix is `""`, length 0. $\pi[1] = 0$.

At $i = 2$, the substring is `"aba"`. Proper prefixes: `""`, `"a"`, `"ab"`. The suffixes of `"aba"` are `""`, `"a"`, `"ba"`, `"aba"`. The intersection is { `""`, `"a"` } — the empty string and `"a"`. The longest is `"a"`, length 1. So $\pi[2] = 1$.

At $i = 3$, the substring is `"abab"`. Proper prefixes: `""`, `"a"`, `"ab"`, `"aba"`. Suffixes: `""`, `"b"`, `"ab"`, `"bab"`, `"abab"`. The intersection is { `""`, `"ab"` }. The longest is `"ab"`, length 2. So $\pi[3] = 2$.

At $i = 4$, the substring is `"ababa"`. Proper prefixes: everything up to `"abab"`. Suffixes: `""`, `"a"`, `"ba"`, `"aba"`, `"baba"`, `"ababa"`. The intersection is { `""`, `"a"`, `"aba"` }. The longest is `"aba"`, length 3. So $\pi[4] = 3$.

Failure function for `"ababa"` is therefore $[0, 0, 1, 2, 3]$. Read it as a hint sheet: if I matched four characters of `"ababa"` and the fifth disagrees, $\pi[3] = 2$ tells me my recent two characters were `"ab"`, which is a prefix of the pattern, so I should resume comparing from $P[2]$ — saving two characters of re-scanning. That hint is what the brute force in chapter 23 was throwing away at every shift.

25.3 Computing the failure function

The algorithm that computes π is, beautifully, KMP running on the pattern itself, against itself shifted by one. I'm walking i from 1 to $m - 1$, maintaining a variable k that's the current "candidate length of the matching prefix" — exactly the same kind of state the matching loop will maintain when it walks the text.

Here's the function.

```
def failure_function(pattern: str) -> list[int]:
    m = len(pattern)
    pi = [0] * m
    k = 0
    for i in range(1, m):
        while k > 0 and pattern[k] != pattern[i]:
            k = pi[k - 1]
        if pattern[k] == pattern[i]:
            k += 1
        pi[i] = k
    return pi
```

Three things happen per iteration. First, the `while` loop falls back: as long as my current candidate length k is positive and the character at position k of the pattern doesn't match the character at position i, I shrink k to $\pi[k - 1]$ — the next-best candidate. Second, after the loop, if the characters now match, I extend k by one. Third, I record k as $\pi[i]$ and move on.

Let me trace on `"ababa"` so you can watch k evolve. Start with $k = 0$ and $\pi = [0, 0, 0, 0, 0]$.

At $i = 1$: $k = 0$, the `while` doesn't enter (the guard $k > 0$ fails immediately). Is `pattern[0] == pattern[1]`? `'a' == 'b'`? No. So k stays at 0. $\pi[1] = 0$.

At $i = 2$: $k = 0$ still. The `while` skips. Is `pattern[0] == pattern[2]`? `'a' == 'a'`? Yes. So k becomes 1. $\pi[2] = 1$.

At $i = 3$: $k = 1$. Is `pattern[1] == pattern[3]`? `'b' == 'b'`? Yes. So the `while` doesn't enter (the second clause is false). k becomes 2. $\pi[3] = 2$.

At $i = 4$: $k = 2$. Is `pattern[2] == pattern[4]`? `'a' == 'a'`? Yes. The `while` doesn't enter. k becomes 3. $\pi[4] = 3$.

Final $\pi = [0, 0, 1, 2, 3]$. Matches the hand derivation above, character-for-character.

```python
from codex.strings.kmp import failure_function

print(f"failure_function('ababa') = {failure_function('ababa')}")
print(f"failure_function('abracadabra') = {failure_function('abracadabra')}")
print(f"failure_function('aaaa') = {failure_function('aaaa')}  (every prefix is a suffix)")
print(f"failure_function('abcd') = {failure_function('abcd')}  (no overlaps anywhere)")
```

```
failure_function('ababa') = [0, 0, 1, 2, 3]
failure_function('abracadabra') = [0, 0, 0, 1, 0, 1, 0, 1, 2, 3, 4]
failure_function('aaaa') = [0, 1, 2, 3]  (every prefix is a suffix)
failure_function('abcd') = [0, 0, 0, 0]  (no overlaps anywhere)
```

You can see the structure in all four. `"aaaa"` has $\pi = [0, 1, 2, 3]$ because every prefix is also a suffix — the pattern is its own degenerate cycle. `"abcd"` has $\pi = [0, 0, 0, 0]$ because no proper prefix of any of its substrings matches any suffix — there's no internal repetition to fall back on. `"abracadabra"` is the one I want to look at more carefully in a moment, because it's the canonical demo.

Bilingually: the loop reads i from 1 to $m - 1$ exactly once, and k is bounded above by i, so the amortized cost is $O(m)$. Each iteration either increments k (at most m times total across the whole loop) or shrinks k via the `while` (each shrink reduces k by at least one, and k can only shrink as many times as it has been incremented). That amortized accounting gives the entire failure-function build in linear time — a factor of $m^2 \cdot |\Sigma|$ improvement over chapter 24's `_delta`-by-brute-force.

25.4 The matching loop

Now the matching algorithm itself. It looks structurally identical to the failure-function build — same `while`/`if`/extend skeleton — but operates on the text instead of the pattern.

```python
def find(text: str, pattern: str) -> int | None:
    n, m = len(text), len(pattern)
    if m == 0:
        return 0
    if m > n:
        return None
    pi = failure_function(pattern)
    j = 0
    for i in range(n):
        while j > 0 and pattern[j] != text[i]:
            j = pi[j - 1]
        if pattern[j] == text[i]:
            j += 1
        if j == m:
            return i - m + 1
    return None
```

Two invariants do all the work. The first: **the text pointer `i` runs once over the text, left to right, and never decreases.** That's the literal text of the `for` loop — there's no inner mechanism that ever resets or rewinds i. Every character of the text is read exactly once, and the entire scan is therefore $\Theta(n)$, period. The brute force from chapter 23 had nothing like this — its inner loop re-read text characters at every shift.

The second invariant: **on a mismatch, `j` falls back via π to the longest pattern prefix that's still salvageable as a suffix of the text I've just read.** That's the `while` loop's job. The line `j = pi[j - 1]` says "I just matched j characters, the next one disagrees, so my recent text suffix of length j equals $P[0..j-1]$, and the longest proper prefix of that string that's also a suffix is $\pi[j-1]$ characters long. Resume the comparison from there." After enough fallbacks, either the next character of the pattern at the new j matches the current text character (and the algorithm extends), or j has fallen all the way to zero (and the algorithm gives up on the current text character and moves on).

When j reaches m, the entire pattern has matched, ending at text position i. The match started at $i - m + 1$. For `find`, I return that index immediately.

`find_all` is the same loop, with two differences: I append the match index instead of returning, and on $j == m$ I set $j = \pi[m-1]$ instead of zero. That's the same trick chapter 24's `find_all` used — the DFA's accept state has its own outgoing transitions, and the failure function encodes them. Setting j to $\pi[m-1]$ keeps the longest possible amount of context alive for overlapping matches.

```
def find_all(text: str, pattern: str) -> list[int]:
    n, m = len(text), len(pattern)
    if m == 0 or m > n:
        return []
    pi = failure_function(pattern)
    matches: list[int] = []
    j = 0
    for i in range(n):
        while j > 0 and pattern[j] != text[i]:
            j = pi[j - 1]
        if pattern[j] == text[i]:
            j += 1
        if j == m:
            matches.append(i - m + 1)
            j = pi[m - 1]
    return matches
```

Eight lines of loop body. No backtracking on the text. One bounded fallback chain per text character. The total work across all text positions is $\Theta(n)$ by the same amortized argument that bounded the failure-function build: j increments at most n times total, and the `while` decrements it, so the total number of decrement steps across the whole scan is at most n too. Linear in the text length, independent of the pattern's internal structure.

25.5 Running on the canonical text

Time to verify everything works on the same input I've been using since chapter 23. Pattern `"ain"` on the canonical text.

```
from codex.strings.kmp import find, find_all

TEXT = "the rain in spain stays mainly in the plain"

idx = find(TEXT, "ain")
print(f"first match of 'ain' at index {idx} - same answer as ch 23 and ch 24")

all_ain = find_all(TEXT, "ain")
print(f"all matches of 'ain': {all_ain}  (four occurrences, identical to ch 23 and ch 24)")

miss = find(TEXT, "frog")
print(f"searching for 'frog': {miss}  (no 'f' anywhere - j stays at 0 throughout)")
```

```
first match of 'ain' at index 5 - same answer as ch 23 and ch 24
all matches of 'ain': [5, 14, 25, 40]  (four occurrences, identical to ch 23 and ch 24)
searching for 'frog': None  (no 'f' anywhere - j stays at 0 throughout)
```

Four matches at 5, 14, 25, 40 — the same answer chapter 23's brute force and chapter 24's DFA produced. KMP did it in $\Theta(n)$ scan time with $\Theta(m)$ preprocessing instead of the DFA's $\Theta(m \cdot |\Sigma|)$ preprocessing. Same answer, cheaper preparation.

Now the overlap case that made the DFA's accept-state transitions interesting in chapter 24 — pattern `"ababa"` on `"abababababa"`.

```
from codex.strings.kmp import find_all

matches = find_all("abababababa", "ababa")
print(f"find_all('abababababa', 'ababa') = {matches}  (three overlapping matches: 0, 2, 4)")
```

```
find_all('abababababa', 'ababa') = [0, 2, 4]  (three overlapping matches: 0, 2, 4)
```

Three matches at 0, 2, 4 — exactly what chapter 24's DFA produced for the same input. The mechanism is different (a failure-function fallback to $\pi[4] = 3$ on match versus an explicit DFA transition out of state 5), but the externally observable behaviour is identical. That's the cross-chapter consistency I want — KMP and the DFA agree on every input, by design.

25.6 abracadabra: the canonical demo

The pattern `"abracadabra"` is the canonical KMP demo for a reason. It's long enough to have non-trivial internal structure (the prefix `"abra"` reappears as the suffix), short enough to trace by hand, and recognizable enough that the matching examples are memorable. Its failure function is $[0, 0, 0, 1, 0, 1, 0, 1, 2, 3, 4]$ — let me walk through what those numbers mean.

```
from codex.strings.kmp import failure_function

pattern = "abracadabra"
pi = failure_function(pattern)
print(f"pattern = {pattern!r}")
print(f"pi      = {pi}")
print()
print("by index:")
for i, val in enumerate(pi):
    prefix = pattern[:i + 1]
    overlap = pattern[:val] if val > 0 else '""'
    print(f"  pi[{i:>2}] = {val}   substring {prefix!r:>14}   longest proper prefix=suffix: {overlap}")
```

```
pattern = 'abracadabra'
pi      = [0, 0, 0, 1, 0, 1, 0, 1, 2, 3, 4]

by index:
  pi[ 0] = 0   substring            'a'   longest proper prefix=suffix: ""
  pi[ 1] = 0   substring           'ab'   longest proper prefix=suffix: ""
  pi[ 2] = 0   substring          'abr'   longest proper prefix=suffix: ""
  pi[ 3] = 1   substring         'abra'   longest proper prefix=suffix: a
  pi[ 4] = 0   substring        'abrac'   longest proper prefix=suffix: ""
  pi[ 5] = 1   substring       'abraca'   longest proper prefix=suffix: a
  pi[ 6] = 0   substring      'abracad'   longest proper prefix=suffix: ""
  pi[ 7] = 1   substring     'abracada'   longest proper prefix=suffix: a
  pi[ 8] = 2   substring    'abracadab'   longest proper prefix=suffix: ab
  pi[ 9] = 3   substring   'abracadabr'   longest proper prefix=suffix: abr
  pi[10] = 4   substring  'abracadabra'   longest proper prefix=suffix: abra
```

The interesting indices are 7 through 10. At $i = 7$, the substring is `"abracada"` — eight characters — and the longest proper prefix that's also a suffix is `"a"`, length 1. At $i = 8$, the substring is `"abracadab"` and the answer is `"ab"`, length 2. At $i = 9$, the substring is `"abracadabr"` and the answer is `"abr"`, length 3. At $i = 10$, the full pattern `"abracadabra"`, and the answer is `"abra"`, length 4 — the four-character prefix that also closes out the pattern.

That last value, $\pi[10] = 4$, is what `find_all` uses after a successful match. After reading the full pattern, the algorithm keeps four characters of context alive — the trailing `"abra"` — instead of restarting from scratch. If the text continues with `"cadabra..."`, those four characters are immediately reusable as the prefix of a second match.

Let me trace one mismatch fallback by hand, so you can see the failure function in action. Suppose I'm scanning the text `"abracadabrx"` (note the `'x'` at the end where `'a'` should be) with pattern `"abracadabra"`.

At text positions 0 through 9, the algorithm extends j once per text character: j goes $1, 2, 3, 4, 5, 6, 7, 8, 9, 10$. Now at text position 10, the text character is `'x'` but `pattern[10]` is `'a'`. Mismatch. The `while` enters: $j = \pi[9] = 3$. Is `pattern[3] == text[10]`? `pattern[3]` is `'a'`. Still `'x'`. Mismatch again. Fall back: $j = \pi[2] = 0$. Loop exits (because j is no longer positive). Now check if `pattern[0] == text[10]`: `'a' == 'x'`? No. j stays at 0. Move on to text position 11.

What just happened: the algorithm matched ten characters of the pattern, mismatched on the eleventh, and the failure function let it skip directly to "what if my recent suffix of length three (`"abr"`) is the start of a new match?" without re-scanning any of those three characters in the text. Two fallbacks total, then a clean reset. That skipping is the KMP win, made visible.

```
from codex.strings.kmp import find, find_all

text = "abracadabra is a magic abracadabra"
matches = find_all(text, "abracadabra")
print(f"matches of 'abracadabra' in {text!r}: {matches}")
print(f"first match index: {find(text, 'abracadabra')}")
```

```
matches of 'abracadabra' in 'abracadabra is a magic abracadabra': [0, 23]
first match index: 0
```

Two matches at positions 0 and 23. KMP reads each of the 34 text characters exactly once and reports both. The failure-function fallbacks happen invisibly inside the inner `while`, never costing more than amortized constant time per text character.

25.7 KMP and the DFA are the same machine

Now the punchline. I claimed at the top of this chapter that KMP is the DFA from chapter 24 in a smaller storage format, and I want to make that claim concrete by reconstructing the entire DFA from the failure function alone, cell by cell, and verifying that the two tables agree everywhere.

The reconstruction rule is short. For a pattern P of length m, the DFA transition $\delta(q, c)$ — defined for states $0 \le q \le m$ and characters c — can be written recursively using only π and the pattern itself.

For $q = 0$: $\delta(0, c) = 1$ if $P[0] = c$, else 0. (Either the first character matches, or it doesn't.)

For $0 < q < m$: $\delta(q, c) = q + 1$ if $P[q] = c$, else $\delta(\pi[q-1], c)$. (Either the next pattern character matches and I extend by one, or I fall back to the failure-function state and recursively ask the same question there.)

For $q = m$ (the accept state, which has no $P[m]$): $\delta(m, c) = \delta(\pi[m-1], c)$. (After accepting, I fall back to the longest proper-prefix-suffix and continue from there.)

That recursion always terminates because $\pi[q-1] < q$ — each recursive step strictly decreases q — and the base case $q = 0$ is non-recursive.

Here's the function that implements that rule.

```
def dfa_transition_via_failure(pattern: str, pi: list[int], q: int, c: str) -> int:
    m = len(pattern)
    if q < m and pattern[q] == c:
        return q + 1
    if q == 0:
        return 0
    return dfa_transition_via_failure(pattern, pi, pi[q - 1], c)
```

I'm letting the recursion bottom out via `pi[q - 1]` chains. The recursion depth is bounded by the chain length of repeated failure-function applications starting from q, which is at most $O(\log m)$ on most patterns and at most $O(m)$ in the worst case (a pathological pattern like `"aaaa..."`). For the patterns I'm running it on here, the recursion is shallow.

Now let me build chapter 24's DFA on `"ababa"` and the KMP failure function on `"ababa"`, then reconstruct the full DFA from the failure function and compare cell-by-cell against the original.

```
from codex.strings.automaton import build_dfa
from codex.strings.kmp import failure_function

pattern = "ababa"
transitions, accept = build_dfa(pattern)
pi = failure_function(pattern)
alphabet = "ab"

print(f"pattern = {pattern!r}")
print(f"pi (failure function) = {pi}")
print()
print(f"reconstructing DFA from pi, comparing to build_dfa output:")
print(f"  state | char | build_dfa | via_pi | match")
print(f"  ------+------+-----------+--------+------")

all_match = True
for q in range(len(pattern) + 1):
    for c in alphabet:
        dfa_val = transitions[q].get(c, 0)
        pi_val = dfa_transition_via_failure(pattern, pi, q, c)
        ok = dfa_val == pi_val
        all_match = all_match and ok
        print(f"    {q}   |  {c}   |     {dfa_val}     |   {pi_val}    |  {ok}")

print()
print(f"all cells agree: {all_match}")
```

```
pattern = 'ababa'
pi (failure function) = [0, 0, 1, 2, 3]

reconstructing DFA from pi, comparing to build_dfa output:
  state | char | build_dfa | via_pi | match
  ------+------+-----------+--------+------
    0   |  a   |     1     |   1    |  True
    0   |  b   |     0     |   0    |  True
    1   |  a   |     1     |   1    |  True
    1   |  b   |     2     |   2    |  True
    2   |  a   |     3     |   3    |  True
    2   |  b   |     0     |   0    |  True
    3   |  a   |     1     |   1    |  True
    3   |  b   |     4     |   4    |  True
    4   |  a   |     5     |   5    |  True
    4   |  b   |     0     |   0    |  True
    5   |  a   |     1     |   1    |  True
    5   |  b   |     4     |   4    |  True

all cells agree: True
```

Every cell matches. The DFA built in chapter 24 by brute-force enumeration of suffix-prefix overlaps and the DFA reconstructed here from KMP's failure function are character-for-character the same table. The failure function loses no information about the DFA — it's a different encoding of the same object.

Run the same check on `"abracadabra"` for a more substantial example.

```
from codex.strings.automaton import build_dfa
from codex.strings.kmp import failure_function
```

```
pattern = "abracadabra"
transitions, accept = build_dfa(pattern)
pi = failure_function(pattern)
alphabet = "abcdr"  # the characters that actually appear in the pattern

mismatches = 0
for q in range(len(pattern) + 1):
    for c in alphabet:
        dfa_val = transitions[q].get(c, 0)
        pi_val = dfa_transition_via_failure(pattern, pi, q, c)
        if dfa_val != pi_val:
            mismatches += 1

cells = (len(pattern) + 1) * len(alphabet)
print(f"pattern = {pattern!r}")
print(f"DFA cells checked: {cells}")
print(f"mismatches between build_dfa and dfa_transition_via_failure: {mismatches}")
```

```
pattern = 'abracadabra'
DFA cells checked: 60
mismatches between build_dfa and dfa_transition_via_failure: 0
```

Sixty cells checked, zero mismatches. The compression is exact on every pattern I've tried, and the chapter's claim is now verified by code: KMP and the DFA are the same machine.

The storage difference is dramatic. The DFA for `"abracadabra"` over its 5-character alphabet has $12 \cdot 5 = 60$ integer slots. The failure function has 11 integer slots. That's a five-and-a-half-fold reduction, and it grows with $|\Sigma|$ — the larger the alphabet, the bigger the relative win for KMP. The trade-off, of course, is that each text character now costs a bounded number of failure-pointer chases instead of a single table lookup. In practice that overhead is small: the amortized fallback cost across the whole scan is $\Theta(n)$, same as the DFA's lookup cost.

25.8 The three questions, applied

Is it correct?

I need two invariants. The first is on the failure function itself: **after `failure_function` returns, $\pi[i]$ equals the length of the longest proper prefix of $P[0..i]$ that is also a suffix of $P[0..i]$, for every $i \in [0, m-1]$.** That follows by induction on i. At $i = 0$, $\pi[0] = 0$ is correct by convention and by the lack of any proper prefix. For $i > 0$, suppose $\pi[0..i-1]$ are all correct. The variable k entering iteration i equals $\pi[i-1]$ — the longest proper prefix-suffix of $P[0..i-1]$. If $P[k] == P[i]$, then $P[0..k]$ is a prefix-suffix of $P[0..i]$ of length $k+1$, and you can show it's the longest such by contradiction (any longer one would imply, after dropping the last character, a prefix-suffix of $P[0..i-1]$ longer than k, which contradicts the induction hypothesis). If $P[k] \neq P[i]$, the fallback chain $k = \pi[k-1]$ enumerates candidate shorter prefix-suffixes in decreasing length until one extends or none do, and the first extension is the answer.

The second invariant is on the matching loop: **at the top of each iteration of the `for i` loop, the variable j equals the length of the longest prefix of P that is also a suffix of $T[0..i-1]$.** This is the same invariant the DFA from chapter 24 maintained on its `state` variable — exactly the same. The `while` loop's role is to maintain it across a mismatch: starting from "I just matched j characters and the next one disagrees," the longest still-salvageable prefix is the longest proper prefix-suffix of $P[0..j-1]$ — which is $\pi[j-1]$ by definition. Chaining until either the character matches or j reaches zero finds the longest prefix that the new character can extend.

When j reaches m, the invariant says my recent text suffix of length m equals P, i.e., the pattern just finished matching at position i. Reporting $i - m + 1$ as the match start is then correct by arithmetic. `find_all` is correct because setting $j = \pi[m-1]$ after a match preserves the invariant for the next iteration — the longest proper prefix-suffix of P is exactly the longest prefix that's still alive as a candidate continuation.

How efficient is it?

The failure function builds in $\Theta(m)$ amortized time and $\Theta(m)$ space. The matching loop runs in $\Theta(n)$ amortized time and $O(m)$ extra space (for π). Total time for `find` or `find_all` is $\Theta(n + m)$; total space is $\Theta(m)$ for the failure function plus $O(k)$ for the output if you're collecting matches.

The amortized argument is the same on both halves. In `failure_function`, the variable k is incremented at most $m - 1$ times (once per iteration when the character matches), and the `while` decrements it — each decrement strictly reduces k, and k can only decrease as many times as it has been increased. So total `while` work across the whole function is $O(m)$, and the outer loop is $O(m)$. In `find`, the same accounting holds with j in place of k and n in place of m.

Bilingually: searching a million-character text for a hundred-character pattern costs about a million read operations for the scan and about a hundred read operations for the preprocessing. The DFA from chapter 24 would have done the same million-read scan but with about a million preprocessing operations on a 26-letter alphabet — a ten-thousand-fold preprocessing reduction for KMP, with the same scan time. For a single search in a large text, the preprocessing savings dominate; for repeated searches with the same pattern across many texts, the savings compound the way they do for the DFA, because the failure function is computed once and reused.

Is it optimal?

For exact pattern matching, $\Omega(n)$ is the lower bound on the text scan — every character has to be read to be sure the pattern doesn't appear at that position. KMP matches that bound and the DFA matches that bound; they're both worst-case optimal on the scan.

The preprocessing is more nuanced. KMP's $\Theta(m)$ preprocessing is information-theoretically tight: any algorithm that wants to use the structure of the pattern at scan time needs to spend $\Omega(m)$ time reading the pattern at least once. Galil's 1979 paper proved that $\Theta(m)$ preprocessing combined with a $\Theta(n)$ scan is optimal even in a stricter delay-bounded model, where every text character is required to be processed in $O(1)$ worst-case time rather than amortized. KMP's amortized $O(1)$-per-character bound can be tightened to a worst-case $O(1)$-per-character bound with a small modification, which is what Galil's variant achieves.

So KMP is optimal for the exact-matching problem in essentially every formal sense — preprocessing time, scan time, total space — except for the constant factor in the scan, where the brute force from chapter 23 still wins on cache-friendly hardware whenever the average case is what matters. The next chapter starts answering "what if I'm willing to do *more* preprocessing in exchange for *sublinear* scanning?" — which is the door Boyer-Moore opens.

25.9 What KMP teaches you

Two lessons I want you to carry forward.

First, the right representation can collapse an algorithm down to one variable. Chapter 24's DFA stored a two-dimensional table — state by character — and chapter 25's KMP stores a one-dimensional array, the failure function. Same algorithm, same scan-time behaviour, dramatically less space. The collapse happened because most of the table's entries were answering the same question ("on a mismatch, what's the longest still-salvageable prefix?") and that question has a one-number answer per state. When you see a table whose rows are dense but whose entries collapse along a single axis, you have a candidate for a sparse-encoding rewrite. Compilers, parsers, regular-expression engines, and string-matching libraries all live on variants of this idea.

Second, an algorithm that "never goes backward" deserves a closer look. The text pointer in KMP is a one-way variable: it strictly increases over the scan, and that monotonicity is the entire reason the scan is $\Theta(n)$. Whenever you can convert a re-scan into a stay-and-fall-back, you turn a multiplicative cost into an additive one. This pattern recurs across the whole book — the Knuth-Morris-Pratt matcher, the linear-time amortized analyses of dynamic arrays, the union-find with path compression, the Fibonacci-heap decrease-key — every one of them is a story about replacing repeated traversal with a smarter local-state update. KMP is the cleanest example of the pattern, which is why it's a fixture of algorithms curricula.

25.10 Notes and further reading

KMP was introduced in Knuth, Morris, and Pratt's 1977 paper "Fast Pattern Matching in Strings" in *SIAM Journal on Computing* 6(2):323–350. The paper proves both correctness and the linear-time scan bound using an amortized potential argument — the technique I sketched in the "How efficient is it?" section is theirs, simplified. The historical context is worth knowing: Morris and Pratt independently invented the algorithm in 1970, Knuth saw a connection to automaton theory and contributed the DFA construction that chapter 24 covers, and the three of them published jointly. The DFA-first presentation I've been using over the past two chapters mirrors Knuth's own preferred exposition.

CLRS section 32.4 covers KMP directly — same algorithm, slightly different notation. Sedgewick and Wayne's *Algorithms* (4th ed.) section 5.3 covers KMP without the DFA detour, presenting the failure function as a stand-alone trick rather than as a compression of the DFA. I prefer the DFA-first path because it makes the failure function feel inevitable, but the failure-function-first path is the more common textbook treatment and worth seeing for contrast.

Zvi Galil's 1979 paper "On Improving the Worst-Case Running Time of the Boyer-Moore String Matching Algorithm" in *Communications of the ACM* 22(9):505–508 — confusingly titled, given the topic — also contains the optimization of KMP that achieves true $O(1)$-per-character delay rather than the amortized bound. The technique is to precompute, in addition to the failure function, a second table called the "delay function" or "next function" that handles the mismatch case in true constant time. Most production KMP implementations don't bother — amortized is fast enough — but the result is theoretically clean.

For deeper reading on the connection between automata and string matching, Crochemore, Hancart, and Lecroq's *Algorithms on Strings* (Cambridge 2007) is the modern reference. The book treats KMP, the DFA construction from chapter 24, Aho-Corasick (chapter 28), and the entire family of suffix-prefix algorithms as variations on a single theme — the same theme I've been pointing at across these last three chapters.

In the next chapter I'll change the lens completely. Instead of scanning the text left to right and using preprocessing to skip work after a mismatch, I'll scan each window right-to-left and use preprocessing to skip *entire windows* on a mismatch — the Boyer-Moore algorithm, whose best case beats $\Theta(n)$ by reading only a fraction of the text characters and whose worst case is the question that motivated Galil's paper.

26 Skipping ahead by reading the mismatch

I closed chapter 25 by promising a change of lens. KMP and the chapter-24 DFA both scan left to right, both read every text character at least once, and both spend preprocessing on the question "after a mismatch, how much of my recent progress is salvageable?" Their answer is $\Theta(n)$ — every text character is read exactly once, and that bound is information-theoretically tight if you insist on scanning the text in order. Boyer-Moore breaks that assumption. By scanning each window of the text *right to left* and preprocessing the pattern with a different question in mind, the algorithm earns the right to skip entire stretches of the text without reading them at all. **The wrong letter is a hint — it tells you how far you can jump.** When a window mismatches at some position, the character that disagreed names a shift distance directly, and that distance is often far larger than one.

By the end of this chapter you'll have built two preprocessing tables — the bad-character table and the good-suffix table — combined them into a matcher that runs in $O(n/m)$ in the favorable case and $O(nm)$ in the worst, watched it recover the same four matches of `"ain"` you've been verifying since chapter 23, and seen it do dramatically fewer character comparisons than brute force on the same input. You'll also see, honestly, where the sublinear win evaporates: there are pathological patterns where neither heuristic rescues the algorithm, and the worst case is genuinely $O(nm)$ until Galil's 1979 modification patches it. I'll point at Galil but won't implement him — the patch is intricate and the pedagogical payoff is small.

The chapter's promise is that you'll leave with a working, instrumented Boyer-Moore matcher and a clear intuition for why scanning order matters: right-to-left exposes a class of skips that left-to-right structurally cannot see.

26.1 Reading the window right-to-left

Imagine you're matching the pattern `"abracadabra"` against a window of text that starts with `"xxxxxxxxxxx"`. Brute force compares `pattern[0]` to `text[0]`, sees a mismatch, shifts by one, compares `pattern[0]` to `text[1]`, sees another mismatch, shifts by one, and so on — eleven text characters consumed across eleven failed shifts before the algorithm has even cleared the first window. KMP improves on that slightly by using the pattern's internal structure, but on this input KMP also stalls: there's no overlap to exploit because no prefix of the pattern matches any suffix of `"xxxxxxx"` — everything is `'x'`, the pattern has no `'x'`, and KMP ends up scanning every text character once.

Now flip the comparison order. Look at `text[10]` first — the character at the *end* of the window. It's `'x'`. The pattern has no `'x'` anywhere. So not only does this window fail to match, but **no window that contains `text[10]` can match either**. I can shift the window so that the right edge moves past position 10 entirely — eleven positions of shift in one step — and try again. Eleven characters of text accounted for in a single comparison. That's the qualitative difference Boyer-Moore makes, and it's the entire reason you'd accept the algorithmic complexity that comes next.

The two heuristics in the rest of this chapter formalize that observation. The **bad-character rule** says: when a window mismatches at position j of the pattern against character c of the text, shift the pattern so that the rightmost occurrence of c inside the pattern lines up with that text position — and if c doesn't appear in the pattern at all, shift past it entirely. The **good-suffix rule** says: when a window has matched some suffix of the pattern before mismatching, shift so that the matched suffix lines up with its next occurrence elsewhere in the pattern. Both rules are independently motivated, both are correct, and Boyer-Moore takes the maximum of the two shifts — the larger jump is always safe because both rules are conservative.

KMP couldn't do any of this. Left-to-right scanning has no analog of "the rightmost occurrence of the mismatched character" — by the time you've seen the mismatch, you've already read every character before it, and there's nothing further to skip. The right-to-left scan is the key insight; everything else is bookkeeping.

26.2 The bad-character heuristic

I'll start with the simpler rule. The bad-character table records, for each character that appears in the pattern, the rightmost index where it appears. Characters that don't appear at all are absent from the table, and the matcher will treat them as if they had index -1 — which is the sentinel that says "shift past entirely."

```
def bad_character_table(pattern: str) -> dict[str, int]:
    """For each character that appears in pattern, the rightmost index where
    it appears. Used by the bad-character heuristic.
    """
    table: dict[str, int] = {}
    for i, c in enumerate(pattern):
        table[c] = i
    return table
```

One left-to-right pass; later writes for the same character overwrite earlier ones, so each character ends up mapped to its rightmost position. Bilingually: for a pattern of length m over an alphabet of size σ, this is $\Theta(m)$ work and produces a table of at most σ entries. For `"abracadabra"`, the table will have five keys.

Watch.

```
from codex.strings.boyer_moore import bad_character_table

bc = bad_character_table("abracadabra")
print(f"bad_character_table('abracadabra') = {bc}")
print()
print("rightmost positions, by character:")
for c, idx in sorted(bc.items()):
    print(f"  '{c}' -> {idx}")
```

```
bad_character_table('abracadabra') = {'a': 10, 'b': 8, 'r': 9, 'c': 4, 'd': 6}

rightmost positions, by character:
  'a' -> 10
  'b' -> 8
  'c' -> 4
  'd' -> 6
  'r' -> 9
```

You can read off the table directly. `'a'` last appears at position 10 (the final character), `'b'` at position 8, `'r'` at position 9, `'c'` at position 4, `'d'` at position 6. The characters `'e'`, `'f'`, and so on aren't keys — they don't appear in the pattern, and on a mismatch against any of them the algorithm will treat their "rightmost index" as -1.

The shift formula is short. If you mismatched at pattern position j against text character c, the bad-character shift is

$$\text{bc_shift} = \max(1, j - \text{bc.get}(c, -1)).$$

Two cases. If c appears in the pattern at some position $\leq j$, the rule lines that occurrence up with the text position you just mismatched. If c appears in the pattern only at positions $> j$, the formula would produce a negative or zero shift — that's why the $\max(1, \cdot)$ clamps it to a forward step. And if c doesn't appear at all, the sentinel -1 gives $j - (-1) = j + 1$, which shifts the pattern entirely past the mismatched text position.

Let me trace one shift by hand. Pattern `"abracadabra"`, text starts with `"...abracadabri..."`, and you've just matched all but the last character — you scanned `pattern[10]` against `text[i + 10]`, then `pattern[9]` against `text[i + 9]`, all the way back to `pattern[1] = 'b'` against `text[i + 1] = 'b'`. Now `pattern[0] = 'a'` against `text[i + 0]` — actually, that case is a full match because I built the example backward. Let me try a real mismatch. Suppose the text window is `"abracadaXra"` — same as the pattern except a `'X'` at position 8. You scan right-to-left: positions 10, 9, then mismatch at position 8 (pattern `'b'` vs text `'X'`). The bad-character

rule looks up `'X'` in the table; it's absent, so the rightmost-index is -1. The shift is $\max(1, 8-(-1)) = 9$. The pattern jumps nine positions forward, past the `'X'` entirely.

That single nine-position jump is what makes Boyer-Moore feel different from KMP. KMP would have stayed within the window and consulted its failure function; Boyer-Moore looked at one bad character and skipped most of the window in response.

26.3 The good-suffix heuristic

The bad-character rule alone isn't enough. Some mismatches happen on characters that *do* appear elsewhere in the pattern, and the bad-character shift can degenerate to 1 — no skip at all. The good-suffix rule covers those cases.

The setup is this: at the moment of mismatch, you've successfully matched the suffix `pattern[j+1:]` against the corresponding window of text. The mismatch is between `pattern[j]` and the text character beside it. The "good suffix" is that matched suffix — the run of characters that *did* agree. The good-suffix rule asks: where else in the pattern does this suffix appear, in a position where the character preceding it is *different* from `pattern[j]`? If it appears elsewhere, shift so that occurrence lines up. If it doesn't appear elsewhere, fall back to the longest pattern *prefix* that's also a suffix of the good suffix. If neither applies, the entire pattern can be shifted past.

Two cases, written precisely:

- **Case 1.** Look for the rightmost index $i < j$ such that `pattern[i+1 : i+1+k]` equals the good suffix (where k is the length of the good suffix) *and* `pattern[i]` is different from `pattern[j]`. The "different" requirement is what makes the rule conservative — if the preceding character were the same, you'd just repeat the same mismatch.
- **Case 2.** If no such i exists, find the longest prefix of the pattern that is also a suffix of the good suffix; the shift aligns that prefix with the tail of the matched window. If even that prefix is empty, the shift is the full pattern length.

Worked example on `"abracadabra"`. Suppose the mismatch is at $j = 7$. The good suffix is `pattern[8:11] = "bra"`, length 3. Does `"bra"` appear elsewhere in the pattern? Yes — at positions 1 through 3. The character preceding that occurrence is `pattern[0] = 'a'`. The character at the mismatch is `pattern[7] = 'a'`. They're the same. So Case 1 *doesn't* apply at $i = 0$ (the rule requires the preceding characters to differ). Does `"bra"` appear at any other position with a different preceding character? It doesn't appear anywhere else at all. So Case 1 fails entirely. Now Case 2: the longest prefix of `"abracadabra"` that's also a suffix of `"bra"`. The suffixes of `"bra"` are `""`, `"a"`, `"ra"`, `"bra"`. The prefixes of `"abracadabra"` are `"a"`, `"ab"`, and so on. Their intersection is `{"", "a"}`. The longest is `"a"`, length 1. So the shift is $m - 1 = 11 - 1 = 10$. The pattern jumps ten characters forward, leaving only one character of overlap.

That's a large skip for one mismatch. Here's the function that computes the shift for any mismatch position j.

```
def good_suffix_shift(pattern: str, j: int) -> int:
    """Compute the good-suffix shift when the mismatch is at position j of the
    pattern (so pattern[j+1:m] matched against the text - the 'good suffix').

    Case 1: the rightmost reoccurrence of the good suffix elsewhere in the
    pattern, where the character preceding the reoccurrence differs from
    pattern[j] (so the same mismatch isn't repeated).

    Case 2: if no such reoccurrence, the longest prefix of pattern that is
    also a suffix of the good suffix.

    If neither applies, shift past the entire pattern (return m).
    """
    m = len(pattern)
    good_suffix = pattern[j + 1:]
    k = len(good_suffix)

    # Case 1
    for shift_pos in range(j - 1, -1, -1):
        end = shift_pos + 1 + k
```

```
        if end > m:
            continue
        if pattern[shift_pos + 1:end] == good_suffix and (
            shift_pos < 0 or pattern[shift_pos] != pattern[j]
        ):
            return j - shift_pos

    # Case 2
    for prefix_len in range(k, 0, -1):
        if good_suffix.endswith(pattern[:prefix_len]):
            return m - prefix_len

    return m
```

The Case 1 loop walks `shift_pos` from $j - 1$ down to 0, looking for an earlier position whose following k characters equal the good suffix and whose preceding character differs from `pattern[j]`. The Case 2 loop walks shrinking prefix lengths, looking for the longest pattern prefix that matches as a suffix of the good suffix. If both fail, the shift is the entire pattern length m — meaning the window can move past every character it just saw.

I want to be honest about this implementation: it is **not** asymptotically optimal. For one call with one j, the work is $O(m^2)$ — the outer loop is $O(m)$, and the string-slice comparison inside is $O(m)$ per iteration. There exist linear-time algorithms (the Galil-Giancarlo "Z-array" approach, or the original Boyer-Moore preprocessing using suffix-prefix functions) that produce the whole table in $\Theta(m)$, but the code is intricate and adds a lot of vocabulary you don't need at this point in the book. I'm trading asymptotic optimality on preprocessing for pedagogical clarity. The matching loop's behavior is the same either way; only the build time differs.

To make the function easier to use, I'll precompute the shift for every possible mismatch position once, up front.

```
def good_suffix_table(pattern: str) -> list[int]:
    """Precompute good_suffix_shift for every mismatch position 0..m-1.
    Returns a list of length m where table[j] is the shift on mismatch at j.
    """
    m = len(pattern)
    return [good_suffix_shift(pattern, j) for j in range(m)]
```

That's m calls each costing $O(m^2)$, so the whole table costs $O(m^3)$. For a pattern of length 11, that's about 1,300 operations — invisibly fast. For a pattern of length 1,000, it's a billion operations — clearly bad. I'll repeat the disclaimer in the Notes section: a production Boyer-Moore preprocessor would use the linear-time technique. The matcher I'm about to build is fast on every input; only the preprocessing scales poorly with m.

Let me show you the good-suffix table for `"abracadabra"`.

```
from codex.strings.boyer_moore import good_suffix_table

gs = good_suffix_table("abracadabra")
print(f"good_suffix_table('abracadabra') = {gs}")
print()
print("by mismatch position j:")
pattern = "abracadabra"
for j, shift in enumerate(gs):
    good_suffix = pattern[j + 1:]
    print(f"  j = {j:>2}   mismatch on '{pattern[j]}'   good suffix = {good_suffix!r:<13}   shift = {sh
```

```
good_suffix_table('abracadabra') = [7, 7, 7, 7, 7, 7, 7, 10, 10, 3, 1]

by mismatch position j:
  j =  0   mismatch on 'a'   good suffix = 'bracadabra'    shift = 7
  j =  1   mismatch on 'b'   good suffix = 'racadabra'     shift = 7
  j =  2   mismatch on 'r'   good suffix = 'acadabra'      shift = 7
```

```
  j =  3   mismatch on 'a'   good suffix = 'cadabra'       shift = 7
  j =  4   mismatch on 'c'   good suffix = 'adabra'        shift = 7
  j =  5   mismatch on 'a'   good suffix = 'dabra'         shift = 7
  j =  6   mismatch on 'd'   good suffix = 'abra'          shift = 7
  j =  7   mismatch on 'a'   good suffix = 'bra'           shift = 10
  j =  8   mismatch on 'b'   good suffix = 'ra'            shift = 10
  j =  9   mismatch on 'r'   good suffix = 'a'             shift = 3
  j = 10   mismatch on 'a'   good suffix = ''              shift = 1
```

Read the table from the right. At $j = 10$, the good suffix is empty (you mismatched on the last character without matching anything), so the table reports a fallback shift of 1 — the smallest legitimate move. At $j = 7$, you matched `"bra"` and the shift is 10, as I derived by hand above. The intermediate values reflect the pattern's internal structure: the more the matched suffix resembles internal substrings of the pattern, the smaller the shift; the more uniquely the matched suffix sits at the end of the pattern, the larger the shift.

26.4 Combining the two: the matcher

With both tables built, the matching loop is short. For each candidate alignment i, scan the window right-to-left until either the pattern is exhausted (full match) or a mismatch is found. On a mismatch at pattern position j against text character c, compute both shifts and take the larger one.

```python
def find(text: str, pattern: str) -> int | None:
    n, m = len(text), len(pattern)
    if m == 0:
        return 0
    if m > n:
        return None
    bc = bad_character_table(pattern)
    gs = good_suffix_table(pattern)
    i = 0
    while i <= n - m:
        j = m - 1
        while j >= 0 and pattern[j] == text[i + j]:
            j -= 1
        if j < 0:
            return i
        bc_shift = max(1, j - bc.get(text[i + j], -1))
        gs_shift = gs[j]
        i += max(bc_shift, gs_shift)
    return None
```

The outer `while` walks the candidate alignment forward; the inner `while` does the right-to-left comparison inside the current window. The shift on each iteration is the maximum of `bc_shift` and `gs_shift` — taking the larger jump is always safe because both rules are individually conservative (each guarantees that no match is missed in the skipped range), and the maximum of two conservative bounds is itself conservative.

Let me trace one full execution on a small example. Text `"abracadabraXabracadabra"`, pattern `"abracadabra"`. Length is 11 for the pattern, 23 for the text. The first valid alignment is $i = 0$.

At $i = 0$: scan right-to-left. `pattern[10] = 'a'` versus `text[10] = 'a'`. Match. Continue down to `pattern[0] = 'a'` versus `text[0] = 'a'`. All match. j falls to -1. The function returns 0 — a match at the start.

But that early return is uninteresting; let me force a continuation by looking for the *second* match by hand. After reporting $i = 0$, imagine restarting the scan at $i = 1$ instead. Now `text[1..11]` is `"bracadabraX"`. Scan right-to-left: `pattern[10] = 'a'` versus `text[11] = 'X'`. Mismatch at $j = 10$. Bad-character: `'X'` is not in the pattern, so the shift is $\max(1, 10 - (-1)) = 11$. Good-suffix: $gs[10] = 1$ (degenerate case). Take the max — shift by 11. Now i jumps to $1 + 11 = 12$.

At $i = 12$: the text window is `text[12..23] = "abracadabra"`. Right-to-left scan matches every character. Full match reported at $i = 12$. One bad-character shift skipped eleven positions of text in one step. That's the kind of compression you came here for.

The matcher above only returns the first match. For all matches, I want a parallel `find_all`.

```
def find_all(text: str, pattern: str) -> list[int]:
    """All matches. On a full match, shift by 1 to find every overlapping
    match. Proper overlap handling in BM requires an additional good-suffix
    preprocessing for the case-when-the-whole-pattern-matched, which I'm
    skipping for clarity; shift-by-1 is correct (just not necessarily fast).
    """
    n, m = len(text), len(pattern)
    if m == 0 or m > n:
        return []
    bc = bad_character_table(pattern)
    gs = good_suffix_table(pattern)
    matches: list[int] = []
    i = 0
    while i <= n - m:
        j = m - 1
        while j >= 0 and pattern[j] == text[i + j]:
            j -= 1
        if j < 0:
            matches.append(i)
            i += 1  # overlap-safe shift
        else:
            bc_shift = max(1, j - bc.get(text[i + j], -1))
            gs_shift = gs[j]
            i += max(bc_shift, gs_shift)
    return matches
```

The post-match shift here is conservatively set to 1, which guarantees every overlapping match is discovered. A more aggressive shift exists (the so-called *strong good-suffix* rule extends the table to cover the post-match case) but adds enough preprocessing complexity that I'd rather not. For chapters where overlaps are rare, shift-by-1 is essentially free; for pathological overlap-heavy inputs, this is one of the worst-case spots Boyer-Moore has.

26.5 Running on the canonical text

Time to verify on the input I've been carrying since chapter 23. Pattern `"ain"`, text `"the rain in spain stays mainly in the plain"`.

```
from codex.strings.boyer_moore import find, find_all

TEXT = "the rain in spain stays mainly in the plain"

idx = find(TEXT, "ain")
print(f"first match of 'ain' at index {idx} - same answer as ch 23, 24, 25")

all_ain = find_all(TEXT, "ain")
print(f"all matches of 'ain': {all_ain}  (four occurrences, identical to previous chapters)")

miss = find(TEXT, "frog")
print(f"searching for 'frog': {miss}  (no match - BM skips quickly because 'f' is absent)")
```

```
first match of 'ain' at index 5 - same answer as ch 23, 24, 25
all matches of 'ain': [5, 14, 25, 40]  (four occurrences, identical to previous chapters)
searching for 'frog': None  (no match - BM skips quickly because 'f' is absent)
```

Four matches at 5, 14, 25, 40 — the answer you've been verifying for three chapters now. The mechanism is completely different (right-to-left scan with bad-character and good-suffix preprocessing) but the externally observable behavior is identical, by design.

Now I want to look at the *cost* of finding those matches, because that's where Boyer-Moore distinguishes itself from KMP and brute force. I'll instrument the matcher to count character comparisons and window shifts on each call.

```
def find_counting(text: str, pattern: str) -> tuple[int | None, int, int]:
    """Instrumented version: returns (first match index or None,
    number of character comparisons, number of window shifts).
    """
    n, m = len(text), len(pattern)
    if m == 0:
        return 0, 0, 0
    if m > n:
        return None, 0, 0
    bc = bad_character_table(pattern)
    gs = good_suffix_table(pattern)
    comparisons = 0
    shifts = 0
    i = 0
    while i <= n - m:
        shifts += 1
        j = m - 1
        while j >= 0:
            comparisons += 1
            if pattern[j] != text[i + j]:
                break
            j -= 1
        if j < 0:
            return i, comparisons, shifts
        bc_shift = max(1, j - bc.get(text[i + j], -1))
        gs_shift = gs[j]
        i += max(bc_shift, gs_shift)
    return None, comparisons, shifts
```

The instrumented matcher counts every character read and every window position attempted. Brute force's `find_counting` from chapter 23 reports the same comparison count for the same input, so they're directly comparable.

```
from codex.strings.boyer_moore import find_counting
from codex.strings.brute import find_counting as brute_find_counting

idx_bm, cmps_bm, shifts_bm = find_counting(TEXT, "ain")
idx_bf, cmps_bf = brute_find_counting(TEXT, "ain")

print(f"text length: {len(TEXT)} characters, pattern 'ain' (length 3)")
print(f"Boyer-Moore: first match at {idx_bm}, {cmps_bm} comparisons, {shifts_bm} window shifts")
print(f"brute force: first match at {idx_bf}, {cmps_bf} comparisons")
print(f"ratio: BM does {cmps_bm / cmps_bf:.2f}x as many comparisons as brute force here")
```

```
text length: 43 characters, pattern 'ain' (length 3)
Boyer-Moore: first match at 5, 5 comparisons, 3 window shifts
brute force: first match at 5, 8 comparisons
ratio: BM does 0.62x as many comparisons as brute force here
```

On this short input, Boyer-Moore does fewer comparisons than brute force already, but the win is modest because the pattern is only three characters long and the text isn't structured to favor large skips. The pattern's value shows up when the pattern is longer or the text has long stretches with no overlap.

26.6 Sublinear in practice

The headline claim about Boyer-Moore is that its best case is sublinear — under favorable conditions, it reads only a fraction of the text characters. Here's a demonstration designed to show that clearly.

```
from codex.strings.boyer_moore import find_counting
from codex.strings.brute import find_counting as brute_find_counting

text = "x" * 100 + "abracadabra"
pattern = "abracadabra"

idx_bm, cmps_bm, shifts_bm = find_counting(text, pattern)
idx_bf, cmps_bf = brute_find_counting(text, pattern)

print(f"text length: {len(text)} ({100} x's followed by 'abracadabra')")
print(f"pattern length: {len(pattern)}")
print()
print(f"Boyer-Moore: first match at {idx_bm}")
print(f"  comparisons: {cmps_bm}")
print(f"  window shifts attempted: {shifts_bm}")
print()
print(f"brute force: first match at {idx_bf}")
print(f"  comparisons: {cmps_bf}")
print()
print(f"BM read {cmps_bm} / {len(text)} = {100 * cmps_bm / len(text):.1f}% of the text characters befor
```

```
text length: 111 (100 x's followed by 'abracadabra')
pattern length: 11

Boyer-Moore: first match at 100
  comparisons: 21
  window shifts attempted: 11

brute force: first match at 100
  comparisons: 111

BM read 21 / 111 = 18.9% of the text characters before declaring a match
```

You can see the shape directly. Boyer-Moore did roughly one comparison per window shift across the `"xxxx..."` prefix — `'x'` isn't in the pattern, so each window mismatches at `pattern[10] = 'a'` against `text[i + 10] = 'x'`, the bad-character rule shifts the pattern eleven positions forward (past the `'x'` entirely), and the next window starts beyond where the previous one ended. The pattern almost never "looks back" at characters it already saw. Brute force, by contrast, examined every prefix `"x", "xa", "xab", ...` and gave up after one or two comparisons per window — but it had to attempt every window position, which is roughly one attempt per text character.

The asymptotic claim is $O(n/m)$ in the favorable case, where n is the text length and m is the pattern length. Bilingually: for a 100-character text with an 11-character pattern under favorable conditions, you'd expect roughly $100/11 \approx 9$ comparisons of bad-character work plus a constant for the eventual match — about 20 comparisons, give or take. The actual count above sits in that range. Compare to brute force's ~ 100 comparisons on the same input. The ratio gets more dramatic as the pattern gets longer: a 100-character pattern would skip 100 positions per bad character, and Boyer-Moore would read $\sim n/100$ characters of a long text instead of n.

26.7 When sublinear breaks

Honesty time. Boyer-Moore's worst case is *not* sublinear, and on some inputs it's not even linear. Consider this pathological combination: text `"aaa...a"` (all `'a'`'s), pattern `"aab"`.

```python
from codex.strings.boyer_moore import find_counting
from codex.strings.brute import find_counting as brute_find_counting

text = "a" * 30
pattern = "aab"

idx_bm, cmps_bm, shifts_bm = find_counting(text, pattern)
idx_bf, cmps_bf = brute_find_counting(text, pattern)

print(f"text = 'a' * 30, pattern = {pattern!r}")
print(f"Boyer-Moore: match = {idx_bm}, comparisons = {cmps_bm}, shifts = {shifts_bm}")
print(f"brute force: match = {idx_bf}, comparisons = {cmps_bf}")
print()
print(f"BM comparisons per text character: {cmps_bm / len(text):.2f}")
print(f"brute force comparisons per text character: {cmps_bf / len(text):.2f}")
```

```
text = 'a' * 30, pattern = 'aab'
Boyer-Moore: match = None, comparisons = 28, shifts = 28
brute force: match = None, comparisons = 84

BM comparisons per text character: 0.93
brute force comparisons per text character: 2.80
```

No sublinear win. Boyer-Moore enters each window, scans right-to-left, mismatches immediately at pattern position 2 (`'b'` vs `'a'`), consults the bad-character table — `'a'` is in the pattern at position 1, so the shift is $\max(1, 2 - 1) = 1$, just one position. The good-suffix table also gives shift 1 because the matched suffix is empty. So Boyer-Moore advances by one position per window and does one comparison per window — about n operations total. It still beats brute force on this input (brute force does three comparisons per window, since its inner loop scans `pattern[0..2]` left-to-right and only mismatches at $j = 2$), but only by a small constant factor. The sublinear-in-m scaling I demonstrated above evaporates entirely.

Constructing an input that truly forces $\Theta(nm)$ work out of Boyer-Moore with both heuristics is fussier than it sounds — the good-suffix rule rescues most of the obvious-looking pathological patterns, and Cole's 1994 paper proved a tight $3n + O(m)$ upper bound on comparisons for the version I implemented here, *not* the $\Theta(nm)$ bound the algorithm is often credited with. The $\Theta(nm)$ worst case appears in variants without the good-suffix rule, or in `find_all` workloads where many partial matches force repeated full-pattern scans. The point that survives all the bookkeeping: Boyer-Moore's *expected* behaviour is sublinear on natural text, its *worst* case for `find` is linear (with a constant Cole pinned at 3), and pathological adversarial inputs can degrade `find_all` further without Galil's modification.

For patterns where this pathology is severe, Boyer-Moore can be made truly linear in the worst case by adding a small modification, which is what Galil's 1979 paper introduced. The trick is to track, after a successful partial match, *how much of the next window's right edge is already known to match the pattern* — and skip those characters on the next inner-loop scan rather than re-comparing them. The result is a worst-case bound of $O(n + m)$ comparisons, matching KMP's worst case while preserving Boyer-Moore's sublinear best case.

I'm not implementing Galil's modification. The patch is intricate (it requires maintaining a "memory" of which suffix of the previous window matched the pattern), the standard textbook treatments handle it briefly if at all, and the pedagogical value-per-line-of-code is low. The version I have here is the Boyer-Moore that ships in most undergraduate algorithms courses and in many production string-matching libraries — well-engineered to be fast on real text, with the known caveat that adversarial inputs can degrade it to $O(nm)$.

26.8 The three questions, applied

Is it correct?

Two invariants. The first is that **both shift rules are individually conservative**: each one names a distance that's guaranteed not to skip over any match. The bad-character rule's correctness: if I mismatched at pattern position j against text character c, then any alignment with a smaller shift than `j - bc.get(c, -1)` would

place some pattern position before j over the same text position, where the pattern character is, by definition of the rightmost-index table, either equal to c (impossible because the table records the *rightmost* such position) or different from c (so the alignment fails at that position immediately). Either way, no match is skipped. The good-suffix rule's correctness is symmetric: by construction, every alignment with a smaller shift would either place the same character against the mismatched text position (no progress) or place the good suffix against a pattern region that doesn't equal it (no match).

The second invariant is that **the maximum of two conservative shifts is itself conservative**: if neither shift would skip a match individually, their maximum can't either. The outer loop therefore visits every candidate alignment that could match, and the inner loop's right-to-left comparison detects every match exactly when the entire window has been verified character-for-character. `find` returns the first such i; `find_all` collects all of them, with the post-match shift-by-1 guaranteeing no overlapping match is missed.

How efficient is it?

The preprocessing is $\Theta(m)$ for the bad-character table and $O(m^3)$ for the good-suffix table as I implemented it (linear-time algorithms exist; I'm using the clearer quadratic version). The matching loop's behavior splits by case.

Best case is sublinear: $O(n/m)$ when the bad-character heuristic consistently produces full-pattern shifts. Bilingually: searching a million-character text for a hundred-character pattern under favorable conditions does about ten thousand character comparisons, not a million.

Average case on random text over a large alphabet is also sublinear in practice — empirically about $O(n/m)$ for English text with moderately long patterns. This is why Boyer-Moore is the matcher of choice in many production string-search libraries (including, historically, `grep`).

Worst case for `find` is $\Theta(n+m)$ comparisons (Cole's 1994 tight bound of $3n + O(m)$ for the version I built, including the good-suffix rule). The classic textbook claim of $\Theta(nm)$ worst case applies to the *bad-character-only* variant (Horspool's simplification, which drops good-suffix entirely) and to `find_all` workloads without Galil's modification. For the find-first-match variant I implemented, the good-suffix rule does enough work to keep the bound linear even on adversarial input. Galil's modification matters most when you want a worst-case-linear `find_all`, which the unmodified Boyer-Moore doesn't guarantee.

Space is $O(m+\sigma)$ for the two preprocessing tables (σ keys in the bad-character dict, m entries in the good-suffix list), and $O(1)$ working space during the scan.

Is it optimal?

Boyer-Moore with the good-suffix rule (the version I wrote) achieves $\Theta(n+m)$ worst-case comparisons for `find`, matching KMP's asymptotic bound — Cole's 1994 paper proved this rigorously, with a tight constant of $3n+O(m)$ (versus KMP's $2n+O(m)$). For `find_all`, the unmodified version can degrade further on adversarial input and needs Galil's 1979 patch to recover linear behaviour.

The deeper claim — and the one that's stayed true since 1977 — is that Boyer-Moore is *practically* optimal on the kinds of text humans actually search: long texts, moderately long patterns, large alphabets, no adversarial structure. In that regime, the sublinear best case dominates and the worst case rarely fires. The matcher you'd actually deploy is some variant of Boyer-Moore (or Horspool's simplification, or Sunday's Quick Search, all in this family); the matcher you'd analyze for worst-case bounds is KMP. The two algorithms live in different parts of the cost-shape plane, and choosing between them is an engineering decision about which input distribution you expect.

26.9 What Boyer-Moore teaches you

Two lessons I want you to take from this chapter.

First, scanning order matters. Reading the window right-to-left exposes a class of skips that left-to-right scanning structurally cannot see — specifically, the skip you get when the *last* character of the window doesn't appear in the pattern at all. KMP couldn't exploit that because by the time it'd seen the mismatch, it had already read every character before it. The right-to-left scan is what makes the bad-character rule possible, and the bad-character rule is what makes the sublinear best case possible. Whenever you're designing an algorithm

that consumes a window of input, ask yourself which character to look at first: the answer often determines whether the cost shape is linear, sublinear, or somewhere worse.

Second, two heuristics can be combined by taking the maximum. Bad-character and good-suffix are independently motivated; one is about the mismatching character, the other is about the matched suffix. They're correct for different reasons and they fire on different inputs. The matcher just takes the max of both shifts on every mismatch, and the resulting algorithm is at least as fast as either heuristic alone on every input. That "use them both, take the larger jump" pattern recurs in algorithm design — when you have two independently conservative bounds, their maximum is itself conservative and usually strictly tighter than either one. You'll see the same pattern in alpha-beta pruning (chapter 32 of the book), in branch-and-bound search, and in interval scheduling with multiple constraints.

26.10 Notes and further reading

The original paper is Boyer and Moore's 1977 "A Fast String Searching Algorithm" in *Communications of the ACM* 20(10):762–772. The paper introduces both heuristics, proves the algorithm's correctness, and presents empirical timing data showing the sublinear behavior on English text. The worst-case analysis is left as an open question in the original — it took Galil's 1979 paper "On Improving the Worst-Case Running Time of the Boyer-Moore String Matching Algorithm" in *Communications of the ACM* 22(9):505–508 to patch the worst case to $O(n + m)$, and Cole's 1994 paper "Tight Bounds on the Complexity of the Boyer-Moore String Matching Algorithm" in *SIAM Journal on Computing* 23(5):1075–1091 to nail down the exact constant.

Horspool's 1980 paper "Practical Fast Searching in Strings" in *Software: Practice and Experience* 10(6):501–506 introduced a simplification using only the bad-character heuristic, with a slightly different shift formula based on the *last* character of the window rather than the mismatched one. Horspool's variant is often faster in practice than full Boyer-Moore because the preprocessing is simpler and the inner loop has fewer branches; the trade-off is that the worst case is worse (no good-suffix backup) and the average case is roughly the same. Sunday's 1990 paper "A Very Fast Substring Search Algorithm" in *Communications of the ACM* 33(8):132–142 — sometimes called *Quick Search* — is another bad-character-only variant that looks at the character *just past* the current window, giving an even tighter shift on most inputs.

CLRS doesn't cover Boyer-Moore directly; the closest treatment is Exercise 32.3-5 with a hint pointing at Horspool. Sedgewick and Wayne's *Algorithms* (4th ed.) section 5.3 covers both the bad-character and good-suffix rules in detail, with the same right-to-left presentation I used here. For the full mathematical treatment including Galil's patch and Cole's analysis, Crochemore, Hancart, and Lecroq's *Algorithms on Strings* (Cambridge 2007) is the modern reference — the same book I pointed you at in chapter 25 for the deeper automaton material.

In the next chapter I'll change the lens again. Instead of preprocessing the pattern to enable smart shifts, I'll preprocess the text into a *hash* and ask "does this rolling hash equal the pattern's hash?" — the Rabin-Karp algorithm, which trades exact comparisons for probabilistic ones and earns the right to match multiple patterns in a single scan.

27 Matching by fingerprint

Chapter 26 closed by promising a change of lens. Boyer-Moore preprocessed the pattern so that a mismatch on a single character would expose a large skip; KMP and the chapter-24 automaton preprocessed the pattern so that no character of the text would ever be read twice. All three are scanning algorithms — they walk the text and decide, character by character, how far to shift. I want to do something different in this chapter. I want to **stop comparing strings entirely**, and instead compare *numbers* that stand in for strings — fingerprints that I can compute cheaply and update even more cheaply as the window slides forward. **Hash the pattern once, roll the window's hash forward in $O(1)$ per step**, then verify only on hash equality. That is the Rabin-Karp algorithm, and it earns a cost shape the previous three couldn't.

By the end of this chapter you'll have built a polynomial-hash function, watched it slide across the canonical text in constant time per shift, seen Rabin-Karp recover the four matches of `"ain"` you've now verified across five chapters, and watched it correctly *handle* a deliberately-induced hash collision — because the algorithm is **Las Vegas**, meaning the answer is always right even when the hashes lie. You'll know why $p \approx 10^9 + 7$ is the modulus people actually pick, and you'll have an instrumented matcher that counts the collisions and verifications so you can see the probabilistic shape directly. I'll also tell you why Rabin-Karp's real niche isn't single-pattern matching at all — it's the multi-pattern case, where one hash table replaces k separate scans.

The chapter's empower-promise is that randomization isn't just a trick for treaps and skip lists; it's a string-matching tool too. Once you've seen rolling hashes, the family of fingerprint algorithms — content-defined chunking, document-similarity hashing, network deduplication — stops looking exotic. They're all this same move, applied to different windows.

27.1 The hash table trick, applied to a sliding window

I want to start by reaching back to chapter 12. That chapter sold you on one move: turning a key into an integer in $\{0, 1, \ldots, m-1\}$ via a deterministic, uniform-ish, fast function, and using the integer as a memory address. The payoff was *constant-time membership* — you handed the table a key, it computed the address, it went straight to the slot. The whole structure depended on the hash function being cheap to compute and on collisions being rare enough that the chain at each slot stayed short.

Now I want you to flip the question. Chapter 12 used a hash for *equality testing inside a single table*. What if I used a hash for *equality testing between two strings* — the pattern and a window of the text? If I can compute h(pattern) once and h(window) for each candidate window, then I can compare hashes as a cheap first filter. Equal hashes are not a *proof* of equal strings (collisions exist, the same way they did in chapter 12), but unequal hashes are a *proof* of unequal strings — and that asymmetry is what makes the trick worth doing. I skip the full character-by-character comparison except when the hashes match, and on most windows the hashes won't match.

The catch — and it's a real one — is that recomputing h(window) from scratch for every shift costs $O(m)$ per window, which gives $O(nm)$ for the whole scan, the same as brute force. The hashing has bought me nothing if every window forces me to read every character anyway. So I need a hash function with one extra property: it has to be **rollable**. The hash of `text[i+1..i+m]` should be computable from the hash of `text[i..i+m-1]` in $O(1)$ — without re-reading all m characters of the new window. If I can do that, I pay $O(m)$ once to hash the initial window, then $O(1)$ per shift, and the total scan is $O(n + m)$ in the *expected* case. That's the cost shape Rabin-Karp is built for.

So the design problem is: find a hash function for strings that supports a constant-time roll. The polynomial hash is the answer, and the next section is where I build it.

27.2 A polynomial hash for strings

The trick is to read the string as a number — specifically, as a multi-digit number written in a chosen *base*, with the characters as digits. Pick a base b and a prime modulus p. Define the hash of a string s of length m as

$$h(s) = \left(\sum_{i=0}^{m-1} \text{ord}(s[i]) \cdot b^{m-1-i} \right) \bmod p.$$

You can read that formula by treating each character as a digit and the string as the integer that would result if you wrote those digits in base b. Bilingually: for `"cat"` with $b = 257$, the value before the modulus is $\text{ord}(\text{`c`}) \cdot 257^2 + \text{ord}(\text{`a`}) \cdot 257^1 + \text{ord}(\text{`t`}) \cdot 257^0$. The modulus is what keeps the integer bounded — without it, the values would grow exponentially in the string length and arithmetic would stop being $O(1)$ per operation.

Why polynomial? Because evaluating that sum incrementally is *Horner's rule*, and Horner's rule lets you compute the whole thing in m multiplications and m additions, with the running value never larger than $b \cdot p$. The code is one loop.

```
def polynomial_hash(s: str, base: int, modulus: int) -> int:
    """Horner-form polynomial hash of s in base `base` modulo `modulus`.

    Reads s as a multi-digit number with each character treated as a digit
    in base `base`, then reduces modulo `modulus` to keep the integer
    bounded. Runs in O(len(s)) time and O(1) space.
    """
    h = 0
    for c in s:
        h = (h * base + ord(c)) % modulus
    return h
```

Three lines of body. On each iteration I shift the accumulated value one digit to the left (multiply by `base`), add the new character's ord, and reduce mod `modulus`. That reduction at every step is what keeps `h` bounded by `modulus` throughout the loop, so the arithmetic stays $O(1)$ per character — Python's arbitrary-precision int doesn't charge you for big numbers you never let grow.

Why base 257 and modulus $10^9 + 7$? The base wants to be larger than the largest character code so that different characters genuinely move the value (with $b = 1$ the hash would collapse to a sum of ord values and `"abc"` and `"bac"` would collide). One past 256 — the largest byte value — is the common choice; some textbooks use 31 or 53, which also work for ASCII text but are less spread on byte-valued inputs. The modulus wants to be a *prime* near 2^{30}: large enough that random collisions are rare (the probability is roughly $1/p$ per comparison, so $p \approx 10^9$ gives one expected false positive per billion comparisons), small enough that all intermediate products fit comfortably in a 64-bit machine word so the arithmetic stays cheap even outside Python. The specific value $10^9 + 7$ is a competitive-programming staple — a prime, easy to remember, well within machine-word range. There's nothing magical about the exact constant.

Watch.

```
from codex.strings.rabin_karp import polynomial_hash

for s in ["cat", "dog", "ain", "rain", "spain", "the rain"]:
    h = polynomial_hash(s, base=257, modulus=10**9 + 7)
    print(f"  polynomial_hash({s!r:>10}, 257, 10^9+7) = {h}")
```

```
  polynomial_hash(     'cat', 257, 10^9+7) = 6563896
  polynomial_hash(     'dog', 257, 10^9+7) = 6633530
  polynomial_hash(     'ain', 257, 10^9+7) = 6433848
  polynomial_hash(    'rain', 257, 10^9+7) = 941537443
  polynomial_hash(   'spain', 257, 10^9+7) = 591680858
  polynomial_hash('the rain', 257, 10^9+7) = 482846166
```

Each string maps to a 30-ish-bit integer in $[0, 10^9 + 7)$. The values look unstructured (which is what I want — uniformity is one of the three properties from chapter 12), and they're deterministic — running the same input twice gives the same number every time.

27.3 Rolling the hash forward in $O(1)$

Here is where the polynomial form earns its keep. Suppose I've already computed the hash of `text[i..i+m-1]`, and now I want the hash of `text[i+1..i+m]` — the same window shifted one position to the right. The new window drops the character at position i and gains the character at position $i + m$. The polynomial structure makes this update mechanical.

Think of the old window's hash as a polynomial evaluated at b:

```
window i:     text[i..i+m-1]    hash = ord(text[i])  * b^(m-1) + ord(text[i+1]) * b^(m-
2) + ... + ord(text[i+m-1])
window i+1:   text[i+1..i+m]    hash = ord(text[i+1]) * b^(m-1) + ord(text[i+2]) * b^(m-
2) + ... + ord(text[i+m])
```

Read those two lines and ask: how do I get from the first to the second? Three steps. **Subtract** the leading term, which is $\text{ord}(\text{text}[i]) \cdot b^{m-1}$ — that's the contribution of the character I'm dropping. **Multiply** everything by b — that shifts every remaining term one position to the left in the polynomial. **Add** $\text{ord}(\text{text}[i+m])$ — that's the new character in the lowest-order (rightmost) position. Reduce mod p. Done.

In one formula:

$$h_{i+1} = ((h_i - \text{ord}(\text{text}[i]) \cdot b^{m-1}) \cdot b + \text{ord}(\text{text}[i+m])) \bmod p.$$

Three multiplications, two additions/subtractions, one modulus — six arithmetic operations regardless of how long the window is. That is the $O(1)$ I promised, and the polynomial form is why I get it.

One detail. The quantity $b^{m-1} \bmod p$ shows up on every roll, so I want to precompute it once. Python's three-argument `pow` does *modular exponentiation* by repeated squaring, in $O(\log m)$ multiplications, each on integers bounded by p. That's much faster than `(base ** (m - 1)) % modulus`, which would build the un-modded integer b^{m-1} as a Python big-int (potentially thousands of digits long) before reducing. For a pattern of length 1000, three-argument `pow` does about 10 modular multiplications instead of 1000 unbounded ones. I'll use it without comment in the matcher below.

One more subtle detail worth naming. The intermediate value $h_i - \text{ord}(\text{text}[i]) \cdot b^{m-1}$ can go negative — the subtraction can produce a value less than zero before the modulus reduces it. In C this would be a problem (the language defines the sign of the modulus on negatives as implementation-dependent). In Python, `%` is defined so that `(-3) % 11 == 8` — the result always has the same sign as the divisor, and the divisor is positive, so the result is always in $[0, p)$ regardless of the sign of the dividend. The rolling formula is correct in Python as written. If you're porting this to a language with different modulus semantics, you'll need to add p before the reduction.

Before I write the matcher I want you to *see* the rolling hash in action. The chunk below computes the hash of every length-3 window across a short text in two ways — from scratch each time, and rolled from the previous — and prints both so you can confirm they agree.

```
from codex.strings.rabin_karp import polynomial_hash

text = "the rain in"
m = 3
base = 257
mod = 10**9 + 7
high_power = pow(base, m - 1, mod)

print(f"text = {text!r}, window length = {m}\n")
print(f"{'i':>2}  {'window':<5}  {'hash from scratch':>20}  {'hash rolled':>15}")
print("-" * 50)

# Initial window: hash from scratch
h_rolled = polynomial_hash(text[:m], base, mod)
print(f"{0:>2}  {text[:m]!r:<5}  {polynomial_hash(text[:m], base, mod):>20}  {h_rolled:>15}")

# Each subsequent window: roll forward
```

```
for i in range(1, len(text) - m + 1):
    h_rolled = ((h_rolled - ord(text[i - 1]) * high_power) * base
                + ord(text[i + m - 1])) % mod
    h_scratch = polynomial_hash(text[i:i + m], base, mod)
    marker = "  OK" if h_rolled == h_scratch else "  MISMATCH!"
    print(f"{i:>2}  {text[i:i + m]!r:<5}  {h_scratch:>20}  {h_rolled:>15}{marker}")
```

```
text = 'the rain in', window length = 3

 i  window     hash from scratch      hash rolled
-------------------------------------------------
 0  'the'               7688513          7688513
 1  'he '               6895085          6895085  OK
 2  'e r'               6679287          6679287  OK
 3  ' ra'               2142963          2142963  OK
 4  'rai'               7554620          7554620  OK
 5  'ain'               6433848          6433848  OK
 6  'in '               6963447          6963447  OK
 7  'n i'               7273719          7273719  OK
 8  ' in'               2140663          2140663  OK
```

Two columns, identical at every row. The right-hand column does six arithmetic operations per window regardless of window length; the left-hand column does m operations per window. For $m = 3$ that's a $2\times$ savings; for a 100-character pattern, the rolling approach reads only one character per shift while the scratch approach reads all 100. That ratio is where Rabin-Karp's $O(n + m)$ expected time comes from.

27.4 The matching loop with Las Vegas verification

Now I have the two pieces I need: a polynomial hash, and a constant-time roll. The matcher precomputes $h(\text{pattern})$ and $h(\text{text}[0..m-1])$ once, then walks each candidate window. At each step it compares hashes; on equality, it verifies the actual strings character-by-character to rule out a collision. On a true match, it returns. Otherwise it rolls the hash forward and continues.

```
def find(
    text: str, pattern: str, base: int = 257, modulus: int = 1_000_000_007
) -> int | None:
    """First match index via Rabin-Karp rolling-hash with Las Vegas verification."""
    n, m = len(text), len(pattern)
    if m == 0:
        return 0
    if m > n:
        return None
    pattern_hash = polynomial_hash(pattern, base, modulus)
    window_hash = polynomial_hash(text[:m], base, modulus)
    high_power = pow(base, m - 1, modulus)
    if window_hash == pattern_hash and text[:m] == pattern:
        return 0
    for i in range(1, n - m + 1):
        window_hash = (
            (window_hash - ord(text[i - 1]) * high_power) * base
            + ord(text[i + m - 1])
        ) % modulus
        if window_hash == pattern_hash and text[i : i + m] == pattern:
            return i
    return None
```

The body is short. Two hash computations up front, one `pow` for the high-order coefficient, then a loop that rolls and tests. The check `window_hash == pattern_hash and text[i : i + m] == pattern` is the *Las Vegas*

verification — and I want you to spend a sentence on what it does. The first conjunct is the cheap filter: an integer comparison, $O(1)$, true for most windows only by accident. The second conjunct is the certificate: a full string equality, $O(m)$, run only when the filter passes. Python's `and` short-circuits, so the string comparison fires *only* when hashes already agree. On most windows you pay one integer compare and move on.

This is what makes the algorithm Las Vegas in the technical sense — randomized algorithms that are **always correct** but **sometimes slow**. Rabin-Karp never misses a match (a true match always produces equal hashes), and never reports a false one (the string check catches every collision). The randomness — the hash function and its modulus — affects only how *often* the slow path fires. With a good modulus, almost never; with a bad one, often. Either way, the answer you read off at the end is right.

`find_all` is the same loop, but appends to a list instead of returning, with no early termination.

```
def find_all(
    text: str, pattern: str, base: int = 257, modulus: int = 1_000_000_007
) -> list[int]:
    """All overlapping match indices via Rabin-Karp."""
    n, m = len(text), len(pattern)
    if m == 0 or m > n:
        return []
    pattern_hash = polynomial_hash(pattern, base, modulus)
    window_hash = polynomial_hash(text[:m], base, modulus)
    high_power = pow(base, m - 1, modulus)
    matches: list[int] = []
    if window_hash == pattern_hash and text[:m] == pattern:
        matches.append(0)
    for i in range(1, n - m + 1):
        window_hash = (
            (window_hash - ord(text[i - 1]) * high_power) * base
            + ord(text[i + m - 1])
        ) % modulus
        if window_hash == pattern_hash and text[i : i + m] == pattern:
            matches.append(i)
    return matches
```

And the instrumented variant that I'll use to *see* the algorithm's probabilistic behavior. It returns the first match (or `None`), the number of hash collisions detected (windows where the hash matched but the string didn't — false positives caught by verification), and the total number of full-string verifications performed (every time the inner string compare ran, regardless of outcome).

```
def find_counting(
    text: str, pattern: str, base: int = 257, modulus: int = 1_000_000_007
) -> tuple[int | None, int, int]:
    """Instrumented Rabin-Karp.

    Returns (first match index or None, hash collisions detected,
    full-string verifications performed). A 'collision' is a hash match
    that fails string comparison - a false positive caught by the
    Las Vegas verification. A 'verification' is any full string equality
    check that ran, including the eventual confirmation of a real match.
    """
    n, m = len(text), len(pattern)
    if m == 0:
        return 0, 0, 0
    if m > n:
        return None, 0, 0
    pattern_hash = polynomial_hash(pattern, base, modulus)
    window_hash = polynomial_hash(text[:m], base, modulus)
    high_power = pow(base, m - 1, modulus)
    collisions = 0
    verifications = 0
```

```
    if window_hash == pattern_hash:
        verifications += 1
        if text[:m] == pattern:
            return 0, collisions, verifications
        collisions += 1
    for i in range(1, n - m + 1):
        window_hash = (
            (window_hash - ord(text[i - 1]) * high_power) * base
            + ord(text[i + m - 1])
        ) % modulus
        if window_hash == pattern_hash:
            verifications += 1
            if text[i : i + m] == pattern:
                return i, collisions, verifications
            collisions += 1
    return None, collisions, verifications
```

The counters tell you which path the algorithm took on every window. If `collisions` is zero, the hash function did all the work and every verification was a true match. If `collisions` is large, the modulus was too small — the algorithm was correct, but slow, because the slow path kept firing.

27.5 Running on the canonical text

Same input I've been carrying since chapter 23, same expected output.

```
from codex.strings.rabin_karp import find, find_all, find_counting

TEXT = "the rain in spain stays mainly in the plain"

idx = find(TEXT, "ain")
print(f"first match of 'ain': index {idx} - same answer as ch 23-26")

all_ain = find_all(TEXT, "ain")
print(f"all matches of 'ain': {all_ain}  (four occurrences)")

idx2, collisions, verifications = find_counting(TEXT, "ain")
print(f"\ninstrumented run:")
print(f"  first match: {idx2}")
print(f"  hash collisions detected: {collisions}")
print(f"  full-string verifications: {verifications}")
```

```
first match of 'ain': index 5 - same answer as ch 23-26
all matches of 'ain': [5, 14, 25, 40]  (four occurrences)

instrumented run:
  first match: 5
  hash collisions detected: 0
  full-string verifications: 1
```

Four matches at 5, 14, 25, 40 — the same indices that chapter 26's Boyer-Moore returned, the same indices KMP returned in chapter 25, the same indices brute force returned in chapter 23. The instrumentation tells you the interesting part: **zero hash collisions across the whole scan**. With $p = 10^9 + 7$ and only ~ 40 windows to consider, the expected number of false positives is $40/10^9 \approx 4 \times 10^{-8}$ — essentially impossible to observe by chance. Every verification you see fired on a true match, not a collision. The hash function did all the filtering; the string comparison only ran when it was about to confirm a real match.

27.6 When hashes collide

I want you to *see* a collision, because the algorithm's whole character depends on what happens when one fires. To make collisions actually happen I have to make them likely, which means picking a tiny modulus on purpose. With $p = 11$, the hash output is in $\{0, 1, \ldots, 10\}$ — eleven distinct values total. Any text longer than a handful of windows is guaranteed to produce some hash matches between unequal strings, by pigeonhole alone.

```
from codex.strings.rabin_karp import polynomial_hash, find_counting

# A tiny modulus forces collisions visible to the naked eye.
TINY_TEXT = "zzabczzbaa"
PATTERN = "baa"

# Show the per-window hashes by hand so you can see the collision happen.
print(f"pattern={PATTERN!r}  hash(pattern, 10, 11) = {polynomial_hash(PATTERN, 10, 11)}")
print(f"text={TINY_TEXT!r}\n")
print(f"{'i':>2}  {'window':<5}  {'hash':>4}  outcome")
print("-" * 40)
m = len(PATTERN)
p_hash = polynomial_hash(PATTERN, 10, 11)
for i in range(len(TINY_TEXT) - m + 1):
    win = TINY_TEXT[i:i + m]
    h = polynomial_hash(win, 10, 11)
    if h == p_hash and win == PATTERN:
        outcome = "MATCH (verified)"
    elif h == p_hash:
        outcome = "COLLISION (hash agrees, strings differ - Las Vegas catches it)"
    else:
        outcome = "filtered out by hash"
    print(f"{i:>2}  {win!r:<5}  {h:>4}  {outcome}")

idx, collisions, verifications = find_counting(TINY_TEXT, PATTERN, base=10, modulus=11)
print(f"\nfind_counting result: match at {idx}, "
      f"{collisions} collisions caught, {verifications} verifications total")
```

```
pattern='baa'  hash(pattern, 10, 11) = 10
text='zzabczzbaa'

 i  window  hash  outcome
----------------------------------------
 0  'zza'     9  filtered out by hash
 1  'zab'     2  filtered out by hash
 2  'abc'    10  COLLISION (hash agrees, strings differ - Las Vegas catches it)
 3  'bcz'     0  filtered out by hash
 4  'czz'     0  filtered out by hash
 5  'zzb'    10  COLLISION (hash agrees, strings differ - Las Vegas catches it)
 6  'zba'     0  filtered out by hash
 7  'baa'    10  MATCH (verified)

find_counting result: match at 7, 2 collisions caught, 3 verifications total
```

Two false positives fire — `'abc'` and `'zzb'` both hash to the same value as `'baa'` under this tiny modulus, even though none of the three are remotely equal as strings. Each time Rabin-Karp encounters one, it falls through to the string comparison, discovers the strings disagree, increments the collision counter, and keeps rolling. The final answer is still correct: the only match is at index 7, and the algorithm reports exactly that. The verifications counter is 3 (two collisions plus one real match); without the collisions it would have been 1.

Now re-run with the realistic modulus and watch the collisions disappear.

```python
from codex.strings.rabin_karp import find_counting

idx, collisions, verifications = find_counting(TINY_TEXT, PATTERN)  # default 10^9 + 7
print(f"with modulus = 10^9 + 7:")
print(f"  match at {idx}, {collisions} collisions, {verifications} verifications")
```

```
with modulus = 10^9 + 7:
  match at 7, 0 collisions, 1 verifications
```

Zero collisions. Same input, same correct answer, but the verifications counter dropped to 1 — only the true match triggered a string comparison. The algorithm is correct under both moduli; the *cost shape* is what changes. That's the pedagogical point I most want you to leave with: **the modulus controls only the probability of doing slow work, never the correctness of the answer.**

27.7 Picking the modulus

I want to make the probability claim concrete. Treat the polynomial hash as approximately uniform over $\{0, 1, \ldots, p-1\}$ — that's the *uniform hashing assumption* from chapter 12, applied here to substrings instead of dictionary keys. Then the probability that two random distinct strings happen to hash to the same value is about $1/p$. Across $n - m + 1 \approx n$ windows, the expected number of false-positive collisions is roughly n/p.

Plug in numbers. With $p = 10^9 + 7$ and a text of length $n = 10^6$, the expected collision count is $10^6/10^9 \approx 10^{-3}$. With $p = 11$ and a text of length 10, the expected collision count is $10/11 \approx 1$ — and you just watched two of them fire. The two regimes are wildly different in cost but identically correct.

This is what makes Rabin-Karp's *expected* running time $O(n + m)$ despite a worst case of $O(nm)$. The worst case happens when every window collides — for instance, if an adversary picks the pattern and the text knowing your modulus. With a fixed public modulus, an adversary can construct an input where every window of the text hashes to the same value as the pattern, forcing the verification on every shift. That regenerates the $O(nm)$ cost of brute force. The defenses are well-known: pick the modulus at random at program startup, or use a pair of independent hashes (double hashing — two different (b, p) pairs, require both to match before verifying), or use a cryptographic hash that's hard to invert. For most text-search applications none of this paranoia is necessary — natural text doesn't construct adversarial inputs by accident.

The third detail worth knowing is that **prime moduli matter** for uniformity. If p shares a common factor with b, the hash function's image is restricted to multiples of that factor, which collapses the effective output space. Choosing p prime, with $b < p$, sidesteps the issue entirely.

This whole chapter is the string-matching analog of a move you already saw in chapter 18 on treaps. The treap chapter's claim was that randomization buys you balance — the adversary can pick any input, but you pick the priorities, and the priorities (not the input) determine the tree's shape. Rabin-Karp's claim is the same shape: the adversary can pick any text and any pattern, but you pick the hash function's (b, p), and the hash function (not the input) determines how often you fall to the slow path. The adversary's worst case is constructible only against a *fixed* hash function; against a hash function chosen at runtime, the worst case becomes statistically unreachable.

27.8 The three questions, applied

Is it correct?

Two facts establish correctness. **First**, equal strings always produce equal hashes — the polynomial-hash formula is deterministic and depends only on the string's contents, so $s_1 = s_2$ implies $h(s_1) = h(s_2)$. That means a true match's window-hash will always equal the pattern-hash, and the algorithm will never *miss* a match. **Second**, the verification step compares strings character-by-character only when the hashes match, so any hash collision (where unequal strings happen to share a hash) is caught and rejected. That means the algorithm will never *report a false* match. Together: every match is found, every reported match is real. The rolling-hash recurrence is correctness-preserving by construction — algebraically, h_{i+1} as computed from h_i equals $h(\text{text}[i+1..i+m])$ as computed from scratch, modulo p — and I showed you that empirically in the rolling-hash trace above.

How efficient is it?

The preprocessing is $O(m)$ — one polynomial hash of the pattern and one of the initial window, plus an $O(\log m)$ modular exponentiation for the high-order coefficient. The main loop runs $n - m + 1$ iterations, and each iteration does $O(1)$ arithmetic to roll the hash plus, with probability $\approx 1/p$, an $O(m)$ string comparison.

The **expected** total work is $O(n + m)$ for any reasonable modulus, because the expected number of full-string verifications is $1+n/p$, contributing $O(m+nm/p) = O(m+n \cdot m/p)$, and for $p \gg n$ that second term is negligible. Bilingually: searching a megabyte text for a hundred-character pattern with $p = 10^9 + 7$ does about 10^6 rolls (cheap) and $\sim 10^{-3}$ expected verifications (essentially zero). The cost shape is $O(n)$ in practice, dominated by the rolling.

The **worst case** is $O(nm)$, achieved when every window collides — possible only on adversarial input against a fixed modulus. Randomizing the modulus at runtime makes the adversarial case statistically unreachable, the same way treaps make worst-case BST shapes statistically unreachable.

Space is $O(1)$ working memory — just the running hash, the high-power constant, and a couple of counters. There's no pattern table to build, no automaton to construct, no suffix array. That low memory footprint is part of why Rabin-Karp scales well to enormous texts and patterns.

Is it optimal?

For *single*-pattern search, Rabin-Karp's expected $O(n + m)$ matches KMP's worst-case $O(n + m)$ asymptotically. So neither is strictly better — KMP gives you a deterministic guarantee, Rabin-Karp gives you a smaller constant on natural input (no table to consult, just integer arithmetic) and the freedom to randomize away the worst case. The two algorithms live on different sides of a familiar trade-off: determinism vs randomization, worst-case vs expected-case, table-driven vs arithmetic-driven. For single-pattern matching neither is *optimal* in a strong sense — they meet the same asymptotic bound from different directions.

Where Rabin-Karp pulls strictly ahead is the **multi-pattern** case. If you want to search for k patterns simultaneously, KMP costs $O(k(n + m))$ — one scan per pattern, each with its own failure-function table. Rabin-Karp costs $O(n + km)$: hash all k patterns into a hash table once, do a single $O(n)$ rolling-hash scan over the text, and probe the hash table at each window. Every pattern that hashes to that window's hash gets verified. The win is asymptotic — Rabin-Karp turns the multiplier k in the dominant term into an additive — and it's the algorithm's real-world niche over the single-pattern alternatives.

27.9 What Rabin-Karp teaches you

Two lessons I want you to take from this chapter.

First, randomization buys you a different cost shape. Determinism in algorithm design always carries a worst case the adversary can construct. Randomization in algorithm design replaces the worst case with an expected case that holds *regardless* of the input, at the price of needing the random source to be hidden from the adversary. You've now seen this move twice: in chapter 18 it gave you balanced trees without a balance factor; in this chapter it gives you string matching without preprocessing tables. The pattern is the same — the algorithm makes a random choice that the adversary cannot anticipate, and the analysis shifts from "what's the worst input?" to "what's the average over my random choices, for any input?" That second question often has a much better answer.

Second, fingerprints turn equality into a cheap pre-test. The polynomial hash isn't an exact comparison — it's a *summary* that's much cheaper to compute than the thing it summarizes, with the property that the summary is identical iff the things are (probably) identical. Once you have a cheap-summary-with-rare-collisions, you can put it in front of any expensive equality check as a filter, and pay the expensive check only when the cheap one passes. The same pattern shows up in *content-defined chunking* for backup systems (one hash per content block, deduplicate identical blocks), in *similarity hashing* for near-duplicate document detection (locality-sensitive hashes, an extension of this idea), and in *plagiarism detection* (compare hashes of every k-word window between two documents). Rabin-Karp is the introductory case where you see the fingerprint trick at its cleanest.

27.10 Notes and further reading

Rabin and Karp's original paper is "Efficient randomized pattern-matching algorithms," published in *IBM Journal of Research and Development* 31(2):249–260 in 1987. The paper presents the algorithm as a randomized solution to multi-pattern matching, not single-pattern — that framing is right, and it's the application where Rabin-Karp clearly beats the alternatives. The single-pattern variant I built in this chapter is the textbook simplification; the original is closer in spirit to what `grep -f patterns.txt` does today, scanning once and checking against many fingerprints.

The polynomial-hash idea predates Rabin-Karp by a long way — *Rabin fingerprints* are essentially the same construction, used in network-deduplication systems (think rsync's rolling checksum, or content-defined chunking in backup software like *Borg* and *Restic*). The same arithmetic also underlies the *Karp-Rabin* approach to *substring matching with errors* and to *all-pairs longest common substring* algorithms. If the polynomial-hash trick interests you, follow the trail through content-defined chunking — it's the same algorithm with a different stopping criterion.

For plagiarism detection, the closest practical descendant is the *Manber-Myers winnowing* algorithm, which applies Rabin-Karp-style hashing to overlapping k-grams of two documents and counts matching fingerprints; commercial plagiarism checkers use variants of this with much larger k and additional locality-sensitive hashing on top.

CLRS covers Rabin-Karp in §32.2 with a presentation similar to mine, including the randomized-modulus discussion. Sedgewick and Wayne's *Algorithms* (4th ed.) §5.3 covers it as well, with a slight twist on the hash function. The Crochemore-Hancart-Lecroq reference I keep pointing at — *Algorithms on Strings*, Cambridge 2007 — has the deepest treatment, including the multi-pattern extension and the connection to q-gram indexing.

In the next chapter I'll change the lens one more time. Rabin-Karp's multi-pattern win comes from hashing patterns into a table; Aho-Corasick gets the same multi-pattern win deterministically, by building a single automaton that recognizes *all* patterns simultaneously, with $O(n + \sum m_i + z)$ worst-case time where z is the number of matches found — the algorithm that powers `fgrep`.

28 A trie that fails forward, on many patterns at once

Chapter 27 closed by pointing at the multi-pattern case as Rabin-Karp's real niche, and I told you the next chapter would show you how `grep -F` does it deterministically. I want to keep that promise now. The lens shifts one more time. Brute force, the DFA, KMP, Boyer-Moore, Rabin-Karp — five chapters of single-pattern matching, five different ways to ask "where does pattern P appear in text T?" In this chapter I'm going to ask a different question: **where does *any* of patterns $P_1, P_2, \ldots, P_k$ appear in T, in one pass over the text, finding every match of every pattern simultaneously?** That problem is what virus scanners solve, what intrusion-detection rulesets solve, what `grep -F` solves when you hand it a file of fixed strings. Running KMP k separate times answers the question correctly but pays $O(k(n+m))$ for the privilege. The Aho-Corasick algorithm answers it in essentially one scan of the text.

Trie plus failure links equals KMP's failure function generalized to a tree. Chapter 20 built the trie — one branch per character, the path from root to node spells the key. Chapter 25 built KMP's failure function — for each prefix of one pattern, how much of my recent progress is still salvageable on a mismatch. Aho-Corasick is what you get when you combine those two ideas: build a trie of all k patterns, then add a failure link to every node pointing at the deepest other node whose string is a proper suffix of this node's string. The trie tells you which patterns extend forward; the failure links tell you what to fall back to when extension fails. One scan over the text, one fail-link chase per character on average, every match of every pattern reported.

By the end of this chapter you'll have built that combined structure on the classical demo set `{he, she, his, hers}`, watched the BFS construction of the failure links produce non-trivial fall-backs you can verify by hand, scanned the text `"ushers"` and recovered all three matches that famously fire at nearly the same scan position, swept the canonical text from earlier Part IV chapters with three patterns at once for twelve matches in a single pass, and seen how the same matcher can be built once and reused across many texts the way a virus-scanner ruleset gets reused across every file on disk. The empower-promise is that once you have Aho-Corasick, multi-pattern matching stops being a separate algorithm — it's just the trie chapter and the KMP chapter clicked together.

28.1 What if you wanted to match a thousand patterns at once?

I want to start with the cost shape, because that's what makes the algorithm worth its complexity. Suppose I hand you a thousand virus signatures — short byte strings, average length 30 — and ask you to scan a gigabyte of disk for any of them. The naive thing you might try is to run KMP a thousand times. KMP's scan is $\Theta(n)$ in the text, and the preprocessing per pattern is $\Theta(m)$, so the total cost is $\Theta(k \cdot n + \sum m_i)$ — for the numbers above, about 10^{12} character reads, a thousand passes over the disk. That cost is *linear in the number of patterns*, and for a real scanner with tens of thousands of rules, the multiplier kills you.

What I want is a cost where the text is read *once*. Aho-Corasick gives me that. The build phase is $O(\sum m_i)$ — proportional to the total pattern size, not to the text — and the scan phase is $O(n + z)$ where z is the total number of matches reported. For the virus example, that's about 10^9 character reads for the scan plus the work of emitting any actual matches. A thousand-fold improvement, baked into the algorithm's structure, with no probability handwave like Rabin-Karp's expected case.

The trick — and you'll recognize this from chapter 25 — is to build a state machine that knows about all patterns simultaneously. Each state represents a prefix shared by some subset of the patterns; transitions extend the matched-so-far by one character; failure links handle the "I just mismatched, what's the longest still-salvageable suffix that's also a prefix of *some* pattern?" question. The DFA from chapter 24 answered that question for a single pattern with a two-dimensional table. KMP from chapter 25 compressed the table into a one-dimensional failure function. Aho-Corasick generalizes the failure function from a single pattern's prefixes (a chain) to many patterns' prefixes (a tree). Same idea, larger domain.

28.2 A trie of patterns

The skeleton is already in your hands. Chapter 20's trie stores keys character by character, one character per edge, and the path from the root to any node spells the prefix that node represents. That's what I need here — except I'm going to augment each node with two extra fields: a `fail` pointer (the failure link), and an `output` list (the patterns that finish at this node, including any reachable by walking fail-links — more on that in a couple of sections).

Here's the node class. I'm putting the `deque` import at the top of this same block because both `_ACNode` and the AC builder live in the same file, and `_build_failure_links` will need it in a moment.

```
from collections import deque


class _ACNode:
    """Internal trie node with Aho-Corasick augmentations."""

    def __init__(self) -> None:
        self.children: dict[str, "_ACNode"] = {}
        self.fail: "_ACNode | None" = None
        self.output: list[str] = []  # patterns ending at this node or via fail-chain
        self.depth: int = 0
```

The fields beyond `children` are what make this a *failure-aware* trie. The `fail` pointer will be filled in by a BFS during construction; it stays `None` until then. The `output` list starts empty; the trie-build phase will append a pattern to the node where that pattern ends, and the failure-link phase will merge in additional patterns reachable via fail-links. The `depth` field is bookkeeping — I'll use it to keep some of the prose tractable and to make the structure easier to inspect, even though the algorithm doesn't strictly need it.

Now the matcher class. The constructor takes the pattern list, drops any empty strings (they'd match at every position, which is a degenerate case I'll skip), and runs the two construction phases in order: build the trie, then walk the trie in BFS order to compute fail-links. That order matters and I'll come back to it.

```
class AhoCorasick:
    """Multi-pattern matcher. Build once with a list of patterns, then scan
    arbitrarily many texts. Each scan is a single O(n + total_output) pass.
    """

    def __init__(self, patterns: list[str]) -> None:
        self._root = _ACNode()
        self._patterns = [p for p in patterns if p]  # drop empty patterns
        self._build_trie()
        self._build_failure_links()
```

Two builders, called in sequence. Once they finish, the matcher is ready to scan any text via `find_all`, and I never have to touch the trie again.

The trie-build itself is exactly chapter 20's `insert` loop, repeated once per pattern, with `node.output.append(pat)` at the end of each insertion to record that this pattern terminates at this node. No `terminal` boolean is needed — a non-empty `output` list is the terminal marker, and a list lets multiple patterns end at the same node (a pathological-but-legal case: `{"abc", "abc"}` would put two copies of `"abc"` in the same list, and the scan would emit both).

```
    def _build_trie(self) -> None:
        for pat in self._patterns:
            node = self._root
            for c in pat:
                if c not in node.children:
                    child = _ACNode()
                    child.depth = node.depth + 1
                    node.children[c] = child
```

```
                node = node.children[c]
            node.output.append(pat)
```

For the demo set `{he, she, his, hers}` the resulting trie has nine nodes including the root. Two children of root, on `h` and `s`. Under `h`: an `e` child (giving `"he"` with output `["he"]`) which has a further `r` child, which has a further `s` child (giving `"hers"` with output `["hers"]`); and an `i` child, which has a further `s` child (giving `"his"` with output `["his"]`). Under `s`: an `h` child, which has an `e` child (giving `"she"` with output `["she"]`). Walk that in your head once — I'll lean on it for the next two sections.

28.3 Failure links: KMP's idea on a tree

Now the interesting part. The failure function from chapter 25, restated: for a single pattern P, $\pi[i]$ is the length of the longest proper prefix of $P[0..i]$ that is also a suffix of $P[0..i]$. I want to generalize that from "one pattern's prefixes" to "many patterns' prefixes," and the right way to express the generalization is in trie terms.

For any trie node v representing string s_v (i.e., the path from the root to v spells s_v), define **fail**(v) **as the deepest trie node** $u \neq v$ **such that** s_u **is a proper suffix of** s_v**.** If no such node exists (other than the root, which represents the empty string), then fail(v) is the root. The root's own fail-link is left as `None` and I treat it specially in the matcher.

The single-pattern case from chapter 25 falls out of this definition. For a pattern of length m, the trie is a chain of $m+1$ nodes (root, then one node per pattern character). The fail-link of the node at depth $i+1$ is the node at depth $\pi[i]$. Same numbers, different storage — the failure function packed into pointers instead of an array. That's the connection I want you to hold on to.

Let me trace fail-links by hand on `{he, she, his, hers}` so you can see the definition pay off. I'll write each node by its represented string for legibility.

- Root: `fail = None` (special case).
- Depth-1 nodes (`"h"` and `"s"`): their proper suffixes are just `""`, which is the root. So both fail to the root.
- Node `"sh"` (depth 2, under `s`): its proper suffixes are `""` and `"h"`. The deepest one that's a trie node is `"h"` (depth 1). So `fail("sh") = "h"`.
- Node `"he"` (depth 2, under `h`): proper suffixes are `""` and `"e"`. `"e"` is not a trie node — there's no edge from the root labeled `e`. So fail goes to root.
- Node `"hi"` (depth 2): proper suffixes `""` and `"i"`. `"i"` is not in the trie. Fail to root.
- Node `"she"` (depth 3): proper suffixes `""`, `"e"`, `"he"`. The deepest one that's in the trie is `"he"` (depth 2). So `fail("she") = "he"`.
- Node `"her"` (depth 3): proper suffixes `""`, `"r"`, `"er"`. None of those exist as trie nodes. Fail to root.
- Node `"his"` (depth 3): proper suffixes `""`, `"s"`, `"is"`. The deepest one in the trie is `"s"` (depth 1). So `fail("his") = "s"`.
- Node `"hers"` (depth 4): proper suffixes `""`, `"s"`, `"rs"`, `"ers"`. The only one in the trie is `"s"` (depth 1). So `fail("hers") = "s"`.

You can already feel the algorithm in those fail-links. The node `"she"` fails to `"he"` — meaning when the scan has read `"she"` and the next character disagrees with what `"she"` would extend to, the longest still-salvageable suffix is `"he"`, and the scan can continue from there. That's KMP's "fall back to the next-best candidate" move, generalized to a node that doesn't live on the same trie branch.

Now the construction. I want to compute fail-links in **BFS order** — level by level, shallow before deep. The reason is straightforward: a node's fail target is always strictly shallower than the node itself (because $s_{\text{fail}(v)}$ is a proper suffix of s_v, hence shorter), so by the time I need to compute fail(v), the fail-links of all shallower nodes are already known. BFS gives me that guarantee for free; a DFS would not.

Here's the construction.

```
    def _build_failure_links(self) -> None:
        # BFS so a node's fail link is computed only after its parent's is.
        queue: deque[_ACNode] = deque()
        for child in self._root.children.values():
            child.fail = self._root
            queue.append(child)
```

```
        while queue:
            current = queue.popleft()
            for c, child in current.children.items():
                # Walk fail-pointers from current looking for one with
                # a c-child, or hit the root.
                f = current.fail
                while f is not None and c not in f.children:
                    f = f.fail
                child.fail = self._root if f is None else f.children[c]
                # Merge fail-target's output into ours so a single output
                # check at scan time catches every pattern ending here or
                # via the fail-chain.
                child.output = child.output + (
                    child.fail.output if child.fail is not None else []
                )
                queue.append(child)
```

The structure is BFS with one twist. For each node `current` I pop, I look at every edge `(c, child)` leaving `current`, and I compute `child.fail` from `current.fail` — not from `current` itself. Why? Because `child` represents the string $s_{\text{current}} + c$, and its longest proper-suffix-that's-in-the-trie is found by extending some proper suffix of s_{current} by the character c. The proper suffixes of s_{current}, in decreasing length, are exactly the chain `current.fail`, `current.fail.fail`, and so on up to the root. I walk that chain, looking for the first node that has a `c`-child; that `c`-child is `child.fail`. If the chain runs all the way to `None` without finding one, `child.fail` is the root.

The inner `while` loop is the failure-chain chase. It looks similar to the chain chase in KMP's `failure_function` (chapter 25), and it works for the same amortized-cost reason: the total chain work across the whole BFS is bounded by the total depth of the trie, which is $O(\sum m_i)$. I'll come back to that in the analysis.

After computing `child.fail`, I do one more thing: I merge `child.fail.output` into `child.output`. That merge deserves its own section.

28.4 Output links: don't miss matches buried inside matches

Here's a subtlety that bites if you don't handle it. Consider what happens when the scan, reading `"ushers"`, reaches the node for `"she"`. That node's own `output` is `["she"]` — so far, so good. But `"she"` *contains* `"he"` as a suffix: the trailing two characters of `"she"` spell `"he"`, which is also a stored pattern. If I emit only `"she"` at this scan position, I miss `"he"` — which is wrong, because `"he"` does occur in the text, ending at the same position where `"she"` ends.

The fix follows from the fail-link structure. The fail-link of `"she"` is `"he"`, by the definition I gave you above. So the chain of fail-links from any node enumerates all the *suffixes of* s_v that are themselves trie nodes — and any such suffix that's a stored pattern is a match that fires at the same scan position. Concretely: at scan position i, if I'm at node v, I should emit every pattern in the `output` list of v, *plus* every pattern in the `output` list of fail(v), *plus* every pattern in the `output` list of fail(fail(v)), and so on up the chain.

I could chase the fail-chain at scan time for every text character, but that costs more than I want — it would turn the per-character work from amortized $O(1)$ into amortized $O(\text{depth})$. The cleaner move is to do the chain-chase work *once*, during the BFS construction, and pre-compute the merged output list per node. That's what the line `child.output = child.output + (child.fail.output if child.fail is not None else [])` does. By the time the BFS visits `child`, the BFS has already finished visiting `child.fail` (because fail-targets are shallower and BFS does shallow first), so `child.fail.output` is already the fully merged list of *its* node's patterns plus all patterns reachable from it via further fail-chains. Concatenating with `child`'s own already-recorded patterns gives the full merged set for `child` in one step.

For the demo set, this means:

- `"he"` after merging: `["he"]` (own patterns) + `[]` (root's output) = `["he"]`. No change.
- `"she"` after merging: `["she"]` + `["he"]` (the fail-target's merged output) = `["she", "he"]`. **This is the magic move** — when the scan lands at `"she"`, the merged output lets me emit both matches in one pass over a length-2 list, no recursive fail-chase needed.

- `"his"` after merging: `["his"] + []` (s-node has no patterns) = `["his"]`.
- `"hers"` after merging: `["hers"] + []` (s-node) = `["hers"]`.

That one extra concatenation per node during the BFS — moving the chain-chase work to build time — is what keeps the scan's per-character work at amortized $O(1)$. The trade is a bit more memory at build time (each node's `output` list may contain patterns "borrowed" from ancestors via fail-links), but the gain is a cleaner scan loop. Worth it.

28.5 The single-pass scan

Now I have the structure: a trie of patterns, fail-links computed in BFS order, output lists pre-merged. The scan is short.

```
    def find_all(self, text: str) -> list[tuple[int, str]]:
        """Return (start_index, pattern) for every match of every pattern in
        text. Matches of overlapping patterns are all reported.
        """
        matches: list[tuple[int, str]] = []
        node = self._root
        for i, c in enumerate(text):
            # Follow fail-links looking for a node with a c-child, or hit root.
            while node is not self._root and c not in node.children:
                node = node.fail if node.fail is not None else self._root
            if c in node.children:
                node = node.children[c]
            # Emit all patterns ending at this node (or reachable via fail-chain,
            # already merged into output by _build_failure_links).
            for pat in node.output:
                matches.append((i - len(pat) + 1, pat))
        return matches
```

I keep a single state variable `node`, starting at the root. For each text character `c`, I try to extend by walking to `node.children[c]`. If that child doesn't exist, I follow the fail-link of `node` and try again — exactly the same fall-back move KMP's matching loop makes when it does `j = pi[j - 1]`. I keep falling back until I find an ancestor (in the fail-link sense) with a `c`-child, or until I'm at the root and there's no `c`-child there either (in which case the character contributes nothing and the scan moves on). Then I emit every pattern in `node.output`, which by construction contains every pattern that ends at the new node *or* via any fail-link suffix.

Let me trace the scan on `"ushers"` character by character. I'll write `node` by its represented string.

- Start: `node = root`.
- `i = 0`, `c = 'u'`: root has no `u`-child. Stay at root. Output: nothing.
- `i = 1`, `c = 's'`: root has an `s`-child. Move to `"s"`. Output: nothing (`"s"` has no patterns).
- `i = 2`, `c = 'h'`: `"s"` has an `h`-child. Move to `"sh"`. Output: nothing.
- `i = 3`, `c = 'e'`: `"sh"` has an `e`-child. Move to `"she"`. Output: `["she", "he"]`. Emit two matches: `"she"` started at `3 - 3 + 1 = 1`, and `"he"` started at `3 - 2 + 1 = 2`. Both fire at this same scan step.
- `i = 4`, `c = 'r'`: `"she"` has no `r`-child. Follow fail-link to `"he"`. `"he"` has an `r`-child. Move to `"her"`. Output: nothing.
- `i = 5`, `c = 's'`: `"her"` has an `s`-child. Move to `"hers"`. Output: `["hers"]`. Emit `"hers"` started at `5 - 4 + 1 = 2`.

Final match list: `[(1, "she"), (2, "he"), (2, "hers")]`. Three matches across five text characters past the leading `'u'`, found in a single left-to-right pass, with one fail-link chase total (at the `'r'` step). That's the algorithm doing exactly what I promised: every match of every pattern, in one pass.

Time to actually run it.

```
from codex.strings.aho_corasick import AhoCorasick

ac = AhoCorasick(["he", "she", "his", "hers"])
matches = ac.find_all("ushers")

print("matches on 'ushers' with patterns {he, she, his, hers}:")
for start, pat in matches:
    print(f"  {pat!r:>8} at index {start}")
```

```
matches on 'ushers' with patterns {he, she, his, hers}:
     'she' at index 1
      'he' at index 2
    'hers' at index 2
```

Three matches — `"she"` at 1, `"he"` at 2, `"hers"` at 2 — exactly the trace above. The two matches at index 2 are what makes this example famously pedagogical: a single scan position is responsible for two distinct pattern hits (`"he"` and `"hers"` both have their starts at the same text index, though they end at different positions). That's the multi-pattern character of the algorithm visible at the smallest possible scale.

28.6 Running on the canonical text

Now the same text I've been carrying since chapter 23, but with three patterns at once instead of one.

```
from codex.strings.aho_corasick import AhoCorasick

TEXT = "the rain in spain stays mainly in the plain"
ac = AhoCorasick(["ain", "in", "the"])
matches = ac.find_all(TEXT)

print(f"text: {TEXT!r}\n")
print(f"total matches found in one pass: {len(matches)}\n")

# Group by pattern so the structure is legible
by_pat: dict[str, list[int]] = {}
for start, pat in matches:
    by_pat.setdefault(pat, []).append(start)

for pat in ["the", "ain", "in"]:
    starts = sorted(by_pat.get(pat, []))
    print(f"  {pat!r:>5} found at: {starts}  ({len(starts)} occurrences)")
```

```
text: 'the rain in spain stays mainly in the plain'

total matches found in one pass: 12

  'the' found at: [0, 34]  (2 occurrences)
  'ain' found at: [5, 14, 25, 40]  (4 occurrences)
   'in' found at: [6, 9, 15, 26, 31, 41]  (6 occurrences)
```

Twelve matches across the three patterns: `"the"` at 0 and 34, `"ain"` at 5, 14, 25, and 40, `"in"` at 6, 9, 15, 26, 31, and 41. Every one of them found in a single left-to-right walk over the 43 text characters. Notice the overlap structure — at index 5 the text reads `"...rain..."`, which contains both `"ain"` (starting at 5) and `"in"` (starting at 6). Aho-Corasick reports both, with no special handling for overlap, because each match is emitted at the scan step where it *ends*, and the two matches end at different positions (7 and 7 respectively — actually the same position, since `"rain"` ends at 8 and the two patterns both end at index 7). The point is: the merged `output` lists do the work, and overlaps are reported as a side effect of the structure, not as an explicit feature.

Compare this to what KMP would have done: three separate scans of the text, one per pattern, $3 \cdot 43 = 129$ character reads minimum. Aho-Corasick did 43 character reads (plus a small constant amount of fail-link chasing). For three patterns the gain is modest; for three thousand the gain is dramatic, and the algorithm's shape stays the same.

28.7 Build once, scan many

The build phase produces a matcher object. That object holds the trie, the fail-links, and the merged output lists — everything needed to scan. Nothing about the scan loop modifies any of that state (the only mutable variable is the local `node`, which gets reset to root at the start of each scan via the loop initializer). So one `AhoCorasick` instance can scan any number of texts, paying the build cost exactly once.

This is the operational mode that matters in practice. A virus scanner builds its matcher from the signature database at startup, then scans every file on disk against the *same* matcher. A content filter compiles its rule set once and applies the matcher to every message that arrives. An intrusion-detection system builds the matcher from its alert ruleset at boot and scans every packet's payload through it. The build is the slow part, and you do it once; the scans are the fast part, and you do them millions of times.

```
from codex.strings.aho_corasick import AhoCorasick

# Build the matcher once for a small set of "signatures"
ac = AhoCorasick(["ain", "in", "the"])

# Now scan a batch of independent texts with the same matcher
texts = [
    "the rain in spain",
    "mainly in the plain",
    "the plain truth in the rain",
    "nothing matches here at all",
    "the the the",
]

print("build once, scan many:\n")
for text in texts:
    matches = ac.find_all(text)
    summary = ", ".join(f"{pat}@{i}" for i, pat in matches) if matches else "no matches"
    print(f"  {text!r:<35}  ->  {summary}")
```

```
build once, scan many:

  'the rain in spain'                 ->  the@0, ain@5, in@6, in@9, ain@14, in@15
  'mainly in the plain'               ->  ain@1, in@2, in@7, the@10, ain@16, in@17
  'the plain truth in the rain'       ->  the@0, ain@6, in@7, in@16, the@19, ain@24, in@25
  'nothing matches here at all'       ->  in@4
  'the the the'                       ->  the@0, the@4, the@8
```

Five texts, one matcher, five independent scans. The matcher's internal trie and fail-links are untouched between calls — only the local scan state (`node`, `matches`, `i`) is created fresh each time. This is the streaming use case Aho-Corasick was designed for, and the reason `grep -F` (with the `-F` for "fixed strings") uses it: you compile the pattern set once, then scan stdin (or a file, or a stream) against it without rebuilding.

If you want to see the asymmetry in the cost shape directly: building the matcher does $\Theta(\sum |p_i|)$ work — for the three short patterns above, that's about 8 character-worth of trie nodes plus a comparable amount of fail-link work. Scanning a 35-character text does $\Theta(35 + z)$ work, where z is the (small) number of matches emitted. Run the same matcher over a billion-character text and the scan is $\Theta(10^9 + z)$ — the build cost is amortized across every text you scan, and the per-text cost is linear in just that text's length.

28.8 The three questions, applied

Is it correct?

Two invariants together establish correctness. The first is on the construction: **after `_build_failure_links` returns, every node v in the trie has fail(v) pointing at the deepest trie node $u \neq v$ such that s_u is a proper suffix of s_v (or at the root if no such node exists), and v.output contains every pattern that ends at v or at any node reachable from v by following fail-links.** That invariant holds by induction on the BFS level. At level 1 (root's children), each child's fail-link is set to the root explicitly, and the merge step is a no-op because the root has empty output. At level > 1, the construction reads its parent's already-correct fail-link chain to find the longest matching suffix, and merges the fail-target's already-correct merged output into its own. BFS order is what makes the induction valid: a node is visited only after all its fail-link targets (which are strictly shallower) have been visited.

The second invariant is on the scan: **at the top of each iteration of the `for i` loop in `find_all`, the state `node` is the trie node v such that s_v is the longest suffix of $T[0..i-1]$ that is a prefix of *some* pattern in the trie.** That's the multi-pattern generalization of KMP's "longest pattern-prefix that's a suffix of the text scanned so far" invariant. The `while` loop's fall-back maintains the invariant across a character that can't extend the current state; the children-lookup-and-move maintains it across a character that can. When `node.output` is non-empty, every pattern in that list is a suffix of s_v (by the first invariant), which means every pattern in that list is also a suffix of $T[0..i]$ — i.e., a match ending at position i. Emitting all of them is correct.

A pattern that the algorithm *misses* would have to be a suffix of $T[0..i]$ that isn't in the merged output list at the state reached after processing position i. The first invariant rules that out: the merged output at v includes every pattern that's a suffix of s_v, and s_v by the second invariant *is* the longest such suffix, so every shorter pattern-suffix is a suffix of s_v too and is in the fail-chain. Nothing is missed.

How efficient is it?

The build phase decomposes into two parts. `_build_trie` walks each pattern once, doing $O(1)$ work per character (dictionary lookup, possible insertion). Total: $O(\sum |p_i|)$.

`_build_failure_links` does a BFS over the trie, and for each node processes each outgoing edge. The work per edge is dominated by the inner `while` loop that chases fail-pointers. The amortized argument is identical to KMP's: the total length of all fail-chain chases across the whole BFS is bounded by the total depth of the trie, which is $O(\sum |p_i|)$. Each chase either drops the failure-pointer to a strictly shallower node (limited charges) or terminates by finding a `c`-child (also limited). So the BFS total work is $O(\sum |p_i|)$.

The merge step `child.output = child.output + child.fail.output` is potentially expensive — in the worst case where many patterns share a long common suffix, each node's merged output could be $O(k)$ long, blowing the build cost up to $O(k \cdot \sum |p_i|)$. In practice this rarely happens; for English text with no contrived suffix sharing, output lists stay near constant size and the build is effectively linear in the total pattern length. A more memory-efficient variant stores an `output_link` pointer per node and walks it at scan time; I've used the merge-at-build approach here because it makes the scan code dramatically simpler at a modest memory cost.

The scan phase reads each text character exactly once. At each character, it does $O(1)$ amortized fail-link work (same amortized argument as KMP — the state `node` can only fall back as many times across the whole scan as it has previously advanced, and it advances at most n times) plus the work of emitting any matches. So the scan is $O(n + z)$ where z is the total number of matches reported.

Bilingually: scanning a megabyte of text against a thousand 30-character virus signatures costs about $10^6 + 30 \cdot 1000 = 1.03 \times 10^6$ character-equivalent operations for the build-plus-scan, plus however many matches actually fire. Compare with KMP run a thousand times: $10^3 \cdot 10^6 = 10^9$ character reads. A three-orders-of-magnitude improvement, baked into the asymptotic shape.

Space is $O(\sum |p_i|)$ for the trie plus the fail-links, with each node a constant number of fields plus its children dictionary. The merge variant adds up to $O(k \cdot \text{avg-pattern-length})$ in the worst case for the merged output lists — usually much less.

Is it optimal?

For multi-pattern exact matching with k patterns of total length M in a text of length n reporting z matches, the lower bound is $\Omega(n + M + z)$: every text character must be examined to be sure no pattern starts there, every pattern character must be examined to encode it into the matcher, and every reported match must be emitted. Aho-Corasick hits $O(n + M + z)$ when implemented carefully (a `goto`-and-fail formulation that avoids the per-node `output` merge can hit the bound exactly), and my implementation here hits $O(n + M + z)$ in the typical case and slightly worse in worst-case pathological pattern sets.

For *single*-pattern matching ($k = 1$), Aho-Corasick reduces to KMP — the trie collapses to a chain, the fail-links collapse to the failure function, and the scan loop becomes KMP's matching loop. So Aho-Corasick is at least as good as KMP, and strictly better when $k > 1$ because KMP doesn't share structure across patterns. Rabin-Karp's expected $O(n + km)$ for multi-pattern matching is the closest competitor; Aho-Corasick beats it deterministically and has no notion of a worst case driven by adversarial input.

The Commentz-Walter algorithm extends Aho-Corasick with Boyer-Moore-style skipping — it scans windows right-to-left and can skip multiple characters per window when no pattern's suffix is found. That's a different cost shape (sublinear in good cases, worse in bad ones), and for short pattern sets it can beat Aho-Corasick by a constant factor, but it doesn't change the asymptotic bound.

28.9 What Aho-Corasick teaches you

Two lessons I want you to take from this chapter, and one observation about where the chapter sits in Part IV.

First, two ideas from earlier chapters can compose without distortion. The trie from chapter 20 wasn't designed with string matching in mind — it was designed to support prefix queries on a dictionary. KMP's failure function from chapter 25 wasn't designed for trees — it was designed for a single pattern's prefix chain. Aho-Corasick takes both and adds *one* new field per trie node (the fail-link), with the same definition KMP's failure function had, just applied to many patterns' worth of prefixes at once. Nothing about either source idea has to bend. The trie's path-encodes-key invariant from chapter 20 still holds; the fail-link's "deepest proper-suffix node" definition is the natural generalization of KMP's $\pi[i]$. When two algorithms compose this cleanly, you should suspect there's an underlying unity, and in this case there is — both algorithms are about *suffix-prefix overlap*, and a trie is the right shape for storing many such overlaps at once.

Second, building structure once to enable cheap repeated queries is one of the book's most reliable wins. You've seen this pattern in the hash table from chapter 12 (build once, look up many), in the segment tree from chapter 21 (build once, range-query many), in KMP's failure function from chapter 25 (build once, scan many), and now in Aho-Corasick's full matcher (build once, scan many texts). The shape is always the same: a preprocessing phase whose cost is proportional to the static data, then an answer phase whose cost is proportional to just the current query. When you can amortize a fixed build cost across many queries, the per-query cost can drop to its information-theoretic minimum, and the structure pays for itself many times over.

The chapter sits at the end of Part IV's pattern-matching arc. Chapter 23 gave you brute force as the baseline. Chapter 24 built the DFA — preprocessed match information into a $\Theta(m \cdot |\Sigma|)$ table for a single pattern. Chapter 25 compressed the DFA into KMP's $\Theta(m)$ failure function. Chapter 26 introduced Boyer-Moore's sublinear best case via right-to-left scanning. Chapter 27 gave you Rabin-Karp's randomized $O(n + m)$ expected case. This chapter, Aho-Corasick, generalizes KMP from one pattern to many — and the resulting algorithm is what `grep -F` ships, what virus scanners run, what intrusion-detection systems use to scan packet payloads against rule databases. The arc has been one of progressive sophistication, and the destination is a structure that does multi-pattern matching in essentially the best shape the problem admits.

28.10 Notes and further reading

The algorithm was introduced in Alfred Aho and Margaret Corasick's 1975 paper "Efficient string matching: An aid to bibliographic search," published in *Communications of the ACM* 18(6):333–340. The original motivation was bibliographic search — indexing a corpus of references by keywords pulled from a controlled vocabulary — and the paper presents the algorithm as a `goto`-function and `failure`-function pair, slightly different in surface form from the trie-and-fail-link formulation I used here but equivalent in substance. Aho and Corasick's paper also describes the *output function* explicitly, which is what my pre-merged `output` list does at scan time. Worth

reading the original if you want to see how the same data structure can be presented in two textbook-friendly forms.

CLRS doesn't cover Aho-Corasick. Sedgewick and Wayne's *Algorithms* (4th ed.) mentions it in passing in §5.3 but defers the details. The modern reference is Maxime Crochemore, Christophe Hancart, and Thierry Lecroq's *Algorithms on Strings* (Cambridge University Press, 2007), which treats the algorithm in chapter 3 alongside KMP, Boyer-Moore, and the suffix-tree extensions, and explicitly draws out the structural connection I've been pointing at — that Aho-Corasick is KMP-on-a-trie.

The Commentz-Walter algorithm — Beate Commentz-Walter's 1979 paper "A string matching algorithm fast on the average," *Proceedings of ICALP '79*, LNCS 71, pp. 118–132 — extends Aho-Corasick with Boyer-Moore-style right-to-left window skipping. It's the multi-pattern analog of Boyer-Moore the way Aho-Corasick is the multi-pattern analog of KMP. For short pattern sets and large alphabets, Commentz-Walter often beats Aho-Corasick by reading fewer text characters in the average case. For large pattern sets it falls back to roughly Aho-Corasick performance because the skip distances shrink.

Applications I've alluded to throughout the chapter: `fgrep` and `grep -F` (the F is for "fixed strings") have used Aho-Corasick or variants since the late 1970s. Network intrusion detection systems like Snort and Suricata run thousands of signature rules through an Aho-Corasick matcher on every packet payload. The Yara malware-detection language compiles its rule database into an Aho-Corasick automaton before scanning files. Bibliographic and full-text indexing systems use it to find keyword occurrences in large document corpora. Bioinformatics tools use it for fixed-pattern DNA motif scanning (though for approximate or pattern-with-wildcard matching, more sophisticated structures take over). Plagiarism-detection software uses it for k-gram exact-match overlap scoring, often as a fast first-pass filter before a more expensive similarity computation.

For the connection to automata theory more broadly: Aho-Corasick can be viewed as the *minimum DFA* recognizing the set of strings containing any pattern as a substring. Building it explicitly as a DFA (rather than as a trie with fail-links) takes $\Theta(\sum |p_i| \cdot |\Sigma|)$ space — back to the chapter 24 trade-off — but allows true $O(1)$-per-character scanning with no fail-link chases. Most production implementations choose the trie-with-fail-links form because the asymptotic amortized cost is the same and the memory savings are substantial.

In the next chapter I'll change the question one more time. Every algorithm in Part IV so far has been about scanning a text *given* the patterns. The next chapter — suffix arrays — flips the setup: preprocess the *text* once into a sorted array of all its suffixes, then answer pattern queries in $O(m \log n)$ each, regardless of the pattern. That's the algorithm that powers full-text search, biological sequence alignment, and Burrows-Wheeler compression — and it's the door into the suffix-structures family that closes out Part IV.

29 Indexing every substring with a sorted array

Chapter 28 closed by promising to change the question entirely. Every algorithm in Part IV up to that point answered "find these patterns in this text" — the patterns came first, fixed and known, and the text streamed past them. I want to flip the setup now. Preprocess the *text* once, build an index, and then answer pattern queries against that index — any pattern, as many as you like, each query much cheaper than re-scanning. That's the move full-text search engines make, the move biological-sequence aligners make, the move the Burrows-Wheeler transform makes before it compresses anything. The structure that does it, in its cleanest form, is the **suffix array**.

The sorted list of every suffix's starting position — every substring of the text is a prefix of some suffix. That's the entire structural insight, and once it lands the rest of the chapter is mechanics. A substring of T starting at position i with length m is just the first m characters of the suffix $T[i{:}]$. So if I have every suffix sorted lexicographically, any pattern occurrences cluster as a contiguous range in that sorted list — every suffix whose first m characters spell the pattern sits together — and a pair of binary searches pulls the range out in $O(m \log n)$ time per query. Indexing reduces to sorting suffixes; pattern search reduces to binary search.

By the end of this chapter you'll have built the suffix array two ways — once with an explicit $O(n^2 \log n)$ sort, then again with the prefix-doubling trick that knocks it down to $O(n \log^2 n)$ — paired it with Kasai's beautiful $O(n)$ algorithm for the longest-common-prefix array, used it to answer pattern queries on the canonical text from earlier chapters in $O(m \log n)$ per query, and applied the LCP array to find the longest repeated substring of a text in linear time after the array is built. The empower-promise is that by the chapter's end the phrase "preprocess the text into a suffix array" stops being an opaque move from a research paper and becomes a small, explicit chain of code blocks you've watched run on `"banana"`.

29.1 Every substring is a prefix of some suffix

I want to start where the suffix array starts: at the observation that every substring of a text T has a name. Pick any substring; it begins at some position i and ends at some position j. What you're looking at is $T[i : j]$, which is the same as the first $j - i$ characters of $T[i :]$ — the suffix that starts at position i. So if you write down every suffix of T (there are exactly n of them, one per starting position), every substring you could possibly care about is a *prefix* of one of those suffixes.

That observation has more force than it sounds at first. Pattern matching asks "does pattern P appear in T, and if so where?" Rephrase it: "does P appear as a *prefix* of any suffix of T, and if so which suffixes?" Same question, different shape — and the new shape is one that an ordering can help with.

Sort the suffixes lexicographically. Now suffixes that share a common prefix sit next to each other in the sorted order, because lex order's first move is to compare the first character. Suffixes that share two characters of common prefix sit even closer together. And suffixes that all start with the same length-m string P form a *contiguous block* in the sorted list — they're exactly the suffixes whose first m characters spell P, and the lex order can't intersperse them with anything else, because anything else either has an earlier-character difference and falls outside the block on one side, or has a later-character difference within the prefix and the lex order separates them.

So if I had the sorted list, finding all matches of P in T would be: binary-search for the leftmost suffix whose first m characters are $\geq P$, then binary-search for the leftmost suffix whose first m characters are $> P$, and the suffixes between those two indices are exactly the matches. Each binary search does $O(\log n)$ comparisons, and each comparison is at most m characters of substring compare, so the whole query is $O(m \log n)$. For a text of length 10^9 and a pattern of length 30, that's about $30 \cdot 30 = 900$ character reads per query, no matter how many matches there are.

The suffix array itself doesn't store the suffixes — that would cost $O(n^2)$ memory, since the suffixes of a length-n text have total length $n(n+1)/2$. It stores something much cheaper: **the starting positions of the suffixes, in the order the suffixes themselves would be in if sorted.** A single array of n integers. For `"banana"`,

the suffix array is $[5, 3, 1, 0, 4, 2]$ — the suffixes starting at those positions, read in that order, are `"a"`, `"ana"`, `"anana"`, `"banana"`, `"na"`, `"nana"`, which is exactly the lex-sorted list of all six suffixes.

That's the data structure. The chapter from here is two questions: how do I build the array fast (the suffixes themselves are $O(n^2)$ to write down, so I can't afford to materialize them), and what else can I do with it once I have it?

29.2 The naive sort

The first version is the one you'd write if you hadn't thought about cost yet. Generate every suffix, hand them to Python's sort, and read off the starting positions.

```
def suffix_array_naive(text: str) -> list[int]:
    """Build the suffix array by explicit sort of every suffix.
    O(n^2 log n) time, O(n^2) extra memory. Clear; not for large inputs.
    """
    n = len(text)
    indexed = sorted(range(n), key=lambda i: text[i:])
    return indexed
```

The whole function is one `sorted` call with a key function. `range(n)` is the list of starting positions; the key `lambda i: text[i:]` tells Python's sort to order positions by the suffix that starts at each one. Python's `sorted` returns a new list, and the result is the suffix array.

Let me run it on `"banana"` and walk through what comes out.

```
from codex.strings.suffix_array import suffix_array_naive

text = "banana"
sa = suffix_array_naive(text)

print(f"text     = {text!r}")
print(f"suffix array = {sa}")
print()
print("sorted suffixes:")
for i in sa:
    print(f"  start={i}  suffix={text[i:]!r}")
```

```
text     = 'banana'
suffix array = [5, 3, 1, 0, 4, 2]

sorted suffixes:
  start=5  suffix='a'
  start=3  suffix='ana'
  start=1  suffix='anana'
  start=0  suffix='banana'
  start=4  suffix='na'
  start=2  suffix='nana'
```

Six positions, six suffixes, sorted lex. Position 5 first (the bare `"a"`), then 3 (`"ana"`), then 1 (`"anana"`), then 0 (`"banana"`), then 4 (`"na"`), then 2 (`"nana"`). The `a`-starting suffixes cluster at the top because `a < b < n`; within the `a` block, the shorter suffix wins because lex comparison treats end-of-string as smaller than any character.

Now the cost. Python's sort is `O(n \log n)` comparisons. Each comparison is a string comparison between two suffixes, which in the worst case scans both suffixes to the end — $O(n)$ characters. So the total work is $O(n^2 \log n)$. For a text of length 10^6, that's about $10^6 \cdot 20 \cdot 10^6 = 2 \cdot 10^{13}$ character comparisons — completely impractical. The memory cost is worse: the key function materializes a copy of every suffix as a separate string object, and the suffixes have total length $n(n+1)/2 = O(n^2)$, so the sort needs $O(n^2)$ extra memory. A million-character text would want a terabyte of suffix storage. Naive is the right word.

I'll keep `suffix_array_naive` around as a clarity check — I'll use it later to verify that the fast version produces the same array on a real input — but I won't run it on anything larger than `"banana"`-sized inputs.

29.3 Manber-Myers and the doubling trick

The killer idea — Udi Manber and Gene Myers, 1993 — is that I don't have to compare suffixes character by character. Instead, I assign each suffix a **rank** based on how its first k characters compare to every other suffix's first k characters, and I increase k by doubling between iterations.

The mechanic that makes this work: if I already know the rank of every suffix based on its first k characters, then I can compute the rank based on the first $2k$ characters by sorting the suffixes by the *pair* $(\text{rank}[i], \text{rank}[i+k])$ — the rank of my own first k characters, followed by the rank of the next k characters, which is the rank of the suffix that starts at position $i+k$. Pairs are cheap to compare: two integer comparisons, $O(1)$ apiece. And k doubles each iteration, so after $\lceil \log_2 n \rceil$ doublings, every suffix's rank reflects n characters' worth of order, which is the entire suffix. At that point the rank order *is* the lex order, and I'm done.

The total work is $O(\log n)$ sorts. Each sort is over n items with $O(1)$ comparisons, so each sort is $O(n \log n)$ using Python's timsort. Total: $O(n \log^2 n)$. The proper Manber-Myers algorithm uses radix sort on the pair keys for $O(n)$ per sort and $O(n \log n)$ total — I won't implement that; I'll let timsort eat the extra log factor and gain readability.

The initial rank — what I use at $k = 1$ — is just each suffix's first character. Two suffixes that start with the same character get the same rank; two that start with different characters get ranks ordered by the character order. The cleanest way to bootstrap is to set `rank[i] = ord(text[i])` directly — that's already a consistent ranking by first-character order, even if the rank values aren't dense integers like $0, 1, 2, \ldots$. The doubling step doesn't care about density; it only cares about the *order* the ranks induce.

Here's the function. The body is one outer loop over k values, with two phases inside: sort by the pair key, then recompute ranks.

```
def suffix_array(text: str) -> list[int]:
    """Build the suffix array via prefix-doubling (Manber-Myers style).
    O(n log^2 n) using Python's timsort; the O(n log n) Manber-Myers
    proper would use radix sort instead - mentioned but not implemented.
    """
    n = len(text)
    if n == 0:
        return []
    sa = list(range(n))
    rank = [ord(c) for c in text]
    k = 1
    while True:
        def key(i: int) -> tuple[int, int]:
            return (rank[i], rank[i + k] if i + k < n else -1)
        sa.sort(key=key)
        new_rank = [0] * n
        new_rank[sa[0]] = 0
        for j in range(1, n):
            new_rank[sa[j]] = new_rank[sa[j - 1]]
            if key(sa[j]) != key(sa[j - 1]):
                new_rank[sa[j]] += 1
        rank = new_rank
        if rank[sa[n - 1]] == n - 1:
            break
        k *= 2
        if k >= n:
            break
    return sa
```

A few things are worth pointing at. The sentinel `-1` in the `key` function handles suffixes that fall off the end — a suffix starting at position i with $i + k \geq n$ has no second half, and I want it to sort *before* any suffix that

has a real character there. Using `-1` is one easy way to get that behavior, since any real `rank` value will be a non-negative integer. The early-exit condition `rank[sa[n - 1]] == n - 1` checks whether all ranks have become distinct (the largest rank equals $n-1$, which happens only when every suffix has its own unique rank); when that condition fires, no further doubling can change the order and I'm done.

Let me trace the algorithm on `"banana"` so the doubling is concrete. Initial ranks are the character codes: `[ord('b'), ord('a'), ord('n'), ord('a'), ord('n'), ord('a')]` = `[98, 97, 110, 97, 110, 97]`.

At $k = 1$, the pair keys $(\text{rank}[i], \text{rank}[i+1])$ are: position 0 $\to (98, 97)$, position 1 $\to (97, 110)$, position 2 $\to (110, 97)$, position 3 $\to (97, 110)$, position 4 $\to (110, 97)$, position 5 $\to (97, -1)$. Sorting positions by these keys: 5 $(97, -1)$, then 1 $(97, 110)$, then 3 $(97, 110)$, then 0 $(98, 97)$, then 2 $(110, 97)$, then 4 $(110, 97)$. So `sa` after the first sort is $[5, 1, 3, 0, 2, 4]$. Recompute ranks: position 5 gets rank 0; positions 1 and 3 have equal keys to each other but different from 5's, so they both get rank 1; position 0 has a strictly larger key, rank 2; positions 2 and 4 have equal keys, both rank 3. The new rank array is `[2, 1, 3, 1, 3, 0]`. Largest rank is 3, not yet $n - 1 = 5$, so keep doubling.

At $k = 2$, the pair keys are now $(\text{rank}[i], \text{rank}[i+2])$: position 0 $\to (2, 3)$, position 1 $\to (1, 1)$, position 2 $\to (3, 3)$, position 3 $\to (1, 0)$, position 4 $\to (3, -1)$, position 5 $\to (0, -1)$. Sorting positions: 5 $(0, -1)$, then 3 $(1, 0)$, then 1 $(1, 1)$, then 0 $(2, 3)$, then 4 $(3, -1)$, then 2 $(3, 3)$. So `sa` after the second sort is $[5, 3, 1, 0, 4, 2]$ — and that matches the suffix array I got from the naive sort. Recompute ranks: every key is now distinct, so ranks come out $[3, 2, 5, 1, 4, 0]$ and the largest rank is $5 = n - 1$. Done in two doublings.

Time to actually run it.

```
from codex.strings.suffix_array import suffix_array

text = "banana"
sa = suffix_array(text)

print(f"text     = {text!r}")
print(f"suffix array (prefix-doubling) = {sa}")
print()
print("sorted suffixes:")
for i in sa:
    print(f"  start={i}  suffix={text[i:]!r}")
```

```
text     = 'banana'
suffix array (prefix-doubling) = [5, 3, 1, 0, 4, 2]

sorted suffixes:
  start=5  suffix='a'
  start=3  suffix='ana'
  start=1  suffix='anana'
  start=0  suffix='banana'
  start=4  suffix='na'
  start=2  suffix='nana'
```

Same array as the naive sort, computed without ever materializing the suffix strings. Two doublings sufficed for a length-6 text; for a length-n text, $\lceil \log_2 n \rceil$ doublings is the worst-case bound, and many texts terminate early when their ranks become distinct.

Now a sanity check across a longer input — the canonical text I've been carrying through Part IV — to verify the two builders agree.

```
from codex.strings.suffix_array import suffix_array_naive, suffix_array

TEXT = "the rain in spain stays mainly in the plain"
sa_slow = suffix_array_naive(TEXT)
sa_fast = suffix_array(TEXT)

print(f"length: {len(TEXT)}")
print(f"naive and prefix-doubling agree: {sa_slow == sa_fast}")
```

```
print(f"first 10 sorted suffixes:")
for i in sa_fast[:10]:
    print(f"  start={i:>2}  suffix={TEXT[i:]!r}")
```

```
length: 43
naive and prefix-doubling agree: True
first 10 sorted suffixes:
  start= 8  suffix=' in spain stays mainly in the plain'
  start=30  suffix=' in the plain'
  start=23  suffix=' mainly in the plain'
  start=37  suffix=' plain'
  start= 3  suffix=' rain in spain stays mainly in the plain'
  start=11  suffix=' spain stays mainly in the plain'
  start=17  suffix=' stays mainly in the plain'
  start=33  suffix=' the plain'
  start=40  suffix='ain'
  start= 5  suffix='ain in spain stays mainly in the plain'
```

The arrays match cell by cell. The fast version produced the same answer the naive version did, in much less work — and on a million-character input, the fast version would still be tractable while the naive version would have died long ago.

29.4 Kasai's LCP in linear time

The suffix array on its own is enough to do pattern search. But there's a companion array that turns it into a much more powerful index, and the algorithm that computes it is one of the prettiest I know. The companion is the **longest-common-prefix array**, or **LCP**, defined as follows: `lcp[i]` is the length of the longest common prefix shared by the suffixes at `sa[i - 1]` and `sa[i]` — i.e., between consecutive entries in the sorted order. The first entry `lcp[0]` is defined to be 0 by convention (there's no `sa[-1]` to compare against).

The LCP captures *how similar* adjacent sorted suffixes are. For `"banana"`, the sorted suffixes are `"a"`, `"ana"`, `"anana"`, `"banana"`, `"na"`, `"nana"`, and reading the overlaps left-to-right: `"a"` vs `"ana"` share 1 character (`"a"`); `"ana"` vs `"anana"` share 3 characters (`"ana"`); `"anana"` vs `"banana"` share 0 characters (different first letter); `"banana"` vs `"na"` share 0; `"na"` vs `"nana"` share 2 (`"na"`). So `lcp = [0, 1, 3, 0, 0, 2]`.

The obvious way to compute LCP is to do it directly from the suffix array — for each adjacent pair, walk forward until characters disagree. That costs $O(n)$ per pair in the worst case, so $O(n^2)$ total. Kasai et al.'s 2001 algorithm does it in $O(n)$, by processing suffixes in a non-obvious order that lets the LCP from one suffix's computation jump-start the next.

The insight: process suffixes **in length order** (longest first, or equivalently in text-position order: position 0, 1, 2, …, $n-1$). When I move from the suffix at position i to the suffix at position $i+1$, the new suffix has the same characters as the old one with the first character stripped off. So the LCP between $T[i+1:]$ and *any other* suffix $T[j+1:]$ is at least $h-1$ if the LCP between $T[i:]$ and $T[j:]$ was h — chopping one character off both can shrink the common prefix by at most one. That's the structural fact I want to exploit.

For each position i in text order, I look up its rank — `rank[i]` is the position of suffix i in the sorted suffix array. If its rank is positive (it's not the lex-smallest suffix), I compare it against the suffix one rank lower, which is the suffix that immediately precedes it in sorted order. I walk the common prefix forward starting from offset `h` (the LCP I carried over from the previous text position), incrementing `h` for every matching character. When I move on to the next text position, I decrement `h` by one (because I'm comparing one-character-shorter suffixes), but never below zero.

The total work is bounded by a charging argument: `h` increases at most n times in total across the whole algorithm (each increment reads a matched character, and any specific character pair is read at most once), and `h` decreases at most n times (once per text-position transition), so the total time is $O(n)$. That's the magic.

Here's the algorithm.

```python
def lcp_kasai(text: str, sa: list[int]) -> list[int]:
    """Kasai's O(n) algorithm. lcp[i] = length of longest common prefix
    of the suffixes at sa[i-1] and sa[i]. lcp[0] is defined to be 0.
    """
    n = len(text)
    if n == 0:
        return []
    rank = [0] * n
    for i in range(n):
        rank[sa[i]] = i
    lcp = [0] * n
    h = 0
    for i in range(n):
        if rank[i] > 0:
            j = sa[rank[i] - 1]
            while i + h < n and j + h < n and text[i + h] == text[j + h]:
                h += 1
            lcp[rank[i]] = h
            if h > 0:
                h -= 1
        else:
            h = 0
    return lcp
```

The first block builds the inverse permutation `rank` from `sa` — if `sa[i] = j` then `rank[j] = i`. That's the lookup I need to turn "the suffix at position i" into "where does that suffix sit in the sorted order". The main loop walks i from 0 to $n-1$ in *text-position* order, not sorted order. For each position, it finds the predecessor in sorted order (the suffix at `sa[rank[i] - 1]`, which I call j), extends the running `h` as long as the characters match, and records `lcp[rank[i]] = h`. The decrement `h -= 1` at the end of each iteration is the carry-over: by the structural fact above, the LCP at the next text position is at least the current LCP minus one, so I start with a head-start instead of from zero.

Let me run it on `"banana"` and check the trace I worked out by hand.

```python
from codex.strings.suffix_array import suffix_array, lcp_kasai

text = "banana"
sa = suffix_array(text)
lcp = lcp_kasai(text, sa)

print(f"text = {text!r}")
print()
print(f"  i   sa[i]   lcp[i]   suffix")
for i in range(len(sa)):
    print(f"  {i}   {sa[i]:>3}      {lcp[i]:>3}     {text[sa[i]:]!r}")
```

```
text = 'banana'

  i   sa[i]   lcp[i]   suffix
  0     5        0     'a'
  1     3        1     'ana'
  2     1        3     'anana'
  3     0        0     'banana'
  4     4        0     'na'
  5     2        2     'nana'
```

Reading the table: `lcp[1] = 1` because `"a"` and `"ana"` share the prefix `"a"`; `lcp[2] = 3` because `"ana"` and `"anana"` share the prefix `"ana"`; `lcp[3] = 0` because `"anana"` and `"banana"` share no prefix (`a` vs `b`); `lcp[4] = 0` because `"banana"` and `"na"` share no prefix; `lcp[5] = 2` because `"na"` and `"nana"` share the prefix `"na"`. Six suffixes, six LCP entries, all computed in a single linear pass over the text.

29.5 Pattern search in $O(m \log n)$

With the suffix array built, pattern search is the binary-search lookup I sketched in the opening. I want the range of suffix-array entries whose suffixes begin with the pattern P — call those entries `[left, right)` — and then I return the text positions `sa[left:right]` as the list of match positions.

I find the range with two binary searches. The first is a lower-bound: the smallest index `mid` such that the suffix `text[sa[mid]:]` is lexicographically `>= pattern` when compared up to `len(pattern)` characters. The second is an upper-bound: the smallest index `mid` such that the suffix is lex `> pattern`. Between them lies the contiguous block of matches.

One subtle point worth saying out loud: I compare the suffix's first m characters (the slice `text[sa[mid]:sa[mid] + m]`) against the pattern, not the entire suffix. The truncation is what makes the comparison cost $O(m)$ per binary-search step instead of $O(n)$, and it's correct because a match is exactly the condition "the suffix's first m characters equal the pattern." If the suffix is shorter than m characters (the slice falls off the end), the slice is just the entire suffix, which is strictly shorter than the pattern and therefore lex-less than it — exactly the behavior I want, since a too-short suffix can't be a match.

The function returns `sorted(sa[left:right])` — the matches sorted by text position, not by suffix-array index. That ordering is the one every other chapter's `find_all` has used (Aho-Corasick, KMP, brute force) on the canonical `"ain"` query, so the suffix-array variant lines up with them position-by-position.

```
def find_all(text: str, sa: list[int], pattern: str) -> list[int]:
    """Binary-search the suffix array for the range of suffixes that
    start with the pattern. Return the (sorted) text positions of all
    matches. O(m log n) per query.
    """
    n, m = len(text), len(pattern)
    if m == 0:
        return list(range(n))
    if m > n:
        return []

    # lower bound: smallest mid such that text[sa[mid]:sa[mid]+m] >= pattern
    lo, hi = 0, n
    while lo < hi:
        mid = (lo + hi) // 2
        if text[sa[mid]:sa[mid] + m] < pattern:
            lo = mid + 1
        else:
            hi = mid
    left = lo

    # upper bound: smallest mid such that text[sa[mid]:sa[mid]+m] > pattern
    lo, hi = left, n
    while lo < hi:
        mid = (lo + hi) // 2
        if text[sa[mid]:sa[mid] + m] <= pattern:
            lo = mid + 1
        else:
            hi = mid
    right = lo

    return sorted(sa[left:right])
```

Two binary searches, each $O(\log n)$ iterations with $O(m)$ comparison work per iteration, total $O(m \log n)$. The final `sorted` is over at most z entries where z is the number of matches; for typical queries z is much smaller than n, so the sort cost is dominated by the binary searches.

Run it on the canonical text.

```python
from codex.strings.suffix_array import suffix_array, find_all

TEXT = "the rain in spain stays mainly in the plain"
sa = suffix_array(TEXT)

pattern = "ain"
matches = find_all(TEXT, sa, pattern)
print(f"text    = {TEXT!r}")
print(f"pattern = {pattern!r}")
print(f"matches at text positions: {matches}")
print()

# Same query as every previous Part IV chapter - should match exactly.
for start in matches:
    print(f"  position {start:>2}: {TEXT[start:start + len(pattern) + 5]!r}")
```

```
text    = 'the rain in spain stays mainly in the plain'
pattern = 'ain'
matches at text positions: [5, 14, 25, 40]

  position  5: 'ain in s'
  position 14: 'ain stay'
  position 25: 'ainly in'
  position 40: 'ain'
```

The match positions are `[5, 14, 25, 40]` — exactly the same four positions every previous Part IV chapter found for the same query on the same text. The difference is that every previous algorithm re-scanned the text from scratch per pattern; here I built the suffix array once and the query did four binary-search comparisons, each touching $O(m \log n) \approx 3 \cdot \log_2 43 \approx 17$ characters of work. Even more strikingly, if I run a second pattern against the same index I pay the cost of *only the new query*, not another full text scan.

```python
from codex.strings.suffix_array import find_all

# Same suffix array, several different queries - the build cost amortizes.
for pattern in ["the", "in", "plain", "missing"]:
    matches = find_all(TEXT, sa, pattern)
    summary = matches if matches else "no matches"
    print(f"  pattern {pattern!r:>10}  ->  {summary}")
```

```
  pattern      'the'  ->  [0, 34]
  pattern       'in'  ->  [6, 9, 15, 26, 31, 41]
  pattern    'plain'  ->  [38]
  pattern  'missing'  ->  no matches
```

Four queries, one suffix array, four small binary-search costs. That's the build-once-query-many shape the chapter opened by promising — the same operational mode Aho-Corasick gave you for multi-pattern matching, but now in the orthogonal direction: arbitrary patterns against a fixed text, instead of fixed patterns against arbitrary text.

29.6 Finding the longest repeated substring

Here's where the LCP array starts paying off. The longest substring that appears at least twice in T has a beautiful characterization in terms of LCP: **it's the substring whose length is the maximum value in the LCP array**, and its text position is the suffix-array entry at that LCP index.

Why? Any substring that appears twice in T is a common prefix of two distinct suffixes — namely, the two suffixes starting at the two occurrence positions. Sort all the suffixes; the two suffixes whose common prefix is

this substring don't have to be adjacent in the sorted order, but the LCP between *adjacent* sorted suffixes is at least the common-prefix length whenever the two suffixes nest within an LCP-monotone block. More precisely: the longest repeated substring's length equals the maximum LCP value, because two suffixes with a long common prefix must be adjacent in the sorted order (otherwise the suffix sandwiched between them would also share that prefix, contradicting either the maximality argument or just demonstrating an even-longer repeat). So the maximum entry in the LCP array tells me the length of the longest repeat, and the corresponding suffix-array entry tells me where one occurrence starts — I can read off the substring as `text[sa[i]:sa[i] + lcp[i]]`.

Let me show it on a text with a non-trivial repeat. `"abracadabra"` is the classic example — the substring `"abra"` appears twice, at positions 0 and 7, and that's the longest repeated substring in the text.

```
def longest_repeated_substring(text: str) -> tuple[int, str]:
    """Find the longest substring appearing at least twice in text.
    Returns (length, substring). If text has no repeated substring of
    positive length, returns (0, "").
    """
    n = len(text)
    if n < 2:
        return (0, "")
    sa = suffix_array(text)
    lcp = lcp_kasai(text, sa)
    idx = max(range(n), key=lambda i: lcp[i])
    length = lcp[idx]
    if length == 0:
        return (0, "")
    return (length, text[sa[idx]:sa[idx] + length])
```

The function builds the suffix array and the LCP, picks the index of the maximum LCP entry, and returns the substring at that suffix-array position with that length. Three lines of real work after the two builders. The whole cost is dominated by the suffix-array construction at $O(n \log^2 n)$; everything else — the LCP build, the linear scan for the maximum — is $O(n)$.

```
from codex.strings.suffix_array import longest_repeated_substring

for text in ["abracadabra", "banana", "abcabcaa", "mississippi", "no_repeats"]:
    length, substring = longest_repeated_substring(text)
    if length == 0:
        print(f"  {text!r:>15}  ->  no repeated substring")
    else:
        print(f"  {text!r:>15}  ->  {substring!r} (length {length})")
```

```
    'abracadabra'  ->  'abra' (length 4)
         'banana'  ->  'ana' (length 3)
       'abcabcaa'  ->  'abca' (length 4)
    'mississippi'  ->  'issi' (length 4)
     'no_repeats'  ->  'e' (length 1)
```

`"abracadabra"` → `"abra"` (length 4). `"banana"` → `"ana"` (length 3). `"abcabcaa"` → `"abca"` (length 4, appearing at positions 0 and 3). `"mississippi"` → `"issi"` (length 4). The application is the kind of thing that would be expensive to do by brute force — comparing every pair of substrings is $O(n^4)$ — and the suffix-array approach brings it down to the build cost plus linear time.

The construction generalizes. The k-th longest repeated substring drops out by sorting LCP entries; the *longest substring repeating at least k times* drops out by sliding a window of length $k-1$ over the LCP array and taking the maximum of the per-window minima; the longest substring common to two texts drops out by concatenating them with a separator character and running the same algorithm. The LCP array is the workhorse.

29.7 What about suffix trees and suffix automata?

I should say something about the structures the suffix array competes with, since you'll see them named in the literature and I don't want you to think the array is the only option.

The **suffix tree** is the older structure — Weiner 1973, McCreight 1976, Ukkonen 1995 — and it stores the suffixes as the leaves of a compressed trie. Every internal node has at least two children, the edges are labeled with substrings of T, and the path from the root to any leaf spells a suffix. The structure is $O(n)$ in size if you use a clever edge-labeling scheme, and it supports an enormous range of string-processing queries in time proportional to the query size, not the text size: pattern matching in $O(m)$, longest common substring of two texts, all pairs of repeated substrings, and more.

The suffix tree is gorgeous in theory and a pain in practice. The constant factor on the size is large — typically 15–20 bytes per character of the input — and the implementations of Ukkonen's $O(n)$ construction are notoriously fiddly. The suffix array gives you almost everything the suffix tree gives you (modulo a log factor on some queries) at 4–8 bytes per character and with an implementation you can write in fifty lines. Almost every modern library — `bowtie` and `bwa` for genomics, `sa-is` and `divsufsort` as standalone libraries, the BWT-based compressors like `bzip2` and `xz` — uses a suffix array (or a variant like the FM-index built from one), not a suffix tree.

The **suffix automaton** is a different beast — Blumer et al.'s 1985 directed acyclic word graph (DAWG) — which is the minimal DFA recognizing every substring of T. It's the right structure when you want online construction (build it character by character as the text streams in) and is the basis of some very fast string-matching libraries. Its space is also $O(n)$, and its construction is famously elegant. Outside that niche, the suffix array is still usually the right call.

What about achieving $O(n)$ suffix-array construction? It's possible — the **SA-IS algorithm** by Nong, Zhang, and Chan (2009) achieves it via an induced-sorting scheme that exploits the structure of LMS-suffixes ("leftmost S-type"), and several other linear-time algorithms exist (DC3 by Kärkkäinen-Sanders 2003, KA by Kärkkäinen-Aluru 2003). I'm not going to implement any of them; they're significantly harder to get right than Manber-Myers, and for most workloads $O(n \log^2 n)$ on a length-n text is fast enough. The pointer if you want the linear-time version: SA-IS is the modern standard, and `divsufsort` is the reference C implementation.

29.8 The three questions, applied

Is it correct?

There are three correctness claims to nail down: the suffix-array builders return the right permutation; the LCP builder returns the right LCP values; and the pattern-search routine returns exactly the set of match positions.

For the **suffix-array builders**, `suffix_array_naive` is correct by appeal to Python's sort: `sorted(range(n), key=lambda i: text[i:])` returns the positions in the order of their suffixes' lex order, which is the suffix-array definition. For `suffix_array`, the invariant maintained across iterations is: **after the kth iteration, the array `rank` assigns each suffix a value such that `rank[i] < rank[j]` if and only if the first k characters of suffix i are lex-less than the first k characters of suffix j, with the sentinel `-1` treated as smaller than any character.** That invariant holds at $k = 1$ because the initial ranks are character codes. It's preserved by the doubling step because the pair $(\text{rank}[i], \text{rank}[i+k])$ encodes exactly the first $2k$ characters of suffix i under the previous invariant, and re-ranking by sorted-key order under that pair gives the new rank that satisfies the $2k$-character version of the invariant. Termination at $\text{rank}[\text{sa}[n-1]] = n-1$ means all ranks are distinct, which means the lex order is fully determined, which means `sa` is the suffix array.

For **Kasai's LCP**, the invariant is on the running `h`: at the top of iteration i of the outer loop, `h` equals the length of the common prefix between `text[i:]` and `text[sa[rank[i]] - 1] + ...` — wait, that's not quite right. Let me restate it. The invariant is that when I enter iteration i, `h` is a *lower bound* on the LCP I'm about to compute, justified by the structural fact that chopping one character off two suffixes shrinks their LCP by at most one. The inner `while` loop extends `h` to the actual LCP by matching forward, and the `h -= 1` at the end of the iteration carries the lower bound forward to iteration $i+1$ for free. Each value of `lcp[rank[i]]` is set to the correct LCP at the moment of assignment, because the inner loop runs until characters disagree (or one of the suffixes ends), which is the definition of LCP.

For `find_all`, the two binary searches compute the lower and upper bounds of the range of suffix-array entries whose suffixes have first m characters equal to the pattern. Lex order's transitivity makes the binary search correct: suffixes with first m characters less than the pattern form a contiguous prefix of `sa`, suffixes with first m

characters equal to the pattern form a contiguous middle block, and suffixes with first m characters greater than the pattern form a contiguous suffix. The lower-bound search finds the boundary between the first two; the upper-bound search finds the boundary between the last two. The slice `sa[left:right]` is exactly the set of starting positions of matching suffixes, which is exactly the set of match positions in the text.

How efficient is it?

The cost table for the suffix-array family on a text of length n and a pattern of length m:

Operation	Time	Extra space
`suffix_array_naive(text)`	$O(n^2 \log n)$	$O(n^2)$
`suffix_array(text)`	$O(n \log^2 n)$	$O(n)$
`lcp_kasai(text, sa)`	$O(n)$	$O(n)$
`find_all(text, sa, pattern)`	$O(m \log n + z)$	$O(z)$
`longest_repeated_substring(text)`	$O(n \log^2 n)$	$O(n)$

The $O(n \log^2 n)$ on the prefix-doubling build comes from $O(\log n)$ doubling iterations, each doing a Python sort over n pair-keys at $O(n \log n)$ cost. The proper Manber-Myers algorithm replaces each sort with a radix sort over the pair-keys at $O(n)$ per pass, giving the overall $O(n \log n)$ bound — the same bound a comparison-sort-of-suffixes would naturally hit if each comparison were $O(1)$ instead of $O(n)$.

Kasai's $O(n)$ LCP is the showcase of the family. The amortization argument is the one I sketched in the section: `h` increases at most n times across the whole algorithm (each increment matches one character pair, and any specific text position can be the right endpoint of an LCP extension at most once), and `h` decreases at most n times (once per text-position iteration). So the total work in the inner loop is $O(n)$, and the outer loop adds another $O(n)$ for the bookkeeping.

Pattern search is $O(m \log n)$ per query, where $m \log n$ is the cost of $O(\log n)$ binary-search steps each doing an $O(m)$ substring comparison. The plus-z term is the cost of returning the z matches (the final `sorted` over the slice). For typical workloads $z \ll n$, so the binary searches dominate.

Bilingually: on a megabyte-sized text, building the suffix array does about $10^6 \cdot 20^2 = 4 \cdot 10^8$ work in the prefix-doubling version — fast enough to run in a few seconds in Python, fraction of a second in C. Each subsequent pattern query at length 30 does $30 \cdot 20 = 600$ characters of work, which is essentially instantaneous. Run a million queries against the same text and you've paid the build cost once and amortized it across all of them. Compare with Aho-Corasick from chapter 28: same operational mode, but Aho-Corasick fixes the patterns at build time and lets the text vary, whereas the suffix array fixes the text at build time and lets the patterns vary. They occupy orthogonal niches.

Is it optimal?

For suffix-array construction, the lower bound is $\Omega(n)$ — every text character must be examined to encode it into the array — and the **SA-IS** algorithm by Nong et al. (2009) achieves $O(n)$ matching that bound. My prefix-doubling implementation hits $O(n \log^2 n)$, a $\log^2 n$ factor above optimal; the original Manber-Myers algorithm with radix sort hits $O(n \log n)$, a $\log n$ factor above optimal. Both are widely used in production despite being suboptimal because their constant factors are small and their implementations are tractable, whereas SA-IS is fiddly enough that most production systems ship a non-linear variant unless they care about asymptotic performance on very large inputs.

For LCP construction, **Kasai's** $O(n)$ matches the trivial $\Omega(n)$ lower bound exactly — you can't do better, and you don't need to. The algorithm is optimal.

For pattern search, $O(m \log n)$ per query is optimal *for a comparison-based search over a sorted suffix-array structure*. With auxiliary data — specifically, an **enhanced suffix array** that augments the SA + LCP with additional tables for $O(1)$ range-minimum queries — pattern search drops to $O(m)$ per query, matching the suffix-tree bound. Abouelhoda, Kurtz, and Ohlebusch (2004) is the canonical reference for that construction. I won't implement it; the bookkeeping is significant and the $\log n$ factor on top of m is usually invisible in practice.

The suffix-tree alternative gives strict $O(m)$ pattern search and $O(n)$ size, but with much larger constants, harder construction, and a clumsy memory profile. The trade-off favors the suffix array in essentially every real workload — that's why the modern string-processing libraries are built on it.

29.9 What suffix arrays teach you

Two big lessons from this chapter, and one observation about the role the suffix array plays inside the wider Codex.

First, sort the *positions* of an object, not the object itself. The naive suffix-array build materializes every suffix as a separate string — total memory $O(n^2)$ — and that's what makes it impractical. The smart move is to recognize that the suffixes don't need to be materialized; the algorithm only needs to *compare* them, and comparison can be expressed in terms of the underlying text and a few index lookups. So I sort positions (which take $O(n)$ memory) and use the positions to do comparisons against the text on the fly. This pattern shows up all over the book: chapter 5's argsort takes a sequence and returns the indices in sorted order rather than reshuffling the values; chapter 21's segment tree stores aggregates of array slices rather than the slices themselves; chapter 28's Aho-Corasick stores trie nodes representing patterns rather than the patterns inline. When you find yourself about to materialize an exponential set of derived objects to sort or process, ask whether you can index into them on the fly instead.

Second, the right processing order can collapse quadratic work to linear. Kasai's algorithm is the cleanest example I've ever seen of this lesson. The naive LCP construction processes adjacent sorted-suffix pairs in the order they appear in the suffix array; that's $O(n)$ pairs, each costing $O(n)$ to compute, total $O(n^2)$. Kasai processes the same set of pairs in *text-position order* instead, which lets the LCP from the previous text position seed the LCP for the next one with a one-character drop. Same outputs, different order, asymptotically less work. The pattern shows up elsewhere — chapter 26's Boyer-Moore preprocesses patterns in reverse character order to enable forward skipping, KMP's failure function builds in pattern-prefix-length order rather than left-to-right — and the meta-lesson is that when you have an apparently-quadratic loop nest, look at the dependency structure between iterations and see whether reordering them creates carry-over opportunities.

The chapter sits at the boundary between Part IV's pattern-matching arc and the broader string-indexing world that opens up after it. Chapters 23–28 all answered the question "given these patterns, find them in this text" with progressively more sophisticated machinery. This chapter answered a different question — "given this text, prepare it so any pattern query is fast" — and that pivot is what the suffix array exists for. Full-text search engines, biological sequence aligners, the Burrows-Wheeler transform underlying `bzip2` and the FM-index, plagiarism detectors looking for long shared substrings, document deduplication systems: every one of those applications builds a suffix-array variant once and runs millions of queries against it. The pattern-matching arc of Part IV led you here, and the door opens onto a much wider field of string algorithms that all start with "first, build the suffix array of T."

29.10 Notes and further reading

The suffix array was introduced by Udi Manber and Gene Myers in their 1993 paper "Suffix arrays: A new method for on-line string searches," published in *SIAM Journal on Computing* 22(5):935–948. The paper presents the prefix-doubling algorithm (with radix sort) and the $O(m \log n)$ pattern-search procedure I've implemented here. The motivation was as an alternative to the suffix tree — Manber and Myers wanted a structure with smaller constant-factor memory cost and a simpler implementation, while preserving the asymptotic indexing benefits.

Kasai's linear-time LCP algorithm appears in Toru Kasai, Gunho Lee, Hiroki Arimura, Setsuo Arikawa, and Kunsoo Park's 2001 paper "Linear-time longest-common-prefix computation in suffix arrays and its applications," published in *Proceedings of CPM 2001*, LNCS 2089, pp. 181–192. The paper is short and worth reading directly — the amortized-cost argument for the $O(n)$ bound is presented cleanly there, and the paper also introduces several of the LCP-based applications I gestured at (longest repeated substring, longest common substring, etc.).

The $O(n)$ suffix-array construction algorithm I named — **SA-IS** — is from Ge Nong, Sen Zhang, and Wai Hong Chan's 2009 paper "Linear suffix array construction by almost pure induced-sorting," published in the *2009 Data Compression Conference* (DCC '09), pp. 193–202. It's the modern standard for production suffix-array construction; Yuta Mori's `sa-is` C implementation is the reference. Earlier linear-time algorithms — Kärkkäinen

and Sanders's **DC3** (2003), and Kim, Sim, Park, and Park's **KSPP** (2003) — are also worth knowing about as historical context.

CLRS does not cover suffix arrays directly. Sedgewick and Wayne's *Algorithms* (4th ed.) discusses them briefly in §6.4. The modern textbook reference is Maxime Crochemore, Christophe Hancart, and Thierry Lecroq's *Algorithms on Strings* (Cambridge University Press, 2007), which devotes chapter 4 to suffix arrays and chapter 5 to the related suffix tree and suffix automaton — read those three chapters together for the full landscape. Mohamed Abouelhoda, Stefan Kurtz, and Enno Ohlebusch's 2004 paper "Replacing suffix trees with enhanced suffix arrays" (*Journal of Discrete Algorithms* 2(1):53–86) is the canonical reference for the enhanced-suffix-array construction that achieves suffix-tree-equivalent query times.

Applications I touched on: the **Burrows-Wheeler transform** by Michael Burrows and David Wheeler (1994 technical report, later published in *Communications of the ACM*) is computed by sorting all rotations of the input, which turns out to be equivalent to constructing the suffix array of `text + "$"` and reading off the previous character at each suffix-array position. The BWT is what powers `bzip2` compression and the FM-index used by `bowtie` and `bwa` for short-read alignment in genomics. The **FM-index** itself was introduced by Paolo Ferragina and Giovanni Manzini in their 2000 paper "Opportunistic data structures with applications" (*Proc. FOCS 2000*), and it provides $O(m)$ pattern-counting queries with $O(nH_k(T))$ space — where $H_k(T)$ is the k-th order empirical entropy of T — which is sublinear in the text size for compressible inputs.

In the next chapter I'll leave string indexing and turn to a different kind of string problem entirely: not "where do these patterns appear?" but "how *similar* are two strings to each other?" Edit distance — the minimum number of insertions, deletions, and substitutions to turn one string into another — is the question, and dynamic programming over a 2D table is the answer. That chapter closes Part IV.

30 Counting the edits between two strings

Chapter 29 closed Part IV's pattern-matching arc and promised a sharp turn. Every algorithm from chapter 23 onward answered the same question in different shapes — "where does this pattern occur exactly in this text?" — and the suffix array's parting move was to flip preprocessing from patterns to text, so any pattern query was cheap. I want to flip the *question* now. Forget exact occurrence. Two strings sit in front of me, and I want a single number that says how *close* they are — how many character-level edits separate one from the other. That number is the **edit distance**, and the structure that computes it is the cynosure of this chapter: **an n×m grid where each cell is the cost of aligning two prefixes**.

By the end of the chapter you'll have built four functions: `hamming` for the trivial case of two equal-length strings, `levenshtein` for the general case where insertions and deletions are also allowed, `align` to recover the actual edits behind the distance number, and `lcs` and `lcss` for two close cousins (longest common subsequence, longest common substring) that share the same grid-of-cells engine. You'll have watched the kitten-to-sitting DP table fill in row by row, traced the backtracking path that produces the alignment, and seen how exact substring search falls out as a special case of longest-common-substring. The empower-promise: by chapter's end, the phrase "dynamic programming over a 2D table" stops being a research-paper incantation and becomes a small Python loop you've watched run on six-character strings.

One framing note I owe you up front: this is also the origin story for Part VI. The book hasn't named "dynamic programming" yet, and won't name it formally until Part VI's landing page, but edit distance is the canonical example — the one every DP textbook reaches for first — so I'll spend the closing sections of this chapter pulling out the pattern explicitly. The grid you fill, the cell-by-cell recurrence, the optimal-substructure property, the overlapping subproblems: all of those are concepts Part VI generalizes. For now, treat the n×m grid as the chapter's central image and let the abstraction wait.

30.1 When the match is fuzzy

Suppose you type `recieve` into a search box. The dictionary has `receive`. Should the search engine say "no match" and walk away? It shouldn't — and the reason it shouldn't is that those two strings differ by one transposition's worth of editing work, and a sensible spell-checker recognizes that as a likely typo. Suppose you have two DNA sequences and you want to know whether they come from related organisms. Their letters don't agree exactly — a few substitutions here, an insertion there from a duplication event, a deletion there from a frameshift — and you want a single similarity number that summarizes the alignment. Suppose you have two text files and you want a `diff`: a minimal description of how to turn one into the other. All three problems reduce to the same primitive: given two strings a and b, count the minimum number of single-character edits — insertions, deletions, substitutions — that transform a into b.

That count is the **Levenshtein distance**, named after Vladimir Levenshtein who introduced it in a 1965 Russian paper (translated to English in 1966). It's the workhorse distance metric for strings in essentially every domain that deals with sequences: bioinformatics (`needle` and `water` in the EMBOSS suite, BLAST's local alignment, every modern DNA aligner), spell correction (every search engine and every word processor), version control (`diff` and `patch`), record linkage (matching `John Smith` against `Jon Smyth`), plagiarism detection, fuzzy text search. If a tool measures string similarity, the Levenshtein distance or a close variant is probably underneath.

The naive approach — try every possible sequence of edits and pick the shortest — is hopeless. A string of length n has exponentially many edit sequences, and brute force would melt down on any input longer than a few characters. The dynamic-programming approach I'll build in this chapter computes the same number in $O(nm)$ time and $O(nm)$ space, which is fast enough to handle string pairs of length tens of thousands without breaking a sweat.

Before I get to the general case, I want to start with the simpler relative — the one where insertions and deletions are off the table and only substitutions are allowed.

30.2 Hamming as the trivial case

When the two strings have equal length and the only edits you allow are substitutions, the edit distance has a name of its own: the **Hamming distance**. Richard Hamming defined it in his 1950 paper on error-detecting codes — the question he cared about was how many bit-flips it took to turn one codeword into another, and the answer was simply the number of positions where the two codewords disagreed.

For strings, the definition is identical: count the positions at which the two strings differ. It's a single-pass $O(n)$ scan with no auxiliary structure beyond a counter.

```
def hamming(a: str, b: str) -> int:
    """Number of positions at which two equal-length strings differ.
    Raises ValueError if lengths disagree."""
    if len(a) != len(b):
        raise ValueError(f"hamming requires equal lengths, got {len(a)} vs {len(b)}")
    return sum(1 for x, y in zip(a, b) if x != y)
```

One line of work plus a guard. The guard is what makes Hamming an honest answer to a narrower question: equal-length strings only. I raise a `ValueError` rather than silently truncating because conflating Hamming with something else would defeat the whole point — Hamming is the distance when alignment is fixed character-by-character, and unequal lengths break that assumption.

Running it on a famously dissimilar pair:

```
from codex.strings.edit_distance import hamming

print(f"hamming('hello', 'world') = {hamming('hello', 'world')}")
print(f"hamming('karolin', 'kathrin') = {hamming('karolin', 'kathrin')}")
print(f"hamming('abcde', 'abcde') = {hamming('abcde', 'abcde')}")
```

```
hamming('hello', 'world') = 4
hamming('karolin', 'kathrin') = 3
hamming('abcde', 'abcde') = 0
```

`hello` versus `world` differs at four of five positions — the two strings agree only on the `l` at index 3 (`hello[3] = 'l'` and `world[3] = 'l'`) — so the distance is 4. `karolin` versus `kathrin` differs at three positions (positions 2, 3, 4: `r` vs `t`, `o` vs `h`, `l` vs `r`) and agrees at the other four, so the distance is 3. Equal strings have distance 0, which is the property a metric needs.

The limitation is severe. Hamming insists that the two strings line up character-by-character with no shifting. If I'm comparing `kitten` against `sitten` (one substitution at position 0), Hamming gives me 1 — fine. But `kitten` against `sitting` — same first four characters, plus an extra letter at the end and a swap in the middle — has different lengths, so Hamming refuses to answer. And even `kitten` against `kittten` (one extra `t` inserted) — same length difference, same intuitive answer of one edit — falls outside Hamming's scope. The whole shape of typo-correction, version control, biological alignment depends on tolerating insertions and deletions, not just substitutions, and Hamming gives me no purchase on those.

That's the pedagogical setup: Hamming is the right answer to a narrow question, and the narrow question shows you what the harder question needs. The harder question is the Levenshtein distance, and the algorithm that computes it is the centerpiece of the chapter.

30.3 The edit-distance recurrence

Three operations are on the table: **insert** a character into a, **delete** a character from a, or **substitute** one character of a with another. Each operation costs 1. The edit distance between strings a and b is the minimum total cost over all sequences of operations that transform a into b.

The trick — and it's the trick that earns dynamic programming its name in this chapter — is to phrase the question in terms of *prefixes* of the two strings. Define `dp[i][j]` to be the edit distance between the prefix

`a[:i]` (the first i characters of a) and the prefix `b[:j]` (the first j characters of b). The full answer I want is `dp[n][m]` where $n = \text{len}(a)$ and $m = \text{len}(b)$.

The recurrence I claim:

$$\text{dp}[i][j] = \begin{cases} j & \text{if } i = 0 \\ i & \text{if } j = 0 \\ \text{dp}[i-1][j-1] & \text{if } i, j > 0 \text{ and } a[i-1] = b[j-1] \\ 1 + \min(\text{dp}[i-1][j],\ \text{dp}[i][j-1],\ \text{dp}[i-1][j-1]) & \text{otherwise} \end{cases}$$

Bilingually: if either prefix is empty, the answer is the length of the other prefix — you have no choice but to insert (or delete) every character of the non-empty one. If both prefixes are non-empty and their last characters agree, the last character is free: the distance between `a[:i]` and `b[:j]` equals the distance between `a[:i-1]` and `b[:j-1]`, because once you've aligned the prefixes-minus-one, the matching final character costs zero. If the last characters disagree, you pay 1 for some edit at the end, and the rest of the work is whichever of three subproblems is cheapest:

- **Delete `a[i-1]`** — the last character of `a` is removed and you're left aligning `a[:i-1]` against the full `b[:j]`. Subproblem: `dp[i-1][j]`.
- **Insert `b[j-1]`** — you tack `b[j-1]` onto the end of `a`, which accounts for the last character of `b[:j]`, and you're left aligning the full `a[:i]` against `b[:j-1]`. Subproblem: `dp[i][j-1]`.
- **Substitute** — replace `a[i-1]` with `b[j-1]`, accounting for the last character of both prefixes, and align `a[:i-1]` against `b[:j-1]`. Subproblem: `dp[i-1][j-1]`.

Add 1 to whichever subproblem cost is smallest and that's your `dp[i][j]`.

That recurrence is the whole algorithm. The work that's left is mechanical: fill the table in an order that has every needed subproblem computed before you ask for it. Since `dp[i][j]` depends on `dp[i-1][j]`, `dp[i][j-1]`, and `dp[i-1][j-1]` — three neighbors above and to the left — filling row-by-row from top to bottom, left-to-right within each row, guarantees that every reference is already populated.

30.4 Filling the table bottom-up

Here's the function. It allocates an $(n+1) \times (m+1)$ table, seeds the first row and column with the base cases, then walks the recurrence cell-by-cell.

```
def levenshtein(a: str, b: str) -> int:
    """Minimum number of insert/delete/substitute edits to transform a into b.
    Bottom-up DP, O(nm) time, O(nm) space."""
    n, m = len(a), len(b)
    if n == 0:
        return m
    if m == 0:
        return n
    # dp[i][j] = edit distance between a[:i] and b[:j]
    dp = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(n + 1):
        dp[i][0] = i
    for j in range(m + 1):
        dp[0][j] = j
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                dp[i][j] = 1 + min(
                    dp[i - 1][j],      # delete a[i-1]
                    dp[i][j - 1],      # insert b[j-1]
                    dp[i - 1][j - 1],  # substitute a[i-1] with b[j-1]
```

```
                )
    return dp[n][m]
```

The early-exit cases handle empty inputs — if either string is empty, the answer is the length of the other, which falls out of the base case but is faster to short-circuit. The double loop is the recurrence transcribed directly. Each cell touches three predecessors, so each cell does $O(1)$ work, and there are $(n+1)(m+1)$ cells — that's where the $O(nm)$ time bound comes from. The table allocation is also $O(nm)$, which I'll come back to in the efficiency discussion.

Running it on the canonical example — Levenshtein's own kitten/sitting pair — and a few neighbors:

```
from codex.strings.edit_distance import levenshtein

print(f"levenshtein('kitten', 'sitting') = {levenshtein('kitten', 'sitting')}")
print(f"levenshtein('Saturday', 'Sunday') = {levenshtein('Saturday', 'Sunday')}")
print(f"levenshtein('', 'abc') = {levenshtein('', 'abc')}")
print(f"levenshtein('abc', '') = {levenshtein('abc', '')}")
print(f"levenshtein('hello', 'hello') = {levenshtein('hello', 'hello')}")
```

```
levenshtein('kitten', 'sitting') = 3
levenshtein('Saturday', 'Sunday') = 3
levenshtein('', 'abc') = 3
levenshtein('abc', '') = 3
levenshtein('hello', 'hello') = 0
```

The kitten/sitting answer is 3, matching Levenshtein's hand-worked example: `kitten` → `sitten` (substitute `k`→`s`) → `sittin` (substitute `e`→`i`) → `sitting` (insert `g`). `Saturday` against `Sunday` is also 3 — delete `a`, delete `t`, substitute `r`→`n`. The two empty-prefix cases are 3 apiece (the length of the non-empty string). Equal strings give 0.

I want to actually look at the table, not just the final number. Let me write a small helper that prints the full DP grid for `levenshtein('kitten', 'sitting')`, with the two strings labeling the rows and columns.

```
def print_dp_table(a: str, b: str) -> None:
    n, m = len(a), len(b)
    dp = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(n + 1):
        dp[i][0] = i
    for j in range(m + 1):
        dp[0][j] = j
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                dp[i][j] = 1 + min(dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1])

    # Header row: blank corner, then "" placeholder, then b's characters
    header = "       ''  " + "  ".join(f" {c}" for c in b)
    print(header)
    for i in range(n + 1):
        label = "''" if i == 0 else f" {a[i - 1]}"
        row_cells = "  ".join(f"{dp[i][j]:>2}" for j in range(m + 1))
        print(f"  {label}  {row_cells}")

print_dp_table("kitten", "sitting")
```

```
       ''   s   i   t   t   i   n   g
  ''   0   1   2   3   4   5   6   7
```

```
   k   1   1   2   3   4   5   6   7
   i   2   2   1   2   3   4   5   6
   t   3   3   2   1   2   3   4   5
   t   4   4   3   2   1   2   3   4
   e   5   5   4   3   2   2   3   4
   n   6   6   5   4   3   3   2   3
```

Read the grid like a coordinate plane. The top-left cell is `dp[0][0] = 0` (two empty prefixes are at distance 0). The first row is the base case for an empty `a`: `dp[0][j] = j` because turning the empty string into `b[:j]` costs j insertions. The first column is symmetric: `dp[i][0] = i` because turning `a[:i]` into the empty string costs i deletions.

Now follow the interior. `dp[1][1]` is the distance between `'k'` and `'s'` — different characters, so it's $1 + \min(\text{dp}[0][1], \text{dp}[1][0], \text{dp}[0][0]) = 1 + \min(1, 1, 0) = 1$. One substitution. `dp[2][2]` is the distance between `'ki'` and `'si'` — last characters agree (`i` = `i`), so it inherits `dp[1][1] = 1`. The match-cost-zero rule shows up on the diagonal whenever the running prefixes' last characters align, and the table's diagonal between `dp[2][2]` and `dp[4][4]` is exactly that — `kit` vs `sit` (last char `t` = `t`, distance 1), `kitt` vs `sitt` (last char `t` = `t`, distance 1).

The final cell `dp[6][7] = 3` is the answer. You can read the value off the bottom-right corner of any edit-distance table — that cell is the distance between the two full strings.

The grid is the cynosure. Every entry is the cost of aligning two prefixes; the final entry is the cost of aligning the two full strings; and every interior cell is one decision step computed from three neighbors. The whole structure of edit distance is on display.

30.5 Reconstructing the alignment

The distance number tells you *how many* edits separate the two strings, but not *which* edits. For many applications — `diff`'s output, biological alignment displays, spell-correction hints — you also want the alignment itself: a column-by-column display showing which characters of `a` match which characters of `b`, where the substitutions happened, and where the gaps (insertions and deletions) fell.

To recover the alignment, I backtrack from `dp[n][m]` to `dp[0][0]`, asking at each cell "which of the three predecessors produced this value?" — and the predecessor I find tells me which edit was at this step:

- If `a[i-1] == b[j-1]` and `dp[i][j] == dp[i-1][j-1]`, the step is a **match** (no edit). Move diagonally to `dp[i-1][j-1]`.
- If `dp[i][j] == dp[i-1][j-1] + 1`, the step is a **substitution**. Move diagonally.
- If `dp[i][j] == dp[i-1][j] + 1`, the step is a **deletion from `a`** (a `'-'` in `b`'s column). Move up to `dp[i-1][j]`.
- If `dp[i][j] == dp[i][j-1] + 1`, the step is an **insertion into `a`** (a `'-'` in `a`'s column). Move left to `dp[i][j-1]`.

I emit each step as a `(char_a, char_b)` pair as I walk, then reverse the list at the end so it reads in forward order.

```python
def align(a: str, b: str) -> list[tuple[str, str]]:
    """Edit-distance alignment as a list of (char_a, char_b) pairs.
    '-' is the gap marker. Reconstructed via backtracking through the DP
    table from dp[n][m] to dp[0][0]."""
    n, m = len(a), len(b)
    dp = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(n + 1):
        dp[i][0] = i
    for j in range(m + 1):
        dp[0][j] = j
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
```

```
            else:
                dp[i][j] = 1 + min(
                    dp[i - 1][j],
                    dp[i][j - 1],
                    dp[i - 1][j - 1],
                )

    # Backtrack to reconstruct alignment
    alignment: list[tuple[str, str]] = []
    i, j = n, m
    while i > 0 or j > 0:
        if i > 0 and j > 0 and a[i - 1] == b[j - 1]:
            alignment.append((a[i - 1], b[j - 1]))
            i -= 1
            j -= 1
        elif i > 0 and j > 0 and dp[i][j] == dp[i - 1][j - 1] + 1:
            alignment.append((a[i - 1], b[j - 1]))  # substitute
            i -= 1
            j -= 1
        elif i > 0 and dp[i][j] == dp[i - 1][j] + 1:
            alignment.append((a[i - 1], "-"))  # delete from a
            i -= 1
        else:
            alignment.append(("-", b[j - 1]))  # insert into a (delete from b)
            j -= 1
    alignment.reverse()
    return alignment
```

The function rebuilds the DP table from scratch — the same lines as `levenshtein`. You'll notice the table-fill code is duplicated verbatim between the two functions. I keep it that way deliberately: each function should stand on its own as a self-contained algorithm you can read top-to-bottom, and tucking the table-fill behind a shared helper would force you to flip between three functions to understand `align`. Explicit duplication wins on pedagogical clarity here, and the cost is a few extra lines of identical Python — Python will define each function fresh when `illiterate` appends the blocks to the same module.

The backtracking loop terminates when both `i` and `j` reach zero — every cell on the way back to `dp[0][0]` consumes either a character from `a`, a character from `b`, or both. The four branches inside the loop correspond to the four edit possibilities I sketched above, with match handled separately from substitution so the match's zero cost is recorded correctly. The trailing `alignment.reverse()` flips the path back to forward order.

Running it on `('kitten', 'sitting')` and printing each pair with a label:

```
from codex.strings.edit_distance import align

def print_alignment(a: str, b: str) -> None:
    pairs = align(a, b)
    for ca, cb in pairs:
        if ca == "-":
            label = "insert"
        elif cb == "-":
            label = "delete"
        elif ca == cb:
            label = "match"
        else:
            label = "substitute"
        print(f"  {ca} | {cb}   {label}")

print_alignment("kitten", "sitting")
```

```
  k | s   substitute
```

```
  i | i   match
  t | t   match
  t | t   match
  e | i   substitute
  n | n   match
  - | g   insert
```

Seven columns: substitute `k`→`s`, three matches (`i`, `t`, `t`), substitute `e`→`i`, match (`n`), insert `g`. Three non-match operations total — the same 3 the distance function gave me. The alignment lets you *see* which characters survived and which got rewritten.

Some pairs have multiple optimal alignments — `('abc', 'cba')` could be solved with three substitutions or with three deletions plus three insertions, both costing 3 substitutions' worth or 3 indels' worth in different unit-cost models. My function returns *one* optimal alignment, the one the backtracking branches happen to prefer (match > substitute > delete > insert in tie-breaking). Other implementations make different choices; for the chapter's purposes, *some* optimal alignment is enough.

30.6 Subsequences and substrings

Edit distance is one specific question about string similarity, but the same grid-of-cells engine answers a small family of related questions. Two of them — **longest common subsequence** (LCS) and **longest common substring** (LCSS) — are worth seeing here, both because they show how the engine generalizes and because they help fix the difference between "subsequence" and "substring" in your head.

A **subsequence** of a string is what you get by deleting zero or more characters without reordering the rest. So `"ace"` is a subsequence of `"abcde"` (delete `b` and `d`), as is `"bd"` and `"abcd"` and the empty string. A subsequence doesn't have to be contiguous in the source string. A **substring**, in contrast, is a contiguous slice — `"bcd"` is a substring of `"abcde"`, but `"ace"` is not.

The **longest common subsequence** of two strings is the longest string that's a subsequence of both. For `"AGCAT"` and `"GAC"`, possible common subsequences include `"AC"`, `"GC"`, `"GA"` — all of length 2 — and the answer can be any of them depending on tie-breaking.

The recurrence for LCS is structurally similar to Levenshtein, but the values count *agreements* instead of *edits*, so `dp[i][j]` is the length of the LCS of `a[:i]` and `b[:j]`:

- If `a[i-1] == b[j-1]`, the last characters agree, so they extend the LCS: `dp[i][j] = dp[i-1][j-1] + 1`.
- Otherwise, the last characters can't both be part of the LCS, so I throw one away: `dp[i][j] = max(dp[i-1][j], dp[i][j-1])`.

Backtracking reconstructs the actual subsequence: walk from `dp[n][m]` to `dp[0][0]`, prepending `a[i-1]` whenever the diagonal-up move was a "match" extension, and following whichever in-edge yielded the max otherwise.

```python
def lcs(a: str, b: str) -> str:
    """Longest common subsequence - a string that is a subsequence of both
    a and b, of maximum length. Returns one such LCS (there can be many)."""
    n, m = len(a), len(b)
    dp = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
    # Backtrack
    result: list[str] = []
    i, j = n, m
    while i > 0 and j > 0:
        if a[i - 1] == b[j - 1]:
            result.append(a[i - 1])
```

```
            i -= 1
            j -= 1
        elif dp[i - 1][j] >= dp[i][j - 1]:
            i -= 1
        else:
            j -= 1
    return "".join(reversed(result))
```

The base cases are implicit — the table is initialized to zero, which is the right answer when either prefix is empty. The interior loop is the recurrence above. The backtracking loop emits matched characters and otherwise moves toward the larger of the two in-edges; ties go upward (`dp[i-1][j] >= dp[i][j-1]`), but any consistent tie-break works.

```
from codex.strings.edit_distance import lcs

print(f"lcs('AGCAT', 'GAC') = {lcs('AGCAT', 'GAC')!r}")
print(f"lcs('ABCBDAB', 'BDCAB') = {lcs('ABCBDAB', 'BDCAB')!r}")
print(f"lcs('hello', 'world') = {lcs('hello', 'world')!r}")
print(f"lcs('identical', 'identical') = {lcs('identical', 'identical')!r}")
```

```
lcs('AGCAT', 'GAC') = 'AC'
lcs('ABCBDAB', 'BDCAB') = 'BCAB'
lcs('hello', 'world') = 'l'
lcs('identical', 'identical') = 'identical'
```

`lcs('AGCAT', 'GAC')` returns `'GA'` of length 2 — my tie-breaking happened to prefer that one over `'AC'` and `'GC'`. `lcs('ABCBDAB', 'BDCAB')` returns `'BDAB'` of length 4. `'hello'` and `'world'` share the single letter `'o'`, length 1. Identical strings have themselves as their LCS.

LCS shows up in `diff` (the longest common subsequence of two files is the "unchanged" part, and everything else is insertions and deletions), in bioinformatics (gap-tolerant similarity scoring), and in version-control merge algorithms. It's also closely related to edit distance: if substitutions are *not* allowed and only insertions and deletions count, the edit distance between a and b is $n + m - 2 \cdot \text{lcs}(a, b)$ — the characters that don't survive in the LCS are the ones you have to delete from a or insert into b.

The **longest common substring** is the contiguous-slice version. The DP is again structurally similar, but the recurrence resets to zero whenever there's a mismatch — a common substring can't span a gap.

```
def lcss(a: str, b: str) -> str:
    """Longest common SUBSTRING - a contiguous run of characters appearing
    in both a and b, of maximum length."""
    n, m = len(a), len(b)
    if n == 0 or m == 0:
        return ""
    dp = [[0] * (m + 1) for _ in range(n + 1)]
    best_len = 0
    best_end_in_a = 0
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
                if dp[i][j] > best_len:
                    best_len = dp[i][j]
                    best_end_in_a = i
    return a[best_end_in_a - best_len:best_end_in_a]
```

`dp[i][j]` now means "the length of the longest common substring ending at `a[i-1]` and `b[j-1]`" — note "ending at," not "of `a[:i]` and `b[:j]`." That's a different per-cell invariant from LCS. Matches extend the run from the diagonal predecessor; mismatches leave `dp[i][j]` at zero (the table's default), because a mismatch means no common substring ends here. I track the overall maximum across all cells as I fill the table, and at the end I slice it out of `a` using the recorded end-position and length.

```
from codex.strings.edit_distance import lcss

print(f"lcss('introduction', 'education') = {lcss('introduction', 'education')!r}")
print(f"lcss('abcdef', 'xyzabcw') = {lcss('abcdef', 'xyzabcw')!r}")
print(f"lcss('hello', 'world') = {lcss('hello', 'world')!r}")
```

```
lcss('introduction', 'education') = 'tion'
lcss('abcdef', 'xyzabcw') = 'abc'
lcss('hello', 'world') = 'l'
```

`introduction` and `education` share `'tion'` — four contiguous characters at the end of `introduction` (positions 8–11) and at the end of `education` (positions 5–8). Spelling both out: `i-n-t-r-o-d-u-c-t-i-o-n` and `e-d-u-c-a-t-i-o-n` both end in `tion`. They share `duc` and `tion` separately, but not as one run — between `duc` and `tion`, `introduction` has a `t` while `education` has an `a`, so the contiguous match breaks. `abcdef` and `xyzabcw` share `abc` of length 3. `hello` and `world` share a single character `'l'` (the same column-3 match Hamming flagged earlier).

One nice connection to the previous chapters' pattern-matching work: if `pattern` appears as a substring of `text`, then `lcss(text, pattern) == pattern`, because the entire pattern is the longest possible common substring. The substring-search problem from Part IV is a degenerate case of longest-common-substring where one of the two strings is the pattern and the answer either is or isn't the pattern itself.

```
from codex.strings.edit_distance import lcss

text = "the rain in spain"
print(f"lcss({text!r}, 'rain')    = {lcss(text, 'rain')!r}    (pattern is in text)")
print(f"lcss({text!r}, 'snow')    = {lcss(text, 'snow')!r}    (pattern is NOT in text)")
print(f"lcss({text!r}, 'rainbow') = {lcss(text, 'rainbow')!r}    (partial match)")
```

```
lcss('the rain in spain', 'rain')    = 'rain'    (pattern is in text)
lcss('the rain in spain', 'snow')    = 'n'    (pattern is NOT in text)
lcss('the rain in spain', 'rainbow') = 'rain'    (partial match)
```

When the pattern is present, the LCSS is the pattern itself — exact substring search is the special case where the longest common substring has the pattern's full length. When the pattern is absent, the LCSS is the *longest prefix-or-substring of the pattern* that does occur, which is sometimes useful as a fuzzy fallback ("you searched for `rainbow` but I found `rain`"). The orientation between exact and approximate matching falls out cleanly from the LCSS perspective.

30.7 A bridge to Part VI

Every algorithm I've written in this chapter — `levenshtein`, `align`, `lcs`, `lcss` — has the same structural shape: allocate a 2D table, fill it cell-by-cell with a recurrence that references a small fixed set of neighbors, read the answer off the table (sometimes also backtracking through it). That shape is **dynamic programming**, and Part VI will give it the name and the theory.

For now, two properties are what matters, because they're what justifies the technique:

Optimal substructure. The edit distance between two full strings depends, via the recurrence, on the edit distances between their prefixes. The optimal answer at `dp[i][j]` is composed from optimal answers at three predecessor cells. If the predecessors had been suboptimal, the recurrence would not have produced an optimal `dp[i][j]` — but they're not, because the same recurrence makes them optimal in turn, all the way down to the base cases. The optimal solution to the big problem is built out of optimal solutions to smaller subproblems of the same shape. That's the property called "optimal substructure," and it's the structural prerequisite for any DP formulation.

Overlapping subproblems. The cell `dp[i][j]` is referenced by `dp[i+1][j]`, `dp[i][j+1]`, and `dp[i+1][j+1]` — three different successors all want to read the same value. A naive recursive computation would re-derive `dp[i][j]` once per reference, leading to exponential blowup; the DP approach materializes each cell once, in

a table, and lets every successor consume the cached value. The exponential collapse to polynomial that DP produces is exactly because the subproblems overlap and the table eliminates re-computation. If the subproblems didn't overlap — if each was referenced exactly once — there'd be no benefit over plain recursion, and the technique wouldn't earn its name.

Part VI's landing page will name those two properties, generalize them, and walk through more DP problems (knapsack, matrix-chain multiplication, longest increasing subsequence, all-pairs shortest paths) whose structure mirrors the edit-distance recurrence. For now, log the pattern: optimal substructure plus overlapping subproblems equals DP. Edit distance is the first textbook example for a reason.

30.8 The three questions, applied

Is it correct?

The correctness argument for `levenshtein` is induction on the size of the subproblem — specifically, on $i + j$. The base case is the row and column of zero-indexed entries: `dp[i][0] = i` and `dp[0][j] = j`. Both are correct by direct argument — transforming a length-i string to the empty string requires i deletions, and transforming the empty string to a length-j string requires j insertions, and there's no cheaper edit sequence.

The inductive step is the recurrence itself. Assume `dp[i'][j']` is correct for every (i', j') with $i' + j' < i + j$. I want to show that `dp[i][j]`, computed by the recurrence, is also correct — i.e., equal to the true edit distance between `a[:i]` and `b[:j]`. The argument splits on the last edit in some optimal edit sequence. Every edit sequence that transforms `a[:i]` to `b[:j]` ends in either a match (which exists only if `a[i-1] == b[j-1]`), a substitution, an insertion, or a deletion. In each case, removing the last edit leaves a shorter edit sequence on smaller prefixes, whose cost is bounded below by the inductive hypothesis. Taking the minimum across the four cases — match takes priority when the characters agree, because it costs zero rather than one — gives the recurrence. The minimum is achieved by some choice, so the recurrence is *tight*, not just an upper bound.

`align`'s correctness rides on `levenshtein`'s. The DP table is the same, so the cell values are correct. The backtracking walk reads off, for each cell visited, *one* of the predecessors that achieved the cell's value — exactly the operation that produced the optimal answer at that cell. The resulting sequence of operations, in reverse, is therefore an optimal edit sequence between the two full strings.

`lcs` and `lcss` are inductions on the same shape with different recurrences. For LCS: the longest common subsequence either does or doesn't include the last character of each prefix. If both last characters agree and the LCS uses them, the LCS extends a smaller LCS by one; if the last characters don't agree or aren't both used, the LCS is the better of dropping one or the other. For LCSS: a common substring ends at `(i, j)` only if `a[i-1] == b[j-1]` and the previous characters formed a (possibly empty) common substring ending one position back; the global maximum scans all cells.

How efficient is it?

The cost table:

Function	Time	Extra space
`hamming(a, b)`	$O(n)$	$O(1)$
`levenshtein(a, b)`	$O(nm)$	$O(nm)$
`align(a, b)`	$O(nm)$	$O(nm)$
`lcs(a, b)`	$O(nm)$	$O(nm)$
`lcss(a, b)`	$O(nm)$	$O(nm)$

Each cell in the $(n+1) \times (m+1)$ table is computed in $O(1)$ work (a min or max over a fixed-size constant number of predecessors), and there are $\Theta(nm)$ cells, so the per-function time and space are both $\Theta(nm)$. Bilingually: for two strings of length 1000 apiece, each function does about 10^6 cell-fills — fraction of a second in Python, much less in C. For strings of length 10^5, that's 10^{10} — already a serious computation, around a minute in Python and seconds in C; for strings of length 10^6, 10^{12} — out of reach for the dense $O(nm)$ approach, and you'd want to reach for one of the algorithmic improvements I mention below.

There's an easy space optimization I'll point at but not implement. Since `dp[i][j]` depends only on row i and row $i-1$, you can throw away every row before the most recent two and bring space down to $O(\min(n, m))$ — keep two rolling arrays and overwrite the older one as you walk forward. This works for the *distance* computation (`levenshtein`, the length output of `lcs`, the length output of `lcss`), but **not** for alignment reconstruction (`align` and the actual-string outputs of `lcs` and `lcss`), because backtracking needs the full table. **Hirschberg's algorithm** (1975) is the clever trick that achieves both at once: it recovers the alignment in $O(\min(n, m))$ space by a divide-and-conquer scheme that recomputes the DP twice in halves rather than storing the full table. I'll point at Hirschberg in the Notes and leave the implementation for a reference.

Is it optimal?

Asking whether the $O(nm)$ bound is asymptotically optimal turns out to be one of the more interesting open questions in fine-grained complexity. The trivial lower bound is $\Omega(n + m)$ — you have to look at every character of both strings — and that bound is achievable for Hamming distance ($O(n)$ in equal-length strings) but seems to fall short of what edit distance requires.

For decades, no algorithm beat $O(nm/\log^2 n)$ asymptotically — modest log-factor improvements via various bit-packing tricks (the **Four Russians** technique from the 1970s), but nothing close to $O(n^{1.5})$ or $O(n\sqrt{m})$ or any other genuinely sub-quadratic bound. In **2014**, Backurs and Indyk showed (in their STOC paper "Edit Distance Cannot Be Computed in Strongly Subquadratic Time Unless SETH is False") that this isn't a failure of imagination: under the **Strong Exponential Time Hypothesis** (SETH), no algorithm can compute edit distance in $O(n^{2-\varepsilon})$ time for any constant $\varepsilon > 0$. That's a conditional lower bound — it's contingent on SETH being true, which is widely believed but unproven — but it's strong enough that the field treats $O(n^2)$ as essentially tight.

So my $O(nm)$ algorithm is, up to log factors, optimal under SETH. The gap between "best known upper bound" and "SETH lower bound" is just polylogarithmic, and closing it isn't expected. If you want better than quadratic for edit distance, you have to either restrict the input (small alphabet, bounded distance) or give up on the exact answer (approximation algorithms — Andoni and Onak's 2009 result gives a $(\log n)^{O(1/\varepsilon)}$ approximation in $n^{1+\varepsilon}$ time, and more recent work pushes this further). For exact edit distance on arbitrary inputs, $O(nm)$ is the wall, and the algorithm in this chapter sits at it.

LCS shares the same conditional lower bound (Abboud et al., 2015), for essentially the same reason — there's a reduction between the two problems that preserves complexity. LCSS is asymptotically easier — Knuth-Morris-Pratt and suffix-tree approaches achieve $O(n + m)$ for the exact-substring-search special case, and suffix automaton or generalized suffix tree achieves $O(n+m)$ for the general LCSS too. My $O(nm)$ DP for LCSS is *not* optimal; I'm presenting the DP version because it sits in the same family as the others and the implementation parallels Levenshtein cleanly. For production LCSS on long strings, reach for the suffix-automaton approach from a string-algorithms library.

30.9 What edit distance teaches you

Three lessons from this chapter, and one structural observation about where the chapter sits in the book.

First, when a problem decomposes over prefixes, build a grid. Edit distance is a question about whole strings, but the recurrence is about *prefixes* of those strings — the answer for `(a, b)` depends on the answers for shorter prefix pairs. Once you've spotted that prefix decomposition, the grid follows mechanically: one axis per string, one cell per prefix pair, and the recurrence fills the grid bottom-up. This pattern shows up well beyond strings — knapsack decomposes over item-count prefixes, matrix chain multiplication decomposes over subchain endpoints, longest increasing subsequence decomposes over array prefixes. Whenever you can express "the answer for the whole input" as "the answer for a smaller input, combined with a local decision," you've found a grid-of-cells problem.

Second, alignment is a path through the grid, not just a number at the corner. The distance is a single cell value, but the *alignment* — the actual sequence of edits — is a path from the top-left corner of the grid to the bottom-right, with each diagonal step a match-or-substitute, each down step a deletion, each right step an insertion. That path-through-a-grid framing extends beyond edit distance: Needleman-Wunsch (global biological alignment) and Smith-Waterman (local biological alignment) are explicitly described in those terms — "path through the alignment matrix" is the standard vocabulary, and the optimal path is the one with the highest cumulative score under whatever scoring function the application uses. Substitution-cost matrices (BLOSUM, PAM), affine gap penalties, anti-diagonal sweeps — all of those are variants on the same grid-and-path framing.

Third, every chapter in this part has been DP without naming it. I won't belabor this — Part VI will do the naming — but it's worth noting that the suffix-array construction in chapter 29 was DP (prefix-doubling fills a rank table from a smaller one), Kasai's LCP was DP (each iteration uses the carry-over from the previous), KMP's failure function was DP (each entry is computed from a smaller one). The 2D grid here is the first textbook *example* of DP because the two-axis decomposition is so visually clear, but the pattern has been with you since Part IV opened. Part VI will pull the pattern up to the top level and run it across half a dozen more problems.

The chapter sits at the closing edge of Part IV — string algorithms in their pattern-matching shape (chapters 23–28) and their indexing shape (chapter 29), now extended to the similarity-measurement shape that doesn't ask "where does this occur?" but "how close are two things?" The grid-of-cells is the bridge object: it's the right tool for similarity measurement, and it's the bridge into Part VI's broader DP territory.

30.10 Notes and further reading

The Levenshtein distance was introduced by Vladimir Levenshtein in his 1965 paper "Binary codes capable of correcting deletions, insertions, and reversals" in *Doklady Akademii Nauk SSSR* 163(4):845–848, translated to English the following year in *Soviet Physics Doklady* 10(8):707–710. Levenshtein's motivation was coding theory — the same concern Hamming had a decade and a half earlier, extended from substitutions to insertions and deletions — and his paper proves bounds on error-correcting codes under the extended distance metric.

The dynamic-programming algorithm for computing Levenshtein distance was published by Robert A. Wagner and Michael J. Fischer in 1974, "The String-to-String Correction Problem" in *Journal of the ACM* 21(1):168–173. Wagner and Fischer's paper is the canonical reference for the algorithm I've implemented here, and the algorithm is often called the **Wagner-Fischer algorithm** in the literature. The paper also handles arbitrary edit costs (rather than the unit-cost model I've used), and extends naturally to the weighted edit distance you find in biological scoring schemes.

Hamming distance is from Richard Hamming's 1950 paper "Error detecting and error correcting codes" in *Bell System Technical Journal* 29(2):147–160 — one of the foundational papers of coding theory, well worth reading in its own right.

For biological sequence alignment, the two essential papers are Saul B. Needleman and Christian D. Wunsch's 1970 *Journal of Molecular Biology* paper "A general method applicable to the search for similarities in the amino acid sequence of two proteins" — global alignment, which is the direct biological analog of `levenshtein` and `align` — and Temple F. Smith and Michael S. Waterman's 1981 *Journal of Molecular Biology* paper "Identification of common molecular subsequences" — local alignment, which finds the highest-scoring aligned region between two sequences rather than the whole alignment, more like `lcss` than `levenshtein`. Both papers are short and readable, and their algorithms are the foundation of modern bioinformatics tools.

The $O(\min(n, m))$ space optimization for edit-distance alignment is Daniel S. Hirschberg's 1975 *Communications of the ACM* paper "A linear space algorithm for computing maximal common subsequences." Hirschberg's trick is the divide-and-conquer scheme I gestured at — compute the middle column of the DP table from both ends, find the optimal split point, and recurse on the two halves — and the same idea applies to edit distance and other prefix-decomposable DPs.

The SETH-conditional quadratic lower bound is Arturs Backurs and Piotr Indyk's 2015 STOC paper "Edit Distance Cannot Be Computed in Strongly Subquadratic Time (unless SETH is false)" — the paper title says it all. The LCS analog is Amir Abboud, Arturs Backurs, and Virginia Vassilevska Williams's 2015 FOCS paper "Tight Hardness Results for LCS and other Sequence Similarity Measures," which extends the same conditional lower bound to LCS and several related problems. For lighter reading on the fine-grained complexity landscape, Vassilevska Williams's 2018 survey "On Some Fine-grained Questions in Algorithms and Complexity" is a good entry point.

CLRS covers edit distance briefly in §15.5 (in the 3rd edition), under the section on optimal-substructure problems. Sedgewick and Wayne's *Algorithms* (4th ed.) doesn't cover edit distance in detail, but their substring-search material in chapter 5 is the right neighbor. The textbook reference for the DP pattern itself — which is what I'll cite when Part VI opens — is Bertsekas's *Dynamic Programming and Optimal Control*, but for the discrete combinatorial version most directly relevant to this book, Kleinberg and Tardos's *Algorithm Design* chapter 6 is the cleanest pedagogical treatment.

This chapter closes Part IV. The next file is `lessons_4.md`, where I step back across all of chapters 23–30 and name the patterns that recurred — pattern-matching as a problem family, the trade between preprocessing

patterns and preprocessing text, the role of automaton and array structures, and the path-through-a-grid framing that opens onto Part VI's territory.

Lessons from Part IV

Eight chapters about strings, and almost none of the cleverness lived in the algorithms themselves. The cleverness lived in *representation*. Every chapter from 24 onward was the same conceptual object — a way of answering the question "have I matched a prefix of the pattern against a recent suffix of the text?" — stored in a different shape with a different access pattern. Knuth-Morris-Pratt is a deterministic automaton compressed to one integer per state. Boyer-Moore is two tables that encode the same question read in the opposite scanning direction. Rabin-Karp is the same question filtered through a rolling fingerprint. Aho-Corasick is the same question generalized from one pattern to a set, with the failure function reshaped into a tree. Suffix arrays flip the entire substrate — preprocess the *text* instead of the pattern — and answer the question for arbitrary future patterns instead of just one. Edit distance closes the part by changing what the question even means. The algorithms got their names from their inventors, but their shapes are mostly variations on a few ideas.

The patterns that kept showing up

Preprocessing-versus-query as an organizing axis. Every chapter sat at a different point on the same trade-off curve. The brute force in chapter 23 did zero preprocessing and paid for it with an $O(nm)$ worst case. The DFA in chapter 24 preprocessed the pattern into a $\Theta(m \cdot |\Sigma|)$ table and bought $\Theta(n)$ scanning. KMP compressed the same preprocessing to $\Theta(m)$ with the same scan time. Boyer-Moore preprocessed two tables but earned sublinear best-case scanning by reading windows right-to-left. Rabin-Karp preprocessed the pattern's hash and rolled a fingerprint forward in $O(1)$ per shift. Aho-Corasick preprocessed a set of patterns into a single trie-with-failure-links and answered multi-pattern queries in one scan. Suffix arrays moved the preprocessing entirely onto the text, which lets you reuse the index across arbitrarily many future pattern queries. Edit distance preprocessed nothing and accepted $O(nm)$ on every call because the question itself doesn't admit shortcuts. The chapters together are a map of the trade-off — every point on the preprocessing-versus-query plane is occupied by some chapter, and choosing among them is an engineering question about the workload, not an open theoretical question.

Automata as the universal lens. The deterministic finite automaton from chapter 24 is the conceptual ancestor of half the part. KMP's failure function is the same automaton with sparse storage — chapter 25's punchline reconstructed every cell of the DFA from the failure pointers alone, proving the compression is lossless. Aho-Corasick is the same automaton lifted from a single pattern to a set, with the failure function generalized from a one-dimensional array to a tree. Even Boyer-Moore's good-suffix rule has an automaton-shaped formal explanation if you squint at the precomputed shift table the right way. Pattern matching *is* automaton design; the choice of algorithm is mostly a choice of how to compress the automaton.

Randomization as a cost-shape strategy. The rolling hash in chapter 27 is the treap argument applied to strings. The adversary controls the input — the text, the patterns, the timing — but you control the hash function's parameters, and a random choice of modulus makes the worst case statistically unreachable. The pattern will recur in Part IX (skip lists, randomized incremental algorithms) and in cryptography wherever someone needs guarantees against an adversary they can't see. When you find yourself worrying about a worst case an attacker could construct, ask whether you can make the worst case depend on a coin you flip rather than on the input the attacker picks.

Indexing the input shifts what's expensive. Chapters 23 through 28 all preprocessed the *pattern* and scanned the text. Chapter 29 broke that — it preprocessed the *text* into a suffix array and answered an arbitrary pattern query in $O(m \log n)$. The flip is one of the deepest moves in algorithm design. Database indexes preprocess the rows; inverted indexes in search engines preprocess the documents; spatial indexes in computational geometry preprocess the points. The query side gets cheap because the preprocessing did the work once. It shows up again in Part VIII when geometric algorithms preprocess point sets into k-d trees.

Restriction is power. Hamming distance only works on equal-length strings, but inside that restriction it's a single $O(n)$ scan — no table, no allocation. Levenshtein generalizes to arbitrary string pairs and pays $O(nm)$ for the privilege. Aho-Corasick's restriction to *fixed strings* is what lets it match thousands of patterns in one pass; extending it to regex would be PCRE territory, an entirely different and much heavier engine. Boyer-Moore's

restriction to *exact match* is what gives it the bad-character rule. Every chapter in the part bought its cost shape by accepting a restriction; lifting the restriction would lose the win. Discipline buys cost shape, and the deeper into the book you go the more often the right answer to a performance question is "what restriction can I accept that would let me deliver the shape I want?"

Meta-lessons

The obvious algorithm is the baseline you have to beat. The brute force in chapter 23 was the algorithm with no clever ideas, and yet it's what production code most often ships. CPython's `str.find`, GNU `memmem`, Rust's `str::find` — all of them are brute force with a small skip table on top, not KMP or Boyer-Moore or any of the cleverer algorithms. The reason is that brute force has a tight inner loop with no branches, which runs faster on modern hardware than any algorithm whose inner loop consults a precomputed table. Asymptotic optimality is a different question from production-code optimality, and when the average case is benign the brute force usually wins on constants.

Representation can collapse cost. Chapter 25's KMP stored the DFA from chapter 24 in one integer per state instead of one entry per state-character pair, and the scan ran in the same asymptotic time on much less memory. Chapter 29's suffix array stored an index over every substring of a text in $O(n)$ space, where a suffix tree storing the same information would have used $O(n \cdot |\Sigma|)$. The same phenomenon appears in Part V when sparse graphs use adjacency lists instead of adjacency matrices, and in Part VII when persistent data structures store version histories in $O(\log n)$ space instead of $O(n)$ per version. When you see a table whose entries collapse along an axis, ask whether a sparser encoding stores the same information with less overhead.

Scanning order is a design choice, not a default. Boyer-Moore made the case starkly. The exact same window of text, examined right-to-left instead of left-to-right, exposed a class of skips that left-to-right scanning structurally could not see. The choice determined whether the cost shape was $\Theta(n)$ or $\Theta(n/m)$. The pattern recurs whenever an algorithm consumes a window of input: the in-order traversal of a BST gives you sorted output; the pre-order traversal gives you serialization; the post-order traversal gives you bottom-up dependency satisfaction. Different orders expose different invariants.

Two conservative bounds combine by taking the max. Boyer-Moore's bad-character rule and good-suffix rule are independently correct and independently motivated. The matcher takes the maximum of both shifts on every mismatch, and the result is at least as fast as either heuristic alone on every input. The pattern recurs in branch-and-bound search (combine multiple admissible heuristics, take the max), in interval scheduling with multiple constraints (combine bounds, take the max), in linear programming relaxations (combine valid inequalities, intersect feasible regions). When you have two correct conservative estimates of the same quantity, their maximum is itself correct and usually strictly tighter than either one alone.

The same DP grid answers a family of related questions. Chapter 30's edit-distance grid was structurally identical to its longest-common-subsequence and longest-common-substring grids — the cells, the indexing, the bottom-up fill order. What changed was the per-cell recurrence and the post-fill backtracking. That observation is the seed of dynamic programming proper. DP isn't a single algorithm — it's a framework for problems with optimal substructure and overlapping subproblems, and the same grid-of-cells machinery handles dozens of distinct questions by varying the recurrence. Part VI develops the pattern.

What's next

Part V is a change of substrate. The next eight chapters drop the assumption that the data being walked is a one-dimensional sequence of characters and let the structure become a *graph* — nodes connected by edges, with no implicit left-or-right order, with cycles and multiple paths and arbitrary connectivity. Chapter 28's Aho-Corasick was already half a graph — a trie with failure links is a graph if you draw the failure edges as edges — and chapter 20's trie was a tree, which is a graph with no cycles. The leap into Part V is just letting the edges go anywhere.

The arc starts with the substrate. Graph representations — adjacency lists, adjacency matrices, edge lists, implicit graphs — name the trade-offs. DFS and BFS are the universal traversals, the way pre-order and level-order were for trees in Part III. Connectivity, articulation points, topological sort, strongly connected components are the structural questions you can ask in linear time over the graph. The middle of Part V is where the shortest-path canon lives — Dijkstra, Bellman-Ford, Floyd-Warshall, A*-style heuristic search —

and where the flow algorithms enter the book. The closing chapters cover NP-hard graph problems and the practical-vs-theoretical gap that defines the rest of the book.

The Part IV atoms keep paying out. KMP's failure function reappears in Aho-Corasick, which reappears in Part V as the basis for finite-state speech recognition. The suffix-array trick reappears as the FM-index for biological sequence alignment. The DP grid from chapter 30 generalizes to Part VI's entire toolkit. Hashing — chapter 12, then Rabin-Karp — reappears in Part VIII for geometric hashing and in Part IX for randomized graph algorithms.

Eight chapters, four questions, one part. Strings turned out to be where the book stops being about *operations on data* and starts being about *representations of structure*. Onward.

Part V

Graphs

The trie at the end of Part IV was already a graph. We followed the edges of a tree-with-failure-links to scan a text in a single pass, and the structure that made the algorithm work was just a set of nodes with directed edges between them. Part V lets the edges go anywhere — no requirement that the structure be a tree, no constraint that the relations point only at children. Graphs are what you reach for when the structure is *any relation on any set of things*, and the rest of this part is about what that level of generality buys you and what it costs.

The eight chapters move from the most basic question — *how do I even store one of these?* — through traversal, shortest paths, spanning trees, ordering, and finally to flow.

- **Representations of graphs** (chapter 31) sets the substrate. Adjacency list, adjacency matrix, edge list — three encodings of the same object, each cheap for different questions. The choice runs through every later chapter.
- **Breadth-first traversal** (chapter 32) is the simplest graph algorithm. It introduces the *layered frontier* — the set of vertices at distance k from the source — and as a side effect it solves shortest paths in unweighted graphs.
- **Depth-first traversal** (chapter 33) is the swiss-army knife. The same recursion that walks the graph also exposes its structure: connected components, cycles, the discovery and finish times that later chapters lean on heavily.
- **Dijkstra's algorithm** (chapter 34) tackles weighted shortest paths under the restriction that all edge weights are non-negative. The chapter is also where the heap from chapter 19 finally earns its keep.
- **Relaxation under negative weights** (chapter 35) lifts that restriction. Bellman-Ford handles negative edges and detects negative cycles; Floyd-Warshall pushes the same idea to all-pairs. Both are *relaxation* algorithms, and the chapter makes the unifying pattern explicit.
- **Minimum spanning trees** (chapter 36) leans on union-find from chapter 14 and the heap from chapter 19. Kruskal and Prim are two reading orders of the same proof — the **cut property** — and the chapter develops both.
- **Topological sort and strong components** (chapter 37) returns to DFS and squeezes more out of it. Kahn's algorithm orders the vertices of a DAG; Tarjan's algorithm finds the strongly connected components in a single DFS pass.
- **Flow networks** (chapter 38) closes the part with the deepest result. Ford-Fulkerson augments paths until no augmenting path remains, Edmonds-Karp shows that picking the shortest augmenting path bounds the running time, and the max-flow min-cut theorem connects the algorithmic question to its LP-dual.

Three patterns recur through the part. **Representation shapes cost** — the adjacency list vs. matrix choice isn't storage hygiene, it determines which questions are $O(1)$ and which are $O(n)$. **Relaxation unifies the shortest-path family** — Dijkstra, Bellman-Ford, Floyd-Warshall, and Prim's MST are all maintaining an upper-bound estimate and repeatedly tightening it; the difference is the order they pick edges. And **DFS is the lens for structural questions** — connectivity, cycles, ordering, components, bridges — every one of them is DFS plus a small bookkeeping tweak.

Part V leaves three families of graph problems for later parts. **Matching and coloring** are decision problems whose hard cases are NP-complete; they fit naturally in Part VIII. **PageRank, random walks, and community detection** are applied; they belong in Part VII's specialized domains. And **dynamic programming on graphs** — longest path on a DAG, vertex cover on a tree, the SCC-condensation reduction — gets its own treatment in Part VI, building on the structures developed here.

31 Three ways to write the same graph

A graph is a set of vertices and a set of edges between them, and the second sentence of every chapter in this part depends on which of three encodings of that pair you have in hand. The trie at the end of Part IV was already a graph — a set of nodes plus the parent-child edges between them — and Aho-Corasick added a second set of edges (the fail-links) over the same vertex set. Both algorithms worked because the structure could answer one question quickly: *given a node, what are its outgoing edges?* Chapter 32's breadth-first search will ask the same question, billions of times if the graph is large enough, and the answer's cost depends entirely on how the graph is stored. Two graphs with the same vertices and the same edges can be represented in ways that differ by orders of magnitude in space and by orders of magnitude in per-query time. **Representation is not a storage choice — it is a cost-shape choice, and it propagates through every algorithm built on top of it.**

By the end of this chapter you will have implemented three concrete representations of a graph (edge list, adjacency matrix, adjacency list), seen the same six-vertex demo graph encoded in all three, watched the per-operation cost shape change as the representation changes, and met a fourth shape — the *implicit graph* — that stores no edges at all and computes them on demand. The rest of Part V leans on the adjacency-list class built here as its default, picks the matrix when density justifies it, and reaches for implicit-graph code when the vertex set is huge and the edges are derivable.

31.1 Vertices, edges, and four combinations

A graph $G = (V, E)$ is a finite set of *vertices* V paired with a finite set of *edges* E, where each edge is either an unordered pair $\{u, v\}$ of vertices (*undirected*) or an ordered pair (u, v) (*directed*). When an edge carries a number attached to it — a cost, a distance, a capacity, a time — the graph is *weighted*; otherwise it is *unweighted*, which is just the special case where every edge has weight 1.

Four combinations, then. Directed-weighted graphs are the most general; the other three are restrictions. Dijkstra's algorithm in chapter 34 wants directed-weighted, but works on undirected graphs by treating each undirected edge $\{u, v\}$ as the pair of directed edges (u, v) and (v, u) with the same weight. Breadth-first search in chapter 32 wants directed-unweighted, but works on weighted graphs by ignoring the weights. The chapters use whichever subset they need, and the representations here support all four.

A few conventions I will hold to across the part. The number of vertices is n, the number of edges is m, and the symbols u, v, w denote vertices (the symbol w is also reused for *weight* in the rare lines where context makes the distinction obvious). The *neighbors* of a vertex u — written $N(u)$ — are the vertices v such that $(u, v) \in E$. The *degree* of u is $|N(u)|$. For directed graphs, *out-neighbors* and *in-neighbors* are distinguished when it matters; *neighbors* without qualification means out-neighbors. A graph is *sparse* when $m = O(n)$ or $m = O(n \log n)$; *dense* when $m = \Theta(n^2)$. Most real graphs are sparse — social networks, road networks, web graphs, citation graphs all have a small constant or a logarithmic factor more edges than vertices — and that fact is what tips the representation choice toward the adjacency list in most chapters.

31.2 One graph to run through all three

I want one graph that recurs across the part so the cost-shape comparisons are concrete. Six vertices labeled `"A"` through `"F"`, nine edges, weighted. The edges form a small mesh that is connected, has multiple paths between most vertex pairs, and has at least one bridge and at least one cycle.

edge	weight
A — B	5
A — C	3
B — C	2
B — D	6

edge	weight
C — D	1
C — E	4
D — E	2
D — F	3
E — F	5

I will use this graph undirected and weighted by default; chapters 33, 37, and 38 will reorient it as a directed or unweighted variant as they need.

31.3 Edge list: the rawest encoding

The simplest representation stores no structure beyond what was given: a list of triples (u, v, w), one per edge. That's it. No vertex set is stored separately — vertices are whichever labels appear in the edges. No adjacency information is precomputed. To find the neighbors of a vertex, you scan the whole list.

```
from collections.abc import Hashable, Iterable, Iterator


class EdgeList[V: Hashable]:
    """Graph stored as a flat list of (u, v, weight) edge triples.

    Cheap to build, cheap to enumerate edges, expensive to query.
    Best for Kruskal's MST (chapter 36), where the whole point is to
    sort the edges once and walk them in order.
    """

    def __init__(self, directed: bool = False) -> None:
        self._directed = directed
        self._edges: list[tuple[V, V, float]] = []
        self._vertices: set[V] = set()

    def add_edge(self, u: V, v: V, weight: float = 1.0) -> None:
        self._edges.append((u, v, weight))
        self._vertices.add(u)
        self._vertices.add(v)

    def __len__(self) -> int:
        return len(self._vertices)

    def vertices(self) -> Iterable[V]:
        return self._vertices

    def edges(self) -> Iterator[tuple[V, V, float]]:
        yield from self._edges

    def neighbors(self, u: V) -> Iterator[V]:
        for x, y, _ in self._edges:
            if x == u:
                yield y
            elif not self._directed and y == u:
                yield x
```

Four methods, one field that matters (`_edges`), and a derived set of vertices kept for the `vertices()` query. The directed flag controls whether undirected edges are walked in both directions during neighbor enumeration; the underlying edge list stores each undirected edge only once.

The cost shape is the point. Adding an edge is $O(1)$. Enumerating all edges is $O(m)$, which is optimal — you have to touch every edge once. Asking "does the edge (u, v) exist?" or "what are the neighbors of u?" both scan the whole edge list and cost $\Theta(m)$. That is bad for almost every algorithm in this part — BFS's inner loop asks for neighbors n times across the whole scan, so BFS on an edge-list-backed graph runs in $\Theta(nm)$ instead of $\Theta(n + m)$. A factor of n slowdown for picking the wrong representation. The edge list is the right call only when the algorithm itself wants to enumerate edges in some global order — Kruskal's MST in chapter 36 sorts the edge list by weight, then walks it; the global enumeration is the algorithm's loop, so an edge list pays for itself.

```
from codex.graphs.edge_list import EdgeList

g = EdgeList[str](directed=False)
for u, v, w in [("A", "B", 5), ("A", "C", 3), ("B", "C", 2), ("B", "D", 6),
                ("C", "D", 1), ("C", "E", 4), ("D", "E", 2), ("D", "F", 3),
                ("E", "F", 5)]:
    g.add_edge(u, v, w)

print(f"|V| = {len(g)}, |E| = {sum(1 for _ in g.edges())}")
print(f"neighbors of 'C': {sorted(g.neighbors('C'))}")
print(f"neighbors of 'F': {sorted(g.neighbors('F'))}")
```

```
|V| = 6, |E| = 9
neighbors of 'C': ['A', 'B', 'D', 'E']
neighbors of 'F': ['D', 'E']
```

Six vertices, nine edges. Vertex `C` has four neighbors (it's the most-connected vertex in this mesh); vertex `F` has two. The neighbor query scanned all nine edges twice — once for each call — even though `C` and `F` each touch only a few of them. That scan cost is what the next two representations eliminate.

31.4 Adjacency matrix: $O(1)$ edge queries, $\Theta(n^2)$ space

The adjacency matrix encodes the graph as a two-dimensional array. With n vertices labeled 0 through $n - 1$, the matrix is an $n \times n$ table where the entry at row u and column v holds the weight of the edge from u to v, or some sentinel meaning "no edge." For an undirected graph the matrix is symmetric: entry (u, v) equals entry (v, u). For an unweighted graph the entries are 0 or 1.

The strength of this representation is that the question "is there an edge from u to v?" is answered by a single array read — $O(1)$, no scan, no hash lookup. The weakness is that the matrix uses $\Theta(n^2)$ space whether the graph is dense or sparse. For a million-vertex sparse graph with five million edges, the adjacency-list representation needs $\Theta(n + m) = \Theta(6 \cdot 10^6)$ slots and the adjacency matrix needs $\Theta(n^2) = \Theta(10^{12})$ slots — a million-fold blow-up on a graph the list representation handles trivially.

I'm going to use `float('inf')` as the sentinel for "no edge" in the weighted case, because it composes well with weight comparisons: any real edge weight is less than infinity, so Dijkstra-style relaxation code can compare without a separate `has_edge` check. The user-facing methods accept arbitrary hashable vertex labels and translate them to row/column indices internally via a vertex-to-index map.

```
from collections.abc import Hashable, Iterable, Iterator, Sequence
from math import inf


class AdjacencyMatrix[V: Hashable]:
    """Graph stored as an n x n matrix of edge weights, with float('inf')
    for absent edges.

    O(1) edge queries, O(n^2) space. Use when the graph is dense
    (m = Theta(n^2)) or when the algorithm hammers has_edge / weight
    lookups, like Floyd-Warshall in chapter 35.
    """
```

```
    def __init__(self, vertices: Sequence[V], directed: bool = False) -> None:
        self._directed = directed
        self._index: dict[V, int] = {v: i for i, v in enumerate(vertices)}
        self._vertices: list[V] = list(vertices)
        n = len(vertices)
        self._matrix: list[list[float]] = [[inf] * n for _ in range(n)]
```

The constructor fixes the vertex set at construction time — once an `AdjacencyMatrix` is built, you can't add new vertices without rebuilding the matrix. That's a deliberate restriction: growing the matrix would mean re-allocating an n^2 table on every vertex insertion, and the resulting cost shape is bad enough that matrix-backed graphs simply don't support it. The adjacency list, which we'll build next, does.

```
    def __len__(self) -> int:
        return len(self._vertices)

    def vertices(self) -> Iterable[V]:
        return self._vertices

    def add_edge(self, u: V, v: V, weight: float = 1.0) -> None:
        i, j = self._index[u], self._index[v]
        self._matrix[i][j] = weight
        if not self._directed:
            self._matrix[j][i] = weight

    def has_edge(self, u: V, v: V) -> bool:
        return self._matrix[self._index[u]][self._index[v]] < inf

    def weight(self, u: V, v: V) -> float:
        return self._matrix[self._index[u]][self._index[v]]

    def neighbors(self, u: V) -> Iterator[V]:
        i = self._index[u]
        for j, w in enumerate(self._matrix[i]):
            if w < inf:
                yield self._vertices[j]

    def edges(self) -> Iterator[tuple[V, V, float]]:
        for i, row in enumerate(self._matrix):
            for j, w in enumerate(row):
                if w < inf and (self._directed or i <= j):
                    yield (self._vertices[i], self._vertices[j], w)
```

Six methods, all of them either $O(1)$ (single-cell reads and writes) or $O(n)$ (row scans for `neighbors`) or $O(n^2)$ (the full-matrix scan in `edges`). `neighbors(u)` walks an entire row of the matrix, including the absent edges — that's the inefficiency that sinks the adjacency matrix on sparse graphs. If the average vertex has 5 neighbors and n is a million, every `neighbors` call reads a million entries to find five of them. The adjacency-list `neighbors` reads exactly five.

```
from codex.graphs.adjacency_matrix import AdjacencyMatrix

vertices = ["A", "B", "C", "D", "E", "F"]
g = AdjacencyMatrix[str](vertices, directed=False)
for u, v, w in [("A", "B", 5), ("A", "C", 3), ("B", "C", 2), ("B", "D", 6),
                ("C", "D", 1), ("C", "E", 4), ("D", "E", 2), ("D", "F", 3),
                ("E", "F", 5)]:
    g.add_edge(u, v, w)

print(f"|V| = {len(g)}")
print(f"weight(C, D) = {g.weight('C', 'D')}  (single cell read, O(1))")
```

```
print(f"weight(A, F) = {g.weight('A', 'F')}  (no edge -> inf, also O(1))")
print(f"has_edge(B, E) = {g.has_edge('B', 'E')}  (B does not connect directly to E)")
print(f"neighbors of 'C' (sorted): {sorted(g.neighbors('C'))}")
print(f"neighbors of 'F' (sorted): {sorted(g.neighbors('F'))}")
```

```
|V| = 6
weight(C, D) = 1  (single cell read, O(1))
weight(A, F) = inf  (no edge -> inf, also O(1))
has_edge(B, E) = False  (B does not connect directly to E)
neighbors of 'C' (sorted): ['A', 'B', 'D', 'E']
neighbors of 'F' (sorted): ['D', 'E']
```

Same neighbors as the edge list — `C` connects to four vertices, `F` to two — but the cost shape underneath has flipped. `weight(C, D)` is now a single cell read instead of a nine-edge scan. The price is the $6 \times 6 = 36$ cells of storage, of which only 18 hold real edge data (nine weights, doubled because the graph is undirected) and the rest are `inf`. On a graph this small the matrix wastes nothing; on a million-vertex sparse graph it wastes essentially all of its memory.

Print it row by row and the shape shows itself.

```
print(f"adjacency matrix (rows/cols = {vertices}):")
print(f"        {'  '.join(vertices)}")
for v in vertices:
    row = [f"{g.weight(v, u):>3.0f}" if g.has_edge(v, u) else "  ."
           for u in vertices]
    print(f"   {v}   {' '.join(row)}")
```

```
adjacency matrix (rows/cols = ['A', 'B', 'C', 'D', 'E', 'F']):
        A  B  C  D  E  F
   A    .  5  3  .  .  .
   B    5  .  2  6  .  .
   C    3  2  .  1  4  .
   D    .  6  1  .  2  3
   E    .  .  4  2  .  5
   F    .  .  .  3  5  .
```

The symmetry across the diagonal is the undirected-graph signature; the `.` cells are the absent edges. For a directed graph the matrix would not be symmetric, and that asymmetry is exactly what makes the matrix representation useful for directed algorithms — `out_neighbors(u)` walks row u, `in_neighbors(u)` walks column u, and both questions are equally cheap.

31.5 Adjacency list: the workhorse

The adjacency list stores, for each vertex, the list (or dictionary) of its neighbors and their edge weights. It uses $\Theta(n + m)$ space — proportional to the actual graph, not to the matrix it could have been. Per-vertex queries are cheap: `neighbors(u)` returns exactly $|N(u)|$ vertices, no scanning of absent edges. Edge queries `has_edge(u, v)` require looking up v in u's adjacency container, which is $O(1)$ amortized if the container is a hash map, or $O(\log n)$ if it's a balanced tree. For the chapters in this part I'll use Python dictionaries, which gives the amortized $O(1)$ guarantee.

The internal storage is a `dict[V, dict[V, float]]` — the outer dictionary maps each vertex to its neighbor dictionary, and each neighbor dictionary maps a destination vertex to the edge weight. This nested-dictionary layout makes the four common queries all cheap: vertex enumeration walks the outer dictionary's keys, neighbor enumeration walks an inner dictionary's keys, edge-existence is an inner-dictionary `in` test, and weight lookup is an inner-dictionary access.

```
from collections.abc import Hashable, Iterable, Iterator


class AdjacencyList[V: Hashable]:
    """Graph stored as a dictionary mapping each vertex to a dictionary
    of (neighbor -> weight) entries.

    O(1) amortized for has_edge, weight, and add_edge. O(deg(u)) for
    neighbors(u). O(n + m) space. The default representation for almost
    every chapter in Part V.
    """

    def __init__(self, directed: bool = False) -> None:
        self._directed = directed
        self._adj: dict[V, dict[V, float]] = {}
```

One field of substance. The dictionary is created empty; vertices are added implicitly when an edge first mentions them, or explicitly via `add_vertex` for isolated vertices that have no edges yet.

```
    def add_vertex(self, v: V) -> None:
        if v not in self._adj:
            self._adj[v] = {}

    def add_edge(self, u: V, v: V, weight: float = 1.0) -> None:
        self.add_vertex(u)
        self.add_vertex(v)
        self._adj[u][v] = weight
        if not self._directed:
            self._adj[v][u] = weight

    def __len__(self) -> int:
        return len(self._adj)

    def vertices(self) -> Iterable[V]:
        return self._adj.keys()

    def has_edge(self, u: V, v: V) -> bool:
        return u in self._adj and v in self._adj[u]

    def weight(self, u: V, v: V) -> float:
        return self._adj[u][v]

    def degree(self, u: V) -> int:
        return len(self._adj[u])

    def neighbors(self, u: V) -> Iterator[V]:
        yield from self._adj[u].keys()

    def weighted_neighbors(self, u: V) -> Iterator[tuple[V, float]]:
        yield from self._adj[u].items()

    def edges(self) -> Iterator[tuple[V, V, float]]:
        seen: set[tuple[V, V]] = set()
        for u, nbrs in self._adj.items():
            for v, w in nbrs.items():
                if not self._directed and (v, u) in seen:
                    continue
                seen.add((u, v))
                yield (u, v, w)
```

Ten methods, all of them either $O(1)$ amortized or proportional to exactly the work the caller asked for. `neighbors(u)` yields $|N(u)|$ items in $O(|N(u)|)$ time, never more. `edges()` yields each undirected edge once by maintaining a `seen` set during the walk; for a directed graph the `seen` check is bypassed. The `weighted_neighbors(u)` method exists because Dijkstra's loop in chapter 34 wants both the neighbor and the edge weight in one pass, and constructing tuples from two dictionary lookups would be wasteful.

```
from codex.graphs.adjacency_list import AdjacencyList

g = AdjacencyList[str](directed=False)
for u, v, w in [("A", "B", 5), ("A", "C", 3), ("B", "C", 2), ("B", "D", 6),
                ("C", "D", 1), ("C", "E", 4), ("D", "E", 2), ("D", "F", 3),
                ("E", "F", 5)]:
    g.add_edge(u, v, w)

print(f"|V| = {len(g)}, |E| = {sum(1 for _ in g.edges())}")
print(f"degree of C: {g.degree('C')} (the hub)")
print(f"degree of F: {g.degree('F')} (a leaf-ish vertex)")
print()
print("neighborhood of C with weights:")
for v, w in sorted(g.weighted_neighbors("C")):
    print(f"   C -- {v}  (weight {w})")
```

```
|V| = 6, |E| = 9
degree of C: 4 (the hub)
degree of F: 2 (a leaf-ish vertex)

neighborhood of C with weights:
   C -- A  (weight 3)
   C -- B  (weight 2)
   C -- D  (weight 1)
   C -- E  (weight 4)
```

Same graph again, same answer. `C` has four neighbors, `F` has two. The internal storage is nine entries in the outer dictionary's inner dictionaries (one per undirected edge, doubled to eighteen because each undirected edge appears in two adjacency entries). Space proportional to the actual edge count, not to n^2.

31.6 When does each representation win?

The three representations have the same expressive power — every graph can be encoded in any of them, and the encodings can be translated into each other in $\Theta(n + m)$ time. They differ only in cost shape. The right summary table:

operation	edge list	adjacency matrix	adjacency list
space	$\Theta(m)$	$\Theta(n^2)$	$\Theta(n + m)$
add edge	$O(1)$	$O(1)$	$O(1)$ amortized
has_edge	$\Theta(m)$	$O(1)$	$O(1)$ amortized
neighbors(u)	$\Theta(m)$	$\Theta(n)$	$O(\deg u)$
iterate edges	$\Theta(m)$	$\Theta(n^2)$	$\Theta(n + m)$
degree(u)	$\Theta(m)$	$\Theta(n)$	$O(1)$

Read it as a forecast for the chapters ahead. BFS and DFS (chapters 32 and 33) hit `neighbors(u)` once per vertex during a traversal — total cost $\Theta(\sum_u \deg u) = \Theta(m)$ on an adjacency list, $\Theta(n^2)$ on a matrix. For a sparse graph with $m = O(n)$, the difference is a factor of n. Dijkstra's algorithm (chapter 34) calls `weighted_neighbors` once per vertex popped from the priority queue and gets the same shape. Floyd-Warshall (chapter 35) is the exception — its $\Theta(n^3)$ triple loop does $\Theta(n^2)$ `weight` queries per iteration of the outer loop, and the matrix is the right shape for those. The matrix wins specifically when m approaches n^2 or when the algorithm's structure asks $\Theta(n^2)$ many edge-existence questions regardless of m.

The edge list's place is more specialized. Kruskal's MST (chapter 36) sorts all edges by weight in a single $O(m \log m)$ pass and then walks them in order, processing each edge exactly once. The natural representation for an algorithm whose outer loop is "for each edge in sorted order" is a list of edges; an adjacency list would require flattening before sorting, costing an extra $\Theta(m)$ pass.

The rule of thumb most chapters in this part follow: **use the adjacency list as the default, switch to the matrix when the graph is dense, and reach for the edge list when the algorithm iterates edges in sorted order.**

31.7 The graph that stores nothing

There's a fourth shape that doesn't appear in the table above because it doesn't store anything. An *implicit graph* is one where the vertex set is too large or too dynamic to materialize, and the edges are computed on demand from a function. The classic examples: the state graph of a sliding puzzle (vertices are puzzle configurations, edges are legal moves), the graph of all reachable webpages from a starting URL (vertices are URLs, edges are hyperlinks discovered by fetching the page), the graph of all length-n words differing by exactly one letter (the word-ladder graph), the grid of cells in a maze (vertices are cells, edges are passable adjacencies).

For an implicit graph, there is no `AdjacencyList` instance to query. The algorithm needs only one function: given a vertex, return its neighbors. Everything else — the vertex set, the edge set, the size of the graph — is either implicit or infinite. BFS and DFS in chapters 32 and 33 are written to accept a neighbor *function* in addition to (or instead of) a concrete graph instance, which is what lets them solve word-ladder puzzles and maze-shortest-path problems against graphs that are never fully materialized.

```
from collections.abc import Callable, Iterable, Iterator

type NeighborFn[V] = Callable[[V], Iterable[V]]
```

That single type alias is the entire interface. A `NeighborFn[V]` is a function from a vertex to an iterable of its neighbors; the graph itself is whatever produces that function. Chapter 32's BFS will accept a `NeighborFn[V]` and a starting vertex and walk outward; no graph object is required. To show the shape, here's a tiny implicit-graph example — the 4-connected grid that mazes are built on.

```
def grid_neighbors(
    width: int,
    height: int,
    blocked: frozenset[tuple[int, int]] = frozenset(),
) -> NeighborFn[tuple[int, int]]:
    """Return a NeighborFn for a width x height 4-connected grid with the
    given blocked cells. The graph has width*height vertices but the cells
    are never materialized as a collection; neighbors are computed on demand.
    """
    def neighbors(cell: tuple[int, int]) -> Iterator[tuple[int, int]]:
        x, y = cell
        for dx, dy in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
            nx, ny = x + dx, y + dy
            if 0 <= nx < width and 0 <= ny < height and (nx, ny) not in blocked:
                yield (nx, ny)
    return neighbors
```

No vertices are stored. No edges are stored. The graph exists only through the closure that captures `width`, `height`, and `blocked`. Asking for the neighbors of a cell does a constant amount of work — four directions, four bounds checks. A million-by-million grid is a 10^{12}-vertex graph and the closure occupies a few hundred bytes.

```
from codex.graphs.implicit import grid_neighbors

nbrs = grid_neighbors(width=4, height=3, blocked=frozenset({(1, 1), (2, 1)}))
print(f"neighbors of (0, 0): {list(nbrs((0, 0)))}  (top-left corner: 2 neighbors)")
print(f"neighbors of (1, 0): {list(nbrs((1, 0)))}  (top edge, (1,1) blocked below)")
```

```
print(f"neighbors of (2, 2): {list(nbrs((2, 2)))}  (bottom edge, (2,1) blocked above)")
print(f"neighbors of (3, 2): {list(nbrs((3, 2)))}  (bottom-right corner: 2 neighbors)")
```

```
neighbors of (0, 0): [(1, 0), (0, 1)]  (top-left corner: 2 neighbors)
neighbors of (1, 0): [(0, 0), (2, 0)]  (top edge, (1,1) blocked below)
neighbors of (2, 2): [(1, 2), (3, 2)]  (bottom edge, (2,1) blocked above)
neighbors of (3, 2): [(2, 2), (3, 1)]  (bottom-right corner: 2 neighbors)
```

Four queries, three or four neighbors each, no graph object anywhere. This is what BFS will walk in chapter 32 when it computes the shortest path through a maze — the maze's grid is the graph, the obstacle set is the edge mask, and nothing is allocated up front. The implicit-graph pattern is what makes graph algorithms apply to the dynamic search problems they were invented for; without it, BFS would be stuck on graphs small enough to fit in memory.

31.8 The three questions, applied

Is it correct?

Each representation is correct if the encoding is *complete* — every edge in E appears once (for directed) or implicitly twice (for undirected) — and *consistent* — the encoding agrees with the graph's directedness and weights. The edge list is correct by construction: it stores literally the edge set. The adjacency matrix is correct if, for every u, v, the entry (u, v) holds the weight of edge (u, v) if it exists and ∞ otherwise; the symmetric update in `add_edge` maintains this for undirected graphs. The adjacency list is correct if `self._adj[u][v]` returns the edge weight for every $(u, v) \in E$ and raises `KeyError` for every non-edge; the same symmetric-update discipline maintains it.

The implicit-graph representation has a different correctness question: the `NeighborFn` is correct if, for every u, the function returns *exactly* the set $N(u)$ — no missing neighbors, no spurious ones. For the grid example, that's a bounds-and-blocked check on each of the four directions. For more interesting implicit graphs (the puzzle-state graph, the word-ladder graph), the neighbor function is the algorithm's only contract with the graph, so a bug in `neighbors` corrupts every traversal of the graph downstream.

How efficient is it?

The cost table in the previous section is the efficiency answer. No representation escapes the space-time trade-off: cheap edge queries cost $\Theta(n^2)$ space, $\Theta(n + m)$ space costs amortized $O(1)$ per query, and bare-edge storage costs $\Theta(m)$ space but turns every neighbor query into a $\Theta(m)$ scan. No representation strictly dominates the others; the right choice depends on the algorithm.

The implicit graph trades a different axis. It uses $\Theta(1)$ space per traversal step (no vertex set is precomputed) and pays for it with whatever cost the `NeighborFn` has per call. For a 4-connected grid the per-call cost is $O(1)$; for the word-ladder graph it's $O(n \cdot |\Sigma|)$ where n is the word length; for the puzzle-state graph it depends on the move-generation function. The implicit graph wins by avoiding the up-front $\Theta(|V|)$ memory cost when the reachable set is much smaller than the full vertex set, which is the usual case for state-space search.

What's the optimal representation?

There isn't one. The lower bound on representing a graph is $\Omega(\log \binom{n^2}{m}) = \Omega(m \log(n^2/m))$ bits in the information-theoretic sense — that's how many bits it takes to distinguish one graph with n vertices and m edges from another. The adjacency list comes within a constant factor of that bound for sparse graphs (each edge takes $O(\log n)$ bits to encode), and the adjacency matrix is tight for dense graphs (one bit per cell, n^2 bits total). For any specific graph, one of these two representations is within a constant factor of optimal. The edge list is loose by a constant factor relative to the adjacency list, and the implicit graph is incomparable — its cost is *zero* up-front, paid in Θ(calls to NeighborFn) instead.

The deeper "optimality" question — *which representation should this chapter's algorithm use?* — has no single answer either, and the chapters that follow each pick their own. *The algorithm and the representation are designed together;* change one without changing the other and you waste a factor of n somewhere.

31.9 What this chapter teaches

First, representation propagates through cost. The same abstract graph encoded in two different ways yields algorithms with two different cost shapes. BFS is $\Theta(n + m)$ on an adjacency list and $\Theta(n^2)$ on a matrix; for a sparse graph that's the difference between minutes and days. The choice of representation isn't a hygiene decision made before the real work starts — it *is* the real work, in the sense that the rest of the algorithm's cost is determined by it. When you read a graph-algorithm pseudocode in a textbook that says "for each neighbor of u do…", the cost of that loop varies by a factor of n depending on what's stored under "graph."

Second, the right representation matches the algorithm's query pattern. BFS, DFS, Dijkstra, and SCC all query neighbors of one vertex at a time, so they want the adjacency list. Floyd-Warshall queries weights of every vertex pair, so it wants the matrix. Kruskal sorts edges globally, so it wants the edge list. Implicit-graph search avoids materializing the graph at all, so it wants a neighbor function. The cost shape of the algorithm is determined by which question is the hot loop, and the representation should make exactly that question $O(1)$. When in doubt, look at the inner loop of the algorithm first, then pick the storage that makes the inner loop's per-iteration cost match its semantic content.

A third observation worth holding, even though it won't pay off until later in the book: the *adjacency-list representation is the bridge between graphs and the data structures from Parts I-III.* The outer dictionary is a hash table (chapter 12). Each inner dictionary is also a hash table. The neighbor enumeration is the iteration order of a hash table's keys. Graph algorithms are mostly built out of the same data structures you've already seen, applied at a slightly higher level — that's why the chapters in this part can keep claiming "and now we use the heap from chapter 19" without apology. Graphs are not a different kind of object; they are a structure over the kinds of objects you already have.

The choice of representation isn't bookkeeping you settle before writing the algorithm. It *is* the algorithm's inner-loop cost, and the inner loop is where the time goes.

31.10 Notes and further reading

The three representations covered here are the textbook trio. CLRS chapter 22 introduces them in roughly this order, with the same cost-shape table I reproduced above; Sedgewick and Wayne's *Algorithms* (4th ed.) chapter 4 covers them with slightly more code, presenting the adjacency list as a list-of-lists rather than a dict-of-dicts. The list-of-lists variant has one constant-factor advantage (no hash overhead on neighbor enumeration) and one disadvantage (no $O(1)$ `has_edge`). For algorithms in this part that need fast edge-existence checks — Floyd-Warshall, augmenting-path algorithms — the dict-of-dicts pays for itself.

For the implicit-graph idea, Russell and Norvig's *Artificial Intelligence: A Modern Approach* (chapter 3) treats state-space search as the canonical implicit-graph application, and the formal treatment of "graphs defined by a successor function" is theirs. The classic reference for graph-algorithm efficiency on different representations is Robert Tarjan's *Data Structures and Network Algorithms* (SIAM, 1983), which establishes the asymptotic bounds for the major algorithms parameterized by representation.

For sparse graphs at production scale there are two refinements I won't cover here. The first is the *compressed sparse row* format (CSR) borrowed from sparse-matrix libraries, which stores the adjacency list as two flat arrays — one of vertex offsets, one of neighbor labels — for excellent cache behaviour during traversal; it sacrifices the $O(1)$ `add_edge` of the dict-based list. Most production graph libraries (NetworkX, igraph, graph-tool) offer CSR or CSR-like representations as a build-once-query-many option. The second refinement is *external-memory graph representations* — the graphs that live on disk because they don't fit in RAM, with edge lists stored in sorted order and processed in streaming passes. Frank McSherry's *Differential Dataflow* and the broader literature on streaming graph algorithms treat that family.

In the next chapter I'll put the adjacency list to work on the simplest non-trivial graph algorithm: breadth-first traversal. The discipline from the queue chapter — first in, first out — turns out to be exactly what a graph traversal needs to enumerate vertices in order of distance from a source. As a side effect, BFS gives the first usable answer to whether two vertices are in the same connected component, computes the shortest path in unweighted graphs, and introduces the layered-frontier structure that chapter 38's augmenting-path algorithm depends on.

32 Expanding the frontier, one ring at a time

The queue from chapter 11 wanted a problem. The adjacency list from chapter 31 wanted an algorithm. Breadth-first search is what happens when you point them at each other. Start at one vertex; put its neighbors in a queue; pop the next vertex off the queue, push its unvisited neighbors; repeat until the queue is empty. That five-line discipline does four useful things at once: it visits every vertex reachable from the source, it visits them *in order of distance from the source*, it builds a shortest-path tree along the way, and it does all of that in $\Theta(n + m)$ time on any graph small enough to fit in memory and many that aren't.

By the end of this chapter you will have implemented BFS as a queue-driven sweep over an adjacency-list graph, augmented it to record distances and parent pointers, used those pointers to reconstruct shortest paths in unweighted graphs, partitioned a disconnected graph into its connected components, and run BFS against an *implicit* graph — a maze whose cells exist only as a neighbor function — to find the shortest passable path between two squares. The pattern recurs across the rest of the part. Dijkstra (chapter 34) replaces the queue with a priority queue and the same skeleton solves weighted shortest paths. Edmonds-Karp (chapter 38) finds augmenting paths in flow networks via BFS in the residual graph. The layered-frontier idea you'll build here is the skeleton every later chapter reuses.

32.1 The five-line skeleton

BFS keeps two pieces of state: a queue of vertices waiting to be expanded, and a set of vertices already seen. Both are initialized with the source vertex. The main loop pops the front of the queue, walks each of the popped vertex's neighbors, and for any neighbor that hasn't been seen yet, marks it seen and pushes it onto the back of the queue. When the queue empties, every reachable vertex has been visited exactly once.

The detail that earns most of the bugs: **mark a vertex as seen the moment you enqueue it, not when you dequeue it.** If you delay the marking, a vertex with two parents in the BFS tree gets enqueued twice — and the second visit either does redundant work or, in the path-reconstruction variant later in this chapter, corrupts the parent pointer. Mark on enqueue, never on dequeue.

Here's the skeleton with distance bookkeeping folded in. The output is a dictionary mapping each reachable vertex to its distance (in edges) from the source. Unreachable vertices simply don't appear in the dictionary.

```
from collections.abc import Hashable, Iterator
from codex.graphs.implicit import NeighborFn
from codex.structures.queue import LinkedQueue


def bfs_distances[V: Hashable](source: V, neighbors: NeighborFn[V]) -> dict[V, int]:
    """Return a dict mapping each vertex reachable from source to its
    distance in edges. Unreachable vertices are absent.
    """
    dist: dict[V, int] = {source: 0}
    queue: LinkedQueue[V] = LinkedQueue()
    queue.enqueue(source)
    while queue:
        u = queue.dequeue()
        for v in neighbors(u):
            if v not in dist:
                dist[v] = dist[u] + 1
                queue.enqueue(v)
    return dist
```

Twelve lines. The signature deliberately takes a `NeighborFn[V]` rather than a graph instance — chapter 31 left the implicit-graph case calling for exactly this shape, and accepting a function means the same BFS runs against `AdjacencyList.neighbors`, against `AdjacencyMatrix.neighbors`, against `grid_neighbors`, against a closure that fetches a webpage to discover its outlinks. The graph object never has to exist as a concrete data structure.

The `dist` dictionary doubles as the "seen" set. A vertex is in `dist` if and only if it's already been enqueued, so checking `v not in dist` is the same as asking "have I seen v yet?" The first time a vertex enters `dist`, it gets its true shortest-path distance — and that distance is correct by construction, because the queue's FIFO discipline guarantees that all vertices at distance k are enqueued before any vertex at distance $k+1$. Take the invariant on trust for the moment and watch the algorithm work on the canonical graph from chapter 31; I'll prove it in the "three questions" section.

```
from codex.graphs.adjacency_list import AdjacencyList
from codex.graphs.bfs import bfs_distances

g = AdjacencyList[str](directed=False)
for u, v, _ in [("A", "B", 5), ("A", "C", 3), ("B", "C", 2), ("B", "D", 6),
                ("C", "D", 1), ("C", "E", 4), ("D", "E", 2), ("D", "F", 3),
                ("E", "F", 5)]:
    g.add_edge(u, v)

dist = bfs_distances("A", g.neighbors)
for v in sorted(dist):
    print(f"   distance(A -> {v}) = {dist[v]}")
```

```
   distance(A -> A) = 0
   distance(A -> B) = 1
   distance(A -> C) = 1
   distance(A -> D) = 2
   distance(A -> E) = 2
   distance(A -> F) = 3
```

Six vertices, all reachable from `A`, distances 0 through 3 in edges. `B` and `C` are direct neighbors of `A`; `D` and `E` are two hops away; `F` is three hops, the farthest from `A` in this graph. The weights I assigned in chapter 31 are silently ignored — BFS treats every edge as cost 1, which is the right answer when you want shortest paths in *number of hops*, the wrong answer when edge weights matter, and the reason chapter 34 will need a different algorithm for the weighted case.

Distances alone tell you *how far*; the next thing the chapter needs is the actual route.

A subtle point worth flagging: the BFS distance from `A` to `D` is 2, but in chapter 31's weighted graph the cheapest weighted path from `A` to `D` is `A → C → D` with cost 4 (versus `A → B → D` with cost 11). BFS won't tell you that — its concept of "shortest" is *fewest edges*, full stop. Dijkstra's algorithm in chapter 34 generalizes BFS to weighted edges; the skeleton stays nearly the same but the queue gets upgraded to a priority queue.

32.2 Recording the path home

To recover the *path* — the actual sequence of vertices from source to target — I need one more piece of bookkeeping: a parent pointer. When BFS first discovers a vertex v via an edge from u, it remembers that u is the vertex it came from. Chasing parent pointers backward from any reachable vertex reconstructs the shortest path to the source.

The augmented BFS returns two dictionaries instead of one: `dist` as before, plus `parent`, mapping each non-source reachable vertex to the vertex it was discovered from. The source itself maps to `None`.

```
def bfs[V: Hashable](
    source: V, neighbors: NeighborFn[V]
) -> tuple[dict[V, int], dict[V, V | None]]:
    """Run BFS from source. Return (dist, parent) where dist maps each
```

```
    reachable vertex to its distance and parent maps each non-source
    reachable vertex to the vertex it was first discovered from. The
    source maps to None in parent.
    """
    dist: dict[V, int] = {source: 0}
    parent: dict[V, V | None] = {source: None}
    queue: LinkedQueue[V] = LinkedQueue()
    queue.enqueue(source)
    while queue:
        u = queue.dequeue()
        for v in neighbors(u):
            if v not in dist:
                dist[v] = dist[u] + 1
                parent[v] = u
                queue.enqueue(v)
    return dist, parent
```

Same structure, one extra dictionary, one extra write per vertex discovery. The dictionaries are kept in lockstep — every vertex in `dist` is also in `parent` and vice versa, so the check `v not in dist` could equally be `v not in parent`. Using one of the two as the "seen" predicate and keeping the other purely for return is conventional; mixing them confuses the bookkeeping for no benefit.

Now reconstruction. Starting from any reachable target, follow parent pointers backward until you reach the source. Reversing the resulting list gives the shortest path from source to target in source-first order.

```
def reconstruct_path[V: Hashable](
    parent: dict[V, V | None], target: V
) -> list[V] | None:
    """Walk parent pointers backward from target to the source (the
    vertex whose parent is None). Return the path in source-first order,
    or None if target is not in parent (i.e., unreachable).
    """
    if target not in parent:
        return None
    path: list[V] = []
    current: V | None = target
    while current is not None:
        path.append(current)
        current = parent[current]
    path.reverse()
    return path
```

Twelve lines, one allocation, one in-place reverse. The path length is the number of vertices, which equals the distance in edges plus one — a useful invariant for sanity-checking the result.

```
from codex.graphs.adjacency_list import AdjacencyList
from codex.graphs.bfs import bfs, reconstruct_path

g = AdjacencyList[str](directed=False)
for u, v, _ in [("A", "B", 5), ("A", "C", 3), ("B", "C", 2), ("B", "D", 6),
                ("C", "D", 1), ("C", "E", 4), ("D", "E", 2), ("D", "F", 3),
                ("E", "F", 5)]:
    g.add_edge(u, v)

dist, parent = bfs("A", g.neighbors)

for target in ["B", "D", "F"]:
    path = reconstruct_path(parent, target)
    print(f"   path A -> {target}: {path}  (length {dist[target]})")
```

```
path A -> B: ['A', 'B']  (length 1)
path A -> D: ['A', 'B', 'D']  (length 2)
path A -> F: ['A', 'B', 'D', 'F']  (length 3)
```

Three paths reconstructed in source-first order. `A → B` is one hop; `A → C → D` (or `A → B → D`, depending on neighbor-iteration order) is two; `A → C → D → F` is three. BFS returns *one* shortest path per target, not all of them. In this graph there are two shortest paths from `A` to `D` (one through `B`, one through `C`) and BFS picks the one whose intermediate vertex was enqueued first — which depends on the iteration order of the adjacency dictionary. The choice is arbitrary but deterministic for a given dictionary insertion order. Enumerating *all* shortest paths is a different algorithm with cost Θ(number of paths) on top of the BFS itself; I'll skip it here.

32.3 The layered frontier

There's a second way to organize the same algorithm, and I find it more honest about what BFS actually computes. Expand the frontier one *layer* at a time, where layer k is the set of vertices at distance exactly k from the source. The level-by-level emission pattern is what gave BFS its name: "breadth-first" means expanding the full breadth of the frontier before moving outward.

The layered formulation is sometimes more natural than the queue-based formulation. For problems like "find all vertices within distance k" or "what's the diameter of this graph?" or "what does the BFS tree look like, level by level?", emitting layers directly is cleaner than reconstructing them from distance values. The implementation keeps two lists — the current frontier and the next frontier — and swaps them at the end of each layer.

```
def bfs_layers[V: Hashable](
    source: V, neighbors: NeighborFn[V]
) -> Iterator[list[V]]:
    """Yield each BFS layer as a list, starting with [source] at layer 0.
    Layer k is the set of vertices at distance exactly k from source.
    """
    seen: set[V] = {source}
    frontier: list[V] = [source]
    while frontier:
        yield frontier
        next_frontier: list[V] = []
        for u in frontier:
            for v in neighbors(u):
                if v not in seen:
                    seen.add(v)
                    next_frontier.append(v)
        frontier = next_frontier
```

No queue this time. The current layer is held as a list (its ordering doesn't matter for correctness), and at the end of each round the next layer takes over as the frontier. The total work is the same $\Theta(n + m)$ — each vertex enters the seen set once, each edge is walked once — but the structure of the computation is now level-synchronous rather than vertex-synchronous. The same idea reappears in parallel BFS implementations on multi-core machines, where each layer can be expanded by many threads in parallel and the synchronization happens once per layer rather than once per vertex.

```
from codex.graphs.adjacency_list import AdjacencyList
from codex.graphs.bfs import bfs_layers

g = AdjacencyList[str](directed=False)
for u, v, _ in [("A", "B", 5), ("A", "C", 3), ("B", "C", 2), ("B", "D", 6),
                ("C", "D", 1), ("C", "E", 4), ("D", "E", 2), ("D", "F", 3),
                ("E", "F", 5)]:
    g.add_edge(u, v)

for k, layer in enumerate(bfs_layers("A", g.neighbors)):
    print(f"   layer {k} (distance {k} from A): {sorted(layer)}")
```

```
layer 0 (distance 0 from A): ['A']
layer 1 (distance 1 from A): ['B', 'C']
layer 2 (distance 2 from A): ['D', 'E']
layer 3 (distance 3 from A): ['F']
```

Four layers, each one ring farther from the source. Layer 0 is just the source. Layer 1 is `A`'s direct neighbors. Layer 2 is the vertices reachable in two hops but not in one. Layer 3 is `F`, the most-distant vertex in the graph. The graph's *diameter from `A`* — the longest shortest-path distance — is 3, which you read off as the last non-empty layer.

The layered view also makes BFS's bipartite-check pay off cheap. A graph is bipartite iff its vertex set can be split into two color classes with no edges within a class — equivalently, iff a BFS from any starting vertex produces no edges that connect two vertices of the same parity layer. Walk the graph with `bfs_layers`, alternate colors layer by layer, and on each edge check that the endpoints have different colors. If any edge violates the rule, the graph isn't bipartite; if no edge does, it is. The check is $\Theta(n + m)$, same as BFS itself.

What if the graph isn't connected? BFS from one source can't reach everything; you need a sweep.

32.4 One BFS per island

BFS from a single source visits everything reachable from that source — which on a disconnected graph is only the source's connected component. To enumerate *all* connected components, sweep over the vertex set: pick any unvisited vertex, run BFS from it, mark its whole component as visited, repeat until no unvisited vertices remain. Each component is one BFS, each BFS visits its component exactly once, and the total work across all components is $\Theta(n + m)$ because every vertex and every edge gets touched exactly once over the whole sweep.

```
def connected_components[V: Hashable](
    vertices: Iterator[V] | list[V] | set[V],
    neighbors: NeighborFn[V],
) -> list[set[V]]:
    """Sweep BFS over the vertex set. Return a list of components, each
    as a set of vertices. The component containing vertex u is the set
    of vertices reachable from u via the undirected closure of neighbors.
    """
    seen: set[V] = set()
    components: list[set[V]] = []
    for start in vertices:
        if start in seen:
            continue
        component: set[V] = {start}
        queue: LinkedQueue[V] = LinkedQueue()
        queue.enqueue(start)
        while queue:
            u = queue.dequeue()
            for v in neighbors(u):
                if v not in component:
                    component.add(v)
                    queue.enqueue(v)
        seen |= component
        components.append(component)
    return components
```

Same skeleton as the basic BFS, wrapped in an outer loop. The `seen` set is shared across components — once a vertex enters any component it can't enter another — but inside one component the local `component` set is the seen-predicate. Returning a list of sets makes downstream queries cheap: "is `u` connected to `v`?" is one membership check per component until you find one containing both, or you reach the end.

For directed graphs, this routine computes *weakly connected* components — the components you'd get if you ignored edge directions. *Strongly connected* components are a different question requiring DFS-based discovery and finish times; chapter 37 handles that.

```
from codex.graphs.adjacency_list import AdjacencyList
from codex.graphs.bfs import connected_components

# Build a graph with three components: {1,2,3,4} chain, {5,6} pair, {7} singleton
g = AdjacencyList[int](directed=False)
for u, v in [(1, 2), (2, 3), (3, 4), (5, 6)]:
    g.add_edge(u, v)
g.add_vertex(7)

components = connected_components(g.vertices(), g.neighbors)
print(f"number of components: {len(components)}")
for i, comp in enumerate(sorted(components, key=lambda c: min(c))):
    print(f"   component {i + 1}: {sorted(comp)}")
```

```
number of components: 3
   component 1: [1, 2, 3, 4]
   component 2: [5, 6]
   component 3: [7]
```

Three components: the 4-chain, the pair, and the isolated vertex. The cost of computing this on a million-vertex graph is the same as one BFS from a single source — the work is shared across the components, not multiplied.

32.5 BFS on a graph that doesn't exist

This is the section I most wanted to write, and the one chapter 31 spent its closing pages setting up. BFS accepts a `NeighborFn[V]` and doesn't care whether that function reads from a stored adjacency list, computes neighbors on the fly, or fetches a webpage and parses its outlinks. The same five-line skeleton finds the shortest passable path through a maze that has never been materialized as a graph object.

Take a small maze on a 5×5 grid. The maze is represented by its set of *blocked* cells; the remaining cells are passable, and a cell's neighbors are its passable 4-connected grid neighbors. Chapter 31's `grid_neighbors` already produces the right `NeighborFn`. BFS finds the shortest path from start to goal, in passable-cell-count, by treating the grid as an implicit graph.

```
from codex.graphs.implicit import grid_neighbors
from codex.graphs.bfs import bfs, reconstruct_path

# 5x5 maze, '#' is a wall, '.' is open. Start at (0,0), goal at (4,4).
blocked = frozenset({
    (1, 0), (1, 1), (1, 3),
    (3, 1), (3, 2), (3, 3),
})
nbrs = grid_neighbors(width=5, height=5, blocked=blocked)

dist, parent = bfs((0, 0), nbrs)
path = reconstruct_path(parent, (4, 4))

print(f"shortest path: {dist[(4, 4)]} steps over {dist[(4, 4)] + 1} cells")
print(f"path: {path}")
print()
# Render the maze with the path overlaid
for y in range(5):
    row = ""
    for x in range(5):
        if (x, y) in blocked:
            row += "# "
        elif (x, y) in (path or []):
            row += "o "
```

```
        else:
            row += ". "
    print(f"   {row}")
```

```
shortest path: 8 steps over 9 cells
path: [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 3), (2, 4), (3, 4), (4, 4)]

   o # . . .
   o # . # .
   o o o # .
   . # o # .
   . . o o o
```

The maze has two `#`-shaped walls that force the path to weave around. BFS finds the shortest path — eight steps, nine cells — without ever materializing the grid as a graph object. The 25-cell grid is trivial here, but the same code works on a 10000×10000 grid where storing the full adjacency list would be $\Theta(10^8)$ entries. The implicit-graph representation lets BFS solve problems whose state space is too large to enumerate; the only cost is one `grid_neighbors` call per cell visit, and the cost stays $\Theta(\text{visited cells} + \text{visited edges})$ rather than scaling with the full grid size.

Word-ladder puzzles work the same way. The vertices are words of a fixed length; the neighbors of a word are the words you can reach by changing exactly one letter; BFS from `"COLD"` to `"WARM"` finds the shortest letter-by-letter transformation. The graph is exponentially large in principle (every length-4 string is a vertex), but BFS only enumerates the reachable subset, and on a dictionary of English words the reachable component from any common word is a few thousand vertices at most. The implicit framing is what makes the problem tractable; storing the full word graph would be wasteful and unnecessary.

32.6 The three questions, applied

Is it correct?

The correctness of BFS rests on one invariant: **when a vertex is dequeued, its recorded distance is its true shortest-path distance from the source.** The proof is induction on the dequeue order. The source itself is dequeued first with distance 0, which is correct. For any later vertex v dequeued at recorded distance d, v was enqueued at the moment some vertex u at distance $d-1$ visited it as a neighbor — and u's distance was correct by the induction hypothesis. So v's recorded distance is at most d. It's also at least d, because any vertex enqueued before v had distance $\leq d-1$, and a shorter path to v would have produced an earlier enqueue. The recorded distance equals the true shortest-path distance.

The "mark on enqueue, not on dequeue" discipline is what keeps this argument tight. If a vertex v were enqueued twice — once at distance d_1 and once at distance d_2 with $d_1 \leq d_2$ — the second visit would either overwrite v's correct shortest distance with a longer one (corrupting the dictionary) or be filtered out by an "already-dequeued" check (adding a layer of bookkeeping). Marking on enqueue prevents the double-enqueue from ever happening, which is what lets the dictionary serve as the seen-set with no additional state.

Path reconstruction is correct by the same induction. Each non-source vertex's parent pointer points at the vertex from which it was first discovered, and the first discovery is via a shortest path (by the dequeue invariant above). Chasing parent pointers backward therefore produces a shortest path, in reverse, which `reconstruct_path` un-reverses.

How efficient is it?

BFS runs in $\Theta(n+m)$ time and $\Theta(n)$ extra space on a graph with n vertices and m edges, when the graph is represented as an adjacency list. Each vertex is enqueued at most once (the dictionary `dist` enforces this) and dequeued at most once, contributing $\Theta(n)$ total queue work. Each edge is walked at most once per endpoint — twice total for undirected graphs, once for directed — and each walk is $O(1)$ on an adjacency list, contributing $\Theta(m)$ total. The two pieces add to $\Theta(n+m)$. Space is dominated by the `dist` and `parent` dictionaries, which together hold one entry per reachable vertex; in the worst case where every vertex is reachable, that's $\Theta(n)$.

On an adjacency *matrix* the cost shape changes. Finding the neighbors of u now takes $\Theta(n)$ work — a row scan — and BFS calls `neighbors` once per dequeued vertex, so the total cost rises to $\Theta(n \cdot n) = \Theta(n^2)$. For a sparse graph with $m = O(n)$ that's a factor of n slowdown over the adjacency-list version; for a dense graph with $m = \Theta(n^2)$ the two costs match. Chapter 31's table predicted this, and BFS is the cleanest demonstration of the prediction in action.

On an implicit graph the cost depends on the `NeighborFn`. For `grid_neighbors`, each call is $O(1)$ (four directions, four bounds checks), so BFS on an implicit grid is Θ(visited cells) — strictly better than building the explicit graph and then running BFS on it. For more expensive neighbor functions (a web crawl that fetches a page, a word-ladder generator that scans the dictionary), the per-call cost dominates and the asymptotic shape becomes Θ(visited vertices $\cdot$ cost per call).

Is it optimal?

For unweighted shortest paths from a single source, $\Theta(n + m)$ is the lower bound. Every reachable vertex has to be reached, and every edge incident to a reachable vertex has to be examined to be sure no shorter path exists. BFS achieves the bound exactly. No algorithm can do better in the unit-weight model.

A stronger optimality claim holds when the graph is *implicit*. BFS's per-vertex cost is the cost of one `neighbors` call plus $O(1)$ bookkeeping, which is asymptotically tight: any algorithm that finds shortest paths in an implicit graph has to consult the neighbor function at least once per visited vertex, and bookkeeping below $O(1)$ per visit is impossible. The implicit-graph BFS is therefore optimal *for problems whose state space is too large to materialize* — and that is what makes it the workhorse algorithm of AI search and combinatorial enumeration.

For weighted shortest paths the question changes. BFS solves the unweighted case in $\Theta(n + m)$; Dijkstra in chapter 34 solves the weighted case (with non-negative weights) in $\Theta((n + m) \log n)$ using a heap. The $\log n$ factor is what the heap pays for, and the price is unavoidable when edge weights stop being unit.

32.7 What this chapter teaches

First, the right discipline at the queue is the right answer at the graph. BFS visits vertices in *the order they were first seen* — that's the FIFO contract from chapter 11, applied to graph traversal. The contract guarantees that vertices come out in distance order, which guarantees that the first-discovered path is a shortest path. Replace the queue with a stack and you get DFS (chapter 33): different discipline, different cost shape, completely different set of facts about the graph. Replace it with a priority queue and you get Dijkstra (chapter 34): the heap's "smallest-first" discipline solves weighted shortest paths the way the queue's FIFO discipline solves unweighted ones. The choice of container determines the algorithm's character; the rest of the code stays nearly the same.

Second, the layered frontier is the structural object, not the queue. The queue is an implementation detail — it's how you maintain the frontier from one layer to the next. The *layered* view of BFS (every vertex at distance k forms layer k) is what the algorithm actually computes, and it's the right mental model for any algorithm that builds on BFS. Edmonds-Karp's augmenting paths in chapter 38 are BFS layers in the residual graph. The bipartite check is a parity argument on BFS layers. Parallel BFS expands one layer at a time across many cores. Once you see BFS as "expand the frontier outward, one ring at a time," the variations stop being separate algorithms and start being variations on a single shape.

A third observation. **The implicit-graph framing is what makes graph algorithms applicable beyond the cases where you can store the graph.** A maze on a 10000×10000 grid has 10^8 vertices — too many to materialize on most machines, trivial for BFS as long as the start-to-goal path doesn't touch a substantial fraction of them. A word-ladder graph has $26^4 \approx 4 \times 10^5$ length-4 strings — too many to enumerate, fine for BFS because most aren't reachable from any English word. The pattern recurs across the rest of this part and through the AI-search literature: define the state graph implicitly via a successor function, and let BFS or its descendants explore only the part that matters.

32.8 Notes and further reading

BFS in its modern form is usually attributed to Edward Moore's 1957 maze-solving algorithm, presented at the International Symposium on the Theory of Switching at Harvard (proceedings published 1959 as "The shortest path through a maze," *Proceedings of an International Symposium on the Theory of Switching*, pp. 285–292). Konrad Zuse's 1945 dissertation contained an essentially identical algorithm for graph traversal but went unpublished for decades; C. Y. Lee's 1961 paper "An Algorithm for Path Connections and Its Applications" (*IRE Transactions on Electronic Computers* EC-10:346–365) independently rediscovered the same algorithm for VLSI routing. The "breadth-first search" name is post-1960s and tracks the algorithm's role in graph theory rather than its earlier applications.

CLRS chapter 22 covers BFS with the same skeleton and the same proof. Sedgewick and Wayne's *Algorithms* (4th ed.) §4.1 presents BFS alongside DFS and walks through the connected-components computation. For the parallel BFS variant — important on multi-core hardware and the basis of the Graph500 benchmark — the canonical reference is Beamer, Asanović, and Patterson's "Direction-Optimizing Breadth-First Search" (*SC '12*), which alternates between the layer-expansion form shown here and a "bottom-up" pass that checks each unvisited vertex for any frontier neighbor; on graphs with high-degree hub vertices the bottom-up pass is dramatically faster than the standard top-down form.

For implicit-graph BFS in AI search, Russell and Norvig's *Artificial Intelligence: A Modern Approach* chapter 3 covers BFS as one of the *uninformed search* strategies, alongside iterative deepening and uniform-cost search (which generalizes BFS to weighted graphs and is essentially Dijkstra's algorithm in different notation). The book also discusses *bidirectional* search, where BFS runs from both source and target simultaneously and meets in the middle — a $\sqrt{n}$-factor improvement on the explored-vertex count when both endpoints are known up front.

The Edmonds-Karp algorithm — Jack Edmonds and Richard Karp's 1972 paper "Theoretical improvements in algorithmic efficiency for network flow problems" (*Journal of the ACM* 19(2):248–264) — uses BFS to find augmenting paths in flow networks; chapter 38 develops it in full. Worth noting now: every iteration of Edmonds-Karp runs one BFS in the residual graph, so the algorithm's correctness ultimately depends on BFS's "shortest path in edges" guarantee.

In the next chapter I'll keep the graph and swap the queue for a stack. The resulting algorithm — depth-first search — gives up BFS's distance guarantee but gains something else: the ability to expose the *structure* of the graph through discovery and finish times. Cycle detection, topological sort, articulation points, and strong components all fall out of DFS plus a few lines of bookkeeping. The trade is the one chapter 11 already made for you: FIFO answers "what's the layered frontier," LIFO answers "what's the structural decomposition."

33 What you see when you go deep before you go wide

Swap the queue for a stack and the algorithm from chapter 32 becomes a completely different thing. The skeleton is the same — five lines of "mark, expand neighbors, repeat" — but where BFS visited the graph one ring at a time outward from the source, depth-first search rams as far down a single branch as it can, backtracks when it has to, and on its way home leaves behind a structural fingerprint the graph didn't appear to have. That fingerprint is what makes DFS the substrate every structural graph algorithm builds on. Cycle detection, topological ordering, articulation points, strongly connected components, bridge detection — none of these are separate algorithms. They are DFS with a few lines of bookkeeping each, and they all rest on a single property that recursive DFS exposes for free: every vertex is visited inside a time interval, and the intervals from any DFS run form a properly nested parenthesis structure over the vertex set.

By the end of this chapter you will have implemented DFS in two forms (recursive, which is the natural way to think about it, and iterative with an explicit stack, which is the only way to walk a graph deep enough that Python's recursion limit kicks in), recorded discovery and finish timestamps for every vertex, used those timestamps to detect cycles in a directed graph, and read the same timestamps as a *parenthesization* of the DFS tree that makes the algorithm's structural output visible. The vocabulary developed here — discovery times, finish times, tree edges, back edges — is what chapter 37's topological sort and strongly connected components stand on directly. Both of those algorithms are this chapter's DFS plus one extra pass.

33.1 DFS in its natural form

The form I'd write at the keyboard before writing anything cleaner is a recursion. Visit a vertex, mark it, recurse on each of its unvisited neighbors. The recursion stops at vertices whose neighbors are all already visited, the call stack unwinds, and the algorithm finishes when every reachable vertex has been touched exactly once. The whole thing is about ten lines including the wrapper.

```
from collections.abc import Hashable, Iterable
from sys import setrecursionlimit
from codex.graphs.implicit import NeighborFn
from codex.structures.stack import ArrayStack


def dfs_recursive[V: Hashable](source: V, neighbors: NeighborFn[V]) -> set[V]:
    """Return the set of vertices reachable from source via depth-first
    recursion. Visits each reachable vertex exactly once.
    """
    seen: set[V] = set()
    def visit(u: V) -> None:
        seen.add(u)
        for v in neighbors(u):
            if v not in seen:
                visit(v)
    visit(source)
    return seen
```

Nine lines of substance. The inner `visit` is the recursion; the outer wrapper exists so the `seen` set is shared across the recursive calls without being a global. The order in which a vertex's neighbors are visited determines which path DFS goes down first — the canonical demo from chapter 31 traverses `A → B → C → D → E → F` because that's the insertion order of the adjacency dictionaries, but a different insertion order would give a different traversal. DFS produces *one* depth-first ordering per starting vertex and per neighbor iteration order; the ordering depends on the data structure underneath.

```
from codex.graphs.adjacency_list import AdjacencyList
from codex.graphs.dfs import dfs_recursive

g = AdjacencyList[str](directed=False)
for u, v, _ in [("A", "B", 5), ("A", "C", 3), ("B", "C", 2), ("B", "D", 6),
                ("C", "D", 1), ("C", "E", 4), ("D", "E", 2), ("D", "F", 3),
                ("E", "F", 5)]:
    g.add_edge(u, v)

reached = dfs_recursive("A", g.neighbors)
print(f"vertices reached from A: {sorted(reached)}")
print(f"all six? {len(reached) == 6}")
```

```
vertices reached from A: ['A', 'B', 'C', 'D', 'E', 'F']
all six? True
```

All six vertices, same set BFS produced in chapter 32. Reachability is a structural property of the graph; both algorithms agree on it. What they disagree on is the *order* of discovery. That order is what later algorithms care about.

33.2 When recursion overflows the stack

Python's default recursion limit is around 1000 frames. A DFS over a graph with a chain of 10000 vertices — perfectly reasonable in practice; think of a linked-list-shaped graph, a path through a long maze, a chain of dependencies in a build system — will hit `RecursionError` before it finishes. The fix is either to raise the limit (`sys.setrecursionlimit`, which trades a stack overflow for a process crash if you go too far) or to convert the recursion into an explicit loop with an explicit stack. The explicit-stack form is the one you ship.

The translation is mechanical. Push the source onto a stack; while the stack is non-empty, pop a vertex; if it hasn't been seen yet, mark it seen and push each of its unvisited neighbors onto the stack. The "mark on push" discipline (the LIFO analogue of BFS's "mark on enqueue") prevents the same vertex from being pushed twice via two different paths, which would otherwise cause $O(n^2)$ work in the worst case.

```
def dfs_iterative[V: Hashable](source: V, neighbors: NeighborFn[V]) -> set[V]:
    """Iterative DFS using an explicit stack. Same reachability set as
    dfs_recursive, no recursion limit, $O(n)$ stack space.
    """
    seen: set[V] = {source}
    stack: ArrayStack[V] = ArrayStack()
    stack.push(source)
    while stack:
        u = stack.pop()
        for v in neighbors(u):
            if v not in seen:
                seen.add(v)
                stack.push(v)
    return seen
```

Eleven lines. Same skeleton as BFS, swapped queue for stack, same `seen`-as-dictionary trick. The visit order is *not* identical to the recursive form — the iterative version expands the last-pushed neighbor first, which reverses the order in which neighbors of each vertex are explored — but the reachable set is the same. For algorithms that only care about reachability (the connected-components sweep at the end of the chapter, the cycle check in the next section), the iterative form is the right call: it shares the same correctness guarantees with the recursive form and never crashes the interpreter.

```
from codex.graphs.adjacency_list import AdjacencyList
from codex.graphs.dfs import dfs_iterative

g = AdjacencyList[str](directed=False)
for u, v, _ in [("A", "B", 5), ("A", "C", 3), ("B", "C", 2), ("B", "D", 6),
                ("C", "D", 1), ("C", "E", 4), ("D", "E", 2), ("D", "F", 3),
                ("E", "F", 5)]:
    g.add_edge(u, v)

reached = dfs_iterative("A", g.neighbors)
print(f"vertices reached from A: {sorted(reached)} (iterative)")
```

```
vertices reached from A: ['A', 'B', 'C', 'D', 'E', 'F'] (iterative)
```

Same six vertices. The iterative form has one subtlety the recursive form doesn't: if you need a specific DFS *order* — say, to match a textbook proof that walks neighbors left-to-right — you have to push neighbors in *reverse* iteration order so the last-pushed (first-popped) is the leftmost neighbor. The recursive form does this implicitly via the order of the `for v in neighbors(u)` loop. The iterative form needs `for v in reversed(list(neighbors(u)))` if order matters. For the timestamp algorithm below I'll use the recursive form precisely because the order matters.

33.3 Discovery, finish, and the parenthesis structure

Here's the move I want you to remember from this chapter. DFS can attach two timestamps to every vertex with almost no extra code, and the timestamps are the structural fingerprint the rest of the book reads. The *discovery time* `d[v]` is the value of a global counter at the moment DFS first reaches `v`; the *finish time* `f[v]` is the counter's value at the moment DFS returns from the recursive call on `v`. The counter ticks up by one on each event, so over a DFS that visits n vertices, the counter goes from 1 to $2n$, with each vertex contributing two ticks: one on discovery, one on finish.

Recording the timestamps is a one-line addition to the recursive DFS at each of the two events.

```
def dfs_timestamps[V: Hashable](
    source: V, neighbors: NeighborFn[V]
) -> tuple[dict[V, int], dict[V, int]]:
    """Recursive DFS recording discovery and finish times for every vertex
    reachable from source. Times are integers in [1, 2*reached], with
    each vertex contributing one discovery tick and one finish tick.
    Returns (discovery, finish).
    """
    discovery: dict[V, int] = {}
    finish: dict[V, int] = {}
    time = [0]  # boxed so the closure can increment it
    def visit(u: V) -> None:
        time[0] += 1
        discovery[u] = time[0]
        for v in neighbors(u):
            if v not in discovery:
                visit(v)
        time[0] += 1
        finish[u] = time[0]
    visit(source)
    return discovery, finish
```

The boxed `time` is the only piece of Python ergonomics worth flagging — closures over integers in Python need a wrapper because plain integer assignment inside a nested function creates a new local. A one-element list lets the inner function mutate the counter without `nonlocal`. The rest is just the recursive DFS from earlier with two extra lines per visit.

Run it on the canonical graph.

```
from codex.graphs.adjacency_list import AdjacencyList
from codex.graphs.dfs import dfs_timestamps

g = AdjacencyList[str](directed=False)
for u, v, _ in [("A", "B", 5), ("A", "C", 3), ("B", "C", 2), ("B", "D", 6),
                ("C", "D", 1), ("C", "E", 4), ("D", "E", 2), ("D", "F", 3),
                ("E", "F", 5)]:
    g.add_edge(u, v)

discovery, finish = dfs_timestamps("A", g.neighbors)
print("vertex |  d  |  f  | interval")
print("-------+-----+-----+------------")
for v in sorted(discovery, key=lambda x: discovery[x]):
    print(f"   {v}   |  {discovery[v]:>2} |  {finish[v]:>2} | [{discovery[v]}, {finish[v]}]")
```

```
vertex |  d  |  f  | interval
-------+-----+-----+------------
   A   |   1 |  12 | [1, 12]
   B   |   2 |  11 | [2, 11]
   C   |   3 |  10 | [3, 10]
   D   |   4 |   9 | [4, 9]
   E   |   5 |   8 | [5, 8]
   F   |   6 |   7 | [6, 7]
```

Six vertices, twelve ticks. Read the table from top to bottom — that's the order DFS discovered the vertices. `A`'s interval is `[1, 12]`, the outermost; it brackets every other vertex's interval. `B`'s is `[2, 11]`, nested inside `A`'s; it brackets `C`, `D`, `E`, `F`. `F`'s is `[6, 7]`, the innermost — it's the deepest leaf of this DFS tree, reached last and finished first.

The bracketing is not coincidence. Write each vertex's interval as a pair of parentheses — `(` at the discovery time, `)` at the finish time — and the resulting string is a *valid parenthesization*: every opening parenthesis has a matching closing one, and the pairs nest cleanly. This is the **parenthesis theorem**: for any two vertices u and v in a DFS, their intervals `[d[u], f[u]]` and `[d[v], f[v]]` are either *disjoint* (one finishes before the other starts) or *one contains the other* (one's interval is entirely inside the other's). They never partially overlap.

That's the theorem in three lines, and the consequence is the algorithmic payoff. *When one interval contains another, the corresponding vertices stand in an ancestor-descendant relationship in the DFS tree.* If `[d[u], f[u]]` contains `[d[v], f[v]]`, then `u` is an ancestor of `v` in the DFS tree, and the path from `u` down to `v` is part of the DFS recursion stack at the moment `v` was discovered. Ancestor-descendant relationships, subtree membership, path depth — the timestamps encode them all, with no separate tree object needed.

Rendered explicitly, the parenthesization makes the DFS tree visible:

```
events: list[tuple[int, str, str]] = []
for v, d in discovery.items():
    events.append((d, "open", v))
for v, f in finish.items():
    events.append((f, "close", v))
events.sort()

parens = ""
for _, kind, v in events:
    if kind == "open":
        parens += f"({v}"
    else:
        parens += f" {v})"
print(f"DFS parenthesization: {parens}")
```

```
DFS parenthesization: (A(B(C(D(E(F F) E) D) C) B) A)
```

`(A (B (C (D (E (F F) E) D) C) B) A)` — six pairs of parentheses, properly nested six levels deep. The graph is a mesh, but DFS from `A` saw it as a chain. That's because the canonical graph happens to be densely connected enough that DFS from `A` goes all the way to `F` without ever needing to backtrack until the deepest leaf is reached. On a sparser or differently-shaped graph the parenthesization would be wider and shallower.

The nesting structure isn't just a visualization. It's the mechanism the next algorithm runs on.

33.4 A gray vertex is proof of a cycle

Here's the payoff. The parenthesis structure makes the next algorithm almost trivial. A directed graph has a cycle if and only if, at some point during DFS, the algorithm encounters an edge from the vertex it's currently expanding to a vertex whose interval is *still open* — i.e., a vertex that's an ancestor of the current vertex in the DFS recursion stack. That kind of edge is called a **back edge**, and back edges are precisely the witnesses of cycles in directed graphs.

The implementation uses three colors to track each vertex's state. *White* means undiscovered, *gray* means currently being explored (between discovery and finish), *black* means finished. A gray vertex is one whose interval is open; an edge from any vertex to a gray vertex is a back edge, hence a cycle. The check is a single state-machine transition at each edge.

```
def has_cycle[V: Hashable](
    vertices: Iterable[V], neighbors: NeighborFn[V]
) -> bool:
    """Detect whether the directed graph defined by (vertices, neighbors)
    contains a cycle. Uses a three-color DFS: white = undiscovered,
    gray = on the recursion stack, black = finished. A gray-to-gray
    edge is a back edge, which is the witness of a cycle.
    """
    WHITE, GRAY, BLACK = 0, 1, 2
    color: dict[V, int] = {v: WHITE for v in vertices}
    def visit(u: V) -> bool:
        color[u] = GRAY
        for v in neighbors(u):
            if color.get(v, WHITE) == GRAY:
                return True
            if color.get(v, WHITE) == WHITE and visit(v):
                return True
        color[u] = BLACK
        return False
    for v in list(color.keys()):
        if color[v] == WHITE:
            if visit(v):
                return True
    return False
```

Sixteen lines. The structure is recursive DFS plus a color update at each vertex's entry and exit, plus an early-return at the moment a back edge is detected. The outer `for v in list(color.keys())` is what makes the algorithm work on disconnected graphs and on graphs where the source isn't given — every vertex gets a chance to start a DFS, and any DFS that finds a cycle short-circuits the whole computation.

For an *undirected* graph the algorithm is subtly different: the edge from any non-source vertex back to its DFS-tree parent looks like a back edge but isn't a real cycle, just the edge you came in on. Detecting undirected cycles by DFS requires passing the parent vertex into the recursion and treating only non-parent gray-edges as back edges. I'll skip the undirected variant here — chapter 14's union-find solves the undirected cycle problem directly, and chapter 36's MST algorithms lean on union-find for the same reason.

Here's the directed-cycle check on two graphs: a DAG (which should report no cycle) and a graph with one obvious cycle.

```python
from codex.graphs.adjacency_list import AdjacencyList
from codex.graphs.dfs import has_cycle

# A small DAG: 1 -> 2, 1 -> 3, 2 -> 4, 3 -> 4
dag = AdjacencyList[int](directed=True)
for u, v in [(1, 2), (1, 3), (2, 4), (3, 4)]:
    dag.add_edge(u, v)
print(f"DAG has cycle? {has_cycle(dag.vertices(), dag.neighbors)}")

# A graph with a 3-cycle: 1 -> 2 -> 3 -> 1, plus 4 -> 1 (no cycle through 4)
cyclic = AdjacencyList[int](directed=True)
for u, v in [(1, 2), (2, 3), (3, 1), (4, 1)]:
    cyclic.add_edge(u, v)
print(f"graph with 3-cycle has cycle? {has_cycle(cyclic.vertices(), cyclic.neighbors)}")
```

```
DAG has cycle? False
graph with 3-cycle has cycle? True
```

DAG: no cycle. Cyclic graph: cycle detected. The algorithm's worst case is the same $\Theta(n + m)$ as plain DFS — at most one DFS traversal of the whole graph, with one constant-time check per edge — and the early return on the first back edge means cyclic graphs are typically detected much faster than the worst case.

The cycle/DAG distinction is what chapter 37's topological sort runs on. A directed graph admits a linear ordering of its vertices respecting all edges (every edge $u \to v$ has u earlier than v in the ordering) *if and only if* the graph has no cycle — i.e., is a DAG. Once DFS confirms the graph is a DAG, the *reverse* of the finish-time ordering is a valid topological sort. The whole topological-sort algorithm is `has_cycle`'s framework plus "append `u` to the result list at the moment it turns black."

33.5 Edge classification, briefly

A complete DFS on a directed graph classifies every edge into one of four types according to the colors of its endpoints at the moment the edge is examined:

- **Tree edge**: u is gray, v is white. The edge is the one DFS follows to discover v; it's part of the DFS tree.
- **Back edge**: u is gray, v is gray. The edge points to an ancestor in the DFS tree. Back edges are cycle witnesses.
- **Forward edge**: u is gray, v is black, *and* $d[u] < d[v]$. The edge points to a proper descendant via a non-tree edge.
- **Cross edge**: u is gray, v is black, *and* $d[u] > d[v]$. The edge points to a vertex in an unrelated, already-finished subtree.

For undirected graphs the classification collapses: every non-tree edge is a back edge, because forward and cross edges can't exist in a graph with no edge directions. That's also why undirected cycle detection has a different shape — for an undirected graph, a back edge to the immediate parent is *not* a real cycle, just the edge you came in on.

The classification is rarely needed as an end product, but it's the vocabulary other algorithms use. Strongly connected components (chapter 37) read tree-edge and back-edge structure via a per-vertex integer called the *low-link* — the earliest discovery time reachable from a vertex's subtree. Articulation points (the cut vertices whose removal disconnects the graph) are characterized by their relationship to back edges in the DFS tree. Bridges (the cut edges whose removal disconnects the graph) similarly. All of these are DFS plus one extra pass to compute one extra integer per vertex; the classification names the integers they care about.

33.6 One DFS per island

The connected-components sweep is the same algorithm as chapter 32's BFS variant, with DFS as the inner traversal. Pick an unvisited vertex, DFS from it to find its whole component, mark the component as visited, repeat. Total work is $\Theta(n + m)$ for the same reason BFS's sweep is $\Theta(n + m)$: every vertex and edge gets touched exactly once across the whole sweep.

```
def connected_components[V: Hashable](
    vertices: Iterable[V], neighbors: NeighborFn[V]
) -> list[set[V]]:
    """Sweep DFS over the vertex set. Return a list of components, each
    as the set of vertices reachable from one starting vertex via the
    undirected closure of neighbors. Same contract as the BFS sweep in
    chapter 32; this version uses iterative DFS to avoid recursion-limit
    issues on long chains.
    """
    seen: set[V] = set()
    components: list[set[V]] = []
    for start in vertices:
        if start in seen:
            continue
        component: set[V] = {start}
        stack: ArrayStack[V] = ArrayStack()
        stack.push(start)
        while stack:
            u = stack.pop()
            for v in neighbors(u):
                if v not in component:
                    component.add(v)
                    stack.push(v)
        seen |= component
        components.append(component)
    return components
```

Identical structure to the BFS sweep in chapter 32, with `ArrayStack` in place of `LinkedQueue` and `stack.push`/`stack.pop` in place of `queue.enqueue`/`queue.dequeue`. The output is the same list of component sets; only the within-component visit order changes. For component enumeration the distinction is invisible: both algorithms produce the same component sets.

```
from codex.graphs.adjacency_list import AdjacencyList
from codex.graphs.dfs import connected_components

g = AdjacencyList[int](directed=False)
for u, v in [(1, 2), (2, 3), (3, 4), (5, 6)]:
    g.add_edge(u, v)
g.add_vertex(7)

components = connected_components(g.vertices(), g.neighbors)
print(f"number of components: {len(components)}")
for i, comp in enumerate(sorted(components, key=lambda c: min(c))):
    print(f"   component {i + 1}: {sorted(comp)}")
```

```
number of components: 3
   component 1: [1, 2, 3, 4]
   component 2: [5, 6]
   component 3: [7]
```

Three components, same as the BFS sweep produced. Pick whichever sweep you prefer; for component enumeration alone they're interchangeable.

33.7 The three questions, applied

Is it correct?

The correctness of recursive DFS is one invariant: **at the moment `visit(u)` is called, `u` has been added to the `seen` set and every previously-seen vertex either has had `visit` called on it already or is on the call stack.** The invariant holds at the first call (only `source` is seen, nothing is on the stack yet) and is preserved by each recursive call (when `visit(v)` is called from inside `visit(u)`, `v` was just added to `seen` and `u` is on the stack as an ancestor). The recursion terminates because each call adds at least one vertex to `seen`, the graph is finite, and the invariant prevents the same vertex from being visited twice.

For `dfs_timestamps`, the parenthesis theorem is the correctness claim. Two vertices' intervals are properly nested or disjoint because at any moment, only the vertices currently on the recursion stack have open intervals. When `visit(v)` is called from inside `visit(u)`, `u`'s interval opens before `v`'s and closes after `v`'s does — `v`'s recursive call must return before `u`'s does, by the structure of nested function calls. So `[d[v], f[v]] \subseteq [d[u], f[u]]`. For two vertices that aren't on the same root-to-leaf path of the DFS tree, one's interval closes before the other's opens — they were discovered in different subtrees and the recursion left one before entering the other.

For `has_cycle`, correctness follows from "a gray vertex is one whose interval is open." If DFS encounters an edge from u to a gray v, then v is an ancestor of u in the DFS recursion — the path from v down to u via tree edges, plus the back edge from u to v, forms a cycle. Conversely, if the graph has a cycle, then on the first vertex of the cycle that DFS reaches, the recursion will eventually find an edge into another vertex of the cycle while that vertex is still gray. The three-color machine is the smallest representation of the gray/black distinction that lets the algorithm short-circuit on detection.

How efficient is it?

DFS runs in $\Theta(n + m)$ time and $\Theta(n)$ extra space on an adjacency-list graph. Each vertex is added to `seen` (or `discovery`, or the gray set) at most once and the addition is $O(1)$ amortized. Each edge is examined at most once per endpoint — twice total for undirected graphs, once for directed — and the examination is $O(1)$. The two pieces add to $\Theta(n + m)$. Space is dominated by either the `seen` set ($\Theta(n)$) or the recursion stack ($\Theta(n)$ in the worst case where the DFS tree is a chain); both are $\Theta(n)$.

The iterative form has the same time and space bounds, with the recursion-stack space moved to an explicit `ArrayStack` whose memory is on the heap rather than on the C call stack. For deep DFS trees this matters in practice — the iterative form can handle graphs that crash the recursive form — but the asymptotics are identical.

The recursive form's per-vertex constant factor is smaller (no stack object, no explicit push/pop) but is dominated by the call-frame overhead Python imposes on every function call. For dense graphs traversed entirely in cache, the iterative form is often a constant factor *faster* than the recursive form because the explicit stack avoids the Python interpreter's call-frame machinery. The two forms are asymptotically equivalent and which one is fastest in practice depends on the workload.

Is it optimal?

For visiting every vertex reachable from a source, $\Theta(n + m)$ is the lower bound — every reachable vertex has to be visited, every edge incident to a reachable vertex has to be examined to be sure the algorithm hasn't missed anything. DFS hits the bound, BFS hits the bound, both are worst-case optimal for the reachability problem.

For *cycle detection* in directed graphs, $\Theta(n + m)$ is the lower bound too: any algorithm must examine every edge at least once to be sure no cycle exists. The three-color DFS hits this bound. There is no faster way to detect cycles in the comparison model; the only improvement available is the constant factor that comes from short-circuiting on the first back edge.

For the *timestamping* output, recording $2n$ events takes $\Theta(n)$ time, plus the $\Theta(n + m)$ DFS traversal. Total $\Theta(n + m)$. Optimal in the same sense.

The deeper "optimality" question for DFS is about *which* DFS — the recursive form, the iterative form, a hybrid? — and the answer depends on the workload. For a fixed graph the algorithms agree on the reachable set and on the timestamps (given the same neighbor iteration order), but they differ in cache behavior, stack

overhead, and recursion-limit safety. Production graph libraries usually pick the iterative form for robustness and pay the small constant-factor cost.

33.8 What this chapter teaches

First, the choice of container determines what the algorithm computes. The five-line skeleton from chapter 32 — visit, mark, expand, repeat — is the same in BFS and DFS. The container in the middle is the entire difference. A queue gives FIFO, which gives distance order, which gives shortest paths. A stack gives LIFO, which gives the DFS tree, which gives discovery and finish times, which give cycle detection and topological order and articulation points. Their only difference is the data structure at the center: queue or stack. Everything else follows from that choice.

Second, DFS exposes structure the graph didn't appear to have. Walk the canonical graph from `A` and you visit six vertices. Walk it with timestamps and you get a parenthesization — a *tree* structure imposed on a graph that's a mesh. The DFS tree is not a property of the graph; it's a property of the DFS traversal, which depends on the neighbor iteration order. But the tree's structural facts — ancestor-descendant relationships, the existence of back edges that close cycles, the existence of cross edges that link unrelated subtrees — are graph-theoretic facts. DFS is how you compute them, and the parenthesization is how you express them.

A third observation, which the rest of the part will keep paying out. **Every "structural" question about a graph reduces to DFS plus one extra integer per vertex.** Cycle detection: one color per vertex. Topological sort: one finish-time per vertex (chapter 37). Strongly connected components: one *low-link* value per vertex, computed during DFS (chapter 37). Articulation points and bridges: one *discovery low* value per vertex. Each of these algorithms runs DFS, accumulates one or two integers per vertex at discovery and finish, and reads the answer off those integers — the same three moves, different integers. Chapter 37 will be one full application of the recipe; the rest of Part V will use the vocabulary developed here without re-deriving it.

33.9 Notes and further reading

The recursive form of DFS is essentially as old as recursion itself; the timestamp formulation and the parenthesis theorem are usually attributed to Robert Tarjan's 1972 paper "Depth-First Search and Linear Graph Algorithms" (*SIAM Journal on Computing* 1(2):146–160), which introduced both as ways to make DFS a tool for structural problems rather than just a traversal. Tarjan's algorithm for strongly connected components — the subject of chapter 37 — is the headline result of that paper, but the timestamping infrastructure he developed for the SCC algorithm is what underlies the cycle-detection, articulation-point, and bridge-finding algorithms that came after.

CLRS chapter 22 covers DFS with the same three-color machine and the same parenthesis theorem; the textbook treatment is essentially Tarjan's, expanded with explicit proofs and exercises. Sedgewick and Wayne's *Algorithms* (4th ed.) §4.1 covers DFS alongside BFS and walks through cycle detection and topological sort in the same chapter. For the iterative-DFS implementation issue — Python's recursion limit and the moral equivalent in other languages — the standard reference is the discussion in the Python documentation for `sys.setrecursionlimit`, which is more honest than most about why the limit exists (C stack overflow is unrecoverable) and why iterative-DFS is the production answer.

For the algorithmic uses of DFS beyond what this chapter covers: Hopcroft and Tarjan's 1973 paper "Algorithm 447: Efficient Algorithms for Graph Manipulation" (*CACM* 16(6):372–378) collects the DFS-based linear-time algorithms for articulation points, bridges, and biconnected components. The 2-SAT solver — determining whether a 2-CNF Boolean formula is satisfiable in polynomial time — reduces to SCC on the implication graph, another DFS application. Planarity testing in linear time is Hopcroft and Tarjan's 1974 result, again DFS-based. Every question this part keeps asking — does this graph have a cycle? what order should these tasks run in? which vertices, if removed, disconnect the network? — reduces to DFS plus one or two integers per vertex.

In the next chapter I'll change the container one more time. Where BFS used a queue and DFS used a stack, *Dijkstra's algorithm* uses a priority queue — the heap from chapter 19 finally earns its keep. The skeleton stays the same: visit, mark, expand neighbors, repeat. The new discipline at the container makes the algorithm extract vertices in order of *weighted distance from the source* instead of order of discovery or order of hop count, and the result is single-source shortest paths in graphs whose edges carry non-negative real weights.

34 BFS with a heap

The skeleton from chapter 32 had one job. Pull the next vertex from the container, expand its neighbors, repeat. When the container was a FIFO queue, vertices came out in order of *fewest hops* from the source and the algorithm computed shortest paths in unweighted graphs. Swap the queue for a min-heap keyed on tentative distance, and the vertices come out in order of *smallest accumulated weight* from the source. That is the entire change. The five-line skeleton stays. The greedy choice — always expand the vertex closest to the source, not the one discovered earliest — is what Dijkstra's algorithm contributes, and the heap from chapter 19 is the data structure that makes the greedy choice cheap. By the end of this chapter you'll have implemented Dijkstra's algorithm as one tight loop, computed weighted shortest paths from `A` to every other vertex of the canonical graph from chapter 31, watched the heap-based priority queue evolve through a hand-traceable seven-iteration run, and seen exactly why the algorithm breaks the moment an edge can carry a negative weight.

The vocabulary developed here — *relaxation*, *finalization*, *the greedy choice* — is what chapter 35 reuses when it generalizes to negative-weight edges, what chapter 36 reuses for Prim's minimum-spanning-tree algorithm (which is Dijkstra with `dist[u] + w(u, v)` replaced by just `w(u, v)`), and what chapter 38 reuses inside the augmenting-path loop of the max-flow algorithm. Three later chapters of Part V live downstream of this one; getting the algorithm and its invariant clear here makes those chapters short.

34.1 The relaxation step

The single move at the heart of Dijkstra is the *relaxation* of an edge. Maintain `dist[v]` as an upper bound on the true shortest-path distance from the source to `v` — initialized to infinity for every vertex other than the source, where it starts at zero. When the algorithm considers an edge (u, v) with weight w, it asks: *is the path that goes from the source to `u` and then takes this one edge to `v` shorter than my current upper bound on `dist[v]`?* If yes, update the bound. If no, do nothing. That's the entire relaxation step: a three-line conditional comparing `dist[u] + w` against `dist[v]`.

```
if dist[u] + weight < dist[v]:
    dist[v] = dist[u] + weight
    parent[v] = u
```

Dijkstra wraps this step in a specific traversal order. At any moment, the algorithm maintains two sets: *finalized* vertices, whose `dist` value is known to equal the true shortest-path distance, and *tentative* vertices, whose `dist` value is an upper bound that might still tighten. The algorithm repeatedly picks the tentative vertex with the smallest `dist`, moves it to the finalized set, and relaxes its outgoing edges. When every reachable vertex has been finalized, the algorithm halts and the `dist` dictionary holds the answer.

The *greedy choice* — picking the smallest-`dist` tentative vertex to finalize next — is what makes the algorithm work, and it's also what restricts it to non-negative weights. The correctness section shows exactly why; for now, take the algorithm on its terms and watch it run.

34.2 Twenty lines around a heap

The greedy choice requires a data structure that returns the smallest tentative `dist` in better than $O(n)$ time per query. A linear scan over the tentative set would give an overall $\Theta(n^2)$ algorithm — fine for dense graphs, wasteful for sparse ones. The min-heap from chapter 19 returns the smallest-priority item in $O(\log n)$ per operation, and pushing each vertex once per `dist` improvement gives $\Theta((n + m) \log n)$ total. That's the bound this implementation hits.

I'll use `HeapPriorityQueue[float, V]` from `codex.trees.heap` as the priority queue. The keys are tentative distances; the values are vertices. The wrinkle: a binary heap doesn't support efficient *decrease-key*, so when the

algorithm discovers a shorter path to an already-pushed vertex it can't lower that vertex's existing entry. The standard workaround is *lazy deletion* — push a new entry with the new smaller distance, and when the heap eventually returns one of the now-stale older entries, skip it. The `finalized` set is what makes the skip cheap: any popped entry whose vertex is already in `finalized` is stale and should be discarded.

```
from collections.abc import Callable, Hashable, Iterable
from math import inf
from codex.trees.heap import HeapPriorityQueue

type WeightedNeighborFn[V] = Callable[[V], Iterable[tuple[V, float]]]


def dijkstra[V: Hashable](
    source: V, neighbors: WeightedNeighborFn[V]
) -> tuple[dict[V, float], dict[V, V | None]]:
    """Dijkstra's single-source shortest-path algorithm. Returns
    (dist, parent), where dist[v] is the weighted shortest-path distance
    from source to v (math.inf if v is unreachable), and parent[v] is
    the predecessor of v on a shortest path. Requires every edge weight
    returned by neighbors to be non-negative; raises ValueError on a
    negative weight.
    """
    dist: dict[V, float] = {source: 0.0}
    parent: dict[V, V | None] = {source: None}
    finalized: set[V] = set()
    pq: HeapPriorityQueue[float, V] = HeapPriorityQueue()
    pq.push(0.0, source)
    while pq:
        d, u = pq.pop()
        if u in finalized:
            continue
        finalized.add(u)
        for v, w in neighbors(u):
            if w < 0:
                raise ValueError(f"Dijkstra cannot handle negative edge weight {w}")
            new_d = d + w
            if new_d < dist.get(v, inf):
                dist[v] = new_d
                parent[v] = u
                pq.push(new_d, v)
    return dist, parent
```

Twenty lines, including the docstring. The structure is the BFS skeleton from chapter 32 with three changes: the queue is a priority queue keyed by distance, the `seen` set is replaced by a `finalized` set (a vertex is "seen" the first time it's pushed but only `finalized` when it's popped for the first time), and the relaxation conditional has replaced the simple "v not in seen" gate. Every push has the same `(new_d, v)` shape, every pop checks `finalized` and skips stale entries, every relaxation either tightens the bound or does nothing. There's no other state.

The negative-weight check is a hard runtime error rather than a silent return of wrong answers. It's the kind of defensive validation worth keeping because the failure mode is so dangerous — without it, Dijkstra on a negative-weight graph would return *plausible-looking* but *wrong* distances, and silently. Chapter 35 will introduce Bellman-Ford, which handles negative weights correctly; until then, the check makes the failure visible.

The next section runs the algorithm against the canonical graph from chapter 31 and watches the heap evolve one pop at a time.

34.3 A run, traced by hand

Take the canonical six-vertex graph from chapter 31, with the weights I assigned there. Run Dijkstra from `A` and watch the heap and the `dist` dictionary evolve through each pop.

Initialization: `dist[A] = 0`, all other vertices implicit at ∞. The priority queue holds `(0, A)`.

Pop `(0, A)`. Finalize `A`. Relax `A`'s edges: `A → B` with weight 5 updates `dist[B] = 5`, `A → C` with weight 3 updates `dist[C] = 3`. Push `(5, B)` and `(3, C)`.

Pop `(3, C)`. Finalize `C`. Relax `C`'s edges: `C → A` would tighten `dist[A]`, but `A` is already finalized, so the relaxation is harmless (the conditional refuses to update because $0 + 3 = 3$ isn't less than 0). `C → B` offers $3 + 2 = 5$, which isn't an improvement over the current `dist[B] = 5`. `C → D` offers $3 + 1 = 4$, an improvement: `dist[D] = 4`, push `(4, D)`. `C → E` offers $3 + 4 = 7$: `dist[E] = 7`, push `(7, E)`.

Pop `(4, D)`. Finalize `D`. Relax: `D → B` offers $4 + 6 = 10$, worse. `D → C` blocked by finalization. `D → E` offers $4 + 2 = 6$, an improvement over the current 7: `dist[E] = 6`, push `(6, E)`. `D → F` offers $4 + 3 = 7$: `dist[F] = 7`, push `(7, F)`. The heap now holds `(5, B)`, `(6, E)`, `(7, E)`, `(7, F)` — note that *two* entries for `E` are alive in the heap, one stale (the `(7, E)` from earlier) and one current. Lazy deletion will sort it out.

Pop `(5, B)`. Finalize `B`. Relax: `B`'s neighbors are `A`, `C`, `D` — all already finalized, so nothing to do.

Pop `(6, E)`. Finalize `E`. Relax `E`'s edges: `C` and `D` finalized; `E → F` offers $6 + 5 = 11$, worse than the current 7.

Pop `(7, E)`. `E` is in `finalized` already. Skip — this was the stale entry from the third pop. The lazy-delete trick costs one $O(\log n)$ pop plus an $O(1)$ skip; that's the price of avoiding a real decrease-key implementation.

Pop `(7, F)`. Finalize `F`. Relax: `D` and `E` finalized; nothing to do.

The priority queue is now empty. The algorithm halts. Final distances: `dist[A] = 0`, `dist[B] = 5`, `dist[C] = 3`, `dist[D] = 4`, `dist[E] = 6`, `dist[F] = 7`.

Run the actual implementation against the trace.

```
from codex.graphs.adjacency_list import AdjacencyList
from codex.graphs.dijkstra import dijkstra

g = AdjacencyList[str](directed=False)
for u, v, w in [("A", "B", 5), ("A", "C", 3), ("B", "C", 2), ("B", "D", 6),
                ("C", "D", 1), ("C", "E", 4), ("D", "E", 2), ("D", "F", 3),
                ("E", "F", 5)]:
    g.add_edge(u, v, w)

dist, parent = dijkstra("A", g.weighted_neighbors)

print("vertex | dist | parent")
print("-------+------+-------")
for v in sorted(dist):
    print(f"   {v}   |   {dist[v]:>3.0f} |   {parent[v]}")
```

```
vertex | dist | parent
-------+------+-------
   A   |    0 |   None
   B   |    5 |   A
   C   |    3 |   A
   D   |    4 |   C
   E   |    6 |   D
   F   |    7 |   D
```

Same numbers as the hand trace. The parent dictionary records the predecessor on the chosen shortest path, which `reconstruct_path` from chapter 32 turns back into a source-first path.

```python
from codex.graphs.bfs import reconstruct_path

for target in ["B", "D", "E", "F"]:
    path = reconstruct_path(parent, target)
    weight = dist[target]
    print(f"   shortest path A -> {target} (weight {weight:.0f}): {path}")
```

```
   shortest path A -> B (weight 5): ['A', 'B']
   shortest path A -> D (weight 4): ['A', 'C', 'D']
   shortest path A -> E (weight 6): ['A', 'C', 'D', 'E']
   shortest path A -> F (weight 7): ['A', 'C', 'D', 'F']
```

Compare the paths to BFS from chapter 32. BFS reported the path `A → B → D → F` with three hops; Dijkstra reports `A → C → D → F` with weight 7, a fundamentally different route. BFS optimizes hop count, blind to weights. Dijkstra optimizes weighted distance, blind to hop count. The same canonical graph hides two different "shortest" paths depending on what metric you care about.

The `A → B → D → F` path that BFS picked has weight $5 + 6 + 3 = 14$. Twice Dijkstra's cost for the same hop count. If the weights model road distances or transit fares or processing time, the BFS path is the wrong answer. If they model just hop count or one-step costs, the BFS path is fine.

That correctness depends on one assumption the BFS comparison quietly ignored: no edge in the graph carried a negative weight.

34.4 Why non-negative weights

Here is where the non-negativity assumption does its work. Dijkstra's correctness rests on the *greedy choice property*: when the algorithm picks the smallest-`dist` tentative vertex `u` and finalizes it, the current `dist[u]` is already the true shortest-path distance. No path discovered later could be shorter. This holds because every other tentative vertex has `dist` $\geq$ `dist[u]`, and any path from the source that doesn't go through `u` directly has to pass through some other tentative vertex first — and from there, extending by even one more non-negative edge can only increase, not decrease, the total weight. So no detour through any other tentative vertex could beat the path the algorithm has already found.

The non-negativity assumption is exactly what makes that argument work. Allow even a single negative-weight edge and the property collapses. Consider a tiny graph with three vertices `A`, `B`, `C`, edges `A → B` with weight 5, `A → C` with weight 3, and `C → B` with weight -10. The shortest path from `A` to `B` is `A → C → B` with weight $3 - 10 = -7$. Dijkstra from `A` would pop `(0, A)`, finalize `A`, then pop `(3, C)`, finalize `C`, then pop `(5, B)`, finalize `B` with `dist[B] = 5`. That answer is wrong: at the moment `B` was finalized, the relaxation `dist[B] = 3 + (-10) = -7` was still pending in the future, but the greedy choice had already fired and committed to 5.

The implementation refuses to run on negative weights for exactly this reason. Bellman-Ford in chapter 35 handles the negative case by abandoning the greedy choice — it relaxes every edge in every iteration, $n - 1$ times, paying $\Theta(nm)$ for the privilege.

A more subtle issue: even *zero*-weight edges work fine for Dijkstra (the argument above only requires $w \geq 0$, not $w > 0$). A useful sanity check on any Dijkstra implementation is to verify that zero-weight edges don't cause infinite loops, double-counting, or any other pathology — the algorithm should simply treat a zero-weight edge as "instant teleport" and finalize the destination as soon as the source is finalized. The implementation above does that correctly because the lazy-deletion skip ignores re-finalized vertices.

34.5 The three questions, applied

Is it correct?

The correctness invariant is **when Dijkstra finalizes vertex `u` (i.e., pops it from the heap for the first time), `dist[u]` equals the true shortest-path distance from the source to `u`.** The proof is induction on the order of finalization. The source is finalized first with `dist[source] = 0`, which is correct. For any later vertex `u` finalized at distance `d`, consider the true shortest path P from the source to `u`. The first vertex of P that

hasn't been finalized at the moment `u` is popped is `u` itself, by the structure of the algorithm — every prefix of P up to but not including `u` consists of finalized vertices, because otherwise there'd be a tentative vertex on P with `dist` strictly less than `d`, contradicting the greedy choice. So P is the path-from-source-to-`u`-through-finalized-vertices, and Dijkstra has already relaxed every edge of P in the order they appear; by induction the relaxations recorded the true distance of each prefix, and the final relaxation set `dist[u]` to the true distance of P.

The one step the whole argument turns on is "any prefix of P has `dist` $\leq$ d at the moment `u` is popped." Non-negative weights are what make this true: extending any finalized vertex's distance by one more edge weight can only increase, not decrease, the result. With a negative-weight edge in the picture, the prefix-distance bound fails and the greedy choice produces wrong answers.

The lazy-deletion skip preserves correctness because a vertex that's already in `finalized` has, by the invariant above, its true shortest-path distance recorded; any later pop that names the same vertex with a larger distance is a stale upper bound that the algorithm can safely discard.

How efficient is it?

Dijkstra runs in $\Theta((n + m) \log n)$ time and $\Theta(n + m)$ space on an adjacency-list graph using a binary-heap priority queue. The heap holds at most m entries total over the course of the algorithm (one per edge relaxation that improved a distance), with each `push` and `pop` costing $O(\log m) = O(\log n)$ amortized (since $m \leq n^2$). The outer loop runs at most once per heap entry, so the total work is $\Theta(m \log n)$ for the heap operations plus $\Theta(n + m)$ for the constant-time relaxation conditionals. The two add to $\Theta((n + m) \log n)$.

Space is dominated by the heap, the `dist` and `parent` dictionaries, and the `finalized` set — each $O(n)$ in the worst case where every vertex is reachable. The heap can temporarily hold $\Theta(m)$ stale entries before they're popped and skipped; this is the lazy-deletion overhead. A version with explicit decrease-key (using a Fibonacci heap or a pairing heap) eliminates the stale entries and tightens the bound to $\Theta(n \log n + m)$, but the constant factors are bad enough that production implementations almost always stick with the binary heap.

For a dense graph with $m = \Theta(n^2)$, the binary-heap version is $\Theta(n^2 \log n)$, slightly worse than the $\Theta(n^2)$ achievable with a linear-scan implementation. A reasonable implementation picks the algorithm by density: linear scan for $m = \Omega(n^2 / \log n)$, binary heap otherwise. Most graphs in practice are sparse enough that the binary heap is the right call by a wide margin.

Is it optimal?

For single-source shortest paths in graphs with non-negative weights, the lower bound depends on the model. In the comparison-based model (the algorithm can compare edge weights but not, say, sort them in linear time using radix), $\Omega(n \log n + m)$ is tight: you need at least $n \log n$ to maintain the priority queue across n extract-min operations, and you need at least m to examine every edge. Dijkstra with a Fibonacci heap matches this bound. Dijkstra with a binary heap is $O(\log n)$ off for the dense case but within a constant factor for sparse graphs.

In a stronger model where the edge weights are bounded integers, Thorup's 1999 algorithm runs in $O(n+m)$ time — *linear* in the graph size, no $\log n$ factor — but the constant factors and the implementation complexity are large enough that it's almost never used in practice. For real workloads, the binary-heap Dijkstra is the algorithm production graph libraries ship, and the constant factors are good enough that the asymptotic suboptimality rarely matters.

For graphs with negative-weight edges, $\Omega(nm)$ is the lower bound in the comparison model (Bellman-Ford in chapter 35 matches this), and no asymptotic improvement is possible. Dijkstra's advantage over Bellman-Ford — a factor of $n / \log n$ — is bought entirely by the non-negativity assumption.

34.6 What this chapter teaches

First, the container at the center of the loop is the algorithm. This is the third time Part V has made the point. A queue gave BFS and shortest paths in hops; a stack gave DFS and structural decomposition; a heap gives Dijkstra and weighted shortest paths. The skeleton — visit, mark, expand, repeat — is the same in all three. The data structure is what changes, and the algorithm's character changes with it. The pattern is so reliable that the right way to design a new graph algorithm is often to ask: *what discipline at the container would compute the answer I want?* If you can answer that, the rest of the algorithm usually writes itself.

Second, relaxation is the unifying technique. Dijkstra is one of four shortest-path-family algorithms that all share the same per-edge step: compare `dist[u] + w(u, v)` against `dist[v]`, update if smaller. Bellman-Ford in chapter 35 does the same relaxation but applies it $n - 1$ times to every edge instead of using a heap. Floyd-Warshall in chapter 35 generalizes the relaxation to all pairs of vertices, with the intermediate vertex set growing one vertex at a time. Prim's MST in chapter 36 uses the same relaxation step with `dist[u] + w(u, v)` replaced by just `w(u, v)`. Four algorithms, one inner loop body.

The greedy choice property is the algorithm's correctness, and the choice of greedy criterion is what restricts the algorithm's applicability. Dijkstra's greedy choice — finalize the smallest-`dist` tentative vertex — works because non-negative weights guarantee that detours can only make paths longer. Other greedy graph algorithms have their own greedy choices and their own correctness properties: Kruskal's MST in chapter 36 picks the smallest-weight edge that doesn't form a cycle; Prim's MST in chapter 36 picks the smallest-weight edge crossing the cut between the current tree and the rest of the graph. In each case, naming the greedy criterion is half the algorithm; proving it correct is the other half. The pattern recurs throughout the algorithm literature, and getting comfortable with it here makes the rest of Part V's greedy algorithms — and Part VI's dynamic-programming alternatives, which arise precisely when no greedy choice works — easier to read.

34.7 Notes and further reading

Edsger Dijkstra published the algorithm in his 1959 paper "A Note on Two Problems in Connexion with Graphs" (*Numerische Mathematik* 1:269–271). The paper is three pages long and contains two unrelated graph algorithms — the shortest-path algorithm and an MST algorithm — both presented as elegant solutions to problems Dijkstra encountered while testing the ARMAC computer in Amsterdam. The original paper's data structure was a linear-scan tentative set, not a heap; the $\Theta((n+m)\log n)$ heap-based variant came later, and the Fibonacci-heap variant that matches the comparison-model lower bound is due to Fredman and Tarjan's 1984 paper "Fibonacci heaps and their uses in improved network optimization algorithms" (*JACM* 34(3):596–615).

CLRS chapter 24 covers Dijkstra with the same lazy-deletion technique used here and the same correctness argument, plus a treatment of decrease-key-capable priority queues (Fibonacci heaps, pairing heaps) that this chapter has skipped. Sedgewick and Wayne's *Algorithms* (4th ed.) §4.4 presents Dijkstra alongside Prim and shows the structural parallel explicitly — the same code with one substitution. For a deeper treatment of the algorithm's role in the shortest-path family, Robert Tarjan's *Data Structures and Network Algorithms* (SIAM, 1983) chapter 7 develops the relaxation framework and shows Dijkstra, Bellman-Ford, and Floyd-Warshall as variations on a single template.

For applications: Dijkstra powers OSPF (Open Shortest Path First, the dominant routing protocol on the Internet's interior gateways), every modern GPS routing service in some form (usually augmented with bidirectional search and contraction hierarchies, neither of which change the asymptotic complexity but make the constant factors dramatically better for road-network graphs), and the SCION inter-domain routing architecture's path selection. A* search (Russell and Norvig, *AIMA* chapter 3) is Dijkstra with a heuristic added to the priority — if the heuristic is admissible (never overestimates the true remaining distance), A* finds the same optimal paths Dijkstra finds, often expanding far fewer vertices.

For the bidirectional Dijkstra variant that runs the algorithm from both source and target simultaneously and meets in the middle, the canonical reference is Dantzig's 1960 monograph *Linear Programming and Extensions* §17.4; the technique is a $\sqrt{n}$-factor improvement on the explored-vertex count when both endpoints are known up front, and is the standard form in production routing engines.

In the next chapter I'll lift the non-negativity restriction. Bellman-Ford handles negative edges and detects negative-weight cycles; Floyd-Warshall pushes the relaxation framework to all-pairs shortest paths. Both are *relaxation* algorithms in the sense Dijkstra is — the per-edge step is identical — and the next chapter develops the family explicitly. The cost shape changes from $\Theta((n+m)\log n)$ to $\Theta(nm)$ for Bellman-Ford and $\Theta(n^3)$ for Floyd-Warshall, which is the price of giving up the greedy choice.

35 Relaxation under negative weights and across all pairs

Every edge in Dijkstra's algorithm runs through the same three-line test: compare `dist[u] + w(u, v)` against `dist[v]`, tighten the bound if the new path is cheaper, do nothing otherwise. That test — *relaxation* — is correct under any weight sign. What breaks under negative weights is not the test but the order in which Dijkstra applies it. The heap imposes a greedy order: finalize the tentative vertex with the smallest `dist`, relax its edges, and trust that no future path can improve on it. Non-negative weights make that trust warranted. A negative edge anywhere in the graph can break it: a path discovered later, routing through that negative edge, might be cheaper than something the algorithm committed to many pops ago. Remove the greedy order and you're left with the relaxation step running under its own discipline, and two distinct algorithms emerge from choosing what that discipline is.

By the end of this chapter you will have implemented both. Bellman-Ford replaces the heap with brute force: relax every edge, $n - 1$ times, in any order. It handles negative-weight edges correctly and detects the one condition that makes shortest paths undefined — a negative-weight cycle reachable from the source. Floyd-Warshall drops the single-source constraint entirely and computes shortest paths between every ordered pair of vertices by growing the allowed set of intermediate vertices one step at a time, turning the problem into a three-line dynamic-programming loop. Both algorithms share the same per-edge relaxation step from chapter 34. The cost shape changes — from $\Theta((n + m) \log n)$ to $\Theta(nm)$ for Bellman-Ford and $\Theta(n^3)$ for Floyd-Warshall — and that difference is exactly the price of giving up the greedy finalization order.

35.1 Why a negative edge defeats the greedy choice

Walk through a concrete failure. Four vertices: A, B, C, D. Edges: $A \to B$ with weight 3, $A \to C$ with weight 100, $B \to D$ with weight 1, $C \to B$ with weight -200. The true shortest paths from A are $\text{dist}[B] = -100$ (via $A \to C \to B$, weight $100 - 200$) and $\text{dist}[D] = -99$ (via $A \to C \to B \to D$). Dijkstra's algorithm — with the negative-weight check from chapter 34 disabled — would proceed as follows. Pop $(0, A)$, push $(3, B)$ and $(100, C)$. Pop $(3, B)$, finalize B at distance 3, push $(4, D)$. Pop $(4, D)$, finalize D at distance 4. Pop $(100, C)$, finalize C, relax $C \to B$ and discover a path of weight -100… but B is already in `finalized`. The skip fires. The algorithm returns $\text{dist}[B] = 3$, $\text{dist}[D] = 4$. Both are wrong by roughly a factor of a hundred.

The greedy choice bought its correctness from the guarantee that no detour can improve on a finalized vertex: any other tentative path would have to pass through some tentative vertex first, and those vertices all have distances at least as large, and extending forward by any non-negative edge can only increase the total. With a negative edge, that guarantee evaporates. A path that looks expensive early might become cheap when it hits the negative edge; the algorithm has no way to know, and once a vertex is finalized, the improvement that arrives later is ignored.

The fix is to abandon finalization entirely: relax every edge in the graph, repeatedly, until nothing changes. The number of passes required is at most $n - 1$. A shortest simple path in a graph with n vertices has at most $n - 1$ edges — if it had n or more edges, some vertex would appear twice, forming a cycle, and that cycle either has positive weight (making it non-optimal, so the shortest path wouldn't include it) or negative weight (which is the case we detect and flag separately). After k passes of relaxing every edge once, `dist[v]` holds the best path weight using at most k edges. After $n - 1$ passes, that covers all simple paths.

35.2 Relax every edge, over and over

The algorithm structure is three parts: initialize distances, run $n - 1$ relaxation passes over every edge, check for a negative-weight cycle with one additional pass. The implementation follows that outline directly.

```
from collections.abc import Hashable, Iterable
from math import inf

type WeightedEdge[V] = tuple[V, V, float]


def bellman_ford[V: Hashable](
    source: V,
    vertices: Iterable[V],
    edges: Iterable[WeightedEdge[V]],
) -> tuple[dict[V, float], dict[V, V | None]]:
    """Bellman-Ford single-source shortest paths. Handles negative-weight
    edges. Raises ValueError if a negative-weight cycle is reachable from
    source. Returns (dist, parent), where dist[v] is the shortest-path
    distance from source to v (inf if unreachable) and parent[v] is the
    predecessor of v on a shortest path.
    """
    dist: dict[V, float] = {v: inf for v in vertices}
    dist[source] = 0.0
    parent: dict[V, V | None] = {source: None}
    n = len(dist)
    edge_list = list(edges)
    for _ in range(n - 1):
        _relax_pass(edge_list, dist, parent)
    if _relax_pass(edge_list, dist, parent):
        raise ValueError("negative-weight cycle reachable from source")
    return dist, parent
```

The `list(edges)` materializes the edge iterator before the outer loop so that `_relax_pass` always iterates the same sequence in the same order. That matters for reproducibility and for the hand trace below. The outer loop runs exactly $n - 1$ times, where n is the number of vertices. The negative-cycle detection is the n-th pass: if any distance still improves after $n - 1$ passes, a path using n or more edges is shorter than any simple path, and the only explanation is a negative-weight cycle.

The helper that applies the full edge list once is the relaxation step from chapter 34, generalized to every edge at once:

```
def _relax_pass[V: Hashable](
    edges: list[WeightedEdge[V]],
    dist: dict[V, float],
    parent: dict[V, V | None],
) -> bool:
    """Relax every edge once. Return True if any distance improved."""
    improved = False
    for u, v, w in edges:
        d_u = dist.get(u, inf)
        if d_u + w < dist.get(v, inf):
            dist[v] = d_u + w
            parent[v] = u
            improved = True
    return improved
```

Twelve lines. The `dist.get(u, inf)` handles vertices not yet reachable: when `d_u` is `inf`, the expression `inf + w` stays `inf` for any finite `w`, so no update fires. The function returns a boolean flag — whether any distance improved — which is all the negative-cycle check needs: if the flag is `True` after $n - 1$ passes, some shortest path uses more than $n - 1$ edges, which implies a negative-weight cycle.

Run Bellman-Ford on the graph where Dijkstra was wrong.

```python
from codex.graphs.adjacency_list import AdjacencyList
from codex.graphs.relaxation import bellman_ford

g = AdjacencyList[str](directed=True)
for u, v, w in [("A", "B", 3), ("A", "C", 100), ("B", "D", 1), ("C", "B", -200)]:
    g.add_edge(u, v, w)

dist, parent = bellman_ford("A", g.vertices(), g.edges())

print("vertex | dist | parent")
print("-------+------+-------")
for v in sorted(dist):
    p = str(parent.get(v))
    print(f"   {v}   | {dist[v]:>5.0f} | {p!s:>6}")
```

```
vertex | dist | parent
-------+------+-------
   A   |     0 |   None
   B   |  -100 |      C
   C   |   100 |      A
   D   |   -99 |      B
```

`dist[B] = -100`, `dist[D] = -99`. The path through the negative edge is now captured correctly: $A \to C \to B$ beats the direct $A \to B$ by 103 units, and that improvement carries forward to D.

The ordering of edges inside `_relax_pass` determines how quickly the distances converge, not whether they converge. A lucky ordering — one that processes edges in path order from the source — might converge in one pass. An unlucky ordering might need all $n - 1$. The algorithm is correct regardless, because after k passes, `dist[v]` holds the shortest path weight using at most k edges.

35.3 Following the distances pass by pass

Here is a trace that needs more than one pass. Five vertices: A, B, C, D, E. Edges in the order `_relax_pass` will see them: $C \to D$ (weight 2), $B \to C$ (weight -3), $B \to E$ (weight 8), $A \to B$ (weight 4), $A \to C$ (weight 5).

The edges are deliberately ordered so that $C \to D$ appears before $B \to C$ — meaning C's distance hasn't been tightened by the $B \to C$ relaxation when $C \to D$ fires in pass 1. This forces the pass count upward.

Initialize: `dist = {C: ∞, D: ∞, B: ∞, E: ∞, A: 0}`.

Pass 1.

- $C \to D$: $\infty + 2 = \infty$. No update (C is unreachable).
- $B \to C$: $\infty + (-3) = \infty$. No update (B is unreachable).
- $B \to E$: $\infty + 8 = \infty$. No update.
- $A \to B$: $0 + 4 = 4 < \infty$. `dist[B] = 4`.
- $A \to C$: $0 + 5 = 5 < \infty$. `dist[C] = 5`.

After pass 1: `{A: 0, B: 4, C: 5, D: ∞, E: ∞}`. The pass found B and C but couldn't use them to reach D or update C via B: $C \to D$ ran before C was reachable, and $B \to C$ ran before B was reachable.

Pass 2.

- $C \to D$: $5 + 2 = 7 < \infty$. `dist[D] = 7`.
- $B \to C$: $4 + (-3) = 1 < 5$. `dist[C] = 1`.
- $B \to E$: $4 + 8 = 12 < \infty$. `dist[E] = 12`.
- $A \to B$, $A \to C$: no improvement.

After pass 2: `{A: 0, B: 4, C: 1, D: 7, E: 12}`. `dist[D]` is still wrong: the pass saw $C \to D$ before $B \to C$ had a chance to tighten C to 1, so D was updated using the old $C = 5$. The cheaper path $A \to B \to C \to D$ of weight $4 - 3 + 2 = 3$ hasn't been assembled yet.

Pass 3.

- $C \to D$: $1 + 2 = 3 < 7$. `dist[D] = 3`.
- Everything else: no improvement.

After pass 3: `{A: 0, B: 4, C: 1, D: 3, E: 12}`. All distances are correct. The path $A \to B \to C \to D$ used three edges and took three passes to assemble in this edge ordering — exactly the correspondence the $n - 1$ bound captures.

Confirm with the actual implementation:

```
from codex.graphs.adjacency_list import AdjacencyList
from codex.graphs.relaxation import bellman_ford

g = AdjacencyList[str](directed=True)
for u, v, w in [("C", "D", 2), ("B", "C", -3), ("B", "E", 8), ("A", "B", 4), ("A", "C", 5)]:
    g.add_edge(u, v, w)

dist, parent = bellman_ford("A", g.vertices(), g.edges())

print("vertex | dist | path from A")
print("-------+------+------------")
for v in sorted(dist):
    path, cur = [], v
    while cur is not None:
        path.append(cur)
        cur = parent.get(cur)
    path.reverse()
    print(f"   {v}   |   {dist[v]:>2.0f} | {'→'.join(path)}")
```

```
vertex | dist | path from A
-------+------+------------
   A   |    0 | A
   B   |    4 | A→B
   C   |    1 | A→B→C
   D   |    3 | A→B→C→D
   E   |   12 | A→B→E
```

Distances `{A:0, B:4, C:1, D:3, E:12}` with paths `A→B`, `A→B→C`, `A→B→C→D`, `A→B→E`. The parent reconstruction is the same logic as chapter 32's `reconstruct_path`, applied from each target back to the source. The next question is what happens when a n-th pass would still improve a distance — the negative-cycle case.

35.4 One more pass reveals a negative cycle

Negative-weight cycles make shortest paths undefined: follow the cycle repeatedly and the path weight decreases without bound. The n-th Bellman-Ford pass is the witness. After $n-1$ correct passes, any remaining improvement must come from a path using n or more edges — the only way that happens is if a negative-weight cycle is reachable from the source.

The implementation raises `ValueError` rather than returning a silently wrong answer. The rationale is the same as Dijkstra's negative-weight check in chapter 34: a shortest-path distance of negative infinity is not a useful answer, and returning a large negative finite number in its place would mislead any caller who treats the output as a real distance.

```python
from codex.graphs.adjacency_list import AdjacencyList
from codex.graphs.relaxation import bellman_ford

g_neg = AdjacencyList[str](directed=True)
for u, v, w in [("A", "B", 3), ("B", "C", -2), ("C", "A", -2)]:
    g_neg.add_edge(u, v, w)
# cycle A→B→C→A has weight 3 + (-2) + (-2) = -1

try:
    bellman_ford("A", g_neg.vertices(), g_neg.edges())
    print("no negative cycle detected")
except ValueError as e:
    print(f"caught: {e}")
```

```
caught: negative-weight cycle reachable from source
```

The cycle $A \to B \to C \to A$ has weight -1. Traversing it once shaves a unit off every path that passes through it; traversing it k times shaves k units. Bellman-Ford correctly flags the graph rather than returning a finite answer.

The implementation detects whether *any* vertex reachable from the source is on or downstream of a negative cycle. If you need to know which specific vertices are affected — to mark them as "undefined" in the distance table rather than raising an exception — the standard extension is to run the n-th pass, collect all vertices whose distance improved, and then sweep reachability from those vertices forward through the graph. The implementation here keeps the detection minimal; the extension is one DFS away. Bellman-Ford has handled one source at a time throughout; the next algorithm drops that constraint entirely.

35.5 Shortest paths between every pair of vertices

Single-source shortest paths answer "how far is every vertex from this one starting point?" All-pairs shortest paths answer a different question: "for any two vertices, what is the cheapest path between them?" The brute-force approach is to run Bellman-Ford from every vertex — n single-source computations at $\Theta(nm)$ each, for a total of $\Theta(n^2 m)$. Floyd-Warshall solves the same problem in $\Theta(n^3)$, which is better whenever $m = \Omega(n)$. The gain comes from reusing intermediate shortest-path computations across multiple source-destination pairs instead of recomputing them from scratch for every source.

Here is the decomposition I want you to see. Fix two vertices i and j and ask: what is the shortest path from i to j using only vertices $\{v_1, v_2, \ldots, v_k\}$ as intermediates? The answer satisfies a recurrence: either the optimal path doesn't use v_k as an intermediate (and the answer is the same as for $\{v_1, \ldots, v_{k-1}\}$), or it does (and the path goes from i to v_k optimally using $\{v_1, \ldots, v_{k-1}\}$ as intermediates, then from v_k to j optimally the same way). The minimum of those two cases is the answer:

$$\text{dist}_k[i][j] = \min(\text{dist}_{k-1}[i][j],\ \text{dist}_{k-1}[i][v_k] + \text{dist}_{k-1}[v_k][j])$$

The base case is $k = 0$: $\text{dist}_0[i][j]$ is the weight of the direct edge (i, j) if it exists, or ∞ if it doesn't (and 0 for $i = j$). When k reaches n, every vertex is a possible intermediate and $\text{dist}_n[i][j]$ is the true all-pairs shortest path.

This recurrence has the shape of dynamic programming: a two-dimensional table, filled in a topological order (increasing k), where each cell depends only on cells from the previous layer. The edit-distance table from chapter 30 had the same shape with a different recurrence. Part VI will generalize the framework — subproblem structure, overlapping subproblems, optimal substructure — and Floyd-Warshall will appear there as a canonical example alongside edit distance. The intermediate-vertex recurrence produces the algorithm almost automatically: once you write down the DP, the code is three nested loops.

35.6 The three-line loop that builds a matrix

Watch how cleanly the recurrence turns into an implementation. Initialize a two-dimensional distance table from the direct edge weights, then update it one intermediate vertex at a time.

```
def floyd_warshall[V: Hashable](
    vertices: Iterable[V],
    edges: Iterable[WeightedEdge[V]],
) -> dict[V, dict[V, float]]:
    """Floyd-Warshall all-pairs shortest paths. Returns dist where
    dist[u][v] is the shortest-path distance from u to v (inf if
    unreachable). Handles negative-weight edges. A negative-weight cycle
    is indicated by dist[v][v] < 0 for any vertex v after the call.
    """
    vlist = list(vertices)
    dist: dict[V, dict[V, float]] = {
        u: {v: (0.0 if u == v else inf) for v in vlist}
        for u in vlist
    }
    for u, v, w in edges:
        if w < dist[u][v]:
            dist[u][v] = w
    for k in vlist:
        _relax_via(vlist, dist, k)
    return dist
```

The initialization builds an $n \times n$ table: zeros on the diagonal, direct edge weights off-diagonal (keeping the minimum if parallel edges exist), and `inf` for all pairs with no direct edge. Then the outer loop runs once per vertex in `vlist` order, calling the helper that applies the recurrence for one intermediate vertex.

```
def _relax_via[V: Hashable](
    vertices: list[V],
    dist: dict[V, dict[V, float]],
    k: V,
) -> None:
    """Update dist[i][j] for all i, j by routing through intermediate k."""
    for i in vertices:
        for j in vertices:
            through_k = dist[i][k] + dist[k][j]
            if through_k < dist[i][j]:
                dist[i][j] = through_k
```

Seven lines. The inner double loop over all (i, j) pairs is the entire cost driver: n^2 pairs, times n outer iterations, gives n^3 relaxation steps in total, each $O(1)$. There is no heap, no visited set, no recursion. Just three nested loops over the vertex list.

The in-place update is correct for graphs without negative cycles. The standard argument: in the k-th outer iteration, reading `dist[i][k]` yields the shortest path from i to v_k using intermediates $\{v_1, \ldots, v_{k-1}\}$ — the same value the recurrence would compute for `dist[k-1][i][k]`. The reason is that `dist[i][k]` cannot be improved by using v_k as an intermediate for the i-to-v_k path, because that would require v_k to appear twice, forming a cycle, and on a cycle-free shortest path that never happens. The same argument covers `dist[k][j]`. So the in-place reads are correct even though the table is being modified simultaneously.

This implementation does not reconstruct paths — it returns only the distance table. Adding path reconstruction requires a second $n \times n$ table `next[i][j]` initialized with the direct successor on each direct edge and updated alongside `dist`. For the pedagogical goal of this chapter, the distance table is enough; the reconstruction extension is one more table and one more update per inner-loop iteration.

Run the algorithm on a four-vertex directed graph with negative edges.

```
from codex.graphs.adjacency_list import AdjacencyList
from codex.graphs.relaxation import floyd_warshall
from math import inf

g = AdjacencyList[str](directed=True)
for u, v, w in [("A", "B", 3), ("B", "C", -2), ("C", "D", 2),
                ("D", "A", 5), ("A", "D", 10), ("B", "D", 8)]:
    g.add_edge(u, v, w)

dist = floyd_warshall(g.vertices(), g.edges())

vertices = sorted(dist.keys())
header = "     | " + " | ".join(f"  {v}  " for v in vertices)
sep = "-----+" + "------+" * len(vertices)
print(header)
print(sep)
for u in vertices:
    row = " | ".join(
        f"  {dist[u][v]:3.0f} " if dist[u][v] < inf else "  ∞  "
        for v in vertices
    )
    print(f"  {u}  | {row}")
```

```
     |   A   |   B   |   C   |   D
-----+------+------+------+------+
  A  |     0  |     3  |     1  |     3
  B  |     5  |     0  |    -2  |     0
  C  |     7  |    10  |     0  |     2
  D  |     5  |     8  |     6  |     0
```

The distance from A to C is 1: the path $A \to B \to C$ costs $3 + (-2) = 1$, cheaper than any direct edge. From D to C the distance is 6: the path $D \to A \to B \to C$ costs $5 + 3 - 2 = 6$. The negative edge $B \to C$ propagates its savings into all paths that route through B and then continue to C or beyond.

One entry worth checking against the trace above: $\text{dist}[C][B]$. There is no direct edge $C \to B$. The shortest path goes $C \to D \to A \to B$ with weight $2 + 5 + 3 = 10$. Floyd-Warshall assembles this in two steps: in the D-intermediate pass it discovers $\text{dist}[C][A] = 2{+}5 = 7$; in the A-intermediate pass it discovers $\text{dist}[C][B] = 7{+}3 = 10$. The recurrence does the composition one intermediate at a time, and the incremental structure is what avoids the $\Theta(n^2 m)$ cost of running Bellman-Ford from every vertex separately.

35.7 The three questions, applied

Is it correct?

Bellman-Ford's correctness follows from an inductive argument on pass count. **Invariant: after pass k, `dist[v]` holds the weight of the shortest path from `source` to `v` using at most k edges.** The base case holds before any pass: `dist[source] = 0` (a zero-edge path) and `dist[v] = ∞` for all other vertices. For the inductive step: in pass k, when `_relax_pass` considers edge (u, v, w), `dist[u]` is (by induction) the weight of the shortest path to u using at most $k-1$ edges. If `dist[u] + w < dist[v]`, a shorter path to v using at most k edges is recorded. Every such shorter path is found because `_relax_pass` examines every edge. After $n-1$ passes the invariant gives the correct shortest-path weight for every vertex, since shortest simple paths have at most $n-1$ edges.

The ordering of edges within each pass affects convergence speed but not correctness. A lucky ordering (edges processed in topological order from the source) converges in one pass; an unlucky ordering may use all $n-1$. Either way the invariant holds.

Floyd-Warshall's correctness is the standard DP argument. The recurrence correctly covers both cases — path not through v_k, path through v_k — for every pair (i, j). The in-place update is correct for the reason given in

the implementation section: `dist[i][k]` and `dist[k][j]` are not affected by updating `dist[i][j]` in the same k-pass, because using v_k twice would create a cycle and shortest simple paths avoid cycles. For graphs with negative cycles, `dist[v][v]` becomes negative after the algorithm, which is the caller's signal that the distances are undefined.

How efficient is it?

Bellman-Ford runs in $\Theta(nm)$ time and $\Theta(n+m)$ space. The outer loop runs $n-1$ times plus one for the cycle check. Each pass iterates all m edges in $\Theta(m)$ time. Total: $\Theta(nm)$. Space is dominated by the materialized edge list ($\Theta(m)$) and the `dist` and `parent` dictionaries ($\Theta(n)$ each).

As an optimization, the outer loop can terminate early if a pass produces no improvements — `_relax_pass` returning `False` means the distances are already optimal and no further passes are needed. On a graph where the shortest paths are short on average, this early termination makes Bellman-Ford much faster than the $\Theta(nm)$ worst case.

Floyd-Warshall runs in $\Theta(n^3)$ time and $\Theta(n^2)$ space regardless of graph structure. There are no shortcuts: the three nested loops always execute n^3 iterations. The n^2 space is the distance table itself. For sparse graphs, $m \ll n^2$, and running Bellman-Ford from every vertex in turn costs $\Theta(n^2 m) \ll \Theta(n^4)$; Floyd-Warshall at $\Theta(n^3)$ beats that. For very sparse graphs with negative-weight edges, Johnson's algorithm (reweight edges with a Bellman-Ford potential, then run Dijkstra from every vertex) achieves $\Theta(n^2 \log n + nm)$, which beats Floyd-Warshall on sparse graphs. Johnson's algorithm is the all-pairs algorithm production graph libraries use when the graph has negative weights and $m \ll n^2$.

Is it optimal?

For single-source shortest paths with negative-weight edges, $\Theta(nm)$ is essentially tight in the comparison model. Any correct algorithm must examine every edge at least once (otherwise it could miss a negative edge that improves some distance), and on a graph where the shortest path uses $n-1$ edges, a single pass is not enough. Bellman-Ford matches both lower bounds.

For all-pairs shortest paths, the output alone is n^2 values, so $\Omega(n^2)$ is a trivial lower bound. The best general upper bound is $\Theta(n^3)$ from Floyd-Warshall. Closing that gap is open: the fastest algorithms use $(\min, +)$-matrix multiplication and achieve $O(n^3/\log^2 n)$ in theory, but the constant factors are large enough that $\Theta(n^3)$ with SIMD vectorization beats them in practice on any realistic graph size. For production purposes, Floyd-Warshall on dense graphs and Johnson's algorithm on sparse graphs are the state of the art, and the gap between them and the theoretical lower bound doesn't matter at any scale where the algorithm would actually be run.

35.8 What this chapter teaches

The relaxation step is the algorithm; the scheduling is a design choice. Dijkstra, Bellman-Ford, and Floyd-Warshall all use the same three-line test — compare a proposed path weight against the current best, update if cheaper. The algorithms differ only in how they schedule that test: greedy heap order, brute-force pass-over-all-edges order, or DP intermediate-vertex order. When you encounter a new shortest-path algorithm, the useful question is not "what does it compute?" (they all compute shortest paths) but "what order does it relax edges in, and what assumption about the graph makes that order correct?" Answering that question immediately gives you the algorithm's correctness conditions and its cost shape.

Generality costs a factor of $n/\log n$ off the greedy optimum. Dijkstra on a sparse graph runs in $\Theta(m \log n)$; Bellman-Ford runs in $\Theta(nm)$. The extra factor is the cost of giving up the greedy finalization order. The greedy order is what makes Dijkstra visit each vertex exactly once; without it, you can't know when to stop relaxing, so you relax everything repeatedly until nothing moves. Any shortest-path algorithm whose correctness argument is "after k passes, the answer uses paths of at most k hops" will cost at least $\Theta(nm)$ — the $n-1$ worst-case pass count is irreducible for adversarial edge orderings.

Recognizing the DP recurrence in Floyd-Warshall opens the door to Part VI. The intermediate-vertex recurrence $\text{dist}_k[i][j] = \min(\text{dist}_{k-1}[i][j], \text{dist}_{k-1}[i][v_k] + \text{dist}_{k-1}[v_k][j])$ is a textbook example of optimal substructure: the optimal solution to the k-intermediate problem decomposes into optimal solutions to the $(k-1)$-intermediate sub-problems. The edit-distance recurrence from chapter 30 had the same shape — optimal alignment of two strings decomposes into optimal alignments of their prefixes. Part VI will develop the framework

explicitly, name the conditions (optimal substructure, overlapping sub-problems), and show that Floyd-Warshall and edit distance are instances of one design pattern. The SCC condensation from chapter 37 will turn a directed graph into a DAG on which DP runs in linear time, completing the feedback loop between graph algorithms and dynamic programming.

35.9 Notes and further reading

Richard Bellman published the single-source negative-weight algorithm in 1958 in "On a Routing Problem" (*Quarterly of Applied Mathematics* 16(1):87–90). The same algorithm appeared independently in a RAND Corporation technical report by Lester Ford Jr. in 1956 (published 1958), which is why the algorithm is sometimes called Bellman-Ford-Moore — Edward Moore published a closely related traversal in 1959. The $\Theta(nm)$ analysis and negative-cycle detection via the n-th pass are standard across every treatment of the algorithm; CLRS chapter 24 gives the textbook version with explicit proofs.

Robert Floyd published the all-pairs algorithm in 1962 ("Algorithm 97: Shortest Path," *CACM* 5(6):345). Stephen Warshall had published a reachability-only variant one year earlier ("A Theorem on Boolean Matrices," *JACM* 9(1):11–12, 1962), computing whether paths exist rather than their weights. Peter Ingerman showed independently in 1962 that the same recurrence computes shortest-path weights; the combined Floyd-Warshall attribution names both the distance variant (Floyd) and the transitive-closure variant (Warshall). The DP derivation via the intermediate-vertex recurrence is due to Ingerman and is the framing CLRS chapter 25 uses.

Johnson's $\Theta(n^2 \log n + nm)$ algorithm for all-pairs shortest paths on sparse graphs with negative weights appeared in "Efficient Algorithms for Shortest Paths in Sparse Networks" (*JACM* 24(1):1–13, 1977). The reweighting trick it uses — compute a potential function via Bellman-Ford, add the potential to all edge weights to make them non-negative, run Dijkstra from every vertex, subtract the potentials — is an elegant composition of the two algorithms from this chapter. CLRS §25.3 presents it in detail.

The faster-than-$\Theta(n^3)$ algorithms for all-pairs shortest paths use $(\min, +)$-matrix multiplication — the algebraic structure where matrix "product" computes minimum-weight one-hop extensions instead of sums of products. The best known bound is $O(n^3 / \log^2 n)$ by Fredman (1976) and subsequent authors. The constant factors are large enough that these algorithms are never used in practice; hand-tuned Floyd-Warshall with SIMD vectorization achieves roughly an order-of-magnitude speedup over the naive triple loop while remaining algorithmically $\Theta(n^3)$.

Sedgewick and Wayne's *Algorithms* (4th ed.) §4.4 presents Bellman-Ford, Floyd-Warshall, and Dijkstra in the same chapter alongside each other, making the scheduling contrast explicit. For a unified treatment of the relaxation framework — in which Dijkstra, Bellman-Ford, Floyd-Warshall, and Prim are all instances of one meta-algorithm — Tarjan's *Data Structures and Network Algorithms* (SIAM, 1983) chapter 7 is the canonical reference.

In the next chapter I'll move from shortest paths to spanning trees. A minimum spanning tree — the cheapest subset of edges that keeps the graph connected — rests on a single theorem, the cut property, that looks different from the relaxation framework but turns out to share its structure. Kruskal's algorithm processes edges sorted by weight and adds each one that doesn't form a cycle; Prim's algorithm grows a tree from a single vertex using the same heap from this chapter and the same greedy finalization order from Dijkstra. Both are the cut property applied in two different orderings.

36 Spanning trees of minimum weight

A spanning tree of a connected graph is a subgraph that includes every vertex and enough edges to keep them connected, with no cycles. There are many spanning trees of most graphs — the complete graph on n vertices has n^{n-2} of them by Cayley's formula — but among all of them one question stands out: which spanning tree minimizes the total edge weight? Two algorithms answer it. Kruskal's algorithm scans the edge list sorted by weight and adds each edge that doesn't form a cycle, stopping when the tree has $n-1$ edges. Prim's algorithm grows a single tree from one starting vertex, repeatedly adding the minimum-weight edge connecting a tree vertex to a non-tree vertex. Both give the same answer on any graph, and the reason both are correct — the reason *any* locally greedy spanning-tree algorithm is correct, under the right conditions — is a single theorem about graph cuts.

By the end of this chapter you will have implemented both algorithms, confirmed that they produce the same minimum spanning tree on the canonical graph from chapter 31, understood the cut property that makes both correct, and seen explicitly that Kruskal and Prim are the same proof read in two different orders. Kruskal uses the union-find structure from chapter 14 to decide in near-constant time whether adding an edge would close a cycle. Prim uses the heap from chapter 19 in the same role it plays in Dijkstra — as a priority queue that returns the cheapest candidate edge in $O(\log n)$ time. Both algorithms produce the $n-1$ edges of a minimum spanning tree in total time $O(m \log m)$ for Kruskal and $\Theta((n+m)\log n)$ for Prim.

36.1 The theorem both algorithms rest on

Here is the one result both algorithms depend on. A *cut* of a graph is any partition of its vertices into two non-empty sets S and $V \setminus S$. An edge *crosses* the cut if one endpoint is in S and the other is in $V \setminus S$. The cut property is the one theorem the whole chapter turns on:

Cut property. *For any cut $(S, V \setminus S)$ of a connected weighted graph with distinct edge weights, the minimum-weight edge crossing the cut belongs to every minimum spanning tree.*

The proof is short. Let $e = (u, v, w)$ be the minimum-weight edge crossing the cut, and let T be any spanning tree that does not contain e. Since T is a spanning tree, it has a path from u to v. That path must cross the cut — it starts in S (at u) and ends in $V \setminus S$ (at v) — so it uses at least one edge $e' = (u', v', w')$ that also crosses the cut. Since e is the minimum-weight cut-crossing edge and $e' \neq e$, we have $w < w'$. Now swap e' for e: the result is still a spanning tree (we removed one edge and added one that reconnects the two parts), and its total weight is strictly less than T's. So T is not a minimum spanning tree. Contradiction. Therefore every MST contains e.

The converse holds too: for any edge e that is not the minimum-weight crossing edge for *any* cut, there exists an MST that doesn't contain e. The cut property completely characterizes which edges can and must appear in any MST.

What makes the cut property the right tool for analyzing both algorithms is that each algorithm's greedy local choice corresponds to finding a minimum-weight crossing edge for some cut. Kruskal's cut, at each step, is the partition into the two components that would be merged by the candidate edge. Prim's cut, at each step, is the partition into the current tree vertices and everything else. The greedy choices look different in code; in terms of the cut property they are the same choice, made in a different order.

36.2 Sorting edges until the tree is built

Kruskal's algorithm is almost embarrassingly simple once you have the right data structure. Sort all edges by weight. Walk down the sorted list; for each edge, check whether its two endpoints are already in the same component. If yes, adding the edge would form a cycle — skip it. If no, add the edge and merge the two components. Stop when $n-1$ edges have been added.

The component check and merge is exactly the problem the union-find from chapter 14 was built for. Each component is a set in the partition; `find` returns the set's representative; `connected` checks whether two vertices share a representative; `union` merges two sets. With path-compressed union-find, each check and merge costs $O(\alpha(n)) \approx O(1)$ amortized, making the dominant cost the initial sort: $O(m \log m)$.

```
from collections.abc import Callable, Hashable, Iterable
from math import inf
from codex.structures.union_find import UnionFind
from codex.trees.heap import HeapPriorityQueue

type WeightedEdge[V] = tuple[V, V, float]
type WeightedNeighborFn[V] = Callable[[V], Iterable[tuple[V, float]]]


def kruskal[V: Hashable](
    vertices: Iterable[V],
    edges: Iterable[WeightedEdge[V]],
) -> list[WeightedEdge[V]]:
    """Kruskal's minimum spanning tree. Sorts edges by weight and adds
    each one that doesn't form a cycle, using union-find for cycle
    detection. Returns the MST as a list of edges. On a disconnected
    graph, returns a minimum spanning forest (one tree per component).
    """
    vlist = list(vertices)
    index = {v: i for i, v in enumerate(vlist)}
    uf = UnionFind(len(vlist))
    mst: list[WeightedEdge[V]] = []
    for u, v, w in sorted(edges, key=lambda e: e[2]):
        iu, iv = index[u], index[v]
        if not uf.connected(iu, iv):
            uf.union(iu, iv)
            mst.append((u, v, w))
    return mst
```

Twenty lines with the import and type alias. The body is the four-step description above, translated directly: sort, walk, check `connected`, call `union` and append. The `index` dictionary maps each vertex to its integer index in the union-find structure — the mapping exists because `UnionFind` from chapter 14 operates on integers 0 to $n - 1$, not on arbitrary vertex labels. Everything else in the code is the algorithm, unchanged.

Run it on the canonical graph.

```
from codex.graphs.adjacency_list import AdjacencyList
from codex.graphs.mst import kruskal

g = AdjacencyList[str](directed=False)
for u, v, w in [("A", "B", 5), ("A", "C", 3), ("B", "C", 2), ("B", "D", 6),
                ("C", "D", 1), ("C", "E", 4), ("D", "E", 2), ("D", "F", 3),
                ("E", "F", 5)]:
    g.add_edge(u, v, w)

mst = kruskal(g.vertices(), g.edges())

total = sum(w for _, _, w in mst)
print(f"MST total weight: {total}")
print("edges:")
for u, v, w in sorted(mst, key=lambda e: e[2]):
    print(f"  {u}-{v}  weight {w}")
```

```
MST total weight: 11
edges:
```

```
  C-D  weight 1
  B-C  weight 2
  D-E  weight 2
  A-C  weight 3
  D-F  weight 3
```

Five edges for six vertices, total weight 11: `C-D(1)`, `B-C(2)`, `D-E(2)`, `A-C(3)`, `D-F(3)`. The algorithm added them in that order, skipping `A-B(5)`, `C-E(4)`, `B-D(6)`, and `E-F(5)` because each of those edges would have formed a cycle.

The cut-property reading of each decision: when `C-D(1)` is considered, the cut $(C, V \setminus C)$ has `C-D` as its minimum-weight crossing edge, so the cut property says it must be in the MST. When `B-C(2)` is considered next, the relevant cut is $(\text{component}(B), V \setminus \text{component}(B)) = (B, A, C, D, E, F)$, and `B-C(2)` is the cheapest crossing edge. Each Kruskal step is a local application of the cut property, and the global correctness follows. Prim's algorithm uses the same theorem but reads it from a different direction.

36.3 Growing the tree from one vertex

Prim's algorithm starts at one vertex and grows the MST outward, one vertex at a time. At every step, it picks the cheapest edge connecting the current tree to a vertex not yet in the tree, adds that vertex, and continues. The loop body is exactly Dijkstra's loop body from chapter 34, with one substitution: instead of maintaining `dist[v] = dist[u] + w(u, v)` (path distance from source), it maintains `key[v] = w(u, v)` (weight of the single edge connecting v to the current tree). The skeleton stays the same.

The substitution is the entire difference between Dijkstra and Prim. Dijkstra minimizes total path length from a fixed source; Prim minimizes the edge weight to the current tree. Both use a min-heap, both use lazy deletion to handle stale entries, both finalize each vertex exactly once. The cost shape is the same: $\Theta((n+m)\log n)$ with a binary heap.

```python
def prim[V: Hashable](
    source: V,
    neighbors: WeightedNeighborFn[V],
) -> tuple[dict[V, float], dict[V, V | None]]:
    """Prim's minimum spanning tree. Grows the MST from source using
    a min-heap keyed on edge weight (not accumulated path distance).
    Returns (key, parent) where key[v] is the weight of the MST edge
    connecting v to the tree, and parent[v] is v's predecessor in the
    MST. source maps to (0.0, None). Unreachable vertices don't appear.
    """
    key: dict[V, float] = {source: 0.0}
    parent: dict[V, V | None] = {source: None}
    in_mst: set[V] = set()
    pq: HeapPriorityQueue[float, V] = HeapPriorityQueue()
    pq.push(0.0, source)
    while pq:
        k, u = pq.pop()
        if u in in_mst:
            continue
        in_mst.add(u)
        for v, w in neighbors(u):
            if v not in in_mst and w < key.get(v, inf):
                key[v] = w
                parent[v] = u
                pq.push(w, v)
    return key, parent
```

Twenty-two lines. Compare with Dijkstra's twenty lines: the differences are the variable names (`key` instead of `dist`, `in_mst` instead of `finalized`) and the one-line change from `new_d = d + w; if new_d < dist.get(v, inf)` to `if w < key.get(v, inf)`. The heap push is `pq.push(w, v)` instead of `pq.push(new_d, v)`. That's the entire algorithmic difference. Everything else — the lazy-deletion skip on `in_mst`, the parent recording, the early-termination condition, the outer loop — is identical.

```
from codex.graphs.adjacency_list import AdjacencyList
from codex.graphs.mst import prim

g = AdjacencyList[str](directed=False)
for u, v, w in [("A", "B", 5), ("A", "C", 3), ("B", "C", 2), ("B", "D", 6),
                ("C", "D", 1), ("C", "E", 4), ("D", "E", 2), ("D", "F", 3),
                ("E", "F", 5)]:
    g.add_edge(u, v, w)

key, parent = prim("A", g.weighted_neighbors)

total = sum(key[v] for v in key if parent[v] is not None)
print(f"MST total weight: {total}")
print("edges:")
for v in sorted(key):
    if parent[v] is not None:
        print(f"  {parent[v]}-{v}  weight {key[v]:.0f}")
```

```
MST total weight: 11
edges:
  C-B  weight 2
  A-C  weight 3
  C-D  weight 1
  D-E  weight 2
  D-F  weight 3
```

Total weight 11, same MST edges as Kruskal. The specific edges: `A-C(3)`, `C-B(2)`, `C-D(1)`, `D-E(2)`, `D-F(3)`. The edges are identical to Kruskal's output; the order in which they were discovered differs (Prim adds them in the order vertices join the tree starting from `A`, while Kruskal adds them in global weight order), but the MST is the same. The section below shows exactly what makes the two orderings equivalent.

36.4 Two algorithms, one theorem

The structural parallel between Kruskal and Prim is easier to see once you write down what cut each step uses.

Kruskal, step by step. After processing the first k edges, there are some number of components. When the algorithm considers edge (u, v, w), the cut is $(\text{component}(u), V \setminus \text{component}(u))$. Among all edges crossing that cut that haven't already been added, (u, v, w) is the one with minimum weight — because the edges are processed in sorted order and any earlier edge that crossed the same cut would have already been added (it would have connected two different components). The cut property says (u, v, w) must be in some MST.

Prim, step by step. After adding k vertices to the tree, the current set is S. The algorithm pops the vertex v with the smallest `key[v]`, which records the weight of the edge $(\mathit{parent}[v], v)$ connecting v to the current tree. The cut is $(S, V \setminus S)$. Among all edges crossing the cut, $(\mathit{parent}[v], v, \text{key}[v])$ is the minimum-weight one — because the heap gives the minimum across all candidates, and `key[v]` records the best edge from v to the tree. The cut property says this edge must be in some MST.

In both cases the greedy choice is "pick the minimum-weight edge crossing some cut." The difference is which cut: Kruskal uses the cut defined by a single component, applied globally to all components; Prim uses the cut defined by the entire current tree, applied locally to one candidate vertex. They discover the same MST edges in a different order.

This also explains when the algorithms have different performance profiles. Kruskal needs to sort all m edges up front: $O(m \log m)$ before the first edge is added. On a dense graph ($m = \Theta(n^2)$) this dominates everything else

and gives $\Theta(n^2 \log n)$. On a sparse graph ($m = O(n)$) the sort is $O(n \log n)$ and the union-find operations barely register, making Kruskal fast. Prim starts immediately from one vertex and pays $O(\log n)$ per edge relaxation via the heap. On a dense graph where most edges are relaxed, $m = \Theta(n^2)$ heap operations give $\Theta(n^2 \log n)$ — same as Kruskal asymptotically, but with a worse constant because the heap operations dominate. In practice, on sparse graphs Kruskal is competitive because its inner loop is simpler; on dense graphs the right choice is the $O(n^2)$ adjacency-matrix Prim (linear scan instead of heap, same greedy logic), not either of the algorithms in this chapter.

36.5 The three questions, applied

Is it correct?

Both algorithms are correct by the cut property. For Kruskal: when the algorithm adds edge (u, v, w), no earlier edge connected `component(u)` to `component(v)`, so (u, v, w) is the minimum-weight edge crossing the cut $(\text{component}(u), V \setminus \text{component}(u))$ among edges not yet committed to cycles. By the cut property, it's in some MST. Repeating this argument for each added edge shows that the entire set of added edges belongs to some common MST.

For Prim: when the algorithm adds vertex v with parent p and key weight w, (p, v, w) is the minimum-weight edge from the current tree S to $V \setminus S$. By the cut property, this edge belongs to every MST. Repeating for each added vertex gives the full MST.

The cut property's proof assumed distinct edge weights. In practice, graphs frequently have ties. The cut property generalizes to non-distinct weights if you replace "every MST" with "some MST": for any cut, a minimum-weight crossing edge is in *some* MST, though not necessarily all of them. Both algorithms still return a valid MST on graphs with ties; they may return different ones depending on tie-breaking order.

How efficient is it?

Kruskal runs in $O(m \log m)$ time, dominated by the initial sort. The union-find operations cost $O(m\alpha(n))$ amortized total, where α is the inverse Ackermann function — for all practical purposes, $O(m)$. The sort is the bottleneck. Space is $O(n + m)$ for the union-find structure and the sorted edge list.

Prim runs in $\Theta((n + m) \log n)$ time with a binary heap. Each vertex is popped from the heap at most once ($\Theta(n \log n)$ total), and each edge is relaxed at most twice (once per endpoint, $\Theta(m \log n)$ total). Space is $O(n+m)$ for the heap and the `key`/`parent` dictionaries. With a Fibonacci heap the bound improves to $\Theta(n \log n + m)$, which beats Kruskal on dense graphs, but the constant factors of Fibonacci heaps make the practical crossover point larger than most real graphs.

For the specific comparison: on sparse graphs ($m = O(n \log n)$), both algorithms are $O(n \log n)$. On dense graphs ($m = \Theta(n^2)$), both are $O(n^2 \log n)$ — but a simple $O(n^2)$ adjacency-matrix Prim with linear scan replaces the heap beats both. In practice, Kruskal is the first choice for sparse graphs (simpler implementation, one sort, union-find nearly free); adjacency-matrix Prim is the first choice for dense graphs.

Is it optimal?

For MST computation in the comparison model, the lower bound is $\Omega(m \log n)$, which Kruskal matches on sparse graphs and Prim matches in general. The bound comes from the fact that MST computation must determine for each edge whether it belongs to the MST, which requires at least $\Omega(m)$ comparisons, and the tree structure requires distinguishing $\Omega(n)$ possibilities, costing $\Omega(\log n)$ per vertex.

The optimal MST algorithm remains an open problem: Borůvka's 1926 algorithm achieves $O(m \log n)$; Fredman and Tarjan's 1987 Fibonacci-heap Prim achieves $O(m+n \log n)$; the current best general algorithm is a randomized algorithm by Karger, Klein, and Tarjan (1995) that runs in expected $O(m)$ time. The $O(m)$ bound is also achievable deterministically for integer weights. Whether a deterministic $O(m)$ algorithm exists for general weights is open, making this one of the few classical algorithm problems whose optimal complexity remains unresolved.

36.6 What this chapter teaches

The cut property unifies the greedy argument. Two algorithms that look structurally different — one sorts all edges up front, the other grows a tree vertex by vertex — are both instances of the same theorem. Every greedy spanning-tree algorithm that can be described as "repeatedly pick the minimum-weight edge crossing some cut" is correct. The cut property is the precise statement of why greedy works here when it fails for, say, the shortest-path problem on graphs with negative cycles: the cut structure of the graph constrains which edges can be in any MST, and the greedy local choice never conflicts with the global constraint.

Union-find earns its keep. The union-find structure from chapter 14 was introduced for disjoint-set operations in abstract terms. Here it does a specific job: cycle detection in Kruskal's inner loop. Without it, testing whether edge (u, v) forms a cycle would require a DFS or BFS from u to v in the current MST forest, costing $O(n)$ per edge and making the total algorithm $O(nm)$. Path-compressed union-find reduces that to $O(\alpha(n))$ per operation — essentially constant — and the sort dominates. The chapter shows the payoff of a data structure studied in isolation: it enters a new algorithmic context and the cost shape of a well-known algorithm changes by a factor of n.

The Dijkstra—Prim parallel is exact. Dijkstra maintains `dist[v]` as the distance from source to `v` via the cheapest known path; Prim maintains `key[v]` as the weight of the cheapest known edge from the current tree to `v`. Both use a min-heap keyed on that value, both use lazy deletion, both finalize each vertex once. The one-line difference between the two implementations — `new_d = d + w` versus `w` directly — is the difference between "shortest path" and "cheapest tree edge." Recognizing this parallel makes the next algorithm in the chapter easier to read: if you understand Dijkstra, Prim is one substitution.

36.7 Notes and further reading

Otakar Borůvka described the minimum spanning tree problem and the first algorithm for solving it in 1926, motivated by an electrical network design problem in Moravia. Borůvka's algorithm processes components in parallel — each component simultaneously selects its minimum-weight outgoing edge, then all selected edges are added and components merged — and runs in $O(m \log n)$ time. It predates Kruskal's and Prim's algorithms by three decades and is still used in practice for parallel MST computation.

Joseph Kruskal published his algorithm in 1956 ("On the Shortest Spanning Subtree of a Graph and the Traveling Salesman Problem," *Proceedings of the American Mathematical Society* 7(1):48–50). Robert Prim published the vertex-growing algorithm in 1957 ("Shortest Connection Networks and Some Generalizations," *Bell System Technical Journal* 36(6):1389–1401), though Vojtech Jarník had described the same algorithm independently in 1930. The heap-based variant of Prim's algorithm, as presented in this chapter, is due to Dijkstra (1959) and Fredman-Tarjan (1987).

The cut property as a unified framework for MST algorithms appears in CLRS chapter 23, where it is called the "generic MST algorithm" — a meta-algorithm that adds safe edges (minimum-weight edges crossing some cut of the current forest) one at a time. Kruskal and Prim are two implementations of this meta-algorithm. Sedgewick and Wayne's *Algorithms* (4th ed.) §4.3 presents both alongside the cut property and makes the connection explicit.

For a broader treatment of greedy algorithms and matroids — the algebraic structure that explains exactly when greedy algorithms work for optimization problems — Lawler's *Combinatorial Optimization: Networks and Matroids* (1976) is the canonical reference. The graphic matroid of a graph is the structure that makes Kruskal's greedy choice correct: the set of edges forms a matroid, and the greedy algorithm applied to a matroid always produces an optimal independent set. The matroid framework extends to weighted interval scheduling, transversal matching, and other problems where greedy algorithms are known to work.

Regarding the open problem on MST optimality: Karger, Klein, and Tarjan's 1995 randomized linear-time algorithm appears in "A Randomized Linear-Time Algorithm to Find Minimum Spanning Trees" (*JACM* 42(2):321–328). The current best deterministic bound, due to Chazelle (2000), is $O(m\alpha(n))$ — nearly linear — but a deterministic $O(m)$ MST algorithm is open.

In the next chapter I'll turn to structural questions about directed graphs. Topological sorting asks for an ordering of vertices such that every edge goes from earlier to later — possible exactly when the graph has no cycle, which DFS from chapter 33 can check in $\Theta(n + m)$. Strongly connected components ask for the maximal subsets of vertices that can all reach each other — again the answer falls out of DFS, but the argument is more subtle. Both algorithms are direct applications of the DFS vocabulary built in chapter 33.

37 Ordering vertices and finding strong components

Two structural questions about directed graphs fall out of DFS with almost no additional machinery. The first: can the vertices of a directed graph be lined up so that every edge points from earlier to later in the line? The answer is yes exactly when the graph is acyclic, and DFS produces the ordering as a side effect — the vertices in decreasing order of their finish times from chapter 33 form a valid topological order. The second: which maximal subsets of vertices can all reach each other? These are the *strongly connected components*, and Tarjan's 1972 algorithm finds all of them in one DFS pass by maintaining a single integer per vertex — the *low-link value* — that tracks the earliest-discovered ancestor still reachable from each vertex's subtree.

By the end of this chapter you will have implemented topological sort two ways (DFS finish-time order and Kahn's in-degree BFS), confirmed that both algorithms detect cycles and produce the same valid ordering on DAGs, implemented Tarjan's strongly connected components algorithm on a directed graph with three distinct components, and understood why the SCC condensation of any directed graph is always a DAG — a result that Part VI will use directly when it runs dynamic programming on directed graphs. Both topological sort algorithms and Tarjan's SCC run in $\Theta(n + m)$ time. ## A directed graph either admits a linear order or contains a cycle

A *topological ordering* of a directed graph is a linear ordering of all its vertices such that for every directed edge (u, v), u appears before v in the ordering. Scheduling problems, build systems, package dependency resolution, and circuit evaluation all reduce to finding a topological order: each vertex is a task, each edge is a dependency, and the topological order is a valid execution sequence.

The key constraint is that topological orderings exist *if and only if* the graph is a DAG — a directed acyclic graph. If there's a directed cycle $v_1 \to v_2 \to \cdots \to v_k \to v_1$, then a topological order would need v_1 before v_2 before $\cdots$ before v_k before v_1, which is impossible. Conversely, a DFS on a directed graph produces a back edge (an edge to a gray ancestor) if and only if the graph has a cycle. So detecting a topological order and detecting the absence of cycles are the same computation. DFS does both: the cycle check produces the topological ordering as a side effect.

37.1 DFS writes the order in its finish times

The DFS-based topological sort is the `has_cycle` algorithm from chapter 33 with one extra line: append each vertex to a list the moment it turns black (finishes). That list, reversed, is a topological order.

The reason: if (u, v) is a directed edge and DFS visits u before v, then the recursion visits v's entire subtree before returning to finish u, so v finishes before u. If DFS visits v before u, then since there's no path from v to u (the graph is a DAG — there's no back edge from v's subtree to u), u is discovered only after v's entire subtree finishes, so again v finishes before u. In both cases, u finishes after v. In the reversed list, u therefore appears before v. Every edge points from earlier to later in the reversed finish-time order.

```
from collections.abc import Hashable, Iterable
from collections import deque
from codex.graphs.implicit import NeighborFn


def topological_sort_dfs[V: Hashable](
    vertices: Iterable[V], neighbors: NeighborFn[V]
) -> list[V]:
    """Topological sort via DFS finish times. Returns vertices in topological
    order (sources first, sinks last). Raises ValueError if the graph
    contains a directed cycle.
    """
```

```
    WHITE, GRAY, BLACK = 0, 1, 2
    color: dict[V, int] = {}
    finish_order: list[V] = []

    def visit(v: V) -> None:
        color[v] = GRAY
        for w in neighbors(v):
            state = color.get(w, WHITE)
            if state == GRAY:
                raise ValueError("graph contains a cycle - not a DAG")
            if state == WHITE:
                visit(w)
        color[v] = BLACK
        finish_order.append(v)

    for v in vertices:
        if color.get(v, WHITE) == WHITE:
            visit(v)

    return list(reversed(finish_order))
```

The structure is `has_cycle` from chapter 33 with `finish_order.append(v)` added at the moment each vertex turns black. One new line. The return value reverses the list because vertices that finish last (sources, with no incoming edges) should appear first in the topological order. The three-color machine and the gray-vertex cycle check are unchanged.

Run it on a small DAG:

```
from codex.graphs.adjacency_list import AdjacencyList
from codex.graphs.topo_scc import topological_sort_dfs

dag = AdjacencyList[str](directed=True)
for u, v in [("A", "B"), ("A", "C"), ("B", "D"), ("C", "D"), ("D", "E")]:
    dag.add_edge(u, v)

order = topological_sort_dfs(dag.vertices(), dag.neighbors)
print("topological order:", " → ".join(order))

# verify: every edge should point forward
edges = list(dag.edges())
pos = {v: i for i, v in enumerate(order)}
valid = all(pos[u] < pos[v] for u, v, _ in edges)
print(f"all edges point forward? {valid}")
```

```
topological order: A → C → B → D → E
all edges point forward? True
```

`A → B → C → D → E` or `A → C → B → D → E` depending on iteration order, but always with A first and E last. The verification loop checks that every edge satisfies the topological constraint. Kahn's finds the same order from the opposite direction: not finish times, but in-degree counts.

37.2 Peel the sources one level at a time

Kahn's algorithm approaches topological sort from the opposite direction. Instead of running DFS and reading the result in reverse, it processes the graph front-to-back: find all vertices with no incoming edges (sources), add them to the output, remove them and all their outgoing edges, and repeat. The new sources after each removal are the vertices whose last incoming edge was just removed. A FIFO queue tracks the sources waiting to be processed.

The algorithm is cleaner to explain than to implement, because counting incoming edges requires either a precomputed in-degree table or iterating all edges. The implementation materializes both the in-degree dictionary and an adjacency list from the `neighbors` function, then runs the BFS:

```
def topological_sort_kahn[V: Hashable](
    vertices: Iterable[V], neighbors: NeighborFn[V]
) -> list[V]:
    """Topological sort via Kahn's algorithm (BFS from in-degree-0 vertices).
    Returns vertices in topological order (sources first). Raises ValueError
    if the graph contains a directed cycle.
    """
    vlist = list(vertices)
    in_degree: dict[V, int] = {v: 0 for v in vlist}
    adj: dict[V, list[V]] = {v: [] for v in vlist}
    for v in vlist:
        for w in neighbors(v):
            in_degree[w] = in_degree.get(w, 0) + 1
            adj[v].append(w)
    queue: deque[V] = deque(v for v in vlist if in_degree[v] == 0)
    order: list[V] = []
    while queue:
        v = queue.popleft()
        order.append(v)
        for w in adj[v]:
            in_degree[w] -= 1
            if in_degree[w] == 0:
                queue.append(w)
    if len(order) != len(vlist):
        raise ValueError("graph contains a cycle - not a DAG")
    return order
```

The cycle detection is free: if the graph has a cycle, the vertices on the cycle never reach in-degree zero, so they're never added to the queue. At the end, `len(order) < len(vlist)` reveals the problem. Kahn's approach gives a different correctness argument than the DFS approach — instead of proving the finish-time order is topological, you prove that at each step the sources are exactly the vertices with no remaining dependencies — but both algorithms produce a valid topological order on any DAG.

```
from codex.graphs.adjacency_list import AdjacencyList
from codex.graphs.topo_scc import topological_sort_kahn

dag = AdjacencyList[str](directed=True)
for u, v in [("A", "B"), ("A", "C"), ("B", "D"), ("C", "D"), ("D", "E")]:
    dag.add_edge(u, v)

order_kahn = topological_sort_kahn(dag.vertices(), dag.neighbors)
print("Kahn order:", " → ".join(order_kahn))

# cycle detection
cyclic = AdjacencyList[str](directed=True)
for u, v in [("A", "B"), ("B", "C"), ("C", "A")]:
    cyclic.add_edge(u, v)

try:
    topological_sort_kahn(cyclic.vertices(), cyclic.neighbors)
    print("no cycle detected")
except ValueError as e:
    print(f"caught: {e}")
```

```
Kahn order: A → B → C → D → E
caught: graph contains a cycle - not a DAG
```

Kahn's output for the DAG matches DFS's ordering (same valid topological sequence). For the cyclic graph, all three vertices remain at non-zero in-degree and the output list is shorter than the vertex count. Cycle detected.

Topological sort assumes the graph is acyclic. When vertices form mutual-reachability clusters, the right question shifts: which clusters are they?

37.3 Vertices that can all reach each other

A *strongly connected component* of a directed graph is a maximal set of vertices S such that for any two vertices $u, v \in S$, there is a directed path from u to v and from v to u. Every directed graph has a unique partition into SCCs, even if some SCCs contain just one vertex. In the graph below — vertices A, B, C, D, E, F with edges $A \to B \to C \to A$ (a 3-cycle), $C \to D \leftarrow E \to D$ (a 2-cycle), and $C \to F$ (isolated vertex) — the SCCs are $\{A, B, C\}$, $\{D, E\}$, and $\{F\}$.

SCC decomposition matters for two reasons. First, the *condensation* of a directed graph — the DAG formed by collapsing each SCC to a single node and keeping the edges between SCCs — is always a DAG. (If there were a cycle in the condensation, the corresponding SCCs would themselves form a larger SCC, contradicting the maximality of the partition.) That DAG is the input to the graph-DP algorithms in Part VI. Second, many "find if you can get from here to there" questions reduce to SCC: two vertices are in the same SCC if and only if they can reach each other; vertices in different SCCs can be reached only in topological order of the condensation. Finding those components efficiently is the question — and DFS already has everything it needs.

37.4 Tarjan's low-link DFS finds all SCCs in one pass

Here is the move at the heart of Tarjan's algorithm. Run DFS and maintain, for each vertex v, a value `low[v]` defined as the minimum discovery time among all vertices reachable from v's DFS subtree via *at most one back edge*. Formally:

$$\text{low}[v] = \min\Big(\text{disc}[v], \min_{(v,w) \text{ is a back edge}} \text{disc}[w], \min_{(v,w) \text{ is a tree edge}} \text{low}[w]\Big)$$

Two vertices u and v are in the same SCC if and only if they are on the same DFS path and there is a back edge from v's subtree back to u or an ancestor of u. The low-link value is what makes this checkable in $O(1)$: vertex v is the *root* of its SCC — the first vertex of the SCC to be discovered — exactly when $\text{low}[v] = \text{disc}[v]$. No back edge from v's subtree reaches further back than v itself, which means v's subtree (minus vertices already assigned to earlier SCCs) forms a maximal reachable cluster.

The algorithm maintains a separate stack of unassigned vertices. When v is identified as an SCC root, pop the stack until v is popped, and those vertices are v's SCC.

```
def tarjan_scc[V: Hashable](
    vertices: Iterable[V], neighbors: NeighborFn[V]
) -> list[list[V]]:
    """Tarjan's strongly connected components. Returns a list of SCCs, each
    as a list of vertices. The list is in reverse topological order of the
    condensation DAG (sinks first, sources last). O(n + m) time.
    """
    disc: dict[V, int] = {}
    low: dict[V, int] = {}
    on_stack: set[V] = set()
    stack: list[V] = []
    sccs: list[list[V]] = []
    timer = [0]

    def visit(v: V) -> None:
        timer[0] += 1
        disc[v] = low[v] = timer[0]
        stack.append(v)
        on_stack.add(v)
```

```
        for w in neighbors(v):
            if w not in disc:
                visit(w)
                low[v] = min(low[v], low[w])
            elif w in on_stack:
                low[v] = min(low[v], disc[w])
        if low[v] == disc[v]:
            scc: list[V] = []
            while True:
                w = stack.pop()
                on_stack.discard(w)
                scc.append(w)
                if w == v:
                    break
            sccs.append(scc)

    for v in vertices:
        if v not in disc:
            visit(v)
    return sccs
```

Thirty lines. The `visit` function is the recursive DFS from chapter 33's `has_cycle`, with three additions: the low-link bookkeeping (`disc[v] = low[v] = timer[0]`, the two `min` updates), the stack management (`stack.append`, `on_stack.add`), and the SCC extraction at the end of `visit` when the SCC root is detected. The `on_stack` set gives the $O(1)$ check that makes each edge processing $O(1)$ despite the set operations. The two branches at the edge-processing step are: if the neighbor hasn't been discovered yet, it's a tree edge — recurse and update `low[v]` from the child's low; if the neighbor is on the stack, it's a back edge — update `low[v]` from the neighbor's discovery time.

```
from codex.graphs.adjacency_list import AdjacencyList
from codex.graphs.topo_scc import tarjan_scc

g = AdjacencyList[str](directed=True)
for u, v in [("A", "B"), ("B", "C"), ("C", "A"),   # 3-cycle
             ("C", "D"), ("D", "E"), ("E", "D"),     # 2-cycle
             ("C", "F")]:                            # isolated vertex
    g.add_edge(u, v)

sccs = tarjan_scc(g.vertices(), g.neighbors)
print(f"number of SCCs: {len(sccs)}")
for i, scc in enumerate(sccs):
    print(f"  SCC {i + 1}: {sorted(scc)}")
```

```
number of SCCs: 3
  SCC 1: ['D', 'E']
  SCC 2: ['F']
  SCC 3: ['A', 'B', 'C']
```

Three SCCs: `{A, B, C}`, `{D, E}`, `{F}`. The order in which they appear in the output is reverse topological order of the condensation: sinks are found first (the algorithm finishes exploring sinks before finishing the source SCCs), sources last.

37.5 The condensation is always a DAG

Watch what happens when the SCC list is reversed: the SCCs appear in topological order of the condensation. `{A, B, C}` has outgoing edges to both `{D, E}` and `{F}` in the condensation, so it's a source; `{D, E}` and `{F}` are sinks. The condensation's topological order is `{A, B, C}` first, then `{D, E}` and `{F}` in either order. Reverse the SCC list from Tarjan's output and you have it.

This connection is the bridge to Part VI. Many dynamic-programming algorithms on directed graphs work by running DP in topological order on the condensation: compute the answer for each SCC, then propagate answers downstream. For example, the longest path in a DAG is computed by processing vertices in topological order and taking the best incoming path at each step. The same algorithm on a general directed graph reduces to: find SCCs with Tarjan's, collapse each SCC to a single vertex (keeping its best-case answer), then run DAG-DP on the condensation.

For this specific graph, the condensation has three vertices: $A' = \{A, B, C\}$, $D' = \{D, E\}$, $F' = \{F\}$, with edges $A' \to D'$ and $A' \to F'$. Any DP on this graph — "what's the maximum-weight path from source to sink," "which sinks are reachable from a given source," "what's the minimum cost to visit at least one vertex in each SCC" — can now run on the three-vertex DAG rather than on the original six-vertex graph with cycles.

37.6 The three questions, applied

Is it correct?

For DFS-based topological sort, the argument is the one sketched above: for any edge (u, v), v finishes before u (because v's subtree is either explored inside u's recursive call, or v is explored and finished before u is even discovered). In both cases u appears before v in the reversed finish-time list. The gray-vertex cycle check is the same as `has_cycle` from chapter 33, which is correct for the same reason: a gray vertex is one whose recursive call is on the stack, and a gray-to-gray edge is a back edge, which witnesses a cycle.

For Kahn's algorithm, correctness follows from the invariant that every vertex added to the output has all of its predecessors already in the output. When v is dequeued, its in-degree is zero — all its predecessors have already been dequeued and removed. If the graph has no cycle, every vertex eventually reaches in-degree zero and is added. If there's a cycle, the vertices on the cycle keep each other at non-zero in-degree permanently.

For Tarjan's SCC, the correctness argument has two parts. First, the SCC root identification: vertex v with $\text{low}[v] = \text{disc}[v]$ is the first vertex of its SCC to be discovered, and the back edges from its subtree don't reach further back than v. This means no vertex above v on the DFS stack is in v's SCC (they were discovered earlier and any path from v can't complete a cycle back to them). Second, the stack pop is safe: all vertices between v and the top of the stack were discovered after v and can reach v via a path through the DFS tree, so they're in the same SCC.

How efficient is it?

All three algorithms run in $\Theta(n + m)$ time on an adjacency-list graph. DFS-based topological sort is DFS with a constant-time append per vertex — same cost as DFS. Kahn's algorithm computes in-degrees in $\Theta(n + m)$ (iterate all edges), initializes the queue in $\Theta(n)$, and processes each vertex and each edge exactly once in the BFS loop — $\Theta(n + m)$ total. Tarjan's SCC is DFS with constant-time stack operations per vertex and per edge — $O(1)$ amortized for the `on_stack` set (hash set), $O(1)$ for the stack push/pop, $O(1)$ for the low-link min. Every vertex and edge is visited exactly once, giving $\Theta(n + m)$.

Space is $\Theta(n)$ for all three: the `color`/`disc`/`low` dictionaries, the stacks and queues, and the output list are all at most $O(n)$ simultaneously. The `adj` dictionary in Kahn's adds $O(m)$ space but that's dominated by the input graph itself.

Is it optimal?

For topological sorting, $\Theta(n + m)$ is optimal: any algorithm must examine every vertex and every edge to be sure no ordering constraint is missed. Both DFS-based sort and Kahn's algorithm match this lower bound.

For strongly connected components, $\Theta(n + m)$ is similarly optimal — any algorithm must examine every edge at least once to determine whether it's a cross-component or intra-component edge. Tarjan's algorithm is optimal in this sense. The alternative SCC algorithm (Kosaraju's, which runs two DFS passes and requires the reversed graph) has the same asymptotic cost; Tarjan's advantage is that it requires only one DFS pass and no explicit graph reversal.

37.7 What this chapter teaches

DFS finish times encode a topological order. The observation that reversing DFS finish times gives a topological sort is the deepest payoff of the finish-time vocabulary from chapter 33. A finish time is a certificate that everything reachable from a vertex has been fully explored. Reversing by finish time guarantees that sources (no incoming edges) finish last and appear first in the output. The same vocabulary supports Tarjan's SCC in this chapter and cycle-detection in both algorithms.

Kahn's algorithm makes cycle detection visible without DFS. The in-degree BFS approach produces the same topological order as the DFS approach but expresses the algorithm as a queue drain rather than a recursion. This matters for practical implementation: Kahn's has no recursion-depth concern, is easier to parallelize (vertices at the same in-degree level can be processed concurrently), and makes the cycle-detection output explicit (the unprocessed vertices are exactly those on cycles). For build systems and dependency resolvers, Kahn's is typically the right choice; for algorithms that also need DFS timestamps for other purposes (like Tarjan's SCC), the DFS approach comes for free.

Tarjan's SCC is DFS with one bookkeeping integer per vertex. The low-link value adds five lines to the DFS from chapter 33 and computes the entire SCC decomposition in a single pass. Every structural graph question reduces to DFS plus one or two integers per vertex, recorded at discovery and finish. Cycle detection used a color (three-valued integer). Topological sort added a finish list. SCC adds a low-link value and a stack. None of these are separate algorithms in the sense of requiring different traversal strategies; they are all DFS, varying only in what they measure as they go.

37.8 Notes and further reading

Topological sort via DFS finish times is essentially Tarjan's 1972 result, extracted from the SCC paper. The observation that reversed finish times form a topological order is stated explicitly in CLRS chapter 22.4. Kahn's algorithm appeared in Kahn's 1962 paper "Topological Sorting of Large Networks" (*CACM* 5(11):558–562) and predates the DFS-based approach; Kahn was solving a build-system-like problem for a large software project.

Robert Tarjan published the SCC algorithm in "Depth-First Search and Linear Graph Algorithms" (*SIAM Journal on Computing* 1(2):146–160, 1972). The paper introduced both the timestamping infrastructure (discovery times, finish times, the parenthesis theorem) and the SCC algorithm as its main application — Tarjan's primary goal was to show that DFS could answer structural questions in linear time. The low-link value as defined in this chapter is a slight simplification of Tarjan's original formulation; the behavior is equivalent for the SCC problem.

Kosaraju's SCC algorithm, which runs two DFS passes — one on the original graph to record finish times, one on the reversed graph in decreasing finish-time order to collect SCCs — was independently discovered by Kosaraju in 1978 and Sharir in 1981. CLRS chapter 22.5 presents Kosaraju's algorithm alongside Tarjan's. Both are $\Theta(n + m)$; Tarjan's requires one pass and no graph reversal, while Kosaraju's is easier to prove correct.

For the applications of SCC: 2-SAT (determining whether a 2-CNF Boolean formula is satisfiable) reduces to SCC on the implication graph in $O(n + m)$ time. Program optimization (identifying strongly connected regions of a control-flow graph, which correspond to loops) uses SCC as a first step. Social network analysis uses SCC to identify "echo chambers" — maximal subsets of users that can all influence each other through the network. Sedgewick and Wayne's *Algorithms* (4th ed.) §4.2 presents the SCC algorithms with detailed application examples.

For Tarjan's broader influence: the same 1972 paper that introduced DFS timestamping and SCC also described linear-time algorithms for biconnected components (vertex-cut sets) and bridges (edge-cut sets). Hopcroft and Tarjan's follow-up work on planarity testing (1974) and dominator trees (1979) established DFS as the canonical tool for structural graph analysis. Chapter 33's observation that "every structural question reduces to DFS plus one extra integer per vertex" is a reasonable summary of the body of work Tarjan's 1972 paper inaugurated.

In the next chapter I'll add a concept to graphs that neither shortest paths nor spanning trees needed: *capacity*. Flow networks assign to each directed edge both a direction and an upper bound on how much "flow" can travel through it. The maximum-flow problem asks for the most flow that can get from a source vertex to a sink vertex without violating any capacity. The answer turns out to equal the weight of the minimum-weight cut separating the source from the sink — max-flow equals min-cut — and the proof is constructive: the augmenting-path algorithm simultaneously computes both sides of the duality.

38 Flow networks and the max-flow min-cut theorem

Two quantities measure how well a directed graph can route material from a source to a sink: the maximum amount that can flow simultaneously, and the minimum capacity of any set of edges whose removal separates source from sink entirely. These two quantities are always equal. Proving that equality requires constructing the flow: the augmenting-path procedure that finds the maximum flow terminates precisely when it has located the minimum cut. The algorithm and the theorem arrive together.

By the end of this chapter you will have implemented Edmonds-Karp, the BFS variant of the augmenting-path procedure that runs in $O(nm^2)$ and is the standard algorithm for the problem. You will understand the residual graph, which makes the algorithm correct; the max-flow min-cut theorem, which gives the algorithm its termination condition; and why BFS augmentation — rather than any-path augmentation — is essential for the polynomial time guarantee. Flow networks are where graph algorithms meet linear programming duality: the max-flow min-cut theorem is LP duality made constructive.

38.1 Sending flow through a capacity network

A flow network is a directed graph with two distinguished vertices — a source s and a sink t — and a *capacity* $c(u, v) \geq 0$ on each directed edge. A valid *flow* assigns a non-negative value $f(u, v)$ to each edge satisfying two constraints:

- **Capacity constraint:** $0 \leq f(u, v) \leq c(u, v)$ for every edge.
- **Flow conservation:** for every vertex v other than s and t, the total flow into v equals the total flow out of v.

The *value* of the flow is the net flow out of the source: $\sum_v f(s, v) - \sum_v f(v, s)$. The goal is to find a flow of maximum value.

The flow value is bounded by any *cut*: a partition of the vertices into two sets S and $T = V \setminus S$ with $s \in S$ and $t \in T$. The capacity of the cut is $\sum_{u \in S, v \in T} c(u, v)$. All flow from source to sink must cross every cut, so no flow can exceed any cut's capacity. The surprising half of the theorem — the achievability half — says the maximum is always attained: there exists a flow whose value equals the minimum cut capacity.

38.2 The residual graph turns flow into a reachability problem

Given a current flow, the *residual graph* captures how much more flow the network can handle and in which directions. For each edge (u, v) with capacity c and current flow f:

- A *forward* arc (u, v) with residual capacity $c - f$: can push up to $c - f$ more units forward.
- A *backward* arc (v, u) with residual capacity f: can cancel up to f units of existing flow by routing backward.

An *augmenting path* is any source-to-sink path in the residual graph where every arc has positive residual capacity. The *bottleneck* of the path is the minimum residual capacity along it. Pushing that many units along the path — increasing forward arcs, decreasing backward arcs — increases the total flow value by exactly the bottleneck amount while preserving both constraints.

The Ford-Fulkerson method repeats: find any augmenting path, push the bottleneck along it, update the residual graph. Terminate when no augmenting path exists. At termination, every vertex reachable from the source in the residual graph forms the S side of a cut, and that cut's capacity equals the current flow value. The flow is maximum; the cut is minimum. The Python implementation maintains this residual table directly.

38.3 Pushing flow along augmenting paths

A single residual-capacity table fuses forward and backward arcs. Augmenting along a path updates the table in-place: forward arcs lose capacity, backward arcs gain it.

```
from collections.abc import Hashable, Iterable
from collections import deque

type FlowEdge[V] = tuple[V, V, float]


def edmonds_karp[V: Hashable](
    source: V,
    sink: V,
    vertices: Iterable[V],
    capacities: Iterable[FlowEdge[V]],
) -> tuple[float, dict[tuple[V, V], float]]:
    """Edmonds-Karp maximum flow (BFS augmenting paths). O(nm^2) time.
    Returns (max_flow_value, flow) where flow[(u, v)] is the flow on each
    original edge. Assumes no antiparallel edges in capacities.
    """
    vlist = list(vertices)
    cap: dict[V, dict[V, float]] = {v: {} for v in vlist}
    for u, v, c in capacities:
        cap.setdefault(u, {})
        cap.setdefault(v, {})
        cap[u][v] = cap[u].get(v, 0.0) + c   # merge parallel edges
        cap[v].setdefault(u, 0.0)              # ensure reverse arc exists

    res: dict[V, dict[V, float]] = {u: dict(nb) for u, nb in cap.items()}
```

The initial residual graph is a copy of the capacity table with zero-capacity reverse arcs added for every original edge. Reverse arcs start at zero: no flow has been pushed yet, so no cancellation is possible. Now the main loop.

```
    max_flow = 0.0
    while True:
        parent: dict[V, V | None] = {source: None}
        queue: deque[V] = deque([source])
        while queue and sink not in parent:
            u = queue.popleft()
            for v, r in res[u].items():
                if v not in parent and r > 0:
                    parent[v] = u
                    queue.append(v)
        if sink not in parent:
            break

        path_flow: float = float("inf")
        v = sink
        while parent[v] is not None:
            u = parent[v]  # type: ignore
            path_flow = min(path_flow, res[u][v])
            v = u

        v = sink
        while parent[v] is not None:
            u = parent[v]  # type: ignore
            res[u][v] -= path_flow
            res[v][u] = res[v].get(u, 0.0) + path_flow
```

```
            v = u
        max_flow += path_flow

    flow: dict[tuple[V, V], float] = {
        (u, v): c - res[u].get(v, 0.0)
        for u, nb in cap.items()
        for v, c in nb.items()
        if c > 0
    }
    return max_flow, flow
```

The BFS loop is the BFS from chapter 32, reading residual capacities instead of adjacency. The path-tracing loop runs twice: once to find the bottleneck, once to apply it. The flow extraction reconstructs per-edge flow from the difference between original capacity and remaining residual: $f(u, v) = c(u, v) - \text{res}(u, v)$.

Run it on a six-vertex example.

```
from codex.graphs.flow import edmonds_karp

edges = [
    ("s", "a", 10), ("s", "b", 10),
    ("a", "c", 4), ("a", "b", 2), ("a", "t", 8),
    ("b", "d", 9),
    ("c", "t", 10),
    ("d", "t", 10),
]
vertices = ["s", "a", "b", "c", "d", "t"]
max_flow, flow = edmonds_karp("s", "t", vertices, edges)
print(f"max flow: {max_flow:.0f}")
print()
cap_map = {(u, v): c for u, v, c in edges}
for (u, v), f in sorted(flow.items()):
    if f > 0:
        print(f"  {u} -> {v}: {f:.0f} / {cap_map[u, v]:.0f}")
```

```
max flow: 19

  a -> c: 2 / 4
  a -> t: 8 / 8
  b -> d: 9 / 9
  c -> t: 2 / 10
  d -> t: 9 / 10
  s -> a: 10 / 10
  s -> b: 9 / 10
```

The maximum flow is 19. The algorithm augments along three BFS-shortest paths: $s \to a \to t$ (8 units), $s \to a \to c \to t$ (2 units), and $s \to b \to d \to t$ (9 units). After termination, the residual graph can reach b from s (one unit of spare capacity on $s \to b$) but cannot reach a, c, d, or t. The reachable set is $\{s, b\}$. The minimum cut $(\{s, b\}, \{a, c, d, t\})$ has capacity $c(s, a) + c(b, d) = 10 + 9 = 19$, confirming that flow value equals cut capacity. That works for this example, but why does BFS guarantee termination in polynomial time?

38.4 Why shortest augmenting paths converge in $O(nm)$ augmentations

Ford-Fulkerson with arbitrary path selection is not a polynomial algorithm. With real-valued capacities it may fail to terminate: a carefully constructed example uses irrational capacities to produce infinitely many augmentations converging to a value less than the true maximum. With integer capacities it terminates but may need $2U$ augmentations — exponential in the input bit-length — where U is the maximum capacity value.

Edmonds-Karp fixes both problems by requiring BFS to select the *shortest* augmenting path (fewest edges) at each step. The key property: with BFS augmentation, the distance from the source to any vertex in the residual graph never decreases across augmentations. This is not obvious — an augmentation adds backward arcs that could create new short paths. The argument is that a backward arc (v, u) created by augmenting along (u, v) can only be used in a future path if it hasn't already been "consumed," and using that backward arc lengthens any path that traverses it compared to the path that created it.

From this monotonicity: each edge (u, v) becomes a *bottleneck* — the minimum-residual arc on the path — at most $O(n)$ times. Every time (u, v) is a bottleneck, its residual drops to zero, so any future path using it must first traverse the backward arc (v, u), which requires the shortest path from s to v to lengthen by at least 2. Since path lengths are bounded by n, each edge becomes a bottleneck at most $O(n)$ times. With m edges and at most $O(n)$ bottleneck events each, the total number of augmentations is $O(nm)$. Each augmentation runs a BFS in $O(m)$ time. Total: $O(nm^2)$.

The $O(nm^2)$ bound depends only on graph size, not on capacity values. That is what makes Edmonds-Karp a polynomial algorithm when Ford-Fulkerson is not.

38.5 Maximum flow equals minimum cut

The reachability argument that terminates the algorithm also proves the theorem. When BFS finds no augmenting path, let S be the set of vertices reachable from the source in the residual graph. By construction, $s \in S$ and $t \notin S$.

Examine the cut $(S, V \setminus S)$. For any original edge (u, v) with $u \in S$ and $v \notin S$: if the residual capacity $c(u, v) - f(u, v)$ were positive, v would be reachable from s in the residual graph, contradicting $v \notin S$. Every forward edge from S to T is therefore saturated: $f(u, v) = c(u, v)$. For any original edge (v, u) with $v \notin S$ and $u \in S$: if $f(v, u) > 0$, the backward arc (u, v) in the residual graph would put v in the reachable set, contradicting $v \notin S$. So $f(v, u) = 0$.

The flow value equals the net flow from S to T:

$$|f| = \sum_{u \in S, v \in T} f(u, v) - \sum_{u \in T, v \in S} f(v, u) = \sum_{u \in S, v \in T} c(u, v) - 0 = \text{cap}(S, T)$$

The flow value equals the capacity of this cut. Since every flow is bounded above by every cut's capacity, and this flow matches this cut's capacity, both are optimal: the flow is maximum, and the cut is minimum.

This argument is the constructive proof of the max-flow min-cut theorem. Every flow is bounded by every cut; a flow that saturates a cut achieves the bound; therefore maximum flow equals minimum cut capacity. The augmenting-path termination condition and the LP-duality result are the same statement.

38.6 The three questions, applied

Is it correct?

Each augmentation preserves feasibility. The bottleneck is at most the residual capacity of every arc on the path, so no arc's residual drops below zero. For every non-source, non-sink vertex on the path, the incoming and outgoing flow changes cancel, preserving conservation. At termination, the reachability argument constructs a min-cut of capacity equal to the current flow value, proving the flow is maximum by the upper-bound argument.

The *integrality theorem* follows directly: if all capacities are integers, every augmentation pushes integer flow (path_flow is the minimum of integer residual values), so the final flow assigns integer values to every edge. Integer flows correspond to combinatorial objects — a flow of value k through a unit-capacity network equals k edge-disjoint paths, which is the connection to matching and connectivity.

How efficient is it?

Edmonds-Karp runs in $O(nm^2)$ time and $O(n+m)$ space. The $O(nm)$ augmentation bound times $O(m)$ per BFS gives the time bound. Space is dominated by the residual table ($O(n+m)$) and the BFS parent table ($O(n)$).

The Dinic algorithm improves on Edmonds-Karp by computing an entire *blocking flow* in the layered graph before advancing to the next BFS distance, reducing the total work to $O(n^2m)$ on general graphs and $O(m\sqrt{n})$ on unit-capacity graphs. For bipartite matching — where unit capacities make the flow value equal the matching size — $O(m\sqrt{n})$ is the Hopcroft-Karp bound and the practical standard. Push-relabel algorithms (Goldberg and Tarjan, 1988) achieve $O(n^2\sqrt{m})$ on general graphs and outperform augmenting-path methods on dense instances.

Is it optimal?

The fastest known max-flow algorithms achieve $\tilde{O}(m)$ time using sophisticated dynamic data structures (Chen et al., 2022, using randomization). For unit-capacity graphs, $O(m\sqrt{n})$ from Hopcroft-Karp is optimal under standard conjectures about triangle detection. For practical purposes, Edmonds-Karp is competitive on sparse graphs with moderate capacity values; Dinic's algorithm is the standard for larger instances.

38.7 What this chapter teaches

The residual graph is the central object. Every step of Edmonds-Karp is a BFS on the residual graph, not on the original. The residual encodes both "room to push forward" and "room to cancel backward" for every edge. The backward arcs are what enables the algorithm to recover from locally suboptimal choices — they are not mere bookkeeping. An augmenting-path algorithm without backward arcs would find a maximal but not necessarily maximum flow.

BFS converts an exponential procedure into a polynomial one. Ford-Fulkerson with arbitrary path selection is not a polynomial algorithm in the input size. Forcing BFS removes the capacity-dependent blowup: the $O(nm)$ augmentation bound depends only on the graph's vertex and edge counts. This is a recurring pattern: when a greedy iteration works in principle but not in polynomial time, choosing the right inner subroutine fixes the complexity without changing the outer structure. Dijkstra (greedy + heap) and Edmonds-Karp (augmenting path + BFS) are both instances.

Max-flow min-cut is LP duality made constructive. The max-flow LP and the min-cut LP are dual programs; strong duality says their optima are equal. The max-flow min-cut theorem is that equality, and the augmenting-path argument is the constructive proof: the algorithm terminates when it simultaneously certifies a flow (primal feasible) and a cut (dual feasible) of the same value. This constructive proof structure — finding a primal solution and a dual certificate together — appears in Kruskal's MST (greedy solution + cut property certificate), Dijkstra's shortest path (tentative distances + relaxation certificate), and Tarjan's SCC (DFS tree + low-link certificate). The max-flow min-cut theorem makes the LP interpretation explicit.

38.8 Notes and further reading

L.R. Ford Jr. and D.R. Fulkerson introduced augmenting paths in "Maximal Flow Through a Network" (*Canadian Journal of Mathematics* 8:399–404, 1956). The max-flow min-cut theorem was proved independently by Ford and Fulkerson and by P. Elias, A. Feinstein, and C.E. Shannon in the same year. Jack Edmonds and Richard Karp showed in 1972 that BFS augmentation gives $O(nm)$ augmentations and $O(nm^2)$ time in "Theoretical Improvements in Algorithmic Efficiency for Network Flow Problems" (*JACM* 19(2):248–264). CLRS chapter 26 covers Ford-Fulkerson, Edmonds-Karp, and push-relabel with complete proofs.

E.A. Dinic's blocking-flow algorithm appeared in "Algorithm for Solution of a Problem of Maximum Flow in a Network" (*Soviet Mathematics Doklady* 11:1277–1280, 1970). John Hopcroft and Richard Karp's $O(m\sqrt{n})$ bipartite matching algorithm (1973, *SIAM Journal on Computing* 2(4):225–231) achieves the same bound for unit-capacity flows. Andrew Goldberg and Robert Tarjan's push-relabel framework appeared in "A New Approach to the Maximum-Flow Problem" (*JACM* 35(4):921–940, 1988).

Bipartite maximum matching reduces to max-flow by adding a supersource connected to every left vertex and a supersink connected from every right vertex, all with unit capacity. A flow of value k in the resulting network corresponds to a matching of size k. The same reduction connects max-flow to assignment problems, network reliability, and project scheduling. Sedgewick and Wayne's *Algorithms* (4th ed.) §6.4 covers the reductions and their flow formulations.

The 2022 near-linear-time result is "Maximum Flow and Minimum-Cost Flow in Almost-Linear Time" by Chen, Kyng, Liu, Peng, Gutenberg, and Sachdeva (*IEEE FOCS 2022*). It achieves $m^{1+o(1)}$ time using interior-point methods and dynamic data structures; the $o(1)$ in the exponent depends on polylogarithmic factors. The algorithm is not practical — the constant factors are enormous — but it settles a 50-year open question about the gap between $O(nm)$ and $O(m)$.

In the next chapter I'll close Part V with a synthesis of the five graph algorithm families — BFS/DFS for reachability, relaxation for shortest paths, cuts for spanning trees, DFS timestamps for structural decomposition, and augmenting paths for flow. Each rests on one invariant, and each invariant is a specialization of the same principle: an algorithm is correct when its termination condition simultaneously proves its own optimality.

Lessons from Part V

Eight chapters about graphs, and the same four moves kept appearing. Every algorithm in the part either explored the graph by tracking what it had already reached, maintained an estimate it tightened until it was tight, searched for a certificate that proved the current answer optimal by exhausting a natural bound, or found an augmenting path through a transformed graph. The chapter titles had different names — BFS, Dijkstra, Kruskal, Tarjan, Edmonds-Karp — but the underlying operations were few. What changed was the substrate: how the graph was represented, what estimates were maintained, which bound was the termination condition.

The patterns that kept showing up

Representation determines the cost of basic operations. The first question in each chapter was not "how does the algorithm work?" but "what graph representation does it need?" Adjacency lists make neighbor enumeration $O(\deg(v))$ per vertex but make "does edge (u, v) exist?" an $O(\deg(u))$ lookup. Adjacency matrices flip both: edge existence is $O(1)$, but iterating over neighbors pays $\Theta(n)$ per vertex even in a sparse graph. Every algorithm in the part chose its representation first, because that choice cascaded into every complexity claim. Dijkstra's $O((n + m) \log n)$ is an adjacency-list bound; on an adjacency matrix it degrades to $O(n^2)$. Bellman-Ford's $O(nm)$ edge-relaxation loop is an edge-list bound. Floyd-Warshall's triple loop is an adjacency-matrix bound. The representation is the algorithm's cost model.

Relaxation is the unifying primitive for distance. Dijkstra, Bellman-Ford, Floyd-Warshall, and Prim all shared the same outer skeleton: maintain an upper-bound estimate, relax edges to tighten it, repeat until no tightening is possible. What differed was the order. Dijkstra relaxed the globally nearest vertex at each step, making the estimate tight in one pass per vertex — but only when edge weights are non-negative. Bellman-Ford relaxed every edge at every step and ran $n - 1$ rounds, tolerating negative weights at the cost of an $O(nm)$ outer loop. Floyd-Warshall relaxed through intermediate vertices in a fixed order, computing all-pairs distances in one $O(n^3)$ cube of iterations. Prim's MST used the same priority-queue loop as Dijkstra but maintained minimum edge weight to the growing tree instead of path distance from the source. These are not four different algorithms. They are four orderings of the same relaxation loop, each ordering justified by a different invariant.

DFS timestamps are structural certificates. DFS discovery and finish times are not integers for the sake of having integers. Each timestamp certified something. A finish time $f(u) > f(v)$ for an unreachable pair (u, v) proves that v was fully explored inside u's subtree — which is exactly the topological-sort argument. A vertex u with the maximum finish time in a weakly connected component is the root of a DFS tree that spans the component's reverse — which is Kosaraju's SCC argument. Tarjan's low-link is a compressed finish time: the minimum finish time reachable from v's subtree via at most one back edge. The same integer, annotated in a different way, answered every structural question the part raised: cycle detection, topological order, cut vertices, bridges, SCCs. DFS is a systematic annotator that encodes the tree's structure in its timestamps.

Termination condition equals optimality certificate. The cleanest pattern in the part is also the least obvious one: every algorithm stopped precisely when it had proven its own answer was optimal. Dijkstra's priority queue reaches a vertex when all shorter paths have been settled — the settled-set invariant is the proof. Kruskal's greedy choice of the minimum-weight crossing edge is correct by the cut property — the cut property is the proof. Tarjan's algorithm identifies an SCC when the low-link of its root equals its own discovery time — the low-link equality is the certificate. Edmonds-Karp terminates when BFS finds no augmenting path — the unreachable set defines a minimum cut of capacity equal to the current flow, which is the proof that no more flow is possible. In each case, the algorithm's stopping rule is not "nothing left to do" but "I have built a certificate." The algorithm and its proof of correctness terminate together.

The residual graph converts a flow problem into a reachability problem. The key move in chapter 38 was the backward arc, not the BFS loop or the capacity table. Every edge (u, v) in the original network gets a reverse arc (v, u) in the residual graph with capacity equal to the current flow. That reverse arc is what lets the algorithm cancel a locally suboptimal routing decision: sending flow backward along (v, u) is equivalent to reducing flow on (u, v), and that reduction can free capacity for a better path. Without the backward arcs, an augmenting-path procedure would find a maximal flow but not necessarily a maximum one. The residual graph

is the transformation that makes the problem correct. Max-flow becomes: find any path in this derived graph. That reduction — from optimization to reachability — is the chapter's central argument.

Meta-lessons

The greedy exchange argument is the proof technique for local optimality. Kruskal's MST is correct not because greedy algorithms are correct in general — they are not — but because of the cut property: any minimum spanning tree includes the minimum-weight edge crossing any cut. The proof is an exchange argument: assume an MST that omits the minimum-weight edge and show that swapping it in produces an MST of equal or lesser weight. The same argument structure underlies Huffman coding, interval scheduling, and every greedy algorithm in the book. When you ask "why is the greedy choice safe here?", you are asking for the exchange argument, and the answer is always a bound that shows the swap cannot increase cost.

Choosing the right inner subroutine changes the complexity class. Ford-Fulkerson with arbitrary path selection is not a polynomial algorithm for real-valued capacities and is $O(nU)$ for integers. Replacing the arbitrary path selection with BFS — a single-line change to the inner subroutine — converts it to $O(nm^2)$. Replacing the linear scan in Prim's algorithm with a binary heap converts it from $O(n^2)$ to $O((n+m)\log n)$. The outer loop structure stays identical. What changes is whether the loop body maintains a useful invariant cheaply or expensively. This pattern — same outer loop, different inner work — is the main lever available for turning a correct but slow algorithm into a correct and fast one.

Every algorithm in the part proved its optimality by finding a dual bound. Dijkstra's settled set and the relaxation invariant form a primal-dual pair: primal is the set of known shortest paths; dual is the lower bound that no shorter path exists. The min-cut certificate in max-flow is the dual of the flow: primal is the amount flowing from source to sink; dual is the cut capacity that bounds it. Kruskal's cut property is the MST dual: the primal spanning tree; the dual the sequence of cuts that certify each edge choice. These are all instances of LP duality in combinatorial disguise. The insight is not "you can write a linear program for each of these problems" — although you can — but that the primal-dual structure is what makes the algorithm's termination condition and its correctness proof the same statement.

What's next

Floyd-Warshall's triple loop is already dynamic programming. The graph is implicit in the recurrence — $d[i][j][k]$ is the shortest path from i to j using only vertices $1, \ldots, k$ as intermediates — and the algorithm fills the table bottom-up exactly as a DP algorithm would. The SCC condensation turns any directed graph into a DAG, and a DAG has a topological order, and a topological order is the correct bottom-up evaluation order for DP on graphs. Every DP on a DAG is a shortest-path computation in disguise, and every shortest-path computation on a DAG can be solved in linear time with DP. Part VI develops both.

The relaxation primitive does not disappear in Part VI. It generalizes. Where Part V's relaxation tightened path-length estimates, Part VI's recurrences tighten optima over subsets, intervals, prefixes, and independent sets. The BFS and DFS idioms carry over directly: BFS-order is topological order on DAGs, and topological order is the correct evaluation order for DP. The relaxation loop recurs in every Part VI recurrence; the traversal idioms show up in every Part VI evaluation order. Part V did not end. It extended.

Part VI

Dynamic Programming and Greedy Algorithms

Part VII

Specialized Domains

Part VIII

Advanced Complexity

A A Python Primer

This Codex uses Python 3.13 as its executable notation. That choice carries some baggage — Python is slower than C, less expressive than Haskell, less type-safe than Rust — but it earns its keep on one axis I care about: *you can read it the first time*. The code I write in the chapters is meant to be the algorithm, not an obstacle between you and the algorithm. This appendix is the glossary that makes that promise honest. If you've programmed in any modern language, most of what's here will look familiar; the parts that won't are usually about the *type system*, which Python 3.13 changed substantially in 2024 and which I lean on heavily.

I won't try to teach you Python. There are better books for that. What I'll do is name the features I actually use, show the idiomatic shape, and call out the parts that look surprising if you're coming from another language. If something in a chapter looks unfamiliar, this appendix is where to look first.

A.1 Reading code as algorithms

Throughout the book I follow a small set of conventions to keep code readable as algorithm pseudocode:

- **Type annotations are mandatory** on function signatures, class attributes, and module-level variables. If a name has a type written next to it, you should never have to guess what shape it holds.
- **No `Optional` where a sentinel will do.** When a function returns "the index, or -1 if not found", I write `-> int` and document the convention, not `-> int | None`. The cost is a single comment; the gain is that the type signature stays simple.
- **One thing per name.** A variable named `result` is only ever the final answer; a variable named `i` is only ever a loop index. I don't reuse names across responsibilities.
- **No clever one-liners.** A four-line loop that's obviously a loop beats a list comprehension that's secretly an algorithm.

A.2 Variables and the assignment model

A name in Python is a reference to an object. The object has a type; the name doesn't. That asymmetry is what people mean when they say Python is *dynamically typed*: the same name can be rebound to a different type during execution. The annotations I add are checked by tools like `mypy` and `pyright` but not by the interpreter — they're documentation that the tooling enforces.

```
n: int = 42
name: str = "Codex"
is_active: bool = True
pi: float = 3.14159
```

Assignment is reference-copy, not value-copy. After `b = a`, the names `a` and `b` point at the same object. For immutable types (int, str, tuple) this distinction never matters because you can't change the object in place. For mutable types (list, dict, set, your own classes) it matters a great deal — modifying `b` modifies what `a` sees too. The chapters that build mutable data structures lean on this constantly.

A.3 Built-in collections

Four built-in collection types do most of the work in the standard library:

- `list[T]` — a dynamic array (chapter 8 builds one from scratch to show why it's $O(1)$ amortized for append).
- `dict[K, V]` — a hash table (chapter 12 builds one from scratch).

- `set[T]` — a hash table with no values, used for fast membership.
- `tuple[T, ...]` — an immutable, fixed-arity sequence.

The chapters I write almost always *use* the built-in `list` for input or output, because it's the natural shape in Python. But when a chapter is *about* a sequence structure, I build the structure from scratch on top of a fixed-size array (a Python list pre-allocated with `[None] * cap`) so the cost model stays visible.

A.4 Slicing

A slice expression `seq[a:b]` creates a new sequence containing items at indices `a, a+1, ..., b-1`. The right endpoint is exclusive. Slices of `list` and `str` allocate fresh memory; they are not views. That's an important cost: writing `arr[mid:]` inside a recursive call allocates a new list of half the size every level, turning what looks like in-place recursion into $\Theta(n \log n)$ extra space. When a chapter needs in-place behavior, it passes indices `(lo, hi)` rather than slicing.

```
xs = [10, 20, 30, 40, 50]
xs[1:4]      # [20, 30, 40] - new list
xs[::2]      # [10, 30, 50] - step of 2
xs[::-1]     # [50, 40, 30, 20, 10] - reversed
```

Negative indices count from the end: `xs[-1]` is the last element.

A.5 Control flow

Two looping constructs cover essentially everything algorithmic:

```
for item in iterable:
    ...

while condition:
    ...
```

`for` iterates anything that implements the iteration protocol (`__iter__` returning an iterator with `__next__`). `while` runs until a boolean expression goes false. Inside either loop, `break` exits, `continue` skips to the next iteration, and `else` (yes, loops have an `else` clause) runs only if the loop completed without a `break`.

Conditional logic uses `if` / `elif` / `else`. Python's truthiness rules treat empty containers, `0`, `0.0`, `""`, and `None` as false; everything else as true. I rely on this for idiomatic checks like `if not stack:` to mean "the stack is empty".

A.6 Functions

Functions are defined with `def`. Arguments can be positional, keyword, or both. Default values are evaluated *once at definition time*, which is a notorious source of bugs if the default is mutable:

```
def bad(items: list[int] = []) -> list[int]:  # don't do this
    items.append(1)
    return items

def good(items: list[int] | None = None) -> list[int]:
    if items is None:
        items = []
    items.append(1)
    return items
```

The Codex never writes `def f(x=[])`. The pattern above (default `None`, materialize inside) is the safe shape.

Functions are first-class objects. You can pass them as arguments, return them from other functions, and store them in data structures. This is how chapter 5 implements custom ordering — by accepting a comparison function as an argument.

A.7 Lambdas

A `lambda` is an anonymous function restricted to a single expression. I use them only when the function body is genuinely one short expression, typically as a key for sorting or as a custom comparator:

```
points.sort(key=lambda p: (p.y, p.x))
```

If the body grows beyond an expression, I write a `def` with a real name. Multi-line lambdas don't exist in Python by design, and that's a feature.

A.8 Classes

A class bundles state (attributes) with behavior (methods). Python's class syntax is lightweight:

```
class Point:
    def __init__(self, x: float, y: float) -> None:
        self.x = x
        self.y = y

    def distance_to(self, other: "Point") -> float:
        dx = self.x - other.x
        dy = self.y - other.y
        return (dx * dx + dy * dy) ** 0.5
```

The first argument to every method is `self` — the instance being operated on. The `__init__` method runs at construction time. The string `"Point"` in the type hint is a *forward reference* needed because the class isn't fully defined yet inside its own body; from Python 3.13 onward you can also write `from __future__ import annotations` at the top of the file to make all annotations lazy.

A.9 Dunder methods

Dunder methods (double-underscore methods like `__init__`, `__len__`, `__getitem__`) hook into Python's syntax. Implementing them is how a custom class gets to behave like a built-in. The ones I use most often:

Method	Triggers
`__init__(self, ...)`	construction (`Point(1, 2)`)
`__repr__(self)`	the interactive repr / `repr(obj)`
`__eq__(self, other)`	`obj == other`
`__hash__(self)`	use as dict key or in a set
`__len__(self)`	`len(obj)`
`__getitem__(self, k)`	`obj[k]`
`__setitem__(self, k, v)`	`obj[k] = v`
`__iter__(self)`	use in a `for` loop
`__contains__(self, x)`	`x in obj`
`__lt__`, `__le__`, …	`<`, `<=`, … (with `@functools.total_ordering` to fill in the rest)

Chapter 8's `DynamicArray[T]` implements `__len__`, `__getitem__`, `__setitem__`. Chapter 9's `LinkedList[T]` implements `__iter__`. Chapter 12's `HashMap[K, V]` implements `__getitem__`, `__setitem__`, `__contains__`, and `__len__`. Each one is a deliberate choice about *which language affordances the structure participates in.*

A.10 Generators and `yield`

A function containing `yield` is a *generator*. Calling it doesn't run the body; it returns a generator object that lazily produces values as they're requested. Generators are how you write iteration over your own structures cheaply, without materializing an intermediate list:

```
def naturals(stop: int) -> Iterator[int]:
    i = 0
    while i < stop:
        yield i
        i += 1
```

The `for` loop calls `next(gen)` repeatedly until it raises `StopIteration`. In chapter 9 I use a generator inside `__iter__` to walk a linked list without copying its contents into a Python list.

A.11 Type annotations and modern generics

This is the place where Python 3.13 differs most from older codebases. Two features matter:

PEP 695 generic syntax. Pre-3.12, generics required `TypeVar` declarations:

```
# old style
from typing import TypeVar
T = TypeVar("T")
def first(xs: list[T]) -> T:
    return xs[0]
```

3.13 lets you write the type parameter inline in brackets after the name:

```
# new style - used throughout this book
def first[T](xs: list[T]) -> T:
    return xs[0]

class Stack[T]:
    ...
```

The new syntax is cleaner, scoped properly (the `T` only exists inside `first`), and removes the boilerplate. Every generic function and class in the Codex uses it.

PEP 695 type aliases. A type alias gives a name to a complex type expression. Old style required `TypeAlias` from `typing`; new style uses the `type` keyword:

```
type Comparator[T] = Callable[[T, T], int]
type Graph[V] = dict[V, list[V]]
```

The alias is lazily evaluated, so it can reference itself recursively (useful for trees in Part III).

A.12 Protocols and structural typing

The `typing.Protocol` mechanism is Python's answer to *structural typing* — describing a type by what it can *do* rather than what it *is*. The standard library defines several protocols I lean on:

- `Sequence[T]` — has `__getitem__` and `__len__`; can be iterated.
- `MutableSequence[T]` — additionally has `__setitem__`, `append`, `pop`, etc.
- `Iterable[T]` — has `__iter__`.
- `Iterator[T]` — has `__iter__` and `__next__`.
- `Hashable` — has `__hash__` (and a consistent `__eq__`).

- `Comparable[T]` (custom, but common) — supports `<` against `T`.

Writing a function that takes `Sequence[T]` instead of `list[T]` means it works on lists, tuples, strings (for `str`), and any custom array-like class. The chapters lean on this for the same reason they lean on PEP 695 generics: the algorithm is about the *shape* of the data, not the *Python class* of the data.

You can define your own protocol with `@runtime_checkable` if you want `isinstance` checks; I rarely need that.

A.13 Dataclasses

A `@dataclass` decorator auto-generates `__init__`, `__repr__`, and (optionally) `__eq__` from class attributes:

```
from dataclasses import dataclass

@dataclass
class Edge:
    src: int
    dst: int
    weight: float
```

This is the same as writing the three dunder methods by hand. The Codex uses dataclasses for plain "record" types — the `_Entry[K, V]` in chapter 12's hash table is one, and Part III's tree nodes will be others. When the class needs custom behavior beyond data, I write the dunder methods explicitly.

A.14 Optional, union, and `None`

Python 3.10+ supports the pipe syntax for union types:

```
def find(xs: list[int], target: int) -> int | None:
    for i, x in enumerate(xs):
        if x == target:
            return i
    return None
```

`int | None` is the same as `Optional[int]`. The Codex prefers the pipe syntax. When `None` is a legitimate return value, the caller must check for it before using the result (`mypy --strict` enforces this).

A.15 Exceptions

`raise` throws an exception. `try` / `except` catches it. `finally` runs regardless. The Codex raises exceptions for *programming errors* (e.g., popping from an empty stack) and returns sentinel values for *expected absence* (e.g., a search miss). The split is a judgment call; the rule is to be consistent within a structure.

```
class StackEmpty(Exception):
    pass

class Stack[T]:
    def pop(self) -> T:
        if not self._items:
            raise StackEmpty("pop from empty stack")
        return self._items.pop()
```

I almost never catch broad exceptions (`except Exception`). When I do catch, I catch the specific class.

A.16 Context managers (`with`)

A `with` block manages setup and teardown around a body. The standard use is for files (`with open(...) as f:`), but the protocol works for anything that needs paired enter/exit. I rarely need to *implement* a context manager in the Codex, but I use them whenever a chapter touches a file.

A.17 `match` statements

Python 3.10's `match` is a structural pattern-matching construct. I use it sparingly — usually only when dispatching on the shape of an algebraic data type:

```
match node:
    case Leaf(value):
        return value
    case Branch(left, right):
        return walk(left) + walk(right)
```

For simple value dispatch, an `if / elif` chain is almost always clearer. `match` earns its place when the cases are *structural* — when they're naming and binding parts of the matched object.

A.18 Iteration helpers

A handful of built-ins and `itertools` helpers come up often:

- `enumerate(xs)` yields `(index, value)` pairs.
- `zip(a, b)` yields `(a[i], b[i])` pairs, stopping at the shorter one.
- `range(start, stop, step)` is a memory-cheap arithmetic progression.
- `reversed(xs)` iterates in reverse without copying.
- `sorted(xs, key=..., reverse=...)` returns a new sorted list.
- `min(xs)`, `max(xs)`, `sum(xs)`, `any(xs)`, `all(xs)` are the standard reductions.

From `itertools`: `chain`, `accumulate`, `combinations`, `permutations`, `product` show up occasionally. From `collections`: `deque`, `Counter`, `defaultdict` are tools I sometimes reach for in benchmarks but build from scratch when a chapter is about them.

A.19 Running the code

Every code block in the chapters that begins with ```` ```{python} ```` is executed by Quarto when the book renders. The chunks share state within a chapter (later chunks see names defined by earlier ones). Plain ```` ```python ```` blocks are inert — they're shown but not run. When you see an *output line* under a chunk, that output is the actual result of executing the chunk during the most recent build, not a hand-written example.

The illiterate tool extracts every executable chunk into `src/codex/` so the code is also importable as a regular Python package. If you've cloned the repo, `uv run python -c "from codex.structures.array import DynamicArray; ..."` will give you a REPL-friendly version of any structure from the book.

B Asymptotic Notation and Analysis

When you read an algorithm and ask "is this fast?", you're really asking *how does its cost scale with the input?* The exact number of operations on your laptop on this Tuesday is a number that depends on the compiler, the CPU, the cache, the temperature of the room. The *rate of growth* — how the cost changes when the input doubles, or grows tenfold, or grows to a million — is an intrinsic property of the algorithm. Asymptotic notation is the language for talking about that intrinsic property without getting distracted by the constants the implementation drags along.

This appendix collects the notation and the standard hierarchy in one place. The chapters use Big-O, Big-Ω, and Big-Θ from the very first one (chapter 1), and they assume you can read all three. If you can, you can skim this appendix for reference. If you can't, this is the place to slow down.

B.1 What the notation is hiding

Asymptotic notation deliberately throws away two kinds of information: *constant factors* and *lower-order terms*. An algorithm that runs in $3n^2 + 5n + 100$ steps is called $\Theta(n^2)$, even though for $n = 5$ the linear term might actually dominate. The bet is that for large enough n, the highest-order term swamps everything else, and the constant in front of it is something you can attack with engineering (better data layout, fewer cache misses, SIMD) while the *order* of growth is something you can only attack with a different algorithm.

This bet is right most of the time and wrong sometimes. It's wrong when the inputs are *always* small (a sort over 8 items, where insertion sort beats quicksort by a wide margin because the constant matters more than the order). It's wrong when the hidden constants are *astronomical* (Strassen's matrix-multiply algorithm is asymptotically faster than naive, but its hidden constant doesn't pay off until matrices are quite large). When you see me writing about practical performance in a chapter, I'm usually talking about constants the asymptotic notation hides; when you see me writing about scaling, I'm talking about what the asymptotic notation reveals.

B.2 The Big-O family

Let $f(n)$ and $g(n)$ be functions from natural numbers to non-negative reals. The five families of notation describe different relationships between them.

Big-O — upper bound

$$f(n) = O(g(n)) \iff \exists c > 0,\ n_0 \geq 0 \text{ s.t. } 0 \leq f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0$$

In words: past some threshold n_0, the function f is bounded above by some constant multiple of g. This is the most commonly written notation, but it's also the most overloaded — when someone says "binary search is $O(\log n)$", they often *mean* $\Theta(\log n)$ (it's also a lower bound), but using O has become idiomatic. Big-O is a *promise about the worst behavior*; it doesn't say f ever achieves that worst behavior.

Big-Ω — lower bound

$$f(n) = \Omega(g(n)) \iff \exists c > 0,\ n_0 \geq 0 \text{ s.t. } 0 \leq c \cdot g(n) \leq f(n) \text{ for all } n \geq n_0$$

The mirror of Big-O. Past some threshold, the function f is bounded *below* by a constant multiple of g. This is what you use when you want to argue that an algorithm *must* take at least so much time — for instance, comparison-based sorting is $\Omega(n \log n)$ regardless of how clever you are about it (chapter 7 of Part I closes on exactly this kind of argument).

Big-Θ — tight bound

$$f(n) = \Theta(g(n)) \iff f(n) = O(g(n)) \text{ and } f(n) = \Omega(g(n))$$

When both bounds match, the algorithm grows *exactly* like g (up to constants). This is the most informative notation and the one I use when I have a tight bound. "Binary search is $\Theta(\log n)$" tells you both that no instance ever takes more than $c \log n$ steps *and* that for sufficiently many inputs it takes at least $c' \log n$ steps. If you can prove Θ, prefer it to O — you're giving strictly more information.

Little-o — strict upper bound

$$f(n) = o(g(n)) \iff \lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$$

f grows *strictly slower* than g. Not just bounded by — *eventually negligible compared to*. $\log n = o(n)$. $n = o(n^2)$. $n^{100} = o(2^n)$. The little-o notation is what you use to say "this part of the cost is irrelevant in the long run".

Little-ω — strict lower bound

$$f(n) = \omega(g(n)) \iff \lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty$$

f grows *strictly faster* than g. The mirror of little-o. $n = \omega(\log n)$. $2^n = \omega(n^k)$ for any constant k.

B.3 Worst, average, and best case

A single algorithm has *three* asymptotic stories — one per cost regime:

- **Worst case** is the maximum cost over all inputs of size n. This is what most analysis defaults to, because it's the conservative guarantee an algorithm provides.
- **Average case** is the expected cost over some distribution of inputs (usually uniform random). Quicksort is the canonical example: $\Theta(n^2)$ worst case, $\Theta(n \log n)$ average case.
- **Best case** is the minimum cost. It's the *least* useful of the three for engineering, but it's useful for showing that an algorithm is *fundamentally limited* (every comparison sort is $\Omega(n)$ best-case because it has to look at every element to be sure).

When a chapter says "binary search is $\Theta(\log n)$", it's giving the worst case. When it says "insertion sort is $\Theta(n)$ on nearly-sorted input", it's giving the best case under a structural assumption. I try to label the regime when there's any ambiguity.

B.4 Amortized analysis

Some operations are *cheap most of the time and expensive occasionally*, where the expensive operations only happen because the cheap ones piled up the necessary state. Dynamic array append (chapter 8), hash table insert (chapter 12), and union-find union (chapter 14) all have this shape: most calls are constant-time, a few are linear, and a careful accounting shows the *total* cost over a sequence of n calls is $O(n)$ — so the *average per call*, amortized over the sequence, is $O(1)$.

There are three standard methods for amortized analysis: the **aggregate method** (compute the total cost of a sequence of n operations, then divide by n), the **accounting method** (assign each operation a "credit" larger than its actual cost, with the surplus paying for future expensive ops), and the **potential method** (define a potential function that captures the structure's stored work, then bound each operation's cost by its actual cost plus the change in potential). I introduce the aggregate method in chapter 8, the accounting method in chapter 12, and use the potential method implicitly in chapter 14. The mathematical machinery for all three lives in the math appendix; the conceptual entry point is in chapter 8.

The headline: *amortized $O(f(n))$ per operation* is a guarantee over a sequence, not per individual call. The worst single call may be more expensive than $f(n)$ — what's guaranteed is that the *total* over k calls is $O(k \cdot f(n))$.

B.5 Space complexity

The same notation describes memory cost. An algorithm is $O(n)$ in space if it allocates a number of words proportional to the input size. There are a few important subtleties:

- **Input space doesn't count.** The convention is to measure *additional* memory beyond what's needed to hold the input itself.
- **In-place** means $O(1)$ extra space, or sometimes $O(\log n)$ to allow for recursion stack.
- **Recursion costs space.** A recursive algorithm with depth d uses $\Theta(d)$ stack space, even if the function body itself looks constant. Mergesort is $O(n \log n)$ in time and $O(n)$ in extra space — the extra space is the temporary array, not the recursion stack ($O(\log n)$).
- **Output space sometimes counts and sometimes doesn't.** If the output is required to be a new array of size n, that $O(n)$ is unavoidable and not usually called "extra".

I try to state both time and space complexity for every structure and algorithm in the book; the math appendix has the bookkeeping rules.

B.6 The hierarchy of growth

Most algorithms in the book live in one of a small set of complexity classes. From fastest to slowest growth:

Class	Name	Where it appears
$\Theta(1)$	constant	array access, hash lookup (avg), stack push
$\Theta(\alpha(n))$	inverse Ackermann	union-find with path compression (ch 14)
$\Theta(\log \log n)$	doubly logarithmic	van Emde Boas trees, interpolation search
$\Theta(\log n)$	logarithmic	binary search, balanced tree ops
$\Theta(\sqrt{n})$	square root	sqrt decomposition, some range queries
$\Theta(n)$	linear	sequential scan, linked list traversal
$\Theta(n \log n)$	linearithmic	comparison-based sorting, FFT
$\Theta(n^2)$	quadratic	naive sorting, all-pairs over n items
$\Theta(n^3)$	cubic	naive matrix multiply, Floyd-Warshall
$\Theta(n^k)$	polynomial	the class P
$\Theta(2^n)$	exponential	subset enumeration, naive TSP
$\Theta(n!)$	factorial	permutation enumeration

The classes are deeply unequal. For $n = 10^6$, an $O(n)$ algorithm runs in a millisecond on modern hardware; an $O(n \log n)$ runs in 20 milliseconds; an $O(n^2)$ runs in roughly 17 minutes; an $O(n^3)$ would take 30 years. Going from $O(n^2)$ to $O(n \log n)$ is not a minor optimization — it's the difference between a usable tool and an unusable one.

The gap between $O(n^k)$ for any constant k and $O(2^n)$ is the conventional boundary between *tractable* and *intractable*. The book's last part returns to this boundary; for now, treat polynomial = OK and exponential = trouble.

B.7 The Master Theorem

Divide-and-conquer algorithms produce recurrences of the form

$$T(n) = a \cdot T(n/b) + f(n)$$

where $a \geq 1$ is the number of subproblems, $b > 1$ is the factor by which each subproblem shrinks, and $f(n)$ is the work done at the current level (the cost of splitting and combining). The Master Theorem gives a recipe for solving these without setting up an explicit recursion tree.

Let $\log_b a$ be the *critical exponent* — the order of growth at which the leaves of the recursion tree dominate.

1. **Leaf-dominated.** If $f(n) = O(n^{\log_b a - \epsilon})$ for some $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$. The total work is dominated by the work at the bottom of the recursion tree, where there are most subproblems.
2. **Balanced.** If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \log n)$. Each level of the tree does the same total work, and there are $\log_b n$ levels.
3. **Root-dominated.** If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some $\epsilon > 0$ *and* f satisfies the regularity condition $af(n/b) \leq cf(n)$ for some $c < 1$ and large n, then $T(n) = \Theta(f(n))$. The work at the root dominates everything below.

Worked examples in the book:

- **Mergesort** (chapter 4): $T(n) = 2T(n/2) + \Theta(n)$. Here $a = 2, b = 2$, so $\log_b a = 1$, and $f(n) = \Theta(n) = \Theta(n^1)$. Case 2 applies. $T(n) = \Theta(n \log n)$.
- **Binary search** (chapter 2): $T(n) = T(n/2) + \Theta(1)$. Here $a = 1, b = 2$, so $\log_b a = 0$, and $f(n) = \Theta(1) = \Theta(n^0)$. Case 2. $T(n) = \Theta(\log n)$.
- **Strassen's matrix multiply**: $T(n) = 7T(n/2) + \Theta(n^2)$. Here $\log_2 7 \approx 2.807$, and $f(n) = \Theta(n^2)$, which is $O(n^{2.807-\epsilon})$. Case 1. $T(n) = \Theta(n^{\log_2 7})$.

The Master Theorem covers most divide-and-conquer recurrences you'll meet in this book. When it doesn't apply (e.g., $T(n) = T(\sqrt{n}) + 1$, or recurrences with non-polynomial f), the math appendix discusses the substitution and recursion-tree methods that handle them.

B.8 Lower bounds: the model is part of the answer

When a chapter proves a lower bound — "comparison-based sorting requires $\Omega(n \log n)$ comparisons" — the lower bound is *not a fact about the problem*. It's a fact about a *computational model*. The same problem (sorting) is $\Theta(n)$ in the *counting sort* model where elements are integers in a bounded range; the lower bound only applies if your algorithm's only access to the elements is through *comparisons*.

This is one of the most important moves in algorithm analysis, and Part I's closing chapter ends on it. When you see a lower bound, ask: *in what model?* Often the path to a faster algorithm is to escape the model in which the lower bound was proved.

B.9 Notation hygiene

A few conventions I follow that make my chapters readable:

- **Use Θ when you can.** O is a weaker statement; reserve it for cases where you only have an upper bound (e.g., describing a sub-routine whose exact cost depends on its argument).
- **Don't write $O(2n)$.** Constants disappear; $2n$ and n are both $O(n)$. The notation already absorbed the constant.
- **Don't write $\log_2 n$ inside Θ.** All logs differ by a constant factor; $\Theta(\log n)$ and $\Theta(\log_2 n)$ and $\Theta(\ln n)$ are the same class. Constants inside a Θ are noise.
- **Don't conflate worst with typical.** "$O(n^2)$ worst case, $O(n \log n)$ expected" is a real and useful sentence; collapsing it to "$O(n^2)$" hides the structure that matters for picking an algorithm.
- **Name the variable.** $O(n)$ when there are two parameters (say n items and m edges) is ambiguous. Write $O(n + m)$ or $O(nm)$, not $O(n)$.

When in doubt, write the recurrence or the sum, solve it, and report the answer. The math appendix has the toolkit.

C A Mathematical Toolkit

Algorithm analysis lives downstream of a handful of mathematical tools: summations, logarithms, recurrences, probability, and a few specialty objects like the inverse Ackermann function. Most undergraduate algorithms courses assume you've seen this material in discrete math and don't re-derive it; this book takes the same approach in the chapters, but I want to give you one place to look up the identities and the methods when an analysis in the chapter asks you to solve a sum or unroll a recurrence.

This isn't a math textbook either. It's a *toolkit* — the smallest set of facts and methods you need to follow every analysis in this book. If you've seen all of this before, you can skim it. If parts of it look unfamiliar, the chapter analyses will refer back here when they lean on a specific identity.

C.1 Summations

Almost every cost analysis ends in a sum. The five sums below cover the vast majority of what comes up.

Arithmetic series

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2} = \Theta(n^2)$$

The classic. Appears in any algorithm with a nested loop of the shape "for each i, do i units of work" — selection sort, the comparison count for bubble sort, the work done by insertion sort on a reversed array.

More generally, for a sum of the first n natural numbers raised to a power k:

$$\sum_{i=1}^{n} i^k = \Theta(n^{k+1})$$

This is the *power rule for sums* — it's the discrete analog of the integral $\int_0^n x^k\,dx = \frac{n^{k+1}}{k+1}$, and the asymptotic agrees with the integral up to constants.

Geometric series

$$\sum_{i=0}^{n-1} r^i = \frac{r^n - 1}{r - 1}$$

for $r \neq 1$. When $r > 1$, this is $\Theta(r^n)$: the last term dominates the entire sum, up to a constant factor that depends on r. When $0 < r < 1$, the sum converges to $\frac{1}{1-r}$ as $n \to \infty$, so the partial sum is $\Theta(1)$ — bounded regardless of how many terms you take. This is the math behind chapter 8's amortized argument for dynamic array doubling: the cost of all resizes through size n is $1 + 2 + 4 + \cdots + n = 2n - 1 = \Theta(n)$, because a geometric series with ratio 2 and last term n sums to $\Theta(n)$.

Harmonic series

$$H_n = \sum_{i=1}^{n} \frac{1}{i} = \ln n + \gamma + O(1/n)$$

where $\gamma \approx 0.5772$ is the Euler–Mascheroni constant. The asymptotic is $\Theta(\log n)$. This sum shows up in randomized analyses (e.g., quicksort's expected comparison count), in the analysis of skip lists, and in any algorithm that does $1 + 1/2 + 1/3 + \cdots$ work — for instance, sifting up through a heap when you only pay $1/h$ at each level.

Telescoping sums

When consecutive terms cancel, a sum collapses:

$$\sum_{i=1}^{n}(a_i - a_{i-1}) = a_n - a_0$$

This is the engine behind the potential method of amortized analysis, where you express the cost of each operation as "actual cost + (potential after) - (potential before)" and the potential changes telescope across a sequence of operations.

Logarithm sums

$$\sum_{i=1}^{n} \log i = \log(n!) = \Theta(n \log n)$$

Useful when you have to add up the log-time work done across n inserts into a balanced tree, or the heap-build cost across n insertions. The $\Theta(n \log n)$ growth here is exactly the comparison lower bound for sorting (Stirling's approximation, below, makes the constant explicit).

C.2 Logarithms

A few logarithm identities come up often enough that I'll state them once and reuse them throughout the book.

Change of base. $\log_b n = \frac{\log_a n}{\log_a b}$. The ratio is a constant, so $\log_a n = \Theta(\log_b n)$ — all logs are the same complexity class up to constants. That's why the chapters write $\Theta(\log n)$ without specifying the base.

Logarithm of a product/quotient. $\log(xy) = \log x + \log y$. $\log(x/y) = \log x - \log y$. $\log(x^k) = k \log x$.

Inversion. $b^{\log_b x} = x$. $\log_b(b^x) = x$. Useful when an algorithm's cost is expressed as $2^{\log_2 n}$ — that's just n.

Stirling's approximation. $\log(n!) = n \log n - n \log e + \Theta(\log n) = n \log n - \Theta(n)$. The first-order term is what makes comparison sorting $\Omega(n \log n)$ in the decision-tree model: a sorting algorithm must distinguish $n!$ permutations, requiring at least $\log_2(n!) = \Theta(n \log n)$ comparisons.

Iterated logarithm. $\log^*(n)$ is the number of times you have to take log before the result drops below 1. It grows *unbelievably slowly*: $\log^*(2^{65536})$ is just 5. The iterated logarithm shows up in some pointer-machine algorithms and in the analysis of the union-by-rank-only version of union-find.

C.3 Recurrences

A recurrence is an equation that defines a function in terms of its values at smaller inputs. Every recursive algorithm has a cost recurrence. Three methods solve them.

The substitution method

Guess the form of the solution and prove it by induction.

To solve $T(n) = 2T(n/2) + n$, guess $T(n) = O(n \log n)$ — formally, that $T(n) \le c \cdot n \log n$ for some c and all $n \ge n_0$. Substitute the guess into the recurrence:

$$T(n) \le 2 \cdot c \cdot (n/2) \log(n/2) + n = cn \log n - cn + n = cn \log n - (c-1)n$$

For this to be at most $cn \log n$, the condition is $(c-1)n \ge 0$, i.e., $c \ge 1$. The base case fixes c for small n. The guess holds.

Substitution is the most general method but also the hardest — you have to guess right. It's a good fallback when the others don't apply.

The recursion-tree method

Draw the recursion tree, sum the work at each level, then sum the levels.

For $T(n) = 2T(n/2) + n$: the root does n work. Its two children do $n/2$ each, totaling n. The four grandchildren do $n/4$ each, totaling n. Each level does $\Theta(n)$ work, and there are $\Theta(\log n)$ levels before subproblems shrink to size 1. Total: $\Theta(n \log n)$.

For $T(n) = T(n/2) + 1$ (binary search): the root does 1 work. Its child does 1. Each level does 1 work, and there are $\Theta(\log n)$ levels. Total: $\Theta(\log n)$.

For $T(n) = 2T(n/2) + n^2$: the root does n^2 work. Its two children do $(n/2)^2 = n^2/4$ each, totaling $n^2/2$. The work at each level is geometrically decreasing, so the root dominates. Total: $\Theta(n^2)$.

Recursion trees are visual and concrete. They're my preferred method when I'm not sure what shape the answer will take.

The Master Theorem

The notation appendix has the statement and three worked examples. The Master Theorem is a pre-packaged answer to recurrences of the form $T(n) = aT(n/b) + f(n)$. Use it when it applies; fall back to substitution or trees when it doesn't.

The shapes it *doesn't* handle:

- $T(n) = T(n-1) + 1$ (linear recursion, not divide-and-conquer): unroll directly to get $T(n) = \Theta(n)$.
- $T(n) = T(\sqrt{n}) + 1$: change variable $m = \log n$, get $T(2^m) = T(2^{m/2}) + 1$, which is $\Theta(\log m) = \Theta(\log \log n)$.
- $T(n) = 2T(n/2) + n \log n$: doesn't match any case cleanly because $f(n) = n \log n$ is $\Theta(n^{\log_b a})$ times a logarithm. The extended master theorem (or substitution) gives $T(n) = \Theta(n \log^2 n)$.

C.4 Amortized analysis methods

When a data structure does expensive work occasionally and cheap work the rest of the time, three methods quantify the average cost over a sequence.

The aggregate method

Compute the total cost of any sequence of n operations, then divide by n.

For chapter 8's dynamic array: a sequence of n appends triggers resizes at sizes $1, 2, 4, 8, \ldots, n$. The cost of all resizes is $1 + 2 + 4 + \cdots + n < 2n$. Plus the n append-after-resize operations, each costing 1. Total: $O(n)$. Per-operation: $O(1)$ amortized.

The aggregate method is the most intuitive of the three. Use it when the sequence has a recognizable pattern of expensive operations.

The accounting method

Assign each operation a *credit* — an upper bound on its cost that may exceed its actual cost. The surplus credit is "saved up" and used to pay for future expensive operations. The amortized cost per operation is the credit, *provided you can prove the accumulated savings never go negative.*

For chapter 12's chained hash table: charge 3 per insert. 1 pays for the actual insert. The other 2 are credit: 1 deposit on the new item, 1 deposit on an old item. When the table doubles and rehashes, every item has accumulated enough credit to pay for its move (each move costs 1, and the doubling guarantees at least as many credits as old items). Amortized cost: $O(1)$.

The accounting method is good when the work has a clear "I'll pay you back later" structure.

The potential method

Define a *potential function* $\Phi : \text{state} \to \mathbb{R}_{\geq 0}$ that captures the structure's stored work. The amortized cost of an operation is its actual cost plus the change in potential:

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$$

If you choose Φ so that $\Phi(D_n) \geq \Phi(D_0)$, then summing telescopes: $\sum \hat{c}_i \geq \sum c_i$, so the amortized total bounds the actual total.

For chapter 14's union-find with path compression and union by rank: the potential function counts something like "the sum of the levels of the rank hierarchy each node lives below". Operations that compress paths decrease the potential by a lot, paying for their own work; operations that just walk paths increase it by a little. The careful analysis (which I sketch in the chapter and which Tarjan worked out fully in 1975) yields the inverse-Ackermann bound.

The potential method is the most powerful and the most abstract. It's what you reach for when the cost structure is too tangled for credits to cover cleanly.

C.5 The inverse Ackermann function

The Ackermann function $A(m, n)$ is a fast-growing function defined by a double recursion. Its values grow faster than any primitive recursive function — $A(4, 4)$ is a number too large to write down with conventional notation.

The *inverse* Ackermann function $\alpha(n)$ is defined (informally) as the smallest k such that $A(k, k) \geq n$. It grows so slowly that for any n you'll ever see in practice — atoms in the universe, microseconds since the Big Bang, you name it — $\alpha(n) \leq 4$. It is *not* a constant function (it does grow), but for engineering purposes you can treat it as one.

The inverse Ackermann appears in exactly one place in this book: the amortized cost of union-find with both path compression and union by rank (chapter 14). The bound $O(\alpha(n))$ per operation is the tightest known, and it's also a *lower bound* in the pointer-machine model — you cannot do better with the operations on offer.

C.6 Probability essentials

A handful of probability facts come up in the book, especially in the chapters on randomized algorithms (quicksort's expected analysis, Bloom filters' false-positive rate, hashing's expected chain length).

Linearity of expectation. $E[X + Y] = E[X] + E[Y]$, regardless of whether X and Y are independent. This is the most useful single fact in randomized algorithm analysis. To bound the expected total work of an algorithm, decompose the work into indicator variables for each "event" (e.g., a comparison happens), bound the expectation of each indicator (the probability the event happens), and add them up.

Indicator variables. $X_A = 1$ if event A happens, 0 otherwise. Then $E[X_A] = \Pr[A]$. This is how chapter 4's expected analysis of randomized quicksort works: define $X_{ij} = 1$ if elements at ranks i and j are compared, 0 otherwise. The total comparisons is $\sum X_{ij}$. By linearity, the expected total is $\sum \Pr[\text{comparison happens}]$, which works out to $\Theta(n \log n)$.

Independence. Events A and B are independent if $\Pr[A \cap B] = \Pr[A] \cdot \Pr[B]$. The Bloom filter analysis assumes the k hash functions are independent — under that assumption, the probability all k bits for a never-inserted key are set is $(1 - e^{-kn/m})^k$, which the chapter optimizes over k.

Markov's inequality. For a non-negative random variable X with finite mean: $\Pr[X \geq t] \leq E[X]/t$. The weakest of the standard concentration bounds, but it requires no assumptions beyond non-negativity. Often the first tool you reach for.

Chernoff bounds. For a sum of independent indicator variables $X = \sum X_i$ with mean μ: $\Pr[X \geq (1 + \delta)\mu] \leq e^{-\delta^2 \mu/3}$ for $\delta \in (0, 1]$. This is *exponentially* tighter than Markov and is what makes randomized algorithms reliable in practice — the probability of being far from the expected behavior decays exponentially fast.

The book uses Markov implicitly in some upper-bound arguments and gestures at Chernoff in the analysis of skip lists and randomized hashing. Detailed Chernoff derivations are out of scope here; the takeaway is that *sums of independent random variables concentrate around their mean fast*, which is why randomized algorithms work as well as they do.

C.7 Working with floors and ceilings

Recurrences and analyses often involve $\lfloor n/2 \rfloor$ and $\lceil n/2 \rceil$. Two useful facts:

- $\lfloor n/2 \rfloor + \lceil n/2 \rceil = n$.
- $\lfloor \log_2 n \rfloor + 1 = \lceil \log_2(n+1) \rceil$ — the number of bits to represent n.
- When solving recurrences, you can usually drop floors and ceilings asymptotically: $T(n) = T(\lfloor n/2 \rfloor) + 1$ has the same Θ behavior as $T(n) = T(n/2) + 1$. The formal justification involves bounding the recurrence above and below by integer-valued recurrences, but the practical shortcut is fine for almost all the analyses in this book.

C.8 When the math gets in the way

Most of the analyses in this book are not deep mathematics. They're a sequence of moves: write down what the algorithm does at each step, count the steps as a sum or a recurrence, simplify using the identities above, report the answer in Θ form. When an analysis feels intractable, the most common culprits are:

1. **Not separating cases.** Worst, average, and best can have very different shapes. Pick one and stick to it.
2. **Trying to be precise about constants when Θ would do.** Inside a Θ, constants and lower-order terms are noise. Drop them early.
3. **Reaching for the Master Theorem when the recurrence isn't divide-and-conquer.** Linear-recursion recurrences ($T(n) = T(n-1) + f(n)$) unroll directly to a sum; trying to force them through the Master Theorem just wastes time.
4. **Forgetting that amortized analysis is over a sequence.** A single operation may exceed the amortized bound. What's guaranteed is the *total* over a long enough sequence.

When in doubt: write the sum or the recurrence on paper, solve it with the toolkit above, sanity-check the answer against a small numerical experiment, and report the asymptotic. The notation appendix has the language for the report; this appendix has the tools for the work.